UNISYS
imagine it. done.

OS 2200

ASCII COBOL
Programming Reference Manual

Level 7R3K

June 2010

7830 7709–002

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THIS DOCUMENT. Any product or related information described herein is only furnished pursuant and subject to the terms and conditions of a duly executed agreement to purchase or lease equipment or to license software. The only warranties made by Unisys, if any, with respect to the products described in this document are set forth in such agreement. Unisys cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, special, or consequential damages.

You should be very careful to ensure that the use of this information and/or software material complies with the laws, rules, and regulations of the jurisdictions with respect to which it is used.

The information contained herein is subject to change without notice. Revisions may be issued to advise of such changes and/or additions.

Notice to U.S. Government End Users: This is commercial computer software or hardware documentation developed at private expense. Use, reproduction, or disclosure by the Government is subject to the terms of Unisys standard commercial license for the products, and where applicable, the restricted/limited rights provisions of the contract data rights clauses.

Unisys is a registered trademark of Unisys Corporation in the United States and other countries.
All other brands and products referenced in this document are acknowledged to be the trademarks or registered trademarks of their respective holders.

Acknowledgment

Any organization interested in reproducing the COBOL standard and specifications in whole or in part, using ideas from this document as the basis for an instruction manual or for any other purpose, is free to do so. However, all such organizations are requested to reproduce the following acknowledgement paragraphs in their entirety as part of the preface to any such publication (any organization using a short passage from this document, such as in a book review, is requested to mention "COBOL" in acknowledgement of the source, but need not quote the acknowledgement):

COBOL is an industry language and is not the property of any company or group of companies, or of any organization or group of organizations.

No warranty, expressed or implied, is made by any contributor or by the CODASYL Programming Language Committee as to the accuracy and functioning of the programming system and language. Moreover, no responsibility is assumed by any contributor, or by the committee, in connection therewith.

The authors and copyright holders of the copyrighted material used herein:

> FLOW-MATIC Programming for the UNIVAC® I and II, Data Automation Systems copyrighted 1958, 1959, by Unisys Corporation; IBM® Commercial Translator Form No. F 28-8013, copyrighted 1959 by IBM; FACT DSI 27A5260-2760, copyrighted 1960 by Minneapolis-Honeywell.

have specifically authorized the use of this material in whole or in part, in the COBOL specifications. Such authorization extends to the reproduction and use of the COBOL specifications in programming manuals or similar publications.

UNIVAC is a registered trademark of Unisys Corporation.
IBM is a registered trademark of International Business Machines Corporation.

Contents

Section 1. Introduction

1.1. General 1–1
1.2. Concepts 1–3
1.2.1. Features of the Language 1–3
1.2.2. Record Ordering 1–3
1.2.3. Report Writer 1–3
1.2.4. Table Handling 1–4
1.2.4.1. Table Definition 1–4
1.2.4.2. References to Table Items 1–5
1.2.4.3. Table Searching 1–6
1.2.5. File Organization and Access Methods 1–6
1.2.5.1. Sequential Organization 1–6
1.2.5.2. Relative Organization 1–7
1.2.5.3. Indexed Organization 1–7
1.2.6. Rerun 1–7
1.2.7. Program Modularity 1–8
1.2.7.1. Interprogram Communications 1–8
1.2.7.1.1. Transfer of Control 1–8
1.2.7.1.2. Interprogram Data Storage 1–9
1.2.7.2. Segmentation 1–9
1.2.8. Communication Facility 1–10
1.2.8.1. Message Control System 1–10
1.2.8.2. The COBOL Object Program 1–10
1.2.8.3. Relationship of COBOL Program to MCS and Communication Devices 1–11
1.2.8.3.1. Invoking the COBOL Object Program 1–12
1.2.8.4. The Concept of Messages and Message Segments 1–13
1.2.8.5. The Concept of Queues 1–14
1.2.8.5.1. Independent Enqueuing and Dequeuing 1–14
1.2.8.5.2. Enabling and Disabling Logical Connectives 1–14
1.2.8.5.3. Enqueuing and Dequeuing Methods 1–15
1.2.8.5.4. Queue Hierarchy 1–15
1.2.9. Debugging 1–17
1.2.10. Library 1–17
1.3. Overall Language Considerations 1–18
1.3.1. General 1–18
1.3.2. Notation Used in Formats and Rules 1–18
1.3.2.1. Definition of a General Format 1–18
1.3.2.1.1. Syntax Rules 1–18
1.3.2.1.2. General Rules 1–18

1.3.2.1.3. Elements....1–18
1.3.2.1.4. Words....1–19
1.3.2.1.5. Level–Numbers....1–19
1.3.2.1.6. Brackets and Braces....1–19
1.3.2.1.7. The Ellipsis....1–19
1.3.2.1.8. Format Punctuation....1–20
1.3.2.1.9. Use of Certain Special Characters in Formats....1–20
1.3.3. Language Concepts....1–20
1.3.3.1. Character Set....1–20
1.3.3.2. Language Structure....1–21
1.3.3.2.1. Separators....1–21
1.3.3.2.2. Character–Strings....1–22
1.3.3.3. Concept of Computer Independent Data Description....1–29
1.3.3.3.1. Logical Record and File Concept....1–29
1.3.3.3.2. Concept of Levels....1–30
1.3.3.3.3. Concept of Classes of Data....1–31
1.3.3.3.4. Selection of Character Representation and Radix....1–31
1.3.3.3.5. Algebraic Signs....1–32
1.3.3.3.6. Standard Alignment Rules....1–32
1.3.3.3.7. Item Alignment for Increased Object–Code Efficiency....1–32
1.3.3.3.8. Uniqueness of Reference....1–33
1.3.3.4. Explicit and Implicit Specifications....1–37
1.3.3.4.1. Explicit and Implicit Procedure Division References....1–37
1.3.3.4.2. Explicit and Implicit Transfers of Control....1–37
1.3.3.4.3. Explicit and Implicit Attributes....1–38
1.3.4. Control Division....1–39
1.3.4.1. General Description....1–39
1.3.4.2. Organization....1–39
1.3.4.3. Control Division Structure....1–39
1.3.4.3.1. General Format....1–39
1.3.5. Identification Division....1–39
1.3.5.1. General Description....1–39
1.3.5.2. Organization....1–39
1.3.5.3. Structure....1–40
1.3.5.3.1. General Format....1–40
1.3.6. Environment Division....1–40
1.3.6.1. General Description....1–40
1.3.6.2. Organization....1–40
1.3.6.3. Environment Division Structure....1–41
1.3.6.3.1. General Format....1–41
1.3.6.3.2. Syntax Rules....1–41
1.3.7. Data Division....1–41
1.3.7.1. Overall Approach....1–41
1.3.7.2. Physical and Logical Aspects of Data Description....1–42
1.3.7.2.1. Data Division Organization....1–42

1.3.7.2.2. Data Division Structure 1–42
1.3.8. Procedure Division 1–44
1.3.8.1. General Description 1–44
1.3.8.1.1. Declaratives 1–44
1.3.8.1.2. Procedures 1–44
1.3.8.1.3. Execution 1–44
1.3.8.1.4. Procedure Division Structure 1–45
1.3.8.2. Statements and Sentences 1–45
1.3.8.2.1. Conditional Statements and Conditional Sentences 1–45
1.3.8.2.2. Compiler-Directing Statements and Compiler Directing Sentences 1–46
1.3.8.2.3. Imperative Statements and Imperative Sentences 1–46
1.3.8.2.4. Categories of Statements 1–47
1.3.9. Reference Format 1–49
1.3.9.1. General Description 1–49
1.3.9.2. Reference Format Representation 1–49
1.3.9.2.1. Sequence Numbers 1–50
1.3.9.2.2. Continuation of Lines 1–50
1.3.9.2.3. Blank Lines 1–50
1.3.9.3. Division, Section, Paragraph Formats 1–50
1.3.9.3.1. Division Header 1–50
1.3.9.3.2. Section Header 1–50
1.3.9.3.3. Paragraph Header, Paragraph-Name, and Paragraph 1–50
1.3.9.4. Data Division Entries 1–51
1.3.9.5. Declaratives 1–52
1.3.9.6. Comment Lines 1–52

Section 2. Nucleus

2.1. General 2–1
2.1.1. Function 2–1
2.1.2. Level Characteristics 2–1
2.1.3. Level Restrictions on Overall Language 2–1
2.1.3.1. Format Notation 2–1
2.1.3.2. Name Characteristics 2–1
2.1.3.3. Figurative Constants 2–2
2.1.3.4. Reference Format 2–2
2.2. Control Division 2–3
2.2.1. General Description 2–3
2.2.2. Organization 2–3
2.2.2.1. Structure 2–3
2.2.2.2. General Format 2–3
2.2.2.3. Syntax Rule 2–3
2.2.3. The Alphabet Section 2–3
2.2.3.1. The SOURCE–ALPHABET Clause 2–3
2.2.3.1.1. Function 2–3
2.2.3.1.2. General Format 2–4
2.2.3.1.3. Syntax Rules 2–4

2.2.3.1.4. General Rules 2–4
2.2.3.2. Example....2–4
2.3. Identification Division 2–5
2.3.1. General Description 2–5
2.3.2. Organization 2–5
2.3.2.1. Structure....2–5
2.3.2.2. General Format....2–5
2.3.2.3. Syntax Rules....2–5
2.3.3. The PROGRAM–ID Paragraph 2–6
2.3.3.1. Function....2–6
2.3.3.2. General Format....2–6
2.3.3.3. Syntax Rules....2–6
2.3.3.4. General Rules 2–6
2.3.4. The DATE–COMPILED Paragraph 2–6
2.3.4.1. Function....2–6
2.3.4.2. General Format....2–6
2.3.4.3. Syntax Rules....2–6
2.3.4.4. General Rules 2–6
2.4. Environment Division....2–7
2.4.1. Configuration Section....2–7
2.4.1.1. The SOURCE–COMPUTER Paragraph....2–7
2.4.1.1.1. Function 2–7
2.4.1.1.2. General Format 2–7
2.4.1.1.3. Syntax Rules 2–7
2.4.1.1.4. General Rules 2–7
2.4.1.2. The OBJECT–COMPUTER Paragraph....2–7
2.4.1.2.1. Function 2–7
2.4.1.2.2. General Format 2–7
2.4.1.2.3. Syntax Rules 2–7
2.4.1.2.4. General Rules 2–8
2.4.1.3. The SPECIAL–NAMES Paragraph 2–9
2.4.1.3.1. Function 2–9
2.4.1.3.2. General Format 2–9
2.4.1.3.3. Syntax Rules 2–10
2.4.1.3.4. General Rules 2–10
2.5. Data Division....2–13
2.5.1. Working–Storage Section 2–13
2.5.1.1. Noncontiguous Working–Storage 2–13
2.5.1.2. Working–Storage Records 2–13
2.5.1.3. Initial Values 2–13
2.5.2. The Data Description—Complete Entry Skeleton....2–14
2.5.2.1. Function....2–14
2.5.2.2. General Format....2–14
2.5.2.3. Syntax Rules....2–15
2.5.2.4. General Rules 2–15
2.5.3. The BLANK WHEN ZERO Clause 2–15
2.5.3.1. Function....2–15
2.5.3.2. General Format....2–15
2.5.3.3. Syntax Rules....2–15
2.5.3.4. General Rules 2–16
2.5.4. The Data–Name or FILLER Clause 2–16
2.5.4.1. Function....2–16

2.5.4.2. General Format 2–16
2.5.4.3. Syntax Rules 2–16
2.5.4.4. General Rules 2–16
2.5.5. The JUSTIFIED Clause 2–17
2.5.5.1. Function 2–17
2.5.5.2. General Format 2–17
2.5.5.3. Syntax Rules 2–17
2.5.5.4. General Rules 2–17
2.5.6. Level–Number 2–18
2.5.6.1. Function 2–18
2.5.6.2. General Format 2–18
2.5.6.3. Syntax Rules 2–18
2.5.6.4. General Rules 2–18
2.5.7. The PICTURE Clause 2–19
2.5.7.1. Function 2–19
2.5.7.2. General Format 2–19
2.5.7.3. Syntax Rules 2–19
2.5.7.4. General Rules 2–19
2.5.7.5. Editing Rules 2–25
2.5.7.6. Precedence Rules 2–30
2.5.8. The REDEFINES Clause 2–34
2.5.8.1. Function 2–34
2.5.8.2. General Format 2–34
2.5.8.3. Syntax Rules 2–34
2.5.8.4. General Rules 2–34
2.5.9. The RENAMES Clause 2–35
2.5.9.1. Function 2–35
2.5.9.2. General Format 2–35
2.5.9.3. Syntax Rules 2–35
2.5.9.4. General Rules 2–37
2.5.10. The SIGN Clause 2–37
2.5.10.1. Function 2–37
2.5.10.2. General Format 2–37
2.5.10.3. Syntax Rules 2–37
2.5.10.4. General Rules 2–38
2.5.11. The SYNCHRONIZED Clause 2–38
2.5.11.1. Function 2–38
2.5.11.2. General Format 2–39
2.5.11.3. Syntax Rules 2–39
2.5.11.4. General Rules 2–39
2.5.12. The USAGE Clause 2–40
2.5.12.1. Function 2–40
2.5.12.2. General Format 2–40
2.5.12.3. Syntax Rules 2–40
2.5.12.4. General Rules 2–41
2.5.13. The VALUE Clause 2–43
2.5.13.1. Function 2–43
2.5.13.2. General Format 2–43
2.5.13.3. Syntax Rules 2–43
2.5.13.4. General Rules 2–43
2.5.13.5. Condition–Name Rules 2–44

2.5.13.6. Data Description Entries Other Than Condition–Names....2–44
2.6. Procedure Division....2–46
2.6.1. Arithmetic Expressions....2–46
2.6.1.1. Definition of an Arithmetic Expression....2–46
2.6.1.2. Arithmetic Operators....2–46
2.6.1.3. Formation And Evaluation Rules....2–47
2.6.2. Conditional Expressions....2–50
2.6.2.1. Simple Conditions....2–50
2.6.2.1.1. Relation Condition....2–50
2.6.2.1.2. Class Condition....2–55
2.6.2.1.3. Condition-Name Condition (Conditional Variable)....2–56
2.6.2.1.4. Switch–Status Condition....2–57
2.6.2.1.5. Sign Condition....2–58
2.6.2.2. Complex Conditions....2–59
2.6.2.2.1. Negated Simple Conditions....2–59
2.6.2.2.2. Combined and Negated Combined Conditions....2–60
2.6.2.3. Abbreviated Combined Relation Conditions....2–61
2.6.2.4. Condition Evaluation Rules....2–63
2.6.3. Common Phrases and General Rules for Statement Formats....2–63
2.6.3.1. The ROUNDED Phrase....2–63
2.6.3.2. The SIZE ERROR Phrase....2–64
2.6.3.3. The CORRESPONDING Phrase....2–65
2.6.3.4. The Arithmetic Statements....2–67
2.6.3.5. Overlapping Operands....2–67
2.6.3.6. Multiple Results in Arithmetic Statements....2–68
2.6.3.7. Arithmetic Intermediate Results....2–68
2.6.3.8. Incompatible Data....2–79
2.6.4. The ACCEPT Statement....2–80
2.6.4.1. Function....2–80
2.6.4.2. General Format....2–80
2.6.4.3. Syntax Rules....2–80
2.6.4.4. General Rules....2–80
2.6.5. The ADD Statement....2–83
2.6.5.1. Function....2–83
2.6.5.2. General Format....2–83
2.6.5.3. Syntax Rules....2–83
2.6.5.4. General Rules....2–84
2.6.6. The ALTER Statement....2–85
2.6.6.1. Function....2–85
2.6.6.2. General Format....2–85
2.6.6.3. Syntax Rules....2–85
2.6.6.4. General Rules....2–85
2.6.7. The COMPUTE Statement....2–86
2.6.7.1. Function....2–86
2.6.7.2. General Format....2–86
2.6.7.3. Syntax Rules....2–86
2.6.7.4. General Rules....2–86
2.6.8. The DISPLAY Statement....2–87

2.6.8.1. Function ... 2–87
2.6.8.2. General Format ... 2–87
2.6.8.3. Syntax Rules ... 2–87
2.6.8.4. General Rules ... 2–87
2.6.9. The DIVIDE Statement ... 2–89
2.6.9.1. Function ... 2–89
2.6.9.2. General Format ... 2–89
2.6.9.3. Syntax Rules ... 2–90
2.6.9.4. General Rules ... 2–90
2.6.10. The EXIT Statement ... 2–91
2.6.10.1. Function ... 2–91
2.6.10.2. General Format ... 2–91
2.6.10.3. Syntax Rules ... 2–91
2.6.10.4. General Rules ... 2–92
2.6.11. The GO TO Statement ... 2–92
2.6.11.1. Function ... 2–92
2.6.11.2. General Format ... 2–92
2.6.11.3. Syntax Rules ... 2–92
2.6.11.4. General Rules ... 2–93
2.6.12. The IF Statement ... 2–93
2.6.12.1. Function ... 2–93
2.6.12.2. General Format ... 2–93
2.6.12.3. Syntax Rules ... 2–93
2.6.12.4. General Rules ... 2–93
2.6.13. The INSPECT Statement ... 2–95
2.6.13.1. Function ... 2–95
2.6.13.2. General Format ... 2–95
2.6.13.3. Syntax Rules ... 2–96
2.6.13.4. General Rules ... 2–97
2.6.13.5. Examples ... 2–100
2.6.14. The MOVE Statement ... 2–101
2.6.14.1. Function ... 2–101
2.6.14.2. General Format ... 2–101
2.6.14.3. Syntax Rules ... 2–101
2.6.14.4. General Rules ... 2–102
2.6.15. The MULTIPLY Statement ... 2–105
2.6.15.1. Function ... 2–105
2.6.15.2. General Format ... 2–105
2.6.15.3. Syntax Rules ... 2–105
2.6.15.4. General Rules ... 2–105
2.6.16. The PERFORM Statement ... 2–106
2.6.16.1. Function ... 2–106
2.6.16.2. General Format ... 2–106
2.6.16.3. Syntax Rules ... 2–107
2.6.16.4. General Rules ... 2–107
2.6.17. The SET Statement ... 2–113
2.6.17.1. Function ... 2–113
2.6.17.2. General Format ... 2–113
2.6.17.3. Syntax Rules ... 2–113
2.6.17.4. General Rules ... 2–113
2.6.18. The STOP Statement ... 2–113
2.6.18.1. Function ... 2–113

2.6.18.2. General Format....2–113
2.6.18.3. Syntax Rules....2–113
2.6.18.4. General Rules....2–114
2.6.19. The STRING Statement....2–114
2.6.19.1. Function....2–114
2.6.19.2. General Format....2–114
2.6.19.3. Syntax Rules....2–114
2.6.19.4. General Rules....2–115
2.6.20. The SUBTRACT Statement....2–117
2.6.20.1. Function....2–117
2.6.20.2. General Format....2–117
2.6.20.3. Syntax Rules....2–118
2.6.20.4. General Rules....2–118
2.6.21. The UNSTRING Statement....2–119
2.6.21.1. Function....2–119
2.6.21.2. General Format....2–119
2.6.21.3. Syntax Rules....2–119
2.6.21.4. General Rules....2–120

Section 3. Table Handling

3.1. General....3–1
3.1.1. Function....3–1
3.1.2. Level Characteristics....3–1
3.2. Data Division....3–2
3.2.1. The OCCURS Clause....3–2
3.2.1.1. Function....3–2
3.2.1.2. General Format....3–2
3.2.1.3. Syntax Rules....3–2
3.2.1.4. General Rules....3–3
3.2.2. The USAGE Clause....3–6
3.2.2.1. Function....3–6
3.2.2.2. General Format....3–6
3.2.2.3. Syntax Rules....3–6
3.2.2.4. General Rules....3–6
3.3. Procedure Division....3–7
3.3.1. Relation Condition....3–7
3.3.1.1. Comparisons Involving Index-Names and/or Index Data Items....3–7
3.3.2. Overlapping Operands....3–7
3.3.3. The SEARCH Statement....3–7
3.3.3.1. Function....3–7
3.3.3.2. General Format....3–7
3.3.3.3. Syntax Rules....3–8
3.3.3.4. General Rules....3–9
3.3.4. The SET Statement....3–11
3.3.4.1. Function....3–11
3.3.4.2. General Format....3–11
3.3.4.3. Syntax Rules....3–12
3.3.4.4. General Rules....3–12

Section 4. Sequential I-O

4.1. General ... 4–1
4.1.1. Function ... 4–1
4.1.2. Level Characteristics ... 4–1
4.1.3. Language Concepts ... 4–1
4.1.3.1. Organization ... 4–1
4.1.3.2. Access Mode ... 4–1
4.1.3.3. Current Record Pointer ... 4–2
4.1.3.4. I–O Status ... 4–2
4.1.3.4.1. Status Key 1 ... 4–2
4.1.3.4.2. Status Key 2 ... 4–3
4.1.3.4.3. Valid Combinations of Status Keys 1 and 2 ... 4–3
4.1.3.5. The AT END Condition ... 4–3
4.1.3.6. LINAGE–COUNTER ... 4–3
4.2. Environment Division ... 4–4
4.2.1. INPUT–OUTPUT Section ... 4–4
4.2.1.1. The FILE–CONTROL Paragraph ... 4–4
4.2.1.1.1. Function ... 4–4
4.2.1.1.2. General Format ... 4–4
4.2.1.2. The File Control Entry ... 4–4
4.2.1.2.1. Function ... 4–4
4.2.1.2.2. General Format ... 4–4
4.2.1.2.3. Syntax Rules ... 4–4
4.2.1.2.4. General Rules ... 4–5
4.2.1.3. The I–O–CONTROL Paragraph ... 4–6
4.2.1.3.1. Function ... 4–6
4.2.1.3.2. General Format ... 4–6
4.2.1.3.3. Syntax Rules ... 4–6
4.2.1.3.4. General Rules ... 4–7
4.3. Data Division ... 4–8
4.3.1. File Section ... 4–8
4.3.2. Record Description Structure ... 4–8
4.3.3. The File Description—Complete Entry Skeleton ... 4–8
4.3.3.1. Function ... 4–8
4.3.3.2. General Format ... 4–9
4.3.3.3. Syntax Rules ... 4–9
4.3.4. The BLOCK CONTAINS Clause ... 4–9
4.3.4.1. Function ... 4–9
4.3.4.2. General Format ... 4–10
4.3.4.3. General Rules ... 4–10
4.3.5. The CODE–SET Clause ... 4–10
4.3.5.1. Function ... 4–10
4.3.5.2. General Format ... 4–10
4.3.5.3. Syntax Rules ... 4–10
4.3.5.4. General Rules ... 4–11
4.3.6. The DATA RECORDS Clause ... 4–11
4.3.6.1. Function ... 4–11
4.3.6.2. General Format ... 4–11
4.3.6.3. Syntax Rules ... 4–11
4.3.6.4. General Rules ... 4–11

4.3.7. The DENSITY Clause 4–11
4.3.7.1. Function....4–11
4.3.7.2. General Format....4–11
4.3.7.3. Syntax Rules....4–11
4.3.7.4. General Rules 4–12
4.3.8. The LABEL RECORDS Clause 4–12
4.3.8.1. Function....4–12
4.3.8.2. General Format....4–12
4.3.8.3. Syntax Rules....4–12
4.3.8.4. General Rules 4–12
4.3.9. The LINAGE Clause 4–12
4.3.9.1. Function....4–12
4.3.9.2. General Format....4–12
4.3.9.3. Syntax Rules....4–13
4.3.9.4. General Rules 4–13
4.3.10. The RECORD CONTAINS Clause 4–15
4.3.10.1. Function....4–15
4.3.10.2. General Format....4–15
4.3.10.3. General Rules 4–15
4.3.11. The VALUE OF Clause 4–15
4.3.11.1. Function....4–15
4.3.11.2. General Format....4–15
4.3.11.3. Syntax Rules....4–16
4.3.11.4. General Rules 4–16
4.4. Procedure Division....4–17
4.4.1. The CLOSE Statement....4–17
4.4.1.1. Function....4–17
4.4.1.2. General Format....4–17
4.4.1.3. Syntax Rules....4–17
4.4.1.4. General Rules 4–17
4.4.2. The OPEN Statement 4–21
4.4.2.1. Function....4–21
4.4.2.2. General Format....4–21
4.4.2.3. Syntax Rules....4–21
4.4.2.4. General Rules 4–21
4.4.3. The READ Statement....4–25
4.4.3.1. Function....4–25
4.4.3.2. General Format....4–25
4.4.3.3. Syntax Rules....4–25
4.4.3.4. General Rules 4–25
4.4.4. The REWRITE Statement 4–27
4.4.4.1. Function....4–27
4.4.4.2. General Format....4–27
4.4.4.3. Syntax Rules....4–27
4.4.4.4. General Rules 4–27
4.4.5. The USE Statement 4–28
4.4.5.1. Function....4–28
4.4.5.2. General Format....4–28
4.4.5.3. Syntax Rules....4–28
4.4.5.4. General Rules 4–28
4.4.6. The WRITE Statement 4–29
4.4.6.1. Function....4–29

4.4.6.2. General Format 4–29
4.4.6.3. Syntax Rules 4–29
4.4.6.4. General Rules 4–30
4.5. Example 4–33
4.5.1. Source Program 4–33
4.5.2. Generated Report 4–38
4.6. Sequential File Assignment 4–39
4.6.1. Non-Symbiont Files 4–39
4.6.1.1. Assignment Examples 4–39
4.6.1.2. COBOL Program Links 4–39
4.6.1.3. Device Independency for ANSI Tape Files 4–39
4.6.1.4. Device Independency for Sequential DISC Files 4–40
4.6.2. Symbiont Files 4–40
4.6.2.1. Assignment Rules 4–40
4.6.2.2. COBOL Program Links 4–40
4.6.3. Alternate Symbiont Files 4–40
4.6.3.1. Assignment Rules 4–40
4.6.3.2. Assignment Examples 4–40
4.6.3.3. COBOL Program Links 4–41

Section 5. Relative I-O

5.1. General 5–1
5.1.1. Function 5–1
5.1.2. Level Characteristics 5–1
5.1.3. Language Concepts 5–1
5.1.3.1. Organization 5–1
5.1.3.2. Access Modes 5–2
5.1.3.3. Current Record Pointer 5–2
5.1.3.4. I–O Status 5–2
5.1.3.4.1. Status Key 1 5–2
5.1.3.4.2. Status Key 2 5–3
5.1.3.4.3. Valid Combinations of Status Keys 1 and 2 5–3
5.1.3.5. The INVALID KEY Condition 5–4
5.1.3.6. The AT END Condition 5–4
5.2. Environment Division 5–5
5.2.1. INPUT–OUTPUT Section 5–5
5.2.1.1. The FILE–CONTROL Paragraph 5–5
5.2.1.1.1. Function 5–5
5.2.1.1.2. General Format 5–5
5.2.1.2. The File Control Entry 5–5
5.2.1.2.1. Function 5–5
5.2.1.2.2. General Format 5–5
5.2.1.2.3. Syntax Rules 5–5
5.2.1.2.4. General Rules 5–6
5.2.1.3. The I–O–CONTROL Paragraph 5–7
5.2.1.3.1. Function 5–7
5.2.1.3.2. General Format 5–7
5.2.1.3.3. Syntax Rules 5–7

5.2.1.3.4. General Rules 5–8
5.3. Data Division 5–9
5.3.1. File Section 5–9
5.3.2. Record Description Structure 5–9
5.3.3. The File Description—Complete Entry Skeleton 5–9
5.3.3.1. Function 5–9
5.3.3.2. General Format 5–9
5.3.3.3. Syntax Rules 5–10
5.3.4. The BLOCK CONTAINS Clause 5–10
5.3.4.1. Function 5–10
5.3.4.2. General Format 5–10
5.3.4.3. Syntax Rules 5–10
5.3.5. The DATA RECORDS Clause 5–10
5.3.5.1. Function 5–10
5.3.5.2. General Format 5–10
5.3.5.3. Syntax Rules 5–10
5.3.5.4. General Rules 5–10
5.3.6. The LABEL RECORDS Clause 5–11
5.3.6.1. Function 5–11
5.3.6.2. General Format 5–11
5.3.6.3. Syntax Rules 5–11
5.3.6.4. General Rules 5–11
5.3.7. The RECORD CONTAINS Clause 5–11
5.3.7.1. Function 5–11
5.3.7.2. General Format 5–11
5.3.7.3. General Rules 5–11
5.3.8. The VALUE OF Clause 5–12
5.3.8.1. Function 5–12
5.3.8.2. General Format 5–12
5.3.8.3. Syntax Rules 5–12
5.3.8.4. General Rules 5–12
5.4. Procedure Division 5–13
5.4.1. The CLOSE Statement 5–13
5.4.1.1. Function 5–13
5.4.1.2. General Format 5–13
5.4.1.3. Syntax Rules 5–13
5.4.1.4. General Rules 5–13
5.4.2. The DELETE Statement 5–14
5.4.2.1. Function 5–14
5.4.2.2. General Format 5–14
5.4.2.3. Syntax Rules 5–14
5.4.2.4. General Rules 5–14
5.4.3. The OPEN Statement 5–15
5.4.3.1. Function 5–15
5.4.3.2. General Format 5–15
5.4.3.3. Syntax Rules 5–15
5.4.3.4. General Rules 5–15
5.4.4. The READ Statement 5–17
5.4.4.1. Function 5–17
5.4.4.2. General Format 5–17
5.4.4.3. Syntax Rules 5–17
5.4.4.4. General Rules 5–18

5.4.5. The REWRITE Statement 5–19
5.4.5.1. Function 5–19
5.4.5.2. General Format 5–19
5.4.5.3. Syntax Rules 5–20
5.4.5.4. General Rules 5–20
5.4.6. The START Statement 5–21
5.4.6.1. Function 5–21
5.4.6.2. General Format 5–21
5.4.6.3. Syntax Rules 5–21
5.4.6.4. General Rules 5–21
5.4.7. The USE Statement 5–22
5.4.7.1. Function 5–22
5.4.7.2. General Format 5–22
5.4.7.3. Syntax Rules 5–22
5.4.7.4. General Rules 5–22
5.4.8. The WRITE Statement 5–23
5.4.8.1. Function 5–23
5.4.8.2. General Format 5–23
5.4.8.3. Syntax Rules 5–23
5.4.8.4. General Rules 5–23
5.5. Relative I–O File Assignment 5–25
5.5.1. Assignment Examples 5–25
5.5.2. COBOL Program Links 5–25

Section 6. Indexed I-O

6.1. General 6–1
6.1.1. Function 6–1
6.1.2. Level Characteristics 6–1
6.1.3. Language Concepts 6–1
6.1.3.1. Organization 6–1
6.1.3.2. Access Modes 6–2
6.1.3.3. Current Record Pointer 6–2
6.1.3.4. I–O Status 6–2
6.1.3.4.1. Status Key 1 6–2
6.1.3.4.2. Status Key 2 6–3
6.1.3.4.3. Valid Combinations of Status Keys 1 and 2 6–4
6.1.3.5. The INVALID KEY Condition 6–4
6.1.3.6. The AT END Condition 6–4
6.1.4. Information Statistics 6–5
6.1.4.1. Function 6–5
6.1.4.2. Calling Format 6–5
6.1.4.3. Syntax Rules 6–5
6.1.4.4. General Rules 6–5
6.1.4.5. Data–Item Format 6–6
6.2. Environment Division 6–8
6.2.1. INPUT–OUTPUT Section 6–8
6.2.1.1. The FILE–CONTROL Paragraph 6–8
6.2.1.1.1. Function 6–8
6.2.1.1.2. General Format 6–8

6.2.1.2. The File Control Entry....6–8
6.2.1.2.1. Function....6–8
6.2.1.2.2. General Format....6–8
6.2.1.2.3. Syntax Rules....6–8
6.2.1.2.4. General Rules....6–9
6.2.1.3. The I–O–CONTROL Paragraph....6–10
6.2.1.3.1. Function....6–10
6.2.1.3.2. General Format....6–10
6.2.1.3.3. Syntax Rules....6–10
6.2.1.3.4. General Rules....6–11
6.3. Data Division....6–12
6.3.1. File Section....6–12
6.3.2. Record Description Structure....6–12
6.3.3. The File Description—Complete Entry Skeleton....6–12
6.3.3.1. Function....6–12
6.3.3.2. General Format....6–12
6.3.3.3. Syntax Rules....6–13
6.3.4. The BLOCK CONTAINS Clause....6–13
6.3.4.1. Function....6–13
6.3.4.2. General Format....6–13
6.3.4.3. General Rules....6–13
6.3.5. The DATA RECORDS Clause....6–13
6.3.5.1. Function....6–13
6.3.5.2. General Format....6–13
6.3.5.3. Syntax Rules....6–14
6.3.5.4. General Rules....6–14
6.3.6. The LABEL RECORDS Clause....6–14
6.3.6.1. Function....6–14
6.3.6.2. General Format....6–14
6.3.6.3. Syntax Rules....6–14
6.3.6.4. General Rules....6–14
6.3.7. The RECORD CONTAINS Clause....6–14
6.3.7.1. Function....6–14
6.3.7.2. General Format....6–14
6.3.7.3. General Rules....6–15
6.3.8. The VALUE OF Clause....6–15
6.3.8.1. Function....6–15
6.3.8.2. General Format....6–15
6.3.8.3. Syntax Rules....6–15
6.3.8.4. General Rules....6–16
6.4. Procedure Division....6–17
6.4.1. The CLOSE Statement....6–17
6.4.1.1. Function....6–17
6.4.1.2. General Format....6–17
6.4.1.3. Syntax Rules....6–17
6.4.1.4. General Rules....6–17
6.4.2. The DELETE Statement....6–18
6.4.2.1. Function....6–18
6.4.2.2. General Format....6–18
6.4.2.3. Syntax Rules....6–18
6.4.2.4. General Rules....6–19
6.4.3. The OPEN Statement....6–19

6.4.3.1. Function 6–19
6.4.3.2. General Format 6–19
6.4.3.3. Syntax Rules 6–19
6.4.3.4. General Rules 6–19
6.4.4. The READ Statement 6–21
6.4.4.1. Function 6–21
6.4.4.2. General Format 6–21
6.4.4.3. Syntax Rules 6–21
6.4.4.4. General Rules 6–22
6.4.5. The REWRITE Statement 6–24
6.4.5.1. Function 6–24
6.4.5.2. General Format 6–24
6.4.5.3. Syntax Rules 6–24
6.4.5.4. General Rules 6–24
6.4.6. The START Statement 6–25
6.4.6.1. Function 6–25
6.4.6.2. General Format 6–25
6.4.6.3. Syntax Rules 6–26
6.4.6.4. General Rules 6–26
6.4.7. The USE Statement 6–27
6.4.7.1. Function 6–27
6.4.7.2. General Format 6–27
6.4.7.3. Syntax Rules 6–27
6.4.7.4. General Rules 6–28
6.4.8. The WRITE Statement 6–28
6.4.8.1. Function 6–28
6.4.8.2. General Format 6–28
6.4.8.3. Syntax Rules 6–28
6.4.8.4. General Rules 6–28
6.5. Indexed I–O File Assignment 6–31
6.5.1. Assignment Examples 6–31
6.5.2. COBOL Program Links 6–31

Section 7. Sort-Merge

7.1. General 7–1
7.1.1. Function 7–1
7.1.2. Level Characteristics 7–1
7.1.3. Relationship with Sequential I–O Module 7–1
7.2. Environment Division 7–2
7.2.1. INPUT–OUTPUT Section 7–2
7.2.1.1. The FILE–CONTROL Paragraph 7–2
7.2.1.1.1. Function 7–2
7.2.1.1.2. General Format 7–2
7.2.1.2. The File Control Entry 7–2
7.2.1.2.1. Function 7–2
7.2.1.2.2. General Format 7–2
7.2.1.2.3. Syntax Rules 7–2
7.2.1.2.4. General Rules 7–2
7.2.1.3. The I–O–CONTROL Paragraph 7–3
7.2.1.3.1. Function 7–3

7.2.1.3.2. General Format 7–3
7.2.1.3.3. Syntax Rules 7–3
7.2.1.3.4. General Rules 7–4
7.3. Data Division 7–5
7.3.1. File Section 7–5
7.3.2. The Sort-Merge File Description—Complete Entry Skeleton 7–5
7.3.2.1. Function 7–5
7.3.2.2. General Format 7–5
7.3.2.3. Syntax Rules 7–5
7.3.3. The DATA RECORDS Clause 7–5
7.3.3.1. Function 7–5
7.3.3.2. General Format 7–5
7.3.3.3. Syntax Rules 7–6
7.3.3.4. General Rules 7–6
7.3.4. The RECORD CONTAINS Clause 7–6
7.3.4.1. Function 7–6
7.3.4.2. General Format 7–6
7.3.4.3. General Rules 7–6
7.4. Procedure Division 7–7
7.4.1. The MERGE Statement 7–7
7.4.1.1. Function 7–7
7.4.1.2. General Format 7–7
7.4.1.3. Syntax Rules 7–7
7.4.1.4. General Rules 7–8
7.4.2. The RELEASE Statement 7–11
7.4.2.1. Function 7–11
7.4.2.2. General Format 7–11
7.4.2.3. Syntax Rules 7–11
7.4.2.4. General Rules 7–11
7.4.3. The RETURN Statement 7–12
7.4.3.1. Function 7–12
7.4.3.2. General Format 7–12
7.4.3.3. Syntax Rules 7–12
7.4.3.4. General Rules 7–12
7.4.4. The SORT Statement 7–13
7.4.4.1. Function 7–13
7.4.4.2. General Format 7–13
7.4.4.3. Syntax Rules 7–13
7.4.4.4. General Rules 7–14
7.5. Examples 7–18

Section 8. Report Writer

8.1. General 8–1
8.1.1. Function 8–1
8.1.2. Language Concepts 8–1
8.1.2.1. LINE–COUNTER 8–1
8.1.2.2. PAGE–COUNTER 8–1
8.1.2.3. Subscripting 8–2
8.1.3. Relationship with Sequential I–O Module 8–2

8.2. Data Division 8–3
8.2.1. File Section 8–3
8.2.2. Report Section 8–3
8.2.2.1. Report Description Entry 8–3
8.2.2.2. Report Group Description Entry 8–3
8.2.3. The File Description—Complete Entry Skeleton 8–3
8.2.3.1. Function 8–3
8.2.3.2. General Format 8–4
8.2.3.3. Syntax Rules 8–4
8.2.4. The Report Description—Complete Entry Skeleton 8–4
8.2.4.1. Function 8–4
8.2.4.2. General Format 8–5
8.2.4.3. Syntax Rules 8–5
8.2.4.4. PAGE–COUNTER Rules 8–5
8.2.4.5. LINE–COUNTER Rules 8–6
8.2.5. The Report Group Description—Complete Skeleton 8–7
8.2.5.1. Function 8–7
8.2.5.2. General Format 8–7
8.2.5.3. Syntax Rules 8–9
8.2.5.4. General Rules 8–10
8.2.6. The BLOCK CONTAINS Clause 8–10
8.2.6.1. Function 8–10
8.2.6.2. General Format 8–10
8.2.6.3. General Rules 8–10
8.2.7. The CODE Clause 8–11
8.2.7.1. Function 8–11
8.2.7.2. General Format 8–11
8.2.7.3. Syntax Rules 8–11
8.2.7.4. General Rules 8–11
8.2.8. The CODE-SET Clause 8–11
8.2.8.1. Function 8–11
8.2.8.2. General Format 8–11
8.2.8.3. Syntax Rules 8–12
8.2.8.4. General Rules 8–12
8.2.9. The COLUMN NUMBER Clause 8–12
8.2.9.1. Function 8–12
8.2.9.2. General Format 8–12
8.2.9.3. Syntax Rules 8–12
8.2.9.4. General Rules 8–12
8.2.10. The CONTROL Clause 8–13
8.2.10.1. Function 8–13
8.2.10.2. General Format 8–13
8.2.10.3. Syntax Rules 8–13
8.2.10.4. General Rules 8–13
8.2.11. The Data–Name Clause 8–14
8.2.11.1. Function 8–14
8.2.11.2. General Format 8–14
8.2.11.3. Syntax Rule 8–14
8.2.11.4. General Rules 8–14
8.2.12. The DENSITY Clause 8–14

8.2.12.1. Function....8–14
8.2.12.2. General Format....8–14
8.2.12.3. Syntax Rule....8–14
8.2.12.4. General Rule....8–14
8.2.13. The GROUP INDICATE Clause....8–15
8.2.13.1. Function....8–15
8.2.13.2. General Format....8–15
8.2.13.3. Syntax Rule....8–15
8.2.13.4. General Rules....8–15
8.2.14. The LABEL RECORDS Clause....8–15
8.2.14.1. Function....8–15
8.2.14.1.1. General Format....8–15
8.2.14.2. Syntax Rule....8–15
8.2.14.3. General Rules....8–15
8.2.15. The LINE NUMBER Clause....8–16
8.2.15.1. Function....8–16
8.2.15.2. General Format....8–16
8.2.15.3. Syntax Rules....8–16
8.2.15.4. General Rules....8–16
8.2.16. The NEXT GROUP Clause....8–18
8.2.16.1. Function....8–18
8.2.16.2. General Format....8–18
8.2.16.3. Syntax Rules....8–18
8.2.16.4. General Rules....8–19
8.2.17. The PAGE Clause....8–19
8.2.17.1. Function....8–19
8.2.17.2. General Format....8–19
8.2.17.3. Syntax Rules....8–19
8.2.17.4. General Rules....8–20
8.2.17.5. PAGE Regions Table....8–22
8.2.18. The RECORD CONTAINS Clause....8–23
8.2.18.1. Function....8–23
8.2.18.2. General Format....8–23
8.2.18.3. General Rules....8–23
8.2.19. The REPORT Clause....8–23
8.2.19.1. Function....8–23
8.2.19.2. General Format....8–23
8.2.19.3. Syntax Rules....8–23
8.2.19.4. General Rules....8–24
8.2.20. The SOURCE Clause....8–24
8.2.20.1. Function....8–24
8.2.20.2. General Format....8–24
8.2.20.3. Syntax Rules....8–24
8.2.20.4. General Rule....8–24
8.2.21. The SUM Clause....8–24
8.2.21.1. Function....8–24
8.2.21.2. General Format....8–24
8.2.21.3. Syntax Rules....8–25
8.2.21.4. General Rules....8–25
8.2.22. The TYPE Clause....8–27
8.2.22.1. Function....8–27
8.2.22.2. General Format....8–27

8.2.22.3. Syntax Rules ... 8–27
8.2.22.4. General Rules ... 8–28
8.2.23. The VALUE OF Clause ... 8–31
8.2.23.1. Function ... 8–31
8.2.23.2. General Format ... 8–31
8.2.23.3. Syntax Rules ... 8–31
8.2.23.4. General Rules ... 8–32
8.3. Procedure Division ... 8–33
8.3.1. The GENERATE Statement ... 8–33
8.3.1.1. Function ... 8–33
8.3.1.2. General Format ... 8–33
8.3.1.3. Syntax Rules ... 8–33
8.3.1.4. General Rules ... 8–33
8.3.2. The INITIATE Statement ... 8–35
8.3.2.1. Function ... 8–35
8.3.2.2. General Format ... 8–35
8.3.2.3. Syntax Rules ... 8–35
8.3.2.4. General Rules ... 8–35
8.3.3. The SUPPRESS Statement ... 8–36
8.3.3.1. Function ... 8–36
8.3.3.2. General Format ... 8–36
8.3.3.3. Syntax Rules ... 8–36
8.3.3.4. General Rules ... 8–36
8.3.4. The TERMINATE Statement ... 8–36
8.3.4.1. Function ... 8–36
8.3.4.2. General Format ... 8–36
8.3.4.3. Syntax Rules ... 8–36
8.3.4.4. General Rules ... 8–37
8.3.5. The USE Statement ... 8–37
8.3.5.1. Function ... 8–37
8.3.5.2. General Format ... 8–37
8.3.5.3. Syntax Rules ... 8–37
8.3.5.4. General Rules ... 8–38
8.4. Report Format on File other than Printer File ... 8–39
8.5. Examples ... 8–40
8.5.1. Program 1 ... 8–40
8.5.1.1. Input Data ... 8–40
8.5.1.2. Source Program ... 8–41
8.5.1.3. Output Report ... 8–43
8.5.1.4. Relation Between Output Report and Source Program ... 8–45
8.5.2. Program 2 ... 8–47
8.5.2.1. Input Data ... 8–47
8.5.2.1.1. Source Program ... 8–48
8.5.2.1.2. Output Report ... 8–50
8.5.2.2. Relation Between Output Report and Source Program ... 8–54

Section 9. Segmentation

9.1. General ... 9–1

9.1.1. Function 9–1
9.1.2. Level Characteristics....9–1
9.2. General Description 9–2
9.2.1. Scope 9–2
9.2.2. Organization 9–2
9.2.2.1. Program Segments 9–2
9.2.2.2. Fixed Portion 9–2
9.2.2.3. Independent Segments....9–3
9.2.3. Segmentation Classification....9–3
9.2.4. Segmentation Control....9–4
9.3. Structure of Program Segments....9–5
9.3.1. Segment–Numbers....9–5
9.3.1.1. General Format....9–5
9.3.1.2. Syntax Rules....9–5
9.3.1.3. General Rules....9–5
9.3.2. SEGMENT–LIMIT Clause....9–5
9.3.2.1. General Format....9–5
9.3.2.2. Syntax Rules....9–6
9.3.2.3. General Rules....9–6
9.4. Restriction on Program Flow....9–7
9.4.1. The ALTER Statement....9–7
9.4.2. The PERFORM Statement....9–7
9.4.3. The MERGE Statement....9–7
9.4.4. The SORT Statement....9–8

Section 10. Library

10.1. General 10–1
10.1.1. Function 10–1
10.1.2. Level Characteristics....10–1
10.2. The COPY Statement 10–2
10.2.1. Function 10–2
10.2.2. General Format 10–2
10.2.3. Syntax Rules 10–2
10.2.4. General Rules....10–3
10.3. Examples 10–5

Section 11. Debug

11.1. General 11–1
11.1.1. Function 11–1
11.1.2. Level Characteristics....11–1
11.1.3. Language Concepts 11–1
11.1.3.1. DEBUG–ITEM....11–1
11.1.3.2. Compile Time Switch 11–2
11.1.3.3. Object Time Switch....11–2
11.2. Environment Division....11–3
11.2.1. The WITH DEBUGGING MODE Clause....11–3
11.2.1.1. Function....11–3
11.2.1.2. General Format....11–3
11.2.1.3. General Rules....11–3

11.3. Procedure Division 11–4
11.3.1. The USE FOR DEBUGGING Statement 11–4
11.3.1.1. Function 11–4
11.3.1.2. General Format 11–4
11.3.1.3. Syntax Rules 11–4
11.3.1.4. General Rules 11–5
11.3.2. Debugging Lines 11–9
11.4. Table Error Facility 11–10
11.4.1. General 11–10
11.4.2. Compiler Option 11–10
11.4.2.1. Function 11–10
11.4.2.2. General Rule 11–10
11.4.3. Procedure Division 11–10
11.4.3.1. The DECLARATIVE Paragraph 11–10
11.4.3.1.1. Function 11–10
11.4.3.1.2. General Format 11–10
11.4.3.1.3. Syntax Rules 11–10
11.4.3.1.4. General Rules 11–11
11.5. Programmers Advanced Debugging System (PADS 2200) 11–12
11.5.1. General 11–12
11.5.2. Compiler Option 11–12
11.5.2.1. Function 11–12
11.5.2.2. General Rules 11–12
11.5.3. Procedure Division 11–13
11.5.3.1. The CALL Statement 11–13
11.5.3.1.1. Function 11–13
11.5.3.1.2. General Format 11–13
11.5.3.1.3. Syntax Rules 11–13
11.5.3.1.4. General Rules 11–13

Section 12. Interprogram Communication

12.1. General 12–1
12.1.1. Function 12–1
12.1.2. Level Characteristics 12–1
12.2. Environment Division 12–2
12.2.1. The I–O–CONTROL Paragraph 12–2
12.2.1.1. The APPLY Clause 12–2
12.2.1.1.1. Function 12–2
12.2.1.1.2. General Format 12–2
12.2.1.1.3. Syntax Rules 12–2
12.2.1.1.4. General Rules 12–2
12.3. Data Division 12–4
12.3.1. Linkage Section 12–4
12.3.1.1. Noncontiguous Linkage Storage 12–4
12.3.1.2. Linkage Records 12–5
12.3.1.3. Initial Values 12–5
12.3.2. Common-Storage Section 12–5
12.4. Procedure Division 12–6
12.4.1. The Procedure Division Header 12–6

12.4.2. The CALL Statement 12–7
12.4.2.1. Function....12–7
12.4.2.2. General Format....12–7
12.4.2.3. Syntax Rules....12–7
12.4.2.4. General Rules 12–8
12.4.3. The CANCEL Statement 12–9
12.4.3.1. Function....12–9
12.4.3.2. General Format....12–9
12.4.3.3. Syntax Rules....12–9
12.4.3.4. General Rules 12–9
12.4.4. The ENTER Statement....12–10
12.4.4.1. Function....12–10
12.4.4.2. General Format....12–10
12.4.4.3. Syntax Rules....12–10
12.4.4.4. General Rules 12–11
12.4.5. The EXIT PROGRAM Statement 12–11
12.4.5.1. Function....12–11
12.4.5.2. General Format....12–11
12.4.5.3. Syntax Rules....12–11
12.4.5.4. General Rules 12–11

Section 13. Communication

13.1. General 13–1
13.1.1. Function 13–1
13.1.2. Level Characteristics....13–1
13.2. Data Division....13–2
13.2.1. Communication Section....13–2
13.2.2. The Communication Description—Complete Entry Skeleton 13–2
13.2.2.1. Function....13–2
13.2.2.2. General Format....13–2
13.2.2.3. Syntax Rules....13–3
13.2.2.4. General Rules 13–6
13.3. Procedure Division....13–11
13.3.1. The ACCEPT MESSAGE COUNT Statement....13–11
13.3.1.1. Function....13–11
13.3.1.2. General Format....13–11
13.3.1.3. Syntax Rules....13–11
13.3.1.4. General Rules 13–11
13.3.2. The DISABLE Statement 13–11
13.3.2.1. Function....13–11
13.3.2.2. General Format....13–11
13.3.2.3. Syntax Rules....13–11
13.3.2.4. General Rules 13–12
13.3.3. The ENABLE Statement 13–12
13.3.3.1. Function....13–12
13.3.3.2. General Format....13–12
13.3.3.3. Syntax Rules....13–13
13.3.3.4. General Rules 13–13
13.3.4. The RECEIVE Statement....13–13

13.3.4.1. Function 13–13
13.3.4.2. General Format 13–14
13.3.4.3. Syntax Rule 13–14
13.3.4.4. General Rules 13–14
13.3.5. The SEND Statement 13–15
13.3.5.1. Function 13–15
13.3.5.2. General Format 13–15
13.3.5.3. Syntax Rules 13–16
13.3.5.4. General Rules 13–16
13.4. System Considerations 13–19

Section 14. Asynchronous Processing

14.1. General 14–1
14.1.1. Asynchronous Control System (ACS) 14–1
14.1.2. Local and Global Data 14–2
14.2. Environment Division 14–4
14.2.1. Input–Output Section 14–4
14.2.1.1. The FILE–CONTROL Paragraph 14–4
14.2.1.1.1. Function 14–4
14.2.1.1.2. General Format 14–4
14.2.1.2. The File Control Entry 14–4
14.2.1.2.1. Function 14–4
14.2.1.2.2. General Format 14–4
14.2.1.2.3. General Rules 14–4
14.3. Data Division 14–6
14.3.1. The USAGE Clause 14–6
14.3.1.1. Function 14–6
14.3.1.2. General Format 14–6
14.3.1.3. Syntax Rules 14–6
14.3.1.4. General Rules 14–6
14.3.2. File Section 14–6
14.3.2.1. The Saved–Area Description—Complete Entry Skeleton 14–6
14.3.2.1.1. Function 14–6
14.3.2.1.2. General Format 14–7
14.3.2.1.3. Syntax Rules 14–7
14.3.2.2. The AREA CONTAINS Clause 14–7
14.3.2.2.1. Function 14–7
14.3.2.2.2. General Format 14–7
14.3.2.2.3. Syntax Rules 14–7
14.3.2.2.4. General Rules 14–7
14.3.2.3. The WITH RANK Clause 14–8
14.3.2.3.1. Function 14–8
14.3.2.3.2. General Format 14–8
14.3.2.3.3. Syntax Rules 14–8
14.3.2.3.4. General Rules 14–8
14.3.3. Communication Section 14–8
14.3.3.1. The PROCESSING MODE Clause 14–8
14.3.3.1.1. Function 14–8
14.3.3.1.2. General Format 14–8

14.3.3.1.3. General Rules 14–8
14.4. Procedure Division....14–9
14.4.1. The ALL Conditional....14–9
14.4.1.1. Function....14–9
14.4.1.2. General Format....14–9
14.4.1.3. Syntax Rules....14–9
14.4.1.4. General Rules....14–9
14.4.2. The ANY Conditional....14–9
14.4.2.1. Function....14–9
14.4.2.2. General Format....14–9
14.4.2.3. Syntax Rules....14–10
14.4.2.4. General Rules....14–10
14.4.3. The HOLD Statement....14–10
14.4.3.1. Function....14–10
14.4.3.2. General Format....14–10
14.4.3.3. Syntax Rules....14–10
14.4.3.4. General Rules....14–11
14.4.4. The LOCK Statement....14–11
14.4.4.1. Function....14–11
14.4.4.2. General Format....14–11
14.4.4.3. Syntax Rule....14–11
14.4.4.4. General Rules....14–11
14.4.5. The PROCESS Statement....14–11
14.4.5.1. Function....14–11
14.4.5.2. General Format....14–11
14.4.5.3. Syntax Rules....14–12
14.4.5.4. General Rules....14–12
14.4.6. The UNLOCK Statement....14–13
14.4.6.1. Function....14–13
14.4.6.2. General Format....14–13
14.4.6.3. Syntax Rules....14–13
14.4.6.4. General Rules....14–13
14.4.7. The USE Statement....14–13
14.4.7.1. Function....14–13
14.4.7.2. General Format....14–13
14.4.7.3. Syntax Rules....14–13
14.4.7.4. General Rules....14–14

Section 15. IBM Tape Processing

15.1. General....15–1
15.1.1. Function....15–1
15.1.2. Language Concepts....15–1
15.1.2.1. Organization....15–1
15.1.2.2. Access Mode....15–1
15.1.2.3. Current Record Pointer....15–1
15.1.2.4. The AT END Condition....15–1
15.2. Environment Division....15–2
15.2.1. Input–Output Section....15–2
15.2.1.1. The FILE–CONTROL Paragraph....15–2
15.2.1.1.1. Function....15–2

15.2.1.1.2. General Format 15–2
15.2.1.2. The File Control Entry 15–2
15.2.1.2.1. Function 15–2
15.2.1.2.2. General Format 15–2
15.2.1.2.3. Syntax Rules 15–2
15.2.1.2.4. General Rules 15–3
15.2.1.3. The I–O–CONTROL Paragraph 15–3
15.2.1.3.1. Function 15–3
15.2.1.3.2. General Format 15–3
15.2.1.3.3. Syntax Rules 15–3
15.2.1.3.4. General Rules 15–4
15.3. Data Division 15–5
15.3.1. File Section 15–5
15.3.2. The File Description—Complete Entry Skeleton 15–5
15.3.2.1. Function 15–5
15.3.2.2. General Format 15–5
15.3.2.3. Syntax Rules 15–5
15.3.3. The BLOCK CONTAINS Clause 15–6
15.3.3.1. Function 15–6
15.3.3.2. General Format 15–6
15.3.3.3. General Rules 15–6
15.3.4. The DATA RECORDS Clause 15–6
15.3.4.1. Function 15–6
15.3.4.2. General Format 15–6
15.3.4.3. Syntax Rules 15–6
15.3.4.4. General Rules 15–7
15.3.5. The LABEL RECORDS Clause 15–7
15.3.5.1. Function 15–7
15.3.5.2. General Format 15–7
15.3.5.3. Syntax Rules 15–7
15.3.5.4. General Rules 15–7
15.3.6. The RECORD CONTAINS Clause 15–7
15.3.6.1. Function 15–7
15.3.6.2. General Format 15–7
15.3.6.3. General Rules 15–7
15.3.7. The RECORDING MODE Clause 15–8
15.3.7.1. Function 15–8
15.3.7.2. General Format 15–8
15.3.7.3. General Rules 15–8
15.3.8. The VALUE OF Clause 15–9
15.3.8.1. Function 15–9
15.3.8.2. General Format 15–9
15.3.8.3. Syntax Rules 15–9
15.3.8.4. General Rules 15–9
15.3.9. The Record Description—Complete Entry Skeleton 15–10
15.3.9.1. Function 15–10
15.3.9.2. General Format 15–10
15.3.9.3. Syntax Rules 15–11
15.3.9.4. General Rules 15–11
15.3.10. The RANK Clause 15–11
15.3.10.1. Function 15–11

15.3.10.2. General Format....15–11
15.3.10.3. Syntax Rules....15–11
15.3.10.4. General Rules....15–12
15.3.11. The SYNCHRONIZED Clause....15–12
15.3.11.1. Function....15–12
15.3.11.2. General Format....15–12
15.3.11.3. Syntax Rules....15–12
15.3.11.4. General Rules....15–12
15.3.12. The USAGE Clause....15–14
15.3.12.1. Function....15–14
15.3.12.2. General Format....15–14
15.3.12.3. Syntax Rules....15–14
15.3.12.4. General Rules....15–14
15.3.13. The VALUE Clause....15–15
15.3.13.1. Function....15–15
15.3.13.2. General Format....15–15
15.3.13.3. Syntax Rules....15–15
15.3.13.4. General Rules....15–15
15.3.14. Record Selector Fields....15–16
15.3.15. Data Allocation....15–17
15.4. Procedure Division....15–20
15.4.1. The CLOSE Statement....15–20
15.4.1.1. Function....15–20
15.4.1.2. General Format....15–20
15.4.1.3. Syntax Rule....15–20
15.4.1.4. General Rules....15–20
15.4.2. The OPEN Statement....15–23
15.4.2.1. Function....15–23
15.4.2.2. General Format....15–23
15.4.2.3. Syntax Rule....15–23
15.4.2.4. General Rules....15–24
15.4.3. The READ Statement....15–25
15.4.3.1. Function....15–25
15.4.3.2. General Format....15–25
15.4.3.3. Syntax Rule....15–25
15.4.3.4. General Rules....15–25
15.4.4. The WRITE Statement....15–27
15.4.4.1. Function....15–27
15.4.4.2. General Format....15–27
15.4.4.3. Syntax Rules....15–27
15.4.4.4. General Rules....15–27
15.5. IBM Tape File Assignment....15–29
15.5.1. Assignment Rules....15–29
15.5.2. Assignment Examples....15–29
15.5.3. COBOL Program Links....15–30
15.6. EBCDIC Character Codes....15–31

Section 16. Information Interchange Tape Processing Module

16.1. General....16–1
16.2. Environment Division....16–2

16.2.1. Input–Output Section 16–2
16.2.2. The FILE–CONTROL Paragraph 16–2
16.2.2.1. Function 16–2
16.2.2.2. General Format 16–2
16.2.3. The File Control Entry 16–2
16.2.3.1. Function 16–2
16.2.3.2. General Format 16–2
16.2.3.3. Syntax Rules 16–2
16.2.3.4. General Rules 16–3
16.3. Data Division 16–4
16.3.1. File Section 16–4
16.3.2. The RECORDING MODE Clause 16–4
16.3.2.1. Function 16–4
16.3.2.2. General Format 16–4
16.3.2.3. General Rule 16–4
16.3.3. Record Description Restrictions 16–4
16.4. Information Interchange Tape File Assignment 16–5
16.4.1. Assignment Examples 16–5
16.4.2. COBOL Program Links 16–5

Section 17. Optimization

17.1. General 17–1
17.2. Programming Hints 17–2
17.2.1. Program Maintainability 17–2
17.2.2. Execution Speed 17–2
17.2.2.1. General 17–2
17.2.2.2. Input/Output 17–3
17.2.2.3. Sort 17–5
17.2.2.4. Data Alignment and Data Types 17–5
17.2.2.5. Execution of Extended Instruction Set (EIS) Instruction 17–5
17.2.2.6. Generation of Inline Code 17–5
17.2.2.7. Programmer-Controlled Optimization 17–6
17.2.3. Compilation Speed 17–6
17.3. Frequency Table Generation 17–7

Section 18. System Interface

18.1. Processor Call Statement 18–1
18.1.1. General Format 18–1
18.1.2. General Rules 18–1
18.2. XQT Options 18–6
18.2.1. General Format 18–6
18.2.2. General Rule 18–6
18.3. Auto-Flagging Compiler 18–7
18.4. FIPS Flagger 18–9
18.5. COPY Procedures 18–12
18.5.1. Library Preparation 18–12
18.5.2. Library Search Algorithm 18–12
18.6. Sample COBOL Compilation 18–14

18.6.1. Source Listing 18–14
18.6.2. Allocation Listing (D Option) 18–16
18.6.3. Cross Reference Listing (R option) 18–17
18.6.4. Definition of Cross Reference and Allocation Listings 18–18
18.7. Interprogram Communication Considerations....18–19
18.7.1. CALL Identifier 18–19
18.7.2. ASCII COBOL Programs Calling Non–COBOL Programs 18–21
18.7.2.1. Preserving the COBOL Environment 18–21
18.7.2.2. ENTER MASM Calling Sequence....18–21
18.7.2.3. ENTER ASM Calling Sequence 18–22
18.7.2.4. Parameter Format for MASM and ASM 18–22
18.7.2.4.1. Evaluation of Subscripted Arguments 18–24
18.7.2.4.2. Evaluation of Arguments Containing an ODO Clause....18–24
18.7.2.5. ENTER PL1 Calling Sequence 18–25
18.7.2.6. ENTER FTN Calling Sequence....18–26
18.7.2.7. Procedure Name Arguments 18–27
18.7.3. Non-COBOL Programs Calling ASCII COBOL Programs 18–27
18.7.3.1. Considerations for Entering ASCII COBOL Programs....18–27
18.7.3.2. Establishing the ASCII COBOL Environment....18–28
18.7.3.3. Calling Sequence....18–29
18.7.3.4. Parameter Format 18–30
18.8. Executive Request Call Statements 18–31
18.8.1. Call ER ACSF$....18–31
18.8.2. Call ER SYMB$ 18–31
18.8.3. Call ER 'ERERR$' 18–32
18.8.4. Call ER 'EREABT$' 18–32
18.9. Current Date Service Routine (C$CURDATE)....18–33
18.10. Messages to the Operator Console 18–34
18.10.1. ACCEPT . . . FROM CONSOLE....18–34
18.10.2. Blank Tape 18–34
18.10.3. DISPLAY . . . UPON CONSOLE 18–34
18.10.4. Unlabeled Tape 18–34
18.10.5. VALUE OF CARTRIDGE–ID 18–34
18.10.6. VALUE OF FORMS–REQUEST....18–35

Section 19. Collection of Programs

19.1. General 19–1
19.2. Element Naming Conventions....19–2
19.2.1. Main Programs....19–2
19.2.2. Segmented Programs 19–2
19.2.3. Subprograms....19–3
19.3. Collection of ASCII COBOL Programs....19–4
19.3.1. Restrictions 19–4
19.3.1.1. General Restrictions 19–4

19.3.1.2. Installation Configuration Considerations 19–4
19.3.2. Bank-Implied Collections 19–5
19.3.3. Collections Using C$DML 19–5
19.3.4. Bank-Named Collection 19–6
19.3.5. Segmented Collections 19–9
19.3.5.1. Compiler Generated Maps 19–9
19.3.5.2. Non-Compiler Segmented Collections 19–9

Section 20. Related Processors

20.1. Define File Processor (DFP) Interface 20–2
20.1.1. Parameters 20–2
20.1.1.1. Non-MSAM Files 20–2
20.1.1.2. MSAM Files 20–3
20.1.1.2.1. Buffer Size 20–3
20.1.1.2.2. Number of Buffers 20–4
20.1.1.2.3. Block Loading Factor 20–4
20.1.1.3. Record Formats 20–4
20.1.1.4. Prep Factors 20–5
20.1.1.5. Label Records 20–5
20.1.1.6. Verify Parameter 20–6
20.1.1.7. EBCDIC Translate 20–6
20.1.1.8. OPEN Parameter 20–6
20.1.1.9. CLOSE Parameter 20–7
20.1.1.10. SHARE Parameters 20–7
20.1.2. Concatenation of Data Files 20–8
20.2. Processor Common Input/Output System (PCIOS) Interface 20–10
20.3. Shared File System (SFS 2200) Interface 20–11
20.3.1. Rollback and Deadlock Situations 20–12
20.4. Sort-Merge Interface 20–13
20.4.1. Main Storage Size 20–13
20.4.2. Sort–Merge Files 20–13
20.4.3. Suppression of SORT Console and Log Messages 20–13

Section 21. Transaction Processing

21.1. General 21–1
21.2. Message Control System (MCS) 21–2
21.2.1. The RECEIVE Statement 21–2
21.2.2. The SEND Statement 21–3
21.3. Main Storage Requirements 21–4
21.4. Transaction Interface Processing (TIP) Considerations 21–5
21.4.1. Restrictions 21–5
21.4.2. Collection Considerations 21–5
21.4.2.1. Noncommunication Program Example 21–5
21.4.2.2. Communication Program Examples 21–6
21.5. HVTIP Considerations 21–7
21.5.1. General 21–7
21.5.2. Creating Module Name Equate Elements 21–8

21.5.3. TIP Interface....21–9
21.5.4. CALL Statement....21–9
21.5.5. EXIT Statement....21–9
21.5.6. CANCEL Statement....21–9
21.5.7. Writing MASM Subprograms for COBOL/HVTIP....21–9
21.5.8. Restrictions....21–10
21.5.9. Main Storage Requirements....21–11
21.5.10. Automatic Collection of ASCII COBOL HVTIP Transactions....21–11
21.5.10.1. User SGSs....21–12
21.5.10.1.1. HVTIP....21–12
21.5.10.1.2. LOCAL....21–13
21.5.10.1.3. REL....21–13
21.5.10.1.4. ABS....21–13
21.5.10.1.5. CONTROL....21–13
21.5.10.1.6. MAP....21–14
21.5.10.1.7. IBANKS....21–14
21.5.10.1.8. DBANK....21–14
21.5.10.1.9. USING....21–14
21.5.10.1.10. LIST....21–14
21.5.10.1.11. COBOL....21–15
21.5.10.1.12. TIP....21–15
21.5.10.1.13. DMS....21–15
21.5.10.1.14. DPS....21–16
21.5.10.1.15. RDMS....21–16
21.5.10.1.16. OWN....21–17
21.5.10.1.17. DBANKADDRESS....21–17
21.5.10.1.18. NOCOMMON....21–17
21.5.10.2. Examples of Map SGSs....21–17
21.5.11. Program Example....21–18
21.5.11.1. General....21–18
21.5.11.2. COBOL Source Programs....21–18
21.5.11.2.1. Main Program Source....21–19
21.5.11.2.2. COBOL Subprogram COBHV2....21–20
21.5.11.2.3. COBOL Subprogram COBHV3....21–20
21.5.11.3. C$S333....21–21
21.5.11.4. MNQ Elements....21–21
21.5.11.5. SGS Input to MAPHVTIP....21–21
21.5.11.6. Program Input....21–22
21.5.11.7. Program Output....21–22

Appendix A. Reserved Words

Appendix B. Character Codes

Appendix C. Diagnostic Messages

C.1. Compiler Diagnostics....C–1
C.2. Compiler Errors....C–104
C.3. Execution Diagnostics....C–106

C.4. PCIOS Error Messages C–133
C.5. Sort/Merge Error Messages C–134
C.6. OS 2200 Status Key 2 Values C–135

Appendix D. Data Allocation

D.1. General D–1
D.2. Data Allocation and Alignment D–2
D.2.1. Word-Aligned Elementary Items D–2
D.2.2. Quarter-word Aligned Elementary Items D–3
D.2.3. Sixth-Word Aligned Elementary Items D–3
D.2.4. Exact Binary Elementary Items D–4
D.2.5. Alignment in Group Items D–8

Appendix E. Label Processing

E.1. Introduction to Label Processing E–1
E.2. The LABEL RECORDS Clause E–2
E.3. Label Formats E–3
E.3.1. TAPE and INTERCHANGE Files E–3
E.3.1.1. Volume Header Label (VOL1) E–3
E.3.1.2. First File Header Label (HDR1) E–4
E.3.1.3. Second File Header (HDR2) E–5
E.3.1.4. End of File Label (EOF1) E–5
E.3.1.5. Second End of File Label (EOF2) E–5
E.3.1.6. End of Volume Label (EOV1) E–5
E.3.1.7. Second End of Volume Label (EOV2) E–6
E.3.2. IBM OS/360 Compatible Tape Files E–6
E.3.3. INDEXED DISC Files E–6
E.3.4. RELATIVE and SEQUENTIAL DISC Files E–6
E.4. The VALUE OF Clause E–7
E.4.1. FILE–ACCESS E–7
E.4.2. FILE-ID E–7
E.4.3. PURGE-DATE E–7
E.5. Procedure Division I-O Statements E–8
E.5.1. The CLOSE Statement E–8
E.5.1.1. CLOSE File E–8
E.5.1.1.1. Output Files E–8
E.5.1.1.2. Input Files E–8
E.5.1.1.3. I–O Files E–8
E.5.1.1.4. Reversed Files E–9
E.5.1.2. CLOSE REEL E–9
E.5.1.2.1. Output Files E–9
E.5.1.2.2. Input Files E–9
E.5.2. The OPEN Statement E–10
E.5.2.1. Output Files E–10
E.5.2.2. Input or I–O Files E–10
E.5.2.3. Reversed Files E–10
E.5.3. The READ Statement E–10
E.5.4. The WRITE Statement E–11

Appendix F. Language Skeleton

F.1. General Description F–1
F.2. Control Division F–2
F.3. Identification Division F–3
F.4. Environment Division F–4
F.4.1. File-Control Entry F–6
F.5. Data Division F–8
F.5.1. Data Description Entry F–11
F.5.2. Communication Description Entry F–12
F.5.3. Report Group Description Entry F–13
F.6. Procedure Division F–15
F.6.1. The ACCEPT Statement F–15
F.6.2. The ADD Statement F–16
F.6.3. The ALTER Statement F–16
F.6.4. The CALL Statement F–16
F.6.5. The CANCEL Statement F–16
F.6.6. The CLOSE Statement F–17
F.6.7. The COMPUTE Statement F–17
F.6.8. The DELETE Statement F–17
F.6.9. The DISABLE Statement F–17
F.6.10. The DISPLAY Statement F–18
F.6.11. The DIVIDE Statement F–18
F.6.12. The ENABLE Statement F–18
F.6.13. The ENTER Statement F–19
F.6.14. The EXIT Statement F–19
F.6.15. The GENERATE Statement F–19
F.6.16. The GO TO Statement F–19
F.6.17. The HOLD Statement F–19
F.6.18. The IF Statement F–19
F.6.19. The INITIATE Statement F–19
F.6.20. The INSPECT Statement F–20
F.6.21. The LOCK Statement F–20
F.6.22. The MERGE Statement F–21
F.6.23. The MOVE Statement F–21
F.6.24. The MULTIPLY Statement F–21
F.6.25. The OPEN Statement F–22
F.6.26. The PERFORM Statement F–22
F.6.27. The PROCESS Statement F–23
F.6.28. The READ Statement F–23
F.6.29. The RECEIVE Statement F–24
F.6.30. The RELEASE Statement F–24
F.6.31. The RETURN Statement F–24
F.6.32. The REWRITE Statement F–24
F.6.33. The SEARCH Statement F–24
F.6.34. The SEND Statement F–25
F.6.35. The SET Statement F–25
F.6.36. The SORT Statement F–26
F.6.37. The START Statement F–26
F.6.38. The STOP Statement F–26
F.6.39. The STRING Statement F–27
F.6.40. The SUBTRACT Statement F–27

F.6.41. The SUPPRESS Statement F–27
F.6.42. The TERMINATE Statement F–27
F.6.43. The UNLOCK Statement F–27
F.6.44. The UNSTRING Statement F–28
F.6.45. The USE Statement F–28
F.6.46. The WRITE Statement F–29
F.7. Conditions F–30
F.7.1. Relation Condition F–30
F.7.2. ALL Condition F–30
F.7.3. ANY Condition F–30
F.7.4. Class Condition F–30
F.7.5. Sign Condition F–30
F.7.6. Condition-Name Condition F–30
F.7.7. Switch-Status Condition F–30
F.7.8. Negated Simple Condition F–30
F.7.9. Combined Condition F–31
F.7.10. Abbreviated Combined Relation Condition F–31
F.8. Miscellaneous Formats F–32
F.8.1. Qualification F–32
F.8.2. Subscripting F–32
F.8.3. Indexing F–32
F.8.4. Identifier F–32
F.9. The COPY Statement F–33

Glossary 1

Index 1

Figures

1-1. COBOL Communications Environment 1–12
1-2. Hierarchy of Queues 1–16

2-1. +, -, ADD, SUBTRACT for Binary 2–75
2-2. +, -, ADD, SUBTRACT for Decimal 2–75
2-3. * for Binary (N1 + N2 + D1 + D2 ≤ 21) 2–76
2-4. * for Binary (N1 + N2 + D1 + D2 > 21) 2–76
2-5. MULTIPLY for Binary (N1 + N2 + D1 + D2 ≤ 21) 2–76
2-6. MULTIPLY for Binary (N1 + N2 + D1 + D2 > 21) 2–77
2-7. DIVIDE for Binary (N1 + D2 + DR ≤ 21) 2–77
2-8. DIVIDE for Binary (N1 + D2 + DR > 21) 2–78
2-9. / in COMPUTE for Binary (N1 + D2 + (the larger of DR and D1) ≤ 21) 2–78
2-10. / in COMPUTE for Binary (N1 + D2 + (the larger of DR and D1) > 21) 2–79
2-11. / in Arithmetic Expression Other Than COMPUTE 2–79
2-12. Execution Flowchart 2–95

14-1. Example of a COBOL Cycle Structure 14–3

D-1. OS 2200 Word Divisions D–2
D-2. Data Allocation D–6

Tables

2–1. Internal Representation of Overpunched Operational Signs 2–23
2–2. PICTURE Character Precedence Chart 2–31
2-3. Examples of Nonediting PICTURE Strings 2–32
2-4. Examples of Editing PICTURE Strings 2–33
2-5. Meaning and Allocation of Data Types by USAGE 2–42
2-6. Alignment of Data Types by USAGE 2–42
2-7. Combination of Symbols in Arithmetic Expressions 2–47
2-8. Collating Sequence Example 2–54
2-9. Combinations of Conditions, Logical Operators, and Parentheses 2–61
2-10. Size of Intermediate Result Algorithm 2–73

4–1. Relationship of Categories of Files and the Formats of the CLOSE Statement 4–18
4-2. Permissible Statements 4–22

5-1. Permissible Statements 5–16

6-1. Contents of DATA–NAME–n 6–7
6-2. Permissible Statements 6–20

8–1. Permissible Clause Combinations in Format 3 Entries 8–10

13–1. Communication Status Key Condition in Level 1 13–9
13–2. Communication Status Key Condition in Level 2 13–10

15-1. Allocation and Alignment Rules 15–17
15-2. Meaning and Allocation of Data Types by USAGE 15–18
15-3. Alignment of Data Types by USAGE 15–19
15-4. Relationship of Categories of Files and the Formats of the CLOSE Statement 15–21
15-5. Permissible Statements 15–24

18-1. @ACOB Control Statement Options 18–2
18-2. @ACOB Control Statement Extra Options 18–3
18-3. @XQT Options 18–6
18-4. Standard COBOL Levels 18–9
18-5. Flagging Options 18–10
18-6. Library Search Order 18–13

20–1. ACOB Rollback/Deadlock Error Processing 20–12

B–1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order B–1

B–2. Correspondence Between ASCII and EBCDIC in EBCDIC Order..... B–7

C-1. Severity of Printed Diagnostics C–3

D-1. Allocation of COMP and COMP–4 Items D–4

Section 1
Introduction

1.1. General

The COBOL source language implemented by the OS 2200 ASCII COBOL compiler system includes all features of American National Standard COBOL X3.23–1974 and FIPS PUB 21–1 (Federal Information Processing Standards Publication 21–1). In addition, it includes numerous compatibility features for American National Standard COBOL X3.23–1968. It also includes a random processing (multitasking) capability based on the work of the CODASYL COBOL committee.

The ASCII COBOL compiler utilizes the ASCII and ISO character sets in processing source text and object time character data. The compiler is fully reentrant and produces optimized reentrant code usable in multibanked, common banked, or transaction program environments.

The organization of ASCII COBOL specifications in this document is based on a functional processing module concept. This document defines a Nucleus and fourteen functional processing modules: Table Handling, Sequential I–O, Relative I–O, Indexed I–O, Sort–Merge, Report Writer, Segmentation, Library, Debug, Interprogram Communications, Communication, Asynchronous Processing, IBM Tape Processing, and Information Interchange Tape Processing. Each module may contain either one or two levels. In all cases, the lower levels are proper subsets of the higher levels within the same module. When a module has two levels, those levels are defined by X3.23–1974. Specifications unique to the high level are enclosed in solid-line boxes.

Specifications denoting Unisys-defined items are indicated by broken-line boxes or footnotes and fall into the following three categories:

1. Implementer-defined specifications as dictated by X3.23–1974
2. Unisys clarifications and examples
3. Unisys extensions

The objective of boxing and footnotes is to facilitate the creation of programs that conform to FIPS PUB 21–1. These programs may be easily interchanged between different computer systems.

Documentation Updates

This document contains all the information that was available at the time of publication. Changes identified after release of this document are included in problem list entry (PLE) 18734401. To obtain a copy of the PLE, contact your Unisys representative or access the current PLE from the Unisys Product Support Web site:

http://www.support.unisys.com/all/ple/18734401

Note: *If you are not logged into the Product Support site, you will be asked to do so.*

1.2. Concepts

1.2.1. Features of the Language

ASCII COBOL offers many features that allow the user to obtain a necessary function without programming the function in detail. In this section each of these features and the concept of its use will be discussed.

1.2.2. Record Ordering

The sort and merge features of ASCII COBOL assist users in arranging records into a particular order.

While both are concerned with record ordering, the functions and capabilities of the SORT and MERGE statements are different in a number of respects. The sort will produce an ordered file from one or more files that may be completely unordered in the sort sequence whereas the merge can only produce an ordered file from two or more files, each of which is already ordered in the specified sequence.

In many applications it is necessary to apply some special processing to the contents of the sort or merge file(s) before or after sorting or merging. This special processing may consist of addition, deletion, creation, altering, editing, or other modification of the individual records in the file. The ASCII COBOL Sort-Merge feature allows the user to express these procedures in the COBOL language. An ASCII COBOL program may contain any number of sorts and merges, and each of them may have its own independent special procedures. The Sort-Merge feature automatically causes execution of these procedures in such a way that extra passes over the sort or merge files are not required.

1.2.3. Report Writer

The Report Writer feature places its emphasis on the organization, format, and contents of an output report. Although a report can be produced using the standard COBOL language, the Report Writer language features provide a more concise facility for report structuring and report production. Much of the Procedure Division programming which would normally be supplied by the programmer is provided automatically by the Report Writer Control System (RWCS). Thus, the programmer is relieved of writing procedures for moving data, constructing print lines, counting lines on a page, numbering pages, producing heading and footing lines, recognizing the end of logical data subdivisions, updating sum counters, etc. All these operations are accomplished by the RWCS as a consequence of source language statements that appear primarily in the Report Section of the Data Division of the source program.

Data movement to a report is directed by the Report Section clauses SOURCE, SUM, and VALUE. Fields of data are positioned on a print line by the COLUMN NUMBER clause. The PAGE clause specifies the length of the page, the size of the heading and footing areas, and the size of the area in which the detail lines will appear. Data items may be specified to form a control hierarchy. During the execution of a GENERATE statement, the Report Writer Control System uses the control hierarchy to check for control breaks. When a control break occurs, summary information (for example, subtotals) can be presented.

1.2.4. Table Handling

Tables of data are common components of business data processing problems. Although items of data that make up a table could be described as contiguous data items, there are two reasons why this approach is not satisfactory. First, from a documentation standpoint, the underlying homogeneity of the items would not be readily apparent; and second, it would be difficult to make an individual element of such a table available when there is a decision as to which element is to be made available at object time.

Tables composed of contiguous data items are defined in COBOL by including the OCCURS clause in their data description entries. This clause specifies that the item is to be repeated as many times as stated. The item is considered to be a table element and its name and description apply to each repetition or occurrence. Since each occurrence of a table element does not have assigned to it a unique data-name, reference to a desired occurrence may be made only by specifying the data-name of the table element together with the occurrence number of the desired table element. Subscripting and indexing are the two methods that are used to specify the occurrence number of a desired table element.

1.2.4.1. Table Definition

To define a one-dimensional table, the programmer uses an OCCURS clause as part of the data description of the table element, but the OCCURS clause must not appear in the description of group items which contain the table element. Example 1 shows a one-dimensional table.

Example 1

```
01 TABLE-1.
   02 TABLE-ELEMENT OCCURS 20 TIMES.
      03 NAME . . . . . .
      03 SSAN . . . . . .
```

Defining a one-dimensional table within each occurrence of an element of another one-dimensional table gives rise to a two-dimensional table. To define a two-dimensional table, then, an OCCURS clause must appear in the data description of the element of the table, and in the description of only one group item which contains that table element. To define a three-dimensional table, the OCCURS clause should appear in the data description of the element of the table and in the description of two group items which contain the element. In COBOL, tables of up to three dimensions are permitted. Example 2 shows a table which has one dimension for CONTINENT–NAME, two dimensions for COUNTRY–NAME, and three dimensions for CITY–NAME and CITY–POPULATION. The table includes 100,510 data items: 10 for CONTINENT–NAME, 500 for COUNTRY–NAME, 50,000 for CITY–NAME, and 50,000 for CITY–POPULATION. Within the table there are ten occurrences of CONTINENT–NAME. Within each CONTINENT–NAME there are 50 occurrences of COUNTRY–NAME and within each COUNTRY–NAME there are one hundred occurrences of CITY–NAME and CITY–POPULATION.

Example 2

```
01 CENSUS-TABLE.
    05 CONTINENT-TABLE OCCURS 10 TIMES.
       10 CONTINENT-NAME PIC XXXXXX.
       10 COUNTRY-TABLE OCCURS 50 TIMES.
          15 COUNTRY-NAME PIC XXXXXXXX.
          15 CITY-TABLE OCCURS 100 TIMES.
             20 CITY-NAME PIC XXXXXXXXXX.
             20 CITY-POPULATION PIC 999999999999.
```

1.2.4.2. References to Table Items

Whenever the user refers to a table element, the reference must indicate which occurrence of the element is intended. For access to a one-dimensional table, the occurrence number of the desired element provides complete information. For access to tables of more than one dimension, an occurrence number must be supplied for each dimension of the table accessed. In Example 2, a reference to the fourth CONTINENT–NAME would be complete, whereas a reference to the fourth COUNTRY–NAME would not. To refer to COUNTRY–NAME, which is an element of a two-dimensional table, the user must refer to, for example, the fourth COUNTRY–NAME within the sixth CONTINENT–TABLE.

One method by which occurrence numbers may be specified is to append one or more subscripts to the data-name. A subscript is an integer whose value specifies the occurrence number of an element. The subscript can be represented either by a literal which is an integer or by a data-name which is defined elsewhere as a numeric elementary item with no character positions to the right of the assumed decimal point. In either case, the subscript, enclosed in parentheses, is written immediately following the name of the table element. A table reference must include as many subscripts as there are dimensions in the table whose element is being referenced. That is, there must be a subscript for each OCCURS clause in the hierarchy containing the data-name, including the data-name itself. In Example 2, references to CONTINENT–NAME require only one subscript, reference to COUNTRY–NAME requires two, and references to CITY–NAME and CITY–POPULATION require three.

When more than one subscript is required, they are written in order of successively less inclusive dimensions of the data organization. When a data-name is used as a subscript, it may be used to refer to items in many different tables. These tables need not have elements of the same size. The data-name may also appear as the only subscript with one item and as one of two or three subscripts with another item. Also, it is permissible to mix literal and data-name subscripts, for example:

```
CITY-POPULATION (10, NEWKEY, 42).
```

Another method of referring to items in a table is indexing. To use this technique, the programmer assigns one or more index-names to an item whose data description contains an OCCURS clause. There is no separate entry to describe the index-name since its definition is completely hardware-oriented and it is not considered data per se. At object time the contents of the index-name will correspond to an occurrence number for that specific dimension of the table to which the index-name was assigned. The initial value of an index-name at object time is not determinable and the index-name must be initialized by the SET statement before use.

When a reference is made to a table element, or to an item within a table element, and the name of the item is followed by its related index-name or names in parentheses, then each occurrence number required to complete the reference will be obtained from the respective index-name. The index-name thus acts as a subscript whose value is used in any table reference that specifies indexing.

When a reference requires more than one occurrence number for completeness, the programmer must not use a data-name subscript to indicate one occurrence number and an index-name for another. Therefore, if indexing is to be used, each OCCURS clause within the hierarchy (each dimension of the table) must contain an INDEXED BY clause. The programmer may, however, mix literals and index-names within one reference, just as he may mix literals and data-name subscripts.

1.2.4.3. Table Searching

Data that has been arranged in the form of a table is very often searched. In COBOL the SEARCH statement provides facilities, through its two options, for producing serial and nonserial (for example, binary) searches. In using the SEARCH statement, the programmer may vary an associated index-name or data-name. This statement also provides facilities for execution of imperative statements when certain conditions are true.

1.2.5. File Organization and Access Methods

Magnetic tape, punched paper tape, and punched card files are normally organized in a sequential manner and the Procedure Division of COBOL reflects this use. Mass storage media can be used to store sequentially organized files, and this technique has been provided; but, more significantly, mass storage devices have been designed to provide nonsequential organization and access capabilities.

1.2.5.1. Sequential Organization

A file whose organization is sequential can only be accessed in the sequential mode. Records in such a file can be accessed in the sequence established as a result of writing the records to the file. A sequential mass storage file may be used for input and output at the same time. One file maintenance method made possible by this facility is to read a record, process it, and, if it is updated, return it, modified, to its previous position.

1.2.5.2. Relative Organization

A file whose organization is relative can be accessed either sequentially, dynamically, or randomly. Sequential access provides the same results as if the file were organized sequentially. Random access allows the sequence in which the records are accessed to be controlled by the programmer. Each record in a relative file is identified by an integer value greater than zero which specifies the record's logical ordinal position in the file. The desired record is accessed by placing its relative record number in a Relative Key data item. Such a file may be thought of as a serial string of areas, each capable of holding a logical record. Each of these areas is denominated by a relative record number. Records are stored and retrieved based on this number. For example, the tenth record is the one addressed by relative record number 10 and is in the tenth record area, whether or not records have been written in the first through the ninth record areas.

In the dynamic access mode, the programmer may change at will from sequential access to random access using appropriate forms of input-output statements.

1.2.5.3. Indexed Organization

A file whose organization is indexed can be accessed either sequentially, dynamically, or randomly. Sequential access provides access to the records in the ascending order of the record key values. The order of retrieval of records within a set of records having duplicate record key values is the order in which the records were written into the set.

In the random access mode, the sequence in which records are accessed is controlled by the programmer. Each record in the file is identified by the value of one or more keys within that record, and the desired record is accessed by placing the value of its record key in a record key data item before accessing the record.

In the dynamic access mode, the programmer may change at will from sequential access to random access by using appropriate forms of input-output statements.

1.2.6. Rerun

The RERUN feature of COBOL provides a facility for check restart. That is, executing a RERUN takes a snapshot of the program status and stores the information. It is then possible to restart the program from the point of the most recent RERUN. The RERUN clause protects the user from having to start a program over from the beginning in the event of a hardware failure while the job is running.

There are two basic parts to the RERUN clause. The user must designate a medium to receive the data and a criterion from which the frequency of checkpoints may be determined. The receiving medium may be specified by designating a file name or a separate hardware device. The determination of frequency of the dump may be made on the basis of a number of records of a particular file having been processed or of the end of a reel of a particular file having been reached.

1.2.7. Program Modularity

Complex data processing problems are frequently solved by the use of separately compiled but logically coordinated programs, which, at execution time, form logical and physical subdivisions of a single run unit. This approach lends itself to dividing a large problem into smaller, more manageable segments which can be programmed and debugged independently. At execution time, control is transferred from program to program by the use of CALL and EXIT PROGRAM statements.

Under certain circumstances (for example, a shortage of computer storage) it is desirable to subdivide a single program into physical segments, so that, at execution time, it is not necessary to load the entire program into computer storage at one time. This approach will permit the overlaying of some segments, with a corresponding saving in total computer storage required to execute the program. This facility is called segmentation.

There are no special statements in COBOL for communications between segments of such a program. There are, however, some special clauses used by the COBOL programmer to specify how the object program is to be segmented.

1.2.7.1. Interprogram Communications

In COBOL terminology, a program is either a source program or an object program depending on context. A source program is a syntactically correct set of COBOL statements. An object program is the set of instructions, constants, and other machine-oriented data resulting from the operation of a compiler on a source program. A run unit is the total machine language necessary to solve a data processing problem. It includes one or more object programs, and it may include machine language from sources other than a COBOL compiler.

When the statement of a problem is subdivided into more than one program, the constituent programs must be able to communicate with each other. This communication may take two forms: transfer of control and reference to common data.

1.2.7.1.1. Transfer of Control

The CALL statement provides the means whereby control can be passed from one program to another within a run unit. A program that is activated by a CALL statement may itself contain CALL statements. However, results are unpredictable where circularity of control is initiated; for example, where program A calls program B, then program B calls program A or another program that calls program A.

When control is passed to a called program, execution proceeds in the normal way from procedure statement to procedure statement beginning with the first nondeclarative statement. If control reaches a STOP RUN statement, this signals the logical end of the run unit. If control reaches an EXIT PROGRAM statement, this signals the logical end of the called program only, and control then reverts to the point immediately following the CALL statement in the calling program. Stated briefly, the EXIT PROGRAM statement terminates only the program in which it occurs, and the STOP RUN statement terminates the entire run unit.

If the called program is not a COBOL program, the termination of the run unit or the return to the calling program must be programmed in accordance with the language of the called program.

1.2.7.1.2. Interprogram Data Storage

Program interaction requires that both programs have access to the same data items. In the calling program the common data items are described along with all other data items in the File Section, Working–Storage Section, Communication Section, Common–Storage Section, or Linkage Section. At object time main storage is allocated for the entire Data Division. In the called program, common data items are described in the Linkage Section. At object time main storage space is not allocated for this section.

Communication between the called program and the common data items stored in the calling program is effected through USING clauses contained in both programs. The USING clause in the calling program is contained in the CALL statement and the operands are a list of common data-identifiers described in its Data Division. The USING clause in the called program follows the Procedure Division header and the operands are a list of common data identifiers described in its Linkage Section. The identifiers specified by the USING clause of the CALL statement indicate those data items available to a calling program that may be referred to in the called program. The sequence of appearance of the identifiers in the USING clause of the CALL statement and the USING clause in the Procedure Division header is significant. Corresponding identifiers refer to a single set of data which is available to the calling program. The correspondence is positional, and not by name. While the called program is being executed, every reference to an operand whose identifier appears in the called program's USING clause is treated as if it were a reference to the corresponding operand in the USING clause of the active CALL statement.

Once control leaves a called program its state is maintained until a CANCEL statement is executed naming that program. Therefore, initialization of the program in case of repetitive calls is not necessary.

Execution of the CANCEL statement allows the user to indicate that the main storage areas occupied by the called programs may be released. In addition, the CANCEL statement guarantees that the program cancelled will be in its initial state when called by a subsequent CALL statement.

1.2.7.2. Segmentation

The segmentation facility permits the user to subdivide physically the Procedure Division of a COBOL object program. All source paragraphs which contain the same segment–number in their section headers will be considered at object time to be one segment. Since segment–numbers can range from 00 through 99, it is possible to subdivide any object program into a maximum of 100 segments.

Program segments may be of three types: fixed permanent, fixed overlayable, and independent as determined by the programmer's assignment of segment numbers.

Fixed permanent segments are always in computer storage during the execution of the entire program; that is, they cannot be overlaid except when the system is executing another program, in which case fixed segments may be "rolled out" temporarily.

Fixed overlayable segments may be overlaid during program execution, but any such overlaying is transparent to the user; in other words, they are logically identical to fixed permanent segments, but physically different from them.

Independent segments may be overlaid, but such overlaying will result in the initialization of that segment. Therefore, independent segments are logically different from fixed permanent/fixed overlayable segments, and physically different from fixed segments.

1.2.8. Communication Facility

The communication facility provides the ability to access, process, and create messages or portions thereof. It provides the ability to communicate through a Message Control System with local and remote communication devices.

1.2.8.1. Message Control System

The implementation of the communication facility requires that a Message Control System (MCS) be present in the COBOL object program's operating environment.

The MCS is the logical interface to the Executive System under which the COBOL object program operates. The primary functions of the MCS are the following:

1. To act as an interface between the COBOL object program and the network of communication devices, in much the same manner as an operating system acts as an interface between the COBOL object program and such devices as card readers, magnetic tape and mass storage devices, and printers.
2. To perform line discipline, including such tasks as dial-up, polling, and synchronization.
3. To perform device-dependent tasks, such as character translation and insertion of control characters, so that the COBOL user can create device-independent programs.

The first function, that of interfacing the COBOL object program with the communications devices, is the most obvious to the COBOL user. In fact, the COBOL user may be totally unaware that the other two functions exist. Messages from communication devices are placed in input queues by the MCS while awaiting disposition by the COBOL object program. Output messages from the COBOL object program are placed in output queues by the MCS while awaiting transmission to communication devices. The structures, formats, and symbolic names of the queues are defined by the user to the MCS at some time prior to the execution of the COBOL object program. Symbolic names for message sources and destinations are also defined at that time. The COBOL user must specify in his COBOL program symbolic names which are known to the MCS.

During execution of a COBOL object program, the MCS performs all necessary actions to update the various queues as required.

1.2.8.2. The COBOL Object Program

The COBOL object program interfaces with the MCS when it is necessary to send data, receive data, or to interrogate the status of the various queues which are created and maintained by the MCS. In addition, the COBOL object program may direct the MCS to establish or break the logical connection between the communications device and a specified portion of the MCS queue structure. The method of handling the physical connection is a function of the MCS.

1.2.8.3. Relationship of COBOL Program to MCS and Communication Devices

The interfaces which exist in a COBOL communication environment are established by the use of a Communication Description (CD) and associated clauses in the Communication Section of the Data Division. There are two such interfaces:

1. The interface between the COBOL object program and the MCS, and
2. The interface between the MCS and the communications devices.

The COBOL source program uses three statements to control the interface with the MCS:

1. The RECEIVE statement, which causes data in a queue to be passed to the COBOL object program,
2. The SEND statement, which causes data associated with the COBOL object program to be passed to one or more queues, and;
3. The ACCEPT statement with the COUNT phrase, which causes the MCS to indicate to the COBOL object program the number of complete messages in the specified queue structure.

The COBOL source program uses two statements to control the interface between the MCS and the communications devices:

1. The ENABLE statement, which establishes logical connection between the MCS and one or more given communications devices, and;
2. The DISABLE statement, which breaks a logical connection between the MCS and one or more given communications devices.

These relationships are shown in Figure 1-1.

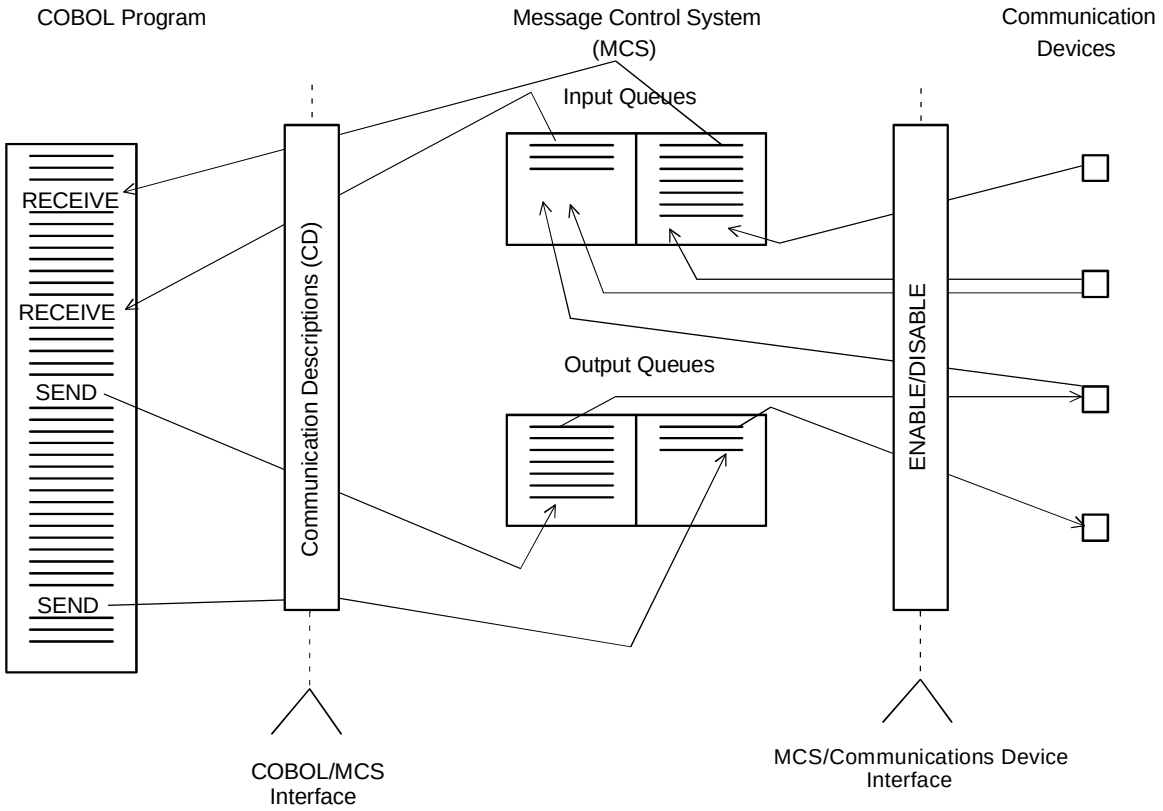


Figure 1-1. COBOL Communications Environment

1.2.8.3.1. Invoking the COBOL Object Program

There are two methods of invoking a COBOL communication object program:

1. Schedule initiation
2. MCS invocation

Regardless of the method of invocation, the only operating difference between the two methods is that MCS invocation causes the areas referenced by the symbolic queue and subqueue names in the specified CD to be filled.

1.2.8.3.1.1. Scheduled Initiation of the COBOL Object Program

A COBOL object program using the communication facility may be scheduled for execution through the normal means available in the program's operating environment, such as job control language. In that case, the COBOL program can use three methods to determine what messages, if any, are available in the input queues:

1. The ACCEPT statement with the COUNT phrase,
2. The RECEIVE statement with a NO DATA phrase, and
3. The RECEIVE statement without a NO DATA phrase (in which case a program wait is implied if no data is available).

1.2.8.3.1.2. Invocation of the COBOL Object Program by the MCS

It is sometimes desirable to schedule a COBOL object communication program only when there is work available for it to do. Such scheduling occurs if the MCS determines what COBOL object program is required to process the available message and subsequently causes that program to be scheduled for execution. Prior to the execution of the COBOL object program, the MCS places symbolic queue and subqueue names in the data items of the CD that specifies the FOR INITIAL INPUT clause.

A subsequent RECEIVE statement directed to that CD will result in the available message being passed to the COBOL object program.

1.2.8.3.1.3. Determining the Method of Scheduling

A COBOL source program can be written so that its object program can operate with either of the two modes of scheduling. In order to determine which method was used to load the COBOL object program, the following technique may be used:

1. One CD must contain a FOR INITIAL INPUT clause.
2. The Procedure Division may contain statements to test the initial value of the symbolic queue name in that CD. If it is space filled, job control statements were used to schedule the COBOL object program. If not space filled, the MCS has invoked the COBOL object program and replaced the spaces with the symbolic name of the queue containing the message to be processed.

1.2.8.4. The Concept of Messages and Message Segments

A message consists of an arbitrary amount of information, usually character data, whose beginning and end are defined or implied. Messages are the fundamental, but not necessarily the most elementary, unit of data to be processed in a COBOL communication environment.

Messages may be logically subdivided into smaller units of data called message segments which are delimited within a message by end of segment indicators (ESI). A message consisting of one or more segments is delimited from the next message by an end of message indicator (EMI). In a similar manner, a group of several messages may be logically separated from succeeding messages by an end of group indicator (EGI). When a message or message segment is received by the COBOL program, a communication description interface area is updated by the MCS to indicate which, if any, delimiter was associated with the text transferred during the execution of that RECEIVE statement. On output the delimiter, if any, to be associated with the text released to the MCS during execution of a SEND statement is specified or referenced in the SEND statement. Thus the presence of these logical indicators is recognized and specified both by the MCS and by the COBOL object program; however, no indicators are included in the message text processed by COBOL programs.

A precedence relationship exists between the indicators EGI, EMI, and ESI. EGI is the most inclusive indicator and ESI is the least inclusive indicator. The existence of an indicator associated with message text implies the association of all less inclusive indicators with that text. For example, the existence of the EGI implies the existence of EMI and ESI.

1.2.8.5. The Concept of Queues

Queues consist of one or more messages from or to one or more communication devices, and as such, form the data buffers between the COBOL object program and the MCS. Input queues are logically separate from output queues.

The MCS logically places in queues or removes from queues only complete messages. Portions of messages are not placed in queues until the entire message is available to the MCS. That is, the MCS will not pass a message segment to a COBOL object program unless all segments of that message are in the input queue, even though the COBOL source program uses the SEGMENT phrase of the RECEIVE statement. For output messages, the MCS will not transmit any segment of a message until all its segments are in the output queue. The number of messages in a given queue reflects only the number of complete messages in the queue.

The process by which messages are placed into a queue is called enqueuing. The process by which messages are removed from a queue is called dequeuing.

1.2.8.5.1. Independent Enqueuing and Dequeuing

It is possible for a message to be received by the MCS from a communication device prior to the execution of the COBOL object program. In this case, the MCS enqueues the message in the proper input queue until the COBOL object program requests dequeuing with the RECEIVE statement. It is also possible for a COBOL object program to cause the enqueuing of messages in an output queue which are not transmitted to a communication device until after the COBOL object program has terminated. Two common reasons for this occurrence are:

1. When data transfer between the specified output queue and its destination is inhibited.
2. When the COBOL object program creates output messages at a speed faster than the destination can receive them.

1.2.8.5.2. Enabling and Disabling Logical Connectives

Usually, the MCS will logically connect and disconnect sources and destinations based on time of day, message activity, or other factors unrelated to the COBOL program. However, the COBOL program has the ability to perform these functions through the ENABLE and DISABLE statements.

A key is required in both statements in order to prevent indiscriminate use of the facility by a COBOL user who is not aware of the total network environment, and who may therefore disrupt system functions by the untimely issuance of ENABLE and DISABLE statements. However, this action never interrupts a transmission.

1.2.8.5.3. Enqueuing and Dequeuing Methods

In systems that allow the user to specify certain MCS functions, it may be necessary that the user specify to the MCS, prior to execution of programs which reference these facilities, the selection algorithm and other designated MCS functions to be used by the MCS in placing messages in the various queues. A typical selection algorithm, for example, would specify that all messages from a given source be placed in a given input queue, or that all messages to be sent to a given destination be placed in a given output queue.

Dequeuing is often done on a first in, first out basis. Thus, messages dequeued from either an input or output queue are those messages which have been in the queue for the longest period of time. However, the MCS can, upon prior specification by the user, dequeue on some other basis; in other words, priority queuing can be employed.

1.2.8.5.4. Queue Hierarchy

In order to control more explicitly the messages being enqueued and dequeued, it is possible to define in the MCS a hierarchy of input queues; in other words, queues comprising queues. In COBOL, four levels of queues are available to the user. In order of decreasing significance, the queue levels are named queue, sub-queue-1, sub-queue-2 and sub-queue-3. The full queue structure is depicted in Figure 1-2, where queues and sub-queues have been named with the letters A through O. Messages have been named with a letter according to their source (X, Y, or Z) and with a sequential number.

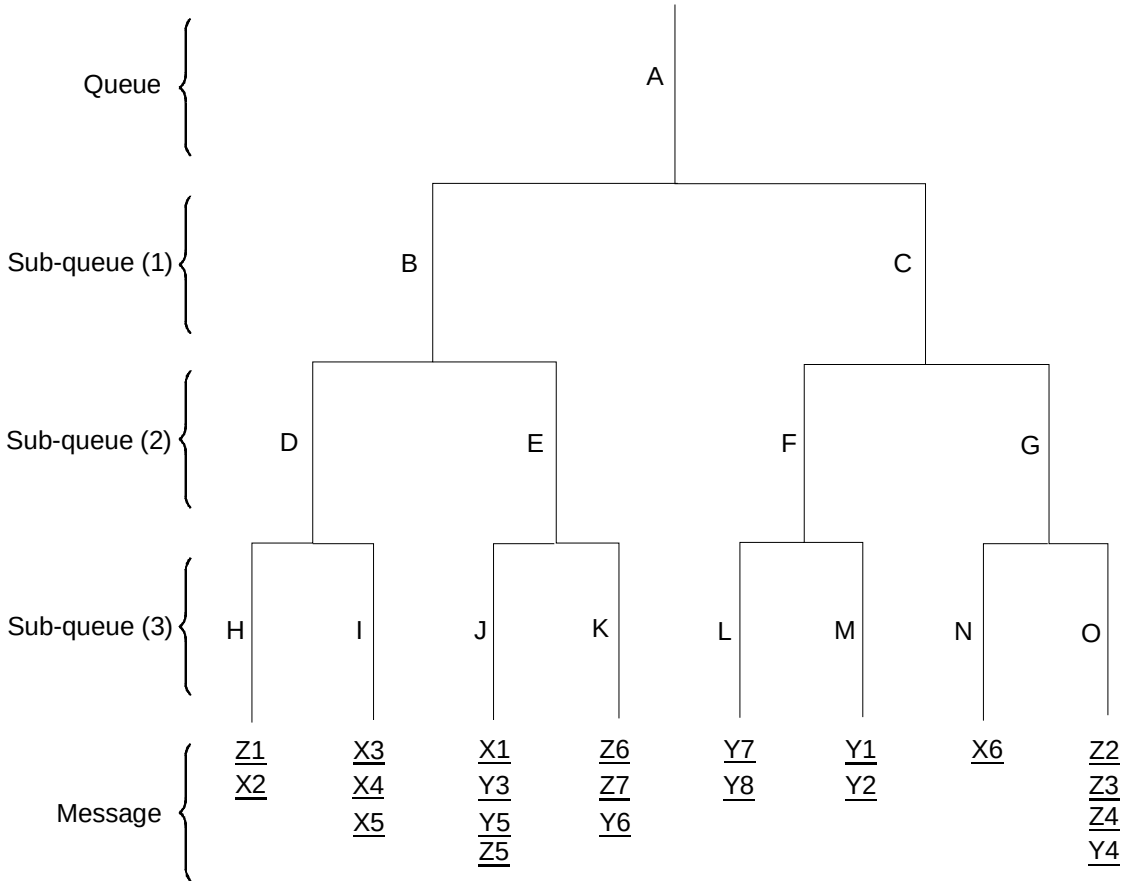


Figure 1-2. Hierarchy of Queues

Assume that the MCS is operating under the following queuing algorithm:

1. Messages are placed in queues according to the contents of some specified data field in each message.
2. With the RECEIVE statement, if the user does not specify a given sub-queue level, the MCS will choose the sub-queue from that level in alphabetical order; for example, if sub-queue-1 is not specified by the user, the MCS will dequeue from sub-queue-1 B.

The following examples illustrate the effect of the algorithms shown in Figure 1-2:

1. The program executes a RECEIVE statement, specifying via the CD:

   Queue A

   MCS returns: Message Z1

2. The program executes a RECEIVE statement, specifying via the CD:

   Queue A

   Sub-queue–1 C

   MCS returns: Message Y7

3. The program executes a RECEIVE statement, specifying via the CD:

   Queue A

   Sub-queue–1 B

   Sub-queue–2 E

   MCS returns: Message X1

4. The program executes a RECEIVE statement, specifying via the CD:

   Queue A

   Sub-queue–1 C

   Sub-queue–2 G

   Sub-queue–3 N

   MCS returns: Message X6

If the COBOL programmer wishes to access the next message in a queue, regardless of which sub-queue that message may be in, he specifies the queue name only. The MCS, when supplying the message, will return to the COBOL object program any applicable sub-queue names via the data items in the associated CD. If, however, he desires the next message in a given sub-queue, he must specify both the queue name and any applicable sub-queue names.

For output, the COBOL user specifies only the destination(s) of the message, and the MCS places the message in the proper output queue structure.

There is no one-to-one relationship between a communication device and a source or destination. A source or destination may consist of one or more physical devices. The device or devices which compose a source or destination are defined to the MCS.

1.2.9. Debugging

To assist in error detection, COBOL provides the facility to monitor during program execution:

1. Transfers of control to user selected procedures, and
2. Values of user selected data items.

The user statements required to accomplish this monitoring are included in the source program and can be compiled or not according to the presence or absence of one clause in the source program. Once compiled into the program, these statements may be executed or ignored at run time according to the setting of a run-time switch.

1.2.10. Library

The library feature provides the facility to copy source text from a "library" of source text material that is available at compile time. A short phrase can cause inclusion of large amounts of source library material into the source program, thus saving repetitious coding. Once established, a source library may be referenced many times by many programs.

1.3. Overall Language Considerations

1.3.1. General

The language considerations and rules specified in this section apply to the highest level of the American National Standard COBOL. When a particular level of a module does not allow all of these language concepts, the restrictions will be pointed out in the section describing that language element. Also, note that restrictions contained in one module might possibly affect other modules. For example, series connectives are not allowed in Level 1 of the Nucleus; therefore, any module which is combined with Level 1 of the Nucleus would have the same restriction.

1.3.2. Notation Used in Formats and Rules

1.3.2.1. Definition of a General Format

A general format is the specific arrangement of the elements of a clause or a statement. A clause or a statement consists of elements as defined in 1.3.2.1.3. Throughout this document a format is shown adjacent to information defining the clause or statement. When more than one specific arrangement is permitted, the general format is separated into numbered formats. Clauses must be written in the sequence given in the general formats. (Clauses that are optional must appear in the sequence shown if they are used.) In certain cases, stated explicitly in the rules associated with a given format, the clauses may appear in sequences other than that shown. Applications, requirements, or restrictions are shown as rules. Throughout this manual, specifications unique to the high level are enclosed in solid line boxes.

1.3.2.1.1. Syntax Rules

Syntax rules are those rules that define or clarify the order in which words or elements are arranged to form larger elements such as phrases, clauses, or statements. Syntax rules also impose restrictions on individual words or elements.

These rules are used to define or clarify how the statement must be written; in other words, the order of the elements of the statement and restrictions on what each element may represent.

1.3.2.1.2. General Rules

A general rule is a rule that defines or clarifies the meaning or relationship of meanings of an element or set of elements. It is used to define or clarify the semantics of the statement and the effect that it has on either execution or compilation.

1.3.2.1.3. Elements

Elements that make up a clause or a statement consist of uppercase words, lowercase words, level-numbers, brackets, braces, connectives, and special characters.

1.3.2.1.4. Words

All underlined uppercase words are called key words and are required when the functions of which they are a part are used. Uppercase words which are not underlined are optional to the user and may or may not be present in the source program. Uppercase words, whether underlined or not, must be spelled correctly.

Lowercase words (in italics), in a general format, are generic terms used to represent COBOL words, literals, PICTURE character-strings, comment-entries, or a complete syntactical entry that must be supplied by the user. Where generic terms are repeated in a general format, a number or letter appendage to the term serves to identify that term for explanation or discussion.

1.3.2.1.5. Level–Numbers

When specific level-numbers appear in data description entry formats, those specific level-numbers are required when such entries are used in a COBOL program. In this manual, the form 01, 02, . . . , 09 is used to indicate level-numbers 1 through 9.

1.3.2.1.6. Brackets and Braces

When a portion of a general format is enclosed in brackets, [ ], that portion may be included or omitted at the user's choice. Braces, { }, enclosing a portion of a general format means a selection of one of the options contained within the braces must be made. In both cases, a choice is indicated by vertically stacking the possibilities. When brackets or braces enclose a portion of a format, but only one possibility is shown, the function of the brackets or braces is to delimit that portion of the format to which a following ellipsis applies. (See 1.3.2.1.7.) If an option within braces contains only reserved words that are not key words, then the option is a default option (implicitly selected unless one of the other options is explicitly indicated).

1.3.2.1.7. The Ellipsis

In text, the ellipsis (. . .) may show the omission of a portion of a source program. This meaning becomes apparent in context.

In the general formats, the ellipsis represents the position at which repetition may occur at the user's option. The portion of the format that may be repeated is determined as follows:

> Given . . . in a clause or statement format, scanning right to left, determine the] or } immediately to the left of the . . .; continue scanning right to left and determine the logically matching [or { ; the . . . applies to the words between the determined pair of delimiters.=

1.3.2.1.8. Format Punctuation

The punctuation characters comma and semicolon are shown in some formats. Where shown in the formats, they are optional and may be included or omitted by the user. In the source program these two punctuation characters are interchangeable and either one may be used anywhere one of them is shown in the formats. Neither one may appear immediately preceding the first clause of an entry or paragraph.

A semicolon or comma may be used between statements in the Procedure Division.

Paragraphs within the Identification and Procedure Divisions, and the entries within the Environment and Data Divisions must be terminated by the separator period.

1.3.2.1.9. Use of Certain Special Characters in Formats

The characters "+", "-", " >", " < ", "=", when appearing in formats, although not underlined, are required when such formats are used.

1.3.3. Language Concepts

1.3.3.1. Character Set

The most basic and indivisible unit of the language is the character. The set of characters used to form COBOL character-strings and separators includes the letters of the alphabet, digits and special characters. The character set consists of 51 or 77 characters as defined under COBOL Character Set in the Glossary. The ASCII COBOL compiler has implicitly extended the COBOL character set to include the lowercase alphabetic characters unless excluded by the M extra option or as indicated in the next paragraph. When included by default, the lowercase alphabetic characters are folded to the uppercase alphabetic characters. In the case of nonnumeric literals, comment-entries, and comment lines, the character set is expanded to include the computer's entire character set. The characters allowable in each type of character-string and as separators are defined in 1.3.3.2 and in the Glossary.

The COBOL character set can be extended to include characters not in the list of 51 characters. The SOURCE–ALPHABET clause of the Control Division enables the extension of the COBOL character set to include other characters of the computer's character set. If the SOURCE-ALPHABET clause contains any of the lowercase alphabetic characters, then only those lowercase characters explicitly specified in the SOURCE-ALPHABET clause are added to the COBOL character set. When any lowercase character is added to the COBOL character set by the SOURCE-ALPHABET clause, the character is not implicitly folded to the uppercase character and is treated as a distinct character throughout the entire COBOL program.

1.3.3.2. Language Structure

The individual characters of the language are concatenated to form character–strings and separators. A separator may be concatenated with another separator or with a character-string. A character-string may only be concatenated with a separator. The concatenation of character-strings and separators forms the text of a source program.

1.3.3.2.1. Separators

A separator is a string of one or more punctuation characters. The rules for formation of separators are:

1. The punctuation character space is a separator. Anywhere a space is used as a separator, more than one space may be used.
2. The punctuation characters comma, semicolon, and period, when immediately followed by a space, are separators. These separators may appear in a COBOL source program only where explicitly permitted by the general formats, by format punctuation rules (see 1.3.2.1.8), by statement and sentence structure definitions (see 1.3.8.2), or reference format rules (see 1.3.9).
3. The punctuation characters right and left parentheses are separators. Parentheses may appear only in balanced pairs of left and right parentheses delimiting subscripts, indices, arithmetic expressions, or conditions.
4. The punctuation character quotation mark is a separator. An opening quotation mark must be immediately preceded by a space or left parenthesis; a closing quotation mark must be immediately followed by one of the separators space, comma, semicolon, period, or right parenthesis.

   Quotation marks may appear only in balanced pairs delimiting nonnumeric literals except when the literal is continued. (See 1.3.9.2.2.)

5. Pseudo-text delimiters are separators. An opening pseudo-text delimiter must be immediately preceded by a space; a closing pseudo-text delimiter must be immediately followed by one of the separators space, comma, semicolon, or period.

   Pseudo-text delimiters may appear only in balanced pairs delimiting pseudo-text.

6. The separator space may optionally immediately precede all separators except:
   a. As specified by reference format rules (see 1.3.9), and
   b. The separator closing quotation mark. In this case, a preceding space is considered as part of the nonnumeric literal and not as a separator.
   c. The opening pseudo-text delimiter, where the preceding space is required.
7. The separator space may optionally immediately follow any separator except the opening quotation mark. In this case, a following space is considered as part of the nonnumeric literal and not as a separator.

Any punctuation character which appears as part of the specification of a PICTURE character-string or numeric literal is not considered as a punctuation character, but rather as a symbol used in the specification of that PICTURE character-string or numeric literal. PICTURE character-strings are delimited only by the separators space, comma, semicolon, or period.

The rules established for the formation of separators do not apply to the characters that compose the contents of nonnumeric literals, comment-entries, or comment lines.

1.3.3.2.2. Character–Strings

A character-string is a character or a sequence of contiguous characters which forms a COBOL word, a literal, a PICTURE character-string, or a comment-entry. A character-string is delimited by separators.

1.3.3.2.2.1. COBOL Words

A COBOL word is a character-string of not more than 30 characters that forms a user-defined word, a system-name, or a reserved word. Within a given source program these classes form disjoint sets; a COBOL word may belong to one and only one of these classes.

1.3.3.2.2.1.1. User–Defined Words

A user-defined word is a COBOL word that must be supplied by the user to satisfy the format of a clause or statement. Each character of a user-defined word is selected from the set of characters "A", "B", "C", ... "Z", "0", ... "9", and "-", except that the "-" may not appear as the first or last character. The characters implicitly or explicitly added to the COBOL character set can be used in the formation of a user-defined word. See 1.3.3.1 for details.

There are 17 types of user-defined words:

alphabet-name
cd-name
condition-name
data-name
file-name
index-name
level-number
library-name
mnemonic-name
paragraph-name
program-name
record-name
report-name
routine-name
section-name
segment-number
text-name

Within a given source program, 15 of these 17 types of user–defined words are grouped into 13 disjoint sets. The disjoint sets are:

alphabet-names
cd-names
condition-names, data-names, and record-names
file-names
index-names
library-names
mnemonic-names
paragraph-names
program-names
report-names
routine-names
section-names
text-names

All user-defined words, except segment-numbers and level-numbers, can belong to one and only one of these disjoint sets. Further, all user-defined words within a given disjoint set must be unique, either because no other user-defined word in the same source program has identical spelling or punctuation, or because uniqueness can be insured by qualification. (See 1.3.3.3.8.)

With the exception of program-name, paragraph-name, section-name, level-number and segment-number, all user-defined words must contain at least one alphabetic character. Segment-numbers and level-numbers need not be unique; a given specification of a segment-number or level-number may be identical to any other segment-number or level-number and may even be identical to a paragraph-name or section-name.

- Condition-Name

  A condition-name is a name which is assigned to a specific value, set of values, or range of values, within a complete set of values that a data item may assume. The data item itself is called a conditional variable.

  Condition-names may be defined in the Data Division or in the SPECIAL–NAMES paragraph within the Environment Division where a condition-name must be assigned to the ON STATUS or OFF STATUS, or both, of implementor-defined switches.

  A condition-name is used in conditions as an abbreviation for the relation condition. This relation condition postulates that the associated conditional variable is equal to one of the set of values to which that condition-name is assigned.

- Mnemonic-Name

  A mnemonic-name assigns a user-defined word to an implementor-name. These associations are established in the SPECIAL–NAMES paragraph of the Environment Division. (See 2.4.1.3.)

- Paragraph-Name

  A paragraph-name is a word which names a paragraph in the Procedure Division. Paragraph-names are equivalent if, and only if, they are composed of the same sequence of the same number of digits and/or characters.

- Section-name

  A section-name is a word which names a section in the Procedure Division. Section-names are equivalent if, and only if, they are composed of the same sequence of the same number of digits and/or characters.

- Other User-Defined Names

  See the Glossary for definitions of all other types of user-defined words.

1.3.3.2.2.1.2. System-Names

A system-name is a COBOL word which is used to communicate with the operating environment. Each character used in the formation of a system–name must be selected from the set of characters "A", "B", "C", . . . "Z", "0", . . . "9", and "-", except that the "-" may not appear as the first or last character.

There are two types of system-names:

computer-name
implementor-name

These two types of system-names form disjoint sets. A given system-name may belong to one and only one of them.

The system-names listed are individually defined in the Glossary.

1.3.3.2.2.1.3. Reserved Words

A reserved word is a COBOL word that is one of a specified list of words which may be used in COBOL source programs, but which must not appear in the programs as user-defined words or system-names. Reserved words can only be used as specified in the general formats. (See Appendix A.)

There are six types of reserved words:

Key words
Optional words
Connectives
Special registers
Figurative constants
Special–character words

- Key Words

  A key word is a word whose presence is required when the format in which the word appears is used in a source program. Within each format, such words are uppercase and underlined.

  Key words are of three types:

  1. Verbs such as ADD, READ, and ENTER.

  2. Required words, which appear in statement and entry formats.

  3. Words which have a specific functional meaning such as NEGATIVE, SECTION.

- Optional Words

  Within each format, uppercase words that are not underlined are called optional words and may appear at the user's option. The presence or absence of an optional word does not alter the semantics of the COBOL program in which it appears.

- Connectives

  There are three types of connectives:

  1. Qualifier connectives that are used to associate a data-name, a condition-name, a text-name, or a paragraph-name with its qualifier: OF, IN
  2. Series connectives that link two or more consecutive operands: , (separator comma) or ; (separator semicolon)
  3. Logical connectives that are used in the formation of conditions: AND, OR

- Special Registers

  Certain reserved words are used to name and reference special registers. Special registers are certain compiler-generated storage areas whose primary use is to store information produced in conjunction with the use of specific COBOL features. These special registers include: LINAGE–COUNTER (see 4.1.3.9), LINE–COUNTER (see 8.1.2.1), PAGE–COUNTER (see 8.1.2.2), and DEBUG–ITEM (see 11.1.3.1).

- Figurative Constants

  Certain reserved words are used to name and reference specific constant values. These reserved words are specified in 1.3.3.2.2.2.3 .

- Special-Character Words

  The arithmetic operators and relation characters are reserved words. (See the Glossary.)

1.3.3.2.2.2. Literals

A literal is a character-string whose value is implied by an ordered set of characters of which the literal is composed or by specification of a reserved word which references a figurative constant. Every literal belongs to one of two types, nonnumeric or numeric.

1.3.3.2.2.2.1. Nonnumeric Literals

A nonnumeric literal is a character-string delimited on both ends by quotation marks and consisting of any allowable character in the computer's character set. Nonnumeric literals of 1 through 132 characters in length are allowed. To represent a single quotation mark character within a nonnumeric literal, two contiguous quotation marks must be used. The value of a nonnumeric literal in the object program is the string of characters itself, except:

1. The delimiting quotation marks are excluded, and
2. Each embedded pair of contiguous quotation marks represents a single quotation mark character.

All other punctuation characters are part of the value of the nonnumeric literal rather than separators. All nonnumeric literals are category alphanumeric. (See 2.5.7.)

1.3.3.2.2.2.2. Numeric Literals

There are two types of numeric literals: fixed-point and floating-point.

- Fixed-Point

  A fixed-point literal is a character-string whose characters are selected from the digits "0" through "9", the plus sign, the minus sign, and/or the decimal point. Fixed-point literals of 1 through 18 digits in length are allowed. The rules for the formation of fixed–point literals are:

  1. A literal must contain at least one digit.
  2. A literal must not contain more than one sign character. If a sign is used, it must appear as the leftmost character of the literal. If the literal is unsigned, the literal is positive.
  3. A literal must not contain more than one decimal point. The decimal point is treated as an assumed decimal point, and may appear anywhere within the literal except as the rightmost character. If the literal contains no decimal point, the literal is an integer.

     If a literal conforms to the rules for the formation of numeric literals, but is enclosed in quotation marks, it is a nonnumeric literal and it is treated as such by the compiler.
  4. The value of a fixed-point literal is the algebraic quantity represented by the characters in the fixed-point literal. Every fixed-point literal is category numeric. (See 2.5.7, the PICTURE clause.) The size of a fixed-point literal in standard data format characters is equal to the number of digits specified by the user.

- Floating-Point

  A floating-point literal is a character-string whose characters are selected from the digits "0" through "9", the plus sign, the minus sign, the decimal point, and the character "E."

  A floating-point literal must have the following format:

  [±] *mantissa* E [±] *exponent*

  where:

  ±

  The two plus or minus signs are optional.

  mantissa

  Consists of 1 to 18 digits with a required decimal point; the decimal point may appear in any position. If the mantissa is signed, the sign must be the leftmost character in the literal string. If the mantissa is unsigned, the literal value is positive.

E

The character "E" must immediately follow the mantissa, and the exponent must appear immediately to the right of the character "E".

exponent

Consists of 1 to 3 digits and an optional sign. If the sign is present, it must be immediately to the right of the symbol "E" and immediately to the left of the digits. Unsigned exponents are positive. The exponent may not contain a decimal point.

The largest number that can be represented by a floating–point literal is 0.83×10^{307}, and the smallest number which can be represented is -0.16×10^{-308}. A zero exponent must be written as 0, 00, or 000.

The value of a floating-point literal is the algebraic quantity product of the mantissa and 10 raised to the power given by the exponent.

Example

`+1.5E-2 = 1.5 x 10^-2`

1.3.3.2.2.2.3. Figurative Constant Values

Figurative constant values are generated by the compiler and referenced through the use of the reserved words that follow. These words must not be bounded by quotation marks when used as figurative constants. The singular and plural forms of figurative constants are equivalent and may be used interchangeably.

The figurative constant values and the reserved words used to reference them are as follows:

ZERO, ZEROS, ZEROES

Represents the value "0", or one or more of the character "0", depending on context. The DISPLAY code is octal 060, and the COMPUTATIONAL code is 000.

SPACE, SPACES

Represents one or more space characters from the computer's character set. The DISPLAY code is octal 040.

HIGH–VALUE, HIGH–VALUES

Represents one or more of the character that has the highest ordinal position in the program collating sequence. The DISPLAY code is octal 177 for the native collating sequence or octal 377 if the D extra option is specified.

LOW–VALUE, LOW–VALUES

Represents one or more of the character that has the lowest ordinal position in the program collating sequence. The DISPLAY code is octal 000.

QUOTE, QUOTES

Represents one or more of the character " or ', depending upon how the compiler is configured. The word QUOTE or QUOTES cannot be used in place of a quotation mark to bound a nonnumeric literal. Thus, QUOTE ABD QUOTE is incorrect as a way of stating the nonnumeric literal "ABD" or 'ABD'. The DISPLAY code is octal 042 for " and octal 047 for '.

ALL *literal*

Represents one or more of the string of characters composing the *literal.* The *literal* must be either a nonnumeric literal or a figurative constant other than ALL *literal*. When a figurative constant is used, the word ALL is redundant and is used for readability only.

When a figurative constant represents a string of one or more characters, the length of the string is determined by the compiler from context according to the following rules.

1. When a figurative constant is associated with another data item, as when the figurative constant is moved to or compared with another data item, the string of characters specified by the figurative constant is repeated character by character on the right until the size of the resultant string is equal to the size in characters of the associated data item. This is done prior to and independent of the application of any JUSTIFIED clause that may be associated with the data item.
2. When a figurative constant is not associated with another data item, as when the figurative constant appears in a DISPLAY, STRING, STOP, or UNSTRING statement, the length of the string is one character.

A figurative constant may be used wherever a literal appears in a format, except that whenever the literal is restricted to having only numeric characters in it, the only figurative constant permitted is ZERO (ZEROS, ZEROES).

When the figurative constants HIGH–VALUE(S) or LOW–VALUE(S) are used in the source program, the actual character associated with each figurative constant depends upon the program collating sequence specified. (See 2.4.1.2, "The OBJECT–COMPUTER Paragraph," and 2.4.1.3, "The SPECIAL–NAMES Paragraph.")

Each reserved word which is used to reference a figurative constant value is a distinct character-string with the exception of the construction ALL *literal* which is composed of two distinct character-strings.

1.3.3.2.2.3. PICTURE Character–Strings

A PICTURE character-string consists of certain combinations of characters in the COBOL character set used as symbols. See 2.5.7 for the discussion of the PICTURE character-string and for the rules that govern its use.

Any punctuation character which appears as part of the specification of a PICTURE character-string is not considered as a punctuation character, but rather as a symbol used in the specification of that PICTURE character-string.

1.3.3.2.2.4. Comment–Entries

A comment-entry is an entry in the Identification Division that may be any combination of characters from the computer's character set.

1.3.3.3. Concept of Computer Independent Data Description

To make data as computer independent as possible, the characteristics or properties of the data are described in relation to a standard data format rather than an equipment-oriented format. This standard data format is oriented to general data processing applications and uses the decimal system to represent numbers (regardless of the radix used by the computer) and the remaining characters in the COBOL character set to describe nonnumeric data items.

1.3.3.3.1. Logical Record and File Concept

The approach taken in defining file information is to distinguish between the physical aspects of the file and the conceptual characteristics of the data contained within the file.

1.3.3.3.1.1. Physical Aspects of a File

The physical aspects of a file describe the data as it appears on the input or output media and include such features as:

- The grouping of logical records within the physical limitations of the file medium.
- The means by which the file can be identified.

1.3.3.3.1.2. Conceptual Characteristics of a File

The conceptual characteristics of a file are the explicit definition of each logical entity within the file itself. In a COBOL program, the input or output statements refer to one logical record.

It is important to distinguish between a physical record and a logical record. A COBOL logical record is a group of related information, uniquely identifiable, and treated as a unit.

A physical record is a physical unit of information whose size and recording mode is convenient to a particular computer for the storage of data on an input or output device. The size of a physical record is hardware dependent and bears no direct relationship to the size of the file of information contained on a device.

A logical record may be contained within a single physical unit; or several logical records may be contained within a single physical unit; or, in the case of mass storage files, a logical record may require more than one physical unit to contain it. There are several source language methods available for describing the relationship of logical records and physical units. When a permissible relationship has been established, control of the accessibility of logical records as related to the physical unit must be provided by the interaction of the object program on the implementor's hardware and software system. In this manual, references to records means to logical records, unless the term "physical record" is specifically used.

The concept of a logical record is not restricted to file data but is carried over into the definition of working storage. Thus, working storage may be grouped into logical records and defined by a series of record description entries.

1.3.3.3.1.3. Record Concepts

The record description consists of a set of data description entries which describe the characteristics of a particular record. Each data description entry consists of a level-number followed by a data-name, if required, followed by a series of independent clauses, as required.

1.3.3.3.2. Concept of Levels

A level concept is inherent in the structure of a logical record. This concept arises from the need to specify subdivisions of a record for the purpose of data reference. Once a subdivision has been specified, it may be further subdivided to permit more detailed data referral.

The most basic subdivisions of a record, that is, those not further subdivided, are called elementary items. Consequently, a record is said to consist of a sequence of elementary items, or the record itself may be an elementary item.

In order to refer to a set of elementary items, the elementary items are combined into groups. Each group consists of a named sequence of one or more elementary items. Groups, in turn, may be combined into groups of two or more groups, etc. Thus, an elementary item may belong to more than one group.

1.3.3.3.2.1. Level-Numbers

A system of level-numbers shows the organization of elementary items and group items. Since records are the most inclusive data items, level-numbers for records start at 01. Less inclusive data items are assigned higher (not necessarily successive) level-numbers not greater in value than 49. There are special level-numbers 66, 77, and 88, which are exceptions to this rule. Separate entries are written in the source program for each level-number used.

A group includes all group and elementary items following it until a level-number less than or equal to the level-number of that group is encountered. All items which are immediately subordinate to a given group item must be described using identical level-numbers greater than the level-number used to describe that group item.

Three types of entries exist for which there is no true concept of level. These are:

1. Entries that specify elementary items or groups introduced by a RENAMES clause
2. Entries that specify noncontiguous working storage and linkage data items
3. Entries that specify condition-names

Entries describing items by means of RENAMES clauses for the purpose of regrouping data items have been assigned the special level-number 66.

Entries that specify noncontiguous data items, which are not subdivisions of other items, and are not themselves subdivided, have been assigned the special level-number 77.

Entries that specify condition-names, to be associated with particular values of a conditional variable, have been assigned the special level-number 88.

1.3.3.3.3. Concept of Classes of Data

The five categories of data items (see 2.5.7, The PICTURE Clause) are grouped into three classes: alphabetic, numeric, and alphanumeric. For alphabetic and numeric, the classes and categories are synonymous. The alphanumeric class includes the categories of alphanumeric edited, numeric edited and alphanumeric (without editing). Every elementary item except for an index data item belongs to one of the classes and further to one of the categories. The class of a group item is treated at object time as alphanumeric regardless of the class of elementary items subordinate to that group item. The following table depicts the relationship of the class and categories of data items.

Level of Item	Class	Category
Elementary	Alphabetic	Alphabetic
	Numeric	Numeric
	Alphanumeric	Numeric Edited
		Alphanumeric Edited
		Alphanumeric
Nonelementary (Group)	Alphanumeric	Alphabetic
		Numeric
		Numeric Edited
		Alphanumeric Edited
		Alphanumeric

1.3.3.3.4. Selection of Character Representation and Radix

The value of a numeric item may be represented in either binary or decimal form depending on the equipment. In addition there are several ways of expressing decimal. Since these representations are actually combinations of bits, they are commonly called binary-coded decimal forms. The selection of radix is generally dependent upon the arithmetic capability of the computer. If more than one arithmetic radix is provided, the selection is dependent upon factors included in such clauses as USAGE. The binary-coded decimal form is also used to represent characters and symbols that are alphanumeric items.

The selection of the proper binary-coded alphanumeric or binary-coded decimal form is dependent upon the capability of the computer and its external media.

When a computer provides more than one means of representing data, the standard data format must be used if not otherwise specified by the data description. If both the external medium and the computer are capable of handling more than one form of data representation, or if there is no external medium associated with the data, the selection is dependent on factors included in USAGE, PICTURE, etc., clauses. (See 3.2.2 for a description of the possible forms.)

The size of an elementary data item or a group item is the number of characters in standard data format of the item. Synchronization and usage may cause a difference between this size and the actual number of characters required for the internal representation.

1.3.3.3.5. Algebraic Signs

Algebraic signs fall into two categories: operational signs, which are associated with signed numeric data items and signed numeric literals to indicate their algebraic properties; and editing signs, which appear on edited reports to identify the sign of the item.

The SIGN clause permits the programmer to state explicitly the location of the operational sign. The clause is optional; if it is not used, operational signs will be represented as defined by the implementor.

Editing signs are inserted into a data item through the use of the sign control symbols of the PICTURE clause.

1.3.3.3.6. Standard Alignment Rules

The standard rules for positioning data within an elementary item depend on the category of the receiving item. These rules are:

1. If the receiving data item is described as numeric:
    a. The data is aligned by decimal point and is moved to the receiving character positions with zero fill or truncation on either end as required. When a fixed-point binary item is a receiving item or has associated with it an initial value, the truncation is based on bytes allocated, not the decimal PICTURE size.
    b. When an assumed decimal point is not explicitly specified, the data item is treated as if it has an assumed decimal point immediately following its rightmost character and is aligned as in preceding paragraph 1a.
2. If the receiving data item is a numeric edited data item, the data moved to the edited data item is aligned by decimal point with zero fill or truncation at either end as required within the receiving character positions of the data item, except where editing requirements cause replacement of the leading zeros.
3. If the receiving data item is alphanumeric (other than a numeric edited data item), alphanumeric edited, or alphabetic, the sending data is moved to the receiving character positions and aligned at the leftmost character position in the data item with space fill or truncation to the right, as required.

If the JUSTIFIED clause is specified for the receiving item, these standard rules are modified as described in 2.5.5.

1.3.3.3.7. Item Alignment for Increased Object–Code Efficiency

The main storage of some computers is organized in such a way that there are natural addressing boundaries in the computer main storage (for example, word boundaries, half-word boundaries, byte boundaries). The way in which data is stored is determined by the object program, and need not respect these natural boundaries.

However, certain uses of data (for example, in arithmetic operations or in subscripting) may be facilitated if the data is stored so as to be aligned on these natural boundaries. Specifically, additional machine operations in the object program may be required for the accessing and storage of data if portions of two or more data items appear between adjacent natural boundaries, or if certain natural boundaries split a single data item.

Data items which are aligned on these natural boundaries in such a way as to avoid such additional machine operations are defined to be synchronized. A synchronized item is assumed to be introduced and carried in that form; conversion to synchronized form occurs only during the execution of a procedure (other than READ or WRITE) which stores data in the item.

Synchronization can be accomplished in two ways:

- By use of the SYNCHRONIZED clause
- By recognizing the appropriate natural boundaries and organizing the data suitably without the use of the SYNCHRONIZED clause. (See 2.5.11, general rule 9.)

1.3.3.3.8. Uniqueness of Reference

1.3.3.3.8.1. Qualification

Every user-specified name that defines an element in a COBOL source program must be unique, either because no other name has the identical spelling and hyphenation, or because the name exists within a hierarchy of names such that references to the name can be made unique by mentioning one or more of the higher levels of the hierarchy. The higher levels are called qualifiers and the process that specifies uniqueness is called qualification. Enough qualification must be mentioned to make the name unique; however, it may not be necessary to mention all levels of the hierarchy. Within the Data Division, all data-names used for qualification must be associated with a level indicator or a level-number. Therefore, two identical data-names must not appear as entries subordinate to a group item unless they can be made unique through qualification. In the Procedure Division two identical paragraph-names must not appear in the same section.

In the hierarchy of qualification, names associated with a level indicator are the most significant, then those names associated with level-number 01, then names associated with level-number 02, . . . , 49. A section-name is the highest (and the only) qualifier available for a paragraph–name. Thus, the most significant name in the hierarchy must be unique and cannot be qualified. Subscripted or indexed data-names and conditional variables, as well as procedure-names and data-names, may be made unique by qualification. The name of a conditional variable can be used as a qualifier for any of its condition-names. Regardless of the available qualification, no name can be both a data-name and procedure-name.

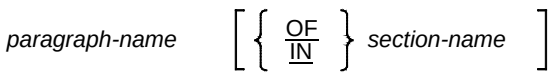
Qualification is performed by following a data-name, a condition-name, a paragraph-name, or a text-name by one or more phrases composed of a qualifier preceded by IN or OF. IN and OF are logically equivalent.

The general formats for qualification are:

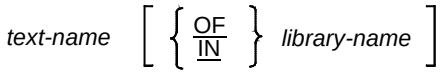
Format 1



Format 2



Format 3



The rules for qualification are:

1. Each qualifier must be of a successively higher level and within the same hierarchy as the name it qualifies.
2. The same name must not appear at two levels in a hierarchy.
3. If a data-name or a condition-name is assigned to more than one data item in a source program, the data-name or condition-name must be qualified each time it is referred to in the Procedure, Environment, and Data Divisions (except in the REDEFINES clause where qualification is unnecessary and must not be used.)
4. A paragraph-name must not be duplicated within a section. When a paragraph-name is qualified by a section-name, the word SECTION must not appear. A paragraph-name need not be qualified when referred to from within the same section.
5. A data-name cannot be subscripted when it is being used as a qualifier.
6. A name can be qualified even though it does not need qualifications; if there is more than one combination of qualifiers that ensures uniqueness, then any such set can be used. The complete set of qualifiers for a data-name must not be the same as any partial set of qualifiers for another data-name. Qualified data-names may have any number of qualifiers up to five.
7. If more than one COBOL library is available to the compiler during compilation, text-name must be qualified each time it is referenced.

1.3.3.3.8.2. Subscripting

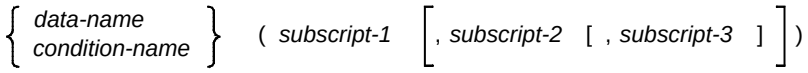
Subscripts can be used only when referencing an individual element within a list or table of like elements that have not been assigned individual data-names (see 3.2.1).

The subscript can be represented either by a numeric literal that is an integer or by a data-name. The data-name must be a numeric elementary item that represents an integer. When the subscript is represented by a data-name, the data-name may be qualified but not subscripted. In the Report Section, neither a sum counter nor the special registers LINE–COUNTER and PAGE–COUNTER can be used as a subscript.

The subscript may be signed and, if signed, it must be positive. The lowest possible subscript value is 1. This value points to the first element of the table. The next sequential elements of the table are pointed to by subscripts whose values are 2, 3, ... The highest permissible subscript value, in any particular case, is the maximum number of occurrences of the item as specified in the OCCURS clause.

The subscript, or set of subscripts, that identifies the table element is delimited by the balanced pair of separators (left parenthesis and right parenthesis) following the table element data-name. The table element data-name appended with a subscript is called a subscripted data-name or an identifier. When more than one subscript is required, they are written in the order of successively less inclusive dimensions of the data organization.

The format is:



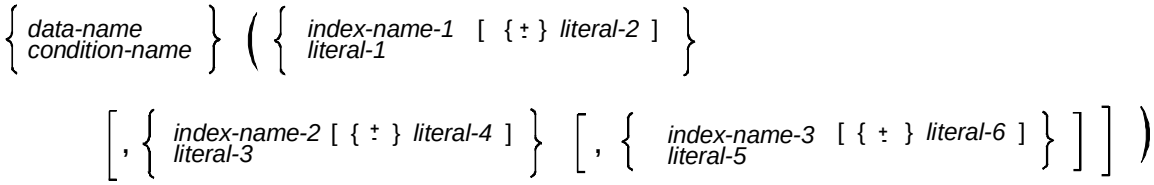
1.3.3.3.8.3. Indexing

References can be made to individual elements within a table of like elements by specifying indexing for that reference. An index is assigned to that level of the table by using the INDEXED BY phrase in the definition of a table. A name given in the INDEXED BY phrase is known as an index-name and is used to refer to the assigned index. The value of an index corresponds to the occurrence number of an element in the associated table. An index-name must be initialized before it is used as a table reference. An index-name can be given an initial value by either a SET, a SEARCH ALL, or a Format 4 PERFORM statement.

Direct indexing is specified by using an index-name in the form of a subscript. Relative indexing is specified when the index-name is followed by the operator "+" or "-", followed by an unsigned integer numeric literal all delimited by the balanced pair of separators (left parenthesis and right parenthesis) following the table element data-name. The occurrence number resulting from relative indexing is determined by incrementing (where the operator "+" is used) or decrementing (when the operator "-" is used), by the value of the literal, the occurrence number represented by the value of the index. When more than one index-name is required, they are written in the order of successively less inclusive dimensions of the data organization.

At the time of execution of a statement which refers to an indexed table element, the value contained in the index referenced by the index-name associated with the table element must neither correspond to a value less than one (1) nor to a value greater than the highest permissible occurrence number of an element of the associated table. This restriction also applies to the value resultant from relative indexing.

The general format for indexing is:

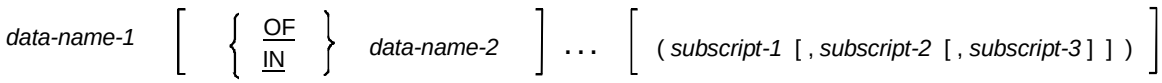


1.3.3.3.8.4. Identifier

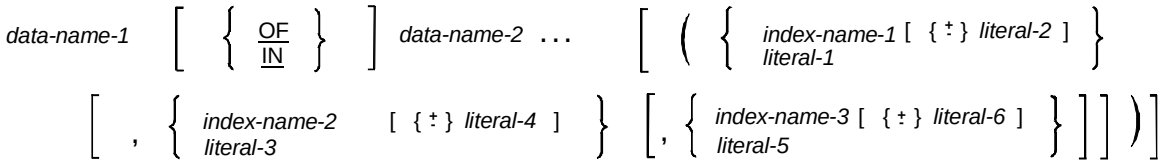
An identifier is a term used to reflect that a data-name, if not unique in a program, must be followed by a syntactically correct combination of qualifiers, subscripts or indices necessary to ensure uniqueness.

The general formats for identifiers are:

Format 1



Format 2



Restrictions on qualification, subscripting, and indexing are:

1. A data-name must not itself be subscripted nor indexed when that data-name is being used as an index, subscript, or qualifier.
2. Indexing is not permitted where subscripting is not permitted.
3. An index may be modified only by the SET, SEARCH, and PERFORM statements. Data items described by the USAGE IS INDEX clause permit storage of the values associated with index-names as data. Such data items are called index data items.
4. *Literal-1, literal-3,* and *literal-5* in the preceding format must be positive numeric integers. *Literal-2, literal-4,* and *literal-6* must be unsigned numeric integers.

1.3.3.3.8.5. Condition-Name

Each condition-name must be unique, or be made unique through qualification, and/or indexing, or subscripting.

If qualification is used to make a condition-name unique, the associated conditional variable may be used as the first qualifier. If qualification is used, the hierarchy of names associated with the conditional variable or the conditional variable itself must be used to make the condition-name unique.

If references to a conditional variable require indexing or subscripting, then references to any of its condition-names also require the same combination of indexing or subscripting.

The format and restrictions on the combined use of qualification and/or subscripting, and indexing of condition-names is exactly that of "identifier" except that *data-name-1* is replaced by *condition-name-1.*

In the general formats, *condition-name* refers to a condition-name qualified, indexed, or subscripted, as necessary.

1.3.3.4. Explicit and Implicit Specifications

There are three types of explicit and implicit specifications that occur in COBOL source programs:

1. Explicit and implicit Procedure Division references
2. Explicit and implicit transfers of control
3. Explicit and implicit attributes.

1.3.3.4.1. Explicit and Implicit Procedure Division References

A COBOL source program can reference data items either explicitly or implicitly in Procedure Division statements. An explicit reference occurs when the name of the referenced item is written in a Procedure Division statement or when the name of the referenced item is copied into the Procedure Division by the processing of a COPY statement. An implicit reference occurs when the item is referenced by a Procedure Division statement without the name of the referenced item being written in the source statement. An implicit reference also occurs, during the execution of a PERFORM statement, when the index or data item referenced by the index-name or identifier specified in the VARYING, AFTER, or UNTIL phrase is initialized, modified, or evaluated by the control mechanism associated with that PERFORM statement. Such an implicit reference occurs if and only if the data item contributes to the execution of the statement.

1.3.3.4.2. Explicit and Implicit Transfers of Control

The mechanism that controls program flow transfers control from statement to statement in the sequence in which they were written in the source program, unless an explicit transfer of control overrides this sequence or there is no next executable statement to which control can be passed. The transfer of control from statement to statement occurs without the writing of an explicit Procedure Division statement and therefore, is an implicit transfer of control.

COBOL provides both explicit and implicit means of altering the implicit control transfer mechanism.

In addition to the implicit transfer of control between consecutive statements, implicit transfer of control also occurs when the normal flow is altered without the execution of a procedure branching statement. COBOL provides the following types of implicit control flow alterations which override the statement–to–statement transfers of control.

1. If a paragraph is being executed under control of another COBOL statement (for example, PERFORM, USE, SORT, and MERGE) and the paragraph is the last paragraph in the range of the controlling statement, then an implied transfer of control occurs from the last statement in the paragraph to the control mechanism of the last executed controlling statement. Further, if a paragraph is being executed under the control of a PERFORM statement which causes iterative execution and that paragraph is the first paragraph in the range of that PERFORM statement, an implicit transfer of control occurs between the control mechanism associated with that PERFORM statement and the first statement in that paragraph for each iterative execution of the paragraph.

2. When a SORT or MERGE statement is executed, an implicit transfer of control occurs to any associated input or output procedures.
3. When any COBOL statement is executed which results in the execution of a declarative section, an implicit transfer of control to the declarative section occurs. Note that another implicit transfer of control occurs after execution of the declarative section, as described in (1) above.

An explicit transfer of control consists of an alteration of the implicit control transfer mechanism by the execution of a procedure branching or conditional statement. (See 1.3.8.2.4.) An explicit transfer of control can be caused only by the execution of a procedure branching or conditional statement. The execution of the procedure branching statement ALTER does not in itself constitute an explicit transfer of control, but affects the explicit transfer of control that occurs when the associated GO TO statement is executed. The procedure branching statement EXIT PROGRAM causes an explicit transfer of control when the statement is executed in a called program.

In this manual, the term "next executable statement" is used to refer to the next COBOL statement to which control is transferred according to the rules just described and the rules associated with each language element in the Procedure Division.

There is no next executable statement following:

1. The last statement in a declarative section when the paragraph in which it appears is not being executed under the control of some other COBOL statement.
2. The last statement in a program when the paragraph in which it appears is not being executed under the control of some other COBOL statement.

1.3.3.4.3. Explicit and Implicit Attributes

Attributes may be implicitly or explicitly specified. Any attribute which has been explicitly specified is called an explicit attribute. If an attribute has not been specified explicitly, then the attribute takes on the default specification. Such an attribute is known as an implicit attribute.

For example, the usage of a data item need not be specified, in which case a data item's usage is DISPLAY.

1.3.4. Control Division

1.3.4.1. General Description

The Control Division is an optional entry in a COBOL source program. This division is used to control the compiler. It directs the compiler what to accept as the valid source character set.

1.3.4.2. Organization

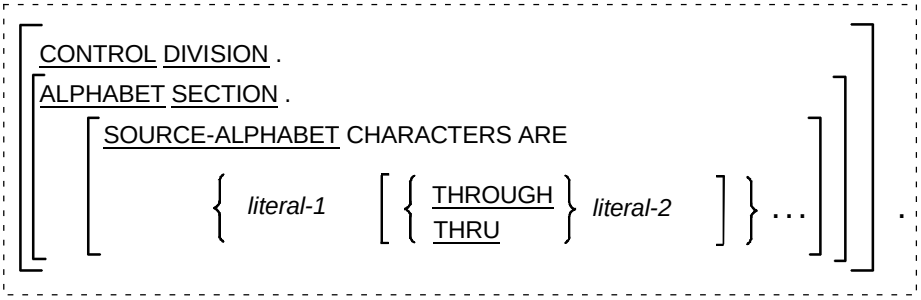
The Control Division contains the Alphabet Section.

The Alphabet Section specifies what characters are to be added to the COBOL character set of the source program.

1.3.4.3. Control Division Structure

The following is the general format of the Control Division.

1.3.4.3.1. General Format



1.3.5. Identification Division

1.3.5.1. General Description

The Identification Division must be included in every COBOL source program. This division identifies both the source program and the resultant output listing. In addition, the user may include the date the program is written, the date the compilation of the source program is accomplished, and other information as shown in the format below.

1.3.5.2. Organization

Paragraph headers identify the type of information contained in the paragraph. The name of the program must be given in the first paragraph, which is the PROGRAM–ID paragraph. The other paragraphs are optional and may be included in this division at the user's choice, in the order of presentation shown in the format that follows.

1.3.5.3. Structure

The following is the general format of the paragraphs in the Identification Division, and it defines the order of presentation in the source program.

1.3.5.3.1. General Format

```
IDENTIFICATION DIVISION.

PROGRAM-ID.  program-name.

[ AUTHOR.  [  comment-entry  ]   . . .  ]

[ INSTALLATION.  [  commen-entry  ]    . . .  ]

[ DATE-WRITTEN.  [  comment-entry  ]    . . .  ]

[ DATE-COMPILED.  [  comment-entry  ]    . . .  ]

[ SECURITY.  [  comment-entry  ]    . . .  ]
```

1.3.6. Environment Division

1.3.6.1. General Description

The Environment Division specifies a standard method of expressing those aspects of a data processing problem that are dependent upon the physical characteristics of a specific computer. This division allows specification of the configuration of the compiling computer and the object computer. In addition, information relating to input-output control, special hardware characteristics and control techniques can be given.

The Environment Division must be included in every COBOL source program.

1.3.6.2. Organization

Two sections make up the Environment Division: the Configuration Section and the Input-Output Section.

The Configuration Section deals with the characteristics of the source computer and the object computer. This section is divided into three paragraphs: the SOURCE–COMPUTER paragraph, which describes the computer configuration on which the source program is compiled; the OBJECT–COMPUTER paragraph, which describes the computer configuration on which the object program produced by the compiler is to be run; and the SPECIAL–NAMES paragraph, which relates the implementor-names used by the compiler to the mnemonic-names used in the source program.

The Input-Output Section deals with the information needed to control transmission and handling of data between external media and the object program. This section is divided into two paragraphs: the FILE–CONTROL paragraph which names and associates the files with external media; and the I–O–CONTROL paragraph which defines special control techniques to be used in the object program.

1.3.6.3. Environment Division Structure

The following is the general format of the sections and paragraphs in the Environment Division, and it defines the order of presentation in the source program.

1.3.6.3.1. General Format

```
ENVIRONMENT  DIVISION.

CONFIGURATION  SECTION.

SOURCE-COMPUTER.  source-computer-entry

OBJECT-COMPUTER.  object-computer-entry

[ SPECIAL-NAMES.  special-names-entry ]

[ INPUT-OUTPUT  SECTION.

  FILE-CONTROL. { file-control-entry }   . . .

[ I-O-CONTROL.  input-output-control-entry  ]  ]
```

1.3.6.3.2. Syntax Rules

1. The Environment Division begins with the reserved words ENVIRONMENT DIVISION followed by a period and a space.

1.3.7. Data Division

1.3.7.1. Overall Approach

The Data Division describes the data that the object program is to accept as input, to manipulate, to create, or to produce as output. Data to be processed falls into three categories:

1. That which is contained in files and enters or leaves the internal memory of the computer from a specified area or areas.
2. That which is developed internally and placed into intermediate or working storage, or placed into specific format for output reporting purposes.
3. Constants which are defined by the user.

1.3.7.2. Physical and Logical Aspects of Data Description

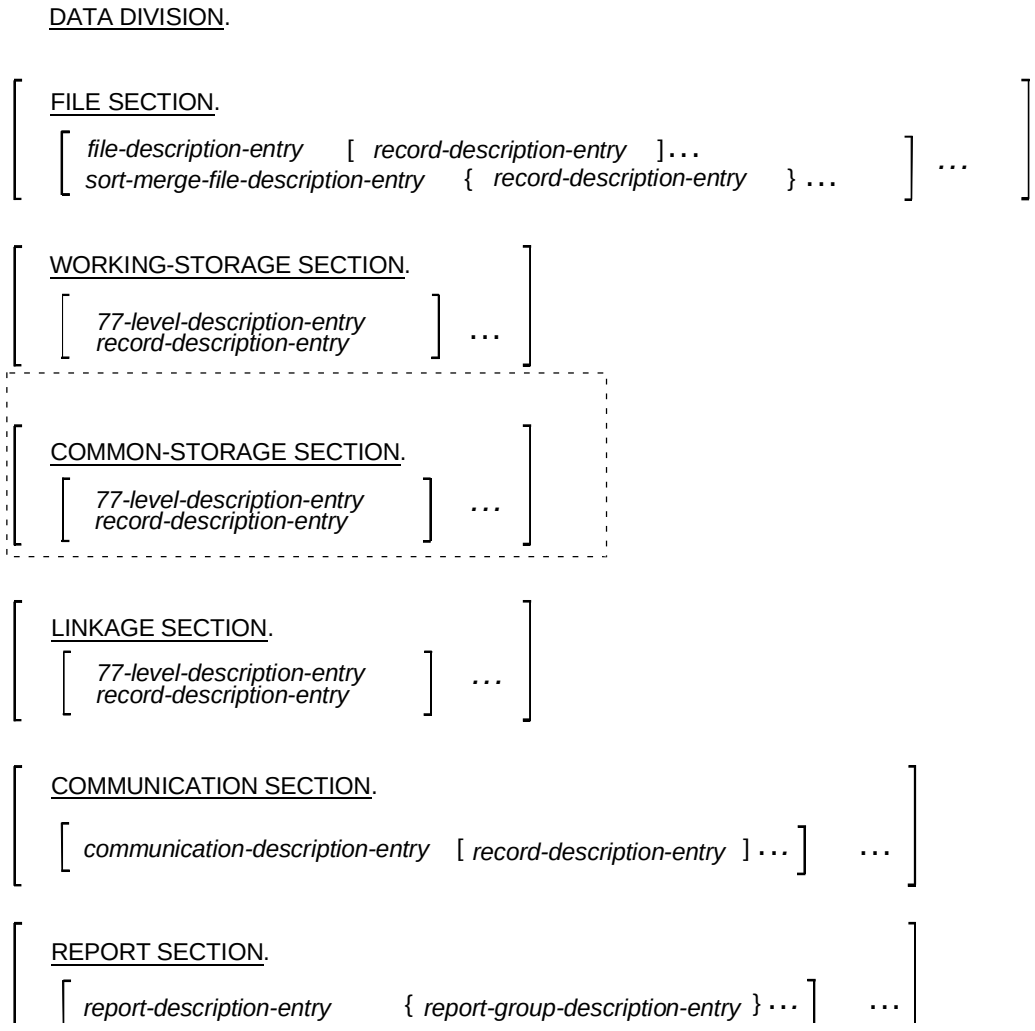
1.3.7.2.1. Data Division Organization

The Data Division, which is one of the required divisions in a program, is subdivided into sections. These are the File, Working–Storage, Common–Storage, Linkage, Communication, and Report Sections.

The File Section defines the structure of data files. Each file is defined by a file description entry and one or more record descriptions, or by a file description entry and one or more report description entries. Record descriptions are written immediately following the file description entry. When the file description specifies a file to be used as a Report Writer output file, no record description entries are permitted for that file. Report description entries appear in a separate section of the Data Division, the Report Section. The Working–Storage Section describes records and noncontiguous data items that are not part of external data files but are developed and processed internally. It also describes data items whose values are assigned in the source program and do not change during the execution of the object program. The Common–Storage Section describes records and noncontiguous data items that are to be shared by all programs that define a Common–Storage Section. The Linkage Section appears in the called program and describes data items that are to be referred to by the calling program and the called program. Its structure is the same as the Working–Storage Section. The Communication Section describes the data items in the source program that will serve as the interface between the MCS and the program. The Report Section describes the content and format of reports that are to be generated.

1.3.7.2.2. Data Division Structure

The following gives the general format of the sections in the Data Division, and defines the order of their presentation in the source program.


DATA DIVISION.
FILE SECTION.
file-description-entry [record-description-entry]...
sort-merge-file-description-entry { record-description-entry }...
WORKING-STORAGE SECTION.
77-level-description-entry
record-description-entry
COMMON-STORAGE SECTION.
77-level-description-entry
record-description-entry
LINKAGE SECTION.
77-level-description-entry
record-description-entry
COMMUNICATION SECTION.
communication-description-entry [record-description-entry]...
REPORT SECTION.
report-description-entry { report-group-description-entry }...

1.3.8. Procedure Division

1.3.8.1. General Description

The Procedure Division must be included in every COBOL source program. This division may contain declaratives and nondeclarative procedures.

1.3.8.1.1. Declaratives

Declarative sections must be grouped at the beginning of the Procedure Division preceded by the key word DECLARATIVES and followed by the key words END DECLARATIVES. (See 4.4.5, 5.4.7, 6.4.7, 8.3.5, and 11.3.1 for the USE statement.)

1.3.8.1.2. Procedures

A procedure is composed of a paragraph or group of successive paragraphs, or a section, or a group of successive sections within the Procedure Division. If one paragraph is in a section, then all paragraphs must be in sections. A procedure-name is a word used to refer to a paragraph or section in the source program in which it occurs. It consists of a paragraph-name (which may be qualified), or a section-name.

The end of the Procedure Division and the physical end of the program is that physical position in a COBOL source program after which no further procedures appear.

A section consists of a section header followed by zero, one, or more successive paragraphs. A section ends immediately before the next section or at the end of the Procedure Division or, in the declaratives portion of the Procedure Division, at the key words END DECLARATIVES.

A paragraph consists of a paragraph-name followed by a period and a space and by zero, one, or more successive sentences. A paragraph ends immediately before the next paragraph-name or section-name or at the end of the Procedure Division or, in the declaratives portion of the Procedure Division, at the key words END DECLARATIVES.

A sentence consists of one or more statements and is terminated by a period followed by a space.

A statement is a syntactically valid combination of words and symbols beginning with a COBOL verb.

The term “identifier” is defined as the word or words necessary to make unique reference to a data item.

1.3.8.1.3. Execution

Execution begins with the first statement of the Procedure Division, excluding declaratives. Statements are then executed in the order in which they are presented for compilation, except where the rules indicate some other order.

1.3.8.1.4. Procedure Division Structure

1.3.8.1.4.1. Procedure Division Header

The Procedure Division is identified by and must begin with the following header:

```
PROCEDURE DIVISION [ USING data-name-1 [ , data-name-2 ] . . . ]

  [ WITH ENTRY POINTS { procedure-name-1 } . . . ] .
```

1.3.8.1.4.2. Procedure Division Body

The body of the Procedure Division must conform to one of the following formats:

Format 1

```
[ DECLARATIVES.

{ section-name SECTION [ segment-number ] . declarative-sentence

[ paragraph-name. [ sentence ] . . . ] . . . } . . .

END DECLARATIVES. ]

{ section-name SECTION [ segment-number ] .

[ paragraph-name. [ sentence ] . . . ] . . . } . . .
```

Format 2

```
{ paragraph-name. [ sentence ] . . . } . . .
```

1.3.8.2. Statements and Sentences

There are three types of statements: conditional statements, compiler directing statements, and imperative statements.

There are three types of sentences: conditional sentences, compiler directing sentences, and imperative sentences.

1.3.8.2.1. Conditional Statements and Conditional Sentences

1.3.8.2.1.1. Definition of Conditional Statement

A conditional statement specifies that the truth value of a condition is to be determined and that the subsequent action of the object program depends on this truth value.

A conditional statement is one of the following:

1. An IF, SEARCH, or RETURN statement.
2. A READ statement that specifies the AT END or INVALID KEY phrase.
3. A WRITE statement that specifies the INVALID KEY or END–OF–PAGE phrase.
4. A START, REWRITE, or DELETE statement that specifies the INVALID KEY phrase.

5. An arithmetic statement (ADD, COMPUTE, DIVIDE, MULTIPLY, SUBTRACT) that specifies the SIZE ERROR phrase.
6. A RECEIVE statement that specifies a NO DATA phrase.
7. A STRING, UNSTRING, or CALL statement that specifies the ON OVERFLOW phrase.

1.3.8.2.1.2. Definition of Conditional Sentence

A conditional sentence is a conditional statement, optionally preceded by an imperative statement, terminated by a period followed by a space.

1.3.8.2.2. Compiler-Directing Statements and Compiler Directing Sentences

1.3.8.2.2.1. Definition of Compiler Directing Statement

A compiler directing statement consists of a compiler directing verb and its operands. The compiler directing verbs are COPY, and USE (see 10.2 for the COPY statement, and 4.4.5, 5.4.7, 6.4.7, 8.3.5, and 11.3.1 for the USE statement.). A compiler directing statement causes the compiler to take a specific action during compilation.

1.3.8.2.2.2. Definition of Compiler Directing Sentences

A compiler directing sentence is a single compiler directing statement terminated by a period followed by a space.

1.3.8.2.3. Imperative Statements and Imperative Sentences

1.3.8.2.3.1. Definition of Imperative Statement

An imperative statement indicates a specific unconditional action to be taken by the object program. An imperative statement is any statement that is neither a conditional statement nor a compiler directing statement. An imperative statement may consist of a sequence of imperative statements, each possibly separated from the next by a separator. The imperative verbs are:

ACCEPT	GENERATE	RELEASE
ADD (1)	GO	REWRITE (2)
ALTER	HOLD	SEND
CALL (3)	INITIATE	SET
CANCEL	INSPECT	SORT
CLOSE	LOCK	START (2)
COMPUTE (1)	MERGE	STOP
DELETE (2)	MOVE	STRING (3)
DISABLE	MULTIPLY (1)	SUBTRACT (1)
DISPLAY	OPEN	SUPPRESS
DIVIDE (1)	PERFORM	TERMINATE
ENABLE	PROCESS	UNLOCK
ENTER	READ (5)	UNSTRING (3)
EXIT	RECEIVE (4)	WRITE (6)
FREE		

1. Without the optional SIZE ERROR phrase.
2. Without the optional INVALID KEY phrase.
3. Without the optional ON OVERFLOW phrase.
4. Without the optional NO DATA phrase.
5. Without the optional AT END phrase or INVALID KEY phrase.
6. Without the optional INVALID KEY phrase or END–OF–PAGE phrase.

When *imperative-statement* appears in the general format of statements, *imperative-statement* refers to that sequence of consecutive imperative statements that must be ended by a period or an ELSE phrase associated with a previous IF statement or a WHEN phrase associated with a previous SEARCH statement.

1.3.8.2.3.2. Definition of Imperative Sentence

An imperative sentence is an imperative statement terminated by a period followed by a space.

1.3.8.2.4. Categories of Statements

Category	Verbs
Arithmetic	ADD COMPUTE DIVIDE INSPECT (TALLYING) MULTIPLY SUBTRACT
Asynchronous Processing	FREE HOLD LOCK PROCESS UNLOCK
Compiler Directing	COPY USE
Conditional	ADD (SIZE ERROR) CALL (OVERFLOW) COMPUTE (SIZE ERROR) DELETE (INVALID KEY) DIVIDE (SIZE ERROR) IF MULTIPLY (SIZE ERROR) READ (AT END or INVALID KEY) RECEIVE (NO DATA) RETURN (END) REWRITE (INVALID KEY) SEARCH START (INVALID KEY) STRING (OVERFLOW) SUBTRACT (SIZE ERROR) UNSTRING (OVERFLOW) WRITE (INVALID KEY or END–OF–PAGE)

Category	Verbs
Data Movement	ACCEPT (DATE, DAY, or TIME) ACCEPT MESSAGE COUNT INSPECT (REPLACING) MOVE STRING UNSTRING
Ending	STOP
Input–Output	ACCEPT (*identifier)* CLOSE DELETE DISABLE DISPLAY ENABLE OPEN READ RECEIVE REWRITE SEND START STOP (*literal*) WRITE
Interprogram Communicating	CALL CANCEL ENTER
Ordering	MERGE RELEASE RETURN SORT
Procedure Branching	ALTER CALL EXIT GO TO PERFORM
Report Writing	GENERATE INITIATE SUPPRESS TERMINATE
Table Handling	SEARCH SET

IF is a "verb" in the COBOL sense; it is recognized that it is not a verb in English.

1.3.8.2.4.1. Specific Statement Formats

The specific statement formats, together with a detailed discussion of the restrictions and limitations associated with each, appear in alphabetic sequence in the appropriate section of this manual. (See the Index to determine the page containing the discussion of a specific verb.)

1.3.9. Reference Format

1.3.9.1. General Description

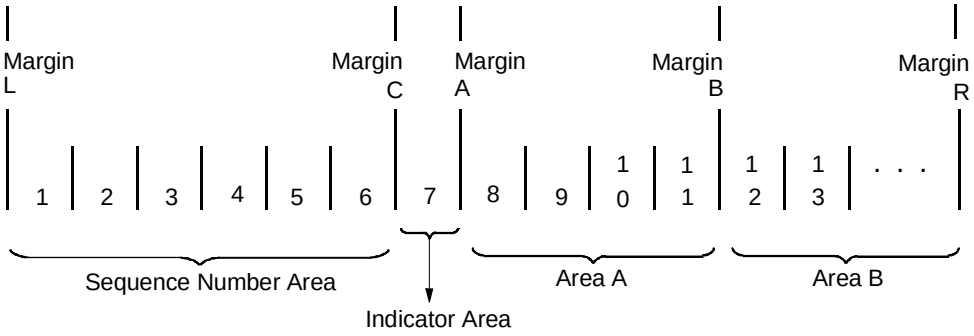
The reference format, which provides a standard method for describing COBOL source programs, is described in terms of character positions in a line on an input-output medium. A line consists of the character positions 1 through 72 of a source program card image. The COBOL compiler accepts source programs written in reference format and produces an output listing of the source program input in reference format.

The rules for spacing given in the discussion of the reference format take precedence over all other rules for spacing.

The divisions of a source program must be ordered as follows: the optional Control Division, then the Identification Division, then the Environment Division, then the Data Division, then the Procedure Division. Each division must be written according to the rules for the reference format.

1.3.9.2. Reference Format Representation

The reference format for a line is:



Margin L is immediately to the left of the leftmost character position of a line.

Margin C is between the 6th and 7th character positions of a line.

Margin A is between the 7th and 8th character positions of a line.

Margin B is between the 11th and 12th character positions of a line.

Margin R is immediately to the right of the 72nd character position of a line.

The sequence number area occupies six character positions (1-6), and is between margin L and margin C.

The indicator area is the 7th character position of a line.

Area A occupies character positions 8, 9, 10, and 11, and is between margin A and margin B.

Area B occupies 61 character positions. It begins immediately to the right of margin B and terminates immediately to the left of margin R.

1.3.9.2.1. Sequence Numbers

A sequence number, consisting of six digits in the sequence area, may be used to label a source program line.

1.3.9.2.2. Continuation of Lines

Whenever a sentence, entry, phrase, or clause requires more than one line, it may be continued by starting subsequent line(s) in area B. These subsequent lines are called the continuation line(s). The line being continued is called the continued line. Any word or literal may be broken in such a way that part of it appears on a continuation line by inserting a hyphen in the indicator area of a continuation line. If the line ends with a separator, the hyphen is not necessary.

A hyphen in the indicator area of a line indicates that the first nonblank character in area B of the current line is the successor of the last nonblank character of the preceding line without any intervening space. However, if the continued line contains a nonnumeric literal without closing quotation mark, the first nonblank character in area B on the continuation line must be a quotation mark, and the continuation starts with the character immediately after that quotation mark. All spaces at the end of the continued line are considered part of the literal. Area A of a continuation line must be blank.

If there is no hyphen in the indicator area of a line, it is assumed that the last character in the preceding line is followed by a space.

1.3.9.2.3. Blank Lines

A blank line is one that is blank from margin C to margin R, inclusive. A blank line can appear anywhere in the source program, except immediately preceding a continuation line. (See 1.3.9.2.2.)

1.3.9.3. Division, Section, Paragraph Formats

1.3.9.3.1. Division Header

The division header must start in area A.

1.3.9.3.2. Section Header

The section header must start in area A.

A section consists of paragraphs in the Control, Environment, Procedure Divisions, and Data Division entries in the Data Division.

1.3.9.3.3. Paragraph Header, Paragraph-Name, and Paragraph

A paragraph consists of a paragraph-name followed by a period and a space, and by zero, one, or more sentences, or a paragraph header followed by one or more entries. Comment entries may be included within a paragraph as indicated in 1.3.9.6. The paragraph header or paragraph-name starts in area A of any line following the first line of a division or a section.

The first sentence or entry in a paragraph begins either on the same line as the paragraph header or paragraph-name or in area B of the next nonblank line that is not a comment line. Successive sentences or entries either begin in area B of the same line as the preceding sentence or entry or in area B of the next nonblank line that is not a comment line.

When the sentences or entries of a paragraph require more than one line they may be continued as described in 1.3.9.2.2.

1.3.9.4. Data Division Entries

Each Data Division entry begins with a level indicator or a level–number, followed by a space, followed by its associated name (except in the Report Section), followed by a sequence of independent descriptive clauses. Each clause, except the last clause of an entry, may be terminated by either the separator semicolon or the separator comma. The last clause is always terminated by a period followed by a space.

There are two types of Data Division entries: those which begin with a level indicator and those which begin with a level–number.

A level indicator is any of the following: FD, SD, SA, RD, CD.

In those Data Division entries that begin with a level indicator, the level indicator begins in area A followed by a space and followed in area B with its associated name and appropriate descriptive information.

Those Data Division entries that begin with level-numbers are called data description entries.

A level-number has a value taken from the set of values 1 through 49, 66, 77, 88. Level-numbers in the range 1 through 9 may be written either as a single digit or as a zero followed by a significant digit. At least one space must separate a level-number from the word following the level-number.

In those data description entries that begin with a level-number 01 or 77, the level-number begins in area A followed by a space and followed in area B by its associated record-name or item-name and appropriate descriptive information.

Successive data description entries may have the same format as the first or may be indented according to level-number. The entries in the output listing need be indented only if the input is indented. Indentation does not affect the magnitude of a level-number.

When level-numbers are to be indented, each new level-number may begin any number of spaces to the right of margin A. The extent of indentation to the right is determined only by the width of the physical medium.

1.3.9.5. Declaratives

The key word DECLARATIVES and the key words END DECLARATIVES that precede and follow, respectively, the declaratives portion of the Procedure Division must appear on a line by itself. Each must begin in area A, and be followed by a period and a space.

1.3.9.6. Comment Lines

A comment line is any line with an asterisk in the continuation indicator area of the line. A comment line can appear as any line in a source program after the Identification Division header. Any combination of characters from the computer's character set may be included in area A and area B of that line. The asterisk and the characters in area A and area B will be produced on the listing but serve as documentation only. A special form of comment line represented by a stroke in the indicator area of the line causes page ejection prior to printing the comment.

Successive comment lines are allowed. Continuation of comment lines is permitted, except that each continuation line must contain an asterisk (*) in the indicator area.

Section 2
Nucleus

2.1. General

2.1.1. Function

The Nucleus provides a basic language capability for the internal processing of data within the basic structure of the five divisions of a program.

2.1.2. Level Characteristics

Nucleus Level 1 does not provide full COBOL facilities for qualification, punctuation characters, data-name formation, connectives, and figurative constants. Within the Procedure Division, Nucleus Level 1 provides limited capabilities for the ACCEPT, ADD, ALTER, DIVIDE, DISPLAY, IF, INSPECT, MOVE, MULTIPLY, PERFORM, and SUBTRACT statements and full capabilities for the ENTER, EXIT, GO, and STOP statements.

Nucleus Level 2 provides full facilities for qualification, punctuation characters, data-name formation, connectives, and figurative constants. Within the Procedure Division, Nucleus Level 2 provides full capabilities for the ACCEPT, ADD, ALTER, DIVIDE, DISPLAY, IF, INSPECT, MOVE, MULTIPLY, PERFORM, and SUBTRACT statements.

2.1.3. Level Restrictions on Overall Language

2.1.3.1. Format Notation

The separators, comma and semicolon, are not included in Level 1. The comma and semicolon are not boxed within the general formats of this manual in order to simplify the formats.

The separators, comma and semicolon, are included in Level 2.

2.1.3.2. Name Characteristics

All data-names must begin with an alphabetic character in Level 1. Qualification is not included; therefore, all data-names, paragraph-names, and text names must be unique in Level 1. In Level 2, data-names need not begin with an alphabetic character. The alphabetic characters may be positioned anywhere within the data-name. Qualification is permitted in Level 2; thus all data-names, condition-names, paragraph names, and text-names need not be unique.

2.1.3.3. Figurative Constants

The only figurative constants that may be used in Level 1 are: ZERO, SPACE, HIGH–VALUE, LOW–VALUE, and QUOTE. In Level 2, all the following figurative constants may be used: ZERO, ZEROS, ZEROES, SPACE, SPACES, HIGH–VALUE, HIGH–VALUES, LOW–VALUE, LOW–VALUES, QUOTE, QUOTES, and ALL literal.

2.1.3.4. Reference Format

In Level 1 a word or numeric literal cannot be broken in such a way that part of it appears on a continuation line. In Level 2, a word or numeric literal can be broken in such a way that part of it appears on a continuation line.

2.2. Control Division

2.2.1. General Description

The optional Control Division directs the compiler during compilation.

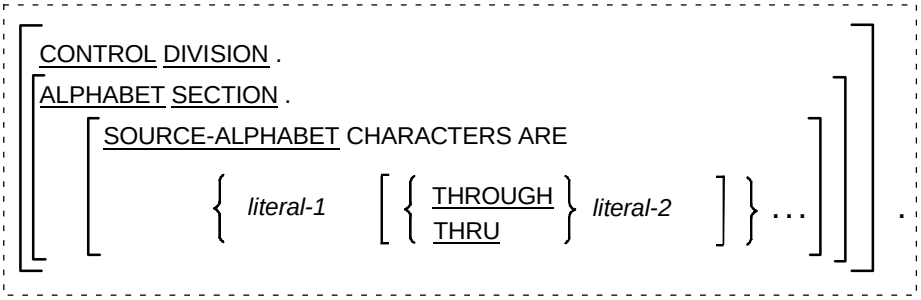
2.2.2. Organization

The Control Division contains the optional Alphabet Section.

2.2.2.1. Structure

The following general format of the Control Division defines the order of presentation in the source program.

2.2.2.2. General Format



2.2.2.3. Syntax Rule

The COPY statement may appear in the Alphabet Section, but the REPLACING option is not allowed.

2.2.3. The Alphabet Section

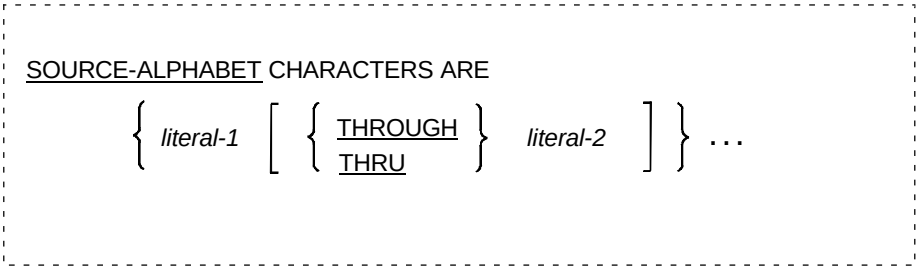
The Alphabet Section describes the extended COBOL character set.

2.2.3.1. The SOURCE–ALPHABET Clause

2.2.3.1.1. Function

The SOURCE–ALPHABET clause extends the allowable COBOL character set.

2.2.3.1.2. General Format



2.2.3.1.3. Syntax Rules

1. The SOURCE–ALPHABET clause, if present, must be the first clause in the Alphabet Section.
2. *Literal-1* and *literal-2* must be nonnumeric and one character in length.
3. A character must not be specified more than once in the SOURCE–ALPHABET clause.
4. *Literal-1* and *literal-2* must be specified in ascending order according to their ordinal position in the ISO 8859-1 character set.
5. *Literal-1* and *literal-2* must be selected from the following: a, b, ..., z, _ (underscore), and any character whose ordinal position is between 128 and 255 in the ISO 8859-1 character set. (See Appendix B.)

2.2.3.1.4. General Rules

1. When the THRU option is used, the set of contiguous characters from *literal-1* through *literal-2* is defined as the source-alphabet (or the extended COBOL character set).
2. The literals specified can only be used in the formulation of nonnumeric literals and user-defined words except for program-names, routine-names, file-names, and library-names.
3. Specification of any lowercase alphabetics will inhibit the folding of any lowercase characters to uppercase.

2.2.3.2. Example

Assume the user has defined C_1=128 and C_n=255 and declares:

```
SOURCE-ALPHABET CHARACTERS ARE
"a" THRU "z", "_", "C1" THRU "Cn"
```

The source-alphabet will consist of the standard COBOL character set, lowercase ASCII (a-z), underscore (_), and the 8-bit ISO 8859-1 characters whose ordinal position are 128 through 255.

2.3. Identification Division

2.3.1. General Description

The Identification Division must be included in every COBOL source program. This division identifies the source program and the resultant output listing. In addition, the user may include the date the program is written and such other information as desired under the paragraphs in the general format shown in the following paragraphs.

2.3.2. Organization

Paragraph headers identify the type of information contained in the paragraph. The name of the program must be given in the first paragraph, which is the PROGRAM–ID paragraph. The other paragraphs are optional and may be included in this division at the user's choice, in the order of presentation shown by the general format in 2.3.2.2.

2.3.2.1. Structure

The following is the general format of the paragraphs in the Identification Division and it defines the order of presentation in the source program. Subsections 2.3.3 and 2.3.4 define the PROGRAM–ID paragraph and the DATE–COMPILED paragraph. While the other paragraphs are not defined, each general format is formed in the same manner.

2.3.2.2. General Format

```
IDENTIFICATION  DIVISION.

PROGRAM-ID.  program-name.

[ AUTHOR.  [  comment-entry  ]  . . . ]

[  INSTALLATION.  [  comment-entry  ]  . . . ]

[ DATE-WRITTEN.  [  comment-entry  ]  . . . ]

[  DATE-COMPILED.  [  comment-entry  ]  . . . ]

[  SECURITY.  [  comment-entry  ]  . . .  ]
```

2.3.2.3. Syntax Rules

1. The Identification Division must begin with the reserved words IDENTIFICATION DIVISION followed by a period and a space.

2. The *comment-entry* may be any combination of the characters from the computer's character set. The continuation of the *comment-entry* by the use of the hyphen in the indicator area is not permitted; however, the *comment-entry* may be contained on one or more lines.

2.3.3. The PROGRAM–ID Paragraph

2.3.3.1. Function

The PROGRAM–ID paragraph gives the name by which a program is identified.

2.3.3.2. General Format

```
PROGRAM-ID.  program-name.
```

2.3.3.3. Syntax Rules

1. The *program-name* must conform to the rules for formation of a user-defined word.
2. Each *program-name* in a run unit must be unique in the first six characters and the last six characters.

2.3.3.4. General Rules

1. The PROGRAM–ID paragraph must contain the name of the program and must be present in every program.
2. The *program-name* identifies the source program and all listings pertaining to a particular program.

2.3.4. The DATE–COMPILED Paragraph

2.3.4.1. Function

The DATE–COMPILED paragraph provides the compilation date in the Identification Division source program listing.

2.3.4.2. General Format

```
DATE-COMPILED.  [  comment-entry  ]  . . .
```

2.3.4.3. Syntax Rules

1. The *comment-entry* may be any combination of the characters from the computer's character set. The continuation of the *comment-entry* by the use of the hyphen in the indicator area is not permitted; however, the *comment-entry* may be contained on one or more lines.

2.3.4.4. General Rules

1. The paragraph-name DATE–COMPILED causes the current date to be inserted during program compilation. If a DATE–COMPILED paragraph is present, it is replaced during compilation with a paragraph of the form:

```
DATE-COMPILED.  current date.
```

2.4. Environment Division

2.4.1. Configuration Section

2.4.1.1. The SOURCE–COMPUTER Paragraph

2.4.1.1.1. Function

The SOURCE–COMPUTER paragraph identifies the computer upon which the program is to be compiled.

2.4.1.1.2. General Format

SOURCE–COMPUTER. *computer-name.*

2.4.1.1.3. Syntax Rules

1. *Computer-name* is a system-name.

2.4.1.1.4. General Rules

1. The computer-name serves as documentation only. Any computer-name reserved word or any legal user-defined COBOL word can be specified. The following computer names are recommended:

   ```
   UNISYS-1100       UNISYS-1100EIS       UNISYS-1100-90

   UNISYS-2200
   ```

2.4.1.2. The OBJECT–COMPUTER Paragraph

2.4.1.2.1. Function

The OBJECT–COMPUTER paragraph identifies the computer on which the program is to be executed.

2.4.1.2.2. General Format

OBJECT-COMPUTER. *computer-name* [, MEMORY SIZE *integer* { WORDS / CHARACTERS / MODULES }]

[, PROGRAM COLLATING SEQUENCE IS *alphabet-name*] .

2.4.1.2.3. Syntax Rules

1. *Computer-name* is a system name.

2.4.1.2.4. General Rules

1. To better use the instruction repertoire of each computer, the ASCII COBOL compiler may generate different code based on the computer-name specified in the OBJECT–COMPUTER paragraph. It is recommended that the computer-name correspond to the system upon which the object program is to be run. Any computer-name reserved word or any legal user-defined COBOL word can be specified.
2. Unless the T-option is specified on the ACOB processor call statement, or the program contains USAGE DISPLAY–1 or COMPUTATIONAL–4 data items, the compiler generates the 1100/60 Extended Instruction Set (EIS) code when the computer-name specified in the OBJECT–COMPUTER paragraph is:

   ```
   UNIVAC-1100-60EIS     UNIVAC-1100-70EIS     SPERRY-SYSTEM-11
   SPERRY-1100EIS        UNISYS-1100EIS        UNIVAC-1100-90
   UNISYS-1100-90        UNISYS-2200
   ```

   The compiler can generate different EIS code depending on which computer-name is specified. To generate EIS code, Unisys recommends one of the following computer-names be used:

   ```
   UNISYS-1100EIS        UNISYS-1100-90        UNISYS-2200
   ```

   If you do not want to generate EIS code, Unisys recommends the computer-name UNISYS–1100 be specified.

3. The MEMORY SIZE clause is required if the data area of the object program is to be collected over a 65K boundary. *Integer* must then be greater than 65,536 words, two modules (one module is 32,768 words), or 262,144 characters (four characters is one word). The instruction area is limited to 65K. Working–Storage, Common–Storage, and the largest record area for any file or Saved-Area are each limited to 65,535 words.

   In the case of subprogramming, if the total collected data bank is over 65K, MEMORY SIZE should be specified in all main programs and subprograms. Otherwise, the results will be unpredictable.

4. If the PROGRAM COLLATING SEQUENCE clause is specified, the collating sequence associated with *alphabet-name* is used to determine the truth value of any nonnumeric DISPLAY comparisons:
   a. Explicitly specified in relation conditions. (See 2.6.2.1.1.)
   b. Explicitly specified in condition–name conditions. (See 2.6.2.1.3.)
   c. Implicitly specified by the presence of a CONTROL clause in a report description entry. (See 8.2.9.)
5. If the PROGRAM COLLATING SEQUENCE clause is not specified, the native collating sequence is used.

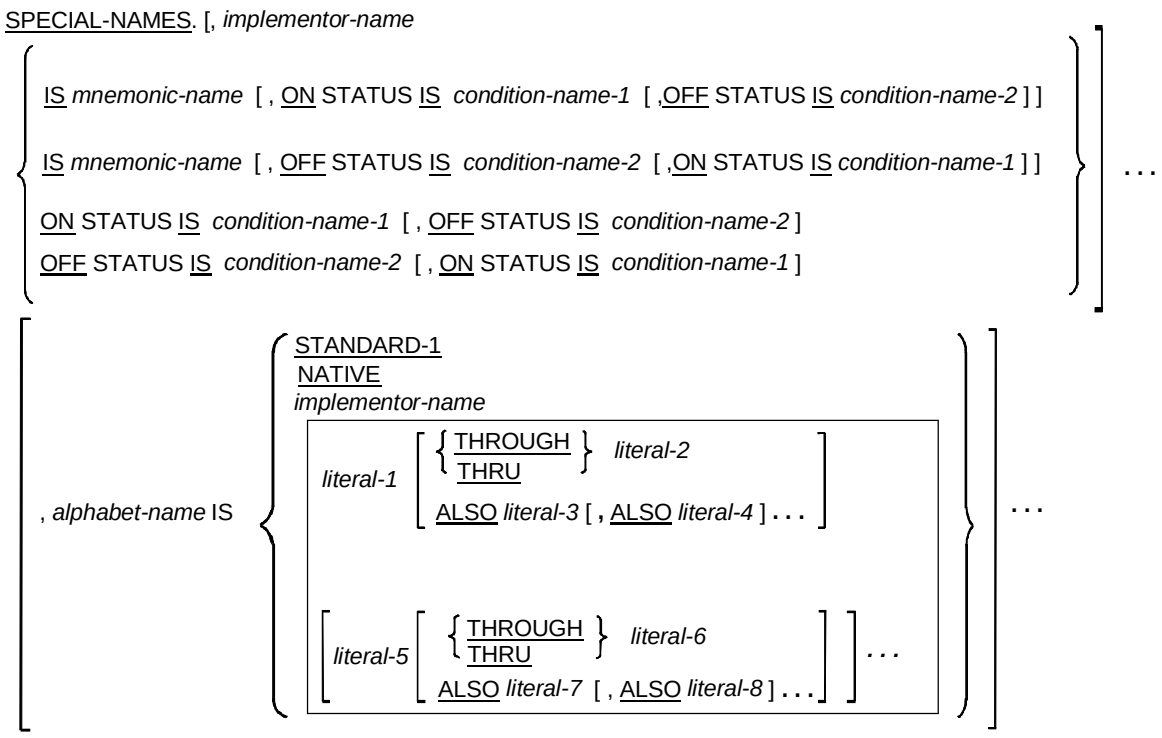
6. If the PROGRAM COLLATING SEQUENCE clause is specified, the program collating sequence is the collating sequence associated with the alphabet–name specified in that clause.
7. The PROGRAM COLLATING SEQUENCE clause is also applied to any nonnumeric DISPLAY merge or sort keys unless the COLLATING SEQUENCE phrase of the respective MERGE or SORT statement is specified. (See 7.4.1 for the MERGE statement, and 7.4.4 for the SORT statement.)
8. The PROGRAM COLLATING SEQUENCE clause applies only to the program in which it is specified.

2.4.1.3. The SPECIAL–NAMES Paragraph

2.4.1.3.1. Function

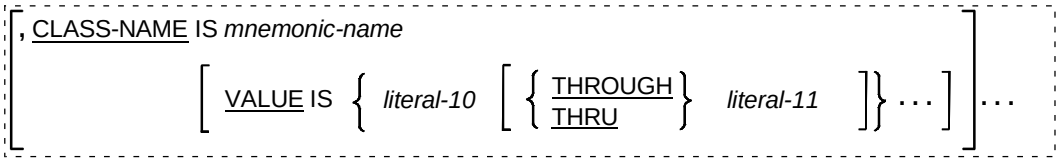
The SPECIAL–NAMES paragraph provides a means of relating implementor-names to user-specified mnemonic-names and of relating alphabet-names to character sets and collating sequences.

2.4.1.3.2. General Format



[, CURRENCY SIGN IS *literal-9*]

[, DECIMAL-POINT IS COMMA]



2.4.1.3.3. Syntax Rules

1. The allowable *implementor–names* are:

CARD-PUNCH	PAGE
CARD-READER	PRINTER
CONSOLE	SWITCH-*n* (where *n* = 1 - 24)

2. The literals specified in the literal phrase of the *alphabet-name* clause:

   a. If numeric, must be unsigned integers and must have a value within the range of one through the maximum number of characters in the native character set.

   b. If nonnumeric and associated with a THROUGH or ALSO phrase, must each be one character in length.

3. If the literal phrase of the *alphabet-name* clause is specified a given character must not be specified more than once in an *alphabet-name* clause.

4. The words THRU and THROUGH are equivalent.

5. In repetition, a comma may be used before *implementor-name.*

2.4.1.3.4. General Rules

1. If the *implementor-name* is not a switch, or a CLASS–NAME , the associated *mnemonic-name* may be used in the following statements:

   a. ACCEPT to specify CONSOLE, DATE–TIME, and CARD–READER

   b. DISPLAY to specify CONSOLE, PRINTER, and CARD–PUNCH

   c. WRITE and SEND to specify PAGE

2. If the *implementor-name* is a switch, at least one *condition-name* must be associated with it. The status of the switch is specified by condition-names and interrogated by testing the condition-names (see 2.6.2.1.2).

3. The *alphabet-name* clause provides a means for relating a name to a specified character code set and/or collating sequence. When *alphabet-name* is referenced in the PROGRAM COLLATING SEQUENCE clause (see 2.4.1.2) or the COLLATING SEQUENCE phrase of a SORT or MERGE statement (see 7.4.1 and 7.4.4), the *alphabet–name* clause specifies a collating sequence. When *alphabet-name* is referenced in a CODE–SET clause in a file description entry (see 4.3.3), the *alphabet-name* clause specifies a character code set.

   a. If the STANDARD–1 phrase is specified, the character code set or collating sequence identified is that defined in American National Standard Code for Information Interchange, X3.4–1968. Each character of the standard character set is associated with its corresponding character of the native character set. The NATIVE and STANDARD–1 character sets are equivalent.

   b. If the NATIVE phrase is specified, the native character code set or native collating sequence is used.

   c. If the literal phrase is specified, the *alphabet-name* may not be referenced in a CODE–SET clause (see 4.3.5). The collating sequence identified is that defined according to the following rules:

      Rule 1: The value of each literal specifies:

      1) The ordinal number of a character within the native character set, if the literal is numeric. This value must not exceed the value which represents the number of characters in the native character set.

      2) The actual character within the native character set, if the literal is nonnumeric. If the value of the nonnumeric literal contains multiple characters, each character in the literal, starting with the leftmost character, is assigned successive ascending positions in the collating sequence being specified.

      Rule 2: The order in which the literals appear in the *alphabet-name* clause specifies, in ascending sequence, the ordinal number of the character within the collating sequence being specified.

      Rule 3: Any characters within the native collating sequence, which are not explicitly specified in the literal phrase, assume a position, in the collating sequence being specified, greater than any of the explicitly specified characters. The relative order within the set of these unspecified characters is unchanged from the native collating sequence.

      Rule 4: If the THROUGH phrase is specified, the set of contiguous characters in the native character set beginning with the character specified by the value of *literal-1,* and ending with the character specified by the value of *literal-2,* is assigned a successive ascending position in the collating sequence being specified. In addition, the set of contiguous characters specified by a given THROUGH phrase may specify characters of the native character set in either ascending or descending sequence.

Rule 5: If the ALSO phrase is specified, the characters of the native character set specified by the value of *literal-1, literal-3, literal-4,* ..., are assigned to the same position in the collating sequence being specified.

4. The character that has the highest ordinal position in the program collating sequence specified is associated with the figurative constant HIGH–VALUE. If more than one character has the highest position in the program collating sequence, the last character specified is associated with the figurative constant HIGH–VALUE.
5. The character that has the lowest ordinal position in the program collating sequence specified is associated with the figurative constant LOW–VALUE. If more than one character has the lowest position in the program collating sequence, the first character specified is associated with the figurative constant LOW–VALUE.
6. The literal which appears in the CURRENCY SIGN IS *literal-9* clause is used in the PICTURE clause to represent the currency symbol. The literal is limited to a single character and must not be one of the following characters:

   a. numerals 0 through 9;

   b. alphabetic characters A, B, C, D, L, P, R, S, V, X, Z, or the space;

   c. special characters "*" "+" "-" "," "." ";" "(" ")" """ "/" "="

   If this clause is not present, only the currency sign is used in the PICTURE clause.

7. The clause DECIMAL–POINT IS COMMA means that the function of comma and period are exchanged in the character–string of the PICTURE clause and in numeric literals.
8. The CLASS–NAME clause allows for an extension to the class condition to allow specification of an alphabetic, numeric or source alphabet subset. The value *literal-10* may be any character belonging to the standard or source-alphabet character set and *literal-11* defines the inclusive range of characters which constitute the class identified by *mnemonic-name*. Class conditions NUMERIC and ALPHABETIC can not be redefined by the CLASS–NAME clause. If the VALUE clause is not specified, a SOURCE–ALPHABET clause must be specified in the CONTROL DIVISION and the characters specified by the literals in the SOURCE–ALPHABET clause define the exclusive set of characters of which the CLASS–NAME consists. The characters of the standard COBOL character set are not implicitly included. More than one CLASS–NAME clause is allowed if the VALUE clause is specified. Only one CLASS–NAME clause without the VALUE clause can be used.

An example of the use of the SPECIAL–NAMES feature is:

```
SPECIAL-NAMES.  CARD-READER IS READER.
```

This paragraph, specified in the Environment Division, would permit reference to the hardware device CARD–READER to be made in the Procedure Division in the following manner:

```
ACCEPT RATE-CHANGE FROM READER.
```

2.5. Data Division

2.5.1. Working–Storage Section

The Working–Storage Section is composed of the section header, followed by data description entries for noncontiguous data items and/or record description entries. Each Working–Storage Section record name and noncontiguous item name must be unique since it cannot be qualified. Subordinate data-names need not be unique if they can be made unique by qualification.

2.5.1.1. Noncontiguous Working–Storage

Items and constants in Working–Storage which bear no hierarchical relationship to one another need not be grouped into records, provided they do not need to be further subdivided. Instead, they are classified and defined as noncontiguous elementary items. Each of these items is defined in a separate data description entry which begins with the special level-number, 77.

The following data clauses are required in each data description entry:

a. level-number 77

b. data-name

c. the PICTURE clause or the USAGE IS INDEX, COMPUTATIONAL–1, COMPUTATIONAL–2, or LOCK clause.

Other data description clauses are optional and can be used to complete the description of the item if necessary.

2.5.1.2. Working–Storage Records

Data elements and constants in Working–Storage which bear a definite hierarchic relationship to one another must be grouped into records according to the rules for formation of record descriptions. All clauses which are used in record descriptions in the File Section can be used in record descriptions in the Working–Storage Section.

2.5.1.3. Initial Values

The initial value of any item in the Working–Storage Section except an index data item is specified by using the VALUE clause with the data item. The initial value of any index data item is unpredictable.

2.5.2. The Data Description—Complete Entry Skeleton

2.5.2.1. Function

A data description entry specifies the characteristics of a particular item of data.

2.5.2.2. General Format

Format 1

level-number { *data-name-1* / FILLER }

[; REDEFINES *data-name-2*]

[; { PICTURE / PIC } IS *character-string*]

[[USAGE IS] { COMPUTATIONAL / COMP / DISPLAY / COMPUTATIONAL-1 / COMP-2 / COMPUTATIONAL-2 / COMP-1 / DISP }]

[; [SIGN IS] { LEADING / TRAILING } [SEPARATE CHARACTER]]

[; { SYNCHRONIZED / SYNC } [{ LEFT / RIGHT }]]

[; { JUSTIFIED / JUST } RIGHT]

[; BLANK WHEN ZERO]

[; VALUE IS *literal*] .

Format 2

66 *data-name-1*; RENAMES *data-name-2* [{ THROUGH / THRU } *data-name-3*] .

Format 3

88 *condition-name* ; { VALUE IS / VALUES ARE } *literal-1* [{ THROUGH / THRU } *literal-2*]

[, *literal-3* [{ THROUGH / THRU } *literal-4*]] . . . .

2.5.2.3. Syntax Rules

1. In Level 1, the *level-number* in Format 1 may be any number from 01–10 or 77.
   In Level 2, the *level-number* in Format 1 may be any number from 01–49 or 77.
2. The clauses may be written in any order with two exceptions: the *data–name–1* or FILLER clause must immediately follow the *level–number;* the REDEFINES clause, when used, must immediately follow the *data–name–1* clause.
3. The PICTURE clause must be specified for every elementary item except an index data item, lock, or floating point data item, in which case use of this clause is prohibited.
4. The words THRU and THROUGH are equivalent.

2.5.2.4. General Rules

1. The clauses SYNCHRONIZED, PICTURE, JUSTIFIED, and BLANK WHEN ZERO must not be specified except for an elementary data item.
2. Format 3 is used for each *condition-name*. Each *condition-name* requires a separate entry with level-number 88. Format 3 contains the name of the condition and the value, values, or range of values associated with the *condition-name*. The c*ondition-name* entries for a particular conditional variable must follow the entry describing the item with which the *condition-name* is associated. A *condition-name* can be associated with any data description entry which contains a level-number except the following:
   a. Another condition-name.
   b. A level 66 item.
   c. A group containing items with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).
   d. An index data item (see 3.2.2).

2.5.3. The BLANK WHEN ZERO Clause

2.5.3.1. Function

The BLANK WHEN ZERO clause permits the blanking of an item when its value is zero.

2.5.3.2. General Format

```
BLANK WHEN ZERO
```

2.5.3.3. Syntax Rules

1. The BLANK WHEN ZERO clause can be used only for an elementary item whose PICTURE is specified as numeric or numeric edited. (See 2.5.7.)

2.5.3.4. General Rules

1. When the BLANK WHEN ZERO clause is used, the item will contain nothing but spaces if the value of the item is zero.
2. When the BLANK WHEN ZERO clause is used for an item whose PICTURE is numeric, the category of the item is considered to be numeric edited.

The following example shows the contents of variable data items following a move of zeros to the field:

PICTURE	BLANK WHEN ZERO SPECIFIED?	EDITED ITEM
$ZZZ9	No	$ 0
$ZZZ9	Yes	
****	No	****
9(4)	No	0000
9(4)	Yes	

2.5.4. The Data–Name or FILLER Clause

2.5.4.1. Function

A data-name specifies the name of the data being described. The word FILLER specifies an item of the logical record that cannot be referred to explicitly.

2.5.4.2. General Format

{ *data-name* / FILLER }

2.5.4.3. Syntax Rules

In the File, Working–Storage, Common Storage, Communication, and Linkage Sections, a *data-name* or the key word FILLER must be the first word following the level-number in each data description entry.

2.5.4.4. General Rules

The key word FILLER can name a data item in a record. Under no circumstances can a FILLER item be referred to explicitly. However, the key word FILLER may be used as a conditional variable because such use does not require explicit reference to the FILLER item, but to its value.

FILLER can be used in place of data-name to indicate a portion that is to contain no directly addressable data; for example, a space between two data items that are to be printed.

As an example, if the following were written:

```
01 EMPLOYEE-HISTORY.
     02 EMPLOYEE-ID.
         03 LIFE-NO PICTURE IS 9(6).
         03 FILLER PICTURE IS X(6) VALUE IS SPACE.
         03 NAME PICTURE IS A(24).
```

A nonaddressable 6-character field would be inserted into the record between LIFE–NO and NAME and preset to contain spaces.

The contents of that 6-character field can then be modified only by changing the contents of the group item EMPLOYEE–ID or the record EMPLOYEE–HISTORY.

2.5.5. The JUSTIFIED Clause

2.5.5.1. Function

The JUSTIFIED clause specifies nonstandard positioning of data within a receiving data item.

2.5.5.2. General Format

{ JUSTIFIED / JUST } RIGHT

2.5.5.3. Syntax Rules

1. The JUSTIFIED clause can be specified only at the elementary item level.
2. JUST is an abbreviation for JUSTIFIED.
3. The JUSTIFIED clause cannot be specified for any data item described as numeric or for which editing is specified.

2.5.5.4. General Rules

1. When a receiving data item is described with the JUSTIFIED clause and the sending data item is larger than the receiving data item, the leftmost characters are truncated. When the receiving data item is described with the JUSTIFIED clause and it is larger than the sending data item, the data is aligned at the rightmost character position in the data item with space fill for the leftmost character positions.

   For example, a field described as X(5) JUSTIFIED RIGHT would contain:

   ΔΔABC if the literal "ABC" were moved to it, or

   BCDEF if the literal "ABCDEF" were moved to it.

   A normally justified field would contain ABCΔΔ and ABCDE if the same literals were moved to it.

2. When the JUSTIFIED clause is omitted, the standard rules for aligning data within an elementary item apply. (See 1.3.3.3.6.)

2.5.6. Level–Number

2.5.6.1. Function

The level-number shows the hierarchy of data within a logical record. In addition, it is used to identify entries for working storage items, linkage items, condition-names, and the RENAMES clause.

2.5.6.2. General Format

level–number

2.5.6.3. Syntax Rules

1. A *level-number* is required as the first element in each data description entry.
2. Data description entries subordinate to an FD, SD, or CD entry must have *level-numbers* with the values 01 through 10 in Level 1, and 0 – 49, 66, or 88 in Level 2. (See 4.3.3 for FD, 7.3.2 for SD, and 13.2.2 for CD.)
3. Data description entries subordinate to an RD entry must have *level–numbers* with the value 01 through 10 in Level 1; 01– 49 in Level 2. (See 8.2.4 for RD.)
4. Data description entries in the Working–Storage Section, Common-Storage Section, and Linkage Section must have *level–numbers* with the values 01–10 or 77 in Level 1, and 01– 49, 66, 77, or 88 in Level 2.

2.5.6.4. General Rules

1. The level-number 01 identifies the first entry in each record description or a report group. (See 8.2.5.)
2. Special level-numbers have been assigned to certain entries where there is no real concept of level:
   a. Level-number 77 is assigned to identify noncontiguous working storage data items, noncontiguous linkage data items, and can be used only as described by Format 1 of the data description skeleton. (See 2.5.2.)
   b. Level-number 66 is assigned to identify RENAMES entries and can be used only as described in Format 2 of the data description skeleton. (See 2.5.2.)
   c. Level-number 88 is assigned to entries which define condition-names associated with a conditional variable and can be used only as described in Format 3 of the data description skeleton. (See 2.5.2.)
3. Multiple level 01 entries subordinate to any given level indicator, other than RD, represent implicit redefinitions of the same area.

2.5.7. The PICTURE Clause

2.5.7.1. Function

The PICTURE clause describes the general characteristics and editing requirements of an elementary item.

2.5.7.2. General Format

```
{ PICTURE }
{ PIC     }  IS character-string
```

2.5.7.3. Syntax Rules

1. A PICTURE clause can be specified only at the elementary item level.
2. A *character-string* consists of certain allowable combinations of characters in the COBOL character set used as symbols. The allowable combinations determine the category of the elementary item.
3. The maximum number of characters allowed in the *character-string* is 30.
4. The PICTURE clause must be specified for every elementary item except an index data item, lock, or floating-point data item, in which case use of this clause is prohibited.
5. PIC is an abbreviation for PICTURE.
6. The asterisk when used as the zero suppression symbol and the clause BLANK WHEN ZERO may not appear in the same entry.

2.5.7.4. General Rules

1. There are five categories of data that can be described with a PICTURE clause: alphabetic, numeric, alphanumeric, alphanumeric edited, and numeric edited.
2. To define an item as alphabetic:
   a. Its PICTURE *character-string* can only contain the symbols "A" or "B"; and
   b. Its contents when represented in standard data format must be any combination of the 26 letters of the Roman alphabet, 26 lowercase letters of the Roman alphabet, and the space from the COBOL character set.

   c. An alphabetic item may have only DISPLAY usage associated with it.

   AAA is an example of a valid alphabetic PICTURE string.
3. To define an item as numeric:
   a. Its PICTURE *character-strin*g can only contain the symbols "9", "P", "S", and "V". The number of digit positions that can be described by the PICTURE *character-string* must range from 1 to 18 inclusive; and

b. If unsigned, its contents when represented in standard data format must be a combination of the Arabic numerals "0", "1", "2", "3", "4", "5", "6", "7", "8", and "9"; if signed, the item may also contain a "+", "-", or other representation of an operational sign. (See 2.5.10.)

SPP9(4) and 9V99 are valid numeric PICTURE strings.

4. To define an item as exact binary:

a. When a PICTURE string contains at least one character "1", and no more than one operational sign "S" and no other symbols, it defines an exact binary data item. Exact binary data items must not have associated USAGE clauses.

b. The total allocated size of an exact binary field may not exceed 72 bits. If the operational sign is present the first bit will be treated as a sign indicator with the remainder of the field assumed to be the absolute value. If the value is negative, the sign bit is 1 and the remainder of the field is 1's complement of the magnitude of the value. Thus, a PICTURE string S1(4) will have one sign bit and 3 magnitude bits.

S1(3) and 1111 are valid exact binary strings.

5. To define an item as alphanumeric:

a. Its PICTURE *character-string* is restricted to certain combinations of the symbols "A", "X", "9", and the item is treated as if the *character-string* contained all X's. A PICTURE *character-string* which contains all A's or all 9's does not define an alphanumeric item; and

b. Its contents when represented in standard data format are allowable characters in the computer's character set.

9XX9, AAXX, and A9(3) are all valid alphanumeric picture strings. Furthermore all three are equivalent and are treated as if X(4) had been written. An alphanumeric item has DISPLAY usage.

6. To define an item as alphanumeric edited:

a. Its PICTURE *character-string* is restricted to certain combinations of the following symbols: "A", "X", "9", "B", "0" , and "/" ; and

1) The character-string must contain at least one "B" and at least one "X" or at least one "0" (zero) and at least one "X" or at least one "/" (stroke) and at least one "X" ; or

2) The *character-string* must contain at least one "0" (zero) and at least one "A" or at least one "/" (stroke) and at least one "A"; and

b. The contents when represented in standard data format are allowable characters in the computer's character set.

An alphanumeric-edited PICTURE string must not cause more than 255 characters to be allocated.

XBBX and XB(15)X0(10) are valid alphanumeric-edited PICTURE strings. An alphanumeric-edited field has DISPLAY usage.

7. To define an item as numeric edited:

   a. Its PICTURE character–string is restricted to certain combinations of the symbols "B", "/", "P", "V", "Z", "0", "9", ",", ".", "*", "+", "-", "CR", "DB", and the currency symbol. The allowable combinations are determined from the order of precedence of symbols and the editing rules; and

      1) The number of digit positions that can be represented in the PICTURE *character-string* must range from 1 to 18 inclusive; and

      2) The *character-string* must contain at least one "0", "B", "/", "Z", "*", "+", ",", ".", "-", "CR", "DB", or currency symbol.

   b. The contents of the character positions of these symbols that are allowed to represent a digit in standard data format, must be one of the numerals.

A numeric-edited item must have DISPLAY usage associated with it.

$$$, $$9.99CR and –ZZZ are valid numeric-edited picture strings.

8. The size of an elementary item, where size means the number of character positions occupied by the elementary item in standard data format, is determined by the number of allowable symbols that represent character positions. An integer which is enclosed in parentheses following the symbols "A", ",", "X", "9", "P", "Z", "*", "B", "/", "0", "+", "-", or the currency symbol indicates the number of consecutive occurrences of the symbol. Note that the following symbols may appear only once in a given PICTURE: "S", "V", ".", "CR", and "DB".

The following examples illustrate the use of replication symbols:

AAAAAA is equivalent to A(6)

PPPP99999 is equivalent to P(4)9(5)

S999V9999 is equivalent to S999V9(4)

9. The functions of the symbols used to describe an elementary item are explained as follows:

   A

   Each "A" in the *character-string* represents a character position which can contain only a letter of the alphabet or a space.

   B

   Each "B" in the *character-string* represents a character position into which the space character will be inserted.

P

Each "P" indicates an assumed decimal scaling position and is used to specify the location of an assumed decimal point when the point is not within the number that appears in the data item. The scaling position character "P" is not counted in the size of the data item. Scaling position characters are counted in determining the maximum number of digit positions (18) in numeric-edited items or numeric items. The scaling position character "P" can appear only to the left or right as a continuous string of P's within a PICTURE description; since the scaling position character "P" implies an assumed decimal point (to the left of P's if P's are leftmost PICTURE characters and to the right if P's are rightmost PICTURE characters), the assumed decimal point symbol "V" is redundant as either the leftmost or rightmost character within such a PICTURE description. The character "P" and the insertion character "." (period) cannot both occur in the same PICTURE character-string. If, in any operation involving conversion of data from one form of internal representation to another, the data item being converted is described with the PICTURE character "P", each digit position described by a "P" is considered to contain the value zero, and the size of the data item is considered to include the digit positions so described.

The following example:

```
02 FLD–1 PIC SPPP99.
02 FLD–2 PIC P999.
02 FLD–3 PIC 999PP.
02 FLD–4 PIC 9(5)V9(5).
          .
          .
          .
MOVE .00123 TO FLD–1, FLD–2.
MOVE 12345 TO FLD–3.
ADD FLD–1, FLD–2, FLD–3 GIVING FLD–4.
```

would produce the following results:

```
FLD–1 would contain 23
FLD–2 would contain 012
FLD–3 would contain 123
FLD–4 would contain 12300.00143
    (.00023 + .0012 + 12300)
```

S

The letter "S" is used in a *character-string* to indicate the presence, but neither the representation nor, necessarily, the position, of an operational sign. It must be written as the leftmost character in the PICTURE. The "S" is not counted in determining the size (in terms of standard data format characters) of the elementary item unless the entry is subject to a SIGN clause which specifies the optional SEPARATE CHARACTER phrase. (See 2.5.10.)

The operational sign is used to indicate whether the value of a data item is positive or negative. When a data item is described with an operational sign, the contents of the data item can be signed positive, signed negative, or unsigned values. Unsigned values are treated conceptually as positive.

The default storage representation of the operational sign of a decimal field is an overpunch on the low-order character. The storage representation of the operational sign of an exact binary or computational field is the high-order bit (a 1's complement form is used for negative values). The status of an operational sign can be printed by using edit sign control symbols. Table 2-1 defines the octal value of the internal representation of the operational sign and the character that will be printed or displayed prior to editing.

Table 2–1. Internal Representation of Overpunched Operational Signs

	Unsigned			Signed Positive			Signed Negative		
Digit	ASCII	Fieldata	Character	ASCII	Fieldata	Character	ASCII	Fieldata	Character
0	060	60	0	173, 77	54	{ , ?	175, 41	55	} , !
1	061	61	1	101	06	A	112	17	J
2	062	62	2	102	07	B	113	20	K
3	063	63	3	103	10	C	114	21	L
4	064	64	4	104	11	D	115	22	M
5	065	65	5	105	12	E	116	23	N
6	066	66	6	106	13	F	117	24	O
7	067	67	7	107	14	G	120	25	P
8	070	70	8	110	15	H	121	26	Q
9	071	71	9	111	16	I	122	27	R

Note that there are two internal sign representations for ASCII: signed positive and signed negative. Both are valid and both are processed correctly.

The ASCII graphic symbols "?" and "{" and FD graphic "?" represent overpunched positive "0".

The ASCII graphic symbols "!" and "}" and FD graphic "!" represent overpunched negative "0".

The presence or absence of an "S" in the PICTURE of a numeric item has the following effects:

1) If the "S" is written for a decimal field, an unsigned value is always handled correctly. The converse is not always true. An unsigned field is always handled correctly when it is modified by direct references to its elementary level. If, however, an unsigned item is contained in a group item which is read into, moved to, or redefined it is possible that the sign position may be incorrect and thereby impact later calculations which reference the item.

2) The result of an arithmetic or MOVE statement is considered signed if any input operand is signed, or if the arithmetic verb is SUBTRACT. When a signed result is stored in an unsigned item, the absolute value is computed and stored.

3) For an exact binary item, it determines whether the high-order bit is treated as a sign bit. Thus, an exact binary PICTURE character-string S1111 can contain a value ranging from -7 to +7 while the PICTURE character-string 1111 can contain values ranging 0 to 15.

V

The "V" is used in a *character-string* to indicate the location of the assumed decimal point and may only appear once in a *character-string*. The "V" does not represent a character position and therefore is not counted in the size of the elementary item. When the assumed decimal point is to the right of the rightmost symbol in the string the "V" is redundant.

X

Each "X" in the *character-string* is used to represent a character position which contains any allowable character from the computer's character set.

Z

Each "Z" in a *character-string* may only be used to represent the leftmost leading numeric character positions which will be replaced by a space character when the contents of that character position is zero. Each "Z" is counted in the size of the item.

1

Each "1" in the *character-string* represents a bit position in an exact binary PICTURE and is counted in the size of the item. An exact binary PICTURE can only contain the symbols "S" and "1". A maximum of 72 bit positions is allowed for signed strings. A maximum of 71 bit positions is allowed if unsigned.

9

Each "9" in the *character-string* represents a character position which contains a numeral and is counted in the size of the item.

0

Each "0" (zero) in the *character-string* represents a character position into which the numeral zero will be inserted. The "0" is counted in the size of the item.

/

Each "/" (stroke) in the *character-string* represents a character position into which the stroke character will be inserted. The "/" is counted in the size of the item.

,

Each "," (comma) in the *character-string* represents a character position into which the character "," will be inserted. This character position is counted in the size of the item. The insertion character "," must not be the last character in the PICTURE *character-string*.

.

When the character "." (period) appears in the *character-string* it is an editing symbol which represents the decimal point for alignment purposes and in addition, represents a character position into which the character "." will be inserted. The character "." is counted in the size of the item. For a given program the functions of the period and comma are exchanged if the clause DECIMAL–POINT IS COMMA is stated in the SPECIAL–NAMES paragraph. In this exchange the rules for the period apply to the comma and the rules for the comma apply to the period wherever they appear in a PICTURE clause. The insertion character "." must not be the last character in the PICTURE *character-string*.

+, -, CR, DB

These symbols are used as editing sign control symbols. When used, they represent the character position into which the editing sign control symbol will be placed. The symbols are mutually exclusive in any one *character-string* and each character used in the symbol is counted in determining the size of the data item.

*

Each "*" (asterisk) in the *character-string* represents a leading numeric character position into which an asterisk will be placed when the contents of that position is zero. Each "*" is counted in the size of the item.

cs

The currency symbol in the *character-string* represents a character position into which a currency symbol is to be placed. The currency symbol in a character–string is represented by either the currency sign or by the single character specified in the CURRENCY SIGN clause in the SPECIAL–NAMES paragraph. The currency symbol is counted in the size of the item.

The alternate currency sign can be any valid character except the digits "0" through "9", the alphabetic characters "A", "B", "C", "D", "L", "P", "R", "S", "V", "X", "Z", or space, and the special characters "=", "*", "+", "-", ",", ".", ";", "(", ")", " " ", "/".

The "$" is used in the following discussion to indicate either the actual "$" or the alternate specified in the CURRENCY SIGN clause.

2.5.7.5. Editing Rules

1. There are two general methods of performing editing in the PICTURE clause: insertion, and suppression and replacement. There are four types of insertion editing available:

   a. Simple insertion

   b. Special insertion

   c. Fixed insertion

   d. Floating insertion

   There are two types of suppression and replacement editing:

   a. Zero suppression and replacement with spaces

   b. Zero suppression and replacement with asterisks

2. The type of editing which may be performed upon an item is dependent upon the category to which the item belongs. The table that follows specifies which type of editing may be performed upon a given category.

Category	Type of Editing
Alphabetic	Simple insertion "B" only
Numeric	None
Alphanumeric	None
Alphanumeric Edited	Simple insertion "0", "B" and "/"
Numeric Edited	All, subject to rules in rule 3

3. Floating insertion editing and editing by zero suppression and replacement are mutually exclusive in a PICTURE clause. Only one type of replacement may be used with zero suppression in a PICTURE clause.

4. Simple Insertion Editing. The "," (comma), "B" (space), "0" (zero), and "/" (stroke) are used as the insertion characters. The insertion characters are counted in the size of the item and represent the position in the item into which the character will be inserted.

   The only circumstance when simple insertion symbols will not appear in a numeric–edited field is when simple insertion characters are either embedded in or preceded on the left by suppression and replacement of floating insertion symbols. In this case the simple insertion characters are suppressed and replaced if the digit position to their left is suppressed and replaced.

   For example, a numeric-edited field defined as:

   ```
   $***,***.99
   ```

   will contain:

   ```
   $**1,000.00
   ```

   if the literal 1000 is moved to it or:

   ```
   $****100.00
   ```

   if the literal 100 is moved to it.

5. Special Insertion Editing. The "." (period) is used as the insertion character. In addition to being an insertion character it also represents the decimal point for alignment purposes. The insertion character used for the actual decimal point is counted in the size of the item. The use of the assumed decimal point, represented by the symbol "V" and the actual decimal point, represented by the insertion character, in the same PICTURE character-string is disallowed. The result of special insertion editing is the appearance of the insertion character in the item in the same position as shown in the *character-string*.

   The period (or the comma if DECIMAL–POINT IS COMMA is specified in the SPECIAL–NAMES paragraph) is the only special insertion symbol.

The result of special insertion editing is the appearance of the insertion character in the item in the same position as shown in the *character-string*. The only exception is if a PICTURE string uses only floating insertion symbols ("+", "-", or currency sign) or the suppression/replacement symbol "Z" to represent the capacity of an edited field and the value being edited to zero, the entire item, including the decimal point, will contain spaces.

For example, moving zero to a field defined by the picture string $$$, $$$. $$ will result in 10 spaces. The effect of moving zero to a field defined by *** , *** . ** is the result *******.**.

6. Fixed Insertion Editing . The currency symbol and the editing sign control symbols, "+", "-", "CR", "DB", are the insertion characters. Only one currency symbol and only one of the editing sign control symbols can be used in a given PICTURE *character-string*. When the symbols "CR" or "DB" are used they represent two character positions in determining the size of the item and they must represent the rightmost character positions that are counted in the size of the item. The symbol "+" or "-", when used, must be either the leftmost or rightmost character position to be counted in the size of the item. The currency symbol must be the leftmost character position to be counted in the size of the item except that it can be preceded by either a "+" or a "-" symbol. Fixed insertion editing results in the insertion character occupying the same character position in the edited item as it occupied in the PICTURE *character-string*. Editing sign control symbols produce the following results depending upon the value of the data item:

Editing Symbol in Picture Character-String	Result	
	Data Item Positive or Zero	**Data Item Negative**
+	+	-
-	space	-
CR	2 spaces	CR
DB	2 spaces	DB

The multiple consecutive occurrence of the currency sign, the "+", or the "-" implies floating insertion editing and must abide by the rules for that type of insertion editing. No picture may specify two separate types of sign control editing.

7. Floating Insertion Editing. The currency symbol and editing sign control symbols "+" or "-" are the floating insertion characters and as such are mutually exclusive in a given PICTURE *character-string*.

   Floating insertion editing is indicated in a PICTURE *character-string* by using a string of at least two of the floating insertion characters. This string of floating insertion characters may contain any of the fixed insertion symbols or have fixed insertion characters immediately to the right of this string. These simple insertion characters are part of the floating string.

The leftmost character of the floating insertion string represents the leftmost limit of the floating symbol in the data item. The rightmost character of the floating string represents the rightmost limit of the floating symbols in the data item.

The second floating character from the left represents the leftmost limit of the numeric data that can be stored in the data item. Nonzero numeric data may replace all the characters at or to the right of this limit.

In a PICTURE *character-string*, there are only two ways of representing floating insertion editing. One way is to represent any or all of the leading numeric character positions on the left of the decimal point by the insertion character. The other way is to represent all of the numeric character positions in the PICTURE *character-string* by the insertion character.

In other words, a numeric edited picture string which represents any digit positions to the right of a decimal point indicator with the symbol "9" must represent all fractional digits with the symbol "9".

If the insertion characters are only to the left of the decimal point in the PICTURE *character-string*, the result is that a single floating insertion character will be placed into the character position immediately preceding either the decimal point or the first nonzero digit in the data represented by the insertion symbol string, whichever is farther to the left in the PICTURE *character-string*. The character positions preceding the insertion character are replaced with spaces.

For example:

```
$$$,$$$.99
```

will contain:

```
$10,000.00
 $1,000.00
   $100.00
      $.01
```

if 10000, 1000, 100 and .01, respectively, are moved to the edited field.

If all numeric character positions in the PICTURE character-string are represented by the insertion character, the result depends upon the value of the data. If the value is zero the entire data item will contain spaces. If the value is not zero, the result is the same as when the insertion character is only to the left of the decimal point.

For example, a field defined by the picture string:

```
++,+++.++
```

will contain:

```
 +1,000.00
   +100.00
     +1.00

      +.01
```

if 1000, 100, 1, 0, and .01, respectively, are moved to the edited field.

To avoid truncation, the minimum size of the PICTURE *character-string* for the receiving data item must be the number of characters in the sending data item, plus the number of nonfloating insertion characters being edited into the receiving data item, plus one for the floating insertion character.

In a given PICTURE string, floating insertion and fixed insertion of the same symbol are disallowed. Also illegal is the specification of more than one type of floating insertion editing, or a mixture of floating insertion and suppression and replacement editing. In addition, no PICTURE may contain more than one type of sign control editing.

8. Zero Suppression Editing. The suppression of leading zeroes in numeric character positions is indicated by the use of the alphabetic character "Z" or the asterisk (*) as suppression symbols in a PICTURE *character-string*. These symbols are mutually exclusive in a given PICTURE character–string. Each suppression symbol is counted in determining the size of the item. If "Z" is used the replacement character will be the space and if the asterisk is used, the replacement character will be "*".

   Zero suppression and replacement is indicated in a PICTURE *character-string* by using a string of one or more of the allowable symbols to represent leading numeric character positions which are to be replaced when the associated character position in the data contains a zero. Any of the simple insertion characters embedded in the string of symbols or to the immediate right of this string are part of the string.

   In a PICTURE *character-string*, there are only two ways of representing zero suppression. One way is to represent any or all of the leading numeric character positions to the left of the decimal point by suppression symbols. The other way is to represent all of the numeric character positions in the PICTURE *character-string* by suppression symbols.

   If the suppression symbols appear only to the left of the decimal point, any leading zero in the data which corresponds to a symbol in the string is replaced by the replacement character. Suppression terminates at the first nonzero digit in the data represented by the suppression symbol string or at the decimal point, whichever is encountered first.

If all numeric character positions in the PICTURE *character-string* are represented by suppression symbols and the value of the data is not zero the result is the same as if the suppression characters were only to the left of the decimal point. If the value is zero and the suppression symbol is "Z", the entire data item will be spaces. If the value is zero and the suppression symbol is "*", the data item will be all "*" except for the actual decimal point.

9. The symbols "+", "-", "*", "Z", and the currency symbol, when used as floating replacement characters, are mutually exclusive within a given *character-string*.

2.5.7.6. Precedence Rules

Table 2-2 shows the order of precedence when using characters as symbols in a *character-string*. An "X" at an intersection indicates that the symbols at the top of the column may precede, in a given *character-string*, the symbols at the left of the row. Arguments appearing in braces indicate that the symbols are mutually exclusive. The currency symbol is indicated by the symbol "cs".

At least one of the symbols "A", "X", "Z", "9" or "*", or at least two of the symbols "+", "-" or "cs" must be present in a PICTURE string.

Table 2–2. PICTURE Character Precedence Chart

Second Symbol \ First Symbol		Non-Floating Insertion Symbols									Floating Insertion Symbols						Other Symbols						
		B	0	/	,	.	{+ –}	{+ –}	{CR DB}	CS	{Z *}	{Z *}	{+ –}	{+ –}	CS	CS	9	A X	S	V	P	P	1
Non-Floating Insertion Symbols	B	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	0	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	/	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X		X		X	
	,	X	X	X	X	X	X			X	X	X	X	X	X	X	X			X		X	
	.	X	X	X	X		X			X	X		X		X		X						
	{+ –}																						
	{+ –}	X	X	X	X	X				X	X	X			X	X	X			X	X	X	
	{CR DB}	X	X	X	X	X				X	X	X			X	X	X			X	X	X	
	CS						X																
Floating Insertion Symbols	{Z *}	X	X	X	X		X			X	X												
	{Z *}	X	X	X	X	X	X			X	X	X								X		X	
	{+ –}	X	X	X	X					X			X										
	{+ –}	X	X	X	X	X				X			X	X						X		X	
	CS	X	X	X	X		X								X								
	CS	X	X	X	X	X	X								X	X				X		X	
Other Symbols	9	X	X	X	X		X			X	X		X		X		X	X	X	X		X	
	A X	X	X	X													X	X					
	S																						
	V	X	X	X	X		X			X	X		X		X		X		X		X		
	P	X	X	X			X			X	X		X		X		X		X		X		
	P						X			X									X	X		X	
	1																		X				X

Nonfloating insertion symbols "+" and "-", floating insertion symbols "Z", "*", "+", "-", and "CS", and other symbol "P" appear twice in Table 2-2. The leftmost column and uppermost row for each symbol represents its use to the left of the decimal point position. The second appearance of the symbol in the chart represents its use to the right of the decimal point position.

Table 2-3 and Table 2-4 summarize the nonediting and editing picture strings.

Table 2-3. Examples of Nonediting PICTURE Strings

If Picture is:	and Field in Main Storage is:	Then Field Will be Used as:	and Class is:
99999	56789	56789	Numeric
9(3)V9(2)	56789	567.89	Numeric
S999V99	(-)56789	(-)567.89	Numeric
SV9(5)	(+)56789	(+).56789	Numeric
S111	binary 011	(+)3	Exact Binary Numeric
XXXXX	ABCDE	ABCDE	Alphanumeric
AAAAA	BCDEF	BCDEF	Alphabetic
X(5)	12345	12345	Alphanumeric
99X99	67.89	67.89	Alphanumeric
99AA9	23FG4	23FG4	Alphanumeric
A(2)X(3)	CD2M3	CD2M3	Alphanumeric
99P(3)	34	34000	Numeric
P(3)99	34	.00034	Numeric
P(3)9(2)	34	.00034	Numeric

Note: *The "+" and "-" indicate the operational sign status of a field in main storage.*

Table 2-4. Examples of Editing PICTURE Strings

Source Area		Receiving Area	
Picture	**Field as in Storage**	**Picture**	**Edited Field**
9(5)	45678	$ZZ,ZZ9.99	$45,678.00
9(3)V99	45678	$ZZ,ZZ9.99	$ 456.78
9(3)V99	00067	+ZZ,ZZZ.ZZ	+ .67
9(3)V99	00004	+ZZ,ZZZ.ZZ	+ .04
9(5)	00000	ZZ,ZZZ.ZZ	
9(5)	00000	**,***.**	******.**
S9(4)	(-)5678	$$,$$$.99BCR	$5,678.00 CR
S9PP	(-)3	$$,$$$.99DB	$300.00DB
SPP99	(+)23	+++.9999	+.0023
SPP99	(+)23	+++.++++	+.0023
S9(4)	(-)1234	- -900PP	-1200
S9(3)99	(+)56789	999B999B99-	000 567 89
999	567	9/9/9	5/6/7
AAA	XYZ	XBXBXB00	X Y Z 00
X(5)	12345	X0X0X0XX	10203045
X(5)	12345	XXXBB	123
9(5)	56789	$***.00	$789.00

Note: *The "+" and "-" indicate the status of the operational sign of the numeric value.*

2.5.8. The REDEFINES Clause

2.5.8.1. Function

The REDEFINES clause allows the same computer storage area to be described by different data description entries.

2.5.8.2. General Format

level–number data–name–1; REDEFINES *data–name–2*

Note: *Level-number, data-name-1 and the semicolon are shown in this format to improve clarity. Level-number and data-name-1 are not part of the REDEFINES clause.*

2.5.8.3. Syntax Rules

1. The REDEFINES clause, when specified, must immediately follow *data-name-1.*
2. The level-numbers of *data-name-1* and *data-name-2* must be identical, but must not be 66 or 88.
3. This clause must not be used in level 01 entries in the File Section. (See 4.3.6, General Rule 2.)
4. This clause must not be used in level 01 entries in the Communication Section.
5. The data description entry for *data-name-2* cannot contain a REDEFINES clause. In Level 1, *data-name-2* cannot be subordinate to an entry which contains a REDEFINES clause. In Level 2, *data-name-2* may be subordinate to an entry which contain a REDEFINES clause. The data description entry for *data-name-2* cannot contain an OCCURS clause. However, *data-name-2* may be subordinate to an item whose data description entry contains an OCCURS clause. In this case, the reference to *data-name-2* in the REDEFINES clause may not be subscripted or indexed. Neither the original definition nor the redefinition can include an item whose size is variable as defined in the OCCURS clause. (See 3.2.1, The OCCURS Clause.)
6. No entry having a *level-number* numerically lower than the *level-number* of *data-name-2* and *data-name-1* may occur between the data description entries of *data-name-2* and *data-name-1.*

2.5.8.4. General Rules

1. Redefinition starts at *data-name-2* and ends when a level-number less than or equal to that of *data-name-2* is encountered.
2. When the level–number of *data-name-1* is other than 01, it must specify the same number of character positions that the data item referenced by *data-name-2* contains. It is important to observe that the REDEFINES clause specifies the redefinition of a storage area, not of the data items occupying the area.

3. Multiple redefinitions of the same character positions are permitted. The entries giving the new descriptions of the character positions must follow the entries defining the area being redefined, without intervening entries that define new character positions. Multiple redefinitions of the same character positions must all use the data-name of the entry that originally defined the area.
4. The entries giving the new description of the character positions must not contain any VALUE clauses, except in condition-name entries.
5. Multiple level 01 entries subordinate to any given level indicator represent implicit redefinitions of the same area.
6. If *data-name-2* is synchronized, its unused bit positions are included in the storage allocation of *data-name-1.*
7. REDEFINES clauses may be included in RENAMES areas so long as the start of the RENAMES area precedes the terminating field of the RENAMES area both in the record description and in the storage allocation.

2.5.9. The RENAMES Clause

2.5.9.1. Function

The RENAMES clause permits alternative, possibly overlapping, groupings of elementary items.

The RENAMES data item permits an item or items established by a record description entry to be assigned a new name. Unlike REDEFINES, this clause does not redefine existing data descriptions but merely allows data to be accessed and grouped under alternative names while maintaining the previously defined data description.

2.5.9.2. General Format

66 *data-name-1*; RENAMES *data-name-2* [{ THROUGH / THRU } *data-name-3*] .

Note: *Level-number 66, data-name-1, and the semicolon are shown in the previous format to improve clarity. Level-number and data-name-1 are not part of the RENAMES clause.*

2.5.9.3. Syntax Rules

1. ALL RENAMES entries referring to data items within a given logical record must immediately follow the last data description entry of the associated record description entry**.**
2. *Data-name-2* and *data-name-3* must be names of elementary items or groups of elementary items in the same logical record, and cannot be the same data-name. A 66 level entry cannot rename another 66 level entry nor can it rename a 77, 88, or 01 level entry.

3. *Data-name-1* cannot be used as a qualifier, and can only be qualified by the names of the associated level 01, FD, CD or SD entries. Neither *data-name-2* nor *data-name-3* may have an OCCURS clause in its data description entry nor may be subordinate to an item that has an OCCURS clause in its data description entry. (See 3.2.1.)

The RENAMES entry may, however, span an entire table or array. If it does, the size of the RENAMES item will include any unused main storage introduced by elementary or group item synchronization.

For example:

```
02 B
     03 C OCCURS 3 TIMES
     03 D OCCURS 3 TIMES
         04 E
         04 F
     03 G
 02 H
 66 J RENAMES B THRU H
```

is a legal sequence while:

```
66 K RENAMES C THRU F
```

alone is not correct.

4. The beginning of the area described by *data-name-3* must not be to the left of the beginning of the area described by *data-name-2*. The end of the area described by *data-name-3* must be to the right of the end of the area described by *data-name-2*. *Data-name-3,* therefore, cannot be subordinate to *data-name-2*.

For example:

```
02 A.
     03 D PIC X(2).
     03 C PIC X(2).
     03 G
         04 E PIC X(2).
         04 F PIC X(2).
02 B REDEFINES A.
     03 H PIC X(3).
     03 I PIC X(4).
     03 J PIC X.
66 K RENAMES D THRU E.
66 L RENAMES C THRU I.
```

will result in K being the first six characters in either A or B, and L being the third through seventh character in either A or B.

5. *Data-name-2* and *data-name-3* may be qualified.
6. The words THRU and THROUGH are equivalent.

7. None of the items within the range, including *data-name-2* and *data-name-3,* if specified, can be an item whose size is variable as defined in the OCCURS clause (see 3.2.1).

2.5.9.4. General Rules

1. One or more RENAMES entries can be written for a logical record.
2. When *data-name-3* is specified, *data-name-1* is a group item which includes all elementary items starting with *data-name-2* (if *data-name-2* is an elementary item) or the first elementary item in *data-name-2* (if *data-name-2* is a group item), and concluding with *data-name-3* (if *data-name-3* is an elementary item) or the last elementary item in *data-name-3* (if *data-name-3* is a group item).
3. When *data-name-3* is not specified, *data-name-2* can be either a group or an elementary item; when *data-name-2* is a group item, *data-name-1* is treated as a group item, and when *data-name-2* is an elementary item, *data-name-1* is treated as an elementary item.
4. The usage, and therefore the initial alignment, of a RENAMES entry is determined by the usage of the *data-name-2* or the first elementary item in *data-name-2* (if *data-name-2* is a group item).

2.5.10. The SIGN Clause

2.5.10.1. Function

The SIGN clause specifies the position and the mode of representation of the operational sign when it is necessary to describe these properties explicitly.

2.5.10.2. General Format

[SIGN IS] { LEADING / TRAILING } [SEPARATE CHARACTER]

2.5.10.3. Syntax Rules

1. The SIGN clause may be specified only for a numeric data description entry whose PICTURE contains the character "S", or a group item containing at least one such numeric data description entry.
2. The numeric data description entries to which the SIGN clause applies must be described as usage is DISPLAY.
3. At most one SIGN clause may apply to any given numeric data description entry.
4. If the CODE–SET clause is specified, any signed numeric data description entries associated with that file description entry must be described with the SIGN IS SEPARATE clause.

2.5.10.4. General Rules

1. The optional SIGN clause, if present, specifies the position and the mode of representation of the operational sign for the numeric data description entry to which it applies, or for each numeric data description entry subordinate to the group to which it applies. The SIGN clause applies only to numeric data description entries whose PICTURE contains the character "S"; the "S" indicates the presence of, but neither the representation nor, necessarily, the position of the operational sign.
2. A numeric data description entry whose PICTURE contains the character "S", but to which no optional SIGN clause applies, has an operational sign, but neither the representation nor, necessarily, the position of the operational sign is specified by the character "S". In this (default) case, the operational sign is represented as an overpunch character (see Table 2-1). General rules 3 through 5 do not apply to such signed numeric data items.
3. If the optional SEPARATE CHARACTER phrase is not present, then:
   a. The operational sign will be presumed to be associated with the leading (or, respectively, trailing) digit position of the elementary numeric data item.
   b. The letter "S" in a PICTURE *character-string* is not counted in determining the size of the item (in terms of standard data format characters).
   c. The valid signs for data items are listed in Table 2-1.
4. If the optional SEPARATE CHARACTER phrase is present, then:
   a. The operational sign will be presumed to be the leading (or, respectively, trailing) character position of the elementary numeric data item. This character position is not a digit position.
   b. The letter "S" in a PICTURE *character-string* is counted in determining the size of the item (in terms of standard data format characters).
   c. The operational signs for positive and negative are the standard data format characters "+" and "-", respectively.
5. Every numeric data description entry whose PICTURE contains the character "S" is a signed numeric data description entry. If a SIGN clause applies to such an entry and conversion is necessary for purposes of computation or comparisons, conversion takes place automatically.
6. The SIGN clause specified at the elementary level overrides the SIGN clause specified at the group level.

2.5.11. The SYNCHRONIZED Clause

2.5.11.1. Function

The SYNCHRONIZED clause specifies the alignment of an elementary item on the natural boundaries of the computer storage (see 1.3.3.3.7).

2.5.11.2. General Format

```
{ SYNCHRONIZED } [ LEFT  ]
{ SYNC         } [ RIGHT ]
```

2.5.11.3. Syntax Rules

1. This clause may appear only with an elementary item.
2. SYNC is an abbreviation for SYNCHRONIZED.

2.5.11.4. General Rules

1. This clause specifies that the subject data item is to be aligned in the computer such that no other data item occupies any of the character positions between the leftmost and rightmost natural boundaries delimiting this data item. If the number of character positions required to store this data item is less than the number of character positions between those natural boundaries, the unused character positions (or portions thereof) must not be used for any other data item. Such unused character positions, however, are included in:
   a. The size of any group items to which the elementary item belongs, and
   b. The character positions redefined when this data item is the object of a REDEFINES clause.
2. SYNCHRONIZED not followed by either RIGHT or LEFT specifies that the elementary item is to be SYNCHRONIZED LEFT if alphanumeric and SYNCHRONIZED RIGHT if numeric.
3. SYNCHRONIZED LEFT specifies that the elementary item is to be positioned such that it will begin at the left boundary of the first word in which the elementary item is placed. Unused portions of the word, if any, will be to the right of the lower order character or digit position of the data item.
4. SYNCHRONIZED RIGHT specifies that the elementary item is to be positioned such that it will terminate on the right boundary of the last word in which the elementary item is placed. Unused portions of the word, if any, will be to the left of the high order character or digit position of the data item.
5. Whenever a SYNCHRONIZED item is referenced in the source program, the original size of the item, as shown in the PICTURE clause, is used in determining any action that depends on size, such as justification, truncation or overflow.
6. If the data description of an item contains the SYNCHRONIZED clause and an operational sign, the sign of the item appears in the normal operational sign position, regardless of whether the item is SYNCHRONIZED LEFT or SYNCHRONIZED RIGHT.
7. When the SYNCHRONIZED clause is specified in a data description entry of a data item that also contains an OCCURS clause, or in a data description entry of a data item subordinate to a data description entry that contains an OCCURS clause, then:
   a. Each occurrence of the data item is SYNCHRONIZED.
   b. Any implicit FILLER generated for other data items within that same table are generated for each occurrence of those data items. (See general rule 8b.)

8. This clause is hardware-dependent and in addition to rules 1 through 7, the elementary items associated with this clause are handled as follows:

   a. The format on the external media of records or groups containing elementary items whose data description contains the SYNCHRONIZED clause is the same as the internal format.

   b. Any necessary generation of implicit FILLER, if the elementary item immediately preceding an item containing the SYNCHRONIZED clause does not terminate at a word boundary, is not included as part of the elementary item. Such automatically generated FILLER positions are included in:

      1) The size of any group to which the FILLER item belongs; and

      2) The number of character positions allocated when the group item of which the FILLER item is a part appears as the object of a REDEFINES clause.

9. In the absence of the SYNCHRONIZED clause, ASCII COBOL will align data items on the next available word, quarter word, sixth word, or bit boundary in accord with the data item's USAGE clause (see 2.5.12) or position in a word. A full description of data allocation and alignment may be found in D.2.

10. The specification of SYNCHRONIZED for a data item which is already synchronized by virtue of its usage (COMPUTATIONAL–1, COMPUTATIONAL–2, or level 01 data item) is redundant although permitted.

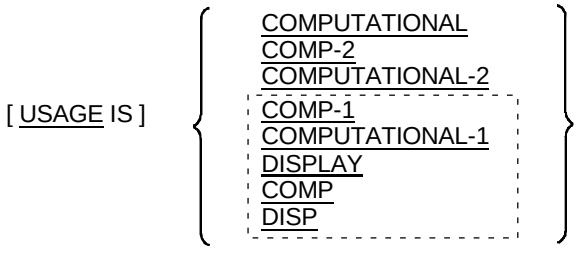
11. Level 77 items are implicitly synchronized left and need a SYNCHRONIZED or SYNCHRONIZED RIGHT if synchronization different from the default is desired.

2.5.12. The USAGE Clause

2.5.12.1. Function

The USAGE clause specifies the format of a data item in storage.

2.5.12.2. General Format



2.5.12.3. Syntax Rules

1. The PICTURE character-string of a COMPUTATIONAL item can contain only 9's, the operational sign character "S", the implied decimal point character "V", and one or more P's. (See 2.5.7, the PICTURE Clause.)

2. COMP, COMP–1, COMP–2, and DISP are abbreviations for COMPUTATIONAL, COMPUTATIONAL–1, COMPUTATIONAL–2 and DISPLAY respectively.

3. COMPUTATIONAL–1, and COMPUTATIONAL–2 usage specifications may not appear in a data description which has a PICTURE clause, because the computer storage format of these items is fixed.
4. The USAGE clause cannot be used to describe elementary items with a PICTURE *character-string* containing 1's.

2.5.12.4. General Rules

1. The USAGE clause can be written at any level. If the USAGE clause is written at a group level, it applies to each elementary item in the group. The USAGE clause of an elementary item cannot contradict the USAGE clause of a group to which the item belongs. The USAGE clause has the following meanings for group items:

   a. Any explicit usage for a group item applies to all the contained group or elementary items.

   b. If the group usage is defined as DISPLAY or COMPUTATIONAL, the group is considered to be an alphanumeric string of ASCII characters. If the group usage is not explicitly defined, the group is considered to be an alphanumeric string of ASCII characters.

2. This clause specifies the manner in which a data item is represented in the storage of a computer. It does not affect the use of the data item, although the specifications for some statements in the Procedure Division may restrict the USAGE clause of the operands referred to. The USAGE clause may affect the radix or type of character representation of the item. Table 2-5 describes the meaning and internal representation of data. Alignment for the various USAGE clauses is specified in Table 2-6.

Table 2-5. Meaning and Allocation of Data Types by USAGE

Usage	Meaning	Allocation
DISPLAY	ASCII characters	9 bits, quarter word, per allocatable character represented in PICTURE clause
COMPUTATIONAL	Fixed-point binary; also called ASCII sync-binary	1–2 digits, 1/4 word, 9 bits 3–5 digits, 1/2–word, 18 bits 6–7 digits, 3/4 word, 27 bits 8–10 digits, 1 word, 36 bits 11–13 digits, 1–1/4 word, 45 bits 14–15 digits, 1–1/2 word, 54 bits 16–18 digits, 1–3/4 word, 63 bits
COMPUTATIONAL–1	Single precision signed floating point	1 word, synchronized in one computer word
COMPUTATIONAL–2	Double precision signed floating point	2 words, synchronized in two computer words

Table 2-6. Alignment of Data Types by USAGE

Usage	Alignment
DISPLAY	Quarter–word boundary
COMPUTATIONAL	Quarter–word boundary
COMPUTATIONAL–1	Word boundary
COMPUTATIONAL–2	Word boundary

3. A COMPUTATIONAL, COMPUTATIONAL–1, or COMPUTATIONAL–2 item is capable of representing a value to be used in computations and must be numeric. If a group item is described as COMPUTATIONAL, COMPUTATIONAL–1, or COMPUTATIONAL–2, the elementary items in the group are COMPUTATIONAL, COMPUTATIONAL–1, or COMPUTATIONAL–2, respectively. The group item itself is not COMPUTATIONAL, COMPUTATIONAL–1, or COMPUTATIONAL–2 (cannot be used in computations).
4. The USAGE IS DISPLAY clause indicates that the format of the data is a standard data format.
5. If the USAGE clause is not specified for an elementary item, or for any group to which the item belongs, the usage is implicitly DISPLAY.

6. When a USAGE COMPUTATIONAL–1 or COMPUTATIONAL–2 data item is either implicitly or explicitly moved to a USAGE COMPUTATIONAL or DISPLAY data item, the result may be imprecise in the least significant positions. For arithmetic statements, see 2.6.3.7.

2.5.13. The VALUE Clause

2.5.13.1. Function

The VALUE clause defines the value of constants, the value of Report Section printable items, the initial value of working storage and Common-Storage Section items, the initial value of data items in the Communication Section, and the values associated with a condition-name.

2.5.13.2. General Format

Format 1

VALUE IS *literal*

Format 2

{ VALUE IS / VALUES ARE } *literal-1* [{ THROUGH / THRU } *literal-2*]

[, *literal-3* [{ THROUGH / THRU } *literal-4*]] ...

2.5.13.3. Syntax Rules

1. The words THRU and THROUGH are equivalent.
2. The VALUE clause cannot be stated for any items whose size is variable. (See 3.2.1.)
3. A signed numeric literal must have associated with it a signed numeric PICTURE *character-string.*
4. All numeric literals in a VALUE clause of an item must have a value which is within the range of values indicated by the PICTURE clause, and not have a value which would require truncation of nonzero digits. Nonnumeric literals in a VALUE clause of an item must not exceed the size indicated by the PICTURE clause.

2.5.13.4. General Rules

1. The VALUE clause must not conflict with other clauses in the data description of the item or in the data description within the hierarchy of the item. The following rules apply:
   a. If the category of the item is numeric, all literals in the VALUE clause must be numeric. If the literal defines the value of a working storage item, the literal is aligned in the data item according to the standard alignment rules. (See 1.3.3.3.6.)

b. If the category of the item is alphabetic, alphanumeric, alphanumeric edited or numeric edited, all literals in the VALUE clause must be nonnumeric literals. The literal is aligned in the data item as if the data item had been described as alphanumeric. (See 1.3.3.3.6.) Editing characters in the PICTURE clause are included in determining the size of the data item but have no effect on initialization of the data item. Therefore, the VALUE for an edited item is presented in an edited form.

c. Initialization takes place independent of any BLANK WHEN ZERO or JUSTIFIED clause that may be specified.

2. A figurative constant may be substituted in both Format 1 and Format 2 wherever a literal is specified.

2.5.13.5. Condition–Name Rules

1. In a condition-name entry, the VALUE clause is required. The VALUE clause and the condition-name itself are the only two clauses permitted in the entry. The characteristics of a condition-name are implicitly those of its conditional variable.
2. Format 2 can be used only in connection with condition-names. (See 1.3.3.3.8.5.) Wherever the THRU phrase is used, *literal-1* must be less than *literal-2, literal-3* less than *literal-4,* etc.

2.5.13.6. Data Description Entries Other Than Condition–Names

1. Rules governing the use of the VALUE clause differ with the respective sections of the Data Division:

   a. In Level 1, the VALUE clause cannot be used in the File Section. In the File Section, the VALUE clause may be used only in condition-name entries.

   b. In the Working–Storage Section, Common-Storage Section, and the Communication Section, the VALUE clause must be used in condition-name entries. The VALUE clause may also be used to specify the initial value of any other data item, in which case the clause causes the item to assume the specified value at the start of the object program. If the VALUE clause is not used in an item's description, the initial value is undefined.

   c. In Level 1, the VALUE clause cannot be used in the Linkage Section. In the Linkage Section, the VALUE clause may be used only in condition-name entries.

   d. In the Report Section, if the elementary report entry containing the VALUE clause does not contain a GROUP INDICATE clause, then the printable item will assume the specified value each time its report group is printed. However, when the GROUP INDICATE clause is also present, the specified value will be presented only when certain object time conditions exist. (See 8.2.12.)

2. The VALUE clause must not be stated in a data description entry that contains an OCCURS clause, or in an entry that is subordinate to an entry containing an OCCURS clause. This rule does not apply to condition-name entries. (See 3.2.1.)

3. The VALUE clause must not be stated in a data description entry that contains a REDEFINES clause, or in an entry that is subordinate to an entry containing a REDEFINES clause. This rule does not apply to condition-name entries.
4. If the VALUE clause is used in an entry at the group level, the literal must be a figurative constant or a nonnumeric literal, and the group area is initialized without consideration for the individual elementary or group items contained within this group. The VALUE clause cannot be stated at the subordinate levels within this group.
5. The VALUE clause must not be written for a group containing items with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).

2.6. Procedure Division

2.6.1. Arithmetic Expressions

2.6.1.1. Definition of an Arithmetic Expression

An arithmetic expression can be an identifier of a numeric elementary item, a numeric literal, such identifiers and literals separated by arithmetic operators, two arithmetic expressions separated by an arithmetic operator, or an arithmetic expression enclosed in parentheses. Any arithmetic expression may be preceded by a unary operator. The permissible combinations of variables, numeric literals, arithmetic operators, and parentheses are given in Table 2-7.

Those identifiers and literals appearing in an arithmetic expression must represent either numeric elementary items or numeric literals on which arithmetic may be performed.

2.6.1.2. Arithmetic Operators

There are five binary arithmetic operators and two unary arithmetic operators that may be used in arithmetic expressions. These operators are represented by specific characters that must be preceded by a space and followed by a space.

Binary Arithmetic Operators	**Meaning**
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponentiation
Unary Arithmetic Operators	**Meaning**
+	The effect of multiplication by numeric literal +1
-	The effect of multiplication by numeric literal -1

2.6.1.3. Formation And Evaluation Rules

1. Parentheses may be used in arithmetic expressions to specify the order in which elements are to be evaluated. Expressions within parentheses are evaluated first, and within nested parentheses, evaluation proceeds from the least inclusive set to the most inclusive set. When parentheses are not used, or parenthesized expressions are at the same level of inclusiveness, the following hierarchical order of execution is implied:

   1st Unary plus and minus

   2nd Exponentiation

   3rd Multiplication and division

   4th Addition and subtraction

2. Parentheses are used either to eliminate ambiguities in logic where consecutive operations of the same hierarchical level appear or to modify the normal hierarchical sequence of execution in expressions where it is necessary to have some deviation from the normal precedence. When the sequence of execution is not specified by parentheses, the order of execution of consecutive operations of the same hierarchical level is from left to right.

3. The ways in which operators, variables, and parentheses may be combined in an arithmetic expression are summarized in Table 2-7, where:

   a. The letter "P" indicates a permissible pair of symbols.

   b. The character "-" indicates an invalid pair.

   c. "Variable" indicates an identifier or literal.

Table 2-7. Combination of Symbols in Arithmetic Expressions

First Symbol	Second Symbol				
	Variable	*** / ** – +**	**Unary + or –**	**(**	**)**
Variable	-	P	-	-	P
* / ** + -	P	-	P	P	-
Unary + or -	P	-	-	P	-
(	P	-	P	P	-
)	-	P	-	-	P

4. An arithmetic expression may only begin with the symbol "(", "+", "-", or a variable and may only end with a ")" or a variable. There must be a one–to–one correspondence between left and right parentheses of an arithmetic expression such that each left parenthesis is to the left of its corresponding right parenthesis.

5. Arithmetic expressions allow the user to combine arithmetic operations without the restrictions on composite of operands and/or receiving data items. See, for example, syntax rule 3, 2.6.5.

See 2.6.3.7 for a description of techniques used in handling arithmetic expressions.

The following examples illustrate natural and parentheses enforced ordering of COBOL arithmetic expressions. In these examples let A = 4, B = 6, C = 2, D = 3, and E = 12.

Example	**Evaluation**
A+B∗C∗∗D	52
A+B∗C	16
E-A/C	10
(A+B∗C)∗∗D	4096
(A+B) ∗C	20
(E-A)/C	4
((A+B)∗C)∗∗D	8000

There are instances where the rules of precedence are insufficient to represent the precise meaning of an algebraic formula without using parentheses. For example:

$$\frac{A+B}{A-B}$$

cannot be expressed as a COBOL expression without using parentheses, according to the rules of precedence established previously. To write it as:

A + B/A – B

is incorrect since this represents the expression:

$$A + \frac{B}{A} - B$$

In this case, parentheses must be used and the desired result is obtained by writing

(A+B)/(A–B)

Parentheses have higher precedence than any operator. Operations within parentheses, therefore, have precedence over any other in a calculation. In the following example, let A = 4, B = 3, and C = 6:

A + B * A – C / 2

This produces 13 as a result, whereas:

((A + B) * A – C) / 2

produces 11, with the sequence of evaluation proceeding from the innermost parenthesized operation to the outermost as follows:

1. A + B = 7
2. 7 * A = 28
3. 28 - C = 22
4. 22/2 = 11 (Result)

Double exponentiation is sometimes used in an algebraic expression. For example:

A^{b^c}

may not be written as:

```
A ** B ** C
```

Parentheses must be used to indicate which exponentiation is to be done first. The correct expression is:

```
A ** (B ** C)
```

As an example, let A = 2, B = 2, and C = 3. The resulting evaluations are:

$A ** B ** C = 4^3 = 64$

$A ** (B ** C) = 2^8 = 256$

In ensuing paragraphs, it will be shown how arithmetic expressions may be used in a conditional case to determine if the expression defines a specific value.

2.6.2. Conditional Expressions

Conditional expressions identify conditions that are tested to enable the object program to select between alternate paths of control depending upon the truth value of the condition. Conditional expressions are specified in the IF, PERFORM and SEARCH statements. There are two categories of condition associated with conditional expressions: simple conditions and complex conditions. Each may be enclosed within any number of paired parentheses, in which case its category is not changed.

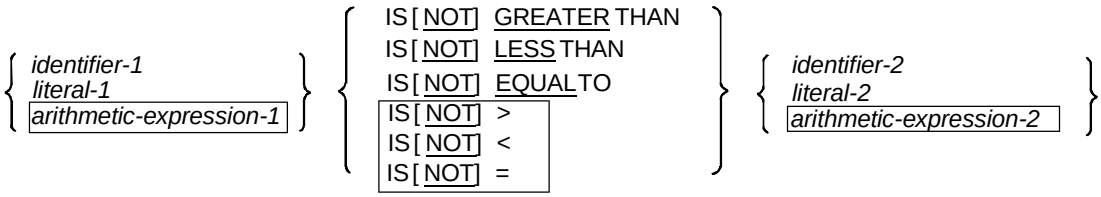
2.6.2.1. Simple Conditions

The simple conditions are the relation, class condition-name, switch-status, and sign conditions. A simple condition has a truth value of "true" or "false". The inclusion in parentheses of simple conditions does not change the simple truth value.

2.6.2.1.1. Relation Condition

A relation condition causes a comparison of two operands, each of which may be the data item referenced by an identifier, a literal, or the value resulting from an arithmetic expression. A relation condition has a truth value of "true" if the relation exists between the operands. Comparison of two numeric operands is permitted regardless of the formats specified in their respective USAGE clauses. However, for all other comparisons the operands must have the same usage. If either of the operands is a group item, the nonnumeric comparison rules apply.

The general format of a relation condition is:



Note: *The required relational characters " > ", " < ", and " = " are not underlined, to avoid confusion with other symbols such as "≥" (greater than or equal to).*

The first operand (*identifier-1, literal-1,* or *arithmetic-expression-1*) is called the subject of the condition; the second operand (*identifier-2, literal-2,* or *arithmetic-expression-2*) is called the object of the condition. The relation condition must contain at least one reference to a variable.

The relational operator specifies the type of comparison to be made in a relation condition. A space must precede and follow each reserved word comprising the relational operator. When used, "NOT" and the next key word or relation character are one relational operator that defines the comparison to be executed for truth value; for example, "NOT EQUAL" is a truth test for an "unequal" comparison; "NOT GREATER" is a truth test for an "equal" or "less" comparison. The meaning of the relational operators is as follows.

Meaning	Relational Operator
Greater than or not greater than	IS [NOT] GREATER THAN IS [NOT] >
Less than or not less than	IS [NOT] LESS THAN IS [NOT] <
Equal to or not equal to	IS [NOT] EQUAL TO IS [NOT] =

Note: *The required relational characters " > ", " < ", and " = " are not underlined, to avoid confusion with other symbols such as "≥" (greater than or equal to).*

2.6.2.1.1.1. Comparison of Numeric Operands

For operands whose class is numeric (see 1.3.3.3.3), a comparison is made with respect to the algebraic value of the operands. The length of the literal or arithmetic expression operands, in terms of number of digits represented, is not significant. Zero is considered a unique value regardless of the sign.

For example, a comparison of a data-item which has a value of +000003 with a data-item which has a value of +03 will result in an "equal" condition. Similarly, the value of 000000 is equal to the value +0000. Following the rules of algebra, +01 is greater than –155.

Comparison of these operands is permitted regardless of the manner in which their usage is described. Unsigned numeric operands are considered positive for purposes of comparison.

For example, A is equal to B if they are described as:

```
02 A PIC 99V99 VALUE 0.01.
02 B COMP-1 VALUE 10.E-3.
```

2.6.2.1.1.2. Comparison of Nonnumeric Operands

For nonnumeric DISPLAY operands, or one numeric DISPLAY and one nonnumeric DISPLAY operand, a comparison is made with respect to a specified collating sequence of characters (see 2.4.1.2). If one of the operands is specified as numeric, it must be an integer data item or an integer literal and:

1. If the nonnumeric operand is an elementary data item or nonnumeric literal, the numeric operand is treated as though it were moved to an elementary alphanumeric data item of the same size as the numeric data item (in terms of standard data format characters), and the contents of this alphanumeric data item were then compared to the nonnumeric operand. (See 2.6.14 and 2.5.7.)
2. If the nonnumeric operand is a group item, the numeric operand is treated as though it were moved to a group item of the same size as the numeric data item (in terms of standard data format characters), and the contents of this group item were then compared to the nonnumeric operand. (See 2.6.14, the MOVE Statement, and 2.5.7, the PICTURE character "P".)

3. A noninteger numeric operand cannot be compared to a nonnumeric operand.

   The size of an operand is the total number of standard data format characters in the operand. Numeric and nonnumeric operands may be compared only when their usage is the same.

There are two cases to consider: operands of equal size and operands of unequal size.

1. Operands of equal size. If the operands are of equal size, comparison effectively proceeds by comparing characters in corresponding character positions starting from the high order end and continuing until either a pair of unequal characters is encountered or the low order end of the operand is reached, whichever comes first. The operands are determined to be equal if all pairs of characters compare equally through the last pair, when the low order end is reached.

   The first encountered pair of unequal characters is compared to determine their relative position in the collating sequence. The operand that contains the character that is positioned higher in the collating sequence is considered to be the greater operand.

2. Operands of unequal size. If the operands are of unequal size, comparison proceeds as though the shorter operand were extended on the right by sufficient spaces to make the operands of equal size.

The following shows the effect of a collating sequence on alphanumeric comparisons:

```
ENVIRONMENT DIVISION.
   .
   .
   .
OBJECT-COMPUTER.  UNISYS-1100 PROGRAM COLLATING SEQUENCE IS NEW-ALPHABET.
SPECIAL-NAMES.
    NEW-ALPHABET IS “A” “B” “CDE” “F” ALSO “f” 72 THRU “U” “Z” THRU “V” “0” THRU 58.
   .
   .
   .
DATA DIVISION.
WORKING-STORAGE SECTION.
01  DATA-NAMES.
    02 DATA-NAME1 PIC XXX VALUE ‘FFF’.
    02 DATA-NAME2 PIC XXX VALUE ‘fff’.
    02 DATA-NAME3 PIC XXX VALUE ‘42X’.
    02 DATA-NAME4 PIC 99 VALUE 42.
01  COUNTER PIC 9(10) COMP VALUE 0.
   .
   .
   .
PROCEDURE DIVISION.
P1.
    IF DATA-NAME1 = DATA-NAME2 MOVE LOW-VALUES TO DATA-NAME1.
    IF DATA-NAME2 LESS THAN DATA-NAME3 ADD 1 TO COUNTER.
    IF DATA-NAME3 > DATA-NAME4 GO TO P2.
   .
   .
   .
P2.
    STOP RUN.
```

A new program collating sequence is defined as described in Table 2-8. Note that new low and high values are also defined such that LOW–VALUE = “A” and HIGH–VALUE = 0377.

Also note that a numeric literal represents the ordinal position of that character in the native collating sequence. Thus, the ordinal position of any character will be one greater than its decimal order as defined in Table B–1. For example, 72 represents the 72nd character in the ASCII collating sequence which is the character “G” (octal 107 or decimal 71).

Table 2-8. Collating Sequence Example

Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal	Source	Coll Seq Order	ASCII Char	ASCII Octal
"A"	0	A	101	Unused	044	None	00	Unused	0110	$	44	Unused	0154	m	0155
"B"	1	B	102	Characters	045	None	01	Characters	0111	%	45	Characters	0155	n	0156
"CDE"	2	C	103	In	046	None	02	In	0112	&	46	In	0156	o	0157
	3	D	104	Native	047	None	03	Native	0113	'	47	Native	0157	p	0160
	4	E	105	Collating	050	None	04	Collating	0114	(	50	Collating	0160	q	0161
"F" ALSO "f"	5	F f	106 146	Sequence	051	None	05	Sequence	0115	)	51	Sequence	0161	r	0162
72	6	G	107												
	7	H	110	Order	052	None	06	Order	0116	*	52	Order	0162	s	0163
	010	I	111	.	053	None	07	.	0117	+	53	.	0163	t	0164
	011	J	112												
	012	K	113	.	054	None	10	.	0120	+	54	.	0164	u	0165
	013	L	114												
	014	M	115	.	055	None	11	.	0121	-	55	.	0165	v	0166
THRU	015	N	116		056	None	12		0122	.	56		0166	w	0167
	016	O	117												
	017	P	120		057	None	13		0123	/	57		0167	x	0170
	020	Q	121		060	None	14		0124	:	72		0170	y	0171
	021	R	122		061	None	15		0125	;	73		0171	z	0172
	022	S	123		062	None	16		0126	<	74		0172	{	0173
	023	T	124												
"U"	024	U	125		063	None	17		0127	=	75		0173	|	0174
					064	None	20		0130	>	76		0174	}	0175
"Z"	025	Z	132												
					065	None	21		0131	?	77		0175	~	0176
	026	Y	131		066	None	22		0132	@	100		0176	Del	0177
THRU	027	X	130												
					067	None	23		0133	[	133		0177	None	0200
	030	W	127		070	None	24		0134	\	134		0200	None	0201
"V"	031	V	126		071	None	25		0135	[	135		.	.	.
"0"	032	0	60		072	None	26		0136	^	136		.	.	.
					073	None	27		0137	_	137		.	.	.
	033	1	61		074	None	30		0140	'	140				
					075	None	31		0141	a	141				
	034	2	62		076	None	32		0142	b	142				
	035	3	63												
					077	None	33		0143	c	143				
	036	4	64		0100	None	34		0144	d	144				
THRU	037	5	65		0101	None	35		0145	e	145				
					0102	None	36		0146	g	147				
	040	6	66		0103	None	37		0147	h	150				
					0104	SP	40		0150	i	151				
	041	7	67		0105	!	41		0151	j	152				
					0106	"	42		0152	k	153				
	042	8	70		0107	#	43		0153	l	154				
	043	9	71												
58													0376	None	377

The comparison DATA–NAME1 = DATA–NAME2 will be true since the relative collating sequence numbers of the characters "F" in DATA–NAME1 and "f" in DATA–NAME2 are equal. Thus the value of DATA–NAME1 will be changed to "AAA". See Table 2-8.

The comparison DATA–NAME2 < DATA–NAME3 will also be true since the first pair of characters encountered in the values of DATA–NAME2 and DATA–NAME3 have relative collating sequence numbers of 05_8 (f) and 036_8 (4) respectively.

The comparison DATA–NAME3 > DATA–NAME4 will be false since the relative collating sequence number of the character "X" in the value of DATA–NAME3 is less than the relative collating sequence number of the character "Δ" in the value of DATA–NAME4.

2.6.2.1.2. Class Condition

The class condition determines whether the operand is numeric, that is, consists entirely of the characters "0", "1", "2", "3", ..., "9", with or without the operational sign, alphabetic, that is, consists entirely of the characters "A", "B", "C", ..., "a", "b", "c", ..., "z", space; or CLASS–NAME, that is, consists entirely of the characters specified in the CLASS–NAME clause of the SPECIAL–NAMES paragraph. The general format for the class condition is as follows:

identifier IS [NOT] { NUMERIC / ALPHABETIC / CLASS-NAME *mnemonic-name* }

The usage of the operand being tested must be described as display. The USAGE of the operand may also be COMP–3, when IBM OS/360 allocation rules apply (see Table 15-2). When used, "NOT" and the next key word specify one class condition that defines the class test to be executed for truth value; for example, "NOT NUMERIC" is a truth test for determining that an operand is nonnumeric.

The NUMERIC test cannot be used with an item whose data description describes the item as alphabetic or as a group item composed of elementary items whose data description indicates the presence of operational signs. If the data description of the item being tested does not indicate the presence of an operational sign, the item being tested is determined to be numeric only if the contents are numeric and an operational sign is not present. If the data description of the item does indicate the presence of an operational sign, the item being tested is determined to be numeric only if the contents are numeric and a valid operational sign is present. Valid operational signs for data items described with the SIGN IS SEPARATE clause are the standard data format characters, "+" and "-". See Table 2-1 for what constitutes valid signs for data items not described with the SIGN IS SEPARATE clause.

The ALPHABETIC test cannot be used with an item whose data description describes the item as numeric. The item being tested is determined to be alphabetic only if the contents consist of any combination of the alphabetic characters "A" through "Z", "a" through "z", and the space.

The CLASS–NAME test allows specification of an alphabetic, numeric, or SOURCE-ALPHABET subset.

2.6.2.1.3. Condition-Name Condition (Conditional Variable)

In a condition-name condition, a conditional variable is tested to determine whether or not its value is equal to one of the values associated with a condition-name. The general format for the condition-name condition is:

condition-name

If the condition-name is associated with a range or ranges of values, then the conditional variable is tested to determine whether or not its value falls in this range, including the end values.

The rules for comparing a conditional variable with a condition-name value are the same as those specified for relation conditions.

The result of the test is true if one of the values corresponding to the condition-name equals the value of its associated conditional variable.

For example, the data description of a particular conditional variable might appear as:

```
02 MARITAL-STATUS PICTURE IS 9.
     88 SINGLE VALUE IS 1.
     88 MARRIED VALUE IS 2.
     88 DIVORCED VALUE IS 3.
     88 WIDOWED VALUE IS 4.
     88 EVER-MARRIED VALUE 2 THRU 4.
```

Using an IF statement, the test to determine marital status may appear as:

```
IF MARRIED . . .
```

which is equivalent to:

```
IF MARITAL-STATUS IS EQUAL TO 2
```

Again, as in the relational condition, NOT is used to specify the opposite of what the condition-name test would normally specify. So, for example:

```
IF NOT SINGLE . . .
```

is equivalent to:

```
IF MARITAL-STATUS IS NOT EQUAL TO 1
```

The same situation could be checked by writing:

```
IF EVER-MARRIED . . .
```

which is equivalent to:

```
IF MARITAL-STATUS NOT < 2 AND
   MARITAL-STATUS NOT > 4 . . .
```

When a single condition-name is associated with a list of values or ranges of values, a programmer may express what is equivalent to rather complex compound relationals in a simple condition-name condition.

For example, the code names for various bolts in inventory might be:

```
03 PART-NUMBER PIC X(5).
  88 BOLT VALUE IS "AA000" THRU
    "AA999", "AX010" THRU "AZ999",
      "P1000", "P1010".
```

Using the IF statement to determine whether the part is a bolt, a programmer can write:

```
IF BOLT GO TO IS-BOLT.
```

which is equivalent to writing:

```
IF PART-NUMBER NOT < "AA000" AND
        PART-NUMBER NOT > "AA999" OR
        PART-NUMBER NOT > "AX010" AND
        PART-NUMBER NOT > "AZ999" OR
        PART-NUMBER = "P1000" OR "P1010"
        GO TO IS-BOLT .
```

2.6.2.1.4. Switch–Status Condition

A switch-status condition determines the "on" or "off" status of an implementor-defined switch. The implementor-name and the "on" or "off" value associated with the condition must be named in the SPECIAL–NAMES paragraph of the Environment Division. The general format for the switch-status condition is:

condition-name

The result of the test is true if the switch is set to the specified position corresponding to *condition-name.*

The implementor-names SWITCH–13, SWITCH–14, . . ., SWITCH–24 correspond to the bits in the last third of the condition word set by the SET mnemonic-name verb. Thus, SWITCH–13 is bit 24, SWITCH–14 is bit 25, . . ., SWITCH–24 is bit 35. The switch-status condition ON checks for a 1 in the appropriate bit, OFF checks for a 0.

The implementor-names SWITCH–1, SWITCH–2, . . ., SWITCH–12 correspond to the bits in the middle or second third of the condition word set by the @SETC command statement. Thus, SWITCH–1 is bit 12, SWITCH–2 is bit 13, . . ., SWITCH–12 is bit 23. The switch-status condition ON checks for a 1 in the appropriate bit, OFF checks for a 0.

For example:

```
@SETC 153/T2
```

will set SWITCH–6, SWITCH–7, SWITCH–9, SWITCH–11, and SWITCH–12 ON; other switches will be OFF.

Thus, a program which is executed contains the statements:

```
ENVIRONMENT DIVISION.
  .
  .
  .
SPECIAL-NAMES.
    SWITCH-6 IS DAILY ON IS DAILY-CYCLE
    SWITCH-7 IS WEEKLY ON IS WEEKLY-CYCLE
    SWITCH-9 IS BI-WEEKLY ON IS BI-WEEKLY-CYCLE
    SWITCH-11 IS MONTH ON IS MONTHLY-CYCLE
    SWITCH-12 IS ANNUAL ON IS ANNUAL-CYCLE.
PROCEDURE DIVISION.
  .
  .
  .
    IF ANNUAL-CYCLE GO TO FISCAL-CLOSEOUT.
    IF MONTHLY-CYCLE GO TO MONTHLY-BILLING.
    IF BI-WEEKLY-CYCLE GO TO TWO-WEEK-REPORT.
    IF WEEKLY-CYCLE GO TO WEEK-END-SUMMARY.
    IF DAILY-CYCLE GO TO ACCOUNTING-UPDATE.
```

2.6.2.1.5. Sign Condition

The sign condition determines whether or not the algebraic value of an arithmetic expression is less than, greater than, or equal to zero. The general format for a sign condition follows.

arithmetic-expression IS [NOT] { POSITIVE | NEGATIVE | ZERO }

When used, "NOT" and the next key word specify one sign condition that defines the algebraic test to be executed for truth value; for example, "NOT ZERO" is a truth test for a nonzero (positive or negative) value. An operand is positive if its value is greater than zero, negative if its value is less than zero, and zero if its value is equal to zero. The zero test will be true for any numeric item containing a zero even if the field contains a positive or negative operational sign. The arithmetic expression must contain at least one reference to a variable.

Any condition that can be expressed in a numeric status test may also be expressed by a relational expression. For example:

```
A + B + C IS POSITIVE
```

may also be expressed in a relational expression as:

```
A + B + C IS GREATER THAN 0
```

2.6.2.2. Complex Conditions

A complex condition is formed by combining simple conditions, combined conditions and/or complex conditions with logical connectors (logical operators "AND" and "OR"), or negating these conditions with logical negation (the logical operator "NOT"). The truth value of a complex condition, whether parenthesized or not, is that truth value which results from the interaction of all the stated logical operators on the individual truth values of simple conditions, or the intermediate truth values of conditions logically connected or logically negated.

The logical operators and their meanings are:

Logical Operator	Meaning
AND	Logical conjunction; the truth value is "true" if both of the conjoined conditions are true; "false" if one or both of the conjoined conditions is false
OR	Logical inclusive OR; the truth value is "true" if one or both of the included conditions is true; "false" if both included conditions are false
NOT	Logical negation or reversal of truth value; the truth value is "true" if the condition is false; "false" if the condition is true.

The logical operators must be preceded by a space and followed by a space.

2.6.2.2.1. Negated Simple Conditions

A simple condition (see 2.6.2.1) is negated through the use of the logical operator "NOT". The negated simple condition affects the opposite truth value for a simple condition. Thus the truth value of a negated simple condition is "true" if and only if the truth value of the simple condition is "false"; the truth value of a negated simple condition is "false" if and only if the truth value of the simple condition is "true". The inclusion in parentheses of a negated simple condition does not change the truth value.

The general format for a negated simple condition is:

```
NOT  simple-condition
```

The NOT reverses the true/false value of the condition. Therefore,

```
A = B
```

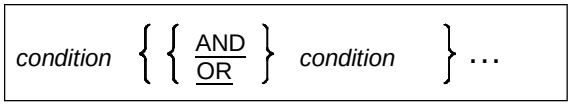
and:

```
NOT A = B
```

will have the effect that if A = B is true, NOT A = B will be false, and vice versa.

2.6.2.2.2. Combined and Negated Combined Conditions

A combined condition results from connecting conditions with one of the logical operators "AND" or "OR". The general format of a combined condition is:



where *condition* may be:

1. A simple condition, or
2. A negated simple condition, or
3. A combined condition, or
4. A negated combined condition, in other words, the "NOT" logical operator followed by a combined condition enclosed within parentheses, or
5. Combinations of 1 through 4, specified according to the rules summarized in Table 2-9.

Although parentheses need never be used when either "AND" or "OR" (but not both) is used exclusively in a combined condition, parentheses may be used to effect a final truth value when a mixture of "AND", "OR", and "NOT" is used. (See Table 2-9 and the condition evaluation rules in 2.6.2.4.)

Table 2-9 indicates the ways in which conditions and logical operators may be combined and parenthesized. There must be a one–to–one correspondence between left and right parentheses such that each left parenthesis is to the left of its corresponding right parenthesis.

Table 2-9. Combinations of Conditions, Logical Operators, and Parentheses

Given the following element:	Location in conditional expression		In a left-to-right sequence of elements:	
	First	Last	Element, when not first, may be immediately preceded by only:	Element, when not last, may be immediately followed by only:
Simple-condition	Yes	Yes	OR, NOT, AND, (	OR, AND,)
OR or AND	No	No	Simple-condition,)	Simple-condition, NOT, (
NOT	Yes	No	OR, AND, (	Simple-condition, (
(	Yes	No	OR, NOT, AND, (	Simple-condition, NOT, (
)	No	Yes	Simple-condition,)	OR, AND,)

Thus, the element pair "OR NOT" is permissible while the pair "NOT OR" is not permissible; "NOT (" is permissible while "NOT NOT" is not permissible.

The logical operators express a logical relationship between the conditions and determine, according to rules for such operations, the truth or falsity of the entire operation. The logical determination of true or false is based upon the rules defined in the following table. Here A and B represent simple conditions.

If A is:	and B is:	A AND B will be:	A OR B will be:	A AND NOT B will be:	A OR NOT B will be:
True	True	True	True	False	True
False	True	False	True	False	False
True	False	False	True	True	True
False	False	False	False	False	True

2.6.2.3. Abbreviated Combined Relation Conditions

When simple or negated simple relation conditions are combined with logical connectives in a consecutive sequence such that a succeeding relation condition contains a subject or subject and relational operator that is common with the preceding relation condition, and no parentheses are used within such a consecutive sequence, any relation condition except the first may be abbreviated by:

1. The omission of the subject of the relation condition, or
2. The omission of the subject and relational operator of the relation condition.

The format for an abbreviated combined relation condition is:

relation-condition { { AND / OR } [NOT] [*relational-operator*] *object* } ...

Within a sequence of relation conditions both of these forms of abbreviation may be used. The effect of using such abbreviations is as if the last preceding stated subject were inserted in place of the omitted subject, and the last stated relational operator were inserted in place of the omitted relational operator. The result of such implied insertion must comply with the rules of Table 2-9. This insertion of an omitted subject and/or relational operator terminates once a complete simple condition is encountered within a complex condition.

The interpretation applied to the use of the word "NOT" in an abbreviated combined relation condition is:

1. If the word immediately following "NOT" is "GREATER", " > ", "LESS", " < ", "EQUAL", "=", then the "NOT" participates as part of the relational operator; otherwise
2. The "NOT" is interpreted as a logical operator and, therefore, the implied insertion of subject or relational operator results in a negated relation condition.

Some examples of abbreviated combined and negated combined relation conditions and expanded equivalents follow.

Abbreviated Combined Relation Condition	Expanded Equivalent
a > b AND NOT < c OR d	((a > b) AND (a NOT < c)) OR (a NOT < d)
a NOT EQUAL b OR c	(a NOT EQUAL b) OR (a NOT EQUAL c)
NOT a = b OR c	(NOT (a = b)) OR (a = c)
NOT (a GREATER b OR < c)	NOT ((a GREATER b) OR (a < c))
NOT (a NOT > b AND c AND NOT d)	NOT (((a NOT > b) AND (a NOT > c)) AND (NOT (a NOT > d)))
a > b AND NOT NOT > c OR d	((a > b) AND NOT (a NOT > c)) OR (a NOT > d)

2.6.2.4. Condition Evaluation Rules

Parentheses may be used to specify the order in which individual conditions of complex conditions are to be evaluated when it is necessary to depart from the implied evaluation precedence. Conditions within parentheses are evaluated first and, within nested parentheses, evaluation proceeds from the least inclusive condition to the most inclusive condition. When parentheses are not used, or parenthesized conditions are at the same level of inclusiveness, the following hierarchical order of logical evaluation is implied until the final truth value is determined:

1. Values are established for arithmetic expressions. (See 2.6.1.3.)
2. Truth values for simple conditions are established in the following order:

   relation (following the expansion of any abbreviated relation condition)

   class

   condition-name

   switch-status

   sign
3. Truth values for negated simple conditions are established
4. Truth values for combined conditions are established:

   "AND" logical operators, followed by

   "OR" logical operators.
5. Truth values for negated combined conditions are established.
6. When the sequence of evaluation is not completely specified by parentheses, the order of evaluation of consecutive operations of the same hierarchical level is from left to right.

2.6.3. Common Phrases and General Rules for Statement Formats

In the statement descriptions that follow, several phrases appear frequently: the ROUNDED phrase, the SIZE ERROR phrase, and the CORRESPONDING phrase.

In the discussion which follows, a resultant-identifier is that identifier associated with a result of an arithmetic operation.

2.6.3.1. The ROUNDED Phrase

If, after decimal point alignment, the number of places in the fraction of the result of an arithmetic operation is greater than the number of places provided for the fraction of the resultant-identifier, truncation is relative to the size provided for the resultant-identifier. When rounding is requested, the absolute value of the resultant-identifier is increased by one whenever the most significant digit of the excess is greater than or equal to five.

When the low-order integer positions in a resultant-identifier are represented by the character "P" in the picture for that resultant-identifier, rounding or truncation occurs relative to the rightmost integer position for which storage is allocated.

For example, if addition operands are:

```
02 A PIC 99V99 VALUE 10.51.
02 B PIC 99V9(3) VALUE 96.299.
02 C PIC 999.
02 D PIC 999V9(3).
02 E PIC 999V9.
02 F PIC 999P.
```

and the statements executed are:

```
ADD A , B GIVING C ROUNDED.
ADD A , B GIVING C.
ADD A , B GIVING D ROUNDED.
ADD A , B GIVING D.
ADD A , B GIVING E ROUNDED.
ADD A , B GIVING F ROUNDED.
```

the results will be C = 107, C = 106, D = 106.809, D = 106.809, E = 106.8, and F = 011, respectively.

2.6.3.2. The SIZE ERROR Phrase

If, after decimal point alignment, the absolute value of a result exceeds the largest value that can be contained in the associated resultant-identifier, a size error condition exists. Division by zero always causes a size error condition. The size error condition applies only to the final results of an arithmetic operation and does not apply to intermediate results, except in the MULTIPLY and DIVIDE statements, in which case the size error condition applies to the intermediate results as well. If the ROUNDED phrase is specified, rounding takes place before checking for size error. When such a size error condition occurs, the subsequent action depends on whether or not the SIZE ERROR phrase is specified.

1. If the SIZE ERROR phrase is not specified and a size error condition occurs, the value of those resultant-identifiers affected is undefined. Values of resultant-identifiers for which no size error condition occurs are unaffected by size errors that occur for other resultant-identifiers during execution of this operation.
2. If the SIZE ERROR phrase is specified and a size error condition occurs, then the values of resultant-identifiers affected by the size errors are not altered. Values of resultant-identifiers for which no size error condition occurs are unaffected by size errors that occur for other resultant-identifiers during execution of this operation. After completion of the execution of this operation, the imperative statement in the SIZE ERROR phrase is executed.

For the ADD statement with the CORRESPONDING phrase and the SUBTRACT statement with the CORRESPONDING phrase, if any of the individual operations produces a size error condition, the imperative statement in the SIZE ERROR phrase is not executed until all of the individual additions or subtractions are completed.

For example, if data fields are defined as:

```
02 A PIC 99V9.
02 B PIC 99V9.
02 C PIC 999V9.
```

the statement ADD A, B GIVING C will not produce a size error condition if A and B do indeed contain numeric values. If, however, a sum is being developed such as:

```
ADD A TO B.
```

a size error condition could occur.

Note: *For an item with the USAGE COMPUTATIONAL clause, the test for SIZE ERROR condition uses the maximum decimal value according to the original PICTURE regardless of bit size. For example:*

```
A1 PIC 99 COMP.
```

A1 is allocated 9 bits. A value of 100 can fit in 9 bits but it would cause a size error condition.

For floating-point arithmetic a size error condition will occur if a hardware floating-point interrupt occurs or if a floating-point intermediate result cannot be contained in the designated receiving field. Exponentiation is accomplished in floating-point arithmetic. If a floating-point interrupt occurs during an exponentiation, it is treated as a run-time error condition rather than a size error condition and the execution is terminated. Termination occurs if:

1. *The result has a zero characteristic and a mantissa less than or equal to zero.*
2. *The result has a characteristic less than zero and a nonintegral mantissa.*
3. *The result is such that the mantissa times the LN of the characteristics is greater than 709.089.*

2.6.3.3. The CORRESPONDING Phrase

For this discussion, d_1 and d_2 must each be identifiers that refer to group items. A pair of data items, one from d_1 and one from d_2 correspond if the following conditions exist:

1. A data item in d_1 and a data item in d_2 are not designated by the key word FILLER and have the same data-name and the same qualifiers up to, but not including, d_1 and d_2.
2. At least one of the data items is an elementary data item in the case of a MOVE statement with the CORRESPONDING phrase; and both of the data items are elementary numeric data items in the case of the ADD statement with the CORRESPONDING phrase or the SUBTRACT statement with the CORRESPONDING phrase.

3. The description of d_1 and d_2 must not contain level-number 66, 77, or 88 or the USAGE IS INDEX clause.
4. A data item that is subordinate to d_1 and d_2 and contains a REDEFINES, RENAMES, OCCURS, or USAGE IS INDEX clause is ignored, as well as those data items subordinate to the data item that contains the REDEFINES, OCCURS, or USAGE IS INDEX clause. However, d_1 and d_2 may have REDEFINES or OCCURS clauses or be subordinate to data items with REDEFINES or OCCURS clauses. (See 3.2.1.)

The following example shows the result of a CORRESPONDING addition and move. Assume that the data is defined as:

```
02 GROUP1.
     03 A.
         04 A1 PIC S99.
         04 FILLER PIC X.
         04 A2 PIC S9(4)V99.
     03 B OCCURS 2 TIMES.
         04 B1 PIC 9(3).
         04 B2 PIC SV999.
     03 C.
         04 C1.
             05 C11 PIC PP99.
             05 C12 PIC 9(6) COMP.
         04 C2 PIC 99.

02 GROUP2.
     05 C.
         07 C1.
             09 C11 PIC 9V9(4).
             09 C12 PIC 9(6).
         07 C2.
             09 C21 PIC 9.
             09 C22 PIC 9.
     05 B.
         07 B1 PIC 9(3).
         07 B2 PIC SV999.
     05 A.
         07 A2 COMP-1.
         07 A1 COMP-2.
         07 FILLER PIC X.
```

the statement:

```
ADD CORRESPONDING GROUP1 TO GROUP2.
```

is equivalent to:

```
ADD C11 OF GROUP1 TO C11 OF GROUP2.
ADD C12 OF GROUP1 TO C12 OF GROUP2.
ADD A1 OF GROUP1 TO A1 OF GROUP2.
ADD A2 OF GROUP1 TO A2 OF GROUP2.
```

while the statement:

```
MOVE CORRESPONDING GROUP1 TO GROUP2
```

is equivalent to:

```
MOVE C11 OF GROUP1 TO C11 OF GROUP2.
MOVE C12 OF GROUP1 TO C12 OF GROUP2.
MOVE C2 OF GROUP1 TO C2 OF GROUP2.
MOVE A1 OF GROUP1 TO A1 OF GROUP2.
MOVE A2 OF GROUP1 TO A2 OF GROUP2.
```

B, B1, and B2 are never involved because of the presence of an OCCURS clause on B in GROUP1. The field C2 is moved but not added because C2 in GROUP2 is not an elementary item.

The statement:

```
MOVE CORRESPONDING B OF GROUP1(2) TO B OF GROUP2
```

is equivalent to:

```
MOVE B1 OF GROUP1(2) TO B1 OF GROUP2.
MOVE B2 OF GROUP1(2) TO B2 OF GROUP2.
```

2.6.3.4. The Arithmetic Statements

The arithmetic statements are the ADD, COMPUTE, DIVIDE, MULTIPLY, and SUBTRACT statements. They have several common features.

1. The data descriptions of the operands need not be the same. Any necessary conversion and decimal point alignment is supplied throughout the calculation.
2. The maximum size of each operand is 18 decimal digits. The composite of operands, which is a hypothetical data item resulting from the superimposition of specified operands in a statement aligned on their decimal points (see 2.6.5 for the ADD statement, 2.6.9 for the DIVIDE statement, 2.6.15 for the MULTIPLY statement, and 2.6.20 for the SUBTRACT statement) must not contain more than 18 decimal digits.

2.6.3.5. Overlapping Operands

When a sending and a receiving item in an arithmetic statement or an INSPECT, MOVE, SET, STRING, or UNSTRING statement share a part of their storage areas, the result of the execution of such a statement is undefined.

2.6.3.6. Multiple Results in Arithmetic Statements

The ADD, COMPUTE, DIVIDE, MULTIPLY, and SUBTRACT statements may have multiple results. Such statements behave as though they had been written in the following way:

1. A statement which performs all arithmetic necessary to arrive at the result to be stored in the receiving items, and stores that result in a temporary storage location.
2. A sequence of statements transferring or combining the value of this temporary location with a single result. These statements are considered to be written in the same left–to–right sequence that the multiple results are listed.

The result of the statement:

```
ADD a, b, c TO c, d (c), e
```

is equivalent to:

```
ADD a, b, c GIVING temp
ADD temp TO c
ADD temp TO d (c)
ADD temp TO e
```

where *temp* is an intermediate result item provided by the compiler.

2.6.3.7. Arithmetic Intermediate Results

Whenever an arithmetic expression is resolved or an arithmetic statement is executed, there is an intermediate result which has characteristics determined by the characteristics of the operands and in some cases by the characteristics of the eventual resultant field.

These characteristics are mode (USAGE of the items), sign (the presence or absence of an operational sign), size (the number of digit positions which the items can contain), and scaling (the position of the assumed decimal point). The characteristics determine what values an item may assume and how the calculations involving the item are made.

When the characteristics and the values are not compatible, either a truncation or a size error condition exists. Truncation conditions occur when the value, after decimal alignment, contains excessive low order digits. Size error conditions occur when the value, after decimal alignment, contains excessive high order digits.

The statements ADD, SUBTRACT, MULTIPLY, DIVIDE, and COMPUTE allow a programmer to correct or compensate when either situation occurs by using the ROUNDED and SIZE ERROR options.

Arithmetic expressions in the PERFORM, IF, and SEARCH statements, however, do not provide alternate actions and can produce either left or right truncation of the expression's resolved value.

The characteristics of intermediate results, regardless of the statement type, are described in the following paragraphs.

The compiler deals with arithmetic operations separately for each pair of operands. For example, in the statement:

```
COMPUTE A = B + C - D
```

B and C determine the characteristics of the first intermediate result of the addition. The sum and the field D determine the characteristics of the final intermediate result. Then the characteristics of A are used to determine the correct way to move the result to the final receiving field. Thus, the preceding example is equivalent to:

```
COMPUTE temp1 = B + C

COMPUTE temp2 = temp1 - D

MOVE temp2 TO A
```

and each value is aligned and each result is computed based solely on the characteristics of the operands involved in the intermediate calculation. Only for division are the characteristics of scale and size of the receiving field used to determine the characteristics of the intermediate result.

Arithmetic results are calculated in one of three ways. The method is determined by the USAGE of the fields involved and the operator specified. The three modes of calculation are decimal, binary, and floating-point.

Decimal arithmetic is addition or subtraction which is performed directly on the characters composing the operands:

D1	scaling positions in operand 1
D2	scaling positions in operand 2
DMAX	greater of D1 and D2
NB	number of bytes per word (4)
SB1	starting byte of operand 1 (0-3)
SB2	starting byte of operand 2 (0-3)
SZ1	size of operand 1 in bytes
SZ2	size of operand 2 in bytes
///	remainder from division

Decimal arithmetic is done only if all of the following conditions are met:

1. SZ1 + SB1 ≤ 3 * NB
2. SZ2 + SB2 ≤ 3 * NB
3. (DMAX - D1 + SB1) /// NB + SZ1 ≤ 3 * NB
4. (DMAX - D2 + SB2) /// NB + SZ2 ≤ 3 * NB
5. Both operands must be array synchronized (subscripted items must always begin at the same byte number)
6. Both operands must be DISPLAY.

If either or both operands of any calculation are floating-point (COMPUTATIONAL–1 or COMPUTATIONAL–2) or if exponentiation is required, calculations are performed in floating-point with any nonfloating-point items being converted to floating-point format.

In all other cases calculations are performed in binary and operands which are not in binary format will be converted to binary prior to calculations.

Like scaling and decimal alignment, the mode of calculation is determined separately for each intermediate calculation. For example, assuming:

```
A, B, E, and X are DISPLAY

C, D, and F are COMP
```

the statement:

```
COMPUTE X = A + B - C ** D/(E - F)
```

will be resolved as follows:

1. Decimal mode A + B = *temp1*
2. Convert C and D to floating point
3. Floating mode C ** D = *temp2*
4. Convert E to binary
5. Binary mode E - F = *temp3*
6. Convert *temp3* to floating point
7. Floating mode *temp2*/*temp3* = *temp4*
8. Convert *temp1* to floating point
9. Floating mode *temp1* - *temp4* = *temp5*
10. Convert *temp5* to DISPLAY and store in X

An understanding of the different calculation modes is important when determining the size of intermediate results. When calculations are performed in decimal mode no more than 18 digits of significance is possible and if a calculation would result in exceeding the maximum low order characters are dropped without rounding.

However, since the OS 2200 System is intrinsically a binary rather than a decimal computer, binary fields can actually contain more digits of precision than is indicated by their PICTURE clause. For example, an S9(10) COMPUTATIONAL item is allocated as a 36-bit field. Those 36 bits can contain an absolute value up to 34,359,738,367.

If, during intermediate calculations, such a field contains a value greater than 9,999,999,999 but less than 2^{35}, the full value is used in subsequent intermediate calculations.

Similarly, a full two-word binary number can contain 21 digits of significance. Therefore, intermediate results of binary calculations are held and used in subsequent intermediate calculations as maximum 21-digit capacity items. During calculations involving binary items the compiler will assure that intermediate results are sufficiently large, up to 21 digits, to retain precision and significance.

Floating-point intermediate results are maintained as either single or double precision floating-point items, whichever will suffice to contain the calculated results. The capacity of floating-point data items is sufficiently large for most applications. Single precision floating-point (COMP–1) can represent a binary characteristic (exponent) of 127 through -128 applied to a mantissa (base) of $\pm 2^{27} - 1$.

COMPUTATIONAL–2 fields and double precision floating-point intermediate results can represent a binary characteristic of 1023 through -1024 applied to a mantissa of $\pm 2^{60} - 1$. Floating-point calculations which result in invalid floating-point values are explained in 2.6.3.2, the SIZE ERROR phrase.

A user should be aware of difficulties which may arise because of floating point arithmetic and take corrective action if necessary. When a floating-point intermediate result is converted and stored in a binary or character field, the result may be imprecise in the least significant position. For example, 4 * * 3 might result in 63.999... instead of 64.000... . Therefore, it is advisable, if a process requires a precise answer, to specify ROUNDED for nonfloating-point receiving fields when floating-point arithmetic is done.

The algorithms for calculating the size and assumed decimal point of nonfloating-point arithmetic intermediate results are defined in Table 2-10. The symbols in the table and their meanings are:

N1 and N2

are the number of integer places in operand 1 and 2.

N3

is the number of integer places in the intermediate sum, difference, product, or quotient.

N4

is the number of integer places in the remainder of a division operation.

D1,D2,D3, and D4

are the number of scaling positions in the field. If a field has scale *n,* this means that its integer value is implicitly multiplied by 10^{-n} whenever the field is used. For a decimal field, having scale *n* is equivalent to having *n* decimal places following an assumed decimal point.

NR and DR

are the number of integer and decimal digits in the ultimate receiving field. If there are multiple receiving fields, the largest NR and the largest DR is used. In this case, NR and DR do not necessarily derive from the same receiving field. When the receiving field is a floating point item, NR is assumed to be 18 and DR is 0.

For division, operand1 is the dividend and operand2 is the divisor. For other operations, the use of the operands is not differentiated when determining the size of an intermediate result.

Table 2-10. Size of Intermediate Result Algorithm

Arithmetic Statement and Expression			
Operation	Calculation Mode	Integer Places	Decimal Places
+ - ADD SUBTRACT	Binary Figure 2-1	N3= { N1+1 N2+1 whichever is greater If N3+D3>21 New N3=21-D3	D3= { D1 D2 whichever is greater
	Decimal Figure 2-2	N3= { N1+1 N2+1 whichever is greater If N3+D3>18 New N3=18-D3	D3= { D1 D2 whichever is greater
*	Binary; if N1+N2+D1+D2<21 Figure 2-3	N3= N1+N2	D3= D1+D2
	Binary; if N1+N2+D1+D2>21 (decimal truncation) Figure 2-4	N3= { N1+N2 21 whichever is smaller	D3=21-N3
MULTIPLY	Binary; if N1+N2+D1+D2<=21 Figure 2-5	N3= N1+N2	D3=D1+D2
	Binary; if N1+N2+D1+D2>21 integer truncation if N1+N2+DR>21 Figure 2-6	N3= N1+N2 21-D3 whichever is smaller	D3=DR

Table 2-10. Size of Intermediate Result Algorithm

Arithmetic Statement and Expressions			
Operation	Calculation Mode	Integer Places	Decimal Places
DIVIDE	Binary; if N1 + D2 + DR ≤ 21 Figure 2-7	N3 = N1 + D2 N4 = N3 + D3 – D4	D3 = DR D4 = { D2 + DR, D1 } whichever is greater
	Binary; if N1 + D2 + DR > 21 (decimal truncation) Figure 2-8	N3 = 21 – D3 N4 = 21 – D4	D3 = { 21 – NR, DR } whichever is smaller D4 = { D2 + DR, D1 } whichever is smaller
/ in COMPUTE	Binary; if N1 + D2 + (the larger of DR and D1) ≤ 21 Figure 2-9	N3 = N1 + D2	D3 = { DR, D1 } whichever is greater
	Binary; if N1 + D2 + (the larger of DR and D1) > 21 (integer truncation) Figure 2-10	N3 = 21 – D3	D3 = { DR, D1 } whichever is greater
/ in arithmetic expression other than COMPUTE	Binary Figure 2-11	N3 = 21 – D1	D3 = D1

Integer truncation means that the intermediate result is truncated left-to-right (going past the decimal point if necessary–N3 negative). Decimal truncation means that the intermediate result is truncated right-to-left to the decimal point and then the integer digits are truncated left-to-right if necessary.

If the OBJECT–COMPUTER is UNIVAC–1100–60EIS and the operation is ADD, SUBTRACT, "+", or "-", the size of the intermediate result is > 16, and all rules for decimal arithmetic have been satisfied (see 2.6.3.7), then the operation will be done in binary and the truncation rules for calculation mode of binary will apply.

The following figures are organized according to calculation mode. The shadow area indicates truncation.

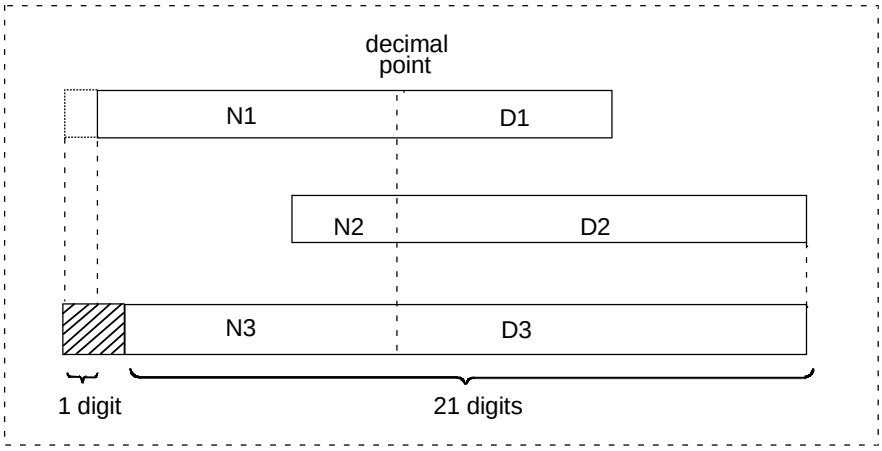


Figure 2-1. +, -, ADD, SUBTRACT for Binary

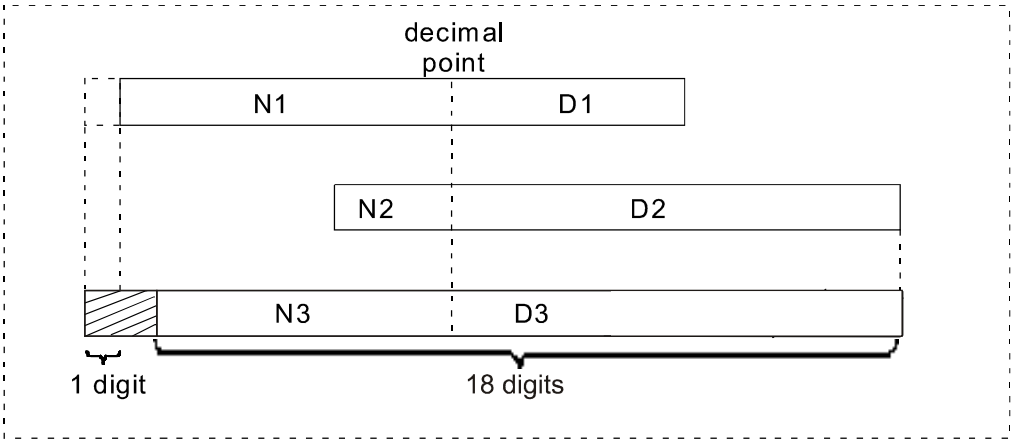


Figure 2-2. +, -, ADD, SUBTRACT for Decimal

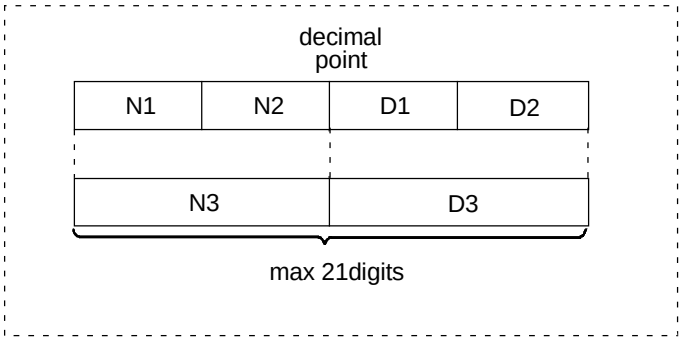


Figure 2-3. * for Binary (N1 + N2 + D1 + D2 ≤ 21)

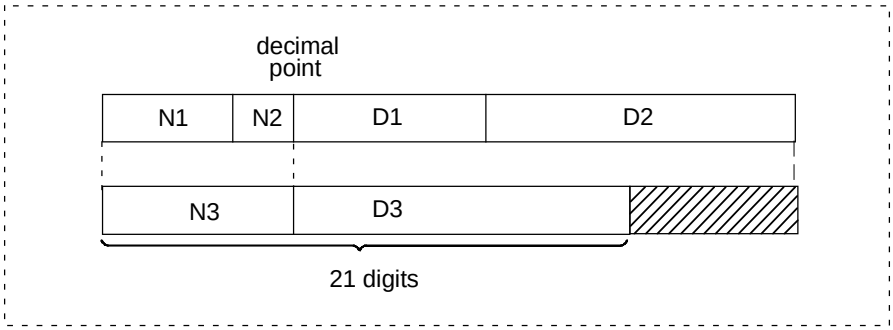


Figure 2-4. * for Binary (N1 + N2 + D1 + D2 > 21)

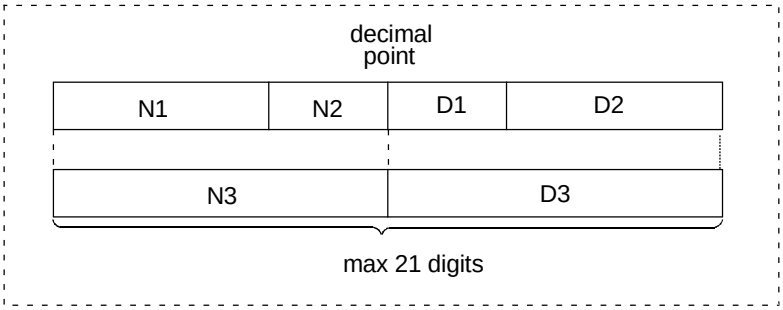
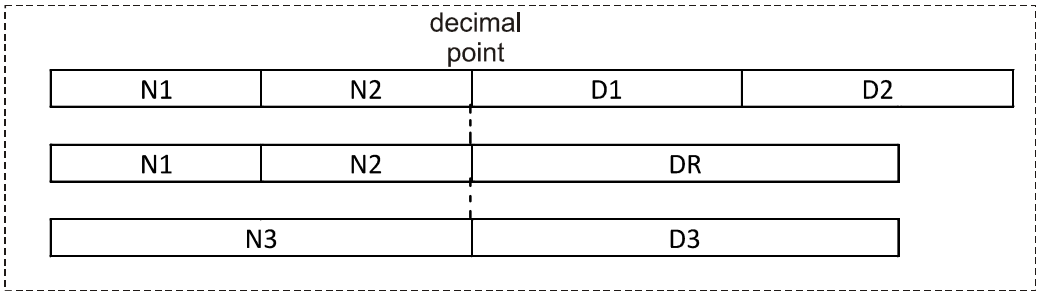


Figure 2-5. MULTIPLY for Binary (N1 + N2 + D1 + D2 ≤ 21)

- If N1 + N2 + DR <= 21



- If N1 + N2 + DR > 21

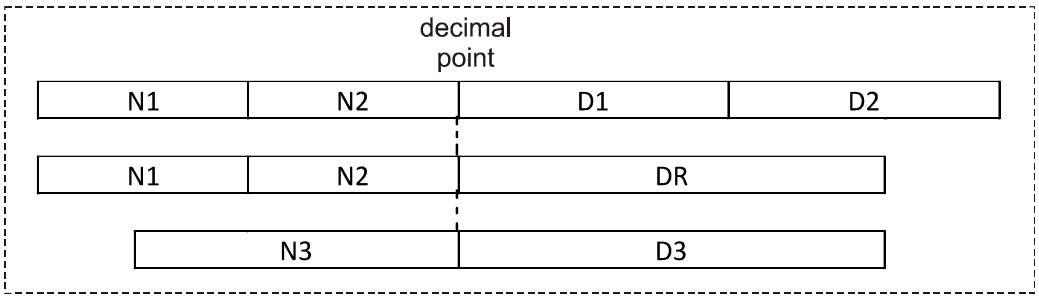


Figure 2-6. MULTIPLY for Binary (N1 + N2 + D1 + D2 > 21)

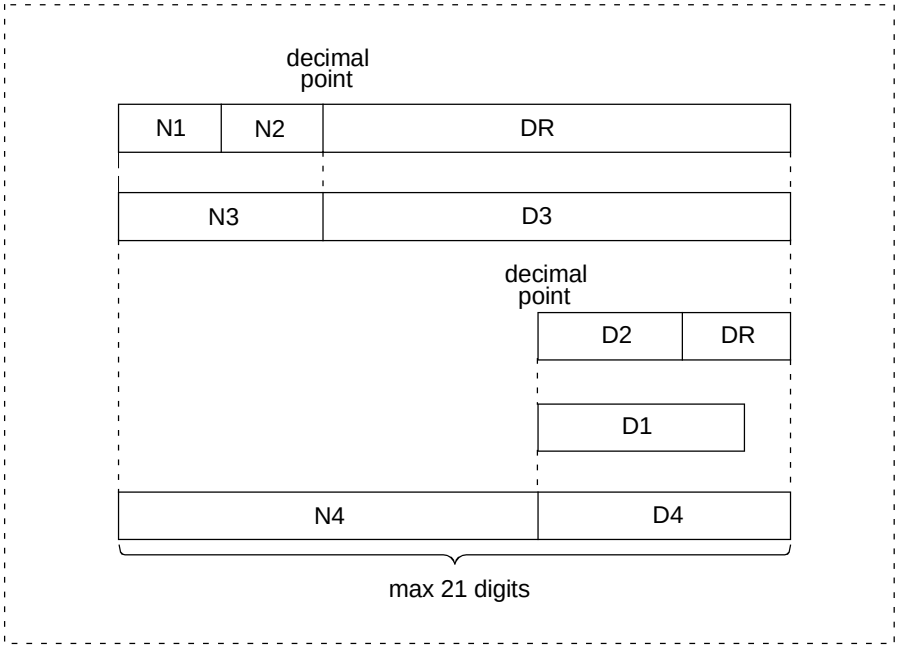


Figure 2-7. DIVIDE for Binary (N1 + D2 + DR ≤ 21)

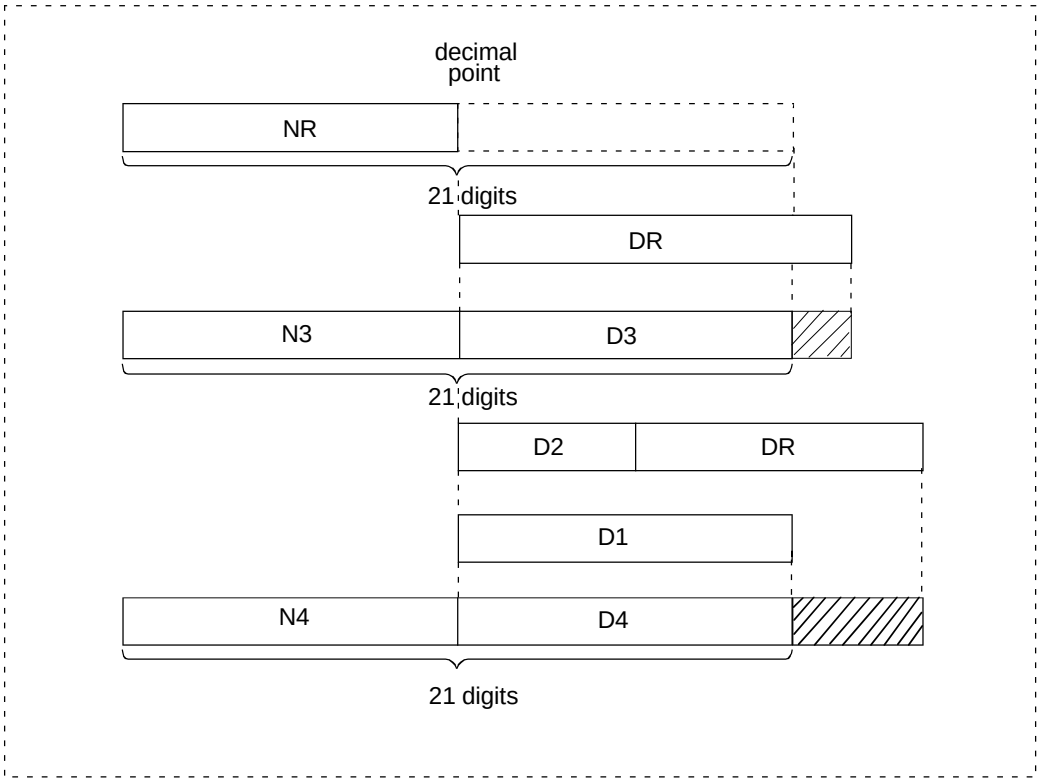


Figure 2-8. DIVIDE for Binary (N1 + D2 + DR > 21)

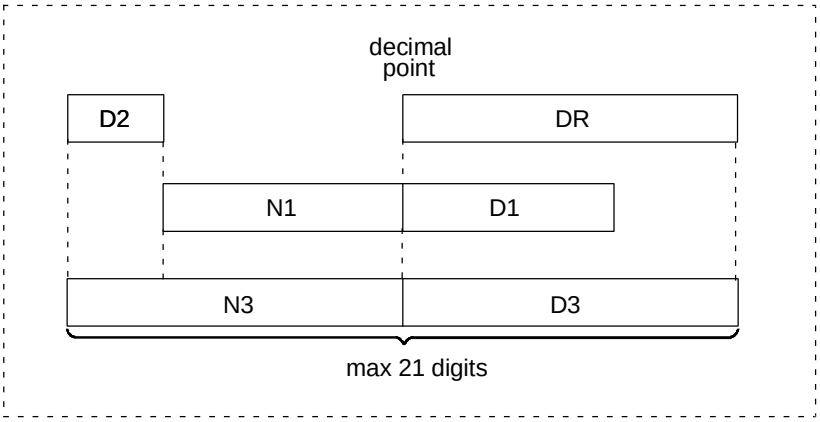


Figure 2-9. / in COMPUTE for Binary (N1 + D2 + (the larger of DR and D1) ≤ 21)

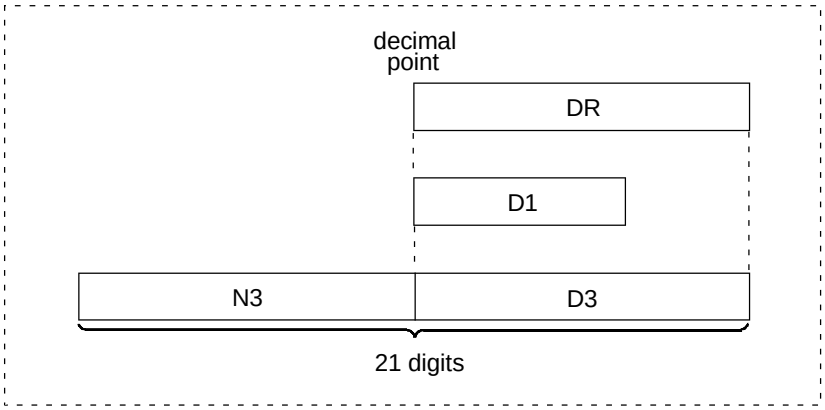


Figure 2-10. / in COMPUTE for Binary (N1 + D2 + (the larger of DR and D1) > 21)

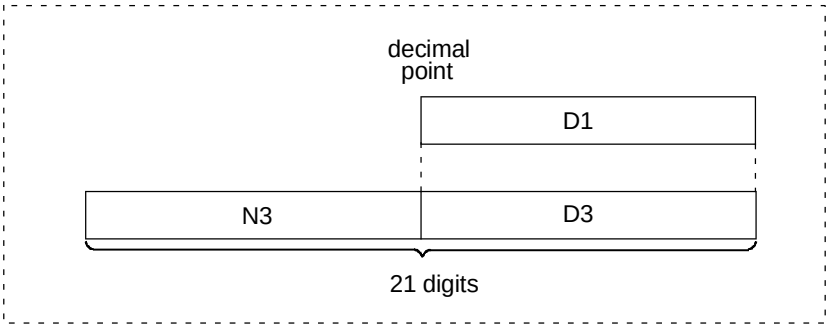


Figure 2-11. / in Arithmetic Expression Other Than COMPUTE

2.6.3.8. Incompatible Data

Except for the class condition (see 2.6.2.1.2) when the contents of a data item are referenced in the Procedure Division and the contents of that data item are not compatible with the class specified for that data item by its PICTURE clause, the result of such a reference is undefined.

2.6.4. The ACCEPT Statement

2.6.4.1. Function

The ACCEPT statement causes low volume data to be made available to the specified data item.

2.6.4.2. General Format

Format 1

```
ACCEPT identifier  [ FROM mnemonic-name ]
```

Format 2

```
ACCEPT identifier FROM { DATE [YYYYMMDD]
                       { DAY  [YYYYDDD]
                       { TIME
                       { DATE-TIME [MMDDYYYY]
```

2.6.4.3. Syntax Rules

1. The *mnemonic-name* in Format 1 must also be specified in the SPECIAL–NAMES paragraph of the Environment Division and must be associated with the implementor-names CARD–READER or CONSOLE.

2.6.4.4. General Rules

Format 1

1. The ACCEPT statement causes the transfer of data from the hardware device. This data replaces the contents of the data item named by the *identifier*.

When the device is the CONSOLE, programs executing in batch mode will accept data from the operator's console. In batch mode CARD–READER specifies the program card file device. For programs executing in demand mode, CONSOLE and CARD–READER both specify the demand terminal.

The execution of an ACCEPT statement specifying the operator's terminal or demand terminal will not permit execution of subsequent instructions until a response is made. It is therefore usually advisable to precede such ACCEPT statements with a DISPLAY to the console or terminal explaining what data is desired.

The presence of the B option on the @XQT statement will cause a program to operate in batch mode for purposes of the ACCEPT statement.

2. The amount of data which can be handled in a single transfer differs with the type of device.

Specified Device Implementor-Name	Size of a Single Data Transfer
CARD–READER	80 characters for programs executing in batch mode 72 characters for programs executing in demand mode
CONSOLE	50 characters for programs executing in batch mode 72 characters for programs executing in demand mode

3. If a hardware device can transfer data of the same size as the receiving data item, the transferred data is stored in the receiving data item.
4. If a hardware device cannot transfer data of the same size as the receiving data item, and the receiving item is alphanumeric, then:
   a. If the size of the receiving data item (or of the portion of the receiving data item not yet currently occupied by transferred data) exceeds the size of the transferred data, the transferred data is stored aligned to the left in the receiving data item (or the portion of the receiving data item not yet occupied), and additional data is requested. In Level 1, only one transfer of data is provided.
   b. If the size of the transferred data exceeds the size of the receiving data item (or of the portion of the receiving data item not yet occupied by transferred data), only the leftmost characters of the transferred data are stored in the receiving data item (or in the portion remaining). The remaining characters of the transferred data which do not fit into the receiving data item are ignored.
5. For a numeric receiving field: data preceding a space in the field is assumed to be an integer value which is converted as necessary, aligned to the right and zero-filled, and treated as an integer for subsequent transfer to the receiving field. A signed data item should be entered as a sign is trailing (not separate) item.
6. The acceptance devices all use character data representation. Thus, input is in ASCII characters. When the receiving field has a usage of other than ASCII the data is converted, then placed in the receiving area.

   The actual value of the accepted data is not examined. Thus, coding a value in the form of a floating point constant will be treated simply as a character string which happens to have the character "E" embedded in it.

7. If the FROM phrase is not given, the CONSOLE is assumed to be the input device.

Format 2

8. The ACCEPT statement causes the information requested to be transferred to the data item specified by identifier according to the rules of the MOVE statement. DATE, DAY, and TIME are conceptual data items and, therefore, are not described in the COBOL program.
9. DATE, without the YYYYMMDD phrase, is composed of the data elements year of century, month of year, and day of month. The sequence of the data element codes shall be from high order to low order (left to right), year of century, month of year, and day of month. Therefore, July 1, 1968 would be expressed as 680701. DATE, without the YYYYMMDD phrase, when accessed by a COBOL program, behaves as if it had been described in the COBOL program as an unsigned elementary numeric integer data item six digits in length.

DATE with the YYYYMMDD phrase is composed of the data elements year including century, month of year, day of month. The sequence of the data element codes shall be from high order to low order (left to right), year including century, month of year and day of month. Therefore, July 1, 1968 would be expressed as 19680701. DATE with the YYYYMMDD phrase when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item eight digits in length.

10. DAY without the YYYYDDD phrase is composed of the data elements year of century and day of year. The sequence of the data element codes shall be from high order to low order (left to right), year of century, and day of year. Therefore, July 1, 1968 would be expressed as 1968183. DAY without the YYYYDDD phrase, when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item five digits in length.

DAY with the YYYYDDD phrase is composed of the data elements year including century and day of year. The sequence of the data element codes shall be from high order to low order (left to right), year including century and day of year. Therefore, July 1, 1968 would be expressed as 1968183. DAY with the YYYYDDD phrase when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item seven digits in length.

11. TIME is composed of the data elements hours, minutes, seconds, and hundredths of a second. TIME is based on elapsed time after midnight on a 24–hour clock basis. Thus, 2:41 p.m. would be expressed as 14410000. TIME, when accessed by a COBOL program behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item eight digits in length. The minimum value of TIME is 00000000; the maximum value of TIME is 23595999.

12. DATE–TIME without the MMDDYYYY phrase is composed of the data elements month of year, day of month, year of century, hour of day, minute of hour, and second of minute. The sequence of the data element codes shall be from high order to low order (left to right), month of year, day of month, year in century, hour of day, minute of hour, and second of minute. Time is based upon elapsed time after midnight on a 24-hour basis. Therefore, 2:41 p.m. July 1, 1998 would be expressed as 070198144100. DATE–TIME without the MMDDYYYY phrase when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item twelve digits in length.

DATE–TIME with the MMDDYYYY phrase is composed of the data elements month of year, day of month, year including century, hour of day, minute of hour, and second of minute. The sequence of the data element codes shall be from high order to low order (left to right), month of year, day of month, year including century, hour of day, minute of hour, and second of minute. Time is based upon elapsed time after midnight on a 24-hour basis. Therefore, 2:41 p.m. July 1, 1998 would be expressed as 07011998144100. DATE–TIME with the MMDDYYYY phrase when accessed by a COBOL program, behaves as if it had been described in a COBOL program as an unsigned elementary numeric integer data item fourteen digits in length.

Please see 18.9 for another method for obtaining a date with a four-digit year.

2.6.5. The ADD Statement

2.6.5.1. Function

The ADD statement causes two or more numeric operands to be summed and the result to be stored.

2.6.5.2. General Format

Format 1

```
ADD { identifier-1 } [ , identifier-2 ] ... TO identifier-m [ ROUNDED ]
    { literal-1    } [ , literal-2    ]

[ , identifier-n [ ROUNDED ] ] ... [ ; ON SIZE ERROR imperative-statement ]
```

Format 2

```
ADD { identifier-1 } , { identifier-2 } [ , identifier-3 ] ...
    { literal-1    }   { literal-2    } [ , literal-3    ]

GIVING identifier-m [ ROUNDED ] [ , identifier-n [ ROUNDED ] ] ...

[ ; ON SIZE ERROR imperative-statement ]
```

Format 3

```
ADD { CORRESPONDING } identifier-1 TO identifier-2 [ ROUNDED ]
    { CORR          }

[ ; ON SIZE ERROR imperative-statement ]
```

2.6.5.3. Syntax Rules

1. In Formats 1 and 2, each *identifier* must refer to an elementary numeric item, except that in Format 2, each *identifier* following the word GIVING must refer to either an elementary numeric item or an elementary numeric–edited item. In Format 3, each *identifier* must refer to a group item.
2. Each *literal* must be a numeric literal.

3. The composite of operands must not contain more than 18 digits (see 2.6.3.4).
   a. In Format 1, the composite of operands is determined by using all of the operands in a given statement.
   b. In Format 2, the composite of operands is determined by using all of the operands in a given statement, excluding the data items that follow the word GIVING.
   c. In Format 3, the composite of operands is determined separately for each pair of corresponding data items.
4. CORR is an abbreviation for CORRESPONDING.

2.6.5.4. General Rules

1. See 2.6.3.1 for the ROUNDED phrase, 2.6.3.2 for the SIZE ERROR phrase, 2.6.3.3 for the CORRESPONDING phrase, 2.6.3.4 for arithmetic statements, 2.6.3.5 for overlapping operands, 2.6.3.6 for multiple results in arithmetic statements, and 2.6.3.7 for arithmetic intermediate results.
2. If Format 1 is used, the values of the operands preceding the word TO are added together, then the sum is added to the current value of *identifier-m* storing the result immediately into *identifier-m,* and repeating this process respectively for each operand following the word TO.
3. If Format 2 is used, the values of the operands preceding the word GIVING are added together, then the sum is stored as the new value of each *identifier-m, identifier-n, ...,* the resultant–identifiers.
4. If Format 3 is used, data items in *identifier-1* are added to and stored in corresponding data items in *identifier-2*.
5. The compiler insures that enough places are carried so as not to lose any significant digits during execution.
6. The individual addends are decimal point aligned prior to addition. The size of such aligned items must not exceed 18 decimal digits.

   For example, a field defined as V9(18) cannot be added to a field defined as 9(18).
7. As long as the aligned size of items does not exceed 18 digits, the compiler ensures that enough places are carried so as not to lose any significant digits during execution.

   For example, a field defined as S9V9(5) and 99V9(4) can be added. The intermediate result can be conceptually considered as S9(3)V9(5), and is therefore large enough to contain the result of any legal values contained in the addends.

2.6.6. The ALTER Statement

2.6.6.1. Function

The ALTER statement modifies a predetermined sequence of operations.

2.6.6.2. General Format

```
ALTER  procedure-name-1  TO  [ PROCEED TO ]  procedure-name-2

       [ ,  procedure-name-3  TO  [ PROCEED TO ]  procedure-name-4 ]  . . .
```

2.6.6.3. Syntax Rules

1. Each *procedure-name-1,* [*procedure-name-3, . . .,*] is the name of a paragraph that contains a single sentence consisting of a GO TO statement without the DEPENDING phrase.
2. Each *procedure-name-2,* [*procedure-name-4, . . .,*] is the name of a paragraph or section in the Procedure Division.

2.6.6.4. General Rules

1. Execution of the ALTER statement modifies the GO TO statement in the paragraph named *procedure-name-1,* [*procedure-name-3, . . .,*] so that subsequent executions of the modified GO TO statement cause transfer of control to *procedure-name-2* [*procedure-name-4, . . .,* respectively.] Modified GO TO statements in independent segments may, under some circumstances, be returned to their initial states (see 9.2.2.3).
2. A GO TO statement in a section whose segment-number is greater than or equal to 50 must not be referred to by an ALTER statement in a section with a different segment–number.

   All other uses of the ALTER statement are valid and are performed even if *procedure-name-1,* [*procedure-name-3*] is in an overlayable fixed segment. (See Section 9.)

For example, in the following code sequence:

```
EX-HDR.
    GO TO SET-MON.
SET-MON.
    .
    .
    .
EX-HDR-A.
    GO TO EX-HDR-REST.
EX-HDR-REST.
    ALTER EX-HDR-A TO PROCEED TO SET-UP-EX EX-HDR TO PROCEED TO EX-HDR-MV.
```

The two GO TO paragraphs might have been switches originally set to fall through to the next paragraph in sequence. After being altered, EX–HDR–A will show GO TO SET–UP–EX, causing control to pass to the procedure–name SET–UP–EX instead of EX–HDR–REST. Upon completion of a specified routine, EX–HDR–A may then be altered back to its original status, or to some other operand by subsequent ALTER verbs. Note that EX–HDR and EX–HDR–A are one sentence paragraphs containing a GO TO statement.

Control is transferred to the next statement in sequence, following the execution of the ALTER statement.

2.6.7. The COMPUTE Statement

2.6.7.1. Function

The COMPUTE statement assigns to one or more data items the value of an arithmetic expression.

2.6.7.2. General Format

```
COMPUTE identifier-1 [ ROUNDED ] [ , identifier-2 [ ROUNDED ] ] . . .

    = arithmetic-expression [ ; ON SIZE ERROR imperative-statement ]
```

2.6.7.3. Syntax Rules

1. Identifiers that appear only to the left of the equal sign must refer to either an elementary numeric item or an elementary numeric-edited item.

2.6.7.4. General Rules

1. See 2.6.3.1 for the ROUNDED phrase, 2.6.3.2 for the SIZE ERROR phrase, 2.6.3.4 for arithmetic statements, 2.6.3.5 for overlapping operands, 2.6.3.6 for multiple results in arithmetic statements, and 2.6.3.7 for arithmetic intermediate results.
2. An arithmetic expression consisting of a single identifier or literal provides a method of setting the values of *identifier-1, identifier-2,* etc., equal to the value of the single identifier or literal. (See 2.6.1.1.)

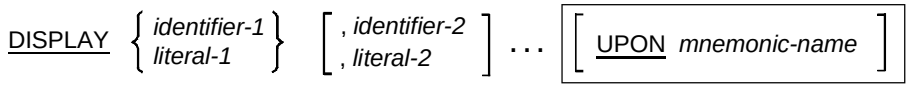
3. If more than one identifier is specified for the result of the operation, that is, preceding the equal sign, the value of the arithmetic expression is computed, and then this value is stored as the new value of each of *identifier-1, identifier-2,* etc., in turn.
4. The COMPUTE statement allows the user to combine arithmetic operations without the restrictions on composite of operands or receiving data items imposed by the arithmetic statements ADD, SUBTRACT, MULTIPLY, and DIVIDE.

2.6.8. The DISPLAY Statement

2.6.8.1. Function

The DISPLAY statement causes low volume data to be transferred to an appropriate hardware device.

2.6.8.2. General Format



2.6.8.3. Syntax Rules

1. The *mnemonic-name* is associated in the SPECIAL–NAMES paragraph in the Environment Division to the implementor-names PRINTER, CONSOLE, and CARD–PUNCH.
2. Each *literal* may be any figurative constant, except ALL.
3. If the *literal* is numeric, then it must be an unsigned integer.

2.6.8.4. General Rules

1. The DISPLAY statement causes the contents of each operand to be transferred to the hardware device in the order listed.
2. For programs executing in batch mode, CONSOLE specifies the operator's console and PRINTER specifies the program print file device. For programs executing in demand mode, CONSOLE and PRINTER both specify the demand terminal. CARD–PUNCH always specifies the program punch file device.

   The presence of the B option on the @XQT statement will cause a program to operate in batch mode for purposes of the DISPLAY statement and PADS (see 11.5).
3. The maximum size of data which can be handled in a single transfer depends on the receiving device.

Device	Size of a Single Data Transfer
PRINTER	
(batch mode)	132
(demand mode)	132
CONSOLE	
(batch mode)	50
(demand mode)	132
CARD–PUNCH	80

4. If a figurative constant is specified as one of the operands, only a single occurrence of the figurative constant is displayed.
5. If the hardware device is capable of receiving data of the same size as the data item being transferred, then the data item is transferred.
6. If the hardware device is not capable of receiving data of the same size as the data item being transferred, then one of the following applies:
   a. If the size of the data item being transferred exceeds the size of the data that the hardware device is capable of receiving in a single transfer, the data beginning with the leftmost character is stored aligned to the left in the receiving hardware device and additional data is requested. The last transfer will be stored aligned to the left and space filled. In Level 1, only one transfer of data is provided.
   b. If the size of the data item that the hardware device is capable of receiving exceeds the size of the data being transferred, the transferred data is stored aligned to the left in the receiving hardware device and space filled.
7. When a DISPLAY statement contains more than one operand, the size of the sending item is the sum of the sizes associated with the operands, and the values of the operands are transferred in the sequence in which the operands are encountered.
8. If the UPON phrase is not used, CONSOLE is assumed.
9. Numeric items which have an assumed decimal point are converted as necessary, and displayed without an indication of the decimal location. Thus, a field defined as 9(5)V9(5) will be displayed as a 10-character value.

   Binary identifiers are converted to the number of digits stated in the PICTURE clause before being output. Exact binary identifiers are converted to the number of decimal digits required by standard binary-decimal equivalence. For example, an item identified as 1(3) will be converted and displayed as an item identified as 9 while an item identified as 1(36) will be converted and displayed as an item identified as 9(10).

   Floating-point identifiers are converted to and displayed as integers with any fractional digits truncated. Thus, a COMPUTATIONAL–1 item will be displayed as a 10-character number and COMPUTATIONAL–2 as an 18-character number.

Each DISPLAY statement begins a new image and continues over as many images as are necessary to display specified items, up to a maximum of 512 characters.

For example:

```
DISPLAY "TOTAL AMOUNT" SPACE TOTAL-AMOUNT
```

results in:

```
TOTAL AMOUNT 04800
```

if the data item TOTAL–AMOUNT contained the value 04800.

The location of the assumed decimal point is not considered. Therefore, the image in the previous example would be the same if TOTAL–AMOUNT were defined as 9(5), 99V999 or even V9(5) with usages DISPLAY, COMPUTATIONAL, or exact binary 1(18).

2.6.9. The DIVIDE Statement

2.6.9.1. Function

The DIVIDE statement divides one numeric data item into others and sets the values of data items equal to the quotient and remainder.

2.6.9.2. General Format

Format 1

DIVIDE { *identifier-1* / *literal-1* } INTO *identifier-2* [ROUNDED]

[, *identifier-3* [ROUNDED]] . . . [; ON SIZE ERROR *imperative-statement*]

Format 2

DIVIDE { *identifier-1* / *literal-1* } INTO { *identifier-2* / *literal-2* } GIVING *identifier-3* [ROUNDED]

[, *identifier-4* [ROUNDED]] . . . [; ON SIZE ERROR *imperative-statement*]

Format 3

DIVIDE { *identifier-1* / *literal-1* } BY { *identifier-2* / *literal-2* } GIVING *identifier-3* [ROUNDED]

[, *identifier-4* [ROUNDED]] . . . [; ON SIZE ERROR *imperative-statement*]

Format 4

DIVIDE { *identifier-1* / *literal-1* } INTO { *identifier-2* / *literal-2* } GIVING *identifier-3* [ROUNDED]

REMAINDER *identifier-4* [; ON SIZE ERROR *imperative-statement*]

Format 5

```
DIVIDE { identifier-1 } BY { identifier-2 } GIVING identifier-3 [ ROUNDED ]
       { literal-1    }    { literal-2    }

    REMAINDER identifier-4 [ ; ON SIZE ERROR imperative-statement ]
```

2.6.9.3. Syntax Rules

1. Each *identifier* must refer to an elementary numeric item, except that any identifier associated with the GIVING or REMAINDER phrase must refer to either an elementary numeric item or an elementary numeric-edited item.
2. Each *literal* must be a numeric literal.
3. The composite of operands, which is the hypothetical data item resulting from the superimposition of all receiving data items (except the REMAINDER data item) of a given statement aligned on their decimal points, must not contain more than eighteen digits.

2.6.9.4. General Rules

1. See 2.6.3.1 for the ROUNDED phrase, 2.6.3.2 for the SIZE ERROR phrase, 2.6.3.4 for arithmetic statements, 2.6.3.5 for overlapping operands, 2.6.3.6 for multiple results in arithmetic statements, and 2.6.3.7 for arithmetic intermediate results. See also general rules 5 through 7 which follow for a discussion of the ROUNDED phrase and the SIZE ERROR phrase as they pertain to Formats 4 and 5.
2. When Format 1 is used, the value of *identifier-1* or *literal-1* is divided into the value of *identifier-2*. The value of the dividend (*identifier-2*) is replaced by this quotient; this is done similarly for *identifier-1* or *literal-1* and *identifier-3,* etc.
3. When Format 2 is used, the value of *identifier-1* or *literal-1* is divided into *identifier-2* or *literal-2* and the result is stored in *identifier-3, identifier-4,* etc.
4. When Format 3 is used, the value of *identifier-1* or *literal-1* is divided by the value of *identifier-2* or *literal-2* and the result is stored in *identifier-3, identifier-4,* etc.
5. Formats 4 and 5 are used when a remainder from the division operation is desired, namely *identifier-4. T*he remainder in ASCII COBOL is defined as the result of subtracting the product of the quotient (*identifier-3*) and the divisor from the dividend. If *identifier-3* is defined as a numeric-edited item, the quotient used to calculate the remainder is an intermediate field which contains the unedited quotient. If ROUNDED is used, the quotient used to calculate the remainder is an intermediate field which contains the quotient of the DIVIDE statement, truncated rather than rounded.
6. In Formats 4 and 5, the accuracy of the REMAINDER data item (*identifier-4*) is defined by the calculation described above. Appropriate decimal alignment and truncation (not rounding) will be performed for the content of the data item referenced by *identifier-4,* as needed.

7. When the ON SIZE ERROR phrase is used in Formats 4 and 5, the following rules pertain:

   a. If the size error occurs on the quotient, no remainder calculation is meaningful. Thus, the contents of the data items referenced by both *identifier-3* and *identifier-4* will remain unchanged.

   b. If the size error occurs on the remainder, the contents of the data item referenced by *identifier-4* remains unchanged. However, as with other instances of multiple results of arithmetic statements, the user will have to do his own analysis to recognize which situation has actually occurred.

In the example:

```
DIVIDE A BY B GIVING C REMAINDER D
```

if the fields are defined as:

```
A: 99V9

B: 9V99

C: 9999V99

D: 9V99
```

and contain only legal numeric values, no truncation or size error condition should arise, except for the zero divisor case.

2.6.10. The EXIT Statement

2.6.10.1. Function

The EXIT statement provides a common end point for a series of procedures.

2.6.10.2. General Format

```
EXIT.
```

2.6.10.3. Syntax Rules

1. The EXIT statement must appear in a sentence by itself.
2. The EXIT sentence must be the only sentence in the paragraph.

2.6.10.4. General Rules

1. An EXIT statement assigns a procedure-name to a given point in a program. An EXIT statement has no other effect on the compilation or execution of the program.

The following example shows the case of the simple EXIT as a common transfer point.

```
        .
        .
        .
       MOVE 1 TO A.
       MOVE 9 TO B.
       PERFORM PARA1 THRU PARA2.
       .
       .
       .
   PARA1.
       IF A = 0 GO TO PARA2.
       ADD 1 TO A.
       IF A = B GO TO PARA2.
       GO TO PARA1.
   PARA2.
       EXIT.
   PARA3.
```

2.6.11. The GO TO Statement

2.6.11.1. Function

The GO TO statement causes control to be transferred from one part of the Procedure Division to another.

2.6.11.2. General Format

Format 1

```
GO TO [ procedure-name-1 ]
```

Format 2

```
GO TO procedure-name-1 [ ,  procedure-name-2  ] . . . , procedure-name-n

      DEPENDING ON identifier
```

2.6.11.3. Syntax Rules

1. *Identifier* is the name of a numeric elementary item described without any positions to the right of the assumed decimal point.
2. When a paragraph is referenced by an ALTER statement, that paragraph can consist only of a paragraph header followed by a Format 1 GO TO statement.

3. A Format 1 GO TO statement, without *procedure-name-1,* can only appear in a single statement paragraph.

4. If a GO TO statement represented by Format 1 appears in a consecutive sequence of imperative statements within a sentence, it must appear as the last statement in that sequence.

2.6.11.4. General Rules

1. When a GO TO statement, represented by Format 1 is executed, control is transferred to *procedure-name-1* or to another procedure–name if the GO TO statement has been modified by an ALTER statement.

2. If *procedure-name-1* is not specified in Format 1, an ALTER statement, referring to this GO TO statement, must be executed prior to the execution of this GO TO statement.

3. When a GO TO statement represented by Format 2 is executed, control is transferred to *procedure-name-1, procedure-name-2,* etc., depending on the value of the identifier being 1, 2, . . . *n.* If the value of the identifier is anything other than the positive or unsigned integers 1, 2, . . . *n,* then no transfer occurs and control passes to the next statement in the normal sequence for execution.

2.6.12. The IF Statement

2.6.12.1. Function

The IF statement causes a condition (see 2.6.1.2) to be evaluated. The subsequent action of the object program depends on whether the value of the condition is true or false.

2.6.12.2. General Format

IF *condition* ; { *statement-1* / NEXT SENTENCE } { ; ELSE *statement-2* / ; ELSE NEXT SENTENCE }

2.6.12.3. Syntax Rules

1. *Statement-1* and *statement-2* represent either an imperative statement or a conditional statement, and either may be followed by a conditional statement.

2. The ELSE NEXT SENTENCE phrase may be omitted if it immediately precedes the terminal period of the sentence.

2.6.12.4. General Rules

1. When an IF statement is executed, the following transfers of control occur:

   a. If the condition is true, *statement-1* is executed if specified. If *statement-1* contains a procedure branching or conditional statement, control is explicitly transferred in accordance with the rules of that statement. (See 1.3.8.2.4.) If *statement-1* does not contain a procedure branching or conditional statement, the ELSE phrase, if specified, is ignored and control passes to the next executable sentence.

b. If the condition is true and the NEXT SENTENCE phrase is specified instead of *statement-1,* the ELSE phrase, if specified, is ignored and control passes to the next executable sentence.

c. If the condition is false, *statement-1* or its surrogate NEXT SENTENCE is ignored, and *statement-2,* if specified, is executed. If *statement-2* contains a procedure branching or conditional statement, control is explicitly transferred in accordance with the rules of that statement. (See 1.3.8.2.4.) If *statement-2* does not contain a procedure branching or conditional statement, control passes to the next executable sentence. If the ELSE *statement-2* phrase is not specified, *statement-1* is ignored and control passes to the next executable sentence.

d. If the condition is false, and the ELSE NEXT SENTENCE phrase is specified, *statement-1* is ignored, if specified, and control passes to the next executable sentence.

2. *Statement-1* and/or *statement-2* may contain an IF statement. In this case the IF statement is said to be nested.

IF statements within IF statements may be considered as paired IF and ELSE combinations, proceeding from left to right. Thus, any ELSE encountered is considered to apply to the immediately preceding IF that has not been already paired with an ELSE.

For example:

```
IF condition-1 statement-1
    IF condition-2
         IF condition-3 statement-2
         ELSE statement-3
     ELSE statement-4
         IF condition-4
              IF condition-5 statement-5
              ELSE statement-6
                  IF condition-6 NEXT SENTENCE
                  ELSE statement-7.
New sentence.
```

The correspondence of IFs and ELSEs is shown by indentation, a frequently used means of making such complex sentences more readable.

The execution of this example, shown in the flowchart in Figure 2-12, assumes that no statement contains a GO TO.

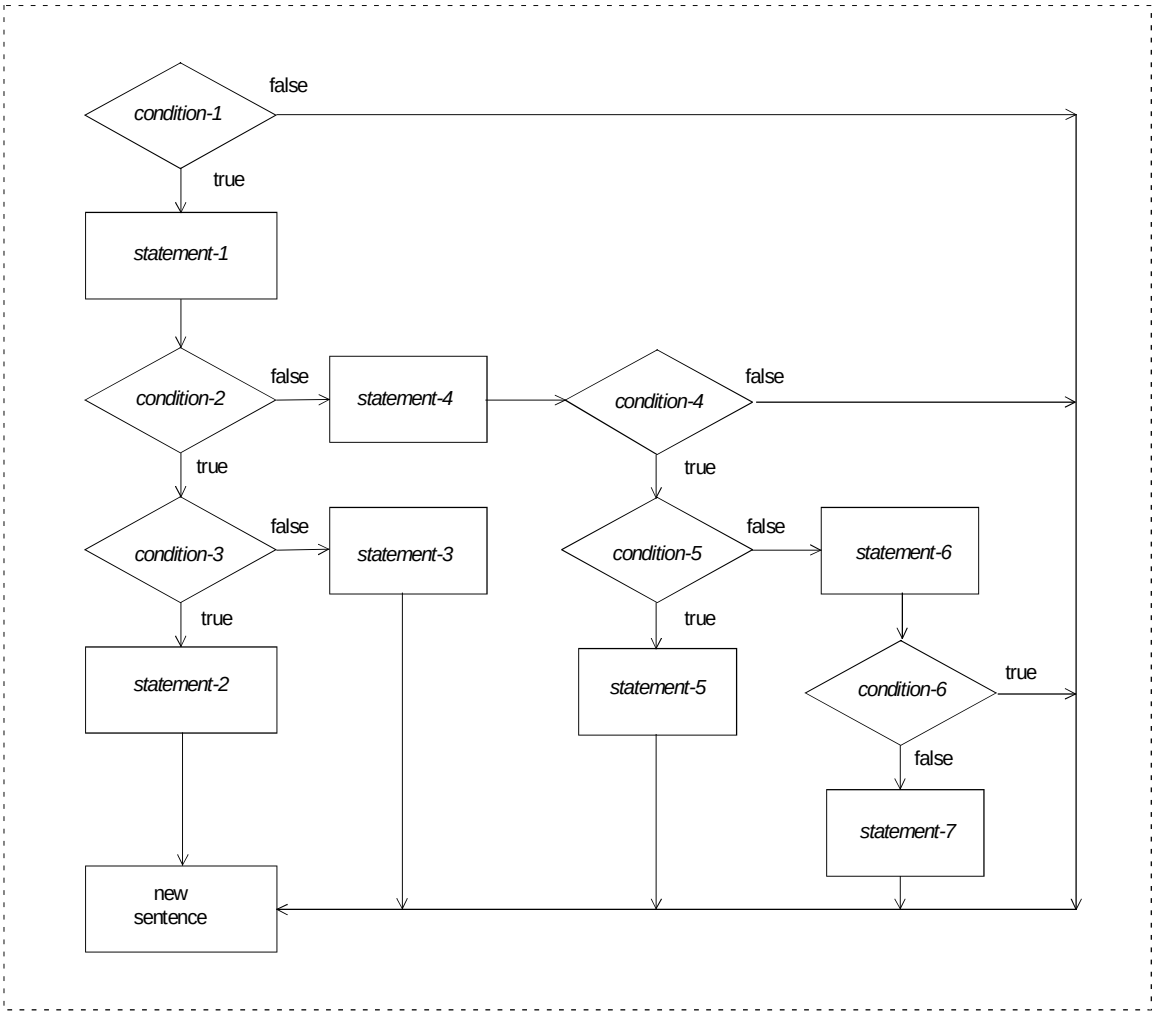


Figure 2-12. Execution Flowchart

2.6.13. The INSPECT Statement

2.6.13.1. Function

The INSPECT statement provides the ability to tally (Format 1), replace (Format 2), or tally and replace (Format 3) occurrences of single characters or groups of characters in a data item.

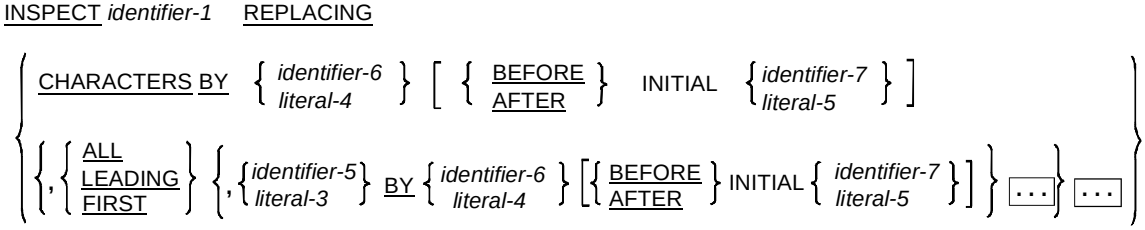
2.6.13.2. General Format

Format 1

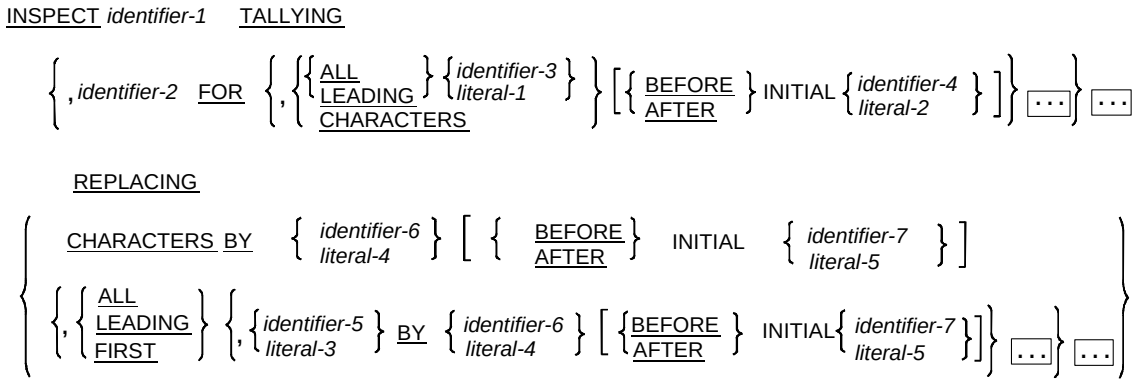
INSPECT *identifier-1* TALLYING

{ , *identifier-2* FOR { , { { ALL / LEADING } { *identifier-3* / *literal-1* } / CHARACTERS } } [{ BEFORE / AFTER } INITIAL { *identifier-4* / *literal-2* }] } ... ...

Format 2



Format 3



2.6.13.3. Syntax Rules

All Formats:

1. *Identifier-1* must reference either a group item or any category of elementary item, described (either implicitly or explicitly) as usage is DISPLAY.
2. *Identifier-3 . . . identifier-n* must reference either an elementary alphabetic, alphanumeric or numeric item described (either implicitly or explicitly) as usage is DISPLAY.
3. Each literal must be nonnumeric and may be any figurative constant, except ALL.
4. In Level 1, *literal-1, literal-2, literal-3, literal-4,* and *literal-5,* and the data items referenced by *identifier-3, identifier-4, identifier-5, identifier-6,* and *identifier-7* must be one character in length.

   Except as specifically noted in syntax and general rules, this restriction on length does not apply to Level 2.

Formats 1 and 3 only:

5. *Identifier-2* must reference an elementary numeric data item.
6. If either *literal-1* or *literal-2* is a figurative constant, the figurative constant refers to an implicit one character data item.

Formats 2 and 3 only:

7. The size of the data referenced by *literal-4* or *identifier-6* must be equal to the size of the data referenced by *literal-3* or *identifier-5.* When a figurative constant is used as *literal-4,* the size of the figurative constant is equal to the size of *literal-3* or the size of the data item referenced by *identifier-5.*
8. When the CHARACTERS phrase is used, *literal-4, literal-5,* or the size of the data item referenced by *identifier-6, identifier-7* must be one character in length. A size greater than one character will be accepted for literal-5 and identifier-7; however, the appropriate leveling diagnostic will be issued.
9. When a figurative constant is used as *literal-3,* the data referenced by *literal-4* or *identifier-6* must be one character in length.

2.6.13.4. General Rules

1. Inspection (which includes the comparison cycle, the establishment of boundaries for the BEFORE or AFTER phrase, and the mechanism for tallying or replacing) begins at the leftmost character position of the data item referenced by *identifier-1,* regardless of its class, and proceeds from left to right to the rightmost character position as described in general rules 4 through 6.
2. For use in the INSPECT statement, the contents of the data item referenced by *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6, or identifier-7* will be treated as follows:
   a. If any of *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6, or identifier-7* are described as alphanumeric, the INSPECT statement treats the contents of each such identifier as a character–string.
   b. If any of *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6, or identifier-7* are described as alphanumeric edited, numeric edited, or unsigned numeric, the data item is inspected as though it had been redefined as alphanumeric (see general rule 2a) and the INSPECT statement had been written to reference the redefined data item.
   c. If any of the *identifier-1, identifier-3, identifier-4, identifier-5, identifier-6, or identifier-7* are described as signed numeric, the data item is inspected as though it had been moved to an unsigned numeric data item of the same length and then the rules in general rule 2b had been applied. (See 2.6.14.)
3. In general rules 4 through 11, all references to *literal-1, literal-2, literal-3, literal-4,* and *literal-5* apply equally to the contents of the data item referenced by *identifier-3, identifier-4, identifier-5, identifier-6,* and *identifier-7,* respectively.
4. During inspection of the contents of the data item referenced by *identifier-1,* each properly matched occurrence of *literal-1* is tallied (Formats 1 and 3) and each properly matched occurrence of *literal-3* is replaced by *literal-4* (Formats 2 and 3).

5. The comparison operation to determine the occurrences of *literal-1* to be tallied and/or occurrences of *literal-3* to be replaced, occurs as follows:

    a. The operands of the TALLYING and REPLACING phrases are considered in the order they are specified in the INSPECT statement from left to right. The first *literal-1, literal-3* is compared to an equal number of contiguous characters, starting with the leftmost character position in the data item referenced by *identifier-1*. *Literal-1, literal-3* and that portion of the contents of the data item referenced by *identifier-1* match if, and only if, they are equal, character for character.

    b. If no match occurs in the comparison of the first *literal-1, literal-3*, the comparison is repeated with each successive *literal-1, literal-3*, if any, until either a match is found or there is no next successive *literal-1, literal-3*. When there is no next successive *literal-1*, *literal-3*, the character position in the data item referenced by *identifier-1* immediately to the right of the leftmost character position considered in the last comparison cycle is considered as the leftmost character position, and the comparison cycle begins again with the first *literal-1, literal-3*.

    c. Whenever a match occurs, tallying and/or replacing takes place as described in the general rules 8 through 10. The character position in the data item referenced by *identifier-1* immediately to the right of the rightmost character position that participated in the match is now considered to be the leftmost character position of the data item referenced by *identifier-1*, and the comparison cycle starts again with the first *literal-1, literal-3*.

    d. The comparison operation continues until the rightmost character position of the data item referenced by *identifier-1* has participated in a match or has been considered as the leftmost character position. When this occurs, inspection is terminated.

    e. If the CHARACTERS phrase is specified, an implied one character operand participates in the cycle described in the preceding paragraphs 5a through 5d, except that no comparison to the contents of the data item referenced by *identifier-1* takes place. This implied character is considered always to match the leftmost character of the contents of the data item referenced by *identifier-1* participating in the current comparison cycle.

6. The comparison operation defined in general rule 5 is affected by the BEFORE and AFTER phrases as follows:

    a. If the BEFORE or AFTER phrase is not specified, *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase participates in the comparison operation as described in general rule 5.

b. If the BEFORE phrase is specified, the associated *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase participates only in those comparison cycles which involve that portion of the contents of the data item referenced by *identifier-1* from its leftmost character position up to, but not including, the first occurrence of *literal-2, literal-5* within the contents of the data item referenced by *identifier-1*. The position of this first occurrence is determined before the first cycle of the comparison operation described in general rule 5 is begun. If, on any comparison cycle, *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase is not eligible to participate, it is considered not to match the contents of the data item referenced by *identifier-1*. If there is no occurrence of *literal-2, literal-5* within the contents of the data item referenced by *identifier-1,* its associated *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase participates in the comparison operation as though the BEFORE phrase had not been specified.

c. If the AFTER phrase is specified, the associated *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase may participate only in those comparison cycles which involve that portion of the contents of the data item referenced by *identifier-1* from the character position immediately to the right of the rightmost character position of the first occurrence of *literal-2, literal-5* within the contents of the data item referenced by *identifier-1* and the rightmost character position of the data item referenced by *identifier-1*. The position of this first occurrence is determined before the first cycle of the comparison operation described in general rule 5 is begun. If, on any comparison cycle, *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase is not eligible to participate, it is considered not to match the contents of the data item referenced by *identifier-1*. If there is no occurrence of *literal-2, literal-5* within the contents of the data item referenced by *identifier-1,* its associated *literal-1, literal-3,* or the implied operand of the CHARACTERS phrase is never eligible to participate in the comparison operation.

Format 1

7. The contents of the data item referenced by *identifier-2* is not initialized by the execution of the INSPECT statement.

8. The rules for tallying are as follows:

   a. If the ALL phrase is specified, the contents of the data item referenced by *identifier-2* is incremented by one for each occurrence of *literal-1* matched within the contents of the data item referenced by *identifier-1*.

   b. If the LEADING phrase is specified, the contents of the data item referenced by *identifier-2* is incremented by one for each contiguous occurrence of *literal-1* matched within the contents of the data item referenced by *identifier-1*, provided that the leftmost such occurrence is at the point where comparison began in the first comparison cycle in which *literal-1* was eligible to participate.

   c. If the CHARACTERS phrase is specified, the contents of the data item referenced by *identifier-2* is incremented by one for each character matched, in the sense of general rule 5e, within the contents of the data item referenced by *identifier-1*.

Format 2

9. The required words ALL, LEADING, and FIRST are adjectives that apply to each succeeding BY phrase until the next adjective appears.

10. The rules for replacement are:

    a. When the CHARACTERS phrase is specified, each character matched, in the sense of general rule 5e, in the contents of the data item referenced by *identifier-1* is replaced by *literal-4*.

    b. When the adjective ALL is specified, each occurrence of *literal-3* matched in the contents of the data item referenced by *identifier-1* is replaced by *literal-4*.

    c. When the adjective LEADING is specified, each contiguous occurrence of *literal-3* matched in the contents of the data item referenced by *identifier-1* is replaced by *literal-4*, provided that the leftmost occurrence is at the point where comparison began in the first comparison cycle in which *literal-3* was eligible to participate.

    d. When the adjective FIRST is specified, the leftmost occurrence of *literal-3* matched within the contents of the data item referenced by *identifier-1* is replaced by *literal-4*.

Format 3

11. A Format 3 INSPECT statement is interpreted and executed as though two successive INSPECT statements specifying the same *identifier-1* had been written with one statement being a Format 1 statement with TALLYING phrases identical to those specified in the Format 3 statement, and the other statement being a Format 2 statement with REPLACING phrases identical to those specified in the Format 3 statement. The general rules given for matching and counting apply to the Format 1 statement and the general rules given for matching and replacing apply to the Format 2 statement.

2.6.13.5. Examples

Following are six examples of the INSPECT statement:

```
INSPECT word TALLYING count FOR LEADING "L" BEFORE INITIAL "A", count-1 FOR LEADING
"A" BEFORE INITIAL "L".
```

Where *word* = LARGE, *count* = 1 and *count–1* = 0.

Where *word* = ANALYST, *count* = 0 and *count–1* = 1.

```
INSPECT word TALLYING count FOR ALL "L", REPLACING LEADING "A" BY "E" AFTER INITIAL "L".
```

Where *word* = CALLAR, *count* = 2 and *word* = CALLAR.

Where *word* = SALAMI, *count* = 1 and *word* = SALEMI.

Where *word* = LATTER, *count* = 1 and *word* = LETTER.

```
INSPECT word REPLACING ALL "A" BY "G" BEFORE INITIAL "X".
```

Where *word* = ARXAX, *word* = GRXAX.

Where *word* = HANDAX, *word* = HGNDGX.

```
INSPECT word TALLYING count FOR CHARACTERS AFTER INITIAL "J" REPLACING ALL "A" BY "B".
```

Where *word* = ADJECTIVE, *count* = 6 and *word* = BDJECTIVE.

Where *word* = JACK, *count* = 3 and *word* = JBCK.

Where *word* = JUJMAB, *count* = 5 and *word* = JUJMBB.

```
INSPECT word REPLACING ALL "X" BY "Yö, "B" BY "Z", "W" BY "Q" AFTER INITIAL "R".
```

Where *word* = RXXBQWY, *word* = RYYZQQY.

Where *word* = YZACDWBR, *word* = YZACDWZR.

Where *word* = RAWRXEB, *word* = RAQRYEZ.

```
INSPECT word REPLACING CHARACTERS BY "B" BEFORE INITIAL "A".
```

word before: `1 2 X Z A B C D`

word after: `B B B B B A B C D`

2.6.14. The MOVE Statement

2.6.14.1. Function

The MOVE statement transfers data, in accordance with the rules of editing, to one or more data areas.

2.6.14.2. General Format

Format 1

MOVE { *identifier-1* / *literal* } TO *identifier-2* [, *identifier-3*] ...

Format 2

MOVE { CORRESPONDING / CORR } *identifier-1* TO *identifier-2*

2.6.14.3. Syntax Rules

1. *Identifier-1* and *literal* represent the sending area; *identifier-2, identifier-3, . . .*, represent the receiving area.
2. CORR is an abbreviation for CORRESPONDING.

3. When the CORRESPONDING phrase is used, both identifiers must be group items.

4. An index data item cannot appear as an operand of a MOVE statement. (See 3.2.2.)

2.6.14.4. General Rules

1. If the CORRESPONDING phrase is used, selected items within *identifier-1* are moved to selected items within *identifier-2,* according to the rules given in 2.6.3.3. The results are the same as if the user had referred to each pair of corresponding identifiers in separate MOVE statements.

2. The data designated by *literal* or *identifier-1* is moved first to *identifier-2,* then to *identifier-3,* . . . . The rules referring to *identifier-2* also apply to the other receiving areas. Any subscripting or indexing associated with *identifier-2,* . . ., is evaluated immediately before the data is moved to the respective data item.

   Any subscripting or indexing associated with *identifier-1* is evaluated only once, immediately before data is moved to the first of the receiving operands. The result of the statement:

   ```
   MOVE a (b) TO b, c (b)
   ```

   is equivalent to:

   ```
   MOVE a (b) TO temp
   MOVE temp TO b
   MOVE temp TO c (b)
   ```

   where *temp* is an intermediate result item provided by the compiler.

3. Any move in which the sending and receiving items are both elementary items is an elementary move. Every elementary item belongs to one of the following categories: numeric, alphabetic, alphanumeric, numeric edited, alphanumeric edited. These categories are described in 2.5.7. Numeric literals belong to the category numeric, and nonnumeric literals belong to the category alphanumeric. The figurative constant ZERO belongs to the category numeric. The figurative constant SPACE belongs to the category alphabetic. All other figurative constants belong to the category alphanumeric.

   The following rules apply to an elementary move between these categories:

   a. The figurative constant SPACE, a numeric edited, alphanumeric edited, or alphabetic data item must not be moved to a numeric or numeric–edited data item.

   b. A numeric literal, the figurative constant ZERO, a numeric data item or a numeric-edited data item must not be moved to an alphabetic data item.

   c. A noninteger numeric literal or a noninteger numeric data item must not be moved to an alphanumeric or alphanumeric-edited data item.

   d. All other elementary moves are legal and are performed according to the rules given in general rule 4.

4. Any necessary conversion of data from one form of internal representation to another takes place during legal elementary moves, along with any editing specified for the receiving data item:

    a. When an alphanumeric-edited or alphanumeric item is a receiving item, alignment and any necessary space filling takes place as defined in 1.3.3.3.6. If the size of the sending item is greater than the size of the receiving item, the excess characters are truncated on the right after the receiving item is filled. If the sending item is described as being signed numeric, the operational sign will not be moved; if the operational sign occupied a separate character position (see 2.5.10), that character will not be moved and the size of the sending item will be considered to be one less than its actual size (in terms of standard data format characters).

    b. When a numeric or numeric-edited item is the receiving item, alignment by decimal point and any necessary zero filling takes place as defined in 1.3.3.3.6, except where zeroes are replaced because of editing requirements.

        1) When a signed-numeric item is the receiving item, the sign of the sending item is placed in the receiving item. (See 2.5.10.) Conversion of the representation of the sign takes place as necessary. If the sending item is unsigned, a positive sign is generated for the receiving item.

        2) When an unsigned-numeric item is the receiving item, the absolute value of the sending item is moved and no operational sign is generated for the receiving item.

        3) When a data item described as alphanumeric is the sending item, data is moved as if the sending item were described as an unsigned numeric integer.

    c. When a receiving field is described as alphabetic, justification and any necessary space-filling takes place as defined in 1.3.3.3.6. If the size of the sending item is greater than the size of the receiving item, the excess characters are truncated on the right after the receiving item is filled.

5. Any move that is not an elementary move is treated exactly as if it were an alphanumeric to alphanumeric elementary move, except that there is no conversion of data from one form of internal representation to another. In such a move, the receiving area will be filled without consideration for the individual elementary or group items contained within either the sending or receiving area, except as noted in general rule 4 of the OCCURS clause (see 3.2.1.4).

6. Data in the following chart summarizes the legality of the various types of MOVE statements. The general rule reference indicates the rule that prohibits the move or the behavior of a legal move.

Category Of Sending Data Item	Category of Receiving Data Item		
	Alphabetic	Alphanumeric Edited Alphanumeric	Numeric Integer Numeric Noninteger Numeric Edited
Alphabetic	Yes/4c	Yes/4a	No/3a
Alphanumeric	Yes/4c	Yes/4a	Yes/4b
Alphanumeric Edited	Yes/4c	Yes/4a	No/3a
Numeric Integer	No/3b	Yes/4a	Yes/4b
Numeric Noninteger	No/3b	No/3c	Yes/4b
Numeric Edited	No/3b	Yes/4a	No/3a

The following table shows the results of elementary moves:

Source Area		Receiving Area	
Picture	Value	Picture	Value After Moving
9V9	12	99V99	0120
9V999	8765	V99	76
9999V99	567891	9999V99	567891
9999V99	567891	9999V9	56789
9V9	78	999V99	00780
XXX	M8N	XXXXX	M8NΔΔ
99V99	6789	999.99	067.89
AAAAAA	WARREN	AAA	WAR
99V99	6789	$$$99.99	ΔΔ$67.89

7. If the last byte is partially filled as in the case of exact binary items, the residue bits will be undefined.
8. The size of a group item will include slack bits or bytes if any. Therefore, if a group contains mixed usages, synchronized, justified, exact binary, floating-point (COMPUTATIONAL–1, or COMPUTATIONAL–2), or dimensioned items, it is advisable to refer to Appendix D and construct moves which will handle the slack areas properly.

2.6.15. The MULTIPLY Statement

2.6.15.1. Function

The MULTIPLY statement causes numeric data items to be multiplied and sets the values of data items equal to the results.

2.6.15.2. General Format

Format 1

```
MULTIPLY { identifier-1 } BY identifier-2 [ ROUNDED ]
         { literal-1    }

    [, identifier-3 [ ROUNDED ] ] ...   [; ON SIZE ERROR imperative-statement ]
```

Format 2

```
MULTIPLY { identifier-1 } BY { identifier-2 } GIVING identifier-3 [ ROUNDED ]
         { literal-1    }    { literal-2    }

    [, identifier-4 [ ROUNDED ] ] ...   [ ; ON SIZE ERROR imperative-statement ]
```

2.6.15.3. Syntax Rules

1. Each identifier must refer to a numeric elementary item, except that in Format 2 each identifier following the word GIVING must refer to either an elementary numeric item or an elementary numeric edited item.
2. Each literal must be a numeric literal.
3. The composite of operands, which is that hypothetical data item resulting from the superimposing of all receiving data items of a given statement aligned on their decimal points, must not contain more than 18 digits.

2.6.15.4. General Rules

1. See 2.6.3.1 for the ROUNDED phrase, 2.6.3.2 for the SIZE ERROR phrase, 2.6.3.4 for arithmetic statements, 2.6.3.5 for overlapping operands, 2.6.3.6 for multiple results in arithmetic statements, and 2.6.3.7 for arithmetic intermediate results.
2. When Format 1 is used, the value of *identifier-1* or *literal-1* is multiplied by the value of *identifier-2*. The value of the multiplier (*identifier-2*) is replaced by this product; similarly for *identifier-1* or *literal-1* and *identifier-3,* etc.
3. When Format 2 is used, the value of *identifier-1* or *literal-1* is multiplied by *identifier-2* or *literal-2* and the result is stored in *identifier-3, identifier-4,* etc.

2.6.16. The PERFORM Statement

2.6.16.1. Function

The PERFORM statement is used to transfer control explicitly to one or more procedures and to return control implicitly whenever execution of the specified procedure is complete.

2.6.16.2. General Format

Format 1

```
PERFORM procedure-name-1 [ { THROUGH } procedure-name-2 ]
                         { THRU    }
```

Format 2

```
PERFORM procedure-name-1 [ { THROUGH } procedure-name-2 ] { identifier-1 } TIMES
                         { THRU    }                     { literal-1    }
```

Format 3

```
PERFORM procedure-name-1 [ { THROUGH } procedure-name-2 ] UNTIL condition-1
                         { THRU    }
```

Format 4

```
PERFORM procedure-name-1 [ { THROUGH } procedure-name-2 ]
                         { THRU    }

    VARYING { identifier-2 } FROM { identifier-3 }
            { index-name-1 }      { index-name-2 }
                                  { literal-1    }

        BY { identifier-4 } UNTIL condition-1
           { literal-2    }

  [ AFTER { identifier-5 } FROM { identifier-6 }
          { index-name-3 }      { index-name-4 }
                                { literal-3    }

        BY { identifier-7 } UNTIL condition-2
           { literal-4    }

  [ AFTER { identifier-8 } FROM { identifier-9 }
          { index-name-5 }      { index-name-6 }
                                { literal-5    }

        BY { identifier-10 } UNTIL condition-3 ] ]
           { literal-6     }
```

2.6.16.3. Syntax Rules

1. Each identifier represents a numeric elementary item described in the Data Division. In Format 2, *identifier-1* must be described as a numeric integer.
2. Each literal represents a numeric literal.
3. The words THRU and THROUGH are equivalent.
4. If an index-name is specified in the VARYING or AFTER phrase, then:
    a. The identifier in the associated FROM and BY phrases must be an integer data item.
    b. The literal in the associated FROM phrase must be a positive integer.
    c. The literal in the associated BY phrase must be a nonzero integer.
5. If an index–name is specified in the FROM phrase, then:
    a. The identifier in the associated VARYING or AFTER phrase must be an integer data item.
    b. The identifier in the associated BY phrase must be an integer data item.
    c. The literal in the associated BY phrase must be an integer.
6. Literal in the BY phrase must not be zero.
7. *Condition-1, condition-2, condition-3* may be any conditional expression as described in 2.6.2.1.3.
8. Where *procedure-name-1* and *procedure-name-2* are both specified and either is the name of a procedure in the declarative section of the program then both must be procedure-names in the same declarative section.

2.6.16.4. General Rules

1. The data items referenced by *identifier-4*, *identifier-7*, and *identifier-10* must not have a zero value.
2. If an index-name is specified in the VARYING or AFTER phrase, and an identifier is specified in the associated FROM phrase, then the data item referenced by the identifier must have a positive value.
3. When the PERFORM statement is executed, control is transferred to the first statement of the procedure named *procedure-name-1* (except as indicated in general rules 6b, 6c, and 6d). This transfer of control occurs only once for each execution of a PERFORM statement. For those cases where a transfer of control to the named procedure does take place, an implicit transfer of control to the next executable statement following the PERFORM statement is established as follows:
    a. If *procedure-name-1* is a paragraph-name and *procedure-name-2* is not specified, then the return is after the last statement of *procedure-name-1*.
    b. If *procedure-name-1* is a section-name and *procedure-name-2* is not specified, then the return is after the last statement of the last paragraph in *procedure-name-1*.

c. If *procedure-name-2* is specified and it is a paragraph-name, then the return is after the last statement of the paragraph.

d. If *procedure-name-2* is specified and it is a section-name, then the return is after the last statement of the last paragraph in the section.

4. There is no necessary relationship between *procedure-name-1* and *procedure-name-2* except that a consecutive sequence of operations is to be executed beginning at the procedure named *procedure-name-1* and ending with the execution of the procedure named *procedure-name-2*. In particular, GO TO and PERFORM statements may occur between *procedure-name-1* and the end of *procedure-name-2*. If there are two or more logical paths to the return point, then *procedure-name-2* may be the name of a paragraph consisting of the EXIT statement, to which all of these paths must lead.

5. If control passes to these procedures by means other than a PERFORM statement, control will pass through the last statement of the procedure to the next executable statement as if no PERFORM statement mentioned these procedures.

6. The PERFORM statements operate as follows with rule 5 applying to all formats:

   a. Format 1 is the basic PERFORM statement. A procedure referenced by this type of PERFORM statement is executed once and then control passes to the next executable statement following the PERFORM statement.

   b. Format 2 is the PERFORM. . . TIMES. The procedures are performed the number of times specified by *integer-1* or by the initial value of the data item referenced by *identifier-1* for that execution. If, at the time of execution of a PERFORM statement, the value of the data item referenced by *identifier-1* is equal to zero or is negative, control passes to the next executable statement following the PERFORM statement. Following the execution of the procedures the specified number of times, control is transferred to the next executable statement following the PERFORM statement.

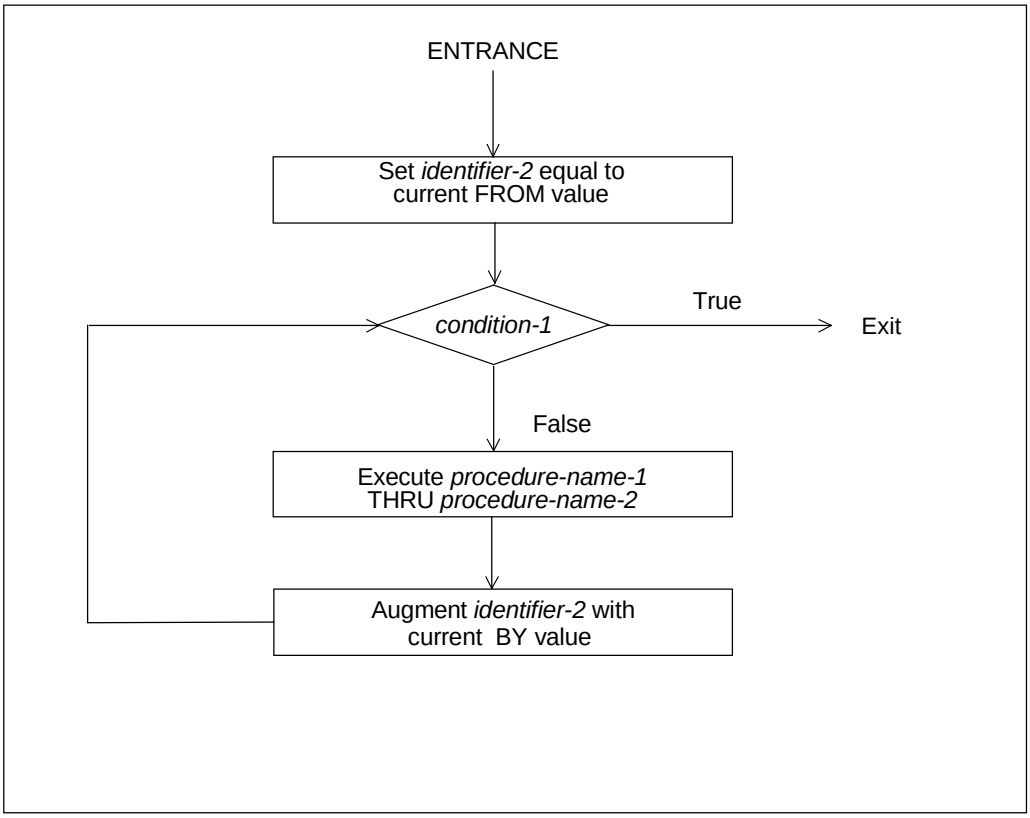
      During execution of the PERFORM statement, references to *identifier-1* cannot alter the number of times the procedures are to be executed from that which was indicated by the initial value of *identifier-1*.

   c. Format 3 is the PERFORM. . . UNTIL. The specified procedures are performed until the condition specified by the UNTIL phrase is true. When the condition is true, control is transferred to the next executable statement after the PERFORM statement. If the condition is true when the PERFORM statement is entered, no transfer to *procedure-name-1* takes place, and control is passed to the next executable statement following the PERFORM statement.

   d. Format 4 is the PERFORM. . . VARYING. This variation of the PERFORM statement is used to augment the values referenced by one or more identifiers or index–names in an orderly fashion during the execution of a PERFORM statement. In the following discussion every reference to identifier as the object of the VARYING, AFTER, and FROM (current value) phrases also refers to index–names. When index–name appears in a VARYING or AFTER phrase, it is initialized and subsequently augmented (as described in the following) according to the rules of the SET statement. When index–name appears in the FROM phrase, identifier, when it appears in an associated VARYING or AFTER phrase, is initialized according to the rules of the SET statement; subsequent augmentation is as described in the following.

In Format 4, when one identifier is varied, *identifier-2* is set to the value of *literal-1* or the current value of *identifier-3* at the point of initial execution of the PERFORM statement; then, if the condition of the UNTIL phrase is false, the sequence of procedures, *procedure-name-1* through *procedure-name-2*, is executed once. The value of *identifier-2* is augmented by the specified increment or decrement value (the value of *identifier-4* or *literal-2*) and *condition-1* is evaluated again. The cycle continues until this condition is true, at which point, control is transferred to the next executable statement following the PERFORM statement. If *condition-1* is true at the beginning of execution of the PERFORM statement, control is transferred to the next executable statement following the PERFORM statement.

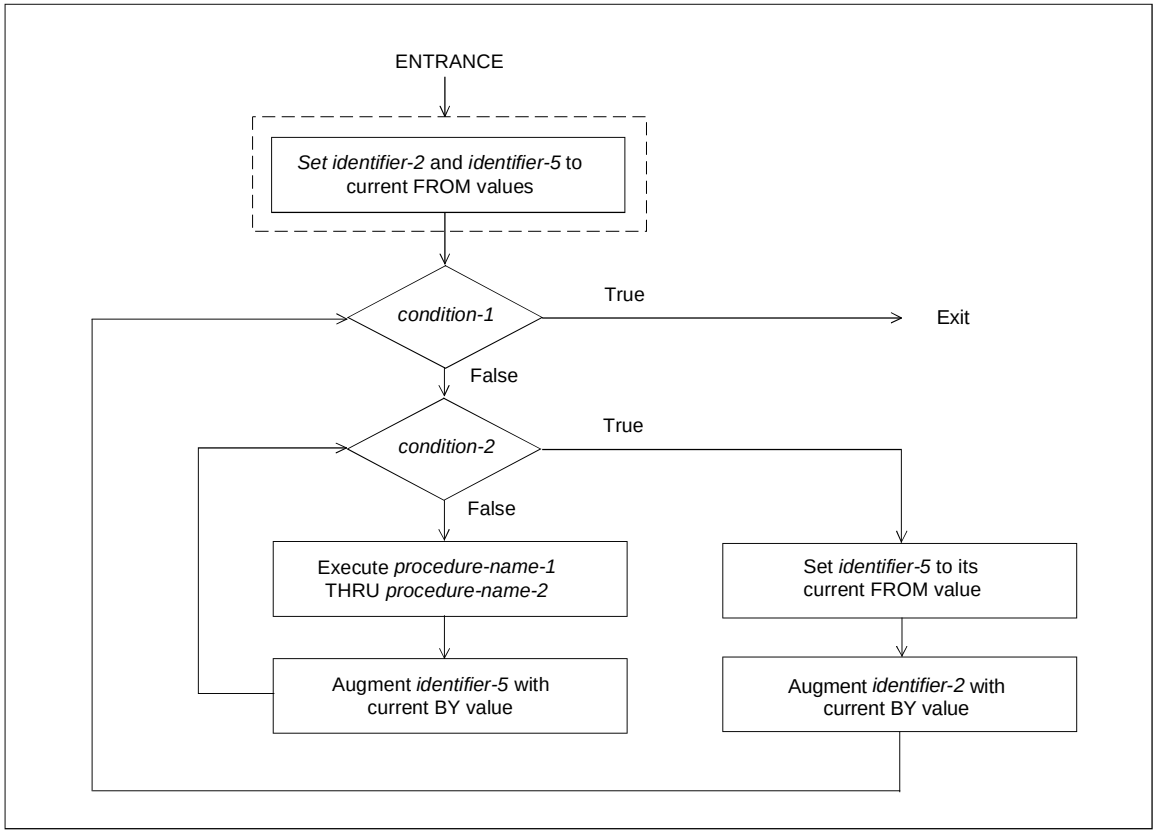
Flowchart for the VARYING phrase of a PERFORM statement having one condition:



In Format 4, when two identifiers are varied, *identifier-2* and *identifier-5* are set to the current value of *identifier-3* and *identifier-6,* respectively. After the identifiers have been set, *condition-1* is evaluated; if true, control is transferred to the next executable statement; if false, *condition-2* is evaluated. If *condition-2* is false, *procedure-name-1* through *procedure-name-2* is executed once, then *identifier-5* is augmented by *identifier-7* or *literal-4* and *condition-2* is evaluated again. This cycle of evaluation and augmentation continues until this condition is true. When *condition-2* is true, *identifier-5* is set to the value of *literal-3* or the current value of *identifier-6, identifier-2* is augmented by *identifier-4* and *condition-1* is reevaluated. The PERFORM statement is completed if *condition-1* is true; if it is not true, the cycles continue until *condition-1* is true.

During the execution of the procedures associated with the PERFORM statement, any change to the VARYING variable (*identifier-2* and *index-name-1*), the BY variable (*identifier-4*), the AFTER variable (*identifier-5* and *index-name-3*), or the FROM variable (*identifier-3* and *index-name-2*) will be taken into consideration and will affect the operation of the PERFORM statement.

Flowchart for the VARYING phrase of a PERFORM statement having two conditions:

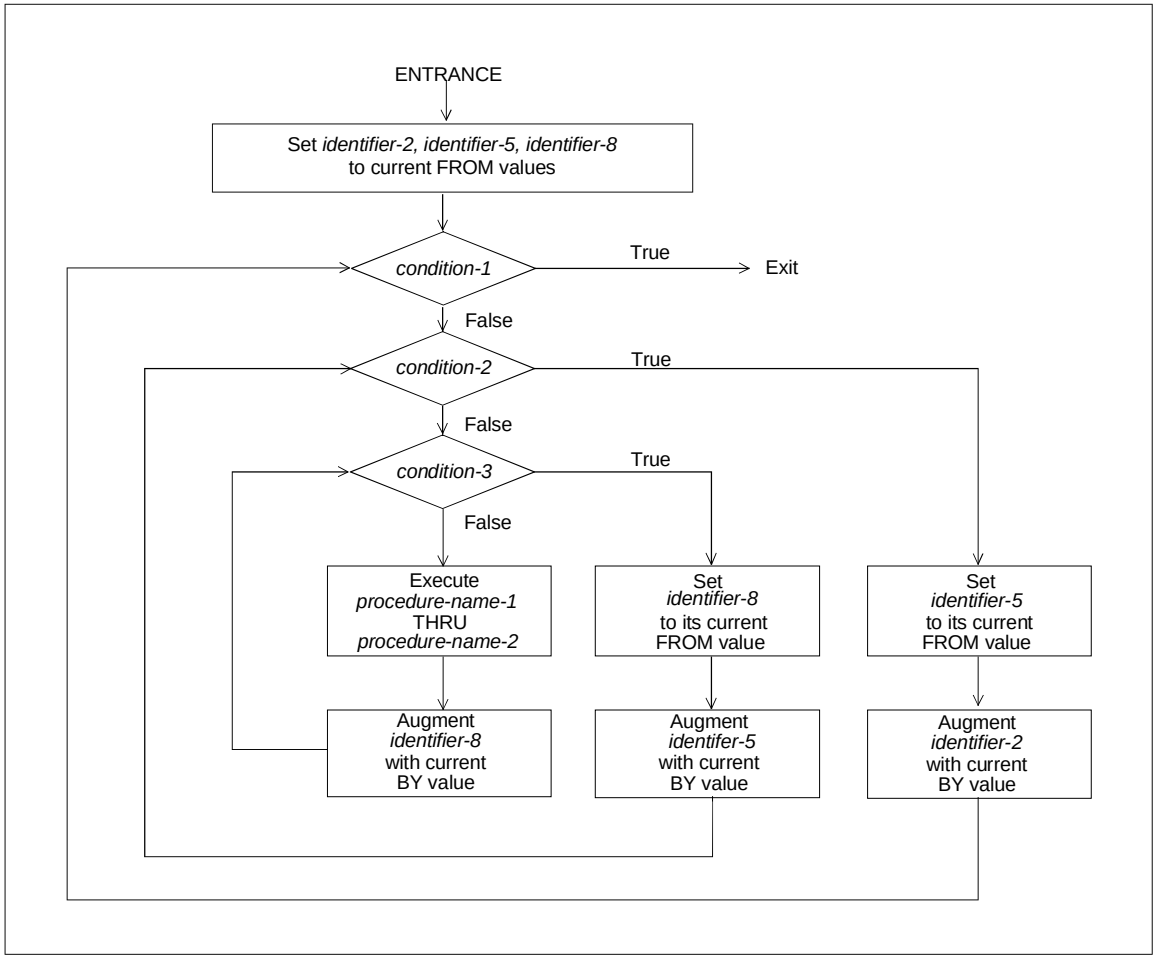


At the termination of the PERFORM statement *identifier-5* contains the current value of *identifier-6*. *Identifier-2* has a value that exceeds the last used setting by an increment or decrement value, unless *condition-1* was true when the PERFORM statement was entered, in which case *identifier-2* contains the current value of *identifier-3*.

When two identifiers are varied, *identifier-5* goes through a complete cycle (FROM, BY, UNTIL) each time *identifier-2* is varied.

For three identifiers the mechanism is the same as for two identifiers except that *identifier-8* goes through a complete cycle each time that *identifier-5* is augmented by *identifier-7* or *literal-4,* which in turn goes through a complete cycle each time *identifier-2* is varied.

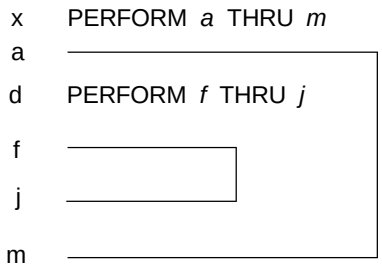
Flowchart for the VARYING phrase of a PERFORM statement having three conditions:

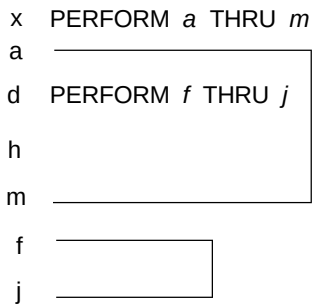


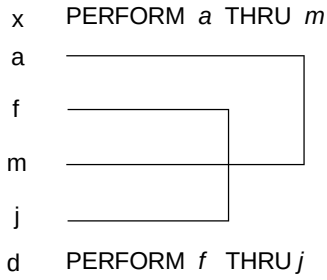
After the completion of a Format 4 PERFORM statement, *identifier-5* and *identifier-8* contain the current value of *identifier-6* and *identifier-9,* respectively. *Identifier-2* has a value that exceeds its last used setting by one increment or decrement value, unless *condition-1* is true when the PERFORM statement is entered, in which case *identifier-2* contains the current value of *identifier-3.*

If a sequence of statements referred to by a PERFORM statement includes another PERFORM statement, the sequence of procedures associated with the included PERFORM must itself either be totally included in, or totally excluded from, the logical sequence referred to by the first PERFORM. Thus, an active PERFORM statement, whose execution point begins within the range of another active PERFORM statement, must not allow control to pass to the exit of the other active PERFORM statement; furthermore, two or more active PERFORM statements may not have a common exit.

This is illustrated by:







8. A PERFORM statement that appears in a section that is not in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:

   a. Sections or paragraphs wholly contained in one or more nonindependent segments.

   b. Sections and/or paragraphs wholly contained in a single independent segment.

9. A PERFORM statement that appears in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:

   a. Sections and/or paragraphs wholly contained in one or more nonindependent segments.

   b. Sections and/or paragraphs wholly contained in the same independent segment as that PERFORM statement.

2.6.17. The SET Statement

2.6.17.1. Function

The SET *mnemonic-name* statement is used to alter the status of external switches.

2.6.17.2. General Format

SET *mnemonic-name* TO { ON / OFF }

2.6.17.3. Syntax Rules

1. The *mnemonic–name* must be associated with an external switch, the status of which can be altered. SWITCH–13, SWITCH–14, . . ., SWITCH–24 can be referenced by the SET statement. See the SPECIAL–NAMES paragraph.
2. ON indicates set a bit in T3 of the program condition word.
3. OFF indicates clear a bit in T3 of the program condition word.

2.6.17.4. General Rules

1. The status of each external switch associated with the specified *mnemonic–name* is modified such that the truth value resultant from evaluation of a condition–name associated with that switch will reflect an "ON" status if the ON phrase is specified, or an "OFF" status if the OFF phrase is specified. (See 2.6.2.1.4.)

2.6.18. The STOP Statement

2.6.18.1. Function

The STOP statement causes a permanent or temporary suspension of the execution of the object program.

2.6.18.2. General Format

STOP { RUN / *literal* }

2.6.18.3. Syntax Rules

1. The *literal* may be numeric or nonnumeric or may be any figurative constant, except ALL.
2. If the *literal* is numeric, then it must be an unsigned integer.
3. If a STOP RUN statement appears in a consecutive sequence of imperative statements within a sentence, it must appear as the last statement in that sequence.

2.6.18.4. General Rules

1. If the RUN phrase is used, the compiler's standard ending procedure is executed.
2. If the literal option is used, the literal is communicated to the operator, using the CONSOLE as defined in the DISPLAY verb. Upon receipt of any keyin by the operator upon the CONSOLE as defined in the ACCEPT verb, one of the following events will occur:

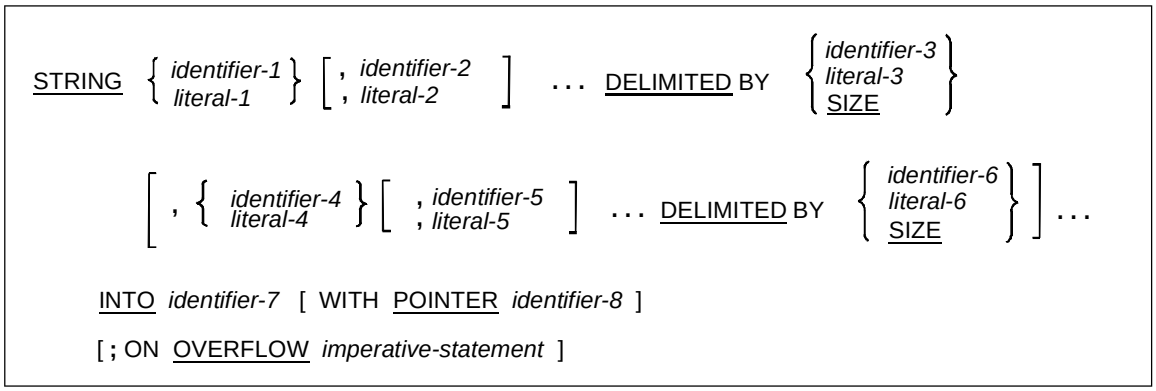
   - If the operator has keyed in GO, continuation of the object program begins with execution of the next statement in sequence.
   - If the operator has keyed in NO, the object program terminates as if a STOP RUN had been encountered.
   - If the operator has keyed in an invalid response, a message informing the operator of the valid response is displayed, and the STOP literal statement is executed again.

2.6.19. The STRING Statement

2.6.19.1. Function

The STRING statement provides juxtaposition of the partial or complete contents of two or more data items into a single data item.

2.6.19.2. General Format



2.6.19.3. Syntax Rules

1. Each literal may be any figurative constant without the optional word ALL.
2. All literals must be described as nonnumeric literals, and all identifiers, except *identifier-8,* must be described implicitly or explicitly as usage is DISPLAY.
3. *Identifier-7* must represent an elementary alphanumeric data item without editing symbols or the JUSTIFIED clause.

4. *Identifier-8* must represent an elementary numeric integer data item of sufficient size to contain a value equal to the size plus 1 of the area referenced by *identifier-7.* The symbol "P" may not be used in the PICTURE character–string of *identifier-8.*
5. Where *identifier-1, identifier-2,* . . ., or *identifier-3* is an elementary numeric data item, it must be described as an integer without the symbol "P" in its PICTURE character-string.

2.6.19.4. General Rules

1. All references to *identifier-1, identifier-2, identifier-3, literal-1, literal-2,* and *literal-3* apply equally *to identifier-4, identifier-5, identifier-6, literal-4, literal-5,* and *literal-6,* respectively, and all recursions thereof.
2. *Identifier-1, literal-1, identifier-2, literal-2*, represent the sending items. *Identifier-7* represents the receiving item.
3. *Literal-3, identifier-3,* indicate the characters delimiting the move. If the SIZE phrase is used, the complete data item defined by *identifier-1, literal-1, identifier-2, literal-2*, is moved. When a figurative constant is used as the delimiter, it stands for a single character nonnumeric literal.
4. When a figurative constant is specified as *literal-1, literal-2, literal-3*, it refers to an implicit one character data item whose usage is DISPLAY.
5. When the STRING statement is executed, the transfer of data is governed by the following rules:

    a. Those characters from *literal-1, literal-2,* or from the contents of the data item referenced by *identifier-1, identifier-2,* are transferred to the contents of *identifier-7* in accordance with the rules for alphanumeric to alphanumeric moves, except that no space-filling will be provided. (See 2.6.14.)

    b. If the DELIMITED phrase is specified without the SIZE phrase, the contents of the data item referenced *by identifier-1, identifier-2,* or the value of *literal-1, literal-2*, are transferred to the receiving data item in the sequence specified in the STRING statement beginning with the leftmost character and continuing from left to right until the end of the data item is reached, or until the characters specified by *literal-3*, or by the contents of *identifier-3* are encountered. The characters specified by *literal-3* or by the data item referenced by *identifier-3* are not transferred.

    c. If the DELIMITED phrase is specified with the SIZE phrase, the entire contents of *literal-1, literal-2,* or the contents of the data item referenced by *identifier-1, identifier-2,* are transferred, in the sequence specified in the STRING statement, to the data item referenced by *identifier-7* until all data has been transferred or the end of the data item referenced by *identifier-7* has been reached.

6. If the POINTER phrase is specified*, identifier-8* is explicitly available to the programmer, and he is responsible for setting its initial value. The initial value must not be less than one.
7. If the POINTER phrase is not specified, the following general rules apply as if the user had specified *identifier-8* with an initial value of one.

8. When characters are transferred to the data item referenced by *identifier-7*, the moves behave as though the characters were moved one at a time from the source into the character position of the data item referenced by *identifier-7* designated by the value associated with *identifier-8*, and then identifier-8 was increased by one prior to the move of the next character. The value associated with *identifier-8* is changed during execution of the STRING statement only by the behavior specified in the preceding rules.
9. At the end of execution of the STRING statement, only the portion of the data item referenced by *identifier-7* that was referenced during the execution of the STRING statement is changed. All other portions of the data item referenced by *identifier-7* will contain data that was present before this execution of the STRING statement.
10. If at any point at or after initialization of the STRING statement, but before execution of the STRING statement is completed, the value associated with *identifier-8* is either less than one or exceeds the number of character positions in the data item referenced by *identifier-7*, no (further) data is transferred to the data item referenced by *identifier-7*, and the imperative statement in the ON OVERFLOW phrase is executed, if specified.
11. If the ON OVERFLOW phrase is not specified when the conditions described in general rule 10 above are encountered, control is transferred to the next executable statement.

The following examples illustrate some of the ways STRING can be used. In each case assume that the receiving field is defined as an X(20) and contains spaces.

Example 1:

```
STRING "DEAR" DELIMITED SIZE,
    TITLE DELIMITED SPACE,
    SPACE DELIMITED SIZE,
    LAST-NAME DELIMITED SPACE,
    ":" DELIMITED SIZE
    INTO LETTER-SALUTE.
```

would result in

```
DEAR MR JONES:
```

or

```
DEAR MISS MARTIN:
```

depending on the contents of TITLE and LAST–NAME.

Example 2:

```
MOVE 1 TO CHAR-CT.
   STRING FLD1, "/", FLD2, "/", FLD3, "/", FLD4, "/"
       DELIMITED BY SPACE INTO
       COMPRESS-FLD POINTER CHAR-CT.
```

assuming:

```
FLD1 contains "AA A"
FLD2 contains "BB    "
FLD3 contains "CCC"
FLD4 contains "D"
```

COMPRESS–FLD will contain:

```
AA/BB/CCC/D/
```

CHAR–CT will equal 13 and could be used to begin a new stringing operation at location 13.

2.6.20. The SUBTRACT Statement

2.6.20.1. Function

The SUBTRACT statement is used to subtract one or the sum of two or more, numeric data items from one or more items, and sets the values of one or more items equal to the results.

2.6.20.2. General Format

Format 1

SUBTRACT { *identifier-1* / *literal-1* } [, *identifier-2* / , *literal-2*] ... FROM *identifier-m* [ROUNDED]

[, *identifier-n* [ROUNDED]] ... [; ON SIZE ERROR *imperative-statement*]

Format 2

SUBTRACT { *identifier-1* / *literal-1* } [, *identifier-2* / , *literal-2*] ... FROM { *identifier-m* / *literal-m* }

GIVING *identifier-n* [ROUNDED] [, *identifier-o* [ROUNDED]] ...

[; ON SIZE ERROR *imperative-statement*]

Format 3

SUBTRACT { CORRESPONDING / CORR } *identifier-1* FROM *identifier-2* [ROUNDED]

[; ON SIZE ERROR *imperative-statement*]

2.6.20.3. Syntax Rules

1. Each identifier must refer to a numeric elementary item except that:
   a. In Format 2, each identifier following the word GIVING must refer to either an elementary numeric item or an elementary numeric-edited item.
   b. In Format 3, where each identifier must refer to a group item.
2. Each literal must be a numeric literal.
3. The composite of operands must not contain more than 18 digits. (See 2.6.3.4.)
   a. In Format 1 the composite of operands is determined by using all of the operands in a given statement.
   b. In Format 2 the composite of operands is determined by using all of the operands in a given statement excluding the data items that follow the word GIVING.
   c. In Format 3 the composite of operands is determined separately for each pair of corresponding data items.
4. CORR is an abbreviation for CORRESPONDING.

2.6.20.4. General Rules

1. See 2.6.3.1 for the ROUNDED phrase, 2.6.3.2 for the SIZE ERROR phrase, 2.6.3.3 for the CORRESPONDING phrase, 2.6.3.4 for arithmetic statements, 2.6.3.5 for overlapping operands, 2.6.3.6 for multiple results in arithmetic statements, and 2.6.3.7 for arithmetic intermediate results.
2. In Format 1, all literals or identifiers preceding the word FROM are added together and this total is subtracted from the current value of *identifier-m* storing the result immediately into *identifier-m,* and repeating this process respectively for each operand following the word FROM.
3. In Format 2, all literals or identifiers preceding the word FROM are added together, the sum is subtracted from *literal-m* or *identifier-m* and the result of the subtraction is stored as the new value of *identifier-n, identifier-o,* etc.
4. If Format 3 is used, data items in *identifier-1* are subtracted from and stored into corresponding data items in *identifier-2.*
5. The compiler ensures enough places are carried so as not to lose significant digits during execution.

For the examples of the SUBTRACT statement, assume the following definition and values for the operands:

```
A   PIC   99V9      VALUE  11.1

B   PIC   S9V9(3)   VALUE  - 0.002

C   PIC   S99V9(5)  VALUE  1.11111

D   PIC   $$99.99.
```

Statement	Calculation	Result
SUBTRACT 16,A,B FROM C	1.11111- (16 + 11.1 - .002)	25V9868R
SUBTRACT A, B FROM D	Illegal because D contains editing characters	—
SUBTRACT A, B FROM 126	Illegal because result cannot be stored in a literal	—
SUBTRACT A, B FROM 126 GIVING D	126 - (11.1 - 0.002)	$114.90

In the first example, V indicates the assumed decimal point and R indicates the operational sign (negative) and last digit (9) of the result.

2.6.21. The UNSTRING Statement

2.6.21.1. Function

The UNSTRING statement causes contiguous data in a sending field to be separated and placed into multiple receiving fields.

2.6.21.2. General Format

```
UNSTRING identifier-1
    [ DELIMITED BY [ ALL ] { identifier-2 } [ , OR [ ALL ] ] { identifier-3 } ] ... ]
                           { literal-1    }                  { literal-2    }
    INTO identifier-4 [ , DELIMITER IN identifier-5 ] [ , COUNT IN identifier-6 ]
        [ , identifier-7 [ , DELIMITER IN identifier-8 ] [ , COUNT IN identifier-9 ] ] ...
    [ WITH POINTER identifier-10 ] [ TALLYING IN identifier-11 ]
    [ ; ON OVERFLOW imperative-statement ]
```

2.6.21.3. Syntax Rules

1. Each literal must be a nonnumeric literal. In addition, each literal may be any figurative constant without the optional word ALL.
2. *Identifier-1, identifier-2, identifier-3, identifier-5*, and *identifier-8* must be described, implicitly or explicitly, as an alphanumeric data item.
3. *Identifier-4* and *identifier-7* may be described as either alphabet (except that the symbol "B" may not be used in the PICTURE character-string), alphanumeric, or numeric (except that the symbol "P" may not be used in the PICTURE character-string), and must be described as USAGE IS DISPLAY.
4. *Identifier-6, identifier-9, identifier-10*, and *identifier-11* must be described as elementary numeric integer data items (except that the symbol "P" may not be used in the PICTURE character-string).

5. No identifier may name a level 88 entry.
6. The DELIMITER IN phrase and the COUNT IN phrase may be specified only if the DELIMITED BY phrase is specified.

2.6.21.4. General Rules

1. All references to *identifier-2, literal-1, identifier-4, identifier-5*, and *identifier-6*, apply equally to *identifier-3, literal-2, identifier-7, identifier-8,* and *identifier-9*, respectively, and all recursions thereof.
2. *Identifier-1* represents the sending area.
3. *Identifier-4* represents the data receiving area. *Identifier-5* represents the receiving area for delimiters.
4. *Literal-1* or the data item referenced by *identifier-2* specifies a delimiter.
5. *Identifier-6* represents the count of the number of characters within the data item referenced by *identifier-1* isolated by the delimiters for the move to *identifier-4*. This value does not include a count of the delimiter characters.
6. The data item referenced by *identifier-10* contains a value that indicates a relative character position within the area defined by *identifier-1*.
7. The data item referenced by *identifier-11* is a counter that records the number of data items acted upon during the execution of an UNSTRING statement.
8. When a figurative constant is used as the delimiter, it stands for a single character nonnumeric literal.

When the ALL phrase is specified, one occurrence or two or more contiguous occurrences of *literal-1* (figurative constant or not) or the contents of the data item referenced by *identifier-2* are treated as if it were only one occurrence, and this occurrence is moved to the receiving data item according to the rules in general rule 13d.

9. When any examination encounters two contiguous delimiters, the current receiving area is either space or zero filled according to the description of the receiving area.
10. *Literal-1* or the contents of the data item referenced by *identifier-2* can contain any character in the computer's character set.
11. Each *literal-1* or the data item referenced by *identifier-2* represents one delimiter. When a delimiter contains two or more characters, all of the characters must be present in contiguous positions of the sending item, and in the order given to be recognized as a delimiter.
12. When two or more delimiters are specified in the DELIMITED BY phrase, an "OR" condition exists between them. Each delimiter is compared to the sending field. If a match occurs, the characters in the sending field is considered to be a single delimiter. No characters in the sending field can be considered a part of more than one delimiter.

Each delimiter is applied to the sending field in the sequence specified in the UNSTRING statement.

13. When the UNSTRING statement is initiated, the current receiving area is the data item referenced by *identifier-4*. Data is transferred from the data item referenced by *identifier-1* to the data item referenced by *identifier-4* according to the following rules:

    a. If the POINTER phrase is specified, the string of characters referenced by *identifier-1* is examined beginning with the relative character position indicated by the contents of the data item referenced by *identifier-10*. If the POINTER phrase is not specified, the string of characters is examined beginning with the leftmost character position.

    b. If the DELIMITED BY phrase is specified, the examination proceeds left to right until either a delimiter specified by the value of *literal-1* or the data item referenced by *identifier-2* is encountered. (See general rule 11.) If the DELIMITED BY phrase is not specified, the number of characters examined is equal to the size of the current receiving area. However, if the sign of the receiving item is defined as occupying a separate character position, the number of characters examined is one less than the size of the current receiving area.

       If the end of the data item referenced by *identifier-1* is encountered before the delimiting condition is met, the examination terminates with the last character examined.

    c. The characters thus examined (excluding the delimiting characters, if any) are treated as an elementary alphanumeric data item, and are moved into the current receiving area according to the rules for the MOVE statement. (See 2.6.14.)

    d. If the DELIMITER IN phrase is specified, the delimiting characters are treated as an elementary alphanumeric data item and are moved into the data item referenced by *identifier-5* according to the rules for the MOVE statement. (See 2.6.14.) If the delimiting condition is the end of the data item referenced by *identifier-1*, then the data item referenced by *identifier-5* is space filled.

    e. If the COUNT IN phrase is specified, a value equal to the number of characters thus examined (excluding the delimiter characters, if any) is moved into the area referenced by *identifier-6* according to the rules for an elementary move.

    f. If the DELIMITED BY phrase is specified, the string of characters is further examined beginning with the first character to the right of the delimiter. If the DELIMITED BY phrase is not specified, the string of characters is further examined beginning with the character to the right of the last character transferred.

    g. After data is transferred to the data item referenced by *identifier-4*, the current receiving area is the data item referenced by *identifier-7*. The behavior described in rule 13b through 13f is repeated until either all the characters are exhausted in the data item referenced by *identifier-1*, or until there are no more receiving areas. If all characters are exhausted by the data-item referenced by *identifier-1* and more receiving areas remain, then these identifiers are left unchanged.

14. The initialization of the contents of the data items associated with the POINTER phrase or the TALLYING phrase is the responsibility of the user.

15. The contents of the data item referenced by *identifier-10* will be incremented by one for each character examined in the data item referenced *by identifier-1*. When the execution of an UNSTRING statement with a POINTER phrase is completed, the contents of the data item referenced by *identifier-10* will contain a value equal to the initial value plus the number of characters examined in the data item referenced by *identifie-1*.
16. When the execution of an UNSTRING statement with a TALLYING phrase is completed, the contents of the data item referenced by *identifier-11* contains a value equal to its initial value plus the number of data receiving items acted upon.
17. Either of the following situations causes an overflow condition:

    a. An UNSTRING is initiated, and the value in the data item referenced by *identifier-10* is less than 1 or greater than the size of the data item referenced by *identifier-1*.

    b. If, during execution of an UNSTRING statement, all data receiving areas have been acted upon, and the data item referenced by *identifier-1* contains characters that have not been examined.

18. When an overflow condition exists, the UNSTRING operation is terminated. If an ON OVERFLOW phrase has been specified, the imperative statement included in the ON OVERFLOW phrase is executed. If the ON OVERFLOW phrase is not specified, control is transferred to the next executable statement.
19. The evaluation of subscripting and indexing for the identifiers is as follows:

    a. Any subscripting or indexing associated *with identifier-1, identifier-10, identifier-11* is evaluated only once, immediately before any data is transferred as the result of the execution of the UNSTRING statement.

    b. Any subscripting or indexing associated *with identifier-2, identifier-3, identifier-4, identifier-5, identifier-6* is evaluated immediately before the transfer of data into the respective data item.

The following examples illustrate some of the ways UNSTRING can be used.

Example 1:

```
02 FLD1 PIC X(5)
02 FLD2 PIC X(5)
02 FLD3 PIC X(5)
02 FLD4 PIC X(5)
02 PACKED-FLD PIC X(14)
           .
           .
           .
    UNSTRING PACKED-FLD
    DELIMITED BY "/" INTO
    FLD1, FLD2, FLD3, FLD4
```

assumed PACKED–FLD contains:

```
AA/BB/CCCCCC/D
```

after execution:

```
FLD1: AA
FLD2: BB
FLD3: CCCCC
FLD4: D
```

Example 2:

```
02 PT-CT PIC 99.
02 UNPACK OCCURS 4 TIMES
    03 RECVR PIC XX.
    03 DELM PIC XX.
    03 CT PIC 99.
    .
    .
    .
MOVE 0 TO TALLY-REG
MOVE 5 TO PT-CT
UNSTRING PACK-FLD
    DELIMITED BY SPACE OR ALL "/" INTO
    RECVR (1) DELIMITER DELM (1) COUNT CT (1)
    RECVR (2) DELIMITER DELM (2) COUNT CT (2)
    RECVR (3) DELIMITER DELM (3) COUNT CT (3)
    RECVR (4) DELIMITER DELM (4) COUNT CT (4)
    POINTER PT-CT
    TALLYING IN TALLY-REG
    ON OVERFLOW STOP RUN.
```

Assuming PACK–FLD is defined as X(17) and contains:

```
//ΔABCDΔ/DE//FΔΔG
```

the execution of the preceding code will result in the STOP RUN being executed and the fields will contain:

N	RECVR (N)	DELM (N)	CT(N)
1	BC	Δ	3
2	ΔΔ	/Δ	0
3	DE	//	2
4	FΔ	ΔΔ	1

```
PT-CT  = 16
TALLY-REG  = 4
```

Example 3:

```
02 NUM1.
    03 THOUSANDS PIC 999.
    03 HUNDREDS PIC 999.
    03 FRCTN PIC X(4).
02 NUM2 REDEFINES NUM1 PIC 9(6)V9(4).
    .
    .
    .
UNSTRING EDIT-NUMBER
    DELIMITED BY SPACE OR "," OR "."
    INTO THOUSANDS, HUNDREDS, FRCTN.
EXAMINE FRCTN REPLACING ALL SPACE BY ZERO
```

Assuming EDIT–NUMBER contains 1,105,345ΔΔΔΔ and is described as X(13), after the execution of the UNSTRING and the EXAMINE NUM2 will contain the value 001105V3450.

Section 3
Table Handling

3.1. General

3.1.1. Function

The Table Handling module provides a capability for defining tables of contiguous data items and accessing an item relative to its position in the table. Language facility is provided for specifying how many times an item is to be repeated. Each item may be identified through use of a subscript or an index. (See 1.3.3.3.8.3.)

3.1.2. Level Characteristics

Table Handling Level 1 provides a capability for accessing items in up to three-dimensional fixed length tables. This level also provides series options and the ability to vary the contents of indices by an increment or decrement.

Table Handling Level 2 provides a capability for accessing items in up to three-dimensional variable length tables. This level also provides the additional facilities for specifying ascending or descending keys and permits searching a dimension of a table for an item satisfying a specified condition.

3.2. Data Division

3.2.1. The OCCURS Clause

3.2.1.1. Function

The OCCURS clause eliminates the need for separate entries for repeated data items and supplies information required for the application of subscripts or indices.

3.2.1.2. General Format

Format 1

```
OCCURS integer-2 TIMES
[ { ASCENDING  } KEY IS data-name-2 [ , data-name-3 ] ... ] ...
  { DESCENDING }
```

Format 2

```
[ INDEXED BY index-name-1 [ , index-name-2 ] ... ]
```

```
OCCURS integer-1 TO integer-2 TIMES DEPENDING ON data-name-1
[ { ASCENDING  } KEY IS data-name-2 [ , data-name-3 ] ... ] ...
  { DESCENDING }
[ INDEXED BY index-name-1 [ , index-name-2 ] ... ]
```

3.2.1.3. Syntax Rules

1. Where both *integer–1* and *integer–2* are used, the value of *integer–1* must be less than the value of *integer–2.*
2. The data description of *data–name–1* must describe a positive integer.
3. *Data–name–1, data–name–2, data–name–3,* . . . may be qualified.
4. *Data–name–2* must either be the name of the entry containing the OCCURS clause or the name of an entry subordinate to the entry containing the OCCURS clause.
5. *Data–name–3,* etc., must be the name of an entry subordinate to the group item which is the subject of this entry.
6. An INDEXED BY phrase is required if the subject of this entry, or an entry subordinate to this entry, is to be referred to by indexing. The index-name identified by this clause is not defined elsewhere since its allocation and format are dependent on the hardware, and not being data, cannot be associated with any data hierarchy.
7. A data description entry that contains Format 2 of the OCCURS clause may only be followed, within that record description, by data description entries which are subordinate to it.

8. The OCCURS clause cannot be specified in a data description entry that:
   a. Has a 01, 66, 77, or an 88 level–number.
   b. Describes an item whose size is variable. The size of an item is variable if the data description of any subordinate item contains Format 2 of the OCCURS clause.
9. In Format 2, the data item defined by *data–name–1* must not occupy a character position within the range of the first character position defined by the data description entry containing the OCCURS clause and the last character position defined by the record description entry containing that OCCURS clause.
10. If *data–name–2* is not the subject of this entry, then:
    a. All of the items identified by the data-names in the KEY IS phrase must be within the group item which is the subject of this entry.
    b. Items identified by the data-name in the KEY IS phrase must not contain an OCCURS clause.
    c. There must not be any entry that contains an OCCURS clause between the items identified by the data-names in the KEY IS phrase and the subject of this entry.
11. *Index–name–1, index–name–2,* ... must be unique words within the program.

3.2.1.4. General Rules

1. The OCCURS clause is used in defining tables and other homogeneous sets of repeated data items. Whenever the OCCURS clause is used, the data–name which is the subject of this entry must be either subscripted or indexed whenever it is referred to in a statement other than SEARCH or USE FOR DEBUGGING. Furthermore, if the subject of this entry is the name of a group item, then all data–names belonging to the group must be subscripted or indexed whenever they are used as operands, except as the object of a REDEFINES clause. (See 1.3.3.3.8.2, 1.3.3.3.8.3, and 1.3.3.3.8.4.)
2. Except for the OCCURS clause itself, all data description clauses associated with an item whose description includes an OCCURS clause apply to each occurrence of the item described. (See restriction in general rule 2 in 2.5.13.5.)
3. The number of occurrences of the subject entry is defined as:
   a. In Format 1, the value of *integer–2* represents the exact number of occurrences**.**
   b. In Format 2, the current value of the data item referenced by *data–name–1* represents the number of occurrences.

      This format specifies that the subject of this entry has a variable number of occurrences. The value of *integer–2* represents the maximum number of occurrences and the value of *integer–1* represents the minimum number of occurrences. This does not imply that the length of the subject of the entry is variable, but that the number of occurrences is variable.

The value of the data item referenced by *data–name–1* must fall within the range *integer–1* through *integer–2. R*educing the value of the data item referenced by *data–name–1* makes the contents of data items, whose occurrence numbers now exceed the value of the data item referenced by *data–name–1,* unpredictable.

4. When a group item, having subordinate to it an entry that specifies Format 2 of the OCCURS clause, is referenced, only that part of the table area that is specified by the value of *data–name–1* will be used in the operation.
5. The KEY IS phrase is used to indicate that the repeated data is arranged in ascending or descending order according to the values contained in *data–name–2, data–name–3,* etc. The ascending or descending order is determined according to the rules for comparison of operands (see 2.6.2.1.1.1 and 2.6.2.1.1.2). The data-names are listed in descending order of significance.
6. In a COBOL statement referencing a group item having subordinate to it an identifier containing the OCCURS DEPENDING ON clause, the number of occurrences as specified by the current value of the object of the DEPENDING ON clause will take part in the operation; the remainder will not. Any items contained in the record structure beginning with the specified group item and ending at the variably occurring item will participate in the operation. However, if the group item is to contain the result of the operation specified by the COBOL verb, the maximum number of occurrences as specified by *integer–2* in Format 2 will be used regardless of the current value of the object of the DEPENDING ON clause. This would be true, for example, on a MOVE statement, where the object of the MOVE was a group item having subordinate to it an identifier containing the OCCURS DEPENDING ON clause.

The statements affected by the OCCURS DEPENDING ON clause are:

CALL	RELEASE
CANCEL	RETURN … INTO
DISABLE	REWRITE
DISPLAY	SEARCH (condition only)
ENABLE	SEND
IF	SOURCE
INSPECT	STRING
MOVE	UNSTRING
PERFORM	WRITE
READ … INTO	

The next example illustrates the effects of the OCCURS DEPENDING ON clause in a WRITE . . . FROM . . . statement.

```
IDENTIFICATION DIVISION.
PROGRAM-ID.  ODO-EXAMPLE.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
    SOURCE-COMPUTER.  UNISYS-2200.
    OBJECT-COMPUTER.  UNISYS-2200.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT FILEA ASSIGN TO PRINTER PRINT-FILE.
DATA DIVISION.
FILE SECTION.
FD FILEA
    LABEL RECORDS ARE OMITTED
    DATA RECORD IS PRINT-RECORD.
01 PRINT-RECORD.
    02 PRINT-SUB PIC 99.
    02 PRINT-ODO-GROUP.
        03 PRINT-ODO-ITEM OCCURS 1 TO 13
           TIMES DEPENDING ON PRINT-SUB.
           04 PRINT4 PIC X(4).
           04 PRINT6 PIC X(6).
WORKING-STORAGE SECTION.
01 ODO.
    02 ODO-SUB PIC 99.
    02 ODO-GROUP.
        03 ODO-ITEM OCCURS 1 TO 5 TIMES
           DEPENDING ON ODO-SUB.
           04 ODO6 PIC X(6).
           04 ODO4 PIC X(4).

PROCEDURE DIVISION.
P1.
    OPEN OUTPUT FILEA.
    MOVE 13 TO PRINT-SUB.
    MOVE ALL "A" TO PRINT-ODO-GROUP.
    WRITE PRINT-RECORD.
P2.
    MOVE 3 TO ODO-SUB.
    MOVE 2 TO PRINT-SUB.
    MOVE ALL "1234567890" TO ODO-GROUP.
    WRITE PRINT-RECORD FROM ODO.
    CLOSE FILEA.
P3.
    STOP RUN.
```

After execution of this program, the external file PRINT–FILE will contain two records. The first record will contain the characters "13" followed by 130 occurrences of the letter "A". The second record will contain the characters "03" followed by three occurrences of the string "1234567890". Note that the value contained in PRINT–SUB prior to the execution of the statement WRITE PRINT–RECORD FROM ODO does not determine the number of occurrences of PRINT–ODO–ITEM written to FILEA.

3.2.2. The USAGE Clause

3.2.2.1. Function

The USAGE clause specifies the format of a data item in storage.

3.2.2.2. General Format

```
[ USAGE IS ] INDEX
```

3.2.2.3. Syntax Rules

1. An index data item can be referenced explicitly only in a SEARCH or SET statement, a relation condition, the USING phrase of a Procedure Division header, or the USING phrase of a CALL statement.
2. The SYNCHRONIZED, JUSTIFIED, PICTURE, VALUE, and BLANK WHEN ZERO clauses cannot be used to describe group or elementary items described with the USAGE IS INDEX clause.

3.2.2.4. General Rules

1. The USAGE clause can be written at any level. If the USAGE clause is written at a group level, it applies to each elementary item in the group. The USAGE clause of an elementary item cannot contradict the USAGE clause of a group to which the item belongs.
2. An elementary item described with the USAGE IS INDEX clause is called an index data item and contains a value which must correspond to an occurrence number of a table element. The elementary item cannot be a conditional variable. If a group item is described with the USAGE IS INDEX clause the elementary items in the group are all index data items. The group itself is not an index data item and cannot be used in the SEARCH or SET statement or in a relation condition.
3. An index data item can be part of a group which is referred to in a MOVE or input–output statement, in which case no conversion will take place.
4. The external and internal format of an index data item is one word with implicit synchronization and implicit COMPUTATIONAL usage.

3.3. Procedure Division

3.3.1. Relation Condition

3.3.1.1. Comparisons Involving Index-Names and/or Index Data Items

Relation tests may be made between:

1. Two index-names. The result is the same as if the corresponding occurrence numbers were compared.
2. An index-name and a data item (other than an index data item) or literal. The occurrence number that corresponds to the value of the index-name is compared to the data item or literal.
3. An index data item and an index-name or another index data item. The actual values are compared without conversion.

   It is recommended that index-names and index data items be compared only if they contain occurrence information for the same table or for tables whose structure and alignment are identical. (See Appendix D.)
4. The result of the comparison of an index data item with any data item or literal not specified above is undefined.

3.3.2. Overlapping Operands

When a sending and a receiving item in a SET statement share a part of their storage areas, the result of the execution of such a statement is undefined.

3.3.3. The SEARCH Statement

3.3.3.1. Function

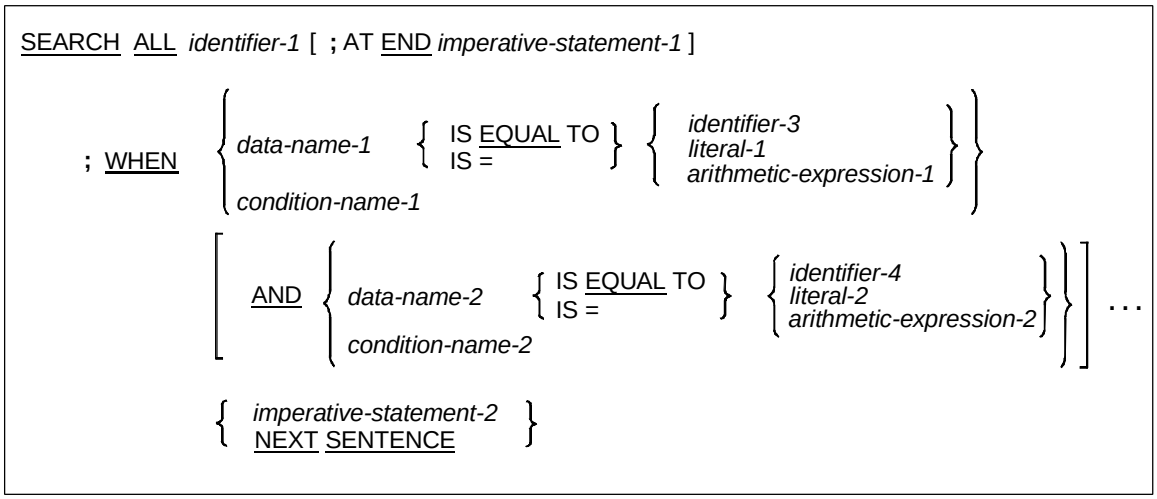
The SEARCH statement is used to search a table for a table element that satisfies the specified condition and adjust the associated index-name to indicate that table element.

3.3.3.2. General Format

Format 1

```
SEARCH identifier-1 [ VARYING { identifier-2 } ]
                              { index-name-1 }
  [ ; AT END imperative-statement-1 ]
    ; WHEN condition-1 { imperative-statement-2 }
                       { NEXT SENTENCE          }
  [ ; WHEN condition-2 { imperative-statement-3 } ] ...
                       { NEXT SENTENCE          }
```

Format 2



Note: *The required relational character " = " is not underlined to avoid confusion with other symbols.*

3.3.3.3. Syntax Rules

1. In both Formats 1 and 2, *identifier-1* must not be subscripted or indexed, but its description must contain an OCCURS clause and an INDEXED BY clause. The description of *identifier-1* in Format 2 must also contain the KEY IS phrase in its OCCURS clause.
2. *Identifier-2,* when specified, must be described as USAGE IS INDEX or as a numeric elementary item without any positions to the right of the assumed decimal point.
3. In Format 1, *condition-1, condition-2,* etc., may be any condition as described in 2.6.1.2.
4. In Format 2, all referenced condition-names must be defined as having only a single value. The data-name associated with a condition-name must appear in the KEY clause of *identifier-1.* Each *data-name-1, data-name-2* may be qualified. Each *data-name-1,data-name–2* must be indexed by the first index-name associated with *identifier-1* along with other indices or literals as required, and must be referenced in the KEY clause of *identifier-1. Identifier-3, identifier-4,* or identifiers specified in *arithmetic-expression-1,arithmetic-expression-2* must not be referenced in the KEY clause of *identifier-1* or be indexed by the first index-name associated with *identifier-1.*

   In Format 2, when a data-name in the KEY clause of *identifier-1* is referenced, or when a condition-name associated with a data-name in the KEY clause of *identifier-1* is referenced, all preceding data-names in the KEY clause of *identifier-1* or their associated condition-names must also be referenced.

3.3.3.4. General Rules

1. If Format 1 of the SEARCH is used, a serial type of search operation takes place, starting with the current index setting.

   a. If, at the start of execution of the SEARCH statement, the index-name associated with *identifier-1* contains a value that corresponds to an occurrence number that is greater than the highest permissible occurrence number for *identifier-1,* the SEARCH is terminated immediately. The number of occurrences of *identifier-1,* the last of which is the highest permissible, is discussed in 3.2.1. Then, if the AT END phrase is specified, *imperative-statement-1* is executed. If the AT END phrase is not specified, control passes to the next executable sentence.

   b. If, at the start of execution of the SEARCH statement, the index-name associated with *identifier-1* contains a value that corresponds to an occurrence number that is not greater than the highest permissible occurrence number for *identifier-1* (the number of occurrences of *identifier-1,* the last of which is the highest permissible is discussed in 3.2.1), the SEARCH statement operates by evaluating the conditions in the order that they are written, making use of the index settings, wherever specified, to determine the occurrence of those items to be tested. If none of the conditions is satisfied, the index-name for *identifier-1* is incremented to obtain reference to the next occurrence. The process is then repeated using the new index-name settings unless the new value of the index-name settings for *identifier-1* corresponds to a table element outside the permissible range of occurrence values, in which case the search terminates as indicated in step 1a. If one of the conditions is satisfied upon its evaluation, the search terminates immediately and the imperative statement associated with that condition is executed; the index-name remains set at the occurrence which caused the condition to be satisfied.

2. In a Format 2 SEARCH, the results of the SEARCH ALL operation are predictable only when:

   a. The data in the table is ordered in the same manner as described in the ASCENDING/DESCENDING KEY clause associated with the description of *identifier-1,* and

   b. The contents of the keys referenced in the WHEN clause are sufficient to identify a unique table element.

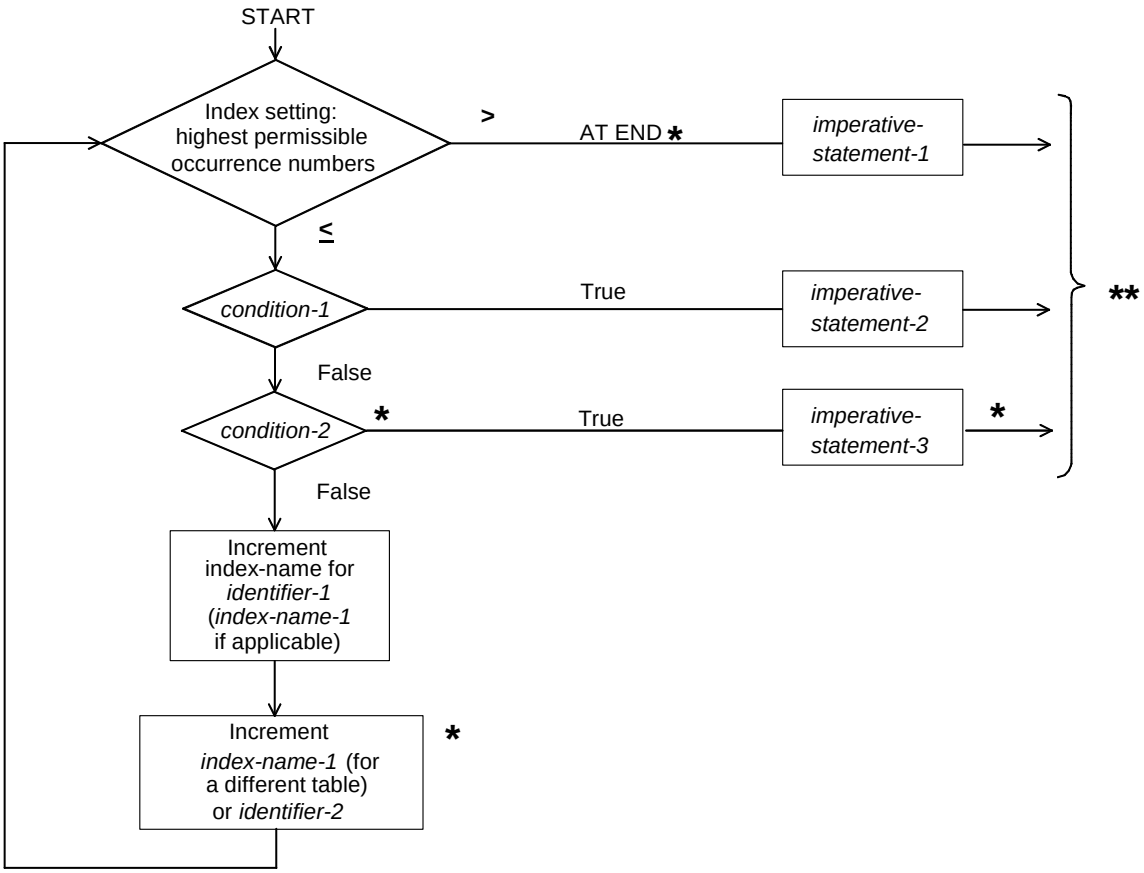
3. If Format 2 of the SEARCH is used, a nonserial type of search operation may take place; the initial setting of the index-name for *identifier-1* is ignored and its setting is varied during the search operation according to a binary search algorithm, with the restriction that at no time is it set to a value that exceeds the value which corresponds to the last element of the table, or that is less than the value that corresponds to the first element of the table. The length of the table is discussed in the OCCURS clause (see 3.2.1). If any of the conditions specified in the WHEN clause cannot be satisfied for any setting of the index within the permitted range, control is passed to *imperative-statement-1* of the AT END phrase, when specified, or to the next executable sentence when this phrase is not specified. In either case the final setting of the index is not predictable. If all the conditions can be satisfied, the index indicates an occurrence that allows the conditions to be satisfied, and control passes to *imperative-statement-2.*
4. After execution of an *imperative-statement-1, imperative-statement-2,* or *imperative-statement-3* that does not terminate with a GO TO statement, control passes to the next executable sentence.
5. In Format 2, the index-name that is used for the search operation is the first (or only) index-name that appears in the INDEXED BY phrase of *identifier-1.* Any other index-names for *identifier-1* remain unchanged.
6. In Format 1, if the VARYING phrase is not used, the index-name that is used for the search operation is the first (or only) index-name that appears in the INDEXED BY phrase of *identifier-1.* Any other index-names for *identifier-1* remain unchanged.
7. In Format 1, if the VARYING *index-name-1* phrase is specified, and if *index-name-1* appears in the INDEXED BY phrase of *identifier-1,* that index-name is used for this search. If this is not the case, or if the VARYING *identifier-2* phrase is specified, the first (or only) index-name given in the INDEXED BY phrase of *identifier-1* is used for the search. In addition, the following operations will occur:

    a. If the VARYING *index-name-1* phrase is used, and if *index-name-1* appears in the INDEXED BY phrase of another table entry, the occurrence number represented by *index-name-1* is incremented by the same amount as, and at the same time as, the occurrence number represented by the index-name associated with *identifier-1* is incremented.

    b. If the VARYING *identifier-2* phrase is specified, and *identifier-2* is an index data item, then the data item referenced by *identifier-2* is incremented by the same amount as, and at the same time as, the index associated with *identifier-1* is incremented. If *identifier-2* is not an index data item, the data item referenced by *identifier-2* is incremented by the value one at the same time as the index referenced by the index-name associated with *identifier-1* is incremented.

8. If *identifier-1* is a data item subordinate to a data item that contains an OCCURS clause (providing for a 2- or 3-dimensional table), an index-name must be associated with each dimension of the table through the INDEXED BY phrase of the OCCURS clause. Only the setting of the index–name associated with *identifier-1* (and the data item *identifier-2* or *index-name-1,* if present) is modified by the execution of the SEARCH statement. To search an entire 2- or 3-dimensional table it is necessary to execute a SEARCH statement several times. Prior to each execution of a SEARCH statement, SET statements must be executed whenever index-names must be adjusted to appropriate settings.

This flowchart shows the Format 1 SEARCH operation containing two WHEN phrases:



* These optional operations are included only when specified in the SEARCH statement.

** Each of these control transfers is to the next executable sentence unless the *imperative-statement* ends with a GO TO statement.

3.3.4. The SET Statement

3.3.4.1. Function

The SET statement establishes reference points for table handling operations by setting index-names associated with table elements.

3.3.4.2. General Format

Format 1

SET { *identifier-1* [, *identifier-2*] ... } TO { *identifier-3* }
{ *index-name-1* [, *index-name-2*] ... } { *index-name-3* }
{ *integer-1* }

Format 2

```
SET  index-name-4  [ , index-name-5 ] ...  { UP BY   }  { identifier-4 }
                                           { DOWN BY }  { integer-2    }
```

3.3.4.3. Syntax Rules

1. All references to *index-name-1, identifier-1,* and *index-name-*4 apply equally to *index-name-2, identifier-2*, and *index-name-5*, respectively.
2. *Identifier-1* and *identifier-3* must name either index data items, or elementary items described as an integer.
3. *Identifier-4* must be described as an elementary numeric integer.
4. *Integer-1* and *integer-2* may be signed. *Integer-1* must be positive.

3.3.4.4. General Rules

1. Index-names are considered related to a given table and are defined by being specified in the INDEXED BY clause.
2. If *index-name-3* is specified, the value of the index before the execution of the SET statement must correspond to an occurrence number of an element in the associated table.

   If *index-name-4* and *index-name-5* are specified, the value of the index both before and after the execution of the SET statement must correspond to an occurrence number of an element in the associated table. If *index-name-1* and *index-name-2* are specified, the value of the index after the execution of the SET statement must correspond to an occurrence number of an element in the associated table. The value of the index associated with an index-name after the execution of a SEARCH or PERFORM statement may be undefined. (See 3.3.3 for the SEARCH statement and 2.6.16 for the PERFORM statement.)
3. In Format 1, the following action occurs:
   a. *Index-name-1* is set to a value causing it to refer to the table element that corresponds in occurrence number to the table element referenced by *index-name-3, identifier-3,* or *integer-1.* If *identifier-3* is an index data item, or if *index-name-3* is related to the same table as *index-name-1,* no conversion takes place.
   b. If *identifier-1* is an index data item, it may be set equal to either the contents of *index-name-3* or *identifier-3* where *identifier-3* is also an index data item. No conversion takes place in either case.
   c. If *identifier-1* is not an index data item, it may be set only to an occurrence number that corresponds to the value of *index-name-3.* Neither *identifier-3* nor *integer-1* can be used in this case.
   d. The process is repeated for *index-name-2, identifier-2,* etc., if specified. Each time the value of *index-name-3* or *identifier-3* is used as it was at the beginning of the execution of the statement. Any subscripting or indexing associated with *identifier-1,* etc., is evaluated immediately before the value of the respective data item is changed.

4. In Format 2, the contents of *index-name-4* are incremented (UP BY) or decremented (DOWN BY) by a value that corresponds to the number of occurrences represented by the value of *integer-2* or *identifier-4;* thereafter, the process is repeated for *index-name-5,* etc. Each time the value of *identifier-4* is used as it was at the beginning of the execution of the statement.
5. Data in the following chart represents the validity of various operand combinations in the SET statement and the applicable general rule.

	Receiving Item		
Sending Item	**Integer Data Item**	**Index–name**	**Index Data Item**
Integer Literal	No/3c	Valid/3a	No/3b
Integer Data Item	No/3c	Valid/3a	No/3b
Index–Name	Valid/3c	Valid/3a	Valid/3b *
Index Data Item	No/3c	Valid/3a *	Valid/3b *

* No conversion takes place

Section 4
Sequential I-O

4.1. General

4.1.1. Function

The Sequential I–O module provides a capability to access records of a file in established sequence. The sequence is established as a result of writing the records to the file. It also provides for the specification of rerun points and the sharing of memory areas among files.

4.1.2. Level Characteristics

Sequential I–O Level 1 does not provide full COBOL facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats of this module. Within the Procedure Division, the Sequential I–O Level 1 provides limited capabilities for the CLOSE, OPEN, USE, and WRITE statements and full capabilities for the READ and REWRITE statements, as specified in the formats of this module.

Sequential I–O Level 2 provides full facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats of this module. Within the Procedure Division, the Sequential I–O Level 2 provides full capabilities for the CLOSE, OPEN, READ, REWRITE, USE, and WRITE statements as specified in the formats of this module. The additional features available in Level 2 include: OPTIONAL files, the RESERVE clause, SAME RECORD AREA, MULTIPLE FILE tapes, REVERSED, EXTEND, and additional flexibility through series options.

4.1.3. Language Concepts

4.1.3.1. Organization

Sequential files are organized such that each record in the file except the first has a unique predecessor record, and each record except the last has a unique successor record. These predecessor-successor relationships are established by the order of WRITE statements when the file is created. Once established, the predecessor-successor relationships do not change except when records are added to the end of the file.

4.1.3.2. Access Mode

In the sequential access mode, the sequence in which records are accessed is the order in which the records were originally written.

4.1.3.3. Current Record Pointer

The current record pointer is a conceptual entity used in this manual to facilitate specification of the next record to be accessed within a given file. The concept of the current record pointer has no meaning for a file opened in the output mode. The setting of the current record pointer is affected only by the OPEN and READ statements.

4.1.3.4. I–O Status

If the FILE STATUS clause is specified in a file control entry, a value is placed into the specified two-character data item during the execution of an OPEN, CLOSE, READ, WRITE, or REWRITE statement and before any applicable USE procedure is executed, to indicate to the COBOL program the status of that input–output operation.

4.1.3.4.1. Status Key 1

The leftmost character position of the FILE STATUS data item is known as status key 1 and is set to indicate one of the following conditions upon completion of the input-output operation:

0 SUCCESSFUL COMPLETION. The input-output statement was successfully executed.

1 AT END. The sequential READ statement was unsuccessfully executed either as a result of an attempt to read a record when no next logical record exists in the file, or as a result of the first READ statement being executed for a file described with the OPTIONAL clause, and that file was not available to the program at the time its associated OPEN statement was executed.

3 PERMANENT ERROR. The input-output statement was unsuccessfully executed as the result of a boundary violation for a sequential file or as the result of an input-output error, such as data check parity error, or transmission error.

9 Unisys Defined. The input-output statement was unsuccessfully executed as a result of a condition other than an AT END or PERMANENT ERROR condition. This value is used only to indicate a condition not indicated by other defined values of status key 1, or by specified combinations of the values of status key 1 and status key 2.

4.1.3.4.2. Status Key 2

The rightmost character position of the FILE STATUS data item is known as status key 2 and is used to further describe the results of the input-output operation. This character will contain a value as follows:

1. If no further information is available concerning the input-output operation, then status key 2 contains a value of "0".
2. When status key 1 contains a value of "3" indicating a permanent error condition, status key 2 may contain a value of "4" indicating a boundary violation. This condition indicates that an attempt has been made to write beyond the externally defined boundaries of a sequential file, as defined on the @ASG statement for the file.
3. When status key 1 contains a value of "9" indicating a Unisys defined condition, the status key 2 values are binary numbers and are described in C.6.

4.1.3.4.3. Valid Combinations of Status Keys 1 and 2

The valid permissible combinations of the values of status key 1 and status key 2 are shown in the following table. An "X" at an intersection indicates a valid permissible combination.

Status Key 1	Status Key 2	
	No Further Information (0)	**Boundary Violation (4)**
SUCCESSFUL COMPLETION (0)	X	
AT END (1)	X	
PERMANENT ERROR (3)	X	X
Unisys Defined (9)	See C.6	

4.1.3.5. The AT END Condition

The AT END condition can occur as a result of the execution of a READ statement. For details of the causes of the condition see 4.4.3.

4.1.3.6. LINAGE–COUNTER

The reserved word LINAGE–COUNTER is a name for a special register generated by the presence of a LINAGE clause in a file description entry. The implicit description is that of an unsigned integer whose size is equal to the size of *integer-1* or the data item referenced by *data-name-1* in the LINAGE clause. See 4.3.9 for the rules governing LINAGE–COUNTER.

4.2. Environment Division

4.2.1. INPUT–OUTPUT Section

4.2.1.1. The FILE–CONTROL Paragraph

4.2.1.1.1. Function

The FILE–CONTROL paragraph names each file and allows specification of other file-related information.

4.2.1.1.2. General Format

```
FILE-CONTROL. { file-control-entry } . . .
```

4.2.1.2. The File Control Entry

4.2.1.2.1. Function

The file control entry names a file and may specify other file–related information.

4.2.1.2.2. General Format

```
SELECT [OPTIONAL] file-name

    ASSIGN TO implementor-name-1 [ , implementor-name-2 ] . . .

    [ ; RESERVE integer-1 [ AREA | AREAS ] ]

    [ ; ORGANIZATION IS SEQUENTIAL ]

    [ ; ACCESS MODE IS SEQUENTIAL ]

    [ ; FILE STATUS IS data-name-1 ] .
```

4.2.1.2.3. Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses which follow the SELECT clause may appear in any order.
2. Each file described in the Data Division must be named once and only once as *file-name* in the FILE–CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. *Implementor-name-1* is any of the following depending on the external media:

CARD-PUNCH	PRINTER
CARD-READER	TAPE
DISC	ANS74-PRINTER

4. For files assigned to TAPE or DISC:

    a. If *implementor-name-2* is specified, it is a 1 to 12-character name used to link the ASCII COBOL file to the OS 2200 Executive File Control and should appear on an @ASG or @USE statement in the run stream for executing the object program. (See 4.6.)

    b. When *implementor-name-2* is not specified, the first 12 characters of *file-name* are used to link the ASCII COBOL file to the OS Executive File Control. (See 4.6.)

5. For unit record files, in other words, CARD–PUNCH, CARD–READER, PRINTER, ANS74–PRINTER:

    a. When present, *implementor-name-2* is a 1 to 12-character name specifying an alternate system input-output file.

    b. When *implementor-name-2* is not specified, *implementor-name-1* is linked to the system input-output files associated with each run unit.

6. All recursions of *implementor-name-1*, *implementor-name-2*, are for documentation only.

7. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.

8. *Data-name-1* must be defined in the Data Division as a two-character data item of the category alphanumeric and must not be defined in the File Section, the Report Section, or the Communication Section. *Data-name-1* must be defined as USAGE DISPLAY.

9. *Data-name-1* may be qualified.

10. When the ORGANIZATION IS SEQUENTIAL clause is not specified, the ORGANIZATION IS SEQUENTIAL clause is implied.

11. The OPTIONAL phrase may only be specified for input files. Its specification is required for input files that are not necessarily present each time the object program is executed.

4.2.1.2.4. General Rules

1. The ASSIGN clause specifies the association of the file referenced by *file–name* to a storage medium.

2. If the RESERVE clause is specified, it is for documentation only. One input-output area is always allocated for CARD–PUNCH, CARD–READER, PRINTER, and ANS74–PRINTER files. Two input-output areas are always allocated for DISC and TAPE files.

3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.

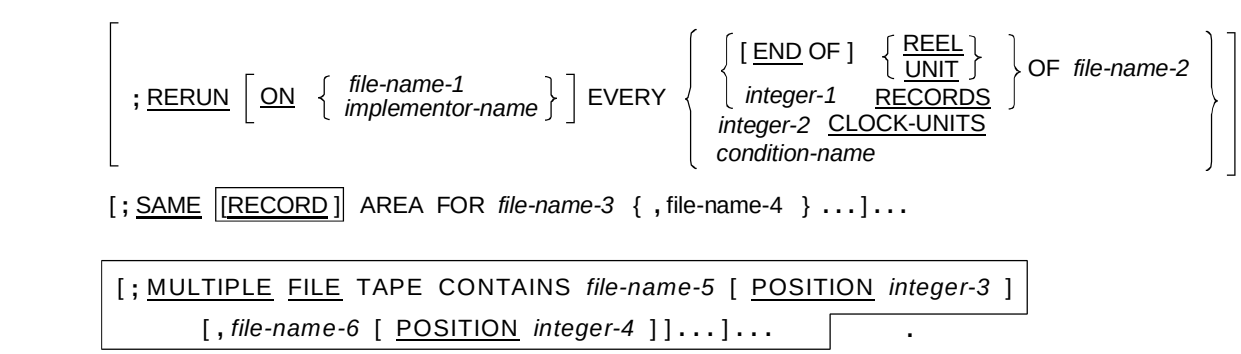
4. Records in the file are accessed in the sequence dictated by the file organization. This sequence is specified by predecessor-successor record relationships established by the execution of WRITE statements when the file is created or extended.
5. When the FILE STATUS clause is specified, a value will be moved by the operating system into the data item specified by *data-name-1* after the execution of every statement that references that file either explicitly or implicitly. This value indicates the status of execution of the statement. (See 4.1.3.4.)

4.2.1.3. The I–O–CONTROL Paragraph

4.2.1.3.1. Function

The I–O–CONTROL paragraph specifies the points at which rerun is to be established, the memory area which is to be shared by different files, and the location of files on a multiple file reel.

4.2.1.3.2. General Format



4.2.1.3.3. Syntax Rules

1. The I–O–CONTROL paragraph is optional.
2. The allowable implementor-names are TAPE and DISC.
3. When the *integer-1* RECORDS clause is specified, *implementor-name* must be given in the RERUN clause.
4. When multiple *integer-1* RECORDS clauses are specified, no two of them may specify the same *file-name-2*.
5. The ON *file-name-1* clause, the *integer-2* CLOCK–UNITS clause, the END OF REEL/UNIT OF *file-name-2* clause, and the *condition-name* clause serve only for documentation.

6. The two forms of the SAME clause (SAME AREA, SAME RECORD AREA) are considered separately in the following:

   More than one SAME clause may be included in a program, however:

   a. A file-name must not appear in more than one SAME AREA clause.

   b. A file-name must not appear in more than one SAME RECORD AREA clause.

   c. If one or more file-names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file-names in that SAME AREA clause must appear in the SAME RECORD AREA clause. However, additional file-names not appearing in that SAME AREA clause may also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at any given time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at any given time.

7. The files referenced in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

4.2.1.3.4. General Rules

1. The RERUN clause specifies when and where the rerun information is recorded.
2. If *implementor-name* is specified, the rerun information is written as a separate file whose file–name is RERUN$.
3. In ASCII COBOL, rerun points can be established when the *integer-1* RECORDS clause is used. In this case, the rerun information is written on the file whose file-name is RERUN$, which must be specified in the ON clause whenever *integer-1* records of *file-name-2* have been processed. *File-name-2* may be either an input or output file with any organization or access.
4. The SAME AREA clause specifies that two or more files that do not represent sort or merge files are to use the same memory area during processing. The area being shared includes all storage area assigned to the files specified; therefore, it is not valid to have more than one of the files open at the same time. (See syntax rule 6c, 4.2.1.3.3.)
5. The SAME RECORD AREA clause specifies that two or more files are to use the same memory area for processing of the current logical record. All of the files may be open at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each opened output file whose file-name appears in this SAME RECORD AREA clause and of the most recently read input file whose file–name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; in other words, records are aligned on the leftmost character position.
6. The MULTIPLE FILE clause is required when more than one file shares the same physical reel of tape. Regardless of the number of files on a single reel, only those files that are used in the object program need be specified. If all file-names have been listed in consecutive order, the POSITION clause need not be given. If any file in the sequence is not listed, the position relative to the beginning of the tape must be given. No more than one file on the same tape reel may be open at one time. Any file on the reel, whether listed or not in the MULTIPLE FILE TAPE clause, must be a labeled file.

4.3. Data Division

4.3.1. File Section

In a COBOL program the file description entry (FD) represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file-name, and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of implementor-defined label items, the names of the data records which compose the file, and the number of lines to be written on a logical printer page. The entry itself is terminated by a period.

4.3.2. Record Description Structure

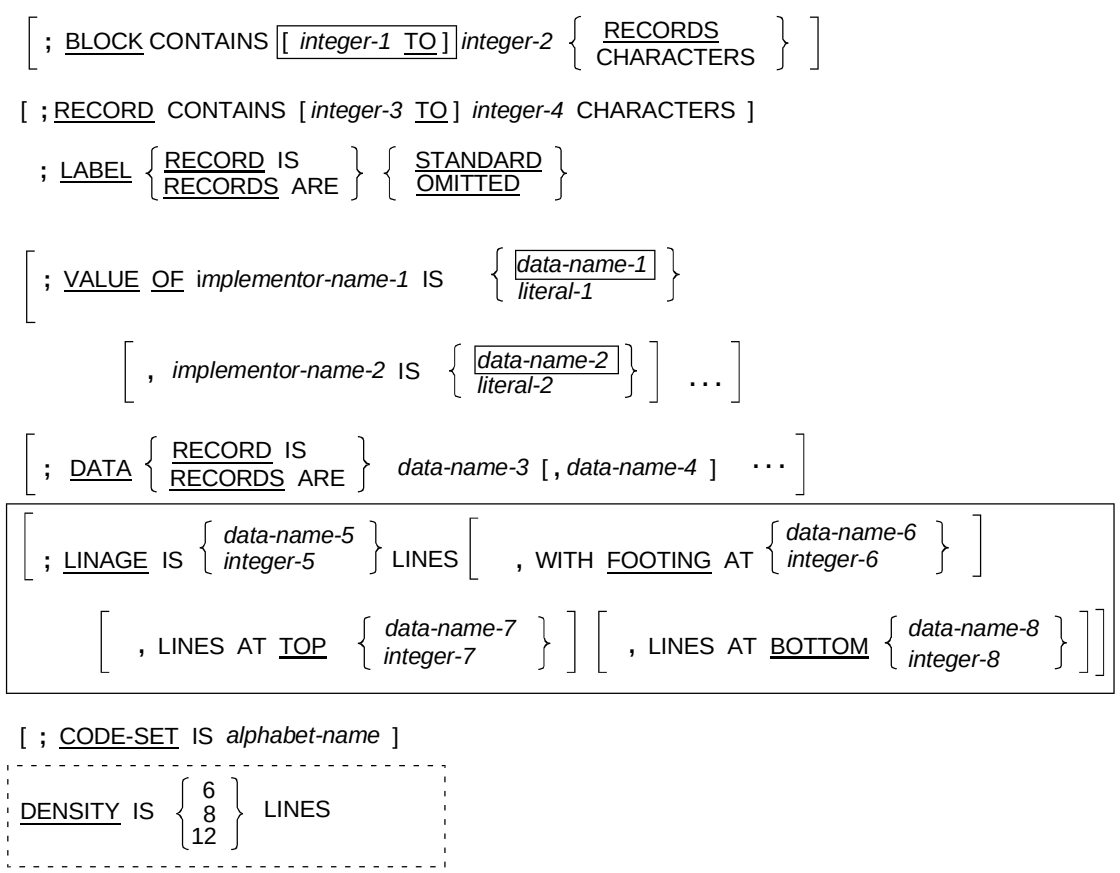
A record description consists of a set of data description entries which describe the characteristics of a particular record. Each data description entry consists of a level-number followed by a data-name if required, followed by a series of independent clauses as required. A record description has a hierarchical structure and therefore the clauses used with an entry may vary considerably, depending upon whether or not it is followed by subordinate entries. The structure of a record description is defined in 1.3.3.3.2, and the elements allowed in a record description are shown in 2.5.2.

4.3.3. The File Description—Complete Entry Skeleton

4.3.3.1. Function

The file description furnishes information concerning the physical structure, identification, and record names pertaining to a given file.

4.3.3.2. General Format



4.3.3.3. Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede *file-name*.
2. The clauses which follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. One or more record description entries must follow the file description entry.

4.3.4. The BLOCK CONTAINS Clause

4.3.4.1. Function

The BLOCK CONTAINS clause specifies the size of a physical record.

4.3.4.2. General Format

```
BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS }
                                          { CHARACTERS }
```

4.3.4.3. General Rules

1. This clause is required except when:
   a. A physical record contains one and only one complete logical record.
   b. The hardware device assigned to the file is an ANS74–PRINTER, CARD–PUNCH, CARD–READER, or PRINTER.
2. The size of the physical record may be stated in terms of RECORDS, unless one of the following situations exists, in which case the RECORDS phrase must not be used.
   a. In mass storage files, where logical records may extend across physical records.
   b. The physical record contains padding (area not contained in a logical record).
   c. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.
3. When the word CHARACTERS is specified, the physical record size is specified in terms of the number of ASCII character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
4. If only *integer-2* is shown, it represents the exact size of the physical record.

   If *integer-1* and *integer-2* are both shown, they refer to the minimum and maximum size of the physical record, respectively.

4.3.5. The CODE–SET Clause

4.3.5.1. Function

The CODE–SET clause specifies the character code set used to represent data on the external media.

4.3.5.2. General Format

```
CODE–SET  IS alphabet-name
```

4.3.5.3. Syntax Rules

1. When the CODE–SET clause is specified for a file, all data in that file must be described as usage is DISPLAY and any signed numeric data must be described with the SIGN IS SEPARATE clause.
2. The *alphabet–name* clause referenced by the CODE–SET clause must not specify the literal phrase.
3. The CODE–SET clause may only be specified for non-mass storage files.

4.3.5.4. General Rules

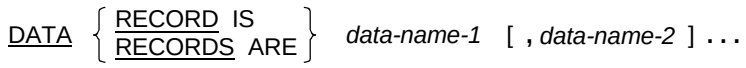
1. If the CODE–SET clause is specified, *alphabet-name* specifies the character code convention used to represent data on the external media. It also specifies the algorithm for converting the character codes on the external media from/to the native character codes. This code conversion occurs during the execution of an input or output operation. (See 2.4.1.3.1, The SPECIAL–NAMES paragraph.)
2. If the CODE–SET clause is not specified, the native character code set is assumed for data on the external media.

4.3.6. The DATA RECORDS Clause

4.3.6.1. Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

4.3.6.2. General Format



4.3.6.3. Syntax Rules

1. *Data-name-1* and *data-name-2* are the names of data records and must have 01 level-number record descriptions, with the same names, associated with them.

4.3.6.4. General Rules

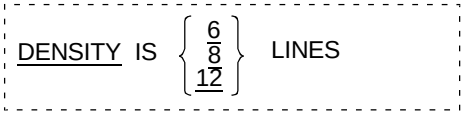
1. The presence of more than one data-name indicates that the file contains more than one type of data record. These records may be of different sizes, different formats, etc. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

4.3.7. The DENSITY Clause

4.3.7.1. Function

The DENSITY clause specifies either 6, 8, or 12 lines per inch on a printer file.

4.3.7.2. General Format



4.3.7.3. Syntax Rules

1. The DENSITY clause is only meaningful for printer files.

4.3.7.4. General Rules

1. Depending on the density chosen, the printer will print 6, 8 or 12 lines per inch. (See 4.4.2.)

4.3.8. The LABEL RECORDS Clause

4.3.8.1. Function

The LABEL RECORDS clause specifies whether labels are present.

4.3.8.2. General Format

```
LABEL { RECORD IS  } { STANDARD }
      { RECORDS ARE} { OMITTED  }
```

4.3.8.3. Syntax Rules

1. This clause is required in every file description entry.

4.3.8.4. General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that labels exist for the file or the device to which the file is assigned (see Appendix E).

4.3.9. The LINAGE Clause

4.3.9.1. Function

The LINAGE clause provides a means for specifying the depth of a logical page in terms of number of lines. It also provides for specifying the size of the top and bottom margins on the logical page, and the line number, within the page body, at which the footing area begins.

4.3.9.2. General Format

```
LINAGE IS { data-name-1 } LINES [ , WITH FOOTING AT { data-name-2 } ]
          { integer-1   }       [                   { integer-2   } ]

[ , LINES AT TOP { data-name-3 } ] [ , LINES AT BOTTOM { data-name-4 } ]
                 { integer-3   }   [                   { integer-4   } ]
```

4.3.9.3. Syntax Rules

1. *Data-name-1*, *data-name-2*, *data-name-3,* and *data-name-4* must reference elementary unsigned numeric integer data items.
2. The value of *integer-1* must be greater than zero.
3. The value of *integer-2* must not be greater than *integer-1*.
4. The value of *integer-3*, *integer-4* may be zero.

4.3.9.4. General Rules

1. The LINAGE clause provides a means for specifying the size of a logical page in terms of number of lines. The logical page size is the sum of the values referenced by each phrase except the FOOTING phrase. If the LINES AT TOP or LINES AT BOTTOM phrases are not specified, the values for these functions are zero. If the FOOTING phrase is not specified, the assumed value is equal to *integer-1*, or the contents of the data item referenced by *data-name-1*, whichever is specified.

   There is not necessarily any relationship between the size of the logical page and the size of a physical page.

2. The value of *integer-1* or the data item referenced by *data-name-1* specifies the number of lines that can be written and spaced on the logical page. The value must be greater than zero. That part of the logical page in which these lines can be written or spaced is called the page body.
3. The value of *integer-3* or the data item referenced by *data-name-3* specifies the number of lines that compose the top margin on the logical page. The value may be zero.
4. The value of *integer-4* or the data item referenced by *data-name-4* specifies the number of lines that compose the bottom margin on the logical page. The value may be zero.
5. The value of *integer-2* or the data item referenced by *data-name-2* specifies the line number within the page body at which the footing area begins. The value must be greater than zero and not greater than the value of *integer-1* or the data item referenced by *data-name-1*.

   The footing area comprises the area of the logical page between the line represented by the value of *integer-2* or the data item referenced by *data-name-2* and the line represented by the value of *integer-1* or the data item referenced by *data-name-1*, inclusive.

6. The value of *integer-1*, *integer-3*, and *integer-4*, if specified, will be used at the time the file is opened by the execution of an OPEN statement with the OUTPUT phrase, to specify the number of lines that comprise each of the indicated sections of a logical page. The value of *integer-2*, if specified, will be used at that time to define the footing area. These values are used for all logical pages written for the file during a given execution of the program.

7. The values of the data items referenced by *data-name-1*, *data-name-3*, and *data-name-4*, if specified, will be used as follows:

    a. The values of the data items, at the time an OPEN statement with the OUTPUT phrase is executed for the file, will be used to specify the number of lines that are to compose each of the indicated sections for the first logical page.

    b. The values of the data items, at the time a WRITE statement with the ADVANCING PAGE phrase is executed or page overflow condition occurs (see 4.4.6), will be used to specify the number of lines that are to compose each of the indicated sections for the next logical page.

8. The value of the data item referenced by *data-name-2*, if specified, at the time an OPEN statement with the OUTPUT phrase is executed for the file, will be used to define the footing area for the first logical page. At the time a WRITE statement with the ADVANCING PAGE phrase is executed or a page overflow condition occurs, it will be used to define the footing area for the next logical page.

9. A LINAGE–COUNTER is generated by the presence of a LINAGE clause. The value in the LINAGE–COUNTER at any given time represents the line number at which the device is positioned within the current page body. The rules governing the LINAGE–COUNTER are:

    a. A separate LINAGE–COUNTER is supplied for each file described in the File Section whose file description entry contains a LINAGE clause.

    b. LINAGE–COUNTER may be referenced, but may not be modified, by Procedure Division statements. Since more than one LINAGE–COUNTER may exist in a program, the user must qualify LINAGE–COUNTER by file-name when necessary.

    c. LINAGE–COUNTER is automatically modified, according to the following rules, during the execution of a WRITE statement to an associated file:

        1) When the ADVANCING PAGE phrase of the WRITE statement is specified, LINAGE–COUNTER is automatically reset to one.

        2) When the ADVANCING *identifier-2* or *integer* phrase of the WRITE statement is specified, LINAGE–COUNTER is incremented by *integer* or the value of the data item referenced by *identifier-2*.

        3) When the ADVANCING phrase of the WRITE statement is not specified, LINAGE–COUNTER is incremented by the value one. (See 4.4.6.)

        4) The value of LINAGE–COUNTER is automatically reset to one when the device is repositioned to the first line that can be written on for each of the succeeding logical pages. (See 4.4.6.)

    d. The value of LINAGE–COUNTER is automatically set to one at the time an OPEN statement is executed for the associated file.

10. Each logical page is contiguous to the next with no additional spacing provided.

4.3.10. The RECORD CONTAINS Clause

4.3.10.1. Function

The RECORD CONTAINS clause specifies the size of data records.

4.3.10.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

4.3.10.3. General Rules

The size of each data record is completely defined within the record description entry, therefore this clause is never required. When present, however, the following notes apply:

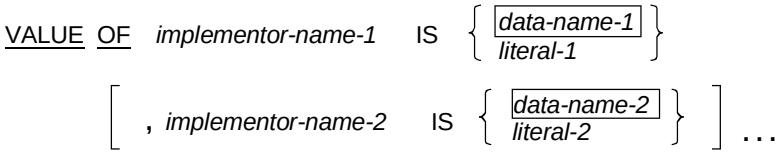
1. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case *integer-2* represents the exact number of characters in the data record. If *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.
2. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. (See 1.3.3.3.4, 2.5.11, and 2.5.12.)

4.3.11. The VALUE OF Clause

4.3.11.1. Function

The VALUE OF clause particularizes the description of an item in the label records associated with a file.

4.3.11.2. General Format



4.3.11.3. Syntax Rules

1. The allowable implementor-names are:

For CARD–PUNCH Files	For PRINTER Files	For TAPE Files	For DISC Files
FORMS–REQUEST	CARTRIDGE–ID FORMS–REQUEST	FILE–ACCESS FILE–ID PURGE–DATE	FILE–ID

These implementor-names have the implied definition of:

```
FORMS-REQUEST      PIC X(24).

CARTRIDGE-ID       PIC XX.

FILE-ID            PIC X(17).

FILE-ACCESS        PIC X.

PURGE-DATE         PIC 9(6).
```

2. *Data-name-1*, *data-name-2*, etc., should be qualified when necessary, but cannot be subscripted or indexed, nor can they be items described with the USAGE IS INDEX clause.
3. *Data-name-1*, *data-name-2*, etc., must be in the Working-Storage Section.
4. *Data-name-1*, *data-name-2*, etc., must be described as USAGE DISPLAY.

4.3.11.4. General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to the value of *literal-1*, or of *data-name-1*, whichever has been specified. (See 4.4.2 and Appendix E.)

   For an output file, at the appropriate time the value of *implementor-name-1* is made equal to the value of *literal-1*, or of a *data-name-1*, whichever has been specified. (See 4.4.2 and Appendix E.)

2. A figurative constant may be substituted in the preceding format wherever a literal is specified.
3. The contents of *literal-1* or the data item referenced by *data-name-1* specified in the VALUE OF CARTRIDGE–ID clause are used to specify the cartridge to be used for this print file. These contents should contain the cartridge-id code that appears on the cartridge case. (See 4.4.2, 8.3.2, and 18.10.)
4. The contents of *literal-1* or the data item referenced by *data-name-1* specified in the VALUE OF FORMS–REQUEST clause are used to specify the special form message for this card-punch or printer file. These contents should contain the form type, name or number required by this file at execution time. (See 4.4.2, 8.3.2, and 18.10.)

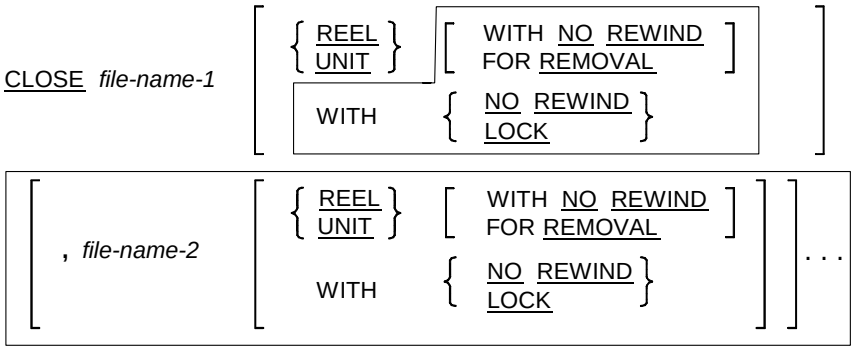
4.4. Procedure Division

4.4.1. The CLOSE Statement

4.4.1.1. Function

The CLOSE statement terminates the processing of reels/units and files with optional rewind and/or lock or removal where applicable.

4.4.1.2. General Format



4.4.1.3. Syntax Rules

1. The REEL or UNIT phrase must only be used for sequential files.
2. The files referenced in the CLOSE statement need not all have the same organization or access.

4.4.1.4. General Rules

Except where otherwise stated in the general rules which follow, the terms "reel" and "unit" are synonymous and completely interchangeable in the CLOSE statement. Treatment of sequential mass storage files is logically equivalent to the treatment of a file on tape or analogous sequential media.

1. The REEL or UNIT phrase must only be used for files assigned to TAPE.
2. A CLOSE statement may only be executed for a file in an open mode.
3. For the purpose of showing the effect of various types of CLOSE statements as applied to various storage media, all files are divided into the following categories:
   a. Non-reel/unit. A file whose input or output medium is such that the concepts of rewind and reels/units have no meaning. This includes all symbiont and mass storage files.
   b. Sequential single-reel/unit. A sequential file that is entirely contained on one reel/unit. This includes only tape files.
   c. Sequential multi-reel/unit. A sequential file that is contained on more than one reel/unit. This includes only tape files.

4. The results of executing each type of CLOSE for each category of file are summarized in Table 4-1.

Table 4–1. Relationship of Categories of Files and the Formats of the CLOSE Statement

CLOSE Statement Format	File Category		
	Non-Reel/Unit	Sequential Single-Reel/Unit	Sequential Multi-Reel/Unit
CLOSE	C	C,G	C,G,A
CLOSE WITH LOCK	C,E	C,G,E	C,G,E,A
CLOSE WITH NO REWIND	X	C,B	C,B,A
CLOSE REEL/UNIT	X	X	F,G
CLOSE REEL/UNIT FOR REMOVAL	X	X	F,D,G
CLOSE REEL/UNIT WITH NO REWIND	X	X	F,B

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output, or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input–output files.

Symbol Description

A Previous Reels/Units Unaffected

Input Files and Input-Output Files:

All reels/units in the file prior to the current reel/unit are processed according to the standard reel/unit swap procedure, except those reels/units controlled by a prior CLOSE REEL/UNIT statement. If the current reel/unit is not the last in the file, the reels/units in the file following the current one are not processed. (See Appendix E.)

Output Files:

All reels/units in the file prior to the current reel/unit are processed according to the standard reel/unit swap procedure, except those reels/units controlled by a prior CLOSE REEL/UNIT statement. (See Appendix E.)

B No Rewind of Current Reel

The current reel/unit is left in its current position.

The No Rewind option is ignored if specified on a CLOSE REEL/UNIT statement.

C Close File

Input Files and Input–Output Files:

If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing. (See Appendix E.)

Output Files:

If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed. (See Appendix E.)

D Reel/Unit Removal

The current reel or unit is rewound when applicable, and the operating system is notified that the reel or unit is logically removed from this run unit; however, the reel or unit may be accessed again, in its proper order of reels or units within the file, if a CLOSE statement without the REEL or UNIT phrase is subsequently executed for this file followed by the execution of an OPEN statement for the file.

E File Lock

This file cannot be opened again during this execution of this run unit.

F Close Reel/Unit

Input Files:

The following operations take place:

1. A reel/unit swap.
2. The standard beginning reel/unit label procedure is executed.

The next executed READ statement for that file makes available the next data record on the new reel/unit.

Output Files and Input-Output Files**:**

The following operations take place:

(1) (For output files only.) The standard ending reel/unit label procedure is executed.

(2) A reel/unit swap.

(3) The standard beginning reel/unit label procedure is executed.

For input-output files, the next executed READ statement that references that file makes the next logical data record on the next mass storage unit available. For output files, the next executed WRITE statement that references that file directs the next logical data record to the next reel/unit of the file.

G Rewind

The current reel or analogous device is positioned at its physical beginning.

X Illegal

This is an illegal combination of a CLOSE option and a file category. The results at object time are undefined.

5. The action taken if the file is in the open mode when a STOP RUN statement is executed or the action taken for a file that has been opened in a called program and not closed in that program prior to the execution of a CANCEL statement for that program are undefined. No implicit CLOSE procedures will be initiated. This could result in the condition which would cause the file to be unusable on subsequent accesses to it.

6. If the OPTIONAL phrase has been specified for the file in the FILE– CONTROL paragraph of the Environment Division and the file is not present, the standard end-of-file processing is not performed for that file.

7. If a CLOSE statement without the REEL or UNIT phrase has been executed for a file, no other statement (except the SORT or MERGE statements with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly, unless an intervening OPEN statement for that file is executed.

8. The WITH NO REWIND and FOR REMOVAL phrases will have no effect at object time if they do not apply to the storage media on which the file resides.

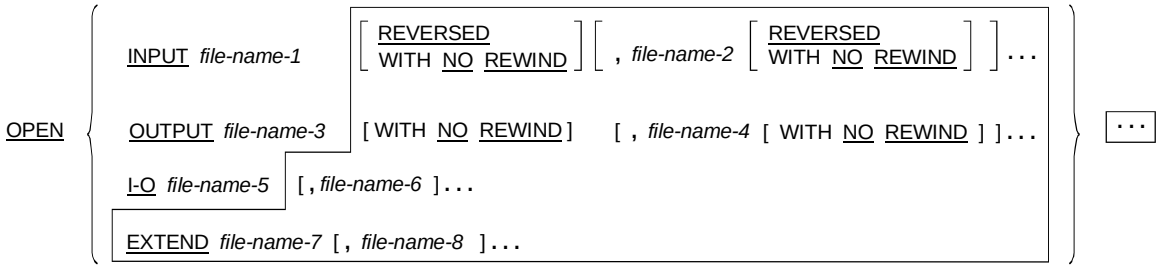
9. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the record area associated with *file-name* is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.

4.4.2. The OPEN Statement

4.4.2.1. Function

The OPEN statement initiates the processing of files. It also performs checking and writing of labels and other input-output operations.

4.4.2.2. General Format



4.4.2.3. Syntax Rules

1. The REVERSED and the NO REWIND phrases can only be used with sequential files. (See 4.4.1.)
2. The I–O phrase can be used only for mass storage files.
3. The EXTEND phrase can be used only for sequential files.
4. The EXTEND phrase must not be specified for multiple file reels. (See 4.2.1.3)
5. The files referenced in the OPEN statement need not all have the same organization or access.

4.4.2.4. General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode.
2. The successful execution of an OPEN statement makes the associated record area available to the program**.**
3. Prior to the successful execution of an OPEN statement for a given file, no statement (except for a SORT or MERGE statement with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 4-2, "X" at an intersection indicates that the specified statement, used in the sequential access mode, may be used with the sequential file organization and open mode given at the top of the column.

Table 4-2. Permissible Statements

Statement	Open Mode			
	Input	Output	Input–Output	Extend
READ	X		X	
WRITE		X		X
REWRITE			X	

5. A file may be opened with the INPUT, OUTPUT, EXTEND, and I–O phrases in the same program. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the REEL, UNIT, or LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file (see Appendix E), the beginning labels are processed as follows:

   a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.

   b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.

8. The file description entry for *file-name-1*, *file-name-2*, *file-name-5*, *file-name-6*, *file-name-7*, or *file-name-8* must be equivalent to that used when this file was created.
9. If an input file is designated with the OPTIONAL phrase in its SELECT clause, the object program causes an interrogation for the presence or absence of this file. If the file is not present, the first READ statement for this file causes the AT END condition to occur. (See 4.4.3.)
10. The REVERSED and NO REWIND phrases can only be used with sequential single reel/unit files. (See 4.4.1.)
11. The REVERSED and WITH NO REWIND phrases will be ignored if they do not apply to the storage media on which the file resides.

12. If the storage medium for the file permits rewinding, the following rules apply:
    a. When neither the REVERSED, the EXTEND, nor the NO REWIND phrase is specified, execution of the OPEN statement positions the file at its beginning.
    b. When the NO REWIND phrase is specified, execution of the OPEN statement does not reposition the file; the file must be already positioned at its beginning prior to execution of the OPEN statement.
    c. When the REVERSED phrase is specified, the file is positioned at its end by execution of the OPEN statement.
13. When the REVERSED phrase is specified, the subsequent READ statements for the file make the data records of the file available in reversed order, that is, starting with the last record.
14. For files being opened with the INPUT or I–O phrase, the OPEN statement sets the current record pointer to the first record currently existing within the file. If no records exist in the file, the current record pointer is set such that the next executed READ statement for the file will result in an AT END condition. If opening an alternate card-reader file, the first read retrieves the first record unless the file has not been closed by the last program that used it. If the file was not closed, the first read will be the next record following the last record read by the last program using the file.
15. When the EXTEND phrase is specified, the OPEN statement positions the file immediately following the last logical record of that file. Subsequent WRITE statements referencing the file will add records to the file as though the file had been opened with the OUTPUT phrase.
16. When the EXTEND phrase is specified and the LABEL RECORDS clause indicates label records are present, the execution of the OPEN statement includes the following steps:
    a. The beginning file labels are processed only in the case of a single reel/unit file.
    b. The beginning reel/unit labels on the last existing reel/unit are processed as though the file was being opened with the INPUT phrase.
    c. The existing ending file labels are processed as though the file is being opened with the INPUT phrase. These labels are then deleted.
    d. Processing then proceeds as though the file had been opened with the OUTPUT phrase.
17. Since the EXTEND phrase implies the existence of the file, it cannot be used to initially create the file.
18. The I–O phrase permits the opening of a mass storage file for both input and output operations. Since this phrase implies the existence of the file, it cannot be used if the mass storage file is being initially created.

19. When the I–O phrase is specified and the LABEL RECORDS clause indicates label records are present (see Appendix E), the execution of the OPEN statement includes the following steps:

    a. The labels are checked in accordance with the conventions for input-output label checking.

    b. The new labels are written in accordance with the conventions for input-output label writing.

20. Upon successful execution of an OPEN statement with the OUTPUT phrase specified, a file is created. At that time the associated file contains no data records.

    If opening an alternate unit record file, no file is created unless the file is a null file or it has been closed by the last program that used it. If the file was not closed, the first write will follow the last record previously written by the last program using the file.

21. When the DENSITY clause is present for other than a report file the execution of the OPEN statement will set the density specified.

22. When the VALUE OF CARTRIDGE–ID clause is present for other than a report file, the execution of the OPEN statement will request the cartridge specified. (See 18.10.)

23. When the VALUE OF FORMS–REQUEST clause is present for other than a report file, the execution of the OPEN statement will request the form specified. (See 18.10.)

24. When the LINAGE and REPORT clauses are not present for a file assigned to PRINTER, execution of the OPEN statement will cause the page margins to be set from the system standard margins (see *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899) (current version)).

25. When the LINAGE and REPORT clauses are not present for a file assigned to ANS74–PRINTER, execution of the OPEN statement will cause the page to be defined as system standard lines with no margins (see *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899) (current version)).

26. For non-report printer files, execution of the OPEN statement will position the printer to the first printable line on the first page.

4.4.3. The READ Statement

4.4.3.1. Function

The READ statement makes available the next logical record from a file.

4.4.3.2. General Format

```
READ file-name RECORD [ INTO  identifier ] [ ;  AT  END  imperative-statement ]
```

4.4.3.3. Syntax Rules

1. The INTO phrase must not be used when the input file contains logical records of various sizes as indicated by their record descriptions. The storage area associated with *identifier* and the record area associated with *file-name* must not be the same storage area.
2. The AT END phrase must be specified if no applicable USE procedure is specified for *file-name*.

4.4.3.4. General Rules

1. The associated file must be open in the INPUT or I–O mode at the time this statement is executed. (See 4.4.2.)
2. The record to be made available by the READ statement is determined as follows:
   a. If the current record pointer was positioned by the execution of the OPEN statement, the record pointed to by the current record pointer is made available.
   b. If the current record pointer was positioned by the execution of a previous READ statement, the current record pointer is updated to point to the next existing record in the file and then that record is made available.
3. The execution of the READ statement causes the value of the FILE STATUS data item, if any, associated with *file-name* to be updated. (See 4.1.3.4.)
4. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged in that a record is available to the object program prior to the execution of any statement following the READ statement.
5. When the logical records of a file are described with more than one record description, these records automatically share the same storage area. This is equivalent to an implicit redefinition of the area. The contents of any data items which lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
6. If the INTO phrase is specified, the record being read is moved from the record area to the area specified by *identifier* according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if the execution of the READ statement was unsuccessful. Any subscripting or indexing associated with *identifier* is evaluated after the record has been read and immediately before it is moved to the data item.

7. When the INTO phrase is used, the record being read is available in both the input record area and the data area associated with *identifier*.
8. If, at the time of execution of a READ statement, the position of current record pointer for that file is undefined, the execution of that READ statement is unsuccessful.
9. If the end of a reel or unit is recognized during execution of a READ statement, and the logical end of the file has not been reached, the following operations are executed:
    a. The standard ending reel/unit label procedure.
    b. A reel/unit swap.
    c. The standard beginning reel/unit label procedure.
    d. The first data record of the new reel/unit is made available.
10. If a file described with the OPTIONAL phrase is not present at the time the file is opened, then at the time of execution of the first READ statement for the file, the AT END condition occurs and the execution of the READ statement is unsuccessful. The standard end-of-file procedures are not performed. (See 4.2.1.1 for the FILE–CONTROL paragraph, 4.4.2 for the OPEN statement, 4.4.5 for the USE statement, and 4.1.3.4 for I–O status.) Execution of the program then proceeds as specified in general rule 12.
11. If, at the time of the execution of a READ statement, no next logical record exists in the file, the AT END condition occurs, and the execution of the READ statement is considered unsuccessful. (See 4.1.3.4.)
12. When the AT END condition is recognized the following actions are taken in the specified order:
    a. A value is placed into the FILE STATUS data item, if specified for this file, to indicate an AT END condition. (See 4.1.3.4.)
    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to the AT END imperative-statement. Any USE procedure specified for this file is not executed.
    c. If the AT END phrase is not specified, then a USE procedure must be specified, either explicitly or implicitly, for this file and that procedure is executed.

       When the AT END condition occurs, execution of the input-output statement which caused the condition is unsuccessful.
13. Following the unsuccessful execution of any READ statement, the contents of the associated record area and the position of the current record pointer are undefined.
14. When the AT END condition has been recognized, a READ statement for that file must not be executed without first executing a successful CLOSE statement followed by the execution of a successful OPEN statement for that file.

4.4.4. The REWRITE Statement

4.4.4.1. Function

The REWRITE statement logically replaces an existing record in a mass storage file.

4.4.4.2. General Format

```
REWRITE record-name [ FROM identifier ]
```

4.4.4.3. Syntax Rules

1. *Record-name* and *identifier* must not refer to the same storage area.
2. *Record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.

4.4.4.4. General Rules

1. The file associated with *record–name* must be a mass storage file and must be open in the I–O mode at the time of execution of this statement. (See 4.4.2.)
2. The last input-output statement executed for the associated file prior to the execution of the REWRITE statement must have been a successfully executed READ statement. The MSCS logically replaces the record that was accessed by the READ statement.
3. The number of character positions in the record referenced by *record-name* must be equal to the number of character positions in the record being replaced.
4. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause, in which case the logical record is available to the program as a record of other files appearing in the same SAME RECORD AREA clause as the associated I–O file, as well as to the file associated with *record-name*.
5. The execution of a REWRITE statement with the FROM phrase is equivalent to the execution of:

   ```
   MOVE identifier TO record-name
   ```

   followed by the execution of the same REWRITE statement without the FROM phrase. The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of the REWRITE statement.

6. The current record pointer is not affected by the execution of a REWRITE statement.
7. The execution of the REWRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 4.1.3.4.)

4.4.5. The USE Statement

4.4.5.1. Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

4.4.5.2. General Format

```
USE AFTER STANDARD { EXCEPTION } PROCEDURE ON { file-name-1 [ , file-name-2 ] ... }
                   { ERROR     }              { INPUT                           }
                                              { OUTPUT                          }
                                              { I-O                             }
                                              { EXTEND                          } .
```

4.4.5.3. Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives section and must be followed by a period followed by a space. The remainder of the section must consist of zero or one or more procedural paragraphs that define the procedures to be used.
2. The USE statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.
3. The same *file-name* can appear in a different specific arrangement of the format. Appearance of a *file-name* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and may be used interchangeably.
5. The files implicitly or explicitly referenced in a USE statement need not all have the same organization or access.

4.4.5.4. General Rules

1. The designated procedures are executed by the input-output system after completing the standard input-output error routine, or upon recognition of the AT END condition, when the AT END phrase has not been specified in the input-output statement.
2. After execution of a USE procedure, control is returned to the invoking routine if it was invoked by the AT END condition or a Unisys defined status condition (See C.6.).
3. Within a USE procedure, there must not be any reference to any nondeclarative procedures. Conversely, in the nondeclarative portion there must be no reference to procedure-names that appear in the declarative portion, except that PERFORM statements may refer to a USE statement or to the procedures associated with such a USE statement.
4. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

4.4.6. The WRITE Statement

4.4.6.1. Function

The WRITE statement releases a logical record for an output file. It can also vertically position lines within a logical page.

4.4.6.2. General Format

```
WRITE record-name [ FROM identifier-1 ]

[ { BEFORE } ADVANCING { { identifier-2 } [ LINE  ] } ]
  { AFTER  }           { { integer      } [ LINES ] }
                       { { mnemonic-name }         }
                       { { PAGE          }         }

[ ; AT { END-OF-PAGE } imperative-statement ]
       { EOP        }
```

4.4.6.3. Syntax Rules

1. *Record-name* and *identifier-1* must not reference the same storage area.
2. When *mnemonic–name* is specified, the name must be associated with PAGE. The *mnemonic-name* is defined in the SPECIAL–NAMES paragraph of the Environment Division.
3. The *record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.
4. When *identifier-2* is used in the ADVANCING phrase, it must be the name of an elementary integer data item.
5. *Integer* or the value of the data item referenced by *identifier-2* may be zero.
6. If the END–OF–PAGE phrase is specified, the LINAGE clause must be specified in the file description entry for the associated file.
7. The words END–OF–PAGE and EOP are equivalent.
8. The ADVANCING *mnemonic-name* phrase cannot be specified when writing a record to a file whose file description entry contains the LINAGE clause.

4.4.6.4. General Rules

1. The associated file must be open in the OUTPUT or EXTEND mode at the time of the execution of this statement. (See 4.4.2.)
2. The logical record released by the execution of the WRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause or the execution of the WRITE statement was unsuccessful due to a boundary violation. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as to the file associated with *record-name*.
3. The results of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of:

   a. The statement:

      ```
      MOVE identifier-1 TO record-name
      ```

      according to the rules specified for the MOVE statement, followed by:

   b. The same WRITE statement without the FROM phrase.

   The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of this WRITE statement.

   After execution of the WRITE statement is complete, the information in the area referenced by *identifier-1* is available, even though the information in the area referenced by *record-name* may not be. (See general rule 2.)

4. The current record pointer is unaffected by the execution of a WRITE statement.
5. The execution of the WRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 4.1.3.4.)
6. The maximum record size for a file is established at the time the file is created and must not subsequently be changed.
7. The number of character positions on a mass storage device required to store a logical record in a file may or may not be equal to the number of character positions defined by the logical description of that record in the program.
8. The execution of the WRITE statement releases a logical record to the operating system.
9. Both the ADVANCING phrase and the END–OF–PAGE phrase allow control of the vertical positioning of each line on a representation of a printed page. If the ADVANCING phrase is not used, automatic advancing will be provided to act as if the user had specified AFTER ADVANCING 1 LINE. If the ADVANCING phrase is used, advancing is provided as follows:

   a. If *identifier-2* is specified, the representation of the printed page is advanced the number of lines equal to the current value associated with *identifier-2*.

   b. If *integer* is specified, the representation of the printed page is advanced the number of lines equal to the value of *integer*.

   c. If the BEFORE phrase is used, the line is presented before the representation of the printed page is advanced according to rules a and b above.

d. If the AFTER phrase is used, the line is presented after the representation of the printed page is advanced according to rules a and b.

e. If PAGE or *mnemonic-name* is specified, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the next logical page. If the record to be written is associated with a file whose file description entry contains a LINAGE clause, the repositioning is to the first line that can be written on the next logical page as specified in the LINAGE clause.

If the record to be written is associated with a file description entry that does not contain a LINAGE clause and the file is assigned to ANS74–PRINTER, the repositioning is to the first line on the next system standard page (see *OS 2200 Exec System Software Executive Requests Programming Reference Manual*, (7830 7899).

If the record to be written is associated with a file description entry that does not contain a LINAGE clause, and the file is assigned to PRINTER, the repositioning is to the first line specified by the system standard page definition. (See *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899). If PAGE is specified but page has no meaning in conjunction with a specific device, then advancing will depend on the device specified.

10. If the logical end of the representation of the printed page is reached during the execution of a WRITE statement with the END–OF–PAGE phrase, the *imperative-statement* specified in the END–OF–PAGE phrase is executed. The logical end is specified in the LINAGE clause associated with *record–name*.

11. An end-of-page condition is reached whenever the execution of a given WRITE statement with the END–OF–PAGE phrase causes printing or spacing within the footing area of a page body. This occurs when the execution of such a WRITE statement causes the LINAGE–COUNTER to equal or exceed the value specified by *integer-2* or the data item referenced by *data-name-2* of the LINAGE clause, if specified. In this case, the WRITE statement is executed and then the imperative statement in the END–OF–PAGE phrase is executed.

An automatic page overflow condition is reached whenever the execution of a given WRITE statement (with or without an END–OF–PAGE phrase) cannot be fully accommodated within the current page body.

This occurs when a WRITE statement, if executed, would cause the LINAGE–COUNTER to exceed the value specified by *integer-1* or the data item referenced by *data-name-1* of the LINAGE clause. In this case, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the first line that can be written on the next logical page as specified in the LINAGE clause. The imperative statement in the END–OF–PAGE clause, if specified, is executed after the record is written and the device has been repositioned.

If *integer-2* or *data-name-2* of the LINAGE clause is not specified, no end-of-page condition distinct from the page overflow condition is detected. In this case, the end-of-page condition and page overflow condition occur simultaneously.

If *integer-2* or *data-name-2* of the LINAGE clause is specified, but the execution of a given WRITE statement would cause LINAGE–COUNTER to simultaneously exceed the value of both *integer-2* or the data item referenced by *data-name-2* and *integer-1* or the data item referenced by *data-name-1*, then the operation proceeds as if *integer-2* or *data-name-2* had not been specified.

12. When a file description entry does not contain a LINAGE clause and the file is assigned to PRINTER, a page overflow condition is reached whenever execution of a given WRITE statement (with or without an END–OF–PAGE phrase) cannot be fully accommodated within the system standard page definition (see 4.4.2.4, general rule 24). In this case, the record is presented on the logical page before or after (depending on the phrase used) the device is repositioned to the first line that can be written on the next logical page, as specified by the system standard page definition.
13. No page overflow condition can occur when a file description entry does not contain a LINAGE clause and the file is assigned to ANS74–PRINTER (see 4.4.2.4, general rule 25).
14. When an attempt is made to write beyond the externally defined boundaries of a sequential file, an exception condition exists and the contents of the record area are unaffected. The following action takes place:
    a. The value of the FILE STATUS data item, if any, of the associated file is set to a value indicating a boundary violation. (See 4.1.3.4.)
    b. If a USE AFTER STANDARD EXCEPTION declarative is explicitly or implicitly specified for the file, that declarative procedure will then be executed.
    c. If a USE AFTER STANDARD EXCEPTION declarative is not explicitly or implicitly specified for the file, the result is undefined.
15. After the recognition of an end-of-reel or an end-of-unit of an output file that is contained on more than one physical reel/unit, the WRITE statement performs the following operations:
    a. The standard ending reel/unit label procedure.
    b. A reel/unit swap.
    c. The standard beginning reel/unit label procedure.

4.5. Example

4.5.1. Source Program

```
000005 IDENTIFICATION DIVISION.
000010 PROGRAM-ID.
000015     SEQUENTIAL-FILE-EXAMPLE.
000020 ENVIRONMENT DIVISION.
000025 CONFIGURATION SECTION.
000030 SOURCE-COMPUTER.  UNISYS-2200.
000035 OBJECT-COMPUTER.  UNISYS-2200.
000040 INPUT-OUTPUT SECTION.
000045 FILE-CONTROL.
000050*
000055     SELECT REPORT-FILE
000060       ASSIGN TO PRINTER ALT-FILE
000065       ORGANIZATION IS SEQUENTIAL.
000070*
000075     SELECT BACKUP-FILE
000080       ASSIGN TO TAPE
000085       ORGANIZATION IS SEQUENTIAL.
000090*
000095     SELECT MASTER-FILE
000100       ASSIGN TO DISC IMP-NAME
000105       ORGANIZATION IS SEQUENTIAL.
000110*
000115     SELECT INPUT-FILE
000120       ASSIGN TO CARD-READER
000125       FILE STATUS IS INPUT-FILE-STATUS.
000130*
000135 I-O-CONTROL.
000140     SAME RECORD AREA FOR BACKUP-FILE, MASTER-FILE.
000145 DATA DIVISION.
000150 FILE SECTION.
000155*
000160 FD  INPUT-FILE
000165     LABEL RECORDS ARE OMITTED.
000170 01  INPUT-RECORDPIC X(50).
000175*
000180 FD  MASTER-FILE
000185     LABEL RECORDS ARE STANDARD
000190     VALUE OF FILE-ID IS "MASTER-FILE"
000195     BLOCK CONTAINS 100 RECORDS.
000200 01  MASTER-RECORD PIC X(80).
000205*
000210 FD  REPORT-FILE
000215     LABEL RECORDS ARE OMITTED
000220     VALUE OF FORMS-REQUEST IS "FORM-118"
000225     LINAGE IS 16 LINES
```

```
000230       WITH FOOTING AT FOOTING-VALUE
000235       LINES AT TOP 4
000240       LINES AT BOTTOM 4
000245     DENSITY IS 8 LINES.
000250 01  REPORT-RECORD PIC X(80).
000255*
000260 FD  BACKUP-FILE
000265     LABEL RECORDS ARE STANDARD
000270     VALUE OF FILE-ID IS "MASTER-BACKUP"
000275     BLOCK CONTAINS 100 RECORDS.
000280 01  BACKUP-RECORD PIC X(80).
000285 WORKING-STORAGE SECTION.
000290 01  INPUT-FILE-ERROR-RECORD.
000295     05 FILLER VALUE
000300                 "INPUT FILE ERROR STATUS KEY EQUALS " PIC A(35).
000305     05 INPUT-FILE-STATUS VALUE ZERO.
000310        10 STATUS-KEY-1 PIC X.
000315           88 INPUT-FILE-AT-END VALUE "1".
000320           88 INPUT-FILE-ERROR VALUE "3"  "9".
000325        10 STATUS-KEY-2 PIC X.
000330 01  LINE-ADVANCE PIC 99.
000335 01  FOOTING-VALUE PIC 99 VALUE 12.
000340 01  REPORT-HEAD.
000345     05 FILLER VALUE "REPORT  HEADING" PIC A(15).
000350     05 FILLER PIC X(3) VALUE SPACES.
000355     05 DATE-VALUE.
000360        10 FILLER PIC XX VALUE SPACES.
000365        10 FILLER VALUE "THE DATE IS "
000370                  PIC X(15).
000375        10 REPORT-DATE-EDIT PIC 99/99/99.
000380 01  REPORT-DATE PIC 9(6).
000385*
000390 01  REPORT-RECORD-FORMAT.
000395     05 REPORT-RECORD-ENTRY PIC X(50).
000400     05 FILLER VALUE
000405              "THIS IS PRINTED ON LINE " PIC X(25).
000410     05 LINAGE-COUNTER-VALUE PIC ZZ9.
000415 01  PAGE-FOOTING.
000420     05 FILLER VALUE "PAGE NUMBER " PIC A(13).
000425     05 PAGE-NUMBER-EDIT PIC ZZZ9.
000430 01  REPORT-FOOTING VALUE "END OF REPORT" PICTURE A(13).
000435 01  PAGE-NUMBER PIC 9(4) VALUE 0.
000440 PROCEDURE DIVISION.
000445 DECLARATIVES.
000450 REPORT-FILE-AT-END SECTION.
000455*
000460     USE AFTER STANDARD EXCEPTION PROCEDURE ON INPUT-FILE.
000465*
```

```
000470 CHECK-INPUT-FILE-ERROR-BEGIN.
000475     IF INPUT-FILE-ERROR
000480       WRITE REPORT-RECORD FROM INPUT-FILE-ERROR-RECORD
000485            BEFORE ADVANCING 1
000490            CLOSE MASTER-FILE
000495            REPORT-FILE
000500       STOP RUN
000505     ELSE
000510       NEXT SENTENCE.
000515 CHECK-INPUT-FILE-ERROR-END.
000520     EXIT.
000525 END DECLARATIVES.
000530*
000535 CONTROLLER SECTION.
000540 MAIN-CONTROL.
000545     PERFORM OPEN-FILES.
000550     PERFORM MAIN-FUNCTION.
000555     PERFORM CLOSE-FILES.
000560     PERFORM CREATE-BACKUP.
000565     PERFORM FINAL-CLOSE.
000570     STOP RUN.
000575*
000580 OPEN-FILES SECTION.
000585 OPEN-BEGIN.
000590     OPEN INPUT INPUT-FILE
000595        OUTPUT REPORT-FILE
000600        EXTEND MASTER-FILE.
000605 OPEN-END.
000610     EXIT.
000615*
000620 MAIN-FUNCTION SECTION.
000625 MAIN-FUNCTION-BEGIN.
000630     PERFORM INITIATE-REPORT.
000635     PERFORM UPDATE-MASTER-GENERATE-REPORT.
000640     PERFORM TERMINATE-REPORT.
000645 MAIN-FUNCTION-END.
000650     EXIT.
000655*
000660 INITIATE-REPORT SECTION.
000665 INITIATE-BEGIN.
000670     ACCEPT REPORT-DATE FROM DATE.
000675     MOVE REPORT-DATE TO REPORT-DATE-EDIT.
000680     MOVE REPORT-HEAD TO REPORT-RECORD-ENTRY.
000685     PERFORM SET-LINE-VALUE.
000690     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
000695        BEFORE ADVANCING 5 LINES.
000700 INITIATE-END.
000705     EXIT.
000710*
```

```
000715 UPDATE-MASTER-GENERATE-REPORT SECTION.
000720 UPDATE-GENERATE-BEGIN.
000725     PERFORM READ-INPUT-FILE.
000730     IF INPUT-FILE-AT-END
000735        GO TO UPDATE-GENERATE-END
000740     ELSE
000745        PERFORM UPDATE-MASTER
000750        PERFORM GENERATE-REPORT
000755      GO TO UPDATE-GENERATE-BEGIN.
000760 UPDATE-GENERATE-END.
000765     EXIT.
000770*
000775 READ-INPUT-FILE SECTION.
000780 READ-INPUT-BEGIN.
000785     READ INPUT-FILE.
000790 READ-INPUT-END.
000795     EXIT.
000800*
000805 UPDATE-MASTER SECTION.
000810 UPDATE-MASTER-BEGIN.
000815     WRITE MASTER-RECORD FROM INPUT-RECORD.
000820 UPDATE-MASTER-END.
000825     EXIT.
000830*
000835 GENERATE-REPORT SECTION.
000840 GENERATE-REPORT-BEGIN.
000845     MOVE INPUT-RECORD TO REPORT-RECORD-ENTRY.
000850     PERFORM SET-LINE-VALUE.
000855     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
000860           BEFORE ADVANCING 2,
000865           AT END-OF-PAGE
000870            PERFORM REPORT-PAGE-FOOTING.
000875 GENERATE-REPORT-END.
000880     EXIT.
000885*
000890 REPORT-PAGE-FOOTING SECTION.
000895 PAGE-FOOTING-BEGIN.
000900     ADD 1 TO PAGE-NUMBER.
000905     MOVE PAGE-NUMBER TO PAGE-NUMBER-EDIT.
000910     MOVE PAGE-FOOTING TO REPORT-RECORD-ENTRY.
000915     PERFORM SET-LINE-VALUE.
000920     WRITE REPORT-RECORD FROM REPORT-RECORD-ûFORMAT
000925           BEFORE ADVANCING 1.
000930 PAGE-EJECT-CHECK.
000935     IF INPUT-FILE-AT-END
000940       GO TO PAGE-FOOTING-END
000945     ELSE
000950       MOVE SPACES TO REPORT-RECORD.
000955       WRITE REPORT-RECORD BEFORE ADVANCING PAGE.
```

```
000960 PAGE-FOOTING-END.
000965     EXIT.
000970*
000975 TERMINATE-REPORT SECTION.
000980 FORCE-PAGE-FOOTING.
000985     SUBTRACT LINAGE-COUNTER FROM FOOTING-VALUE
000990              GIVING LINE-ADVANCE.
000995     MOVE SPACES TO REPORT-RECORD.
001000     WRITE REPORT-RECORD
001005           BEFORE ADVANCING LINE-ADVANCE
001010           AT EOP
001015               PERFORM REPORT-PAGE-FOOTING.
001020 REPORT-FOOTING-CODE.
001025     MOVE REPORT-FOOTING TO REPORT-RECORD-ENTRY.
001030     PERFORM SET-LINE-VALUE.
001035     WRITE REPORT-RECORD FROM REPORT-RECORD-FORMAT
001040           BEFORE ADVANCING 1.
001045 TERMINATE-REPORT-END.
001050     EXIT.
001055*
001060 CLOSE-FILES SECTION.
001065 CLOSE-BEGIN.
001070     CLOSE INPUT-FILE WITH LOCK
001075           MASTER-FILE
001080           REPORT-FILE WITH LOCK.
001085 CLOSE-END.
001090     EXIT.
001095*
001100 CREATE-BACKUP SECTION.
001105 BACKUP-BEGIN.
001110     OPEN INPUT MASTER-FILE
001115          OUTPUT BACKUP-FILE.
001120 COPY-MASTER-TO-BACKUP.
001125     READ MASTER-FILE
001130          AT END GO TO BACKUP-END.
001135     WRITE BACKUP-RECORD.
001140     GO TO COPY-MASTER-TO-BACKUP.
001145 BACKUP-END.
001150     EXIT.
001155*
001160 FINAL-CLOSE SECTION.
001165 FINAL-BEGIN.
001170     CLOSE MASTER-FILE WITH LOCK
001175           BACKUP-FILE WITH LOCK.
001180 FINAL-END.
001185     EXIT.
001190*
```

```
001195 SET-LINE-VALUE SECTION.
001200 LINE-VALUE-BEGIN.
001205     MOVE LINAGE-COUNTER OF REPORT-FILE TO LINAGE-COUNTER-VALUE.
001210 LINE-VALUE-END.
001215     EXIT.
001220*
```

4.5.2. Generated Report

The following is the report produced by the source program in 4.5.1.

```
REPORT  HEADING     THE DATE IS    01/12/22       THIS IS PRINTED ON LINE 1




THIS IS RECORD NUMBER 1                           THIS IS PRINTED ON LINE    6

THIS IS RECORD NUMBER 2                           THIS IS PRINTED ON LINE    8

THIS IS RECORD NUMBER 3                           THIS IS PRINTED ON LINE   10

PAGE NUMBER     1                                 THIS IS PRINTED ON LINE   12




THIS IS RECORD NUMBER 4                           THIS IS PRINTED ON LINE    1

THIS IS RECORD NUMBER 5                           THIS IS PRINTED ON LINE    3

THIS IS RECORD NUMBER 6                           THIS IS PRINTED ON LINE    5







PAGE NUMBER     2                                 THIS IS PRINTED ON LINE   12
END OF REPORT                                     THIS IS PRINTED ON LINE   13
```

4.6. Sequential File Assignment

The link between the ASCII COBOL defined file name and the OS 2200 File Control System name should be specified in the runstream (@ASG or @USE) prior the execution of any program containing sequential file processing.

4.6.1. Non-Symbiont Files

4.6.1.1. Assignment Examples

The following are specific examples of the OS 2200 Executive @ASG control statements for sequential files. Consult *OS 2200 Exec System Software Executive Control Language (ECL) End Use and Programming Reference Manual* (7830 7949) for general rules and formats.

```
@ASG,T  DF1.,F///200     (Disk)
@ASG,T  TAPE1.,U9V,1234  (Tape)
@ASG,T  CARTAPE.,HIC4O/////////BUFON,456
```

4.6.1.2. COBOL Program Links

The links between COBOL-defined files and the File Control System files renamed in 4.6.1.1 are provided by the following COBOL source language statements:

```
SELECT PAYROLL ASSIGN TO DISK DF1.

SELECT OLD-MASTER ASSIGN TO TAPE TAPE1.

SELECT CARTAPE ASSIGN TO TAPE.
```

The COBOL file PAYROLL would be linked with the File Control System name DF1 and have a maximum size of 200 tracks.

The COBOL file OLD–MASTER would be linked with labeled tape number 1234 with the File Control System name TAPE1.

The COBOL file CARTAPE would be linked with labeled tape number 456 which has the File Control System name CARTAPE. The file would be assigned to a 0899 cartridge tape device in the buffered write mode.

4.6.1.3. Device Independency for ANSI Tape Files

ANSI–74 tape files (in other words, SELECT file-name ASSIGN TO TAPE) can be assigned to a mass-storage device at execution time. In this case, an ANSI–74 SDF mass storage file will be created which is the same as if the file was assigned to DISC in the source program.

4.6.1.4. Device Independency for Sequential DISC Files

ANSI–74 mass–storage files (in other words, SELECT file-name ASSIGN TO DISC) can be assigned to a tape device at execution time. When the external device is tape, the file created will be an ANSI–74 SDF formatted tape file.

4.6.2. Symbiont Files

4.6.2.1. Assignment Rules

File assignments are handled automatically by the system for batch runs.

4.6.2.2. COBOL Program Links

The COBOL program links are:

```
SELECT f-n ASSIGN TO PRINTER.
SELECT f-n ASSIGN TO CARD-READER.
SELECT f-n ASSIGN TO CARD-PUNCH.
```

where *f–n* identifies the file for use within the COBOL program.

4.6.3. Alternate Symbiont Files

4.6.3.1. Assignment Rules

1. At the time the file is closed, a @BRKPT on that file is issued.
2. In order to explicitly control the printing or punching of an alternate file, that file must be assigned by the @ASG statement and output by the @SYM statement.
3. For output, if a file is not assigned by the @ASG statement, the system will attempt to dynamically assign it and release it at the termination of the run. If the file cannot be assigned because of an assignment conflict, the run is error terminated with an appropriate symbiont message. The automatic @ASG, @FREE, and @SYM are for batch runs only.

4.6.3.2. Assignment Examples

The following are specific examples of the @ASG control statements for sequential files. Consult the *OS 2200 Exec System Software Executive Control Language (ECL) End Use and Programming Reference Manual* (7830 7949) for general rules and formats.

```
@ASG,T  DF1.,F///1000

@ASG,A  DF2.
```

4.6.3.3. COBOL Program Links

The COBOL program links are:

```
SELECT f-n ASSIGN TO PRINTER DF1.
SELECT f-n ASSIGN TO CARD-READER DF2.
SELECT f-n ASSIGN TO CARD-PUNCH DF1.
```

where *f-n* identifies the file for use within the COBOL program, and DF1 or DF2 identifies the file name referenced on the @ASG statement.

Section 5
Relative I-O

5.1. General

5.1.1. Function

The Relative I–O module provides a capability to access records of a mass storage file in either a random or sequential manner. Each record in a relative file is uniquely identified by an integer value greater than zero that specifies the record's logical ordinal position in the file.

5.1.2. Level Characteristics

Relative I–O Level 1 does not provide full COBOL facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats of this module. Within the Procedure Division, the Relative I–O Level 1 provides limited capabilities for the READ and USE statements and full capabilities for the CLOSE, DELETE, OPEN, REWRITE, and WRITE statements, as specified in the formats of this module.

Relative I–O Level 2 provides full facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats of this module. Within the Procedure Division, the Relative I–O Level 2 provides full capabilities for the CLOSE, DELETE, OPEN, READ, REWRITE, START, USE, and WRITE statements as specified in the formats of this module. The additional features available in Level 2 include: the RESERVE clause, DYNAMIC accessing, SAME RECORD AREA, READ NEXT, and the entire START statement.

5.1.3. Language Concepts

5.1.3.1. Organization

Relative file organization is permitted only on mass storage devices. A relative file consists of records which are identified by relative record numbers. The file may be thought of as composed of a serial string of areas, each capable of holding a logical record. Each of these areas is denominated by a relative record number. Records are stored and retrieved based on this number. For example, the tenth record is the one addressed by relative record number 10 and is in the tenth record area, whether or not records have been written in the first through the ninth record areas.

5.1.3.2. Access Modes

In the sequential access mode, the sequence in which records are accessed is the ascending order of the relative record numbers of all records which currently exist within the file.

In the random access mode, the sequence in which records are accessed is controlled by the programmer. The desired record is accessed by placing its relative record number in a relative key data item.

In the dynamic access mode, the programmer may change at will from sequential access to random access using appropriate forms of input-output statements.

5.1.3.3. Current Record Pointer

The current record pointer is a conceptual entity used in this manual to facilitate the specification of the next record to be accessed within a given file. The concept of the current record pointer has no meaning for a file opened in the output mode. The setting of the current record pointer is affected only by the OPEN, START, and READ statements.

5.1.3.4. I–O Status

If the FILE STATUS clause is specified in a file control entry, a value is placed into the specified 2–character data item during the execution of an OPEN, CLOSE, READ, WRITE, REWRITE, DELETE, or START statement and before any applicable USE procedure is executed, to indicate to the COBOL program the status of that input-output operation.

5.1.3.4.1. Status Key 1

The leftmost character position of the FILE STATUS data item is known as status key 1 and is set to indicate one of the following conditions upon completion of the input-output operation.

0 SUCCESSFUL COMPLETION. The input-output statement was successfully executed.

1 AT END. The Format 1 READ statement was unsuccessfully executed as a result of an attempt to read a record when no next logical record exists in the file.

2 INVALID KEY. The input-output statement was unsuccessfully executed as a result of one of the following:

Duplicate Key

No Record Found

Boundary Violation

3 PERMANENT ERROR. The input-output statement was unsuccessfully executed as the result of an input-output error, such as data check, parity error, or transmission error.

9 Unisys Defined. The input-output statement was unsuccessfully executed as a result of a condition other than AT END, INVALID KEY, or PERMANENT ERROR condition. This value is used only to denote a condition not indicated by other defined values of status key 1, or by a specified combination of the values of status key 1 and status key 2.

5.1.3.4.2. Status Key 2

The rightmost character position of the FILE STATUS data item is known as status key 2 and is used to further describe the results of the input-output operation. This character will contain a value as follows:

1. If no further information is available concerning the input-output operation, then status key 2 contains a value of "0".
2. When status key 1 contains a value of "2" indicating an INVALID KEY condition, status key 2 is used to designate the cause of that condition as follows:
   a. A value of "2" in status key 2 indicates a duplicate key value. An attempt has been made to write a record that would create a duplicate key in a relative file.
   b. A value of "3" in status key 2 indicates no record found. An attempt has been made to access a record, identified by a key, and that record does not exist in the file.
   c. A value of "4" in status key 2 indicates a boundary violation. An attempt has been made to write beyond the externally-defined boundaries of a relative file, as defined on the @ASG statement for the file.
3. When status key 1 contains a value of "9" indicating a Unisys defined condition, the value of status key 2 is a binary number. These values are described in Appendix C.6.

5.1.3.4.3. Valid Combinations of Status Keys 1 and 2

The valid permissible combinations of the values of status key 1 and status key 2 are shown in the following figure. An "X" at an intersection indicates a permissible combination.

Status Key 1	Status Key 2			
	No Further Information (0)	Duplicate Key (2)	No Record Found (3)	Boundary Violation (4)
SUCCESSFUL COMPLETION (0)	X			
AT END (1)	X			
INVALID KEY (2)		X	X	X
PERMANENT ERROR (3)	X			
Unisys Defined (9)	See C.6			

5.1.3.5. The INVALID KEY Condition

The INVALID KEY condition can occur as a result of the execution of a START, READ, WRITE, REWRITE or DELETE statement. For details of the causes of the condition, see 5.4.6 for the START statement, 5.4.4 for the READ statement, 5.4.8 for the WRITE statement, 5.4.5 for the REWRITE statement, and 5.4.2 for the DELETE statement.

When the INVALID KEY condition is recognized, the MSCS takes these actions in the following order:

1. A value is placed into the FILE STATUS data item, if specified for this file, to indicate an INVALID KEY condition. (See 5.1.3.4.)
2. If the INVALID KEY phrase is specified in the statement causing the condition, control is transferred to the INVALID KEY imperative statement. Any USE procedure specified for this file is not executed.
3. If the INVALID KEY phrase is not specified, but a USE procedure is specified, either explicitly or implicitly, for this file, that procedure is executed.

When the INVALID KEY condition occurs, execution of the input-output statement that recognized the condition is unsuccessful and the file is not affected.

5.1.3.6. The AT END Condition

The AT END condition can occur as a result of the execution of a READ statement. For details of the causes of the condition, see 5.4.4.

5.2. Environment Division

5.2.1. INPUT–OUTPUT Section

5.2.1.1. The FILE–CONTROL Paragraph

5.2.1.1.1. Function

The FILE–CONTROL paragraph in the INPUT-OUTPUT section names each file and allows specification of other file-related information.

5.2.1.1.2. General Format

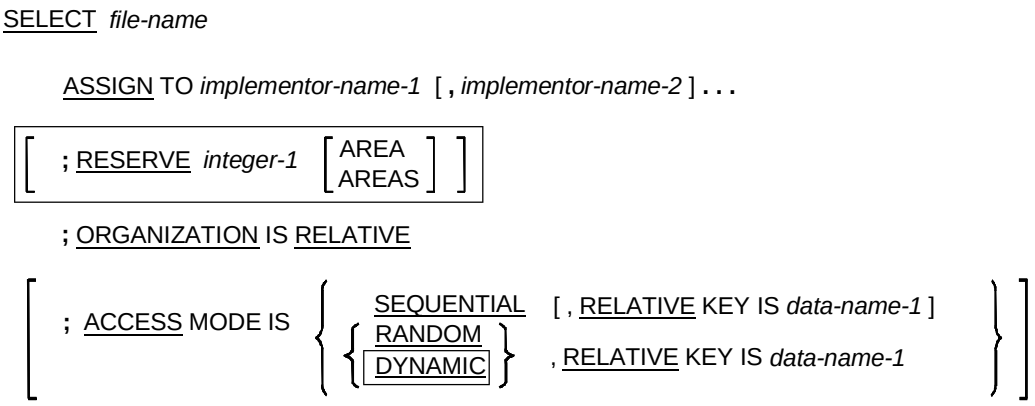
```
FILE–CONTROL. {file-control-entry} . . .
```

5.2.1.2. The File Control Entry

5.2.1.2.1. Function

The file control entry names a file and may specify other file-related information.

5.2.1.2.2. General Format



5.2.1.2.3. Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses that follow the SELECT clause may appear in any order.
2. Each file described in the Data Division must be named once and only once as *file-name* in the FILE–CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. A–SHARED–FILE or DISC must be specified for *implementor-name-1*.

4. If *implementor-name-2* is specified, it is a 1- to 12-character name used to link the ASCII COBOL file to OS 2200 Executive File Control and should appear in an @ASG or @USE statement in the run stream for executing the object program. (See 5.5.)

   When *implementor-name-2* is not specified, the first 12 characters of the *file-name* are used to link the ASCII COBOL file to the OS 2200 Executive File Control. (See 5.5.)

5. All recursions of *implementor-name-1, implementor-name-2* are for documentation only.

6. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.

7. *Data-name-2* must be defined in the Data Division as a 2-character data item of the category alphanumeric and must not be defined in the File Section, the Report Section, or the Communication Section. *Data-name-2* must be defined as USAGE DISPLAY.

8. *Data-name-1* and *data-name-2* may be qualified.

9. If a relative file is to be referenced by a START statement, the RELATIVE KEY phrase must be specified for that file.

10. *Data-name-1* must not be defined in a record description entry associated with that *file-name.*

11. The data item referenced by *data-name-1* must be defined as an unsigned integer.

5.2.1.2.4. General Rules

1. The ASSIGN clause specifies the association of the file referenced by *file-name* to a storage medium.

2. If the RESERVE clause is specified, it is for documentation only. One input-output area is always allocated.

3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.

4. When the access mode is sequential, records in the file are accessed in the sequence dictated by the file organization. This sequence is the order of ascending relative record numbers of existing records in the file.

5. When the FILE STATUS clause is specified, a value will be moved by the operating system into the data item specified by *data-name-2* after the execution of every statement that references that file either explicitly or implicitly. This value indicates the status of execution of the statement. (See 5.1.3.4, I–O Status.)

6. If the access mode is random, the value of the RELATIVE KEY data item indicates the record to be accessed.

7. When the access mode is dynamic, records in the file may be accessed sequentially and/or randomly. (See general rules 4 and 6.)

8. All records stored in a relative file are uniquely identified by relative record numbers. The relative record number of a given record specifies the record's logical ordinal position in the file. The first logical record has a relative record number of 1, and subsequent logical records have relative record numbers of 2, 3, 4, ...
9. The data item specified by *data-name-1* is used to communicate a relative record number between the user and the MSCS.
10. When DISC is specified as *implementor-name-1,* the runtime system interfaces with PCIOS. When A–SHARED–FILE is specified as *implementor-name-1,* the runtime system interfaces with the Shared File System (SFS).

5.2.1.3. The I–O–CONTROL Paragraph

5.2.1.3.1. Function

The I–O–CONTROL paragraph specifies the points at which rerun is to be established and the memory area which is to be shared by different files.

5.2.1.3.2. General Format

```
I-O-CONTROL .

[ ; RERUN ON { file-name-1       } EVERY { integer-1 RECORDS OF file-name-2 } ] ...
             { implementor-name  }       { integer-2 CLOCK-UNITS           }
                                         { condition-name                  }

[ ; SAME [ RECORD ] AREA FOR file-name-3 { , file-name-4 } ... ] ... .
```

5.2.1.3.3. Syntax Rules

1. The I–O–CONTROL paragraph is optional.
2. The allowable *implementor-names* are TAPE and DISC.
3. When the *integer-1* RECORDS clause is specified, *implementor-name* must be given in the RERUN clause.
4. When multiple *integer-1* RECORDS clauses are specified, no two of them may specify the same *file-name-2*.
5. The ON *file-name-1* clause, the *integer-2* CLOCK–UNITS clause and the *condition-name* clause serve only for documentation.
6. The two forms of the SAME clause (SAME AREA, SAME RECORD AREA) are considered separately in the following cases. More than one SAME clause may be included in a program.
   a. A file-name must not appear in more than one SAME AREA clause.
   b. A file-name must not appear in more than one SAME RECORD AREA clause.

c. If one or more file-names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file-names in that SAME AREA clause must appear in the SAME RECORD AREA clause. However, additional file-names not appearing in that SAME AREA clause may also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at any given time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at any given time.

7. The files referenced in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

8. The files referenced in the SAME AREA clause must not specify A–SHARED–FILE as *implementor-name-1* on the ASSIGN clause in the SELECT clause.

5.2.1.3.4. General Rules

1. The RERUN clause specifies where and when rerun information is to be recorded.

2. If *implementor-name* is specified, the rerun information is written as a separate file whose file-name is RERUN$.

3. In ASCII COBOL rerun points can be established when the *integer-1* RECORDS clause is used. In this case, the rerun information is written on the file whose file-name is RERUN$ which must be specified in the ON clause, whenever *integer-1* records of *file-name-2* have been processed. *File-name-2* may be either an input or output file with any organization or access.

4. The SAME AREA clause specifies that two or more files that do not represent sort or merge files are to use the same memory area during processing. The area being shared includes all storage areas assigned to the files specified; therefore, it is not valid to have more than one of the files open at the same time. (See syntax rule 6c, 5.2.1.3.3.)

5. The SAME RECORD AREA clause specifies that two or more files are to use the same memory area for processing of the current logical record. All of the files may be open at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each opened output file whose file-name appears in this SAME RECORD AREA clause and of the most recently read input file whose file-name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area, in other words, records are aligned on the leftmost character position.

5.3. Data Division

5.3.1. File Section

In a COBOL program the file description entry (FD) represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file-name and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of implementor-defined label items, and the names of the data records which comprise the file. The entry itself is terminated by a period.

5.3.2. Record Description Structure

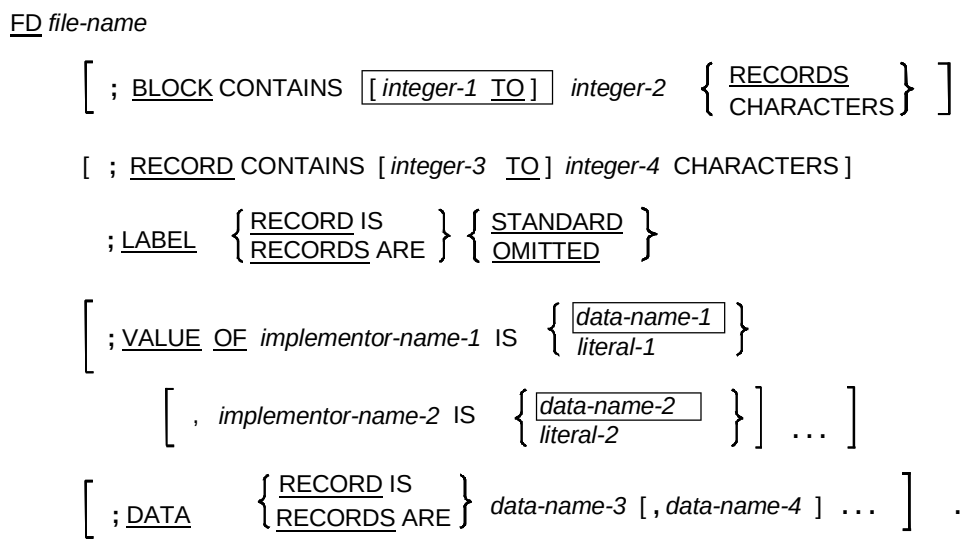
A record description consists of a set of data description entries that describe the characteristics of a particular record. Each data description entry consists of a level-number followed by a data-name if required, followed by a series of independent clauses as required. A record description has a hierarchical structure and therefore the clauses used with an entry may vary considerably, depending upon whether or not it is followed by subordinate entries. The structure of a record description is defined in 1.3.3.3.2, while the elements allowed in a record description are shown in 2.5.2.

5.3.3. The File Description—Complete Entry Skeleton

5.3.3.1. Function

The file description furnishes information concerning the physical structure, identification, and record names pertaining to a given file.

5.3.3.2. General Format



5.3.3.3. Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede the *file-name.*
2. The clauses that follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. One or more record description entries must follow the file description entry.

5.3.4. The BLOCK CONTAINS Clause

5.3.4.1. Function

The BLOCK CONTAINS clause specifies the size of a physical record.

5.3.4.2. General Format

```
BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS }
                                          { CHARACTERS }
```

5.3.4.3. Syntax Rules

1. This clause is for documentation. (See 17.2.2 for blocksize calculations.)

5.3.5. The DATA RECORDS Clause

5.3.5.1. Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

5.3.5.2. General Format

```
DATA { RECORD IS  } data-name-1 [ , data-name-2 ] ...
     { RECORDS ARE }
```

5.3.5.3. Syntax Rules

1. *Data-name-1* and *data-name-2* are the names of data records and must have 01 level-number record descriptions, with the same names, associated with them.

5.3.5.4. General Rules

1. The presence of more than one data-name indicates that the file contains more than one type of data record. These records may be of differing sizes, different formats, etc. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

5.3.6. The LABEL RECORDS Clause

5.3.6.1. Function

The LABEL RECORDS clause specifies whether labels are present.

5.3.6.2. General Format

```
LABEL  { RECORD IS  }  { STANDARD }
       { RECORDS ARE}  { OMITTED  }
```

5.3.6.3. Syntax Rules

1. This clause is required in every file description entry.

5.3.6.4. General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that labels exist for the file or the device to which the file is assigned. (See Appendix E.)

5.3.7. The RECORD CONTAINS Clause

5.3.7.1. Function

The RECORD CONTAINS clause specifies the size of data records.

5.3.7.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

5.3.7.3. General Rules

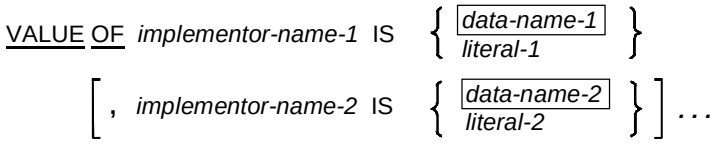
1. The size of each data record is completely defined within the record description entry, therefore this clause is never required. When present, however, the following notes apply:
   a. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case, *integer-2* represents the exact number of characters in the data record. If *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.
   b. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. (See 1.3.3.3.4, 2.5.11, and 2.5.12.)

5.3.8. The VALUE OF Clause

5.3.8.1. Function

The VALUE OF clause particularizes the description of an item in the label records associated with a file.

5.3.8.2. General Format



5.3.8.3. Syntax Rules

1. The allowable *implementor-name* is FILE–ID whose implied definition is PIC X(17).
2. *Implementor-name-2, data-name-2,* and *literal-2* serve only for documentation.
3. *Data-name-1, data-name-2, etc.,* should be qualified when necessary, but cannot be subscripted or indexed, nor can it be an item described with the USAGE IS INDEX clause.
4. *Data-name-1, data-name-2, etc.,* must be in the Working–Storage Section.
5. *Data-name-1* must be described as USAGE DISPLAY.

5.3.8.4. General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to the value of *literal-1,* or of *data-name-1,* whichever has been specified. (See 5.4.3 and Appendix E.)

   For an output file, at the appropriate time the value of *implementor-name-1* is made equal to the value of *literal-1,* or of a *data-name-1,* whichever has been specified. (See 5.4.3 and Appendix E.)
2. A figurative constant may be substituted in the preceding format wherever a literal is specified.

5.4. Procedure Division

5.4.1. The CLOSE Statement

5.4.1.1. Function

The CLOSE statement terminates the processing of files, with optional lock.

5.4.1.2. General Format

```
CLOSE  file-name-1  [ WITH LOCK ] [ , file-name-2  [ WITH LOCK ] ]   . . .
```

5.4.1.3. Syntax Rules

1. The files referenced in the CLOSE statement need not all have the same organization or access.

5.4.1.4. General Rules

1. A CLOSE statement may only be executed for a file in an open mode.
2. Relative files are classified as belonging to the category of nonsequential single/multireel/unit. The results of executing each type of CLOSE for this category of file are summarized in the following table:

CLOSE Statement FORMAT	File Category = Nonsequential Single/Multireel/Unit
CLOSE	A
CLOSE WITH LOCK	A,B

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input-output files.

A. Close File

Input Files and Input-Output Files (Sequential Access Mode):

If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing. (See Appendix E.)

Input Files and Input–Output Files (Random or Dynamic Access Mode); Output Files (Random, Dynamic, or Sequential Access Mode):

If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed. (See Appendix E.)

B. File Lock

This file cannot be opened again during this execution of this run unit.

3. The action taken if the file is in the open mode when a STOP RUN statement is executed or the action taken for a file that has been opened in a called program and not closed in that program prior to the execution of a CANCEL statement for that program are undefined. No implicit CLOSE procedures are initiated. This could result in the condition which would cause the file to be unusable on subsequent accesses to it.
4. If a CLOSE statement has been executed for a file, no other statement can be executed that references that file, either explicitly or implicitly, unless an intervening OPEN statement for that file is executed.
5. Following the successful execution of a CLOSE statement, the record area associated with *file-name* is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.

5.4.2. The DELETE Statement

5.4.2.1. Function

The DELETE statement logically removes a record from a mass storage file.

5.4.2.2. General Format

```
DELETE file-name RECORD [ ; INVALID KEY imperative-statement ]
```

5.4.2.3. Syntax Rules

1. The INVALID KEY phrase must not be specified for a DELETE statement that references a file which is in sequential access mode.
2. The INVALID KEY phrase must be specified for a DELETE statement which references a file which is not in sequential access mode and for which an applicable USE procedure is not specified.

5.4.2.4. General Rules

1. The associated file must be open in the I–O mode at the time of the execution of this statement. (See 5.4.3.)
2. For files in the sequential access mode, the last input-output statement executed for *file-name* prior to the execution of the DELETE statement must have been a successfully executed READ statement. The MSCS logically removes from the file the record that was accessed by that READ statement.

3. For a file in random or dynamic access mode, the MSCS logically removes from the file that record identified by the contents of the RELATIVE KEY data item associated with *file-name.* If the file does not contain the record specified by the key, an INVALID KEY condition exists. (See 5.1.3.8.)
4. After the successful execution of a DELETE statement, the identified record has been logically removed from the file and can no longer be accessed.
5. The execution of a DELETE statement does not affect the contents of the record area associated with *file-name.*
6. The current record pointer is not affected by the execution of a DELETE statement.
7. The execution of the DELETE statement causes the value of the specified FILE STATUS data item, if any, associated with *file-name* to be updated. (See 5.1.3.4.)

5.4.3. The OPEN Statement

5.4.3.1. Function

The OPEN statement initiates the processing of files. It also performs checking and/or writing of labels and other input-output operations.

5.4.3.2. General Format

```
OPEN { INPUT  file-name-1  [ , file-name-2 ] ...          } ...
     { OUTPUT file-name-3  [ , file-name-4 ] ...          }
     { I-O    file-name-5  [ , file-name-6 ] ...          }
```

5.4.3.3. Syntax Rules

1. The files referenced in the OPEN statement need not all have the same organization or access.

5.4.3.4. General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode.
2. The successful execution of the OPEN statement makes the associated record area available to the program.
3. Prior to the successful execution of an OPEN statement for a given file, no statement can be executed that references that file, either explicitly or implicitly.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 5-1, an "X" at an intersection indicates that the specified statement, used in the access mode given for that row, may be used with the relative file organization and the open mode given at the top of the column.

Table 5-1. Permissible Statements

File Access Mode	Statement	Open Mode		
		Input	Output	Input–Output
Sequential	READ	X		X
	WRITE		X	
	REWRITE			X
	START	X		X
	DELETE			X
Random	READ	X		X
	WRITE		X	X
	REWRITE			X
	START			
	DELETE			X
Dynamic	READ	X		X
	WRITE		X	X
	REWRITE			X
	START	X		X
	DELETE			X

5. A file may be opened with the INPUT, OUTPUT, and I–O phrases in the same program. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file (see Appendix E), the beginning labels are processed as follows:

   a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.

   b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. The file description entry for *file-name-1, file-name-2, file-name-5,* or *file-name-6* must be equivalent to that used when this file was created.
9. For files being opened with the INPUT or I–O phrase, the OPEN statement sets the current record pointer to the first record currently existing within the file. If no records exist in the file, the current record pointer is set such that the next executed Format 1 READ statement for the file will result in an AT END condition.
10. The I–O phrase permits the opening of a file for both input and output operations. Since this phrase implies the existence of the file, it cannot be used if the file is being initially created.

11. When the I–O phrase is specified and the LABEL RECORDS clause indicates label records are present (see Appendix E), the execution of the OPEN statement includes the following steps:
    a. The labels are checked in accordance with the conventions for input-output label checking.
    b. The new labels are written in accordance with the conventions for input-output label writing.
12. Upon successful execution of an OPEN statement with the OUTPUT phrase specified, a file is created. At that time the associated file contains no data records.

5.4.4. The READ Statement

5.4.4.1. Function

For sequential access, the READ statement makes available the next logical record from a file. For random access, the READ statement makes available a specified record from a mass storage file.

5.4.4.2. General Format

Format 1

READ *file-name* [NEXT] RECORD [INTO *identifier*] [**;** AT END *imperative-statement*]

Format 2

READ *file-name* RECORD [INTO *identifier*] [**;** INVALID KEY *imperative-statement*]

5.4.4.3. Syntax Rules

1. The INTO phrase must not be used when the input file contains logical records of various sizes as indicated by their record descriptions. The storage area associated with *identifier* and the record area associated with *file-name* must not be the same storage area.
2. Format 1 must be used for all files in sequential access mode.
3. The NEXT phrase must be specified for files in dynamic access mode, when records are to be retrieved sequentially.
4. Format 2 is used for files in random access mode or for files in dynamic access mode when records are to be retrieved randomly.
5. The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE procedure is specified for *file-name.*

5.4.4.4. General Rules

1. The associated files must be open in the INPUT or I–O mode at the time this statement is executed. (See 5.4.3.)
2. The record to be made available by a Format 1 READ statement is determined as follows:
   a. The record, pointed to by the current record pointer, is made available provided that the current record pointer was positioned by the START or OPEN statement and the record is still accessible through the path indicated by the current record pointer. If the record is no longer accessible, which may have been caused by the deletion of the record, the current record pointer is updated to point to the next existing record in the file and that record is then made available.
   b. If the current record pointer was positioned by the execution of a previous READ statement, the current record pointer is updated to point to the next existing record in the file and then that record is made available.
3. The execution of the READ statement causes the value of the FILE STATUS data item, if any, associated with *file-name* to be updated. (See 5.1.3.4.)
4. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged in that a record is available to the object program prior to the execution of any statement following the READ statement.
5. When the logical records of a file are described with more than one record description, these records automatically share the same storage area; this is equivalent to an implicit redefinition of the area. The contents of any data items which lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
6. If the INTO phrase is specified, the record being read is moved from the record area to the area specified by *identifier* according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if the execution of the READ statement was unsuccessful. Any subscripting or indexing associated with *identifier* is evaluated after the record has been read and immediately before it is moved to the data item.
7. When the INTO phrase is used, the record being read is available in both the input record area and the data area associated with *identifier*.
8. If, at the time of execution of a Format 1 READ statement, the position of current record pointer for that file is undefined, the execution of that READ statement is unsuccessful.
9. If, at the time of the execution of a Format 1 READ statement, no next logical record exists in the file, the AT END condition occurs, and the execution of the READ statement is considered unsuccessful. (See 5.1.3.4.)

10. When the AT END condition is recognized the following actions are taken in the specified order:

    a. A value is placed into the FILE STATUS data item, if specified for this file, to indicate an AT END condition. (See 5.1.3.4.)

    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to the AT END *imperative-statement*. Any USE procedure specified for this file is not executed.

    c. If the AT END phrase is not specified, then a USE procedure must be specified, either explicitly or implicitly, for this file, and that procedure is executed.

    When the AT END condition occurs, execution of the input-output statement which caused the condition is unsuccessful.

11. Following the unsuccessful execution of any READ statement, the contents of the associated record area and the position of the current record pointer are undefined.

12. When the AT END condition has been recognized, a Format 1 READ statement for that file must not be executed without first executing one of the following:

    a. A successful CLOSE statement followed by the execution of a successful OPEN statement for that file.

    b. A successful START statement for that file.

    c. A successful Format 2 READ statement for that file.

13. For a file for which dynamic access mode is specified, a Format 1 READ statement with the NEXT phrase specified causes the next logical record to be retrieved from the file as described in general rule 2.

14. If the RELATIVE KEY phrase is specified, the execution of a Format 1 READ statement updates the contents of the RELATIVE KEY data item such that it contains the relative record number of the record made available.

15. The execution of a Format 2 READ statement sets the current record pointer to, and makes available, the record whose relative record number is contained in the data item named in the RELATIVE KEY phrase for the file. If the file does not contain such a record, the INVALID KEY condition exists and execution of the READ statement is unsuccessful. (See 5.1.3.8.)

5.4.5. The REWRITE Statement

5.4.5.1. Function

The REWRITE statement logically replaces a record existing in a mass storage file.

5.4.5.2. General Format

```
REWRITE record-name [ FROM identifier ] [ ; INVALID KEY imperative-statement ]
```

5.4.5.3. Syntax Rules

1. *Record-name* and *identifier* must not refer to the same storage area.
2. *Record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.
3. The INVALID KEY phrase must not be specified for a REWRITE statement that references a file in sequential access mode.
4. The INVALID KEY phrase must be specified in the REWRITE statement for files in the random or dynamic access mode for which an appropriate USE procedure is not specified.

5.4.5.4. General Rules

1. The file associated with *record-name* must be open in the I–O mode at the time of execution of this statement. (See 5.4.3.)
2. For files in the sequential access mode, the last input-output statement executed for the associated file prior to the execution of the REWRITE statement must have been a successfully executed READ statement. The MSCS logically replaces the record that was accessed by the READ statement.
3. The number of character positions in the record referenced by *record-name* must be equal to the number of character positions in the record being replaced.
4. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause, in which case the logical record is available to the program as a record of other files appearing in the same SAME RECORD AREA clause as the associated I–O file, as well as to the file associated with *record-name*.
5. The execution of a REWRITE statement with the FROM phrase is equivalent to the execution of:

   ```
   MOVE identifier TO record-name
   ```

   followed by the execution of the same REWRITE statement without the FROM phrase. The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of the REWRITE statement.

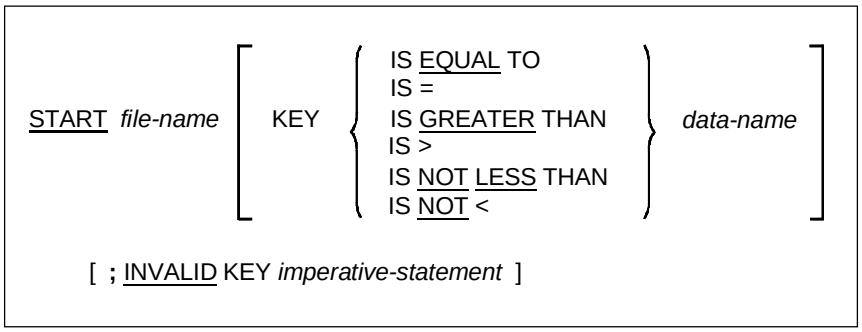
6. The current record pointer is not affected by the execution of a REWRITE statement.
7. The execution of the REWRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 5.1.3.4.)
8. For a file accessed in either random or dynamic access mode, the MSCS logically replaces the record specified by the contents of the RELATIVE KEY data item associated with the file. If the file does not contain the record specified by the key, the INVALID KEY condition exists (See 5.1.3.8). The updating operation does not take place and the data in the record area is unaffected.

5.4.6. The START Statement

5.4.6.1. Function

The START statement provides a basis for logical positioning within a relative file, for subsequent sequential retrieval of records.

5.4.6.2. General Format



Note: *The required relational characters >, < , and = are not underlined to avoid confusion with other symbols such as ≥ (greater than or equal to).*

5.4.6.3. Syntax Rules

1. *File-name* must be the name of a file with sequential or dynamic access.
2. *Data-name* may be qualified.
3. The INVALID KEY phrase must be specified if no applicable USE procedure is specified for *file-name.*
4. *Data-name,* if specified, must be the data item specified in the RELATIVE KEY phrase of the associated file control entry.

5.4.6.4. General Rules

1. *File-name* must be open in the INPUT or I–O mode at the time that the START statement is executed. (See 5.4.3.)
2. If the KEY phrase is not specified the relational operator "IS EQUAL TO" is implied.
3. The type of comparison specified by the relational operator in the KEY phrase occurs between a key associated with a record in the file referenced by *file-name* and a data item as specified in general rule 5.

a. The current record pointer is positioned to the first logical record currently existing in the file whose key satisfies the comparison.

b. If the comparison is not satisfied by any record in the file, an INVALID KEY condition exists, the execution of the START statement is unsuccessful, and the position of the current record pointer is undefined. (See 5.1.3.8.)

4. The execution of the START statement causes the value of the FILE STATUS data item, if any, associated with *file-name* to be updated. (See 5.1.3.4.)
5. The comparison described in general rule 3 uses the data item referenced by the RELATIVE KEY clause associated with *file-name.*

5.4.7. The USE Statement

5.4.7.1. Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

5.4.7.2. General Format

```
USE AFTER STANDARD { EXCEPTION } PROCEDURE ON { file-name-1 [ , file-name-2 ] ... }
                   { ERROR     }              { INPUT                            }
                                              { OUTPUT                           }
                                              { I-O                              }
```

5.4.7.3. Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives section and must be followed by a period followed by a space. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.
3. The same *file-name* can appear in a different specific arrangement of the format. Appearance of a *file-name* in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and may be used interchangeably.
5. The files implicitly or explicitly referenced in a USE statement need not all have the same organization or access.

5.4.7.4. General Rules

1. The designated procedures are executed by the input-output system after completing the standard input-output error routine, or upon recognition of the INVALID KEY or AT END conditions, when the INVALID KEY phrase or AT END phrase, respectively, has not been specified in the input-output statement.
2. After execution of a USE procedure, control is returned to the invoking routine if it was invoked by AT END or INVALID KEY conditions or a Unisys defined condition. (See C.6.)

3. Within a USE procedure, there must not be any reference to any nondeclarative procedures. Conversely, in the nondeclarative portion there must be no reference to procedure-names that appear in the declarative portion, except that PERFORM statements may refer to a USE statement or to the procedures associated with such a USE statement.
4. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

5.4.8. The WRITE Statement

5.4.8.1. Function

The WRITE statement releases a logical record for an output or input–output file.

5.4.8.2. General Format

```
WRITE record-name [ FROM identifier ] [ ; INVALID KEY imperative-statement ]
```

5.4.8.3. Syntax Rules

1. *Record-name* and *identifier* must not reference the same storage area.
2. The *record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.
3. The INVALID KEY phrase must be specified if an applicable USE procedure is not specified for the associated file.

5.4.8.4. General Rules

1. The associated file must be open in the OUTPUT or I–O mode at the time of the execution of this statement. (See 5.4.3.)
2. The logical record released by the execution of the WRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause or the execution of the WRITE statement is unsuccessful due to an INVALID KEY condition. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as to the file associated with *record-name*.
3. The results of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of:

   a. The statement:

      ```
      MOVE identifier TO record-name
      ```

      according to the rules specified for the MOVE statement, followed by:

   b. The same WRITE statement without the FROM phrase.

   The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of this WRITE statement.

After execution of the WRITE statement is complete, the information in the area referenced by *identifier* is available, even though the information in the area referenced by *record-name* may not be. (See general rule 2.)

4. The current record pointer is unaffected by the execution of a WRITE statement.
5. The execution of the WRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 5.1.3.4.)
6. The maximum record size for a file is established at the time the file is created and must not subsequently be changed.
7. The number of character positions on a mass storage device required to store a logical record in a file may or may not be equal to the number of character positions defined by the logical description of that record in the program.
8. The execution of the WRITE statement releases a logical record to the operating system.
9. When a file is opened in the output mode, records may be placed into the file by one of the following:
    a. If the access mode is sequential, the WRITE statement will cause a record to be released to the MSCS. The first record will have a relative record number of one and subsequent records released will have relative record numbers of 2, 3, 4, ... If the RELATIVE KEY data item has been specified in the file control entry for the associated file, the relative record number of the record just released will be placed into the RELATIVE KEY data item by the MSCS during execution of the WRITE statement.
    b. If the access mode is random or dynamic, prior to the execution of the WRITE statement the value of the RELATIVE KEY data item must be initialized in the program with the relative record number to be associated with the record in the record area. That record is then released to the MSCS by execution of the WRITE statement.
10. When a file is opened in the I–O mode and the access mode is random or dynamic, records are to be inserted in the associated file. The value of the RELATIVE KEY data item must be initialized by the program with the relative record number to be associated with the record in the record area. Execution of a WRITE statement then causes the contents of the record area to be released to the MSCS.
11. The INVALID KEY condition exists under either of the following circumstances:
    a. When the access mode is random or dynamic, and the RELATIVE KEY data item specifies a record which already exists in the file, or
    b. When an attempt is made to write beyond the externally defined boundaries of the file.
12. When the INVALID KEY condition is recognized, the execution of the WRITE statement is unsuccessful, the contents of the record area are unaffected, and the FILE STATUS data item, if any, of the associated file is set to a value indicating the cause of the condition. Execution of the program proceeds according to the rules for the INVALID KEY condition (See 5.1.3.4 and 5.1.3.8.)

5.5. Relative I–O File Assignment

The link between the ASCII COBOL defined file name and the OS 2200 File Control System name should be specified in the runstream (@ASG or @USE) prior to the execution of any program containing relative file processing.

5.5.1. Assignment Examples

The following are specific examples of the @ASG control statements for relative files. Consult the *(ECL) Operations and Programming Reference Manual,* for general rules and formats.

```
@ASG,T RF1.,F///1000

@ASG,A RF2.
```

5.5.2. COBOL Program Links

The links between COBOL-defined files and the File Control System files named in 5.5.1 are provided in the following COBOL source language statements:

```
SELECT NEW-MASTER ASSIGN TO DISC RF1.

SELECT RELATIVE-FILE ASSIGN TO DISC RF2.
```

The COBOL file NEW–MASTER would be linked with the File Control System name RF1 and have a maximum size of 1000 tracks.

The COBOL file RELATIVE–FILE would be linked with the File Control System name RF2.

Section 6
Indexed I-O

6.1. General

6.1.1. Function

The Indexed I–O module provides a capability to access records of a mass storage file in either a random or sequential manner. Each record in an indexed file is uniquely identified by the value of one or more keys within that record.

6.1.2. Level Characteristics

Indexed I–O Level 1 does not provide full COBOL facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats of this module. Within the Procedure Division, the Indexed I–O Level 1 provides limited capabilities for the READ and USE statements and full capabilities for the CLOSE, DELETE, OPEN, REWRITE, and WRITE statements, as specified in the formats for this module.

Indexed I–O Level 2 provides full facilities for the FILE–CONTROL, I–O–CONTROL, and FD entries as specified in the formats for this module. Within the Procedure Division, the Indexed I–O Level 2 provides full capabilities for the CLOSE, DELETE, OPEN, READ, REWRITE, START, USE, and WRITE statements as specified in the formats for this module. The additional features available in Level 2 include: the RESERVE clause, DYNAMIC accessing, ALTERNATE KEYS, SAME RECORD AREA, READ NEXT, and the entire START statement.

6.1.3. Language Concepts

6.1.3.1. Organization

A file whose organization is indexed is a mass storage file in which data records may be accessed by the value of a key. A record description may include one or more key data items, each of which is associated with an index. Each index provides a logical path to the data records according to the contents of a data item within each record that is the record key for that index.

The data item named in the RECORD KEY clause of the file control entry for a file is the prime record key for that file. For purposes of inserting, updating, and deleting records in a file, each record is identified solely by the value of its prime record key. This value must, therefore, be unique and must not be changed when updating the record.

A data item named in the ALTERNATE RECORD KEY clause of the file control entry for a file is an alternate record key for that file. The value of an alternate record key may be non-unique if the DUPLICATES phrase is specified for it. These keys provide alternate access paths for retrieval of records from the file.

6.1.3.2. Access Modes

In the sequential access mode, records are accessed in the ascending order of the record key values. The order of retrieval of records within a set of records having duplicate record key values is the order in which the records were written into the set.

In the random access mode, the sequence in which records are accessed is controlled by the programmer. The desired record is accessed by placing the value of its record key in a record key data item.

In the dynamic access mode, the programmer may change at will from sequential access to random access using appropriate forms of input-output statements.

6.1.3.3. Current Record Pointer

The current record pointer is a conceptual entity used in this manual to facilitate specification of the next record to be accessed within a given file. The concept of the current record pointer has no meaning for a file opened in the output mode. The setting of the current record pointer is affected only by the OPEN, START, and READ statements.

6.1.3.4. I–O Status

If the FILE STATUS clause is specified in a file control entry, a value is placed into the specified 2-character data item during the execution of an OPEN, CLOSE, READ, WRITE, REWRITE, DELETE, or START statement and before any applicable USE procedure is executed, to indicate to the COBOL program the status of that input-output operation.

6.1.3.4.1. Status Key 1

The leftmost character position of the FILE STATUS data item is known as status key 1 and is set to indicate one of the following conditions upon completion of the input-output operation.

0 SUCCESSFUL COMPLETION. The input-output statement was successfully executed.

1 AT END. The Format 1 READ statement was unsuccessfully executed as a result of an attempt to read a record when no next logical record exists in the file.

2 INVALID KEY. The input-output statement was unsuccessfully executed as a result of one of the following:

Sequence Error

Duplicate Key

No Record Found

Boundary Violation

3 PERMANENT ERROR. The input-output statement was unsuccessful as the result of an input-output error, such as data check, parity error, or transmission error.

9 Unisys Defined. The input-output statement was unsuccessfully executed as a result of a condition other than an AT END, INVALID KEY, or PERMANENT ERROR condition. This value is used only to indicate a condition not indicated by other defined values of status key 1, or by specified combinations of the value of status key 1 and status key 2.

6.1.3.4.2. Status Key 2

The rightmost character position of the FILE STATUS data item is known as status key 2 and is used to further describe the results of the input-output operation. This character will contain a value as follows:

1. If no further information is available concerning the input-output operation, then status key 2 contains a value of "0".
2. When status key 1 contains a value of"0" indicating a successful completion, status key 2 may contain a value of "2" indicating a duplicate key. This condition indicates:
   a. For a READ statement, the key value for the current key of reference is equal to the value of that same key in the next record within the current key of reference.
   b. For a WRITE or REWRITE statement, the record just written created a duplicate key value for at least one alternate record key for which duplicates are allowed.
3. When status key 1 contains a value of "2" indicating an INVALID KEY condition, status key 2 is used to designate the cause of that condition as follows:
   a. A value of "1" in status key 2 indicates a sequence error for a sequentially accessed indexed file. The ascending sequence requirements of successive record key values have been violated (see 6.4.8), or the prime record key value has been changed by the COBOL program between the successful execution of a READ statement and the execution of the next REWRITE statement for that file.
   b. A value of "2" in status key 2 indicates a duplicate key value. An attempt has been made to write or rewrite a record that would create a duplicate key in an indexed file.
   c. A value of "3" in status key 2 indicates no record found. An attempt has been made to access a record, identified by a key, and that record does not exist in the file.
   d. A value of "4" in status key 2 indicates a boundary violation. An attempt has been made to write beyond the externally defined boundaries of an indexed file as defined on the @ASG statement for the file.
4. When status key 1 contains a value of "9" indicating a Unisys defined condition, the status key 2 values are binary numbers and are described in C.6.

6.1.3.4.3. Valid Combinations of Status Keys 1 and 2

The valid permissible combinations of the values of status key 1 and status key 2 are shown in the following table. An "X" at an intersection indicates a valid combination.

Status Key 1	Status Key 2				
	No Further Information (0)	**Sequence Error (1)**	**Duplicate Key (2)**	**No Record Found (3)**	**Boundary Violation (4)**
SUCCESSFUL COMPLETION (0)	X		X		
AT END (1)	X				
INVALID KEY (2)		X	X	X	X
PERMANENT ERROR (3)	X				
Unisys Defined (9)	See C.6				

6.1.3.5. The INVALID KEY Condition

The INVALID KEY condition can occur as a result of the execution of a START, READ, WRITE, REWRITE or DELETE statement. For details of the causes of the condition, see 6.4.6 for the START statement, 6.4.4 for the READ statement, 6.4.8 for the WRITE statement, 6.4.5 for the REWRITE statement, and 6.4.2 for the DELETE statement.

When the INVALID KEY condition is recognized, the MSCS takes these actions in the following order:

1. A value is placed into the FILE STATUS data item, if specified for this file, to indicate an INVALID KEY condition. (See 6.1.3.4.)
2. If the INVALID KEY phrase is specified in the statement causing the condition, control is transferred to the INVALID KEY imperative statement. Any USE procedure specified for this file is not executed.
3. If the INVALID KEY phrase is not specified, but a USE procedure is specified, either explicitly or implicitly, for this file, that procedure is executed.

When the INVALID KEY condition occurs, execution of the input-output statement which recognized the condition is unsuccessful and the file is not affected.

6.1.3.6. The AT END Condition

The AT END condition can occur as a result of the execution of a READ statement. For details of the causes of the condition, see 6.4.4.

6.1.4. Information Statistics

6.1.4.1. Function

An information statistics table, maintained for each indexed file, contains data on the current status and use of the file. A callable function, C$INFO, provides the user with the ability to access this statistical information.

6.1.4.2. Calling Format

```
ENTER MASM "C$INFO" USING file-name data-name
```

6.1.4.3. Syntax Rules

1. C$INFO is an OS 2200 defined entry point.
2. The parameter *file-name* must be an indexed file assigned to DISC.
3. The parameter *data-name* must be defined as a level 01 entry in the File Section or the Working-Storage Section.

6.1.4.4. General Rules

1. *File-name* must be in the open mode at the time the statement is executed.
2. If the parameter list is in error, the contents of *data-name* will remain unchanged and control is returned to the calling program.
3. If the file represented by *file-name* is not an indexed file assigned to DISC or the file is not in open mode, the contents of *data-name* are cleared to zero and control is returned to the calling program.
4. The successful execution of this function causes the data-item referenced by *data-name* to be initialized to the current contents of the indexed information statistics table for the file referenced by *file-name*.

6.1.4.5. Data–Item Format

The format of the data-item referenced by *data-name* must be defined with the following COBOL record description:

```
01  DATA-NAME.
    10 DATA-NAME-0   PIC 9(10) COMP.
    10 DATA-NAME-1   PIC 9(10) COMP.
    10 DATA-NAME-2   PIC 9(10) COMP.
    10 DATA-NAME-3   PIC 9(10) COMP.
    10 DATA-NAME-4   PIC 9(10) COMP.
    10 DATA-NAME-5   PIC 9(10) COMP.
    10 DATA-NAME-6   PIC 9(10) COMP.
    10 DATA-NAME-7   PIC 9(10) COMP.
    10 DATA-NAME-8   PIC 9(10) COMP.
    10 DATA-NAME-9   PIC 9(10) COMP.
    10 DATA-NAME-10  PIC 9(10) COMP.
    10 DATA-NAME-11  PIC 9(10) COMP.
    10 DATA-NAME-12  PIC 9(10) COMP.
    10 DATA-NAME-13  PIC 9(10) COMP.
    10 DATA-NAME-14  PIC 9(10) COMP.
    10 DATA-NAME-15  PIC 9(10) COMP.
    10 DATA-NAME-16  PIC 9(10) COMP.
    10 DATA-NAME-17  PIC 9(10) COMP.
    10 DATA-NAME-18  PIC 9(10) COMP.
    10 DATA-NAME-19  PIC 9(10) COMP.
    10 DATA-NAME-20  PIC 9(10) COMP.
    10 DATA-NAME-21  PIC 9(10) COMP.
    10 DATA-NAME-22  PIC 9(10) COMP.
```

Table 6-1 defines the contents of each DATA–NAME–n specified in the above record description. The contents of the DATA–NAME–1 through DATA–NAME–11 reflects the files condition prior to the execution of the immediately preceding OPEN statement. The contents of DATA–NAME–12 through DATA–NAME–22 reflect the files condition since the execution of the immediately preceding OPEN statement.

Table 6-1. Contents of DATA–NAME–*n*

DATA–NAME	Contents
DATA–NAME–0	The total number of records written when the file was initially created (between execution of an OPEN OUTPUT statement and execution of the first CLOSE statement for the file).
DATA–NAME–1	The total number of records in the file.
DATA–NAME–2	The total number of records written to the file.
DATA–NAME–3	The total number of records read from the file.
DATA–NAME–4	The total number of records deleted in the file.
DATA–NAME–5	The total number of records modified in the file.
DATA–NAME–6	The total number of start requests.
DATA–NAME–7	The total number of keys modified in the file.
DATA–NAME–8	The total number of invalid key exits.
DATA–NAME–9	The total number of data blocks in the file.
DATA–NAME–10	The total number of index blocks in the file.
DATA–NAME–11	The total number of times the file has been closed.
DATA–NAME–12	The total number of records written to the file.
DATA–NAME–13	The total number of records read from the file.
DATA–NAME–14	The total number of records deleted in the file.
DATA–NAME–15	The total number of records modified in the file.
DATA–NAME–16	The total number of start requests.
DATA–NAME–17	The total number of keys modified in the file.
DATA–NAME–18	The total number of invalid key exits.
DATA–NAME–19	The total number of new data blocks created in the file.
DATA–NAME–20	The total number of new index blocks created in the file.
DATA–NAME–21	The total number of blocks read.
DATA–NAME–22	The total number of blocks written.

6.2. Environment Division

6.2.1. INPUT–OUTPUT Section

6.2.1.1. The FILE–CONTROL Paragraph

6.2.1.1.1. Function

The FILE–CONTROL paragraph in the INPUT-OUTPUT section names each file and allows specification of other file-related information.

6.2.1.1.2. General Format

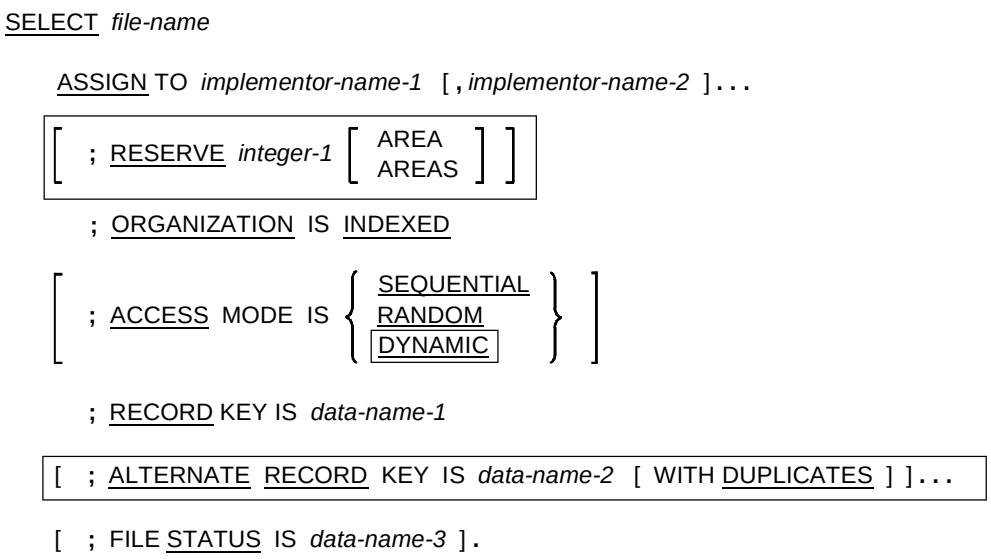
```
FILE-CONTROL. { file-control-entry } . . .
```

6.2.1.2. The File Control Entry

6.2.1.2.1. Function

The file control entry names a file and may specify other file-related information.

6.2.1.2.2. General Format



6.2.1.2.3. Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses which follow the SELECT clause may appear in any order.
2. Each file described in the Data Division must be named once and only once as *file-name* in the FILE–CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. A–SHARED–FILE or DISC must be specified for *implementor-name-1*.

4. If *implementor-name-2* is specified it is a 1 to 12-character name used to link the ASCII COBOL file to OS 2200 Executive File Control. It should appear on an @ASG or @USE statement in the run stream for executing the object program (see 6.5).

   When *implementor-name-2* is not specified, the first 12 characters of the file-name are used to link the ASCII COBOL file to OS 2200 Executive File Control (see 6.5).

5. All recursions of *implementor–name–1*, *implementor–name–2* are for documentation only.

6. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.

7. *Data-name-3* must be defined in the Data Division as a 2-character data item of the category alphanumeric and must not be defined in the File Section, the Report Section, or the Communication Section. *Data-name-3* must be defined as USAGE DISPLAY.

8. *Data-name-1*, *data-name-2,* and *data-name-3* may be qualified.

9. The data items referenced by *data-name-1* and *data-name-2* must each be defined as a data item of the category alphanumeric within a record description entry associated with that *file-name.*

10. Neither *data-name-1* nor *data-name-2* can describe an item whose size is variable (see 3.2.1, The OCCURS Clause).

11. *Data-name-2* cannot reference an item whose leftmost character position corresponds to the leftmost character position of an item referenced by *data-name-1* or by any other *data-name-2* associated with this file.

6.2.1.2.4. General Rules

1. The ASSIGN clause specifies the association of the file referenced by *file-name* to a storage medium.

2. If the RESERVE clause is not specified, the number of input-output areas allocated is equal to the number of keys (prime record key and alternate record keys) specified for the file, plus four. If the RESERVE clause is specified, the number of input-output areas is increased by the value of (*integer-1* - 1). If 0 is specified, the clause has no effect.

3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.

4. When the access mode is sequential, records in the file are accessed in the sequence dictated by the file organization. For indexed files this sequence is the order of ascending record key values within a given key of reference.

5. When the FILE STATUS clause is specified, a value will be moved by the operating system into the data item specified by *data-name-3* after the execution of every statement that references that file either explicitly or implicitly. This value indicates the status of execution of the statement. (See 6.1.3.4.)

6. If the access mode is random, the value of the record key data item indicates the record to be accessed.

7. When the access mode is dynamic, records in the file may be accessed sequentially or randomly. (See general rules 4 and 6.)

8. The RECORD KEY clause specifies the record key that is the prime record key for the file. The values of the prime record key must be unique among records of the file. This prime record key provides an access path to records in an indexed file.

9. An ALTERNATE RECORD KEY clause specifies a record key that is an alternate record key for the file. This alternate record key provides an alternate access path to records in an indexed file.

10. The data descriptions of *data-name-1* and *data-name-2* as well as their relative locations within a record must be the same as that used when the file was created. The number of alternate keys for the file must also be the same as that used when the file was created.

11. The DUPLICATES phrase specifies that the value of the associated alternate record key may be duplicated within any of the records in the file. If the DUPLICATES phrase is not specified, the value of the associated alternate record key must not be duplicated among any of the records in the file.

12. When DISC is specified as *implementor-name-1*, the runtime system interfaces with PCIOS. When A–SHARED–FILE is specified as *implementor-name-1*, the runtime system interfaces with the Shared File System (SFS).

6.2.1.3. The I–O–CONTROL Paragraph

6.2.1.3.1. Function

The I–O–CONTROL paragraph specifies the points at which rerun is to be established and the memory area which is to be shared by different files.

6.2.1.3.2. General Format

```
I-O-CONTROL .

[ ; RERUN ON { file-name-1      } EVERY { integer-1 RECORDS OF file-name-2 } ] ...
             { implementor-name }       { integer-2 CLOCK-UNITS           }
                                        { condition-name                  }

[ ; SAME [ RECORD ] AREA FOR file-name-3 { , file-name-4 } ... ] ... .
```

6.2.1.3.3. Syntax Rules

1. The I–O–CONTROL paragraph is optional.

2. The allowable *implementor-names* are TAPE and DISC.

3. When the *integer-1* RECORDS clause is specified, *implementor-name* must be given in the RERUN clause.

4. When multiple *integer-1* RECORDS clauses are specified, no two of them may specify the same *file-name–2*.

5. The ON *file-name-1* clause, the *integer-2* CLOCK–UNITS clause and *condition-name* clause serve only for documentation.

6. The two forms of the SAME clause (SAME AREA, SAME RECORD AREA) are considered separately in the following:

   More than one SAME clause may be included in a program, however:

   a. A file-name must not appear in more than one SAME AREA clause.

   b. A file-name must not appear in more than one SAME RECORD AREA clause.

   c. If one or more file-names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file-names in that SAME AREA clause must appear in the SAME RECORD AREA clause. However, additional file-names not appearing in that SAME AREA clause may also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at any given time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at any given time.

7. The files referenced in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

8. The files referenced in the SAME AREA clause must not specify A–SHARED–FILE as *implementor-name-1* on the ASSIGN clause in the SELECT clause.

6.2.1.3.4. General Rules

1. The RERUN clause specifies where and when the rerun information is to be recorded.

2. If *implementor–name* is specified, the rerun information is written as a separate file whose file-name is RERUN$.

3. In ASCII COBOL rerun points can be established when the *integer-1* RECORDS clause is used. In this case, the rerun information is written on the file whose file-name is RERUN$, which must be specified in the ON clause, whenever *integer-1* records of *file-name-2* have been processed. *File-name-2* may be either an input or output file with any organization or access.

4. The SAME AREA clause specifies that two or more files that do not represent sort or merge files are to use the same memory area during processing. The area being shared includes all storage areas assigned to the files specified; therefore, it is not valid to have more than one of the files open at the same time. (See syntax rule 6c, 6.2.1.3.3.)

5. The SAME RECORD AREA clause specifies that two or more files are to use the same storage area for processing of the current logical record. All of the files may be open at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each opened output file whose file-name appears in this SAME RECORD AREA clause and of the most recently read input file whose file-name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; in other words, records are aligned on the leftmost character position.

6.3. Data Division

6.3.1. File Section

In a COBOL program the File-Description entry (FD) represents the highest level of organization in the File Section. The File Section header is followed by a File-Description entry consisting of a level indicator (FD), a file-name and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of implementor-defined label items, and the names of the data records which comprise the file. The entry itself is terminated by a period.

6.3.2. Record Description Structure

A record description consists of a set of data description entries which describe the characteristics of a particular record. Each data description entry consists of a level-number followed by a data-name if required, followed by a series of independent clauses as required. A record description has a hierarchical structure and therefore the clauses used with an entry may vary considerably, depending upon whether or not it is followed by subordinate entries. The structure of a record description is defined in 1.3.3.3.2, while the elements allowed in a record description are shown in 2.5.2.

6.3.3. The File Description—Complete Entry Skeleton

6.3.3.1. Function

The file description furnishes information concerning the physical structure, identification, and record names pertaining to a given file.

6.3.3.2. General Format

```
FD file-name

[ ; BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS / CHARACTERS } ]

[ ; RECORD CONTAINS [ integer-3 TO ] integer-4 CHARACTERS ]

  ; LABEL { RECORD IS / RECORDS ARE } { STANDARD / OMITTED }

[ ; VALUE OF implementor-name-1 IS { data-name-1 / literal-1 }
      [ , implementor-name-2 IS { data-name-2 / literal-2 } ] ... ]

[ ; DATA { RECORD IS / RECORDS ARE } data-name-3 [ , data-name-4 ] ... ] .
```

6.3.3.3. Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede *file-name*.
2. The clauses which follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. One or more record description entries must follow the file description entry.

6.3.4. The BLOCK CONTAINS Clause

6.3.4.1. Function

The BLOCK CONTAINS clause specifies the size of a physical record.

6.3.4.2. General Format

```
BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS }
                                          { CHARACTERS }
```

6.3.4.3. General Rules

1. The size of the physical record may be stated in terms of RECORDS, unless one of the following situations exists, in which case the RECORDS phrase must not be used.
   a. Where logical records may extend across physical records.
   b. The physical record contains padding (area not contained in a logical record).
   c. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.
2. When the word CHARACTERS is specified, the physical record size is specified in terms of the number of ASCII character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
3. If only *integer-2* is shown, it represents the exact size of the physical record. If *integer-1* and *integer-2* are both shown, they refer to the minimum and maximum size of the physical record, respectively. (See 17.2.2.)

6.3.5. The DATA RECORDS Clause

6.3.5.1. Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

6.3.5.2. General Format

```
DATA { RECORD IS  } data-name-1 [ , data-name-2 ] ...
     { RECORDS ARE }
```

6.3.5.3. Syntax Rules

Data-name-1 and *data-name-2* are the names of data records and must have 01 level-number record descriptions, with the same names, associated with them.

6.3.5.4. General Rules

1. The presence of more than one data-name indicates that the file contains more than one type of data record. These records may be of differing sizes, different formats, etc. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

6.3.6. The LABEL RECORDS Clause

6.3.6.1. Function

The LABEL RECORDS clause specifies whether labels are present.

6.3.6.2. General Format

```
LABEL { RECORD IS  } { STANDARD }
      { RECORDS ARE} { OMITTED  }
```

6.3.6.3. Syntax Rules

1. This clause is required in every file description entry.

6.3.6.4. General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that labels exist for the file or the device to which the file is assigned. (See Appendix E.)

6.3.7. The RECORD CONTAINS Clause

6.3.7.1. Function

The RECORD CONTAINS clause specifies the size of data records.

6.3.7.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

6.3.7.3. General Rules

1. The size of each data record is completely defined within the record description entry; therefore this clause is never required. When present, however, the following apply:

   a. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case, *integer-2* represents the exact number of characters in the data record. If *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.

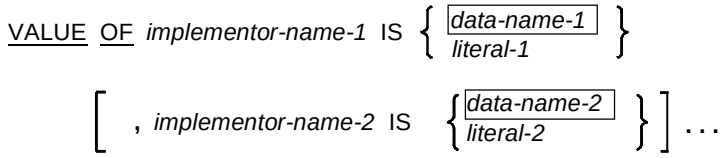
   b. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. (See 1.3.3.3.4, 2.5.11, and 2.5.12.)

6.3.8. The VALUE OF Clause

6.3.8.1. Function

The VALUE OF clause particularizes the description of an item in the label records associated with a file.

6.3.8.2. General Format



6.3.8.3. Syntax Rules

1. The allowable *implementor-name* is FILE–ID whose implied definition is PIC X(17).
2. *Implementor-name-2*, d*ata-name-2*, and *literal-2* serve only for documentation.
3. *Data-name-1, data-name-2, etc.,* should be qualified when necessary, but cannot be subscripted or indexed nor can they be items described with the USAGE IS INDEX clause.
4. *Data-name-1, data-name-2, etc.,* must be in the Working–Storage Section.
5. *Data-name-1* must be described as USAGE DISPLAY.

6.3.8.4. General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to the value of *literal-1*, or of *data-name-1*, whichever has been specified. (See 6.4.3 and Appendix E.)

   For an output file, at the appropriate time the value of *implementor-name-1* is made equal to the value of *literal-1*, or of a *data-name-1*, whichever has been specified. (See 6.4.3 and Appendix E.)

2. A figurative constant may be substituted in the preceding format wherever a literal is specified.

6.4. Procedure Division

6.4.1. The CLOSE Statement

6.4.1.1. Function

The CLOSE statement terminates the processing of files with optional lock.

6.4.1.2. General Format

```
CLOSE file-name-1 [ WITH LOCK ] [ , file-name-2 [ WITH LOCK ] ]  . . .
```

6.4.1.3. Syntax Rules

1. The files referenced in the CLOSE statement need not all have the same organization or access.

6.4.1.4. General Rules

1. A CLOSE statement may be executed only for a file in an open mode.
2. Indexed files are classified as belonging to the category of nonsequential single/multi-reel/unit. The results of executing each type of CLOSE for this category of file are summarized in the following table:

CLOSE Statement Format	File Category = Nonsequential Single/Multi-reel/Unit
CLOSE	A
CLOSE WITH LOCK	A,B

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input, output, or input-output file, alternate definitions are given; otherwise, a definition applies to input, output, and input-output files.

A Close File

Input Files and Input-Output Files (Sequential Access Mode):

If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing. (See Appendix E.)

Input Files and Input-Output Files (Random or Dynamic Access Mode); Output Files (Random, Dynamic, or Sequential Access Mode):

If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed. (See Appendix E.)

B File Lock

This file cannot be opened again during this execution of this run unit.

3. The action taken if the file is in the open mode when a STOP RUN statement is executed, or the action taken for a file that has been opened in a called program and not closed in that program prior to the execution of a CANCEL statement for that program are undefined. No implicit CLOSE procedures are initiated. This could result in rendering the file unusable on subsequent accesses to it.
4. If a CLOSE statement has been executed for a file, no other statement can be executed that references that file, either explicitly or implicitly, unless an intervening OPEN statement for that file is executed.
5. Following the successful execution of a CLOSE statement, the record area associated with file-name is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.

6.4.2. The DELETE Statement

6.4.2.1. Function

The DELETE statement logically removes a record from a mass storage file.

6.4.2.2. General Format

```
DELETE file-name RECORD [ ; INVALID KEY imperative-statement ]
```

6.4.2.3. Syntax Rules

1. The INVALID KEY phrase must not be specified for a DELETE statement that references a file which is in sequential access mode.
2. The INVALID KEY phrase must be specified for a DELETE statement that references a file that is not in sequential access mode and for which an applicable USE procedure is not specified.

6.4.2.4. General Rules

1. The associated file must be open in the I–O mode at the time of the execution of this statement. (See 6.4.3.)
2. For files in the sequential access mode, the last input-output statement executed for *file-name* prior to the execution of the DELETE statement must have been a successfully executed READ statement. The MSCS logically removes from the file the record that was accessed by that READ statement.
3. For a file in random or dynamic access mode, the MSCS logically removes from the file the record identified by the contents of the prime record key data item associated with *file-name*. If the file does not contain the record specified by the key, an INVALID KEY condition exists. (See 6.1.3.8.)
4. After the successful execution of a DELETE statement, the identified record has been logically removed from the file and can no longer be accessed.
5. The execution of a DELETE statement does not affect the contents of the record area associated with *file-name*.
6. The current record pointer is not affected by the execution of a DELETE statement.
7. The execution of the DELETE statement causes the value of the specified FILE STATUS data item, if any, associated with *file-name* to be updated. (See 6.1.3.4.)

6.4.3. The OPEN Statement

6.4.3.1. Function

The OPEN statement initiates the processing of files. It also performs checking and writing of labels and other input-output operations.

6.4.3.2. General Format

```
OPEN { INPUT  file-name-1  [ , file-name-2 ] ...
     { OUTPUT file-name-3  [ , file-name-4 ] ... } ...
     { I-O    file-name-5  [ , file-name-6 ] ...
```

6.4.3.3. Syntax Rules

1. The files referenced in the OPEN statement need not all have the same organization or access.

6.4.3.4. General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode.
2. The successful execution of the OPEN statement makes the associated record area available to the program.
3. Prior to the successful execution of an OPEN statement for a given file, no statement can be executed that references that file, either explicitly or implicitly.

4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 6-2, an "X" at an intersection indicates that the specified statement, used in the access mode given for that row, may be used with the indexed file organization and the open mode given at the top of the column.

Table 6-2. Permissible Statements

		Open Mode		
File Access Mode	**Statement**	**Input**	**Output**	**Input–Output**
Sequential	READ	X		X
	WRITE		X	
	REWRITE			X
	START	X		X
	DELETE			X
Random	READ	X		X
	WRITE		X	X
	REWRITE			X
	START			
	DELETE			X
Dynamic	READ	X		X
	WRITE		X	X
	REWRITE			X
	START	X		X
	DELETE			X

5. A file may be opened with the INPUT, OUTPUT, and I–O phrases in the same program. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file (see Appendix E), the beginning labels are processed as follows:

   a. When the INPUT phrase is specified, the execution of the OPEN statement checks the labels in accordance with the conventions for input label checking.

   b. When the OUTPUT phrase is specified, the execution of the OPEN statement writes the labels in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. The file description entry for *file-name-1*, *file-name-2*, *file-name-5*, or *file-name-6* must be equivalent to that used when the file was created.
9. For files being opened with the INPUT or I–O phrase, the OPEN statement sets the current record pointer to the first record currently existing within the file. For indexed files, the prime record key is established as the key of reference and is used to determine the first record to be accessed. If no records exist in the file, the current record pointer is set such that the next executed Format 1 READ statement for the file will result in an AT END condition.

10. The I–O phrase permits the opening of a file for both input and output operations. Since this phrase implies the existence of the file, it cannot be used if the file is being initially created.
11. When the I–O phrase is specified and the LABEL RECORDS clause indicates label records are present (see Appendix E), the execution of the OPEN statement includes the following steps:
    a. The labels are checked in accordance with the conventions for input-output label checking.
    b. The new labels are written in accordance with the conventions for input-output label writing.
12. Upon successful execution of an OPEN statement with the OUTPUT phrase specified, a file is created. At that time the associated file contains no data records.

6.4.4. The READ Statement

6.4.4.1. Function

For sequential access, the READ statement makes available the next logical record from a file. For random access, the READ statement makes available a specified record from a mass storage file.

6.4.4.2. General Format

Format 1

READ *file-name* [NEXT] RECORD [INTO *identifier*]
[; AT END *imperative-statement*]

Format 2

READ *file-name* RECORD [INTO *identifier* ; KEY IS *data-name*]
[; INVALID KEY *imperative-statement*]

6.4.4.3. Syntax Rules

1. The INTO phrase must not be used when the input file contains logical records of various sizes as indicated by their record descriptions. The storage area associated with *identifier* and the storage area which is the record area associated with *file-name* must not be the same storage area.
2. *Data-name* must be the name of a data item specified as a record key associated with *file-name*.
3. *Data-name* may be qualified.
4. Format 1 must be used for all files in sequential access mode.
5. The NEXT phrase must be specified for files in dynamic access mode, when records are to be retrieved sequentially.
6. Format 2 is used for files in random access mode or for files in dynamic access mode when records are to be retrieved randomly.

7. The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE procedure is specified for *file-name*.

6.4.4.4. General Rules

1. The associated file must be open in the INPUT or I–O mode at the time this statement is executed. (See 6.4.3.)
2. The record to be made available by a Format 1 READ statement is determined as follows:
   a. The record, pointed to by the current record pointer, is made available provided that the current record pointer was positioned by the START or OPEN statement and the record is still accessible through the path indicated by the current record pointer. If the record is no longer accessible, which may have been caused by the deletion of the record or a change in an alternate record key, the current record pointer is updated to point to the next existing record within the established key of reference and that record is then made available.
   b. If the current record pointer was positioned by the execution of a previous READ statement, the current record pointer is updated to point to the next existing record in the file with the established key of reference and then that record is made available.
3. The execution of the READ statement causes the value of the FILE STATUS data item, if any, associated with *file-name* to be updated. (See 6.1.3.4.)
4. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged in that a record is available to the object program prior to the execution of any statement following the READ statement.
5. When the logical records of a file are described with more than one record description, these records automatically share the same storage area. This is equivalent to an implicit redefinition of the area. The contents of any data items which lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
6. If the INTO phrase is specified, the record being read is moved from the record area to the area specified by *identifier* according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if the execution of the READ statement was unsuccessful. Any subscripting or indexing associated with *identifier* is evaluated after the record has been read and immediately before it is moved to the data item.
7. When the INTO phrase is used, the record being read is available in both the input record area and the data area associated with *identifier*.
8. If, at the time of execution of a Format 1 READ statement, the position of current record pointer for that file is undefined, the execution of that READ statement is unsuccessful.
9. If, at the time of the execution of a Format 1 READ statement, no next logical record exists in the file, the AT END condition occurs, and the execution of the READ statement is considered unsuccessful. (See 6.1.3.4.)

10. When the AT END condition is recognized the following actions are taken in the specified order:

    a. A value is placed into the FILE STATUS data item, if specified for this file, to indicate an AT END condition. (See 6.1.3.4.)

    b. If the AT END phrase is specified in the statement causing the condition, control is transferred to the AT END imperative statement. Any USE procedure specified for this file is not executed.

    c. If the AT END phrase is not specified, then a USE procedure must be specified, either explicitly or implicitly, for this file, and that procedure is executed.

    When the AT END condition occurs, execution of the input-output statement that caused the condition is unsuccessful.

11. Following the unsuccessful execution of any READ statement, the contents of the associated record area and the position of the current record pointer are undefined. For indexed files the key of reference is also undefined.

12. When the AT END condition has been recognized, a Format 1 READ statement for that file must not be executed without first executing one of the following:

    a. A successful CLOSE statement followed by the execution of a successful OPEN statement for that file.

    b. A successful START statement for that file.

    c. A successful Format 2 READ statement for that file.

13. For a file for which dynamic access mode is specified, a Format 1 READ statement with the NEXT phrase specified causes the next logical record to be retrieved from that file as described in general rule 2.

14. For an indexed file being sequentially accessed, records having the same value in an alternate record key which is the key of reference are made available in the same order in which they are released by execution of WRITE statements, or by execution of REWRITE statements which create such duplicate values.

15. For an indexed file if the KEY phrase is specified in a Format 2 READ statement, *data-name* is established as the key of reference for this retrieval. If the dynamic access mode is specified, this key of reference is also used for retrievals by any subsequent executions of Format 1 READ statements for the file until a different key of reference is established for the file.

16. If the KEY phrase is not specified in a Format 2 READ statement, the prime record key is established as the key of reference for this retrieval. If the dynamic access mode is specified, this key of reference is also used for retrievals by any subsequent executions of Format 1 READ statements for the file until a different key of reference is established for the file.

17. Execution of a Format 2 READ statement causes the value of the key of reference to be compared with the value contained in the corresponding data item of the stored records in the file, until the first record having an equal value is found. The current record pointer is positioned to this record which is then made available. If no record can be so identified, the INVALID KEY condition exists and execution of the READ statement is unsuccessful. (See 6.1.3.8.)

6.4.5. The REWRITE Statement

6.4.5.1. Function

The REWRITE statement logically replaces a record existing in a mass storage file.

6.4.5.2. General Format

```
REWRITE record-name [ FROM  identifier ] [ ; INVALID KEY imperative-statement ]
```

6.4.5.3. Syntax Rules

1. *Record-name* and *identifier* must not refer to the same storage area.
2. *Record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.
3. The INVALID KEY phrase must be specified in the REWRITE statement for files for which an appropriate USE procedure is not specified.

6.4.5.4. General Rules

1. The file associated with *record-name* must be open in the I–O mode at the time of execution of this statement. (See 6.4.3.)
2. For files in the sequential access mode, the last input–output statement executed for the associated file prior to the execution of the REWRITE statement must have been a successfully executed READ statement. The MSCS logically replaces the record that was accessed by the READ statement.
3. The number of character positions in the record referenced by *record-name* must be equal to the number of character positions in the record being replaced.
4. The logical record released by a successful execution of the REWRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause, in which case the logical record is available to the program as a record of other files appearing in the same SAME RECORD AREA clause as the associated I–O file, as well as to the file associated with *record–name*.
5. The execution of a REWRITE statement with the FROM phrase is equivalent to the execution of:

   ```
   MOVE identifier TO record-name
   ```

   followed by the execution of the same REWRITE statement without the FROM phrase. The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of the REWRITE statement.
6. The current record pointer is not affected by the execution of a REWRITE statement.
7. The execution of the REWRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 6.1.3.4.)

8. For a file in the sequential access mode, the record to be replaced is specified by the value contained in the prime record key. When the REWRITE statement is executed the value contained in the prime record key data item of the record to be replaced must be equal to the value of the prime record key of the last record read from this file.
9. For a file in the random or dynamic access mode, the record to be replaced is specified by the prime record key data item.
10. The contents of alternate record key data items of the record being rewritten may differ from those in the record being replaced. The MSCS utilizes the content of the record key data items during the execution of the REWRITE statement in such a way that subsequent access of the record may be made based upon any of those specified record keys.
11. The INVALID KEY condition exists when:
    a. The access mode is sequential and the value contained in the prime record key data item of the record to be replaced is not equal to the value of the prime record key of the last record read from this file, or
    b. The value contained in the prime record key data item does not equal that of any record stored in the file, or
    c. The value contained in an alternate record key data item for which a DUPLICATES clause has not been specified is equal to that of a record already stored in the file.

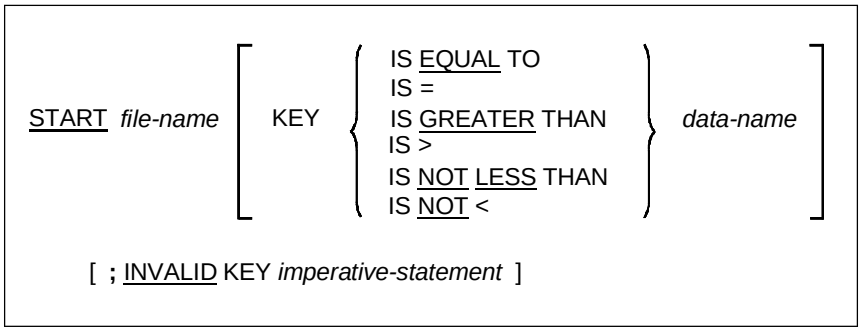
    The updating operation does not take place and the data in the record area is unaffected. (See 6.1.3.8.)

6.4.6. The START Statement

6.4.6.1. Function

The START statement provides a basis for logical positioning within an indexed file, for subsequent sequential retrieval of records.

6.4.6.2. General Format



Note: *The required relational characters " >", " < ", and " = " are not underlined to avoid confusion with other symbols such as " ≥ " (greater than or equal to).*

6.4.6.3. Syntax Rules

1. *File-name* must be the name of an indexed file.
2. *File-name* must be the name of a file with sequential or dynamic access.
3. *Data-name* may be qualified.
4. The INVALID KEY phrase must be specified if no applicable USE procedure is specified for *file-name*.
5. If *file-name* is the name of an indexed file, and if the KEY phrase is specified, *data-name* may reference a data item specified as a record key associated with *file-name*, or it may reference any data item of category alphanumeric subordinate to the *data-name* of a data item specified as a record key associated with *file-name* whose leftmost character position corresponds to the leftmost character position of that record key data item.

6.4.6.4. General Rules

1. *File-name* must be open in the INPUT or I–O mode at the time that the START statement is executed. (See 6.4.3.)
2. If the KEY phrase is not specified the relational operator "IS EQUAL TO" is implied.
3. The type of comparison specified by the relational operator in the KEY phrase occurs between a key associated with a record in the file referenced by *file-name* and a data item as specified in general rule 5. If *file-name* references an indexed file and the operands are of unequal size, comparison proceeds as though the longer one were truncated on the right such that its length is equal to that of the shorter. All other nonnumeric comparison rules apply except that the presence of the PROGRAM COLLATING SEQUENCE clause will have no effect on the comparison. (See 2.6.2.1.1.2.)

   a. The current record pointer is positioned to the first logical record currently existing in the file whose key satisfies the comparison. For records having the same value in an alternate record key which is the key of reference, the first logical record is determined in the same order that these records were released to the file by execution of either WRITE or REWRITE statements which created such duplicate values.

   b. If the comparison is not satisfied by any record in the file, an INVALID KEY condition exists, the execution of the START statement is unsuccessful, and the position of the current record pointer is undefined. (See 6.1.3.8.)

4. The execution of the START statement causes the value of the FILE STATUS data item, if any, associated with *file-name* to be updated. (See 6.1.3.4.)
5. If the KEY phrase is specified, the comparison described in general rule 3 uses the data item referenced by *data-name*.
6. If the KEY phrase is not specified, the comparison described in general rule 3 uses the data item referenced in the RECORD KEY clause associated with *file-name*.

7. Upon completion of the successful execution of the START statement, a key of reference is established and used in subsequent Format 1 READ statements as follows: (See 6.4.4.)

   a. If the KEY phrase is not specified, the prime record key specified for *file-name* becomes the key of reference.

   b. If the KEY phrase is specified, and *data-name* is specified as a record key for *file-name*, that record key becomes the key of reference.

   c. If the KEY phrase is specified, and *data-name* is not specified as a record key for *file-name*, the record key whose leftmost character position corresponds to the leftmost character position of the data item specified by *data-name*, becomes the key of reference.

8. If the execution of the START statement is not successful, the key of reference is undefined.

6.4.7. The USE Statement

6.4.7.1. Function

The USE statement specifies procedures for input-output error handling that are in addition to the standard procedures provided by the input-output control system.

6.4.7.2. General Format



6.4.7.3. Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declarative section and must be followed by a period followed by a space. The remainder of the section must consist of zero, one, or more procedural paragraphs that define the procedures to be used.
2. The USE statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.
3. The same file-name can appear in a different specific arrangement of the format. Appearance of a file-name in a USE statement must not cause the simultaneous request for execution of more than one USE procedure.
4. The words ERROR and EXCEPTION are synonymous and may be used interchangeably.
5. The files implicitly or explicitly referenced in a USE statement need not all have the same organization or access.

6.4.7.4. General Rules

1. The designated procedures are executed by the input-output system after completing the standard input-output error routine, or upon recognition of the INVALID KEY or AT END conditions, when the INVALID KEY phrase or AT END phrase, respectively, has not been specified in the input-output statement.
2. After execution of a USE procedure, control is returned to the invoking routine if it was invoked by the AT END or INVALID KEY condition or a Unisys defined status condition. (See C.6.)
3. Within a USE procedure, there must not be any reference to any nondeclarative procedures. Conversely, in the nondeclarative portion there must be no reference to procedure-names that appear in the declarative portion, except that PERFORM statements may refer to a USE statement or to the procedures associated with such a USE statement.
4. Within a USE procedure, there must not be the execution of any statement that would cause the execution of a USE procedure that had previously been invoked and had not yet returned control to the invoking routine.

6.4.8. The WRITE Statement

6.4.8.1. Function

The WRITE statement releases a logical record for an output or input-output file.

6.4.8.2. General Format

```
WRITE record-name [ FROM identifier ] [ ; INVALID KEY imperative-statement ]
```

6.4.8.3. Syntax Rules

1. *Record-name* and *identifier* must not reference the same storage area.
2. The *record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.
3. The INVALID KEY phrase must be specified if an applicable USE procedure is not specified for the associated file.

6.4.8.4. General Rules

1. The associated file must be open in the OUTPUT or I–O mode at the time of the execution of this statement. (See 6.4.3.)
2. The logical record released by the execution of the WRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause or the execution of the WRITE statement is unsuccessful due to an INVALID KEY condition. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as to the file associated with *record-name*.

3. The results of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of:

   a. The statement:

      ```
      MOVE identifier TO record-name
      ```

      according to the rules specified for the MOVE statement, followed by:

   b. The same WRITE statement without the FROM phrase.

   The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of this WRITE statement.

   After execution of the WRITE statement is complete, the information in the area referenced by *identifier* is available, even though the information in the area referenced by *record-name* may not be. (See general rule 2.)

4. The current record pointer is unaffected by the execution of a WRITE statement.
5. The execution of the WRITE statement causes the value of the FILE STATUS data item, if any, associated with the file to be updated. (See 6.1.3.4.)
6. The maximum record size for a file is established at the time the file is created and must not subsequently be changed.
7. The number of character positions on a mass storage device required to store a logical record in a file may or may not be equal to the number of character positions defined by the logical description of that record in the program.
8. The execution of the WRITE statement releases a logical record to the operating system.
9. Execution of the WRITE statement causes the contents of the record area to be released. The MSCS utilizes the content of the record keys in such a way that subsequent access of the record key may be made based upon any of those specified record keys.
10. The value of the prime record key must be unique within the records in the file.
11. The data item specified as the prime record key must be set by the program to the desired value prior to the execution of the WRITE statement. (See general rule 3.)
12. If sequential access mode is specified for the file, records must be released to the MSCS in ascending order of prime record key values.
13. If random or dynamic access mode is specified, records may be released to the MSCS in any program-specified order.
14. When the ALTERNATE RECORD KEY clause is specified in the file control entry for an indexed file, the value of the alternate record key may be nonunique only if the DUPLICATES phrase is specified for that data item. In this case the MSCS provides storage of records such that when records are accessed sequentially, the order of retrieval of those records is the order in which they are released to the MSCS.

15. The INVALID KEY condition exists under the following circumstances:

    a. When sequential access mode is specified for a file opened in the output mode, and the value of the prime record key is not greater than the value of the prime record key of the previous record, or

    b. When the file is opened in the output or I–O mode, and the value of the prime record key is equal to the value of a prime record key of a record already existing in the file, or

    c. When the file is opened in the output or I–O mode, and the value of an alternate record key for which duplicates are not allowed equals the corresponding data item of a record already existing in the file, or

    d. When an attempt is made to write beyond the externally defined boundaries of the file.

16. When the INVALID KEY condition is recognized, the execution of the WRITE statement is unsuccessful, the contents of the record area are unaffected and the FILE STATUS data item, if any, associated with *file-name* of the associated file is set to a value indicating the cause of the condition. Execution of the program proceeds according to the rules stated in 6.1.3.8. (See also 6.1.3.4.)

6.5. Indexed I–O File Assignment

The link between the ASCII COBOL defined file name and the OS 2200 File Control System name should be specified in the runstream (@ASG or @USE) prior to the execution of any program containing indexed file processing.

6.5.1. Assignment Examples

The following are specified examples of the OS 2200 Executive @ASG control statements for indexed files. Consult *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899), for general rules and formats:

```
@ASG,T INDEX1.,F///2000

@ASG,A PAYROLL.,F
```

6.5.2. COBOL Program Links

The links between COBOL defined files and the File Control System files named in 6.5.1 are provided by the following COBOL source language statements:

```
SELECT INDEX-FILE ASSIGN TO DISC INDEX1.
SELECT PAY-ROLL ASSIGN TO DISC PAYROLL.
```

The COBOL file INDEX–FILE would be linked with the File Control System name INDEX1 and have a maximum size of 2000 tracks.

The COBOL file PAY–ROLL would be linked with the File Control System name PAYROLL.

Section 7
Sort-Merge

7.1. General

7.1.1. Function

The Sort-Merge module provides the capability to order one or more files of records, or to combine two or more identically ordered files of records, according to a set of user–specified keys contained within each record. Optionally, a user may apply some special processing to each of the individual records by input or output procedures. This special processing may be applied before or after the records are ordered by the SORT, or after the records have been combined by the MERGE.

7.1.2. Level Characteristics

Sort-Merge Level 1 provides the facility for sorting a single file only once within a given execution of a COBOL program. Procedures for special handling of each record in the file before and after it has been sorted are also provided.

Sort-Merge Level 2 provides the facility for sorting one or more files, or combining two or more files, one or more times within a given execution of a COBOL program.

7.1.3. Relationship with Sequential I–O Module

The files specified in the USING and GIVING phrases of the SORT and MERGE statements must be described implicitly or explicitly in the FILE CONTROL paragraph as having sequential organization. No input-output statement may be executed for the file named in the sort-merge file description.

7.2. Environment Division

The COBOL Sort feature requires the use of the following facilities in the Environment Division of a COBOL source program.

7.2.1. INPUT–OUTPUT Section

7.2.1.1. The FILE–CONTROL Paragraph

7.2.1.1.1. Function

The FILE–CONTROL paragraph in the INPUT-OUTPUT section names each file and allows specification of other file-related information.

7.2.1.1.2. General Format

```
FILE-CONTROL. { file-control-entry }  . . .
```

7.2.1.2. The File Control Entry

7.2.1.2.1. Function

The file control entry names a sort or merge file and specifies the association of the file to a storage medium.

7.2.1.2.2. General Format

```
SELECT file-name ASSIGN TO implementor-name-1 [,implementor-name-2  ]  . . .
```

7.2.1.2.3. Syntax Rules

1. Each sort or merge file described in the Data Division must be named once and only once as *file-name* in the FILE–CONTROL paragraph. Each sort or merge file specified in the file control entry must have a sort-merge file description entry in the Data Division.
2. Since *file-name* represents a sort or merge file, only the ASSIGN clause is permitted to follow *file-name* in the FILE–CONTROL paragraph.
3. *Implementor-name-1, implementor-name-2,* etc., must conform to the rules specified for Sequential, Relative, or Indexed files. They are for documentation only.

7.2.1.2.4. General Rules

1. The ASSIGN clause specifies the association of the sort or merge file referenced by *file-name* to a storage medium.

7.2.1.3. The I–O–CONTROL Paragraph

7.2.1.3.1. Function

The I–O–CONTROL paragraph specifies the memory area which is to be shared by different files.

7.2.1.3.2. General Format



7.2.1.3.3. Syntax Rules

1. The I–O–CONTROL paragraph is optional.
2. In the SAME AREA clause, SORT and SORT–MERGE are equivalent.
3. If the SAME SORT AREA or SAME SORT–MERGE AREA clause is used, at least one of the file-names must represent a sort or merge file. Files that do not represent sort or merge files may also be named in the clause.
4. The three formats of the SAME clause (SAME RECORD AREA, SAME SORT AREA, SAME SORT–MERGE AREA) are considered separately in the following:

   More than one SAME clause may be included in a program, however:

   a. A *file-name* must not appear in more than one SAME RECORD AREA clause.

   b. A *file-name* that represents a sort or merge file must not appear in more than one SAME SORT AREA or SAME SORT–MERGE AREA clause.

   c. If a *file-name* that does not represent a sort or merge file appears in a SAME AREA clause and one or more SAME SORT AREA or SAME SORT–MERGE AREA clauses, all of the files named in that SAME AREA clause must be named in that SAME SORT AREA or SAME SORT–MERGE AREA clause(s). (See 4.2.1.3.)

5. The files referenced in the SAME SORT AREA, SAME SORT–MERGE AREA, or SAME RECORD AREA clause need not all have the same organization or access.

7.2.1.3.4. General Rules

1. The SAME RECORD AREA clause specifies that two or more files are to use the same memory area for processing of the current logical record. All of the files may be open at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each opened output file whose file-name appears in this SAME RECORD AREA clause and of the most recently read input file whose file-name appears in this SAME RECORD AREA clause. This is equivalent to implicit redefinition of the area, in other words, records are aligned on the leftmost character position.
2. If the SAME SORT AREA or SAME SORT–MERGE AREA clause is used, at least one of the file-names must represent a sort or merge file. Files that do not represent sort or merge files may also be named in the clause. This clause specifies that storage is shared as follows:

   a. The SAME SORT AREA or SAME SORT–MERGE AREA clause specifies a memory area which will be made available for use in sorting or merging each sort or merge file named. Thus any memory area allocated for the sorting or merging of a sort or merge file is available for reuse in sorting or merging any of the other sort or merge files.

   b. In addition, storage areas assigned to files that do not represent sort or merge files may be allocated as needed for sorting or merging the sort or merge files named in the SAME SORT AREA or SAME SORT–MERGE AREA clause. The extent of such allocation will be specified by the implementor.

   c. Files other than sort or merge files do not share the same storage area with each other. If the user wishes these files to share the same storage area with each other, he must also include in the program a SAME AREA or SAME RECORD AREA clause naming these files.

   d. During the execution of a SORT or MERGE statement that refers to a sort or merge file named in this clause, any nonsort-merge files named in this clause must not be open.

7.3. Data Division

The COBOL Sort feature requires the use of a special type of file description called a Sort File Description.

7.3.1. File Section

An SD file description gives information about the size and the names of the data records associated with the file to be sorted or merged. There are no label procedures that the user can control, and the rules for blocking and internal storage are peculiar to the SORT and MERGE statements.

7.3.2. The Sort-Merge File Description—Complete Entry Skeleton

7.3.2.1. Function

The sort-merge file description furnishes information concerning the physical structure, identification, and record names of the file to be sorted or merged.

7.3.2.2. General Format

```
SD  file-name

    [ ; RECORD CONTAINS [ integer-1 TO ] integer-2  CHARACTERS ]

[ ; DATA { RECORD IS   } data-name-1  [ , file-name-2 ] ... ] .
         { RECORDS ARE }
```

7.3.2.3. Syntax Rules

1. The level indicator SD identifies the beginning of the sort-merge file description and must precede the *file-name*.
2. The clauses that follow the name of the file are optional and their order of appearance is immaterial.
3. One or more record description entries must follow the sort-merge file description entry, however, no input-output statements may be executed for this file.

7.3.3. The DATA RECORDS Clause

7.3.3.1. Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

7.3.3.2. General Format

```
DATA { RECORD IS   } data-name-1  [ , data-name-2 ] ...
     { RECORDS ARE }
```

7.3.3.3. Syntax Rules

1. *Data-name-1* and *data-name-2* are the names of data records and must have 01 level-number record descriptions, with the same names, associated with them.

7.3.3.4. General Rules

1. The presence of more than one data-name indicates that the file contains more than one type of data record. These records may be of differing sizes, different formats, etc. The order in which they are listed is not significant.
2. Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

7.3.4. The RECORD CONTAINS Clause

7.3.4.1. Function

The RECORD CONTAINS clause specifies the size of data records.

7.3.4.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

7.3.4.3. General Rules

1. The size of each data record is completely defined within the record description entry, therefore this clause is never required. When present, however, the following notes apply:
   a. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case *integer-2* represents the exact number of characters in the data record. If *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.
   b. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. (See 1.3.3.3.4, 2.5.11, and 2.5.12.)

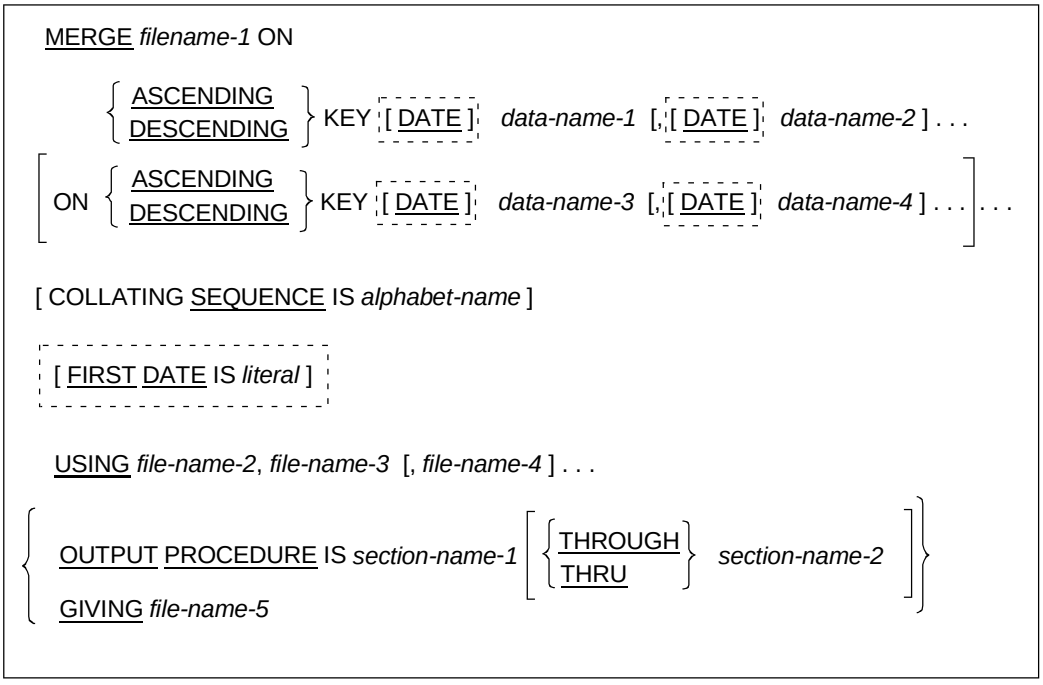
7.4. Procedure Division

7.4.1. The MERGE Statement

7.4.1.1. Function

The MERGE statement combines two or more identically sequenced files on a set of specified keys, and during the process makes records available, in merge order, to an output procedure or to an output file.

7.4.1.2. General Format



7.4.1.3. Syntax Rules

1. *File-name-1* must be described in a sort-merge file description entry in the Data Division.
2. *Section-name-1* represents the name of an output procedure.
3. *File-name-2, file-name-3, file-name-4,* and *file-name-5* must be described in a file description entry, not in a sort-merge file description entry, in the Data Division. The actual size of the logical record(s) described for *file-name-2, file-name-3, file-name-4,* and, *file-name-5* must be equal to the actual size of the logical records described for *file-name-1*. If the data descriptions of the elementary items that make up these records are not identical, it is the programmer's responsibility to describe the corresponding records in such a manner so as to cause an equal number of character positions to be allocated for the corresponding records.
4. The words THRU and THROUGH are equivalent.

5. *Data-name-1, data-name-2, data-name-3,* and *data-name-4* are KEY data-names and are subject to the following rules:

    a. The data items identified by KEY data–names must be described in records associated with *file-name-1.*

    b. KEY data-names may be qualified.

    c. The data items identified by KEY data-names must not be variable length items.

    d. If *file-name-1* has more than one record description, then the data items identified by KEY data-names need be described in only one of the record descriptions.

    e. None of the data items identified by KEY data-names can be described by an entry which either contains an OCCURS clause or is subordinate to an entry which contains an OCCURS clause.

6. No more than one file-name from a multiple file reel can appear in the MERGE statement.

7. File-names must not be repeated within the MERGE statement.

8. MERGE statements may appear anywhere except in the declaratives portion of the Procedure Division or in an input or output procedure associated with a SORT or MERGE statement.

9. *Literal* must be a one or two digit integer in the range 0 through 99.

10. If a DATE reserved word is specified, the KEY data item that immediately follows is assumed to be a date key and the following restrictions and conditions exist:

    a. The data item must be described as a USAGE DISPLAY item that contains at least two characters.

    b. If the data item is numeric it must be unsigned.

7.4.1.4. General Rules

1. The MERGE statement will merge all records contained on *file-name-2, file-name-3,* and *file-name-4.* The files referenced in the MERGE statement must not be open at the time the MERGE statement is executed. These files are automatically opened and closed by the merge operation with all implicit functions performed, such as the execution of any associated USE procedures. The terminating function for all files is performed as if a CLOSE statement, without optional phrases, had been executed for each file.

2. The data-names following the word KEY are listed from left to right in the MERGE statement in order of decreasing significance without regard to how they are divided into KEY phrases. In the format, *data-name-1* is the major key, *data-name-2* is the next most significant key, etc.

    a. When the ASCENDING phrase is specified, the merged sequence will be from the lowest value of the contents of the data items identified by the KEY data-names to the highest value, according to the rules for comparison of operands in a relation condition.

    b. When the DESCENDING phrase is specified, the merged sequence will be from the highest value of the contents of the data items identified by the KEY data-names to the lowest value, according to the rules for comparison of operands in a relation condition.

3. The collating sequence that applies to the comparison of the nonnumeric DISPLAY key data items specified is determined in the following order of precedence:

    a. First, the collating sequence established by the COLLATING SEQUENCE phrase, if specified, in that MERGE statement.

    b. Second, the collating sequence established as the program collating sequence.

4. The output procedure must consist of one or more sections that appear contiguously in a source program and do not form a part of any other procedure. In order to make merged records available for processing, the output procedure must include the execution of at least one RETURN statement. Control must not be passed to the output procedure except when a related SORT or MERGE statement is being executed. The output procedure may consist of any procedures needed to select, modify, or copy the records that are being returned one at a time in merged order, from *file-name-1*. The restrictions on the procedural statements within the output procedure are as follows:

    a. The output procedure must not contain any transfers of control to points outside the output procedure; ALTER, GO TO, and PERFORM statements in the output procedure are not permitted to refer to procedure-names outside the output procedure. COBOL statements are allowed that will cause an implied transfer of control to declaratives.

    b. The output procedures must not contain any SORT or MERGE statements.

    c. The remainder of the Procedure Division must not contain any transfers of control to points inside the output procedures; ALTER, GO TO, and PERFORM statements in the remainder of the Procedure Division are not permitted to refer to procedure-names within the output procedures.

5. If an output procedure is specified, control passes to it during execution of the MERGE statement. The compiler inserts a return mechanism at the end of the last section in the output procedure. When control passes the last statement in the output procedure, the return mechanism provides for termination of the merge, and then passes control to the next executable statement after the MERGE statement. Before entering the output procedure, the merge procedure reaches a point at which it can select the next record in merged order when requested. The RETURN statements in the output procedure are the requests for the next record.

6. Segmentation (see Section 9) can be applied to programs containing the MERGE statement. However, the following restrictions apply:

   a. If the MERGE statement appears in a section that is not in an independent segment, then any output procedure referenced by that MERGE statement must appear:

      - Totally within nonindependent segments, or
      - Wholly contained in a single independent segment.

   b. If a MERGE statement appears in an independent segment, then any output procedure referenced by that MERGE statement must be contained:

      - Totally within nonindependent segments, or
      - Wholly within the same independent segment as that MERGE statement.

7. If the GIVING phrase is specified, all the merged records in *file-name-1* are automatically written on *file-name-5* as the implied output procedure for this MERGE statement.
8. In the case of an equal compare, according to the rules for comparison of operands in a relation condition, on the contents of the data items identified by all the KEY data–names between records from two or more input files (*file-name-2, file-name-3, file-name–4,* . . .), the records are written on *file-name-5* or returned to the output procedure, depending on the phrase specified, in the order that the associated input files are specified in the MERGE statement.
9. The results of the merge operation are predictable only when the records in the files referenced by *file-name-2, file-name-3,* . . ., are ordered as described in the ASCENDING or DESCENDING KEY clause associated with the MERGE statement.
10. If DATE immediately precedes *data-name-1, data-name-2, data-name-3*, or *data-name-4,* the data item is a date key and the two-digit year is assumed to be the leftmost two character positions in the data item. The remaining characters (bits) of the date key field are assumed to be a bit string (unsigned binary number).
11. The FIRST DATE phrase only affects the collation order for date keys and has no affect on the other keys or the MERGE statement if no date keys are specified.
12. Date keys are ordered according to the following conventions:

    a. If the date key is in an ASCENDING KEY phrase, the two character year position is ordered as literal, *literal*+1, *literal*+2,...,'99', '00', '01', ... , *literal*-1. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits cleared to all bits set.

    b. If the date key is in an DESCENDING KEY phrase, the two character year position is ordered as *literal*-1, *literal*-2, ... , '00', '99', '98', ... , *literal*. If *literal* is not specified in the FIRST DATE phrase of the MERGE statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits set to all bits cleared.

7.4.2. The RELEASE Statement

7.4.2.1. Function

The RELEASE statement transfers records to the initial phase of a SORT operation.

7.4.2.2. General Format

```
RELEASE record-name [  FROM identifier ]
```

7.4.2.3. Syntax Rules

1. A RELEASE statement may only be used within the range of an input procedure associated with a SORT statement for a file whose sort-merge file description entry contains *record-name.* (See 7.4.4.)
2. *Record-name* must be the name of a logical record in the associated sort-merge file description entry and may be qualified.
3. *Record-name* and *identifier* must not refer to the same storage area.

7.4.2.4. General Rules

1. The execution of a RELEASE statement causes the record named by *record-name* to be released to the initial phase of a sort operation.
2. If the FROM phrase is used, the contents of the identifier data area are moved to *record-name,* then the contents of *record-name* are released to the sort file. Moving takes place according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The information in the record area is no longer available, but the information in the data area associated with *identifier* is available.
3. After the execution of the RELEASE statement, the logical record is no longer available in the record area unless the associated sort-merge file is named in a SAME RECORD AREA clause. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated sort-merge file, as well as to the file associated with *record-name*. When control passes from the input procedure, the file consists of all those records that were placed in it by the execution of RELEASE statements.

7.4.3. The RETURN Statement

7.4.3.1. Function

The RETURN statement obtains either sorted records from the final phase of a SORT operation or merged records during a MERGE operation.

7.4.3.2. General Format

```
RETURN file-name RECORD [ INTO identifier ] ; AT END imperative-statement
```

7.4.3.3. Syntax Rules

1. *File-name* must be described by a sort-merge file description entry in the Data Division.
2. A RETURN statement may only be used within the range of an output procedure associated with a SORT or MERGE statement for *file-name.*
3. The INTO phrase must not be used when the input file contains logical records of various sizes as indicated by their record descriptions. The storage area associated with *identifier* and the record area associated with *file-name* must not be the same storage area.

7.4.3.4. General Rules

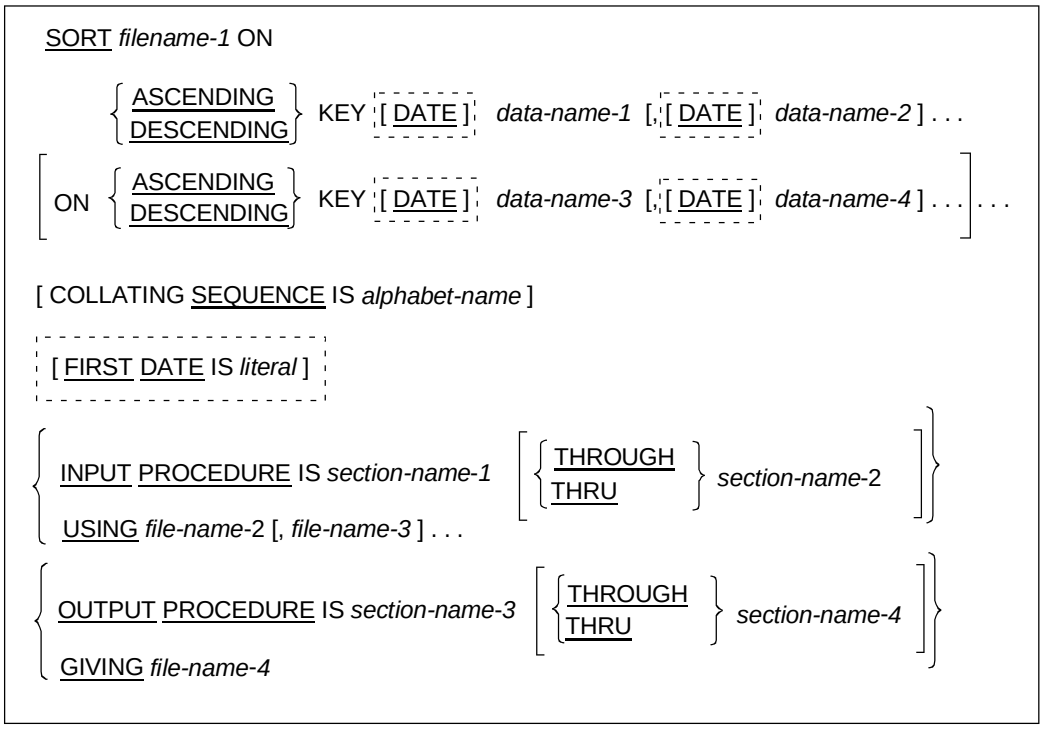
1. When the logical records of a file are described with more than one record description, these records automatically share the same storage area. This is equivalent to an implicit redefinition of the area. The contents of any data items which lie beyond the range of the current data record are undefined at the completion of the execution of the RETURN statement.
2. The execution of the RETURN statement causes the next record in the order specified by the keys listed in the SORT or MERGE statement, to be made available for processing in the record areas associated with the sort or merge file.
3. If the INTO phrase is specified, the current record is moved from the input area to the area specified by identifier according to the rules for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if there is an AT END condition. Any subscripting or indexing associated with *identifier* is evaluated after the record has been returned and immediately before it is moved to the data item.
4. When the INTO phrase is used, the data is available in both the input record area and the data area associated with *identifier*.
5. If no next logical record exists for the file at the time of the execution of a RETURN statement, the AT END condition occurs. The contents of the record areas associated with the file when the AT END condition occurs are undefined. After the execution of the *imperative-statement* in the AT END phrase, no RETURN statement may be executed as part of the current output procedure.

7.4.4. The SORT Statement

7.4.4.1. Function

The SORT statement creates a sort file by executing input procedures or by transferring records from another file, sorts the records in the sort file on a set of specified keys, and in the final phase of the sort operation, makes available each record from the sort file, in sorted order, to some output procedures or to an output file.

7.4.4.2. General Format



7.4.4.3. Syntax Rules

1. *File-name-1* must be described in a sort-merge file description entry in the Data Division.
2. *Section-name-1* represents the name of an input procedure. *Section-name-3* represents the name of an output procedure.
3. *File-name-2, file-name-3* and *file-name-4* must be described in a file description entry, not in a sort-merge file description entry, in the Data Division. The actual size of the logical records described for *file-name-2, file-name-3* and *file-name-4* must be equal to the actual size of the logical records described for *file-name-1.* If the data descriptions of the elementary items that make up these records are not identical, it is the programmer's responsibility to describe the corresponding records in such a manner so as to cause equal amounts of character positions to be allocated for the corresponding records.

4. *Data-name-1, data-name-2, data-name-3,* and *data-name-4* are KEY data-names and are subject to the following rules:
   a. The data items identified by KEY data-names must be described in records associated with *file-name-1.*
   b. KEY data-names may be qualified.
   c. The data items identified by KEY data-names must not be variable length items.
   d. If *file-name-1* has more than one record description, then the data items identified by KEY data-names need be described in only one of the record descriptions.
   e. None of the data items identified by KEY data-names can be described by an entry which either contains an OCCURS clause or is subordinate to an entry which contains an OCCURS clause.
5. The words THRU and THROUGH are equivalent.
6. SORT statements may appear anywhere except in the declaratives portion of the Procedure Division or in an input or output procedure associated with a SORT or MERGE statement.
7. No more than one file-name from a multiple file reel can appear in the SORT statement.
8. *Literal* must be a one or two digit integer in the range 0 through 99.
9. If a DATE reserved word is specified, the KEY data item that immediately follows is assumed to be a date key and the following restrictions and conditions exist:
   a. The data item must be described as a USAGE DISPLAY item that contains at least two characters.
   b. If the data item is numeric it must be unsigned.

7.4.4.4. General Rules

1. In Level 1, the Procedure Division of a program contains one SORT statement and a STOP RUN statement in the first nondeclarative portion. Other sections consist of only the input and output procedures associated with the SORT statement.
2. In Level 2, the Procedure Division may contain more than one SORT statement appearing anywhere except:
   a. in the declaratives portion, or
   b. in the input and output procedures associated with a SORT or MERGE statement.

3. The data-names following the word KEY are listed from left to right in the SORT statement in order of decreasing significance without regard to how they are divided into KEY phrases. In the format, *data-name-1* is the major key, *data-name-2* is the next most significant key, etc.

    a. When the ASCENDING phrase is specified, the sorted sequence will be from the lowest value of the contents of the data items identified by the KEY data-names to the highest value, according to the rules for comparison of operands in a relation condition.

    b. When the DESCENDING phrase is specified, the sorted sequence will be from the highest value of the contents of the data items identified by the KEY data-names to the lowest value, according to the rules for comparison of operands in a relation condition.

4. The collating sequence that applies to the comparison of the nonnumeric DISPLAY key data items specified is determined in the following order of precedence:

    a. First, the collating sequence established by the COLLATING SEQUENCE phrase, if specified, in the SORT statement.

    b. Second, the collating sequence established as the program collating sequence.

5. The input procedure must consist of one or more sections that appear contiguously in a source program and do not form a part of any output procedure. In order to transfer records to the file referenced by *file-name-1,* the input procedure must include the execution of at least one RELEASE statement. Control must not be passed to the input procedure except when a related SORT statement is being executed. The input procedure can include any procedures needed to select, create, or modify records. The restrictions on the procedural statements within the input procedure are as follows:

    a. The input procedure must not contain any SORT or MERGE statements.

    b. The input procedure must not contain any explicit transfers of control to points outside the input procedure; ALTER, GO TO, and PERFORM statements in the input procedure are not permitted to refer to procedure-names outside the input procedure. COBOL statements are allowed that will cause an implied transfer of control to declaratives.

    c. The remainder of the Procedure Division must not contain any transfers of control to points inside the input procedure; ALTER, GO TO and PERFORM statements in the remainder of the Procedure Division must not refer to procedure-names within the input procedure.

6. If an input procedure is specified, control is passed to the input procedure before *file-name-1* is sequenced by the SORT statement. The compiler inserts a return mechanism at the end of the last section in the input procedure and when control passes the last statement in the input procedure, the records that have been released to *file-name-1* are sorted.

7. The output procedure must consist of one or more sections that appear contiguously in a source program and do not form part of any input procedure. In order to make sorted records available for processing, the output procedure must include the execution of at least one RETURN statement. Control must not be passed to the output procedure except when a related SORT statement is being executed. The output procedure may consist of any procedures needed to select, modify or copy the records that are being returned, one at a time in sorted order, from the sort file. The restrictions on the procedural statements within the output procedure are as follows:

    a. The output procedure must not contain any SORT or MERGE statements.

    b. The output procedure must not contain any explicit transfers of control to points outside the output procedure; ALTER, GO TO, and PERFORM statements in the output procedure are not permitted to refer to procedure-names outside the output procedure. COBOL statements are allowed that will cause an implied transfer of control to declaratives.

    c. The remainder of the Procedure Division must not contain any transfers of control to points inside the output procedure. ALTER, GO TO, and PERFORM statements in the remainder of the Procedure Division are not permitted to refer to procedure-names within the output procedure.

8. If an output procedure is specified, control passes to it after *file-name-1* has been sequenced by the SORT statement. The compiler inserts a return mechanism at the end of the last section in the output procedure and when control passes the last statement in the output procedure, the return mechanism provides for termination of the sort and then passes control to the next executable statement after the SORT statement. Before entering the output procedure, the sort procedure reaches a point at which it can select the next record in sorted order when requested. The RETURN statements in the output procedure are the requests for the next record.

9. Segmentation (see Section 9) can be applied to programs containing the SORT statement. However, the following restrictions apply:

    a. If a SORT statement appears in a section that is not in an independent segment, then any input procedures or output procedures referenced by that SORT statement must appear:

        - Totally within nonindependent segments, or
        - Wholly contained in a single independent segment.

    b. If a SORT statement appears in an independent segment, then any input procedures or output procedures referenced by that SORT statement must be contained:

        - Totally within nonindependent segments, or
        - Wholly within the same independent segment as that SORT statement.

10. If the USING phrase is specified, all the records in *file-name-2* and *file-name-3* are transferred automatically to *file–name–1.* At the time of execution of the SORT statement, *file-name-2* and *file-name-3* must not be open. The SORT statement automatically initiates the processing of, makes available the logical records for, and terminates the processing of *filename-2* and *file-name-3.* These implicit functions are performed such that any associated USE procedures are executed. The terminating function for all files is performed as if a CLOSE statement, without optional phrases, had been executed for each file. The SORT statement also automatically performs the implicit functions of moving the records from the file area of *file-name-2* and *file-name-3* to the file area for *file-name-1* and the release of records to the initial phase of the sort operation.
11. If the GIVING phrase is specified, all the sorted records in *file-name-1* are automatically written on *file-name-4* as the implied output procedure for this SORT statement. At the time of execution of the SORT statement *file-name-4* must not be open. The SORT statement automatically initiates the processing of, releases the logical records to, and terminates the processing of *file-name-4.* These implicit functions are performed such that any associated USE procedures are executed. The terminating function is performed as if a CLOSE statement, without optional phrases, had been executed for the file. The SORT statement also automatically performs the implicit functions of the return of the sorted records from the final phase of the sort operation and the moving of the records from the file area for *file–name-1* to the file area for *file-name-4*.
12. If DATE immediately precedes *data-name-1*, *data-name-2*, *data-name-3*, or *data-name-4*, the data item is a date key and the two-digit year is assumed to be the leftmost two character positions in the data item. The remaining characters (bits) of the date key field are assumed to be a bit string (unsigned binary number).
13. The FIRST DATE phrase only affects the collation order for date keys and has no affect on the other keys or the SORT statement if no date keys are specified.
14. Date keys are ordered according to the following conventions:

    a. If the date key is in an ASCENDING KEY phrase, the two character year position is ordered as *literal*, *literal*+1, *literal*+2,...,'99', '00', '01', ... , *literal-1*. If *literal* is not specified in the FIRST DATE phrase of the SORT statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits cleared to all bits set.

    b. If the date key is in an DESCENDING KEY phrase, the two character year position is ordered as *literal-1*, *literal-2*, ... , '00', '99', '98', ... , *literal*. If *literal* is not specified in the FIRST DATE phrase of the SORT statement, the Sort/Merge default is assumed for the collation order. When the year positions of the date key are the same, the remaining part of the key, if any, is ordered from all bits set to all bits cleared.

7.5. Examples

The examples that follow illustrate the two basic organizational approaches that may be adopted by the programmer in writing a Sort program in ASCII COBOL. Note that a file "XA" must be assigned for use by the Sort routine (see the *OS 2200 Sort/Merge Programming Guide,* (7831 0687)).

Example 1:

```
 1     IDENTIFICATION DIVISION.
 2     PROGRAM-ID. SORTA.
 3    *     THIS SORT PROGRAM EXAMPLE ILLUSTRATES
 4    *     THE FOLLOWING SORT OPTIONS
 5    *     INPUT PROCEDURE IS
 6    *     OUTPUT PROCEDURE IS
 7     ENVIRONMENT DIVISION.
 8     CONFIGURATION SECTION.
 9     SOURCE-COMPUTER. UNISYS-1100.
10     OBJECT-COMPUTER. UNISYS-1100.
11     INPUT-OUTPUT SECTION.
12     FILE-CONTROL.
13         SELECT TO-BE-SORTED    ASSIGN TO TAPE D.
14         SELECT AFTER-THE-SORT  ASSIGN TO TAPE E.
15         SELECT WORK-ING        ASSIGN TO DISC XA.
16         SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
17     DATA DIVISION.
18     FILE SECTION.
19     FD  TO-BE-SORTED
20         LABEL RECORDS ARE OMITTED
21         DATA RECORD IS AûFILE.
22     01  A-FILE.
23         02 1-ACCOUNT PICTURE IS X(4).
24         02 1-TYPE PICTURE IS X.
25         02 1-AREA PICTURE IS XXX.
26         02 1-BODY PICTURE IS X(72).
27     FD  AFTER-THE-SORT
28         LABEL RECORDS ARE OMITTED
29         DATA RECORD IS B-FILE.
30     01  B-FILE PICTURE IS X(80).
31     SD  WORK-ING
32         DATA RECORD IS C-FILE.
33     01  C-FILE.
34         02  3-ACCOUNT PICTURE IS X(4).
35         02  3-TYPE PICTURE IS X.
36         02  3-AREA PICTURE IS XXX.
37         02  3-BODY PICTURE IS X(72).
38     FD  FOR-THE-PRINTER
39         LABEL RECORDS ARE OMITTED
40         DATA RECORD IS D-FILE.
41     01  D-FILE PICTURE IS X(80).
```

```
42      PROCEDURE DIVISION.
43      OPEN-SECTION SECTION.
44      OPENING.
45          OPEN INPUT TO-BE-SORTED.
46          OPEN OUTPUT AFTER-THE-SORT FOR-THE-PRINTER.
47      A-SORT SECTION.
48      P1. SORT WORK-ING ON ASCENDING KEY 3-ACCOUNT 3-TYPE 3-AREA
49          INPUT PROCEDURE IS B-SORT THRU B1-SORT
50          OUTPUT PROCEDURE IS C-SORT THRU C1-SORT.
51          GO TO OK-PRINT.
52      B-SORT SECTION.
53      P2. READ TO-BE-SORTED AT END GO TO B1-SORT.
54          RELEASE C-FILE FROM A-FILE.
55          GO TO B-SORT.
56      B1-SORT SECTION.
57      P3. CLOSE TO-BE-SORTED.
58      C-SORT SECTION.
59      P4. RETURN WORK-ING RECORD INTO B-FILE AT END GO TO C1-SORT.
60          WRITE B-FILE.
61          GO TO C-SORT.
62      C1-SORT SECTION.
63      P5. CLOSE AFTER-THE-SORT.
64      OK-PRINT SECTION.
65      P6. OPEN INPUT AFTER-THE-SORT.
66      A-LOOP SECTION.
67      P7. READ AFTER-THE-SORT AT END GO TO A-1-MOVE.
68          PERFORM A-MOVE.
69          WRITE D-FILE.
70          GO TO A-LOOP.
71      A-MOVE SECTION.
72      P8. MOVE B-FILE TO D-FILE.
73      A-1-MOVE SECTION.
74      P9. EXIT.
75      ALL-DONE SECTION.
76      PA. CLOSE AFTER-THE-SORT FOR-THE-PRINTER.
77          STOP RUN.
```

In this example, the files defined by FD entries are opened before the SORT statement is executed. The SORT statement sets the sorting mechanism in motion by transferring control to the INPUT PROCEDURE. The files can be opened with the INPUT PROCEDURE, but the input file must not be opened more than once or closed before input is completed.

Sorting is conducted on an ASCENDING KEY. Note that the KEY identifiers are defined in the sort file WORK–ING, not the input file TO–BE–SORTED.

The INPUT PROCEDURE is completed when the entire input file has been transferred to the sort file. The sort mechanism then transfers control to the OUTPUT PROCEDURE, and records are written on the file AFTER–THE–SORT in the order specified by the KEY parameters. The sorted records are then listed after the sort function has been accomplished.

Example 2:

```
 1     IDENTIFICATION DIVISION.
 2     PROGRAM-ID. SORTB.
 3    *       THIS SORT PROGRAM EXAMPLE ILLUSTRATES
 4    *       THE FOLLOWING SORT OPTIONS
 5    *       USING
 6    *       GIVING.
 7     ENVIRONMENT DIVISION.
 8     CONFIGURATION SECTION.
 9     SOURCE-COMPUTER. UNISYS-1100.
10     OBJECT-COMPUTER. UNISYS-1100.
11     INPUT-OUTPUT SECTION.
12     FILE-CONTROL.
13         SELECT TO-BE-SORTED    ASSIGN TO TAPE D.
14         SELECT AFTER-THE-SORT  ASSIGN TO TAPE E.
15         SELECT WORK-ING        ASSIGN TO DISC XA.
16         SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
17     DATA DIVISION.
18     FILE SECTION.
19     FD  TO-BE-SORTED
20         LABEL RECORDS ARE OMITTED
21         DATA RECORD IS A-FILE.
22     01  A-FILE.
23         02 1-ACCT PICTURE IS X(4).
24         02 1-DEPT PICTURE IS X(4).
25         02 1-BODY PICTURE IS X(72).
26     FD  AFTER-THE-SORT
27         LABEL RECORDS ARE OMITTED
28         DATA RECORD IS B-FILE.
29     01  B-FILE.
30         02 2-ACCT PICTURE IS X(4).
31         02 2-DEPT PICTURE IS X(4).
32         02 2-BODY PICTURE IS X(72).
33     SD  WORK-ING
34         DATA RECORD IS C-FILE.
35     01  C-FILE.
36         02 3-ACCT PICTURE IS X(4).
37         02 3-DEPT PICTURE IS X(4).
38         02 3-BODY PICTURE IS X(72).
39     FD  FOR-THE-PRINTER
40         LABEL RECORDS ARE OMITTED
41         DATA RECORD IS D-FILE.
42     01  D-FILE.
43         02 4-ACCT PICTURE IS X(4).
44         02 4-DEPT PICTURE IS X(4).
45         02 4-BODY PICTURE IS X(72).
46     PROCEDURE DIVISION.
47     OPEN-SECTION SECTION.
48     OPENING.
49         OPEN OUTPUT FOR-THE-PRINTER.
50     A-SORT SECTION.
51     P1. SORT WORK-ING ON ASCENDING KEY 3-ACCT 3-DEPT
52         USING TO-BE-SORTED GIVING AFTER-THE-SORT.
```

```
53     OK-PRINT SECTION.
54     P2. OPEN INPUT AFTER-THE-SORT.
55     A-LOOP SECTION.
56     P3. READ AFTER-THE-SORT AT END GO TO A-1-MOVE.
57         PERFORM A-MOVE.
58         WRITE D-FILE.
59         GO TO A-LOOP.
60     A-MOVE SECTION.
61     P4. MOVE B-FILE TO D-FILE.
62     A-1-MOVE SECTION.
63     P5. EXIT.
64     ALL-DONE SECTION.
65     P6. CLOSE AFTER-THE-SORT FOR-THE-PRINTER.
66         STOP RUN.
```

In this second example, the files that are directly involved in the sort must not be open at the time the SORT statement is encountered. This is because the USING and GIVING options perform the functions of the INPUT PROCEDURE and OUTPUT PROCEDURE (in other words, OPEN, READ, RELEASE, RETURN, USE, WRITE, and CLOSE) automatically. Although the use of the GIVING and USING options requires a minimum amount of coding, it is not possible to select, add, delete, or modify individual records of the sort file in any way (in other words, no special processing procedures may be incorporated into a sort function when this technique is adopted).

Example 3:

```
 1     IDENTIFICATION DIVISION.
 2     PROGRAM-ID. MERGEA.
 3    *       THIS MERGE PROGRAM EXAMPLE ILLUSTRATES
 4    *       THE FOLLOWING MERGE OPTIONS
 5    *       USING
 6    *       GIVING.
 7    ENVIRONMENT DIVISION.
 8    CONFIGURATION SECTION.
 9    SOURCE-COMPUTER. UNISYS-1100.
10    OBJECT-COMPUTER. UNISYS-1100.
11    INPUT-OUTPUT SECTION.
12    FILE-CONTROL.
13        SELECT TO-BE-MERGED1   ASSIGN TO DISC D.
14        SELECT TO-BE-MERGED2   ASSIGN TO DISC F.
15        SELECT AFTER-THE-MERGE ASSIGN TO DISC E.
16        SELECT WORK-ING        ASSIGN TO DISC XA.
17        SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
18    DATA DIVISION.
19    FILE SECTION.
20    FD  TO-BE-MERGED1
21        LABEL RECORDS ARE OMITTED
22        DATA RECORD IS A-FILE1.
23    01  A-FILE1.
24        02 1-ACCT PICTURE IS X(4).
25        02 1-DEPT PICTURE IS X(4).
26        02 1-BODY PICTURE IS X(72).
```

```
27    FD  TO-BE-MERGED2
28        LABEL RECORDS ARE OMITTED
29        DATA RECORD IS A-FILE2.
30    01  A-FILE2.
31        02 1-ACCT PICTURE IS X(4).
32        02 1-DEPT PICTURE IS X(4).
33        02 1-BODY PICTURE IS X(72).
34    FD  AFTER-THE-MERGE
35        LABEL RECORDS ARE OMITTED
36        DATA RECORD IS B-FILE.
37    01  B-FILE.
38        02 2-ACCT PICTURE IS X(4).
39        02 2-DEPT PICTURE IS X(4).
40        02 2-BODY PICTURE IS X(72).
41    SD  WORK-ING
42        DATA RECORD IS C-FILE.
43    01  C-FILE.
44        02  3-ACCT PICTURE IS X(4).
45        02  3-DEPT PICTURE IS X(4).
46        02  3-BODY PICTURE IS X(72).
47    FD  FOR-THE-PRINTER
48        LABEL RECORDS ARE OMITTED
49        DATA RECORD IS D-FILE.
50    01  D-FILE.
51        02 4-ACCT PICTURE IS X(4).
52        02 4-DEPT PICTURE IS X(4).
53        02 4-BODY PICTURE IS X(72).
54    PROCEDURE DIVISION.
55    OPEN-SECTION SECTION.
56    OPENING.
57        OPEN OUTPUT FOR-THE-PRINTER.
58    A-MERGE SECTION.
59    P1. MERGE WORK-ING ON ASCENDING KEY 3-ACCT 3DEPT
60        USING TO-BE-MERGED1 TO-BE-MERGED2
61        GIVING AFTER-THE-MERGE.
62    OK-PRINT SECTION.
63    P2. OPEN INPUT AFTER-THE-MERGE.
61    A-LOOP SECTION.
65    P3. READ AFTER-THE-MERGE AT END GO TO A-1-MOVE.
66        PERFORM A-MOVE.
67        WRITE D-FILE.
68        GO TO A-LOOP.
69    A-MOVE SECTION.
70    P4. MOVE B-FILE TO D-FILE.
71    A-1-MOVE SECTION.
72    P5. EXIT.
73    ALL-DONE SECTION.
74    P6. CLOSE AFTER-THE-MERGE FOR-THE-PRINTER.
75        STOP RUN.
```

In this example, the files that are directly involved in the merge must not be open at the time the MERGE statement is encountered. This is because the USING and GIVING options perform the functions of OPEN, READ, RETURN, WRITE, and CLOSE automatically. Although the use of the USING and GIVING options require a minimum amount of coding, it is not possible to select, add, delete, or modify records of the sort/merge file in any way (in other words, no special processing procedures may be incorporated into a merge function when this technique is used).

This example will merge the files TO–BE–MERGED1 and TO–BE–MERGED2 into the file AFTER–THE–MERGE. This merged file is then listed after the merge function has been accomplished. The files that were merged were already in sorted order according to the keys defined in the MERGE statement.

Example 4:

```
 1     IDENTIFICATION DIVISION.
 2     PROGRAM-ID. MERGEB.
 3    *   THIS MERGE PROGRAM EXAMPLE ILLUSTRATES
 4    *   THE FOLLOWING MERGE OPTIONS
 5    *   USING
 6    *   OUTPUT PROCEDURE.
 7     ENVIRONMENT DIVISION.
 8     CONFIGURATION SECTION.
 9     SOURCE-COMPUTER. UNISYS-1100.
10     OBJECT-COMPUTER. UNISYS-1100.
11     INPUT-OUTPUT SECTION.
12     FILE-CONTROL.
13         SELECT TO-BE-SORTED ASSIGN TO DISC A.
14         SELECT BBB ASSIGN TO DISC D.
15         SELECT CCC ASSIGN TO DISC F.
16         SELECT ABC ASSIGN TO DISC C.
17         SELECT AFTER-THE-SORT ASSIGN TO DISC B.
18         SELECT WORK-ING ASSIGN TO DISC XA.
19         SELECT FOR-THE-PRINTER ASSIGN TO PRINTER.
20     DATA DIVISION.
21     FILE SECTION.
22     FD  TO-BE-SORTED
23         LABEL RECORDS ARE OMITTED
24         DATA RECORD IS A-FILE.
25     01  A-FILE.
26         02 1-ACCOUNT PIC 9999.
27         02 1-TYPE PIC X.
28         02 1-AREA PIC XXX.
29         02 1-BODY PIC X(72).
30     FD  ABC
31         LABEL RECORDS ARE OMITTED
32         DATA RECORD IS ABC-FILE.
33     01  ABC-FILE.
34         02 A-ACCOUNT PIC X(80).
35     FD  BBB
36         LABEL RECORDS ARE OMITTED
37         DATA RECORD IS BBB-FILE.
38     01  BBB-FILE.
39         02 B-ACCOUNT PIC X(80).
```

```
40     FD  CCC
41         LABEL RECORDS ARE OMITTED
42         DATA RECORD IS CCC-FILE.
43     01  CCC-FILE.
44         02 C-ACCOUNT PIC X(80).
45     FD  AFTER-THE-SORT
46         LABEL RECORDS ARE OMITTED
47         DATA RECORD IS B-FILE.
48     01  B-FILE PIC X(80).
49     SD  WORK-ING
50         DATA RECORD IS C-FILE.
51     01  C-FILE.
52         02 3-ACCT PIC XXXX.
53         02 3-TYPE PIC X.
54         02 3-DEPT PIC XXX.
55         02 3-BODY PIC X(72).
56     FD  FOR-THE-PRINTER
57         LABEL RECORDS ARE OMITTED
58         DATA RECORD IS D-FILE.
59     01  D-FILE PIC X(80).
60     WORKING-STORAGE SECTION.
61     01  KNT PIC 9999 VALUE 0.
62     PROCEDURE DIVISION.
63     OPEN-SECTION SECTION.
60     OPENING.
65         OPEN OUTPUT FOR-THE-PRINTER.
66     A-MERGE SECTION.
67     P1. MERGE WORK-ING ON ASCENDING KEY 3-ACCT
68         USING TO-BE-SORTED ABC BBB CCC
69         OUTPUT PROCEDURE IS OK-PRINT.
70         GO TO WWW.
71     OK-PRINT SECTION.
72     P3.
73         RETURN WORK-ING AT END GO TO RRR.
74         DISPLAY 3-ACCT.
75         GO TO P3.
76     RRR.
77         EXIT.
78     WWW SECTION.
79     P4.
80         CLOSE FOR-THE-PRINTER.
81         STOP RUN.
```

In this example, the files that are directly involved in the merge must not be open at the time the MERGE statement is being executed. This is because the USING option in the merge and the RETURN verb in the OUTPUT PROCEDURE perform the functions of OPEN, READ, and CLOSE automatically.

In this program, the files TO–BE–SORTED, ABC, BBB, CCC are merged and records are returned to the sort/merge record one at a time at which time the key field of the record is displayed. The files that were merged were already in sorted order according to the keys defined in the MERGE statement.

Example 5:

```
identification division.
program-id.
    dkst03.
environment division.
configuration section.
source-computer. unisys-2200.
object-computer. unisys-2200.
special-names.
    printer is prt.
input-output section.
file-control.
     select infile assign   to disc in1.
     select outfile assign  to disc out.
     select sortfile assign to disc.
data division.
file section.
fd   infile  label records standard.
01   inrec   pic x(40).
fd   outfile label records standard.
01   outrec  pic x(40).
sd   sortfile.
01   sortrec.
     02    key1   pic 9(6).
     02    filler pic x(8).
     02    key2   pic x(6).
     02    filler pic x(6).
     02    key3   pic 9(14).
working-storage section.
procedure division.
beginsec section.
begin.
    display 'Begin of program dkst03' upon prt.
stpara.
    sort sortfile
       ascending  key DATE key1 DATE key2
       descending key DATE key3
       FIRST DATE is 68
     using infile
     giving outfile.
endpara.
     display 'outfile created ' upon prt.
     display 'end of test dkst03' upon prt.
byepara.
    stop run.
```

This example demonstrates the use of the DATE and FIRST DATE syntax. Please note that DATE and FIRST DATE are in upper case only for emphasis purposes.

Section 8
Report Writer

8.1. General

8.1.1. Function

The Report Writer module provides the facility for producing reports by specifying the physical appearance of a report rather than requiring specification of the detailed procedures necessary to produce that report.

A hierarchy of levels is used in defining the logical organization of a report. Each report is divided into report groups, which in turn are divided into sequences of items. Such a hierarchical structure permits explicit reference to a report group with implicit reference to other levels in the hierarchy. A report group contains one or more items to be presented on one or more lines.

8.1.2. Language Concepts

8.1.2.1. LINE–COUNTER

The reserved word LINE–COUNTER is a name for a special register that is generated for each report description entry in the Report Section of the Data Division. The implicit definition is that of an 18–bit binary integer. The value in LINE–COUNTER is maintained by the Report Writer Control System (RWCS), and is used to determine the vertical positioning of a report. The value in LINE–COUNTER may be accessed by Procedure Division statements; however, only the RWCS may change the value of LINE–COUNTER.

8.1.2.2. PAGE–COUNTER

The reserved word PAGE–COUNTER is a name for a special register that is generated for each report description entry in the Report Section of the Data Division. The implicit definition is that of an 18-bit binary integer. The value in PAGE–COUNTER is maintained by the Report Writer Control System and is used by the program to number the pages of a report. The value in PAGE–COUNTER may be altered by Procedure Division statements. The value in PAGE–COUNTER must be greater than or equal to 1.

8.1.2.3. Subscripting

In the Report Section, neither a sum counter nor the special registers LINE–COUNTER and PAGE–COUNTER can be used as a subscript.

8.1.3. Relationship with Sequential I–O Module

A report file is a sequential file as described in the Sequential I–O module and is subject to the restrictions in the following paragraph.

An OPEN statement, specifying either the OUTPUT or EXTEND phrase, must have been executed prior to the execution of the INITIATE statement, and a CLOSE, without the REEL or UNIT phrase, must be executed for this file subsequent to the execution of the TERMINATE statement. No other input-output statement may be executed for this file.

8.2. Data Division

8.2.1. File Section

A REPORT clause is required in the FD entry to list the names of the reports to be produced.

8.2.2. Report Section

In the Report Section the description of each report must begin with a report description entry (RD entry) and be followed by the entries that describe the report groups within the report.

8.2.2.1. Report Description Entry

In addition to naming the report, the RD entry defines the format of each page of the report by specifying the vertical boundaries of the region within which each type of report group may be printed. The RD entry also specifies the control data items. When the report is produced, changes in the values of the control data items cause the detail information of the report to be processed in groups called control groups.

Each report named in the REPORTS clause of an FD entry in the File Section must be the subject of an RD entry in the Report Section. Furthermore, each report in the Report Section must be named in one and only one FD entry.

8.2.2.2. Report Group Description Entry

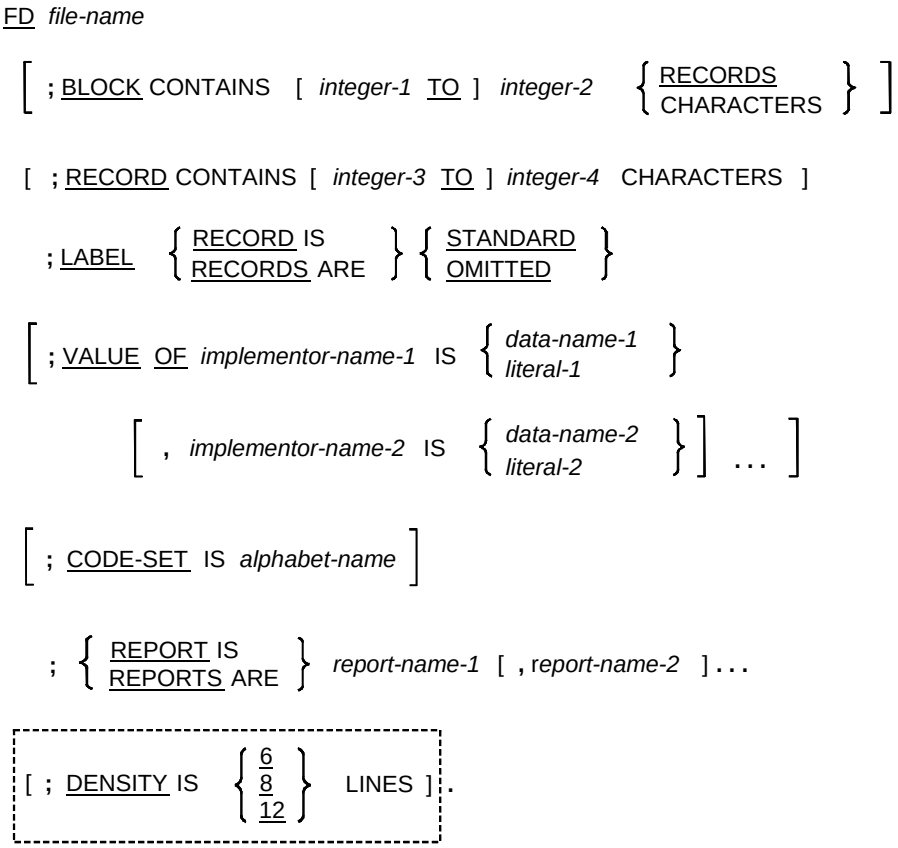
The report groups that will comprise the report are described following the RD entry. The description of each report group begins with a report group description entry; that is an entry that has a 01 level-number and a TYPE clause. Subordinate to the report group description entry, there may appear group and elementary entries that further describe the characteristics of the report group.

8.2.3. The File Description—Complete Entry Skeleton

8.2.3.1. Function

The file description furnishes information concerning the physical structure, identification, and report names pertaining to a given report file.

8.2.3.2. General Format



8.2.3.3. Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede the *file-name*.
2. The clauses which follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. The file referenced by *file-name* must be defined, implicitly or explicitly in the FILE–CONTROL paragraph of the Environment Division, as a sequential file. Further, each report named in the REPORT clause must be the subject of a report description entry in the Report Section.
4. No record description entries are permitted for *file-name* and no input-output statements, except the OPEN with either the OUTPUT or EXTEND phrase and the CLOSE without either the REEL or UNIT phrase, may be executed for this file.

8.2.4. The Report Description—Complete Entry Skeleton

8.2.4.1. Function

The report description entry names a report, specifies any identifying characters to be appended to each print line, and describes the physical structure and organization of that report.

8.2.4.2. General Format

```
RD  report-name

[  ; CODE  literal-1  ]

[  ;  { CONTROL IS  }  { data-name-1 [ , data-name-2 ]...                          }  ]
      { CONTROLS ARE }  { FINAL [ , data-name-1 [ , data-name-2 ]... ]             }

[ ; PAGE  [ LIMIT IS  ]  integer-1  [ LINE  ]  [ , HEADING integer-2 ]
          [ LIMITS ARE ]            [ LINES ]

      [ , FIRST  DETAIL  integer-3 ] [ , LAST  DETAIL  integer-4 ]

      [ , FOOTING  integer-5 ] ] .
```

8.2.4.3. Syntax Rules

1. The *report-name* must appear in one and only one REPORT clause.
2. The order of appearance of the clauses following *report-name* is immaterial.
3. *Report-name* is the highest permissible qualifier that may be specified for LINE–COUNTER, PAGE COUNTER, and all data-names defined within the Report Section.
4. One or more report group description entries must follow the report description entry.

8.2.4.4. PAGE–COUNTER Rules

1. PAGE–COUNTER is the reserved word used to reference a special register that is automatically created for each report specified in the Report Section. (See 8.1.2.2.)
2. In the Report Section, a reference to PAGE–COUNTER can only appear in a SOURCE clause. Outside of the Report Section, PAGE–COUNTER may be used in any context in which a data-name of integral value can appear.
3. If more than one PAGE–COUNTER exists in a program, PAGE–COUNTER must be qualified by a report-name whenever it is referenced in the Procedure Division.

   In the Report Section an unqualified reference to PAGE–COUNTER is implicitly qualified by the name of the report in which the reference is made. Whenever the PAGE–COUNTER of a different report is referenced, PAGE–COUNTER must be explicitly qualified by that report-name. (See 2.1.3.2 for constraints that apply when Report Writer is associated with the Nucleus, Level 1.)
4. Execution of the INITIATE statement causes the Report Writer Control System to set the PAGE–COUNTER of the referenced report to one.
5. PAGE–COUNTER is automatically incremented by one each time the Report Writer Control System executes a page advance.

6. PAGE–COUNTER may be altered by Procedure Division statements.

8.2.4.5. LINE–COUNTER Rules

1. LINE–COUNTER is the reserved word used to reference a special register that is automatically created for each report specified in the Report Section. (See 8.1.2.1.)
2. In the Report Section a reference to LINE–COUNTER can only appear in a SOURCE clause. Outside the Report Section, LINE–COUNTER may be used in any context in which a data-name of integral value may appear. However, only the Report Writer Control System can change the contents of LINE–COUNTER.
3. If more than one LINE–COUNTER exists in a program, LINE–COUNTER must be qualified by a report-name whenever it is referenced in the Procedure Division.

   In the Report Section an unqualified reference to LINE–COUNTER is implicitly qualified by the name of the report in which the reference is made. Whenever the LINE–COUNTER of a different report is referenced, LINE–COUNTER must be explicitly qualified by that report-name. (See 2.1.3.2 for constraints that apply when Report Writer is associated with the Nucleus, Level 1.)
4. Execution of an INITIATE statement causes the Report Writer Control System to set the LINE–COUNTER of the referenced report to zero. The Report Writer Control System also automatically resets LINE–COUNTER to zero each time it executes a page advance.
5. The value of LINE–COUNTER is not affected by the processing of nonprintable report groups nor by the processing of a printable report group whose printing is suppressed by means of the SUPPRESS statement.
6. At the time each print line is presented, the value of LINE–COUNTER represents the line number on which the print line is presented. The value of LINE–COUNTER after the presentation of a report group is governed by the presentation rules for each report group. (See 8.2.14, general rule 4.)

8.2.5. The Report Group Description—Complete Skeleton

8.2.5.1. Function

The report group description entry specifies the characteristics of a report group and of the individual items within a report group.

8.2.5.2. General Format

Format 1

```
01 [ data-name-1 ]

[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] } ]
                   { PLUS integer-2             }

[ ; NEXT GROUP IS { integer-3      } ]
                  { PLUS integer-4 }
                  { NEXT PAGE      }

                                                  { REPORT HEADING }
                                                  { RH             }
                                                  { PAGE HEADING   }
                                                  { PH             }
                                                  { CONTROL HEADING } { data-name-2 }
                                                  { CH              } { FINAL       }
                                                  { DETAIL         }
                                                  { DE             }
                                                  { CONTROL FOOTING } { data-name-3 }
                                                  { CF              } { FINAL       }
                                                  { PAGE FOOTING   }
                                                  { PF             }
                                                  { REPORT FOOTING }
                                                  { RF             }
; TYPE IS

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

Format 2

level-number [*data-name-1*]

```
[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] } ]
                   { PLUS integer-2               }

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

Format 3

level-number [*data-name-1*]

```
[ ; BLANK WHEN ZERO ]

[ ; GROUP INDICATE ]

[ ; { JUSTIFIED } RIGHT ]
    { JUST      }

[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] } ]
                   { PLUS integer-2             }

[ ; COLUMN NUMBER IS integer-3 ]

  ; { PICTURE } IS character-string
    { PIC     }

{ ; SOURCE IS identifier-1                                          }
{ ; VALUE IS literal                                                }
{ ; SUM identifier-2 [ , identifier-3 ] . . .                       }
{       [ UPON data-name-2 [ , data-name-3 ] . . . ] } . . .        }
{       [ RESET ON { data-name-4 } ]                                }
{                  { FINAL       }                                  }

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

8.2.5.3. Syntax Rules

1. The report group description entry can appear only in the Report Section.
2. Except for the *data-name* clause, which when present must immediately follow *level-number*, the clauses may be written in any sequence.
3. In Format 2 the *level-number* may be any integer from 02 to 48 inclusive.
   In Format 3, *level-number* may be any integer from 02 to 49 inclusive.
4. The description of a report group may consist of one, two, or three hierarchic levels:
   a. The first entry that describes a report group must be a Format 1 entry.
   b. Both Format 2 and Format 3 entries may be immediately subordinate to a Format 1 entry.
   c. At least one Format 3 entry must be immediately subordinate to a Format 2 entry.
   d. Format 3 entries must be elementary.
5. In a Format 1 entry, *data-name-1* is required only when:
   a. A DETAIL report group is referenced by a GENERATE statement,
   b. A DETAIL report group is referenced by the UPON phrase of a SUM clause,
   c. A report group is referenced in a USE BEFORE REPORTING sentence,
   d. The name of a CONTROL FOOTING report group is used to qualify a reference to a sum counter.
6. A Format 2 entry must contain at least one optional clause.
7. In a Format 2 entry, *data-name-1* is optional. If present, it may be used only to qualify a sum counter reference.
8. In the Report Section, the USAGE clause is used only to declare the usage of printable items.
   a. If the USAGE clause appears in a Format 3 entry, that entry must define a printable item.
   b. If the USAGE clause appears in a Format 1 or Format 2 entry, at least one subordinate entry must define a printable item.
9. An entry that contains a LINE NUMBER clause must not have a subordinate entry that also contains a LINE NUMBER clause.
10. In Format 3:
    a. A GROUP INDICATE clause may appear only in a TYPE DETAIL report group.
    b. A SUM clause may appear only in a TYPE CONTROL FOOTING report group.
    c. An entry that contains a COLUMN NUMBER clause, but no LINE NUMBER clause, must be subordinate to an entry that contains a LINE NUMBER clause.
    d. *Data-name-1* is optional but may be specified in any entry. *Data-name-1*, however, may be referenced only if the entry defines a sum counter.
    e. A LINE NUMBER clause must not be the only clause specified.

f. An entry that contains a VALUE clause must also have a COLUMN NUMBER clause.

11. Table 8–1 shows all permissible clause combinations for a Format 3 entry. The table is read from left to right along the selected row.

    An "M" indicates that the presence of the clause is mandatory.

    A "P" indicates that the presence of the clause is permitted, but not required.

    A blank indicates that the clause is not permitted.

Table 8–1. Permissible Clause Combinations in Format 3 Entries

PIC	COLUMN	SOURCE	SUM	VALUE	JUST	BLANK WHEN ZERO	GROUP INDICATE	USAGE	LINE
M			M						P
M	M		M			P		P	P
M	P	M			P		P	P	P
M	P	M				P	P	P	P
M	M			M	P		P	P	P

8.2.5.4. General Rules

1. Format 1 is the report group entry. The report group is defined by the contents of this entry and all of its subordinate entries.

8.2.6. The BLOCK CONTAINS Clause

8.2.6.1. Function

The BLOCK CONTAINS clause specifies the size of a physical record.

8.2.6.2. General Format

BLOCK CONTAINS [*integer-1* TO] *integer-2* { RECORDS / CHARACTERS }

8.2.6.3. General Rules

1. This clause is required except when:

   a. A physical record contains one and only one complete logical record.

   b. The hardware device assigned is a printer file.

2. The size of the physical record may be stated in terms of RECORDS, unless one of the following situations exists, in which case the RECORDS phrase must not be used.

   a. In mass storage files, where logical records may extend across physical records.

   b. The physical record contains padding (area not contained in a logical record).

c. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.

3. When the word CHARACTERS is specified, the physical record size is specified in terms of the number of character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
4. If only *integer-2* is shown, it represents the exact size of the physical record. If *integer-1* and *integer-2* are both shown, they refer to the minimum and maximum size of the physical record, respectively.
5. If logical records of differing size are grouped into one physical record, the technique for determining the size of each logical record is specified by the implementor.

8.2.7. The CODE Clause

8.2.7.1. Function

The CODE clause specifies a 2-character literal that identifies each print line as belonging to a specific report.

8.2.7.2. General Format

```
CODE literal-1
```

8.2.7.3. Syntax Rules

1. *Literal-1* is a 2-character nonnumeric literal.
2. If the CODE clause is specified for any report in a file, then it must be specified for all reports in the same file.

8.2.7.4. General Rules

1. When the CODE clause is specified, *literal-1* is automatically placed in the first two character positions of each Report Writer logical record.
2. The positions occupied by *literal-1* are not included in the description of the print line, but are included in the logical record size.
3. The clause is ignored if the report file is assigned to a printer file.

8.2.8. The CODE-SET Clause

8.2.8.1. Function

The CODE-SET clause specifies the character code set used to represent data on the external media.

8.2.8.2. General Format

```
CODE-SET IS alphabet-name
```

8.2.8.3. Syntax Rules

1. When the CODE-SET clause is specified for a file, all data in that file must be described as usage is DISPLAY and any signed numeric data must be described with the SIGN IS SEPARATE clause.
2. The *alphabet-name* clause referenced by the CODE-SET clause must not specify the literal phrase.
3. The CODE-SET clause may only be specified for non-mass storage files.

8.2.8.4. General Rules

1. If the CODE-SET clause is specified, *alphabet-name* specifies the character code convention used to represent data on the external media. It also specifies the algorithm for converting the character codes on the external media from the native character codes. This code conversion occurs during the execution of an output operation. (See 2.4.1.3.)
2. If the CODE-SET clause is not specified, the native character code set is assumed for data on the external media.

8.2.9. The COLUMN NUMBER Clause

8.2.9.1. Function

The COLUMN NUMBER clause identifies a printable item and specifies the column number position of the item on a print line.

8.2.9.2. General Format

```
COLUMN NUMBER IS integer-1
```

8.2.9.3. Syntax Rules

1. The COLUMN NUMBER clause can only be specified at the elementary level within a report group. The COLUMN NUMBER clause, if present, must appear in or be subordinate to an entry that contains a LINE NUMBER clause.
2. Within a given print line, the printable items must be defined in ascending column number order such that each character defined occupies a unique position.

8.2.9.4. General Rules

1. The COLUMN NUMBER clause indicates that the object of a SOURCE clause or the object of a VALUE clause or the sum counter defined by a SUM clause is to be presented on the print line. The absence of a COLUMN NUMBER clause indicates that the entry is not to be presented on a print line.
2. *Integer-1* specifies the column number of the leftmost character position of the printable item.
3. The Report Writer Control System supplies space characters for all positions of a print line that are not occupied by printable items.
4. The first position of the print line is considered to be column number 1.

8.2.10. The CONTROL Clause

8.2.10.1. Function

The CONTROL clause establishes the levels of the control hierarchy for the report.

8.2.10.2. General Format

```
{ CONTROL IS  }  { data-name-1 [ , data-name-2 ] ...                    }
{ CONTROLS ARE}  { FINAL [ , data-name-1 [ , data-name-2 ] ... ]        }
```

8.2.10.3. Syntax Rules

1. *Data-name-1* and *data-name-2* must not be defined in the Report Section. *Data-name-1* and *data-name-2* may be qualified but must not be subscripted or indexed.
2. Each data-name must identify a different data item.
3. *Data-name-1*, *data-name-2*, ... must not have subordinate to it a data item whose size is variable as defined in the OCCURS clause (see 3.2.1).

8.2.10.4. General Rules

1. The data-names and the word FINAL specify the levels of the control hierarchy. FINAL, if specified, is the highest control, *data-name-1* is the major control, *data-name-2* is an intermediate control, etc. The last data-name specified is the minor control.
2. The execution of the chronologically first GENERATE statement for a given report causes the RWCS to save the values of all control data items associated with that report. On subsequent executions of all GENERATE statements for that report, control data items are tested by the RWCS for a change of value. A change of value in any control data item causes a control break to occur. The control break is associated with the highest level for which a change of value is noted. (See 8.3.1.)
3. The Report Writer Control System tests for a control break by comparing the contents of each control data item with the prior contents saved from the execution of the previous GENERATE statement for the same report. The RWCS applies the inequality relation condition as follows:
   a. If the control data item is a numeric data item, the relation test is for the comparison of two numeric operands.
   b. If the control data item is an index data item, the relation test is for the comparison of two index data items.
   c. If the control data item is a data item other than as described in rules 3a and 3b, the relation test is for the comparison of two nonnumeric operands. (See 2.4.1.2.)
4. FINAL is used when the most inclusive control group in the report is not associated with a control data-name.

8.2.11. The Data–Name Clause

8.2.11.1. Function

A data-name specifies the name of the data being described.

8.2.11.2. General Format

data-name

8.2.11.3. Syntax Rule

In the Report Section, a *data-name* need not appear in a data description entry and FILLER must not be used.

8.2.11.4. General Rules

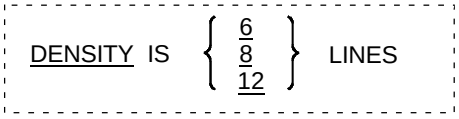
1. In the Report Section, *data–name* must be given in the following cases:
   a. When the *data-name* represents a report group to be referred to by a GENERATE or a USE statement in the Procedure Division.
   b. When reference is to be made to the sum counter in the Procedure Division or Report Section.
   c. When a DETAIL report group is referenced in the UPON phrase of the SUM clause.
   d. When the *data-name* is required to provide sum counter qualification.

8.2.12. The DENSITY Clause

8.2.12.1. Function

The DENSITY clause provides the capability of specifying either 6, 8, or 12 lines per inch on a printer file.

8.2.12.2. General Format



8.2.12.3. Syntax Rule

The DENSITY clause is only meaningful for printer files.

8.2.12.4. General Rule

Depending on the density chosen, the printer will print 6, 8 or 12 lines per inch. (See 8.3.2.)

8.2.13. The GROUP INDICATE Clause

8.2.13.1. Function

The GROUP INDICATE clause specifies that the associated printable item is presented only on the first occurrence of its report group after a control break or page advance.

8.2.13.2. General Format

```
GROUP INDICATE
```

8.2.13.3. Syntax Rule

The GROUP INDICATE clause may only appear in a DETAIL report group entry that defines a printable item.

8.2.13.4. General Rules

1. If a GROUP INDICATE clause is specified, it causes the SOURCE or VALUE clause to be ignored and spaces supplied, except:
   a. On the first presentation of the DETAIL report group in the report, or
   b. On the first presentation of the DETAIL report group after every page advance, or
   c. On the first presentation of the DETAIL report group after every control break.
2. If the Report Description entry specifies neither a PAGE clause nor a CONTROL clause, then a GROUP INDICATE printable item is presented the first time its DETAIL is presented after the INITIATE statement is executed. Thereafter spaces are supplied for indicated items with SOURCE or VALUE clauses.

8.2.14. The LABEL RECORDS Clause

8.2.14.1. Function

The LABEL RECORDS clause specifies whether labels are present.

8.2.14.1.1. General Format

```
LABEL { RECORD IS  } { STANDARD }
      { RECORDS ARE} { OMITTED  }
```

8.2.14.2. Syntax Rule

This clause is required in every File Description entry.

8.2.14.3. General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that labels exist for the file or the device to which the file is assigned and the labels conform to the implementor's label specifications. See Appendix E.

8.2.15. The LINE NUMBER Clause

8.2.15.1. Function

The LINE NUMBER clause specifies vertical positioning information for its report group.

8.2.15.2. General Format

```
LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] }
               { PLUS integer-2             }
```

8.2.15.3. Syntax Rules

1. *Integer-1* and *integer-2* must not exceed three significant digits in length.

   Neither *integer-1* nor *integer-2* may be specified in such a way as to cause any line of a report group to be presented outside of the vertical subdivision of the page designated for the report group type, as defined by the PAGE clause. (See 8.2.16.)

   Integer-2 may be zero.

2. Within a given report group description entry, an entry that contains a LINE NUMBER clause must not contain a subordinate entry that also contains a LINE NUMBER clause.
3. Within a given report group description entry, all absolute LINE NUMBER clauses must precede all relative LINE NUMBER clauses.
4. Within a given report group description entry, successive absolute LINE NUMBER clauses must specify integers that are in ascending order. The integers need not be consecutive.
5. If the PAGE clause is omitted from a given report group description entry, only relative LINE NUMBER clauses can be specified in any report group description entry within that report.
6. Within a given report group description entry a NEXT PAGE phrase can appear only once and, if present, must be in the first LINE NUMBER clause in that report group description entry.

   A LINE NUMBER clause with the NEXT PAGE phrase can appear only in the description of body groups and in a REPORT FOOTING report group.

7. Every entry that defines a printable item (see 8.2.8) must either contain a LINE NUMBER clause, or be subordinate to an entry that contains a LINE NUMBER clause.
8. The first LINE NUMBER clause specified within a PAGE FOOTING report group must be an absolute LINE NUMBER clause.

8.2.15.4. General Rules

1. A LINE NUMBER clause must be specified to establish each print line of a report group.
2. The RWCS effects the vertical positioning specified by a LINE NUMBER clause, before presenting the print line established by that LINE NUMBER clause.

3. *Integer-1* specifies an absolute line number. An absolute line number specifies the line number on which the print line is presented.

4. *Integer-2* specifies a relative line number. If a relative LINE NUMBER clause is not the first LINE NUMBER clause in the Report Group Description entry, then the line number on which its print line is presented is determined by calculating the sum of the line number on which the previous print line of the report group was presented and *integer-2* of the relative LINE NUMBER clause.

If *integer-2* is zero, the line is printed on the same line as the previous print line.

If a relative LINE NUMBER clause is the first LINE NUMBER clause in the Report Groups Description entry and a PAGE clause is specified, then the line number on which its print line is presented is determined as follows:

a. REPORT HEADING

The first print line of the REPORT HEADING report group is presented on the line number obtained by adding the integer of the first LINE NUMBER clause and the value obtained by subtracting one from the value of *integer-2* of the HEADING phrase of the PAGE clause.

b. PAGE HEADING

- If a REPORT HEADING report group has been presented on the page on which the PAGE HEADING report group is to be presented, then the PAGE HEADING is presented relative to the final LINE–COUNTER setting of the REPORT HEADING.
- If no REPORT HEADING has been presented on the page, the sum of the integer of the first LINE NUMBER clause of the PAGE HEADING report group and the value obtained by subtracting one from the value of *integer-2* of the HEADING phrase of the PAGE clause defines the line number on which the first print line of the PAGE HEADING report group is presented.

c. Body Group (DETAIL, CONTROL HEADING, or CONTROL FOOTING)

- If the value in LINE–COUNTER is less than the line number specified by the FIRST DETAIL phrase of the PAGE clause, then the first print line of the body group is presented on the line specified by the FIRST DETAIL phrase.
- If the value in LINE–COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause, and if no body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line immediately following the line indicated by the value contained in LINE–COUNTER.
- If the value in LINE–COUNTER is equal to or greater than the line number specified by the FIRST DETAIL phrase of the PAGE clause, and if a body group has previously been presented on the page to which the report is currently positioned, then the first print line of the current body group is presented on the line that is obtained by adding the contents of LINE– COUNTER and the integer of the first LINE NUMBER clause of the current body group.

d. REPORT FOOTING

- If a PAGE FOOTING report group has been presented on the page to which the report is currently positioned, then the sum of the final LINE–COUNTER setting established by the PAGE FOOTING report group and the integer of the first LINE NUMBER clause of the REPORT FOOTING report group defines the line number on which the first print line of the REPORT FOOTING report group is presented.
- If no PAGE FOOTING group has been presented, the sum of the integer of the first LINE NUMBER clause of the REPORT FOOTING report group, and the line number specified by the value of *integer-5* of the FOOTING phrase of the PAGE clause defines the line number on which the first print line of the REPORT FOOTING report group is presented.

5. The NEXT PAGE phrase specifies that the report group is to be presented beginning on the indicated line number on a new page. The use of this phrase in the first LINE NUMBER clause of a REPORT FOOTING report group indicates the REPORT FOOTING report group is to be presented on a page on which no other report group has been presented.

8.2.16. The NEXT GROUP Clause

8.2.16.1. Function

The NEXT GROUP clause specifies information for vertical positioning of a page following the presentation of the last line of a report group.

8.2.16.2. General Format



8.2.16.3. Syntax Rules

1. A report group entry must not contain a NEXT GROUP clause unless the description of that report group contains at least one LINE NUMBER clause.
2. *Integer-1* and *integer-2* must not exceed three significant digits in length.
3. If the PAGE clause is omitted from the report description entry only a relative NEXT GROUP clause may be specified in any report group description entry within that report.
4. The NEXT PAGE phrase of the NEXT GROUP clause must not be specified in a PAGE FOOTING report group.
5. The NEXT GROUP clause must not be specified in a REPORT FOOTING report group or in a PAGE HEADING report group.

8.2.16.4. General Rules

1. Any positioning of the page specified by the NEXT GROUP clause takes place after the presentation of the report group in which the clause appears. (See 8.2.14, general rule 4.)
2. The vertical positioning information supplied by the NEXT GROUP clause is interpreted by the RWCS along with information from the TYPE and PAGE clauses, and the value in LINE–COUNTER, to determine a new value for LINE–COUNTER. (See 8.2.14, general rule 4.)
3. The NEXT GROUP clause is ignored by the RWCS when it is specified on a CONTROL FOOTING report group that is at a level other than the highest level at which a control break is detected.
4. The NEXT GROUP clause of a body group refers to the next body group to be presented, and therefore can affect the location at which the next body group is presented. The NEXT GROUP clause of a REPORT HEADING report group can affect the location at which the PAGE HEADING report group is presented. The NEXT GROUP clause of a PAGE FOOTING report group can affect the location at which the REPORT FOOTING report group is presented. (See 8.2.14, general rule 4.)

8.2.17. The PAGE Clause

8.2.17.1. Function

The PAGE clause defines the length of a page and the vertical subdivisions within which report groups are presented.

8.2.17.2. General Format

```
PAGE [ LIMIT IS  ]   integer-1 [ LINE  ]
     [ LIMITS ARE ]            [ LINES ]

[ , HEADING integer-2 ]   [ , FIRST DETAIL integer-3 ]

[ , LAST DETAIL integer-4 ]   [ , FOOTING integer-5 ]
```

8.2.17.3. Syntax Rules

1. The HEADING, FIRST DETAIL, LAST DETAIL, and FOOTING phrases may be written in any order.
2. *Integer-1* must not exceed three significant digits in length.
3. *Integer-2* must be greater than or equal to one.
4. *Integer-3* must be greater than or equal to *integer-2*.
5. *Integer-4* must be greater than or equal to *integer-3*.
6. *Integer-5* must be greater than or equal to *integer-4*.
7. *Integer-1* must be greater than or equal to *integer-5*.

8. The following rules indicate the vertical subdivision of the page in which each TYPE of report group may appear when the PAGE clause is specified. (See 8.2.16.5.)

    a. A REPORT HEADING report group that is to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-1*, inclusive.

    A REPORT HEADING report group that is not to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-3* minus 1, inclusive.

    b. A PAGE HEADING report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-3* minus 1, inclusive.

    c. A CONTROL HEADING or DETAIL report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-3* to the line number specified by *integer-4*, inclusive.

    d. A CONTROL FOOTING report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer–3* to the line number specified by *integer-5*, inclusive.

    e. A PAGE FOOTING report group, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-5* plus 1 to the line number specified by *integer-1* inclusive.

    f. A REPORT FOOTING report group that is to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-2* to the line number specified by *integer-1*, inclusive.

    A REPORT FOOTING report group that is not to be presented on a page by itself, if defined, must be defined such that it can be presented in the vertical subdivision of the page that extends from the line number specified by *integer-5* plus 1 to the line number specified by *integer-1*, inclusive.

9. All report groups must be described such that they can be presented on one page. The RWCS never splits a multiline report group across page boundaries.

8.2.17.4. General Rules

1. The vertical format of a report page is established using the integer values specified in the PAGE clause.

    a. *Integer-1* defines the size of a report page by specifying the number of lines available on each page.

    b. HEADING *integer-2* defines the first line number on which a REPORT HEADING or PAGE HEADING report group may be presented.

c. FIRST DETAIL *integer-3* defines the first line number on which a body group may be presented. REPORT HEADING and PAGE HEADING report groups may not be presented on or beyond the line number specified by *integer-3*.

d. LAST DETAIL *integer-4* defines the last line number on which a CONTROL HEADING or DETAIL report group may be presented.

e. FOOTING *integer-5* defines the last line number on which a CONTROL FOOTING report group may be presented. PAGE FOOTING and REPORT FOOTING report groups must follow the line number specified by *integer-5*.

2. If the PAGE clause is specified, the following implicit values are assumed for any omitted phrases:

   a. If the HEADING phrase is omitted, a value of one is assumed for *integer-2*.

   b. If the FIRST DETAIL phrase is omitted, a value equal to *integer-2* (HEADING) is given to *integer-3*.

   c. If the LAST DETAIL and the FOOTING phrases are both omitted, the value of *integer-1* (PAGE LIMIT) is given to both *integer-4* and *integer-5*.

   d. If the FOOTING phrase is specified and the LAST DETAIL phrase is omitted, the value of *integer-5* (FOOTING) is given to *integer-4*.

   e. If the LAST DETAIL phrase is specified and the FOOTING phrase is omitted, the value of *integer-4* (LAST DETAIL) is given to *integer-5*.

3. If the PAGE clause is omitted, the report consists of a single page of indefinite length.

8.2.17.5. PAGE Regions Table

The regions that are established by the PAGE clause are described below:

Report Groups That May Be Presented in the Region	First Line Number of the Region	Last Line Number of the Region	Relative Line Positioning for the First Report Group Within a Region*
REPORT HEADING described with NEXT GROUP NEXT PAGE	*integer-2* (HEADING)	*integer-1* (PAGE LIMIT)	LINE NUMBER plus *integer-2* (HEADING) minus 1
REPORT FOOTING described with LINE *integer-1* NEXT PAGE			
REPORT HEADING not described with NEXT GROUP NEXT PAGE PAGE HEADING	*integer-2* (HEADING)	*integer-3* minus 1 (FIRST DETAIL minus 1)	LINE NUMBER plus *integer-2* (HEADING) minus 1
CONTROL HEADING DETAIL	*integer-3* FIRST DETAIL	*integer-4* LAST DETAIL	If LINE-COUNTER ≥ *integer-3* (FIRST DETAIL), Position on LINE-COUNTER plus 1 If LINE-COUNTER < *integer-3* (FIRST DETAIL), Position on *integer-3* (FIRST DETAIL).
CONTROL FOOTING	*integer-3* FIRST DETAIL	*integer-5* (FOOTING)	Same as above.
PAGE FOOTING	*integer-5* plus 1 (FOOTING PLUS ONE)	*integer-1* (PAGE LIMIT)	
REPORT FOOTING not described with LINE *integer-1* NEXT PAGE			LINE NUMBER PLUS *integer-5* (FOOTING)

***** The last column is a Unisys clarification.

8.2.18. The RECORD CONTAINS Clause

8.2.18.1. Function

The RECORD CONTAINS clause specifies the size of data records.

8.2.18.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

8.2.18.3. General Rules

The size of each data record is completely defined within the record description entry, therefore this clause is never required. When present, however, the following notes apply:

1. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case *integer-2* represents the exact number of characters in the data record. If the *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.
2. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. (See 1.3.3.3.4, 2.5.11, and 2.5.12

8.2.19. The REPORT Clause

8.2.19.1. Function

The REPORT clause specifies the names of reports that comprise a report file.

8.2.19.2. General Format

```
{ REPORT IS  }
{ REPORTS ARE } report-name-1 [ , report-name-2 ] ... .
```

8.2.19.3. Syntax Rules

1. Each *report-name* specified in a REPORT clause must be the subject of a report description entry in the Report Section. The order of appearance of the *report-names* is not significant.
2. A *report-name* must appear in only one REPORT clause.
3. The subject of a file description entry that specifies a REPORT clause may only be referred to by the OPEN OUTPUT, OPEN EXTEND, and CLOSE statements.

8.2.19.4. General Rules

1. The presence of more than one report-name in a REPORT clause indicates that the file contains more than one report.

8.2.20. The SOURCE Clause

8.2.20.1. Function

The SOURCE clause identifies the sending data item that is moved to an associated printable item defined within a report group description entry.

8.2.20.2. General Format

```
SOURCE IS identifier-1
```

8.2.20.3. Syntax Rules

1. *Identifier-1* may be defined in any section of the Data Division. If *identifier-1* is a Report Section item, it can only be:
   a. PAGE–COUNTER, or
   b. LINE–COUNTER, or
   c. A sum counter of the report within which the SOURCE clause appears.
2. *Identifier-1* specifies the sending data item of the implicit MOVE statement that the RWCS will execute to move *identifier-1* to the printable item. *Identifier-1* must be defined such that it conforms to the rules for sending items in the MOVE statement. (See 2.6.14.)

8.2.20.4. General Rule

The RWCS formats the print lines of a report group just prior to presenting the report group. (See 8.2.21.) It is at this time that the implicit MOVE statements specified by SOURCE clauses are executed by the RWCS.

8.2.21. The SUM Clause

8.2.21.1. Function

The SUM clause establishes a sum counter and names the data items to be summed.

8.2.21.2. General Format

```
{ SUM identifier-1  [,identifier-2]...

        [ UPON data-name-1 [,data-name-2 ]...] }...

[ RESET ON { data-name-3 } ]
           { FINAL       }
```

8.2.21.3. Syntax Rules

1. *Identifier-1* and *identifier-2* must be defined as numeric data items. When defined in the Report Section, *identifier-1* and *identifier-2* must be the names of sum counters.

   If the UPON phrase is omitted, any identifiers in the associated SUM clause which are themselves sum counters must be defined either in the same report group that contains this SUM clause or in a report group which is at a lower level in the control hierarchy of this report.

   If the UPON phrase is specified, any identifiers in the associated SUM clause must not be sum counters.

2. *Data-name-1* and *data-name-2* must be the names of DETAIL report groups described in the same report as the CONTROL FOOTING report group in which the SUM clause appears. *Data-name-1* and *data-name-2* may be qualified by a report-name.

3. A SUM clause can appear only in the description of a CONTROL FOOTING report group.

4. *Data-name-3* must be one of the data-names specified in the CONTROL clause for this report. *Data-name-3* must not be a lower level control than the associated control for the report group in which the RESET phrase appears.

   FINAL, if specified in the RESET phrase, must also appear in the CONTROL clause for this report.

5. The highest permissible qualifier of a sum counter is the report-name.

8.2.21.4. General Rules

1. The SUM clause establishes a sum counter. The sum counter is a numeric data item with an optional sign. At object time the RWCS adds directly into the sum counter each of the values contained in *identifier-1* and *identifier-2*. This addition is performed under the rules of the ADD statement. (See 2.6.5.)

2. The printable size of the sum counter is equal to the number of receiving character positions specified by the PICTURE clause that accompanies the SUM clause in the description of the elementary item. Editing of a sum counter occurs only upon the presentation of that sum counter. At all other times the sum counter is treated as a numeric data-item. Sum counters are implicitly described as binary items with a decimal scale equal to the PICTURE clause that accompanies the SUM clause. The binary item is a 36-bit field unless the PICTURE clause that accompanies the SUM clause is greater than ten digits, in which case a signed, 72-bit field is used.

3. Only one sum counter exists for an elementary report entry regardless of the number of SUM clauses specified in the elementary report entry.

4. If the elementary report entry for a printable item contains a SUM clause, the sum counter serves as a source data item. The RWCS moves the data contained in the sum counter, according to the rules of the MOVE statement, to the printable item for presentation.

5. If a data-name appears as the subject of an elementary report entry that contains a SUM clause, the data-name is the name of the sum counter. The data-name is not the name of the printable item that the entry may also define.

   It is permissible for Procedure Division statements to alter the contents of sum counters.

6. Addition of the identifiers into sum counters is performed by the RWCS during the execution of GENERATE and TERMINATE statements. There are three categories of sum counter incrementing called subtotaling, crossfooting, and rolling forward. Subtotaling is accomplished during execution of GENERATE statements only, after any control break processing but before processing of the DETAIL report group (See 8.3.1). Crossfooting and rolling forward are accomplished during the processing of CONTROL FOOTING report groups. (See 8.2.21.)

7. The UPON phrase provides the capability to accomplish selective subtotaling for the DETAIL report groups named in the phrase.

8. The RWCS adds each individual addend into the sum counter at a time that depends upon the characteristics of the addend.

   a. When the addend is a sum counter defined in the same CONTROL FOOTING report group, then the accumulation of that addend into the sum counter is termed crossfooting.

      Crossfooting occurs when a control break takes place and at the time the CONTROL FOOTING report group is processed.

      Crossfooting is performed according to the sequence in which sum counters are defined within the CONTROL FOOTING report group. That is, all crossfooting into the first sum counter defined in the CONTROL FOOTING report group is completed, and then all crossfooting into the second sum counter defined in the CONTROL FOOTING report group is completed. Crossfooting into each sum counter is performed in the order, left to right, of the physical appearance of the crossfooted addends. This procedure is repeated until all crossfooting operations are completed.

   b. When the addend is a sum counter defined in a lower level CONTROL FOOTING report group, then the accumulation of that addend into the sum counter is termed rolling forward. A sum counter in a lower level CONTROL FOOTING report group is rolled forward when a control break occurs and at the time that the lower level CONTROL FOOTING report group is processed.

   c. When the addend is not a sum counter, the accumulation into a sum counter of such an addend is called subtotaling. If the SUM clause contains the UPON phrase, the addends are subtotaled when a GENERATE statement for the designated DETAIL report group is executed. If the SUM clause does not contain the UPON phrase, the addends which are not sum counters are subtotaled when any GENERATE data-name statement is executed for the report in which the SUM clause appears.

9. If two or more of the identifiers specify the same addend, then the addend is added into the sum counter as many times as the addend is referenced in the SUM clause. It is permissible for two or more of the data-names to specify the same DETAIL report group. When a GENERATE data-name statement for such a DETAIL report group is given, the incrementing occurs repeatedly, as many times as data-name appears in the UPON phrase.

10. For the subtotaling that occurs when a GENERATE report-name statement is executed, see 8.3.1.

11. In the absence of an explicit RESET phrase, the RWCS will set a sum counter to zero at the time that the RWCS is processing the CONTROL FOOTING report group within which the sum counter is defined. If an explicit RESET phrase is specified, then the RWCS will set the sum counter to zero at the time that the RWCS is processing the designated level of the control hierarchy. (See 8.2.21.)

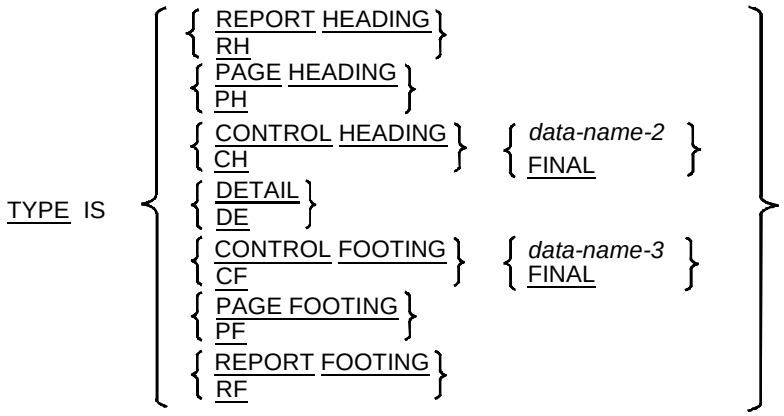
    Sum counters are initially set to zero by the RWCS during the execution of the INITIATE statement for the report containing the sum counter.

8.2.22. The TYPE Clause

8.2.22.1. Function

The TYPE clause specifies the particular type of report group that is described by this entry and indicates the time at which the report group is to be processed by the Report Writer Control System.

8.2.22.2. General Format



8.2.22.3. Syntax Rules

1. RH is an abbreviation for REPORT HEADING.

   PH is an abbreviation for PAGE HEADING.

   CH is an abbreviation for CONTROL HEADING.

   DE is an abbreviation for DETAIL.

   CF is an abbreviation for CONTROL FOOTING.

   PF is an abbreviation for PAGE FOOTING.

   RF is an abbreviation for REPORT FOOTING.

2. REPORT HEADING, PAGE HEADING, CONTROL HEADING FINAL, CONTROL FOOTING FINAL, PAGE FOOTING, and REPORT FOOTING report groups may each appear no more than once in the description of a report.

3. PAGE HEADING and PAGE FOOTING report groups may be specified only if a PAGE clause is specified in the corresponding report description entry.
4. *Data-name-1*, *data-name-2*, and FINAL, if present, must be specified in the CONTROL clause of the corresponding report description entry. At most, one CONTROL HEADING report group and one CONTROL FOOTING report group can be specified for each data-name or FINAL in the CONTROL clause of the report description entry. However, neither a CONTROL HEADING report group nor a CONTROL FOOTING report group is required for a data-name or FINAL specified in the CONTROL clause of the report description entry.
5. In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups, SOURCE clauses and USE statements must not reference any of the following:

   a. Group data items containing a control data item.

   b. Data items subordinate to a control data item.

   c. A redefinition or renaming of any part of a control data item.

   In PAGE HEADING and PAGE FOOTING report groups, SOURCE clauses and USE statements must not reference control data–names.
6. When a GENERATE *report-name* statement is specified in the Procedure Division, the corresponding report description entry must include no more than one DETAIL report group. If no GENERATE *data-name* statements are specified for such a report, a DETAIL report group is not required.
7. The description of a report must include at least one body group.

8.2.22.4. General Rules

1. DETAIL report groups are processed by the RWCS as a direct result of GENERATE statements. If a report group is other than TYPE DETAIL, its processing is an automatic RWCS function.
2. The REPORT HEADING phrase specifies a report group that is processed by the RWCS only once, per report, as the first report group of that report. The REPORT HEADING report group is processed during the execution of the chronologically first GENERATE statement for that report.
3. The PAGE HEADING phrase specifies a report group that is processed by the RWCS as the first report group on each page of that report except under the following conditions:

   a. A PAGE HEADING report group is not processed on a page that is to contain only a REPORT HEADING report group or only a REPORT FOOTING report group.

   b. A PAGE HEADING report group is processed as the second report group on a page when it is preceded by a REPORT HEADING report group that is not to be presented on a page by itself. (See 8.2.14, general rule 4.)

4. The CONTROL HEADING phrase specifies a report group that is processed by the RWCS at the beginning of a control group for a designated control data-name or, in the case of FINAL, is processed during the execution of the chronologically first GENERATE statement for that report. During the execution of any GENERATE statement at which the RWCS detects a control break, any CONTROL HEADING report groups associated with the highest control level of the break and lower levels are processed.
5. The DETAIL phrase specifies a report group that is processed by the RWCS when a corresponding GENERATE statement is executed.
6. The CONTROL FOOTING phrase specifies a report group that is processed by the RWCS at the end of a control group for a designated control data-name.

   In the case of FINAL, the CONTROL FOOTING report group is processed only once per report as the last body group of that report. During the execution of any GENERATE statement in which the RWCS detects a control break, any CONTROL FOOTING report group associated with the highest level of the control break or more minor levels is presented. All CONTROL FOOTING report groups are presented during the execution of the TERMINATE statement if there has been at least one GENERATE statement executed for the report. (See 8.3.4.)
7. The PAGE FOOTING phrase specifies a report group that is processed by the RWCS as the last report group on each page except under the following conditions:
   a. A PAGE FOOTING report group is not processed on a page that is to contain only a REPORT HEADING report group or only a REPORT FOOTING report group.
   b. A PAGE FOOTING report group is processed as the second to last report group on a page when it is followed by a REPORT FOOTING report group that is not to be processed on a page by itself. (See 8.2.14, general rule 4.)
8. The REPORT FOOTING phrase specifies a report group that is processed by the RWCS only once per report and as the last report group of that report. The REPORT FOOTING report group is processed during the execution of a corresponding TERMINATE statement, if there has been at least one GENERATE statement executed for the report. (See 8.3.4.)
9. The sequence of steps that the RWCS executes when it processes a REPORT HEADING, PAGE HEADING, CONTROL HEADING, PAGE FOOTING, or REPORT FOOTING report group is described below.
   a. If there is a USE BEFORE REPORTING procedure that references the data-name of the report group, the USE procedure is executed.
   b. If a SUPPRESS statement has been executed or if the report group is not printable, there is no further processing to be done for the report group.
   c. Otherwise, the RWCS formats the print lines and presents the report group according to the presentation rules for that type of report group. (See 8.2.14, general rule 4.)

10. The sequence of steps that the RWCS executes when it processes a CONTROL FOOTING report group is described below.

    The GENERATE rules specify that when a control break occurs, the RWCS produces the CONTROL FOOTING report groups beginning at the minor level, and proceeding upwards, through the level at which the highest control break was sensed. In this regard, it should be noted that even though no CONTROL FOOTING report group has been defined for a given control data-name, the RWCS will still have to execute the step described in paragraph 10f if a RESET phrase within the report description specifies that control data-name.

    a. Sum counters are crossfooted, in other words, all sum counters defined in this report group that are operands of SUM clauses in the same report group are added to their sum counters. (See 8.2.20.)

    b. Sum counters are rolled forward, in other words, all sum counters defined in the report group that are operands of SUM clauses in higher level CONTROL FOOTING report groups are added to the higher level sum counters. (See 8.2.20.)

    c. If there is a USE BEFORE REPORTING procedure that references the data-name of the report group, the USE procedure is executed.

    d. If a SUPPRESS statement has been executed or if the report group is not printable, the RWCS next executes the step described in paragraph 10f.

    e. Otherwise the RWCS formats the print lines and presents the report group according to the presentation rules for CONTROL FOOTING report groups.

    f. Then the RWCS resets those sum counters that are to be reset when the RWCS processes this level in the control hierarchy. (See 8.2.20.)

11. The DETAIL report group processing that the RWCS executes in response to a GENERATE *data-name* statement is described in paragraphs 11a through 11e.

    When the description of a report includes exactly one DETAIL report group, the detail-related processing that the RWCS executes in response to a GENERATE *report-name* statement is described in paragraphs 11a through 11d. These steps are performed as though a GENERATE *data-name* statement were being executed.

    When the description of a report includes no DETAIL report groups, the detail-related processing that the RWCS executes in response to a GENERATE *report-name* statement is described in paragraph 11a. This step is performed as though the description of the report included exactly one DETAIL report group, and a GENERATE *data-name* statement were being executed.

    a. The RWCS performs any subtotaling that has been designated for the DETAIL report group. (See 8.2.20.)

    b. If there is a USE BEFORE REPORTING procedure that refers to the data–name of the report-group, the USE procedure is executed.

    c. If a SUPPRESS statement has been executed or if the report group is not printable, there is no further processing done for the report group.

    d. If the DETAIL report group is being processed as a consequence of a GENERATE report-name statement, there is no further processing done for the report group.

e. Otherwise, the RWCS formats the print lines and presents the report group according to the presentation rules for DETAIL report groups. (See 8.2.14, general rule 4.)

12. When the RWCS is processing a CONTROL HEADING, CONTROL FOOTING, or DETAIL report group, as described in general rules 9, 10, and 11, the RWCS may have to interrupt the processing of that body group after determining that the body group is to be presented, and execute a page advance (and process PAGE FOOTING and PAGE HEADING report groups) before actually presenting the body group.
13. During control break processing, the values of control data items that the RWCS used to detect a given control break are referred to as prior values.
    a. During control break processing of a CONTROL FOOTING report group, any references to control data items in a USE procedure or SOURCE clause associated with that CONTROL FOOTING report group are supplied with prior values.
    b. When a TERMINATE statement is executed, the RWCS makes the prior control data item values available to SOURCE clause or USE procedure references in CONTROL FOOTING and REPORT FOOTING report groups as though a control break had been detected in the highest control data-name.
    c. All other data item references within report groups and their USE procedures access the current values that are contained within the data items at the time the report group is processed.

8.2.23. The VALUE OF Clause

8.2.23.1. Function

The VALUE OF clause particularizes the description of an item in the label records associated with a file.

8.2.23.2. General Format

```
VALUE OF implementor-name-1 IS { data-name-1 }
                               { literal-1   }

[ , implementor-name-2 IS { data-name-2 } ] ...
                          { literal-2   }
```

8.2.23.3. Syntax Rules

1. *Data-name-1*, *data-name-2*, etc., should be qualified when necessary, but cannot be subscripted or indexed, nor can they be items described with the USAGE IS INDEX clause.
2. *Data-name-1*, *data-name-2*, etc., must be in the Working–Storage Section.
3. *Data-name-1*, *data-name-2*, etc., must be described as USAGE DISPLAY.
4. See 4.3.11 for constraints that apply when Report Writer is associated with Sequential I–O, Level 1.

8.2.23.4. General Rules

1. For an output file, at the appropriate time the value of *implementor-name-1* is made equal to the value of *literal-1*, or of *data-name-1*, whichever has been specified. (See 4.4.2 and Appendix E.)
2. A figurative constant may be substituted in the format above wherever a literal is specified.

8.3. Procedure Division

8.3.1. The GENERATE Statement

8.3.1.1. Function

The GENERATE statement directs the RWCS to produce a report in accordance with the report description that was specified in the Report Section of the Data Division.

8.3.1.2. General Format

GENERATE { *data-name* / *report-name* }

8.3.1.3. Syntax Rules

1. *Data-name* must name a TYPE DETAIL report group and may be qualified by a report-name.
2. *Report-name* may be used only if the referenced report description contains:
   a. A CONTROL clause,
   b. Not more than one DETAIL report group, and
   c. At least one body group.

8.3.1.4. General Rules

1. In response to a GENERATE *report-name* statement, the RWCS performs summary processing. If all of the GENERATE statements that are executed for a report are of the form GENERATE *report-name*, then the report that is produced is called a summary report. A summary report is one in which no DETAIL report group is presented.
2. In response to a GENERATE *data-name* statement, the RWCS performs detail processing that includes certain processing that is specific for the DETAIL report group designated by the GENERATE statement. Normally, the execution of a GENERATE *data-name* statement causes the RWCS to present the designated DETAIL report group.
3. During the execution of the chronologically first GENERATE statement for a given report, the RWCS saves the values within the control data items. During the execution of the second and subsequent GENERATE statements for the same report, and until a control break is detected, the RWCS utilizes this set of control values to determine whether a control break has occurred. When a control break occurs, the RWCS saves the new set of control values, which it thereafter uses to sense for a control break until another control break occurs.
4. During report presentation, an automatic function of the RWCS is to process PAGE HEADING and PAGE FOOTING report groups, if defined, when the RWCS must advance the report to a new page for the purpose of presenting a body group. (See 8.2.14, general rule 4.)

5. When the chronologically first GENERATE statement for a given report is executed, the RWCS processes, in order, the report groups that are named below, provided that such report groups are defined within the report description. The RWCS also processes PAGE HEADING and PAGE FOOTING report groups as described in general rule 4. See 8.2.21 for the actions that the RWCS takes when it processes each type of report group.

    a. The REPORT HEADING report group is processed.

    b. The PAGE HEADING report group is processed.

    c. All CONTROL HEADING report groups are processed from major to minor.

    d. If a GENERATE *data-name* statement is being executed, the processing for the designated DETAIL report group is performed. If a GENERATE *report-name* statement is being executed, certain of the steps that are involved in the processing of a DETAIL report group are performed. (See 8.2.21.)

6. When a GENERATE statement other than the chronologically first is executed for a given report, the RWCS performs the steps enumerated below, as applicable. The RWCS also processes PAGE HEADING and PAGE FOOTING report groups as described in general rule 4. See 8.2.21 for the actions that the RWCS takes when it processes each type of report group.

    a. Sense for control break. The rules for determining the equality of control data items are the same as those specified for relation conditions. If a control break has occurred then:

        - Enable the CONTROL FOOTING USE procedures and CONTROL FOOTING SOURCE clauses to access the control data item values that are described in 8.2.21.

        - Process the CONTROL FOOTING report groups in the order minor to major. Only CONTROL FOOTING report groups that are not more major than the highest level at which a control break occurred are processed.

        - Process the CONTROL HEADING report groups in the order major to minor. Only the CONTROL HEADING report groups that are not more major than the highest level at which a control break occurred are processed.

    b. If a GENERATE *data-name* statement is being executed, the processing for the designated DETAIL report group is performed. If a GENERATE *report-name* statement is being executed, certain of the steps that are involved in the processing of a DETAIL report group are performed. (See 8.2.21.)

7. GENERATE statements for a report can be executed only after an INITIATE statement for the report has been executed and before a TERMINATE statement for the report has been executed.

8.3.2. The INITIATE Statement

8.3.2.1. Function

The INITIATE statement causes the Report Writer Control System to begin the processing of a report.

8.3.2.2. General Format

```
INITIATE report-name-1 [ , report-name-2 ]  . . .
```

8.3.2.3. Syntax Rules

1. Each report-name must be defined by a report description entry in the Report Section of the Data Division.

8.3.2.4. General Rules

1. The INITIATE statement performs the following initialization functions for each named report:
   a. All sum counters are set to zero.
   b. LINE–COUNTER is set to zero.
   c. PAGE–COUNTER is set to one.
2. The INITIATE statement does not open the file with which the report is associated, therefore an OPEN statement with either the OUTPUT phrase or the EXTEND phrase for the file must be executed prior to the execution of the INITIATE statement.
3. A subsequent INITIATE statement for a particular report-name must not be executed unless an intervening TERMINATE statement has been executed for that report-name.
4. When the DENSITY clause is present, the execution of the INITIATE statement sets the density specified.
5. When the VALUE OF CARTRIDGE–ID clause is present, the execution of the INITIATE statement requests the cartridge specified. (See 18.10.5.)
6. When the VALUE OF FORMS–REQUEST clause is present for a report file, the execution of the INITIATE statement requests the form specified. (See 18.10.6.)
7. For printer files, execution of the INITIATE statement causes the page margins to be set from the PAGE LIMIT clause if specified. If the PAGE LIMIT clause has not been specified, execution of the INITIATE statement causes the report to be defined as a single page of indefinite length. In both cases, the printer is positioned to the first printable line on the first (in some cases, and only) page.

8.3.3. The SUPPRESS Statement

8.3.3.1. Function

The SUPPRESS statement causes the Report Writer Control System to inhibit the presentation of a report group.

8.3.3.2. General Format

```
SUPPRESS PRINTING
```

8.3.3.3. Syntax Rules

1. The SUPPRESS statement may only appear in a USE BEFORE REPORTING procedure.

8.3.3.4. General Rules

1. The SUPPRESS statement inhibits presentation only for the report group named in the USE procedure within which the SUPPRESS statement appears.
2. The SUPPRESS statement must be executed each time the presentation of the report group is to be inhibited.
3. When the SUPPRESS statement is executed, the RWCS is instructed to inhibit the processing of the following report group functions:
   a. The presentation of the print lines of the report group,
   b. The processing of all LINE clauses in the report group,
   c. The processing of the NEXT GROUP clause in the report group,
   d. The adjustment of LINE–COUNTER.

8.3.4. The TERMINATE Statement

8.3.4.1. Function

The TERMINATE statement causes the Report Writer Control System to complete the processing of the specified reports.

8.3.4.2. General Format

```
TERMINATE report-name-1 [ , report-name-2 ]  . . .
```

8.3.4.3. Syntax Rules

1. Each report-name given in a TERMINATE statement must be defined by an RD entry in the Report Section of the Data Division.

8.3.4.4. General Rules

1. The TERMINATE statement causes the RWCS to produce all the CONTROL FOOTING report groups beginning with the minor CONTROL FOOTING report group. Then the REPORT FOOTING report group is produced. The RWCS makes the prior set of control data item values available to the CONTROL FOOTING and REPORT FOOTING SOURCE clauses and USE procedures, as though a control break has been sensed in the most major control data-name.
2. If no GENERATE statements have been executed for a report during the interval between the execution of an INITIATE statement and a TERMINATE statement, for that report, the TERMINATE statement does not cause the RWCS to produce any report groups or perform any of the related processing.
3. During report presentation, the RWCS automatically processes PAGE HEADING and PAGE FOOTING report groups, if defined, when the RWCS must advance the report to a new page for the purpose of presenting a body group. (See 8.2.14, general rule 4.)
4. The TERMINATE statement cannot be executed for a report unless the TERMINATE statement was chronologically preceded by an INITIATE statement for that report and for which no TERMINATE statement has yet been executed.
5. The TERMINATE statement does not close the file with which the report is associated; a CLOSE statement for the file must be executed. Every report in a file that is in an initiated condition must be terminated before a CLOSE statement is executed for that file.

8.3.5. The USE Statement

8.3.5.1. Function

The USE sentence specifies Procedure Division statements that are executed just before a report group named in the Report Section of the Data Division is produced.

8.3.5.2. General Format

```
USE BEFORE REPORTING identifier.
```

8.3.5.3. Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives section and must be followed by a period followed by a space. The remainder of the section must consist of zero, one or more procedural paragraphs that define the procedures to be used.
2. *Identifier* represents a report group. *Identifier* must not appear in more than one USE statement.

   The GENERATE, INITIATE, or TERMINATE statements must not appear in a paragraph within a USE BEFORE REPORTING procedure.

   A USE BEFORE REPORTING procedure must not alter the value of any control data item.

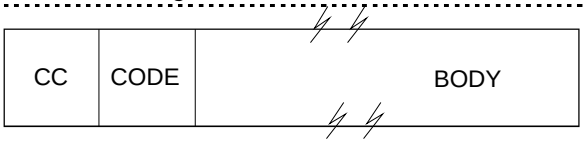
3. The USE statement itself is never executed; it merely defines the conditions calling for the execution of the USE procedures.

8.3.5.4. General Rules

1. The designated procedures are executed by the Report Writer Control System just before the named report group is produced. (See 8.2.21.)
2. Within a USE procedure, there must not be any reference to any nondeclarative procedures. Conversely, in the nondeclarative portion there must be no reference to procedure-names that appear in the declarative portion, except that PERFORM statements may refer to a USE BEFORE REPORTING statement or to the procedures associated with such a USE statement.

8.4. Report Format on File other than Printer File

If the file designated for a report is assigned to other than the printer, a record on the file is in the following format:



where:

CC

is the carriage control character specified as a signed binary item with:

negative (-) = skip to next page

positive (+) = skip specified number of lines

CODE

the 2-character code optionally specified by *literal-1* in CODE clause.

BODY

the report line

The file and record description needed to process this file would be as follows:

```
     FD report-file
         (normal FD clauses)
         DATA RECORD IS report-image .

01 report-image.
    02 carriage-control   PIC     S9(4) COMP.
    02 code               PIC     X(2).
    02 body               PIC     X(size of report line).
```

8.5. Examples

8.5.1. Program 1

The input data, source program, and output report for this sample program are described in the following paragraphs.

8.5.1.1. Input Data

The input data fields are arranged in the following format, using the first record as an example.

```
MONTH     DAY   DEPT #     # OF PURCHASES   ITEM   COST

JANUARY   05    B10        02               A      00300
```

The decimal point in the cost field is assumed to be two places from the right.

The records used in this report are:

```
JANUARY   05  B10  02  A  00300
JANUARY   05  B12  01  A  00400
JANUARY   05  B12  03  C  01500
JANUARY   12  B11  02  B  01200
JANUARY   12  B14  10  A  02000
JANUARY   12  B14  10  C  06000
JANUARY   19  B11  02  B  01200
JANUARY   26  B10  04  B  01200
JANUARY   26  B10  01  C  00500
JANUARY   31  B11  16  A  03200
JANUARY   31  B11  04  B  02400
JANUARY   31  B11  20  D  04000
FEBRUARY  09  B14  05  A  01000
FEBRUARY  09  B14  05  B  02000
FEBRUARY  16  B10  02  A  00300
FEBRUARY  16  B11  02  B  01200
FEBRUARY  16  B12  01  A  00400
FEBRUARY  16  B13  01  A  03000
FEBRUARY  23  B14  10  A  02000
FEBRUARY  23  B15  20  A  02000
FEBRUARY  23  B16  30  A  01500
FEBRUARY  23  B17  30  D  03000
FEBRUARY  28  B10  02  A  00300
FEBRUARY  28  B12  01  A  00400
```

8.5.1.2. Source Program

```
000005 IDENTIFICATION DIVISION.
000010 PROGRAM-ID. EXPENDITURES.
000015 ENVIRONMENT DIVISION.
000020 CONFIGURATION SECTION.
000025 SOURCE-COMPUTER. UNISYS-2200.
000030 OBJECT-COMPUTER. UNISYS-2200.
000035 INPUT-OUTPUT SECTION.
000040 FILE-CONTROL.
000045     SELECT INFILE ASSIGN TO CARD-READER.
000050     SELECT REPORT-FILE ASSIGN TO PRINTER.
000055 DATA DIVISION.
000060 FILE SECTION.
000065 FD  INFILE
000070      LABEL RECORDS ARE OMITTED DATA RECORD IS IN-REC.
000075 01  IN-REC.
000080     02 MONTH  PIC A(9).
000085     02 DAY-1  PIC XX.
000090     02 FILLER PIC X.
000095     02 DEPT   PIC XXX.
000100     02 FILLER PIC X.
000105     02 PURCHS PIC 99.
000110     02 FILLER PIC X.
000115     02 ITEM   PIC X.
000120     02 FILLER PIC X.
000125     02 COST   PIC 999V99.
000130     02 FILLER PIC X(54).
000135 FD  REPORT-FILE
000140      LABEL RECORDS ARE OMITTED REPORT IS EXPENDITURES.
000145 REPORT SECTION.
000150 RD  EXPENDITURES
000155      CONTROLS ARE FINAL, MONTH, DAY-1
000160      PAGE LIMIT IS 66 LINES, FIRST DETAIL 14,
000165      LAST DETAIL 54, FOOTING 59.
000170 01  TYPE RH.
000175     02 LINE NUMBER IS 4.
000180        03 COLUMN NUMBER 28 PICTURE X(26) VALUE IS
000185           "USER MANUFACTURING COMPANY".
000190     02 LINE NUMBER IS 6.
000195        03 COLUMN NUMBER IS 27 PICTURE X(28) VALUE IS
000200           "EXPENDITURES REPORT FOR 2002".
000205 01  TYPE PH.
000210     02 LINE NUMBER IS 10.
000215        03 COLUMN 5 PICTURE X(5) VALUE IS "MONTH".
000220        03 COLUMN 16 PICTURE X(36) VALUE IS
000225          "W/E DEPT NOITEM COST".
000230        03 COLUMN 63 PICTURE X(10) VALUE IS "CUMULATIVE".
000235     02 LINE NUMBER IS 11.
000240        03 COLUMN 28 PICTURE X(16) VALUE IS
000245           "PURCHASES CODE".
```

```
000250        03 COLUMN 66 PICTURE X(4) VALUE IS "COST".
000255 01  TYPE CH MONTH.
000260     02 LINE PLUS 2.
000265        03 COLUMN 1 PICTURE X(9) SOURCE IS MONTH.
000270        03 COLUMN 11 PICTURE X(12) VALUE IS "EXPENDITURES".
000275 01  COST-REPORT TYPE DE.
000280     02 LINE PLUS 2.
000285        03 COLUMN 5 PICTURE X(9) SOURCE MONTH GROUP INDICATE.
000290        03 COLUMN 16 PICTURE XX SOURCE DAY-1 GROUP INDICATE.
000295        03 COLUMN 24 PICTURE XXX SOURCE DEPT.
000300        03 COLUMN 30 PICTURE Z9 SOURCE PURCHS.
000305        03 COLUMN 41 PICTURE X SOURCE ITEM.
000310        03 PRICE COLUMN 47 PICTURE ZZ9.99 SOURCE COST.
000315 01  TYPE CF DAY-1.
000320     02 LINE PLUS 2.
000325        03 COLUMN 1 PICTURE IS X(19) VALUE IS
000330           "TOTAL COSTS FOR W/E".
000335        03 COLUMN 21 PICTURE IS X(9) SOURCE IS MONTH.
000340        03 COLUMN 31 PICTURE IS X(2) SOURCE IS DAY-1.
000345        03 WEEK-PRICE COLUMN 47 PICTURE ZZ9.99 SUM COST.
000350        03 CUME-PRICE COLUMN 64 PICTURE ZZ9.99 SUM WEEK-PRICE
000355           RESET ON FINAL.
000360 01  TYPE CF MONTH.
000365     02 LINE 55.
000370        03 COLUMN 4 PICTURE X(15) VALUE IS
000375           "TOTAL COSTS FOR".
000380        03 COLUMN 20 PICTURE X(9) SOURCE MONTH.
000385        03 COLUMN 31 PICTURE ZZ9.99 SUM COST.
000390 01  TYPE PF.
000395     02 LINE 60.
000400        03 COLUMN 55 PICTURE X(4) VALUE "PAGE".
000405        03 COLUMN 60 PICTURE 99 SOURCE PAGE-COUNTER.
000410 01  TYPE CF FINAL.
000415     02 LINE 58.
000420        04 COLUMN 1 PICTURE X(39) VALUE
000425           "TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS".
000430        04 COLUMN 42 PICTURE 999.99 SOURCE CUME-PRICE.
000435 01  TYPE RF.
000440     02 LINE PLUS 2.
000445        03 COLUMN 32 PICTURE X(10) VALUE IS "END REPORT".
000450 PROCEDURE DIVISION.
000455 P1. OPEN INPUT INFILE OUTPUT REPORT-FILE.
000460     INITIATE EXPENDITURES.
000465 P2. READ INFILE RECORD AT END GO TO P3.
000470     GENERATE COST-REPORT.
000475     GO TO P2.
000480 P3. TERMINATE EXPENDITURES.
000485     CLOSE INFILE.
000490     CLOSE REPORT-FILE.
000495     STOP RUN.
```

8.5.1.3. Output Report

The following report is produced by the source program in 8.5.1.2. The numbers enclosed in dashes are references, used in the explanation in 8.5.1.4.

-1- USER MANUFACTURING COMPANY

-1- EXPENDITURES REPORT FOR 2002

-2- MONTH	W/E	DEPT	NO PURCHASES	ITEM CODE	COST	CUMULATIVE COST	
JANUARY EXPENDITURES -3-							
-4- JANUARY	05	B10	2	A	3.00		
-5-	-5-	B12	1	A	4.00		
		B12	3	C	15.00		
TOTAL COSTS FOR W/E JANUARY			05		22.00	22.00	-6-
-7- JANUARY	12	B11	2	B	12.00		
		B14	10	A	20.00		
		B14	10	C	60.00		
TOTAL COSTS FOR W/E JANUARY			12		92.00	114.00	
JANUARY	19	B11	2	B	12.00		
TOTAL COSTS FOR W/E JANUARY			19		12.00	126.00	
JANUARY	26	B10	4	B	12.00		
		B10	1	C	5.00		
TOTAL COSTS FOR W/E JANUARY			26		17.00	143.00	
JANUARY	31	B11	16	A	32.00		
		B11	4	B	24.00		
		B11	20	D	40.00		
TOTAL COSTS FOR W/E JANUARY			31		96.00	239.00	

-8- TOTAL COSTS FOR JANUARY 239.00

-9- PAGE 01

-10- MONTH	W/E	DEPT	NO PURCHASES	ITEM CODE	COST	CUMULATIVE COST	
FEBRUARY EXPENDITURES -11-							
-4- FEBRUARY	09	B14	5	A	10.00		
-5-	-5-	B14	5	B	20.00		
TOTAL COSTS FOR W/E FEBRUARY	09				30.00	269.00	-6-
-7- FEBRUARY	16	B10	2	A	3.00		
		B11	2	B	12.00		
		B12	1	A	4.00		
		B13	1	A	30.00		
TOTAL COSTS FOR W/E FEBRUARY	16				49.00	318.00	
FEBRUARY	23	B14	10	A	20.00		
		B15	20	A	20.00		
		B16	30	A	15.00		
		B17	30	D	30.00		
TOTAL COSTS FOR W/E FEBRUARY	23				85.00	143.00	
FEBRUARY	28	B10	2	A	3.00		
		B12	1	A	4.00		
TOTAL COSTS FOR W/E FEBRUARY	28				7.00	410.00	

TOTAL COSTS FOR FEBRUARY 171.00

TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS 410.00

-13- PAGE 02

-14- END REPORT

8.5.1.4. Relation Between Output Report and Source Program

In the following key, the numbers enclosed in dashes relate the explanation to the output report in 8.5.1.3.

The six-digit numbers refer to the sequence numbers of the source listing in 8.5.1.2.

-1-

is the REPORT HEADING produced from source lines 000170 through 000200. The output lines are printed on lines 4 and 6, as specified in the report group description. This output is the result of the first GENERATE statement being executed in the object program.

-2-

is the PAGE HEADING produced from source lines 000205 through 000250. The output lines are printed on lines 10 and 11, as specified in the report group description. This output is also the result of the first GENERATE statement being executed in the object program.

-3-

is the CONTROL HEADING produced from source lines 000255 through 000270. The output line is printed on FIRST DETAIL *integer-3* (line 14) since this is the first body group with relative line spacing to be printed on the output page. Likewise, this output is the result of the first GENERATE statement being executed in the object program.

-4-

is the DETAIL produced from source lines 000275 through 000310. Since this is not the first body group to be presented on the output page, it is printed two lines from the last line used by the previous report group. Note that each time the DETAIL is presented, the source item "COST" is added to the sum counter's "WEEK–PRICE" and to the unnamed sum counter in the CONTROL FOOTING "MONTH" report group.

-5-

Note that the data items January and 05 appearing on the second input image do not appear as print output. This is a result of the GROUP INDICATE clauses in source lines 000285 and 000290. The GROUP INDICATE clause causes the printing of the item appearing in the source line to be suppressed at all times except for the first print line appearing after a control break and after a page break.

-6-

is the CONTROL FOOTING "DAY–1" produced from source lines 000315 through 000355. This report group presentation results from the break in the value of the data item "DAY–1" compared to its previous value. Since this is not the first body group to be presented on the output page, it is printed two lines from the last line used by the previous report group. The RESET ON FINAL clause prevents the clearing of the sum counter "CUME–PRICE" until the "FINAL" control break occurs. The sum counter "WEEK–PRICE" is reset to zero.

-7-

The data items "MONTH" and "DAY–1" appear as printed output because this is the first print line following a control break.

-8-

is the CONTROL FOOTING "MONTH" produced from source lines 000360 through 000385. This report group presentation results from the break in the value of the data item "MONTH" compared to its previous value. The output line is printed on line 55 as specified in source line 000365. The control break is considered to occur for the most major identifier at which a change has been recognized. Thus, in this example, the control identifier "MONTH" is recognized as changing (not the control identifier "DAY–1"), and appropriate footings starting from the most minor control ("DAY–1") through the control break ("MONTH") are presented.

-9-

is the PAGE FOOTING produced from source lines 000390 through 000405. The output line is printed on line 60 as specified in source line 000395. This report group presentation results from the following considerations:

a. Following the CONTROL FOOTING "MONTH" report group print line, the next group to be is the CONTROL HEADING "MONTH" report group.

b. However, the last allowable print line for a CONTROL HEADING is LAST DETAIL *integer–4* (line 54). Thus, the RWCS recognizes this and automatically produces a PAGE FOOTING as part of the page break procedure.

-10-

is the PAGE HEADING produced as part of the page break procedure.

-11-

is the CONTROL HEADING "MONTH" still being presented as part of the control break recognized above. Appropriate CONTROL HEADING report groups are presented from the control break level down through the minor CONTROL HEADING report group. In our example only CONTROL HEADING "MONTH" is presented since this is the only CONTROL HEADING specified.

-12-

is the CONTROL FOOTING "FINAL" produced from source lines 000410 through 000430. This report group, as well as the two CONTROL FOOTING report groups preceding it as print output are results of the verb TERMINATE. The output line is printed on line 58 as specified in source line 000415.

-13-

is the PAGE FOOTING produced as a result of executing the TERMINATE verb in the object program.

-14-

is the REPORT FOOTING produced from source lines 000435 through 000445 as a result of executing the TERMINATE verb in the object program. Since a PAGE FOOTING was presented on the page, the REPORT FOOTING is presented on the last line used by the PAGE FOOTING plus 2 (line 62). Note that this falls within the print area defined by the PAGE LIMIT clause, that is, the last allowable print line for the REPORT FOOTING is less than or equal to PAGE LIMIT *integer-1*.

Source line 000460 directs the RWCS to reset all sum counters to zero, to initialize the LINE–COUNTER to zero, and to initialize the PAGE–COUNTER to one. Source line 000470 directs the RWCS to generate the desired report. Source line 00480 in turn directs the RWCS to terminate the desired report, and to present any CONTROL FOOTING, PAGE FOOTING, or REPORT FOOTING report groups necessary to accomplish this termination.

Note: *Column spacing and line spacing in 8.5.1.3 are relative. For exact placement refer to the source code in 8.5.1.2.*

8.5.2. Program 2

The input data, source program, and output report for this sample program are described in the following paragraphs.

8.5.2.1. Input Data

The input data fields are arranged in the following format, using the first record as an example.

```
MONTH     DAY  DEPT #     # OF PURCHASES   ITEM  COST

JANUARY   05   B10        02               A     00300
```

The decimal point in the cost field is assumed to be two places from the right.

The records used in this report are:

```
JANUARY   05  B10  02  A  00300
JANUARY   05  B12  01  A  00400
JANUARY   05  B12  03  C  01500
JANUARY   12  B11  02  B  01200
JANUARY   12  B14  10  A  02000
JANUARY   12  B14  10  C  06000
FEBRUARY  09  B14  05  A  01000
FEBRUARY  09  B14  05  B  02000
```

8.5.2.1.1. Source Program

```
000005 IDENTIFICATION DIVISION.
000010 PROGRAM-ID. EXPENDITURES.
000015 ENVIRONMENT DIVISION.
000020 CONFIGURATION SECTION.
000025 SOURCE-COMPUTER. UNISYS-2200.
000030 OBJECT-COMPUTER. UNISYS-2200.
000035 INPUT-OUTPUT SECTION.
000040 FILE-CONTROL.
000045     SELECT INFILE ASSIGN TO CARD-READER.
000050     SELECT REPORT-FILE ASSIGN TO PRINTER.
000055 DATA DIVISION.
000060 FILE SECTION.
000065 FD  INFILE
000070     LABEL RECORDS ARE OMITTED DATA RECORD IS IN-REC.
000075 01  IN-REC.
000080     02 MONTH   PIC A(9).
000085     02 DAY-1   PIC XX.
000090     02 FILLER  PIC X.
000095     02 DEPT    PIC XXX.
000100     02 FILLER  PIC X.
000105     02 PURCHS  PIC 99.
000110     02 FILLER  PIC X.
000115     02 ITEM    PIC X.
000120     02 FILLER  PIC X.
000125     02 COST    PIC 999V99.
000130     02 FILLER  PIC X(54).
000135 FD  REPORT-FILE
000140     LABEL RECORDS ARE OMITTED REPORT IS EXPENDITURES.
000145 REPORT SECTION.
000150 RD  EXPENDITURES
000155     CONTROLS ARE FINAL, MONTH, DAYû1
000160     PAGE LIMIT IS 66 LINES, FIRST DETAIL 14,
000165     LAST DETAIL 54, FOOTING 59.
000170 01  TYPE RH NEXT GROUP NEXT PAGE.
000175     02 LINE NUMBER 3.
000180        03 COLUMN NUMBER 28 PIC X(26) VALUE IS
000185          "USER MANUFACTURING COMPANY".
000190     02 LINE NUMBER IS PLUS 13.
000195        03 COLUMN NUMBER 27 PIC X(28) VALUE IS
000200          "EXPENDITURES REPORT FOR 2002".
000205 01  TYPE PH.
000210     02 LINE NUMBER IS 4.
000215        03 COLUMN 5 PIC X(5) VALUE IS "MONTH".
000220        03 COLUMN 16 PIC X(36) VALUE IS
000225          "W/E DEPT NOITEM COST".
000230        03 COLUMN 63 PIC X(10) VALUE IS
000235          "CUMULATIVE".
000240     02 LINE NUMBER PLUS 2.
```

```
000245        03 COLUMN 28 PIC X(16) VALUE IS
000250          "PURCHASES CODE".
000255        03 COLUMN 66 PIC X(4) VALUE IS "COST".
000260 01  TYPE CH MONTH.
000265     02 LINE NUMBER PLUS 2.
000270        03 COLUMN 1 PIC X(9) SOURCE IS MONTH.
000275        03 COLUMN 11 PIC X(12) VALUE IS "EXPENDITURES".
000280 01  COST-REPORT TYPE DE NEXT GROUP PLUS 1.
000285     02 LINE PLUS 1.
000290        03 COLUMN  5 PIC X(9) SOURCE MONTH GROUP INDICATE.
000295        03 COLUMN 16 PIC XX   SOURCE DAY-1 GROUP INDICATE.
000300        03 COLUMN 24 PIC XXX  SOURCE DEPT.
000305        03 COLUMN 30 PIC Z9   SOURCE PURCHS.
000310        03 COLUMN 41 PIC X    SOURCE ITEM.
000315        03 PRICE COLUMN 47 PIC ZZ9.99 SOURCE COST.
000320 01  TYPE CF DAY-1 NEXT GROUP PLUS 5.
000325     02 LINE PLUS 1.
000330        03 COLUMN 1 PIC X(19) VALUE IS
000335          "TOTAL COSTS FOR W/E".
000340        03 COLUMN 21 PIC IS X(9) SOURCE IS MONTH.
000345        03 COLUMN 31 PIC IS X(2) SOURCE IS DAY-1.
000350        03 WEEK-PRICE COLUMN 47 PIC ZZ9.99 SUM COST.
000355        03 CUME-PRICE COLUMN 64 PIC ZZ9.99 SUM
000360       WEEK-PRICE RESET ON FINAL.
000365 01  TYPE CF MONTH NEXT GROUP NEXT PAGE.
000370     02 LINE PLUS 2.
000375        03 COLUMN 4 PIC X(15) VALUE IS
000380          "TOTAL COSTS FOR".
000385        03 COLUMN 20 PIC X(9) SOURCE MONTH.
000390        03 COLUMN 31 PIC ZZ9.99 SUM COST.
000395 01  TYPE PF.
000400     02 LINE 60.
000405        03 COLUMN 55 PIC X(4) VALUE "PAGE".
000410        03 COLUMN 60 PIC 99 SOURCE PAGE-COUNTER.
000415 01  TYPE CF FINAL.
000420     02 LINE PLUS 2.
000425        04 COLUMN 1 PIC X(39) VALUE
000430          "TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS".
000435        04 COLUMN 42 PIC 999.99 SOURCE CUME-PRICE.
000440 01  TYPE RF.
000445     02 LINE NUMBER IS 4 NEXT PAGE.
000450        03 COLUMN 32 PIC X(10) VALUE IS "END REPORT".
000455 PROCEDURE DIVISION.
000460 P1. OPEN INPUT INFILE OUTPUT REPORT-FILE.
000465     INITIATE EXPENDITURES.
000470 P2. READ INFILE RECORD AT END GO TO P3.
000475     GENERATE COST-REPORT.
000480     GO TO P2.
```

```
000485 P3. TERMINATE EXPENDITURES.
000490     CLOSE INFILE.
000495     CLOSE REPORT-FILE.
000500     STOP RUN.
```

8.5.2.1.2. Output Report

The following is the report produced by the source program in 8.5.2.1.1. The numbers enclosed in dashes are references, used in the explanation in 8.5.2.2.

```
-1-     USER MANUFATURING COMPANY

-2-     EXPENDITURES REPORT FOR 2002

-3-
```

MONTH	W/E	DEPT	NO PURCHASES	ITEM CODE	COST		CUMULATIVE COST	
JANUARY EXPENDITURES		*-4-*						
JANUARY	05	B10	2	A	3.00	*-5-*		
		B12	1	A	4.00	*-6-*		
		B12	3	C	15.00			
TOTAL COSTS FOR W/E JANUARY			05		22.00		22.00	*-7-*
JANUARY	12	B11	2	B	12.00	*-8-*		
		B14	10	A	20.00			
		B14	10	C	60.00			
TOTAL COSTS FOR W/E JANUARY			12		92.00		114.00	*-9-*
TOTAL COSTS FOR JANUARY			114.00	*-10-*				

PAGE 02

MONTH	W/E	DEPT	NO PURCHASES	ITEM CODE	COST		CUMULATIVE COST	
FEBRUARY EXPENDITURES			*-4-*					
FEBRUARY	09	B14	5	A	10.00	*-5-*		
		B14	5	B	20.00	*-6-*		
TOTAL COSTS FOR W/E FEBRUARY		09			30.00		144.00	*-11-*
TOTAL COSTS FOR FEBRUARY		30.00		*-11-*				
TOTAL EXPENDITURES FOR YEAR-TO-DATE WAS					144.00	*-11-*		

PAGE 03

END REPORT -12-

8.5.2.2. Relation Between Output Report and Source Program

In the key that follows, the numbers enclosed in dashes relate the explanation to the output report in 8.5.2.1.2.

The 6–digit numbers refer to the sequence numbers included in the source listing of 8.5.2.1.1.

-1-

is the first line of REPORT HEADING produced from source lines 000170 through 000185. The first printable line of the report group is presented on line number 3.

-2-

is the second line of the REPORT HEADING produced from source lines 000190 through 000200. The line is presented 13 lines from the last line printed, or line number 16. Note that the use of the NEXT GROUP NEXT PAGE clause has allowed the last allowable print line of the report group to assume the value of PAGE LIMIT *integer–1*, or 66. Failure to include the NEXT GROUP NEXT PAGE clause would have resulted in a diagnostic error condition.

-3-

is the approximate area in which PAGE FOOTING would appear. However, the use of the NEXT GROUP NEXT PAGE clause in a REPORT HEADING results in the suppression of the PAGE FOOTING.

-4-

is the CONTROL HEADING "MONTH" produced from source lines 000260 through 000275. Since this is the first body group to appear on the page, and since relative line spacing is specified, the print line appears on line 14 (FIRST DETAIL).

-5-

is the DETAIL report group produced from source lines 000280 through 000315. Note that it appears one line from the print line used by the last report group.

-6-

is also a DETAIL report group. However, in contrast to *–5-*, it appears two lines from the print line used by *–5-*. This is because *–5-* included a relative NEXT GROUP value of one. Thus, the NEXT GROUP value (1) plus the line value from 000285 (1) places the group two lines down on the print page.

-7-

is the CONTROL FOOTING "DAY–1" produced from source lines 000320 through 000360. It appears because of a break in the control value "DAY–1". Since the break is for "DAY–1", the NEXT GROUP clause on line 000320 is not ignored. Thus, the calculation of the first print line of the next report group will be biased by the value of this NEXT GROUP PLUS 5 value.

-8-

is the DETAIL report group, the printing of which has been biased by the NEXT GROUP value of –*7*-. It is printed (5 plus 1) or six lines from the last print line used.

-9-

Note that the control break is recognized as occurring at the highest changed identifier value, or "MONTH" in this case. For this reason the NEXT GROUP value is ignored.

-10-

Since the break is for "MONTH", the NEXT GROUP clause on source line 000365 is not ignored. This NEXT GROUP IS NEXT PAGE clause refers to the next body group. Thus a page advance is performed.

-11-

Is the result of the verb TERMINATE being executed in the object program.

-12-

Is the REPORT FOOTING produced from source lines 000440 through 000450. It is printed on a page by itself because of the LINE NUMBER IS 4 NEXT PAGE clause on source line 000445. This method of producing a REPORT FOOTING on a separate page is similar to using the NEXT GROUP NEXT PAGE clause for REPORT HEADING report groups in order to print it on a separate page.

Note: *The column spacing and line spacing in the output report of 8.5.2.1.2 are relative. For exact placement refer to the source code of 8.5.2.1.1.*

Section 9
Segmentation

9.1. General

9.1.1. Function

The Segmentation module provides a capability to specify object program overlay requirements.

9.1.2. Level Characteristics

Segmentation Level 1 provides a facility for specifying permanent and independent segments (see 9.2.2.1). All sections with the same segment-number must be contiguous in the source program. All segments specified as permanent segments must be contiguous in the source program.

Segmentation Level 2 provides the facility for intermixing sections with different segment-numbers and allows the fixed portion of the source program to contain segments that may be overlaid (see 9.2.2.2).

9.2. General Description

COBOL segmentation is a facility by which the user may communicate with the compiler to specify object program overlay requirements.

9.2.1. Scope

COBOL segmentation deals only with segmentation of procedures. As such, only the Procedure Division and the Environment Division are considered in determining segmentation requirements for an object program.

9.2.2. Organization

9.2.2.1. Program Segments

Although it is not mandatory, the Procedure Division for a source program is usually written as a consecutive group of sections, each of which is composed of a series of closely related operations that are designed to collectively perform a particular function. However, when segmentation is used, the entire Procedure Division must be in sections. Each section must be classified as belonging either to the fixed portion or to one of the independent segments of the object program. Segmentation in no way affects the need for qualification of procedure-names to ensure uniqueness.

9.2.2.2. Fixed Portion

The fixed portion is defined as that part of the object program which is logically treated as if it were always in memory. This portion of the program is composed of two types of segments: fixed permanent segments and fixed overlayable segments.

A fixed permanent segment is a segment in the fixed portion which cannot be overlaid by any other part of the program.

A fixed overlayable segment is a segment in the fixed portion which, although logically treated as if it were always in memory, can be overlaid by another segment to optimize memory utilization. Variation of the number of fixed permanent segments in the fixed portion can be accomplished by using a special facility called the SEGMENT–LIMIT clause (see 9.3.2). Such a segment, if called for by the program, is always made available in its last used state.

9.2.2.3. Independent Segments

An independent segment is defined as part of the object program which can overlay, and can be overlaid by, either a fixed overlayable segment or another independent segment. An independent segment is in its initial state whenever control is transferred (either implicitly or explicitly) to that segment for the first time during the execution of a program. On subsequent transfers of control to the segment, an independent segment is also in its initial state when:

1. Control is transferred to that segment as a result of the implicit transfer of control between consecutive statements from a segment with a different segment-number.
2. Control is transferred to that segment as the result of the implicit transfer of control between a SORT or MERGE statement, in a segment with a different segment-number, and an associated input or output procedure in that independent segment.
3. Control is transferred explicitly to that segment from a segment with a different segment-number (with the exception noted in paragraph 2 that follows).

On subsequent transfer of control to the segment, an independent segment is in its last-used state when:

1. Control is transferred implicitly to that segment from a segment with a different segment-number (except as noted in preceding paragraphs 1 and 2).
2. Control is transferred explicitly to that segment as the result of the execution of an EXIT PROGRAM statement.

See 1.3.3.4.2 for an explanation of explicit and implicit transfers of control.

9.2.3. Segmentation Classification

Sections which are to be segmented are classified, using a system of segment–numbers (see 9.3.1) and the following criteria:

1. Logic Requirements—Sections which must be available for reference at all times, or which are referred to very frequently, are normally classified as belonging to one of the permanent segments; sections which are used less frequently are normally classified as belonging either to one of the overlayable fixed segments or to one of the independent segments, depending on logic requirements.
2. Frequency of Use—Generally, the more frequently a section is referred to, the lower its segment-number, the less frequently it is referred to, the higher its segment-number.
3. Relationship to Other Sections—Sections that frequently communicate with one another should be given the same segment-numbers.

9.2.4. Segmentation Control

The logical sequence of the program is the same as the physical sequence, except for specific transfers of control. The compiler provides all necessary transfers of control from segment to segment, and provides control necessary for a segment to operate when that segment is called upon to be used. The segment under reference will be brought in before program continuation unless it is already in main storage. Control may be transferred within a source program to any paragraph in the section; that is, it is not mandatory to transfer control to the beginning of a section.

9.3. Structure of Program Segments

9.3.1. Segment–Numbers

Section classification is accomplished by means of a system of segment-numbers. The segment-number is included in the section header.

9.3.1.1. General Format

```
section-name SECTION [ segment-number ] .
```

9.3.1.2. Syntax Rules

1. The *segment-number* must be an integer ranging in value from 0 through 99.
2. If the *segment-number* is omitted from the section header, the *segment-number* is assumed to be 0.
3. Sections in the declaratives must contain *segment-numbers* less than 50.

9.3.1.3. General Rules

1. All sections which have the same *segment-number* constitute a program segment. In Level 1, all sections which have the same *segment-number* must be together in the source program. In Level 2, sections with the same *segment-numbers* need not be physically contiguous in the source program.
2. Segments with *segment-number* 0 through 49 belong to the fixed portion of the object program. In Level 1 all sections with *segment-number* 0 through 49 must be together in the source program.
3. Segments with *segment-number* 50 through 99 are independent segments.

9.3.2. SEGMENT–LIMIT Clause

Ideally, all program segments having *segment-number*s ranging from 0 through 49 should be specified as permanent segments. However, when insufficient memory is available to contain all permanent segments plus the largest overlayable segment, it becomes necessary to decrease the number of permanent segments. The SEGMENT–LIMIT feature provides the user with a means by which he can reduce the number of permanent segments in his program, while still retaining the logical properties of fixed portion segments (segment-numbers 0 through 49).

9.3.2.1. General Format

The SEGMENT–LIMIT clause appears in the OBJECT–COMPUTER paragraph and has the following format:

```
[ , SEGMENT-LIMIT IS segment-number ]
```

9.3.2.2. Syntax Rules

1. *Segment-number* must be an integer ranging in value from 1 through 49.

9.3.2.3. General Rules

1. When the SEGMENT–LIMIT clause is specified, only those segments having segment-numbers from 0 up to, but not including, the segment-number designated as the segment-limit, are considered as permanent segments of the object program.
2. Those segments having segment-numbers from the segment–limit through 49 are considered as overlayable fixed segments.
3. When the SEGMENT–LIMIT clause is omitted, all segments having segment-numbers from 0 through 49 are considered as permanent segments of the object program.

9.4. Restriction on Program Flow

When segmentation is used, the following restrictions are placed on the ALTER, PERFORM, MERGE, and SORT statements.

9.4.1. The ALTER Statement

A GO TO statement in a section whose segment-number is greater than or equal to 50 must not be referred to by an ALTER statement in a section with a different segment-number.

All other uses of the ALTER statement are valid and are performed even if the GO TO to which the ALTER refers is in a fixed overlayable segment.

9.4.2. The PERFORM Statement

A PERFORM statement that appears in a section that is not in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:

1. Sections and/or paragraphs wholly contained in one or more nonindependent segments.
2. Sections and/or paragraphs wholly contained in a single independent segment.

A PERFORM statement that appears in an independent segment can have within its range, in addition to any declarative sections whose execution is caused within that range, only one of the following:

1. Sections and/or paragraphs wholly contained in one or more nonindependent segments.
2. Sections and/or paragraphs wholly contained in the same independent segment as that PERFORM statement

9.4.3. The MERGE Statement

If the MERGE statement appears in a section that is not in an independent segment, then any output procedure referenced by that MERGE statement must appear:

- Totally within nonindependent segments, or
- Wholly contained in a single independent segment.

If a MERGE statement appears in an independent segment, then any output procedure referenced by that MERGE statement must be contained:

- Totally within nonindependent segments, or
- Wholly within the same independent segment as that MERGE statement.

9.4.4. The SORT Statement

If a SORT statement appears in a section that is not an independent segment, then any input procedures or output procedures referenced by that SORT statement must appear:

1. Totally within nonindependent segments, or
2. Wholly contained in a single independent segment.

If a SORT statement appears in an independent segment, then any input procedures or output procedures referenced by that SORT statement must be contained:

1. Totally within nonindependent segments, or
2. Wholly within the same independent segment as that SORT statement.

Section 10
Library

10.1. General

10.1.1. Function

The Library module provides a capability for specifying text that is to be copied from a library.

COBOL libraries contain library texts that are available to the compiler for copying at compile time. The effect of the interpretation of the COPY statement is to insert text into the source program, where it will be treated by the compiler as part of the source program.

COBOL library text is placed in the COBOL library as a function independent of the COBOL program. (See 18.6.)

10.1.2. Level Characteristics

Library Level 1 provides the facility for copying text from a single library into the source program. Text is copied from the library without change.

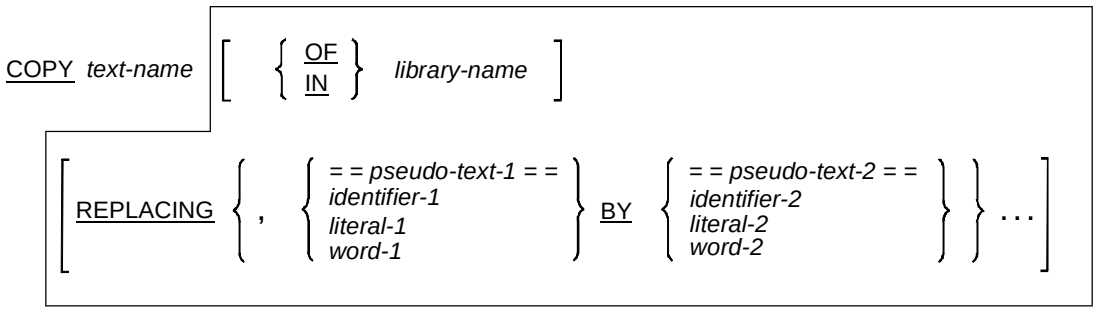
Library Level 2 provides the additional capability of replacing all occurrences of a given literal, identifier, word or group of words in the library text, with alternate text, during the copying process. Level 2 also provides for the availability of more than one COBOL library at compile time. (See 18.5.)

10.2. The COPY Statement

10.2.1. Function

The COPY statement incorporates text into a COBOL source program.

10.2.2. General Format



10.2.3. Syntax Rules

1. If more than one COBOL library is available during compilation, *text–name* must be qualified by the *library-name* identifying the COBOL library in which the text associated with *text-name* resides. (See 2.1.3.2 for constraints that apply when Library is associated with Nucleus, Level 1.)

   Within one COBOL library, each *text-name* must be unique.

2. The COPY statement must be preceded by a space and terminated by the separator period.

3. *Pseudo-text-1* must not be null, nor may it consist solely of the character space, nor may it consist solely of comment lines.

4. *Pseudo-text-2* may be null.

5. Character-strings within *pseudo-text-1* and *pseudo-text-2* may be continued. However, both characters of a pseudo-text delimiter must be on the same line. (See 1.3.9.2.2.) *Pseudo-text-1* and *pseudo-text-2* consist of COBOL text-words.

6. *Word-1* or *word-2* may be any single COBOL word.

7. A COPY statement may occur in the source program anywhere a character-string or a separator may occur, except that a COPY statement must not occur within a COPY statement.

8. If the word COPY appears in a comment-entry or in the place where a comment-entry can appear, it is considered part of the comment-entry.

10.2.4. General Rules

1. The compilation of a source program containing COPY statements is logically equivalent to processing all COPY statements prior to the processing of the resulting source program.
2. The effect of processing a COPY statement is that the library text associated with *text-name* is copied into the source program, logically replacing the entire COPY statement, beginning with the reserved word COPY and ending with the punctuation character period, inclusive. (See 18.5.)
3. If the REPLACING phrase is not specified, the library text is copied unchanged.

   If the REPLACING phrase is specified, the library text is copied or each properly matched occurrence of *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1* in the library text is replaced by the corresponding *pseudo-text-2*, *identifier-2*, *word-2*, or *literal-2*.

4. For purposes of matching, *identifier-1*, *word-1*, and *literal-1* are treated as pseudo-text containing only *identifier-1*, *word-1*, and *literal-1*, respectively.
5. The comparison operation to determine text replacement occurs in the following manner:

   Any separator comma, semicolon, and/or spaces preceding the leftmost library text-word are copied into the source program. Starting with the leftmost library text-word and the first *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1* that was specified in the REPLACING phrase, the entire REPLACING phrase operand that precedes the reserved word BY is compared to an equivalent number of contiguous library text-words.

   Pseudo-text-1, *identifier-1*, *word-1*, or *literal-1* match the library text if, and only if, the ordered sequence of text-words that forms *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1* is equal, character for character, to the ordered sequence of library text-words. For purposes of matching, each occurrence of a separator comma or semicolon in *pseudo-text-1* or in the library text is considered to be a single space except when *pseudo-text-1* consists solely of either a separator comma or semicolon, in which case it participates in the match as a text-word. Each sequence of one or more space separators is considered to be a single space.

   If no match occurs, the comparison is repeated with each next successive *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1*, if any, in the REPLACING phrase until either a match is found or there is no next successive REPLACING operand.

   When all the REPLACING phrase operands have been compared and no match has occurred, the leftmost library text–word is copied into the source program. The next successive library text–word is then considered as the leftmost library text-word, and the comparison cycle starts again with the first *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1* specified in the REPLACING phrase.

   Whenever a match occurs between *pseudo-text–1*, *identifier-1*, *word-1*, or *literal-1* and the library text, the corresponding *pseudo-text-2*, *identifier-2*, *word-2*, or *literal-2* is placed into the source program. The library text-word immediately following the rightmost text–word that participated in the match is then considered as the leftmost library text-word. The comparison cycle starts again with the first *pseudo-text-1*, *identifier-1*, *word-1*, or *literal-1* specified in the REPLACING phrase.

The comparison operation continues until the rightmost text-word in the library text has either participated in a match or been considered as a leftmost library text-word and participated in a complete comparison cycle.

6. A comment line occurring in the library text and *pseudo-text-1* is interpreted, for purposes of matching, as a single space. Comment lines appearing in *pseudo-text-2* and library text are copied into the source program unchanged.
7. Debugging lines are permitted within library text and *pseudo-text-2*. Debugging lines are not permitted within *pseudo-text-1;* text-words within a debugging line participate in the matching rules as if the "D" did not appear in the indicator area. If a COPY statement is specified on a debugging line, the text that is the result of the processing of the COPY statement will appear as though it were specified on debugging lines with the following exception: comment lines in library text will appear as comment lines in the resultant source program.
8. The text produced as a result of the complete processing of a COPY statement must not contain a COPY statement.
9. The syntactic correctness of the library text cannot be independently determined. The syntactic correctness of the entire COBOL source program cannot be determined until all COPY statements have been completely processed.
10. Each word in *pseudo-text-2* that is to be placed in the resultant program is placed in the same area as it appears in *pseudo-text-2*. Library text must conform to the rules for COBOL reference format.
11. For purposes of compilation, text-words after replacement are placed in the source program according to the rules for reference format. (See 1.3.9.) On the source listing, all copied material will be realigned to allow for replacement of greater length.

10.3. Examples

The ensuing paragraphs provide examples of libraries and the manner in which calls are made on the libraries.

Contents of Library A

```
Column

12     8   12

ID PROC
           TEST-PROG.
 END
COMPUTER PROC
       SOURCE-COMPUTER.  UNISYS-2200.
       OBJECT-COMPUTER.  UNISYS-2200.
 END
NAMES PROC
           PRINTER IS PRINT-FILE.
 END
FILECONT PROC
           SELECT FILEA ASSIGN TO PRINT-FILE PRT.
       I-O-CONTROL.  SAME RECORD AREA FOR FILEA, FILEB.
 END
```

Contents of Library B

```
Column

12     8   12

LIBFD PROC
       FILE1 --- etc.
       01 RECORD1 --- etc.
 END
LIBSD PROC
           DATA RECORD SORT-REC --- etc.
 END
LIBRECORD PROC
           02 ELEM-ITEM --- etc.
           02 XYZ --- etc.
 END
```

Contents of Library C

```
Column

12     8   12

P1 PROC
       SECTION-0 SECTION.
       PARA-NAME-1.
            .
            .
            .
 END
P2 PROC
           SECTION.
       PARA-2.
 END
P3 PROC
           SORT-REC FROM RECORD1.
 END
P4 PROC
           ADD A TO B.
 END
```

Library Calls in a COBOL Program Contained in File A

```
Column

12     8   12

       IDENTIFICATION DIVISION.
       PROGRAM-ID. COPY ID.
       ENVIRONMENT DIVISION.
       COPY COMPUTER.
       SPECIAL-NAMES.  COPY NAMES.
             .
             .
             .
       FILE-CONTROL.  COPY FILECONT.
             .
             .
             .
       DATA DIVISION.
       FILE SECTION.
       FD COPY LIBFD IN B.
       SD FILEB COPY LIBSD OF B.
       01 SORT-REC. COPY LIBRECORD IN B REPLACING ELEM-ITEM BY GRP, XYZ
                BY ABC.
            .
            .
            .
       PROCEDURE DIVISION.
       COPY P1 IN C.
            .
            .
            .
       SECTION-1 COPY P2 IN C.
        .
        .
        .
       PARA-3.
           RELEASE COPY P3 OF C.
        .
        .
        .
       PARA-4.
           COPY P4 OF C REPLACING ==ADD A TO== BY ==
           SUBTRACT C FROM==.
```

Section 11
Debug

11.1. General

11.1.1. Function

The Debug module provides a means by which the user can describe his debugging algorithm including the conditions under which data items or procedures are to be monitored during the execution of the object program.

The decision of what to monitor and what information to display on the output device are explicitly in the domain of the user. The ASCII COBOL debug facility simply provides a convenient access to pertinent information.

11.1.2. Level Characteristics

Debug Level 1 provides a basic debugging capability, including the ability to specify selective or full procedure monitoring, and optionally compiled debugging statements.

Debug Level 2 provides the full ASCII COBOL debugging facility.

11.1.3. Language Concepts

The features of the COBOL language that support the Debug module are:

1. A compile time switch—WITH DEBUGGING MODE.
2. An object time switch.
3. A USE FOR DEBUGGING statement.
4. A special register—DEBUG–ITEM.
5. Debugging lines.

11.1.3.1. DEBUG–ITEM

The reserved word DEBUG–ITEM is the name for a special register generated automatically by the compiler. Only one DEBUG–ITEM is allocated per program. The names of the subordinate data items in DEBUG–ITEM are also reserved words.

11.1.3.2. Compile Time Switch

The WITH DEBUGGING MODE clause is written as part of the SOURCE–COMPUTER paragraph. It serves as a compile time switch over the debugging statements written in the program.

When the WITH DEBUGGING MODE clause is specified in a program, all debugging sections and all debugging lines are compiled as specified in this section. When the WITH DEBUGGING MODE clause is not specified, all debugging lines and all debugging sections are compiled as if they were comment lines.

The specification of the U extra option on the ACOB processor call statement will cause the compiler to ignore the WITH DEBUGGING MODE clause if specified. This option provides a means to turn off the compiler time switch without editting the source program.

11.1.3.3. Object Time Switch

An object time switch dynamically activates the debugging code inserted by the compiler. This switch cannot be addressed in the program. It is controlled outside the COBOL environment. If the switch is "on", all the effects of the debugging language written in the source program are permitted. If the switch is "off", all the effects described in 11.3.1 are inhibited. Recompilation of the source program is not required to provide or take away this facility.

The object time switch is the D option on the @XQT statement. If the switch is "on" (the D option is not present), all the effects of the debugging language written in the source program are executed. If the switch is "off" (the D option is present), the effects of USE FOR DEBUGGING are inhibited. It should be noted that the run-time libraries and inline code are still present. What is inhibited is the updating of the DEBUG–ITEM and the execution of the associated debugging section.

The object time switch has no effect on the execution of the object program if the WITH DEBUGGING MODE clause was not specified in the source program at compile time.

11.2. Environment Division

11.2.1. The WITH DEBUGGING MODE Clause

11.2.1.1. Function

The WITH DEBUGGING MODE clause indicates that all debugging sections and all debugging lines are to be compiled. If this clause is not specified, all debugging lines and sections are compiled as if they were comment lines.

11.2.1.2. General Format

```
SOURCE-COMPUTER. computer-name [ WITH DEBUGGING MODE ] .
```

11.2.1.3. General Rules

1. If the WITH DEBUGGING MODE clause is specified in the SOURCE–COMPUTER paragraph of the Configuration Section of a program, all USE FOR DEBUGGING statements and all debugging lines are compiled.
2. If the WITH DEBUGGING MODE clause is not specified in the SOURCE–COMPUTER paragraph of the Configuration Section of a program, any USE FOR DEBUGGING statements and all associated debugging sections, and any debugging lines are compiled as if they were comment lines.

11.3. Procedure Division

11.3.1. The USE FOR DEBUGGING Statement

11.3.1.1. Function

The USE FOR DEBUGGING statement identifies the user items that are to be monitored by the associated debugging section.

11.3.1.2. General Format

section-name SECTION [*segment-number*].

USE FOR DEBUGGING ON { *cd-name-1* / [ALL REFERENCES OF] *identifier-1* / *file-name-1* / *procedure-name-1* / ALL PROCEDURES }

[, { *cd-name-2* / [ALL REFERENCES OF] *identifier-2* / *file-name-2* / *procedure-name-2* / ALL PROCEDURES }] ... .

11.3.1.3. Syntax Rules

1. Debugging sections, if specified, must appear together immediately after the DECLARATIVES header.
2. Except in the USE FOR DEBUGGING statement itself, there must be no reference to any nondeclarative procedure within the debugging section.
3. Statements appearing outside of the set of debugging sections must not reference procedure-names defined within the set of debugging sections.
4. Except for the USE FOR DEBUGGING statement itself, statements appearing within a given debugging section may reference procedure-names defined within a different USE procedure only with a PERFORM statement.
5. Procedure-names defined within debugging sections must not appear within USE FOR DEBUGGING statements.
6. Any given identifier, cd-name, file-name, or procedure-name may appear in only one USE FOR DEBUGGING statement and may appear only once in that statement.
7. The ALL PROCEDURES phrase can appear only once in a program.
8. When the ALL PROCEDURES phrase is specified, *procedure-name-1*, *procedure-name-2*, ... must not be specified in any USE FOR DEBUGGING statement.

9. *Identifier-1*, *identifier-2*, ..., must not reference any data item defined in the Report Section except sum counters.
10. If the data description entry of the data item referenced by *identifier-1*, *identifier-2*, ..., contains an OCCURS clause or is subordinate to a data description entry that contains an OCCURS clause, *identifier-1*, *identifier-2*, ..., must be specified without the subscripting or indexing normally required.
11. References to the special register DEBUG–ITEM are restricted to references from within a debugging section or a table error section.

11.3.1.4. General Rules

1. In the following general rules, all references to *cd-name-1*, *identifier-1*, *procedure-name-1*, and *file-name-1* apply equally to *cd-name-2*, *identifier-2*, *procedure-name-2*, and *file-name-2*, respectively.
2. Automatic execution of a debugging section is not caused by a statement appearing in a debugging section.
3. When *file-name-1* is specified in a USE FOR DEBUGGING statement, that debugging section is executed:

    a. After the execution of any OPEN or CLOSE statement that references *file-name-1*, and

    b. After the execution of any READ statement (after any other specified USE procedure) not resulting in the execution of an associated AT END or INVALID KEY imperative statement, and

    c. After the execution of any DELETE or START statement that references *file-name-1*.

4. When *procedure-name-1* is specified in a USE FOR DEBUGGING statement that debugging section is executed:

    a. Immediately before each execution of the named procedure;

    b. Immediately after the execution of an ALTER statement which references *procedure-name-1*.

5. The ALL PROCEDURES phrase causes the effects described in general rule 4 to occur for every procedure-name in the program, except those appearing within a debugging section.
6. When the ALL REFERENCES OF *identifier-1* phrase is specified, that debugging section is executed for every statement that explicitly references *identifier-1* at each of the following times:

    a. In the case of a WRITE or REWRITE statement immediately before the execution of that WRITE or REWRITE statement and after the execution of any implicit move resulting from the presence of the FROM phrase.

    b. In the case of a GO TO statement with a DEPENDING ON phrase, immediately before control is transferred and prior to the execution of any debugging section associated with the procedure-name to which control is to be transferred.

c. In the case of a PERFORM statement in which a VARYING, AFTER, or UNTIL phrase references *identifier-1*, immediately after each initialization, modification, or evaluation of the contents of the data item referenced by *identifier-1*.

d. In the case of any other COBOL statement, immediately after execution of that statement.

If *identifier-1* is specified in a phrase that is not executed or evaluated, the associated debugging section is not executed.

7. When *identifier-1* is specified without the ALL REFERENCES OF phrase, that debugging section is executed at each of the following times:

   a. In the case of a WRITE or REWRITE statement that explicitly references *identifier-1*, immediately before the execution of that WRITE or REWRITE statement and after the execution of any implicit move resulting from the presence of the FROM phrase.

   b. In the case of a PERFORM statement in which a VARYING, AFTER or UNTIL phrase references *identifier-1*, immediately after each initialization, modification, or evaluation of the contents of the data item referenced by *identifier-1*.

   c. Immediately after the execution of any other COBOL statement that explicitly references and causes the contents of the data item referenced by *identifier-1* to be changed.

   If *identifier-1* is specified in a phrase that is not executed or evaluated, the associated debugging section is not executed.

8. The associated debugging section is not executed for a specific operand more than once as a result of the execution of a single statement, regardless of the number of times that operand is explicitly specified. In the case of a PERFORM statement which causes iterative execution of a referenced procedure, the associated debugging section is executed once for each iteration.

   Within an imperative statement, each individual occurrence of an imperative verb identifies a separate statement for the purpose of debugging.

9. When *cd-name-1* is specified in a USE FOR DEBUGGING statement, that debugging section is executed:

   a. After the execution of any ENABLE, DISABLE, and SEND statement that references *cd-name-1*,

   b. After the execution of a RECEIVE statement referencing *cd-name-1* that does not result in the execution of the NO DATA imperative-statement, and

   c. After the execution of an ACCEPT MESSAGE COUNT statement that references *cd-name-1*.

10. A reference to *file-name-1*, *identifier-1*, *procedure-name-1*, or *cd-name-1* as a qualifier does not constitute reference to that item for the debugging described in the general rules above.

11. Associated with each execution of a debugging section is the special register DEBUG–ITEM, which provides information about the conditions that caused the execution of a debugging section. DEBUG–ITEM has the following implicit description:

```
01  DEBUG-ITEM.
    02 DEBUG-LINE     PICTURE IS X(6).
    02 FILLER         PICTURE IS X VALUE SPACE.
    02 DEBUG-NAME     PICTURE IS X(30).
    02 FILLER         PICTURE IS X VALUE SPACE.
    02 DEBUG-SUB-1    PICTURE IS S9999 SIGN IS LEADING SEPARATE CHARACTER.
    02 FILLER         PICTURE IS X VALUE SPACE.
    02 DEBUG-SUB-2    PICTURE IS S9999 SIGN IS LEADING SEPARATE CHARACTER.
    02 FILLER         PICTURE IS X VALUE SPACE.
    02 DEBUG-SUB-3    PICTURE IS S9999 SIGN IS LEADING SEPARATE CHARACTER.
    02 FILLER         PICTURE IS X VALUE SPACE.
    02 DEBUG-CONTENTS PICTURE IS X(n).
```

12. Prior to each execution of a debugging section, the contents of the data item referenced by DEBUG–ITEM are space filled. The contents of data items subordinate to DEBUG–ITEM are then updated, according to the following general rules, immediately before control is passed to that debugging section. The contents of any data item not specified in the following general rules remains spaces.

    Updating is accomplished in accordance with the rules for the MOVE statement. The sole exception is the move to DEBUG–CONTENTS, when the move is treated exactly as if it were an alphanumeric to alphanumeric elementary move with no conversion of data from one form of internal representation to another.

13. The content of DEBUG–LINE is the number of the source line which causes the execution of a debugging section.

14. DEBUG–NAME contains the first 30 characters of the name that caused the debugging section to be executed.

    All qualifiers of the name are separated in DEBUG–NAME by the word "IN" or "OF". Subscripts or indices, if any, are not entered into DEBUG–NAME.

15. If the reference to a data item that causes the debugging section to be executed is subscripted or indexed, the occurrence number of each level is entered in DEBUG–SUB–1, DEBUG–SUB–2, DEBUG–SUB–3, respectively.

16. DEBUG–CONTENTS is a data item that is large enough to contain the data required by the following general rules.

17. If the first execution of the first nondeclarative procedure in the program causes the debugging section to be executed, the following conditions exist:

    a. DEBUG–LINE identifies the first statement of that procedure.

    b. DEBUG–NAME contains the name of that procedure.

    c. DEBUG–CONTENTS contains "START PROGRAM".

18. If a reference to *procedure-name-1* in an ALTER statement causes the debugging section to be executed, the following conditions exist:
    a. DEBUG–LINE identifies the ALTER statement that references *procedure-name-1*.
    b. DEBUG–NAME contains *procedure-name-1*.
    c. DEBUG–CONTENTS contains the applicable procedure-name associated with the TO phrase of the ALTER statement.
19. If the transfer of control associated with the execution of a GO TO statement causes the debugging section to be executed, the following conditions exist:
    a. DEBUG–LINE identifies the GO TO statement whose execution transfers control to *procedure-name-1*.
    b. DEBUG–NAME contains *procedure-name-1*.
20. If reference to *procedure-name-1* in the INPUT or OUTPUT phrase a SORT or MERGE statement causes the debugging section to be executed, the following conditions exist:
    a. DEBUG–LINE identifies the SORT or MERGE statement that references *procedure-name-1*.
    b. DEBUG–NAME contains *procedure-name-1*.
    c. DEBUG–CONTENTS contains:
        - If the reference to *procedure-name-1* is in the INPUT phrase of a SORT statement, "SORT INPUT".
        - If the reference to *procedure-name-1* is in the OUTPUT phrase of a SORT statement, "SORT OUTPUT".
        - If the reference to *procedure-name-1* is in the OUTPUT phrase of a MERGE statement, "MERGE OUTPUT".
21. If the transfer to control from the control mechanism associated with a PERFORM statement caused the debugging section associated with *procedure-name-1* to be executed, the following conditions exist:
    a. DEBUG–LINE identifies the PERFORM statement that references *procedure-name-1*.
    b. DEBUG–NAME contains *procedure-name-1*.
    c. DEBUG–CONTENTS contains "PERFORM LOOP".
22. If *procedure-name-1* is a USE procedure that is to be executed, the following conditions exist:
    a. DEBUG–LINE identifies the statement that causes execution of the USE procedure.
    b. DEBUG–NAME contains *procedure-name-1*.
    c. DEBUG–CONTENTS contains "USE PROCEDURE".

23. If an implicit transfer of control from the previous sequential paragraph to *procedure-name-1* causes the debugging section to be executed, the following conditions exist:

    a. DEBUG–LINE identifies the previous statement.

    b. DEBUG–NAME contains *procedure-name-1*.

    c. DEBUG–CONTENTS contains "FALL THROUGH".

24. If references to *file-name-1*, *cd-name-1* causes the debugging section to be executed, then:

    a. DEBUG–LINE identifies the source statement that references *file-name-1*, *cd-name-1*.

    b. DEBUG–NAME contains the name of *file-name-1*, *cd-name-1*.

    c. For READ, DEBUG–CONTENTS contains the entire record read.

    d. For all other references to *file-name-1*, DEBUG–CONTENTS contains spaces.

    e. For any reference to *cd-name-1*, DEBUG–CONTENTS contains the contents of the area associated with the cd-name.

25. If a reference to *identifier-1* causes the debugging section to be executed, then:

    a. DEBUG–LINE identifies the source statement that references *identifier-1*,

    b. DEBUG–NAME contains the name of *identifier-1*, and

    c. DEBUG–CONTENTS contains the contents of the data item referenced by *identifier-1* at the time that control passes to the debugging section (see general rules 6 and 7).

11.3.2. Debugging Lines

A debugging line is any line with a "D" in the indicator area of the line. Any debugging line that consists solely of spaces from margin A to margin R is the same as a blank line.

The contents of a debugging line must be such that a syntactically correct program is formed with or without the debugging lines being considered as comment lines.

A debugging line will be considered to have all the characteristics of a comment line, if the WITH DEBUGGING MODE clause is not specified in the SOURCE–COMPUTER paragraph.

Successive debugging lines are allowed. Continuation of debugging lines is permitted, except that each continuation line must contain a "D" in the indicator area, and character-strings may not be broken across two lines.

A debugging line is permitted in the program only after the OBJECT–COMPUTER paragraph.

11.4. Table Error Facility

11.4.1. General

The Table Error facility allows the user to check for validity of table elements referenced in a program by specifying syntax, a compiler option, or both. A table error or subscript out of range occurs any time the subscript value exceeds the maximum number of occurrences of an item as specified in the OCCURS clause.

11.4.2. Compiler Option

11.4.2.1. Function

When the compiler extra option X is specified, a subscript range check will be performed for every table reference in the program.

11.4.2.2. General Rule

When the X extra option is specified, if the data item is not the object of a USE FOR TABLE ERROR procedure and the subscript of the data item is out of range, the name of the data item and its source line number will be displayed, and execution terminated.

11.4.3. Procedure Division

11.4.3.1. The DECLARATIVE Paragraph

11.4.3.1.1. Function

The USE FOR TABLE ERROR statement in the DECLARATIVE paragraph identifies the user table data items that are to be monitored by the associated table error section.

11.4.3.1.2. General Format



11.4.3.1.3. Syntax Rules

1. There must be no reference to any nondeclarative procedure within the table error section.
2. Statements appearing outside of the set of table error sections must not reference procedure-names defined within the set of table error sections.
3. Statements appearing within a given table error section may reference procedure-names defined within a different USE procedure only with a PERFORM statement.
4. *Identifier-1* may appear in only one USE FOR TABLE ERROR statement and may appear only once in that statement.

5. The ALL phrase can appear only once in a program, and when it is specified, *identifier-1* must not be specified in any USE FOR TABLE ERROR statement.
6. *Identifier-1* must not reference any data item in the Report Section.
7. The data description entry of the data item referenced by *identifier–1* must contain an OCCURS clause or be subordinate to a data description entry that contains an OCCURS clause. *Identifier-1* must be specified without the subscripting or indexing normally required.
8. The special register DEBUG–ITEM (see subsections 11.1.3.1 and 11.3.1.4, rule 11) may be referenced from within a table error section.

11.4.3.1.4. General Rules

1. Automatic execution of a table error section is not caused by a statement appearing in table error section.
2. If the data item is the object of a USE FOR TABLE ERROR procedure and the subscript is out of range, DEBUG–ITEM is filled, and then the USE FOR TABLE ERROR procedure is executed.
3. The ALL phrase causes the effects described in general rule 2 to occur for every table reference in the program except those appearing within a table error section.
4. Special register DEBUG–ITEM is filled as follows:

    a. DEBUG–LINE contains the number of the source line which caused the execution of the table error section.

    b. DEBUG–NAME contains the first 30 characters of the name that caused the table error section to be executed. All qualifiers of the name are separated in DEBUG–NAME by the word IN or OF.

    c. DEBUG–SUB–*n* contains the occurrence number of each level.

    d. DEBUG–CONTENTS contains "TABLE ERROR".

11.5. Programmers Advanced Debugging System (PADS 2200)

11.5.1. General

The Programmers Advanced Debugging System (PADS 2200) is a debugging tool which allows users to display and alter the contents of variables defined in their programs during the execution of the programs. It also allows users to set "traps" at section names, paragraph names, and at source lines containing executable statements. The "trap" causes program execution to be suspended when the section name, paragraph name, or line number is reached.

PADS 2200 contains a number of features in addition to these. A complete description of the PADS 2200 command language can be found in *OS 2200 Programmers Advanced Debugging System (PADS) Programming Guide* (7831 0562).

Users may invoke PADS by specifying syntax in the COBOL source language program or by entering a processor call statement immediately before the user program is executed. In either case, it is necessary to compile the source language program with a special option in order to use all the functions of PADS during program execution.

11.5.2. Compiler Option

11.5.2.1. Function

When the compiler extra option K is specified, the compiler generates symbolic debugging tables that enable PADS to locate and reference user-defined program entities during execution of the user program.

These tables include information describing the variables defined in the user program which PADS needs in order to display or alter the contents of the variables. The contents of variables are displayed or altered only upon user command.

11.5.2.2. General Rules

The K extra option should not be used with the J extra option. Should both these options be specified, the J extra option overrides the K extra option, and the compiler does not generate the symbolic debugging tables.

Limited PADS functionality is available within programs not compiled with the K extra option. In these cases, any PADS commands which reference section or paragraph names, variables, or source program line numbers generate PADS 2200 diagnostic messages.

11.5.3. Procedure Division

11.5.3.1. The CALL Statement

11.5.3.1.1. Function

The CALL statement in the procedure division may be used to force PADS to gain control of the user program at the point at which the CALL statement is executed.

11.5.3.1.2. General Format

```
CALL  'PADS$'
```

11.5.3.1.3. Syntax Rules

1. No arguments may be specified with this use of the CALL statement.
2. The ON OVERFLOW clause may not be specified with this use of the CALL statement.

11.5.3.1.4. General Rules

1. When the CALL statement is executed, execution of the user program is suspended, and PADS is given control. PADS prints a message identifying itself and then solicits the user for commands.
2. This form of the CALL statement should not be used in source programs that use the COBOL language segmentation module or that use any features of the Asynchronous Control System.
3. PADS must not be invoked from COBOL subprograms that are called from a main program written in ASCII FORTRAN, PL/I, Fieldata FORTRAN, Fieldata COBOL, or ASM.
4. PADS may be invoked from COBOL subprograms that are called from a main program written in MASM. The MASM main program must preserve the contents of registers R1 through R3, R11 through R15, and A4 and A5, that were set by the Executive when the user program was loaded into storage.
5. PADS may not be used with programs which run in the TIP or HVTIP environment (see *OS 1100 ASCII COBOL, Installation Reference Manual*, UP–8585).
6. PADS can be given control during execution of the user program without the need for any changes to the source language program. See *OS 2200 Programmers Advanced Debugging System (PADS) Programming Guide* (7831 0562) for a description of the manner in which this can be done.

Section 12
Interprogram Communication

12.1. General

12.1.1. Function

The Interprogram Communication module provides a facility by which a program can communicate with one or more programs. This communication is provided by the ability to transfer control from one program to another within a run unit and the ability for both programs to have access to the same data items.

12.1.2. Level Characteristics

Interprogram Communication Level 1 provides a capability to transfer control to one or more programs whose names are known at compile time and for the sharing of data among such programs.

Additionally, Interprogram Communication Level 2 provides the capability to transfer control to one or more programs whose names are not known at compile time as well as the ability to determine the availability of object time memory for the program to which control is being passed.

12.2. Environment Division

12.2.1. The I–O–CONTROL Paragraph

12.2.1.1. The APPLY Clause

12.2.1.1.1. Function

The APPLY clause in the I–O–CONTROL paragraph specifies which compiler-generated tables and data areas are to be shared during execution among two or more separately compiled programs.

12.2.1.1.2. General Format

```
[ { APPLY { EXREF } ON { identifier-1 } ... } ... ]
          { EXDEF }
```

12.2.1.1.3. Syntax Rules

1. *Identifier-1* ..., must specify a cd-name, file-name, sa-name, or Working-Storage level 01 data-name, and must be unique in the first 12 characters.
2. An identifier may be the object of only one APPLY EXDEF clause in a run unit.
3. An identifier may occur only once in an APPLY clause of a program.
4. An identifier may be the object of more than one APPLY EXREF clause in a run unit.
5. An identifier in an APPLY EXREF clause must specify the same type of item as was specified in its associated APPLY EXDEF clause.

12.2.1.1.4. General Rules

1. The tables and data areas associated with the identifier in the APPLY EXDEF clause are accessible by programs that specify the same identifier in an APPLY EXREF clause.
2. The descriptions associated with each identifier in the APPLY EXREF clause must be equivalent to the descriptions associated with that same identifier when specified in an APPLY EXDEF clause.
3. All declarative procedures and Report Descriptions associated with a file must be specified in the same program that has the APPLY EXDEF for that file.
4. The APPLY EXDEF clause must be specified in the main program for any file that potentially invokes a declarative procedure or has a Report attached to it.
5. The identifier specified in the APPLY EXREF clause must not have a REDEFINES clause at the record level. It can, however, be the object of a REDEFINES clause and can contain subordinate items that are redefined.

6. The identifier specified in the APPLY EXREF clause must define the same number of character positions as the same identifier when specified in the APPLY EXDEF clause. It is important to note that the APPLY EXREF clause specifies the redefinition of a storage area, not of the data items occupying the area.
7. When APPLY EXREF is specified, the VALUE clause must not be present in that entry.
8. The APPLY EXDEF clause must be specified in the main program for any IBM compatible tape file which contains an OCCURS DEPENDING ON data item.
9. If a file specified in the APPLY EXDEF clause has a FILE STATUS associated with it, one of the following rules should be used to insure proper reference to that data item in any subprogram.

   a. Define the FILE STATUS data item in the COMMON–STORAGE SECTION of all programs that reference it.
   b. If the file is EXDEF'ed in the main program, define the FILE STATUS in WORKING–STORAGE SECTION. In the subprograms that reference it, define it in the LINKAGE SECTION.
   c. EXDEF the FILE STATUS item in the program where the file is EXDEF'ed, and EXREF that data item in any program where the file is EXREF'ed. In this case, the FILE STATUS clause need not be specified in the SELECT statement of the EXREF'ing programs.

12.3. Data Division

12.3.1. Linkage Section

The Linkage Section in a program is meaningful if, and only if, the object program is to function under the control of a CALL statement, and the CALL statement in the calling program contains a USING phrase.

The Linkage Section is used for describing data that is available through the calling program but is to be referred to in both the calling and the called program. No space is allocated in the program for data items referenced by data-names in the Linkage Section of that program. Procedure Division references to these data items are resolved at object time by equating the reference in the called program to the location used in the calling program. In the case of index-names, no such correspondence is established. Index-names in the called and calling program always refer to separate indices.

Data items defined in the Linkage Section of the called program may be referenced within the Procedure Division of the called program only if they are specified as operands of the USING phrase of the Procedure Division header or are subordinate to such operands, and the object program is under the control of a CALL statement that specifies a USING phrase.

The structure of the Linkage Section is the same as that previously described for the Working-Storage Section, beginning with a section header, followed by data description entries for noncontiguous data items and record description entries.

Each Linkage Section record-name and noncontiguous item name must be unique within the called program since it cannot be qualified. Data items defined in the Linkage Section of the called program must not be associated with data items defined in the Report Section of the calling program.

Of those items defined in the Linkage Section only *data-name-1*, *data-name-2*, ..., in the USING phrase of the Procedure Division header, data items subordinate to these data-names, and condition-names and/or index-names associated with such data-names and/or subordinate data items, may be referenced in the Procedure Division.

12.3.1.1. Noncontiguous Linkage Storage

Items in the Linkage Section that bear no hierarchic relationship to one another need not be grouped into records and are classified and defined as noncontiguous elementary items. Each of these data items is defined in a separate data description entry which begins with the special level-number 77.

The following data clauses are required in each data description entry:

1. Level-number 77
2. data-name
3. the PICTURE clause or the USAGE IS INDEX, COMPUTATIONAL–1, COMPUTATIONAL–2, or LOCK clause.

Other data description clauses are optional and can be used to complete the description of the item if necessary.

12.3.1.2. Linkage Records

Data elements in the Linkage Section which bear a definite hierarchic relationship to one another must be grouped into records according to the rules for formation of record descriptions. Any clause which is used in an input or output record description can be used in a Linkage Section.

12.3.1.3. Initial Values

The VALUE clause must not be specified in the Linkage Section except in condition–name entries (level 88).

12.3.2. Common-Storage Section

The Common-Storage Section is a block of main storage which is shared by all collected COBOL programs which define a Common-Storage Section.

The structure and permissible data definition statements for Common-Storage are identical to those for the Working-Storage Section.

If Common-Storage Sections are different sizes, the total area allocated for an execution is sufficient for the largest Common-Storage Section.

The main storage area shared by two or more COBOL Common-Storage Sections may also be shared by assembly language blank common which is specified by the INFO directive.

The common areas may be used to conserve main storage by providing a "scratch pad" area which each main program and subprogram may use for itself. Or it may be used to allow one program to directly address data that belongs to another program. If Common-Storage is used for this purpose, the user must take the necessary actions to ensure that data field allocation and alignment is consistent among all programs that share the area. The use of the COPY statement for this function is recommended.

When using Common-Storage in nonsegmented programs it is advisable to have all Common-Storage areas defined identically, including initial values. This is because the last one collected is the one whose Common-Storage values will be loaded.

When compiling with a V option on the processor call statement and a 1 option in the Extra Options field, no initial values will be generated for the subprogram's Common-Storage area. If a "1" is not specified in the Extra Options field of the processor call statement with a V option, the subprogram's Common-Storage area will have its initial values generated. If these values are contained in a secondary segment they will overlay the main program's Common-Storage values when the subprogram is called and subsequently loaded.

12.4. Procedure Division

12.4.1. The Procedure Division Header

The Procedure Division is identified by and must begin with the following header:

```
PROCEDURE DIVISION

     [ USING data-name-1 [ , data-name-2 ] ... ]

     [ WITH ENTRY POINTS { procedure-name-1 } ... ] .
```

The USING phrase is present if and only if the object program is to function under the control of a CALL statement, and the CALL statement in the calling program contains a USING phrase.

Each of the operands in the USING phrase of the Procedure Division header must be defined as a data item in the Linkage Section of the program in which this header occurs, and it must have a 01 or 77 level-number.

Within a called program, Linkage Section data items are processed according to their data descriptions given in the called program.

When the USING phrase is present, the object program operates as if *data-name-1* of the Procedure Division header in the called program and *data-name-1* in the USING phrase of the CALL statement in the calling program refer to a single set of data that is equally available to both the called and calling programs. Their descriptions must define an equal number of character positions; however, they need not be the same name. In like manner, there is an equivalent relationship between *data-name-2*, ..., in the USING phrase of the called program and *data-name-2*, ..., in the USING phrase of the CALL statement in the calling program. A data–name must not appear more than once in the USING phrase in the Procedure Division header of the called program; however, a given data-name may appear more than once in the same USING phrase of a CALL statement.

If the USING phrase is specified, the INITIAL clause must not be present in any CD entry. (See syntax rule 2 of 13.2.2.3.)

A COBOL program which may be called by another program within the same run unit is normally executed starting with the first nondeclarative paragraph in the program. To have execution begin at other than the first nondeclarative statement, the ENTRY POINTS phrase is required.

A maximum of 31 data-names may be specified. The combined number of procedure-names and items referenced in the APPLY EXDEF clause must not exceed 100. The procedure-names may be any section-name or paragraph-name that is not contained in the declaratives portion of a program, so long as the procedure-names are not qualified and are unique in the first three characters and the last six characters. The specified procedure-names must not conflict with the program-name or with entries in the APPLY clause. In addition, procedure-names on the ENTRY POINTS phrase must not conflict with the procedure-names on the ENTRY POINTS phrase, program-names, or entries in the APPLY clause for other programs within the run unit.

The execution of a CALL or ENTER statement which references procedure-name on the ENTRY POINTS phrase or program-name of another program transfers control to the first executable statement following the specified name (or the first nondeclarative statement in the program).

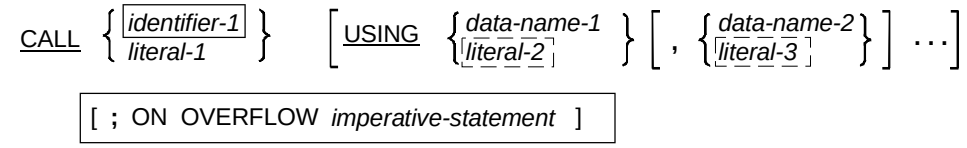
The use of entry-point-names does not permit recursive calling of a subprogram.

12.4.2. The CALL Statement

12.4.2.1. Function

The CALL statement causes control to be transferred from one object program to another, within the run unit.

12.4.2.2. General Format



12.4.2.3. Syntax Rules

1. *Literal-1* must be a nonnumeric literal. This literal must be equal to a program-name or a procedure-name specified on the ENTRY POINTS phrase.
2. *Identifier-1* must be defined as an alphanumeric data item such that its value can be a program-name or a procedure-name specified on the ENTRY POINTS phrase.
3. The USING phrase is included in the CALL statement only if there is a USING phrase in the Procedure Division header of the called program and the number of operands in each USING phrase must be identical.
4. Each of the operands in the USING phrase must have been defined as a data item in the File Section, Working-Storage Section, Communication Section, or Linkage Section, and must have a level-number of 01 or 77. *Data-name-1*, *data-name-2*, ..., may be qualified when they reference data items defined in the File Section or the Communication Section. A maximum of 31 arguments may be specified.

12.4.2.4. General Rules

1. The program whose name is specified by the value of *literal-1* or *identifier-1* is the called program. The program in which the CALL statement appears is the calling program.
2. The execution of a CALL statement causes control to pass to the called program.
3. A called program is in its initial state the first time it is called within a run unit and the first time it is called after a CANCEL to the called program.

   On all other entries into the called program, the state of the program remains unchanged from its state when last exited. This includes all data fields, the status and positioning of all files, and all alterable switch settings.

4. If, during the execution of a CALL statement, the available portion of object time storage is incapable of accommodating the program specified in the CALL statement and the ON OVERFLOW phrase is specified, no action is taken and the *imperative-statement* is executed.

   If this condition exists and the ON OVERFLOW phrase is not specified, a run-time diagnostic will be issued.

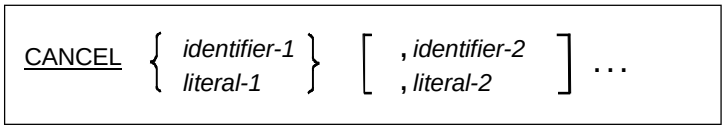
5. Called programs may contain CALL statements. However, a called program must not contain a CALL statement that directly or indirectly calls the calling program.
6. The data-names, specified by the USING phrase of the CALL statement, indicate those data items available to a calling program that may be referred to in the called program. The order of appearance of the data-names in the USING phrase of the CALL statement and the USING phrase in the Procedure Division header is critical. Corresponding data-names refer to a single set of data which is available to the called and calling program. The correspondence is positional, not by name. In the case of index-names, no such correspondence is established. Index-names in the called and calling program always refer to separate indices.
7. The CALL statement may appear anywhere within a segmented program. When a CALL statement appears in a section with a segment–number greater than or equal to 50, that segment is in its last used state when the EXIT PROGRAM statement returns control to the calling program.
8. When *identifier-1* is used, an element named C$S333 must be collected with the object module. This element must be furnished by the user and contains a list of all valid routines to be called. See 18.7.1.

12.4.3. The CANCEL Statement

12.4.3.1. Function

The CANCEL statement releases the memory areas occupied by the referred to program.

12.4.3.2. General Format



12.4.3.3. Syntax Rules

1. *Literal-1*, *literal-2*, ..., must each be a nonnumeric literal. This literal must be equal to a program-name. This literal must not equal a procedure-name specified on the ENTRY POINTS phrase.
2. *Identifier-1*, *identifier–2*, ..., must each be defined as an alphanumeric data item such that its value can be a program-name. *Identifier-1*, *identifier-2*, ..., must not contain a value that equals a procedure–name specified on the ENTRY POINTS phrase.

12.4.3.4. General Rules

1. After the execution of a CANCEL statement, the program cancelled no longer has any logical relationship to the run unit in which the CANCEL statement appears. A subsequently executed CALL statement naming the same program will result in that program being initiated in its initial state. The storage areas associated with the named programs are released so as to be made available for disposition by the operating system. Files referenced in the APPLY EXREF clause are not affected.
2. A program named in the CANCEL statement must not refer to any program that has been called and has not yet executed an EXIT PROGRAM statement.
3. A logical relationship to a cancelled subprogram can be established only by execution of a subsequent CALL statement.
4. A called program is cancelled either by being referred to as the operand of a CANCEL statement or by the termination of the run unit of which the program is a member.
5. No action is taken when a CANCEL statement is executed naming a program that has not been called in this run unit or has been called and is at present cancelled. Control passes to the next statement.
6. The CANCEL statement has no effect on Common-Storage data-items.
7. All files defined in the program named in the CANCEL statement must be closed prior to execution of the CANCEL statement.
8. Any subprogram being cancelled must reside in core prior to the execution of the CANCEL statement.

12.4.4. The ENTER Statement

12.4.4.1. Function

The ENTER statement transfers control to a non-ASCII COBOL program within the same run unit.

12.4.4.2. General Format

Format 1

```
ENTER { MASM | ASM | PL1 | FTN } literal [ USING { argument-1 } ... ]
```

Format 2

```
ENTER MASM "C$INFO" USING file-name, data-name.
```

12.4.4.3. Syntax Rules

Format 1

1. *Literal* must conform to the requirements for entry-points for the languages specified.
2. A maximum of 31 arguments may be specified. However, if ENTER ASM is used, only 11 of these arguments can be from the Linkage Section, and if the program is over 65K, only 10 of the arguments can be from the Linkage Section.
3. Arguments for MASM or ASM may be literals, identifiers, and procedure-names.
   a. Numeric literals are assumed to be fixed-point binary, right justified in single or double precision as necessary. Floating-point literals are assumed to be double precision floating-point, and nonnumeric literals are assumed to be ASCII.
   b. Identifiers may be any data items which are referred to in the calling program. Level 88 items may not be used as arguments. Subscripts and indexes must be integer values.
   c. Procedure-names should contain at least one nonnumeric character to avoid conflict with numeric literal arguments.
4. Arguments for PL1 may be elementary or group items with DISPLAY, COMPUTATIONAL (start on a word boundary and be four or eight bytes in length), COMPUTATIONAL-1, COMPUTATIONAL-2, or Exact Binary Usage.
5. Arguments for FTN may be elementary or group items with DISPLAY, COMPUTATIONAL (start on a word boundary and be four bytes in length), COMPUTATIONAL-1, or COMPUTATIONAL-2 Usage. The arguments can also be COBOL paragraph-names.

12.4.4.4. General Rules

Format 1

1. The program whose entry-point is specified by the value of the literal is the called program; the program in which the ENTER statement appears is the calling program.
2. Execution of an ENTER statement causes control to pass to the called program.
3. A called program must not contain a statement that directly or indirectly calls the calling program.
4. See Section 18 for a description of preserving the COBOL environment, calling sequences, and the parameter format.

Format 2

Refer to 6.1.4 for additional information on this format.

12.4.5. The EXIT PROGRAM Statement

12.4.5.1. Function

The EXIT PROGRAM statement marks the logical end of a called program.

12.4.5.2. General Format

```
EXIT PROGRAM.
```

12.4.5.3. Syntax Rules

1. The EXIT PROGRAM statement must appear in a sentence by itself.
2. The EXIT PROGRAM sentence must be the only sentence in the paragraph.

12.4.5.4. General Rules

An execution of an EXIT PROGRAM statement in a called program causes control to be passed to the calling program. Execution of an EXIT PROGRAM statement in a program which is not called behaves as if the statement were an EXIT statement. (See 2.6.10.)

Section 13
Communication

13.1. General

13.1.1. Function

The Communication module provides the ability to access, process, and create messages or portions thereof. It provides the ability to communicate through a Message Control System (MCS) with local and remote communications devices. (See 1.2.8 for a general discussion of MCS concepts.)

13.1.2. Level Characteristics

Communication Level 1 does not provide the full COBOL facility for the CD entry as specified in the formats for this module. In the Procedure Division, Level 1 provides limited capabilities for the ENABLE, DISABLE, RECEIVE, and SEND statements, as specified in the formats of this module. There is also a provision for determining the number of messages in an input queue.

Communication Level 2 provides the full facility for the CD entry as specified in the formats of this module. Within the Procedure Division, full capabilities are provided for the ENABLE, DISABLE, RECEIVE, and SEND statements, as specified in the formats for this module. The additional features available in Level 2 include: partial messages, segmented messages, multiple destination message processing, and program invocation by the MCS as specified by the INITIAL CD.

13.2. Data Division

13.2.1. Communication Section

In a COBOL program the communication description entries (CD) represent the highest level of organization in the Communication Section. The Communication Section header is followed by a communication description entry consisting of a level indicator (CD), a data-name and a series of independent clauses. These clauses indicate the queues and sub-queues, the message date and time, the source, the text length, the status and end keys, and message count of input. These clauses specify the destination count, the text length, the status and error keys, and destinations for output. The entry itself is terminated by a period. These record areas may be implicitly redefined by user-specified record description entries following the various communication description clauses.

13.2.2. The Communication Description—Complete Entry Skeleton

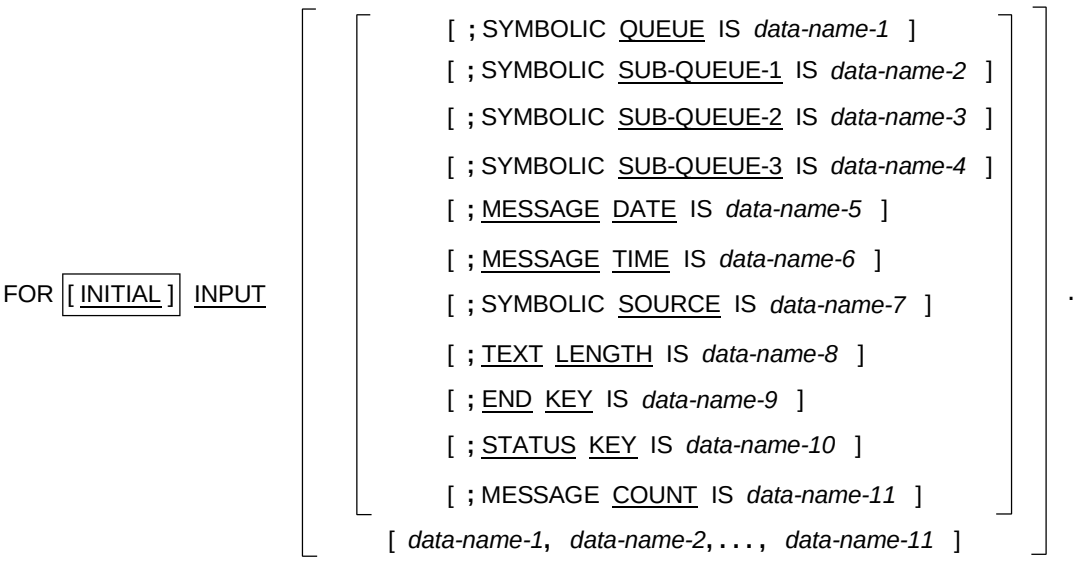
13.2.2.1. Function

The communication description (CD) specifies the interface area between the MCS and a COBOL program.

13.2.2.2. General Format

Format 1

CD *cd-name* ;



Format 2

```
CD  cd-name ;    FOR  OUTPUT

     [ ; DESTINATION  COUNT  IS  data-name-1  ]
     [ ; TEXT  LENGTH  IS  data-name-2  ]
     [ ; STATUS  KEY  IS  data-name-3  ]
     [ ; DESTINATION  TABLE  OCCURS  integer-2  TIMES
          [ ; INDEXED  BY  index-name-1  [ , index-name-2  ] ... ] ]
     [ ; ERROR  KEY  IS  data-name-4  ]
     [ ; SYMBOLIC  DESTINATION  IS  data-name-5  ].
```

13.2.2.3. Syntax Rules

Format 1

1. A CD must appear only in the Communication Section.
2. Within a single program, the INITIAL clause may be specified in only one CD. The INITIAL clause must not be used in a called program. (See 12.4.1.)
3. Except for the INITIAL clause, the optional clauses may be written in any order.
4. If neither option in the format is specified, a level 01 data description entry must follow the CD description entry. Either option may be followed by a level 01 data description entry.
5. For each input CD, a record area of 87 contiguous standard data format characters is allocated. This record area is defined to the MCS as follows:
   a. The SYMBOLIC QUEUE clause defines *data-name-1* as the name of an elementary alphanumeric data item of 12 characters occupying positions 1-12 in the record.
   b. The SYMBOLIC SUB–QUEUE–1 clause defines *data-name-2* as the name of an elementary alphanumeric data item of 12 characters occupying positions 13-24 in the record.
   c. The SYMBOLIC SUB–QUEUE–2 clause defines *data-name-3* as the name of an elementary alphanumeric data item of 12 characters occupying positions 25-36 in the record.
   d. The SYMBOLIC SUB–QUEUE–3 clause defines *data-name-4* as the name of an elementary alphanumeric data item of 12 characters occupying positions 37-48 in the record.
   e. The MESSAGE DATE clause defines *data-name-5* as the name of a data item whose implicit description is that of an integer of 6 digits without an operational sign occupying character positions 49-54 in the record.
   f. The MESSAGE TIME clause defines *data-name-6* as the name of a data item whose implicit description is that of an integer of 8 digits without an operational sign occupying character positions 55-62 in the record.

g. The SYMBOLIC SOURCE clause defines *data-name-7* as the name of an elementary alphanumeric data item of 12 characters occupying positions 63-74 in the record.

h. The TEXT LENGTH clause defines *data-name-8* as the name of an elementary data item whose implicit description is that of an integer of 4 digits without an operational sign occupying character positions 75-78 in the record.

i. The END KEY clause defines *data-name-9* as the name of an elementary alphanumeric data item of 1 character occupying position 79 in the record.

j. The STATUS KEY clause defines *data-name-10* as the name of an elementary alphanumeric data item of 2 characters occupying positions 80-81 in the record.

k. The MESSAGE COUNT clause defines *data-name-11* as the name of an elementary data item whose implicit description is that of an integer of 6 digits without an operational sign occupying character positions 82-87 in the record.

The second option may be used to replace the preceding clauses by a series of data-names which, taken in order, correspond to the data-names defined by these clauses.

Use of either option results in a record whose implicit description is equivalent to the following:

Implicit Description			Comment
01	*data-name-0.*		
02	*data-name-1*	PICTURE X(12).	(symbolic queue)
02	*data-name-2*	PICTURE X(12).	(symbolic sub-queue-1)
02	*data-name-3*	PICTURE X(12).	(symbolic sub-queue-2)
02	*data-name-4*	PICTURE X(12).	(symbolic sub-queue-3)
02	*data-name-5*	PICTURE 9(06).	(message date)
02	*data-name-6*	PICTURE 9(08).	(message time)
02	*data-name-7*	PICTURE X(12).	(symbolic source)
02	*data-name-8*	PICTURE 9(04).	(text length)
02	*data-name-9*	PICTURE X.	(end key)
02	*data-name-10*	PICTURE XX.	(status key)
02	*data-name-11*	PICTURE 9(06).	(message count)

Note: *In the preceding list, the comments within parentheses are for clarification and are not part of the description.*

6. Record description entries following an input CD implicitly redefine the record and must describe a record of exactly 87 characters. Multiple redefinitions of this record are permitted; however, only the first redefinition may contain VALUE clauses. However, the MCS will always reference the record according to the data descriptions defined in syntax rule 5.

7. *Data-name-1, data-name-2, ..., data-name-11* must be unique within the CD. Within this series, any data-name may be replaced by the reserved word FILLER.

Format 2

8. A CD must appear only in the Communication Section.
9. If none of the optional clauses of the CD is specified, a level 01 data description entry must follow the CD description entry.
10. For each output CD, a record area of contiguous standard data format characters is allocated according to the following formula: (10 plus 13 times *integer-2*).
    a. The DESTINATION COUNT clause defines *data-name-1* as the name of a data item whose implicit description is that of an integer without an operational sign occupying character positions 1– 4 in the record.
    b. The TEXT LENGTH clause defines *data-name-2* as the name of an elementary data item whose implicit description is that of an integer of 4 digits without an operational sign occupying character positions 5 – 8 in the record.
    c. The STATUS KEY clause defines *data-name-3* to be an elementary alphanumeric data item of 2 characters occupying positions 9 – 10 in the record.
    d. Character positions 11 – 23 and every set of 13 characters thereafter will form table items of the following description:
        - The ERROR KEY clause defines *data-name-4* as the name of an elementary alphanumeric data item of one character.
        - The SYMBOLIC DESTINATION clause defines *data-name-5* as the name of an elementary alphanumeric data item of 12 characters.

    Use of these clauses results in a record whose implicit description is equivalent to the following:

```
        Implicit Description                          Comment

01 data-name-0.
    02  data-name-1  PICTURE 9(04).                (destination count)
    02  data-name-2  PICTURE 9(04).                (text length)
    02  data-name-3  PICTURE XX.                   (status key)
    02  data-name  OCCURS integer-2 TIMES.         (destination table)
        03 data-name-4  PICTURE X.                  (error key)
        03 data-name-5  PICTURE X(12).              (symbolic destination)
```

Note: *In the preceding list, the comments within parentheses are for clarification and are not part of the description.*

11. Record descriptions following an output CD implicitly redefine this record. Multiple redefinitions of this record are permitted; however, only the first redefinition may contain VALUE clauses. However, the MCS will always reference the record according to the data descriptions defined in syntax rule 10.
12. *Data-name-1, data-name-2, ..., data-name-5* must be unique within a CD.
13. If the DESTINATION TABLE OCCURS clause is not specified, one ERROR KEY and one SYMBOLIC DESTINATION area is assumed. In this case, neither subscripting nor indexing is permitted when referencing these data items.
14. If the DESTINATION TABLE OCCURS clause is specified, *data-name-4* and *data-name-5* may only be referred to by subscripting or indexing.

15. In Level 1, the value of the data item referenced by *data-name-1* and *integer-2* must be 1. In Level 2, there is no restriction on the value of the data item referenced by *data-name-1* and *integer-2*.

13.2.2.4. General Rules

Format 1

1. The input CD information constitutes the communication between the MCS and the program as information about the message being handled. This information does not come from the terminal as part of the message.
2. The contents of the data items referenced by *data-name-2, data-name-3,* and *data-name-4,* when not being used must contain spaces.
3. The data items referenced by *data-name-1, data-name-2, data-name-3,* and *data-name-4* contain symbolic names designating queues, sub-queues, ..., respectively. All symbolic names must follow the rules for the formation of system-names, and must have been previously defined to the MCS.
4. A RECEIVE statement causes the serial return of the "next" message or a portion of a message from the queue as specified by the entries in the CD.

   If during the execution of a RECEIVE statement, a message from a more specific source is needed, the contents of the data item referenced by *data-name-1* can be made more specific by the use of the contents of the data items referenced by *data-name-2, data-name-3,* and in turn *data-name-4*. When a given level of the queue structure is specified, all higher levels must also be specified.

   If less than all the levels of the queue hierarchy are specified, the MCS determines the "next" message or portion of a message to be accessed.

   After the execution of a RECEIVE statement, the contents of the data items referenced by *data-name-1* through *data-name-4* will contain the symbolic names of all the levels of the queue structure.
5. Whenever a program is scheduled by the MCS to process a message, the symbolic names of the queue structure that demanded this activity will be placed in the data items referenced by *data-name-1* through *data-name-4* of the CD associated with the INITIAL clause as applicable. In all other cases, the contents of the data items referenced by *data-name-1* through *data-name-4* of the CD associated with the INITIAL clause are initialized to spaces.

   The symbolic names are inserted or the initialization to spaces is completed prior to the execution of the first Procedure Division statement.

   The execution of a subsequent RECEIVE statement naming the same contents of the data items referenced by *data-name-1* through *data-name-4* will return the actual message that caused the program to be scheduled. Only at that time will the remainder of the CD be updated.

6. If the MCS attempts to schedule a program lacking an INITIAL clause, the results are undefined.

7. *Data-name-5* has the format "YYMMDD" (year, month, day). Its contents represent the date on which the MCS recognizes that the message is complete.

   The contents of the data item referenced by *data-name-5* are only updated by the MCS as part of the execution of a RECEIVE statement.

8. The contents of *data-name-6* has the format "HHMMSSTT" (hours, minutes, seconds, hundredths of a second) and its contents represent the time at which the MCS recognizes that the message is complete.

9. The contents of the data item referenced by *data-name-6* are only updated by the MCS as part of the execution of the RECEIVE statement.

10. During the execution of a RECEIVE statement, the MCS provides, in the data item referenced by *data-name-7,* the symbolic name of the communications terminal that is the source of the message being transferred. However, if the symbolic name of the communication terminal is not known to the MCS, the contents of the data item referenced by *data-name-7* will contain spaces.

11. The MCS indicates via the contents of the data item referenced by *data-name-8* the number of character positions filled as a result of the execution of the RECEIVE statement. (See 13.3.4.)

12. The contents of the data item referenced by *data-name-9* are set only by the MCS as part of the execution of a RECEIVE statement according to the following rules:

    a. When the RECEIVE MESSAGE phrase is specified, then:

       - If an end of group has been detected, the contents of the data item referenced by *data-name-9* are set to 3.
       - If an end of message has been detected, the contents of the data item referenced by *data-name-9* are set to 2.
       - If less than a message is transferred, the contents of the data item referenced by *data-name-9* are set to 0.

    b. When the RECEIVE SEGMENT phrase is specified, then:

       - If an end of group has been detected, the contents of the data item referenced by *data-name-9* are set to 3.
       - If an end of message has been detected, the contents of the data item referenced by *data-name-9* are set to 2.
       - If an end of segment has been detected, the contents of the data item referenced by *data-name-9* are set to 1.
       - If less than a message segment is transferred, the contents of the data item referenced by *data-name-9* are set to 0.

    c. When more than one of the above conditions is satisfied simultaneously, the rule first satisfied in the order listed determines the contents of the data item referenced by *data-name-9.*

12. The contents of the data item referenced by *data-name-10* indicate the status condition of the previously executed RECEIVE, ACCEPT MESSAGE COUNT, ENABLE INPUT, or DISABLE INPUT statement.

    The actual association between the contents of the data item referenced by *data-name-10* and the status condition itself is defined in Table 13-1.

13. The contents of the data item referenced by *data-name-11* indicate the number of messages that exist in a queue, sub-queue-1, … The MCS updates the contents of the data item referenced by *data-name-11* only as part of the execution of an ACCEPT statement with the COUNT phrase.

Format 2

14. The output CD information is not sent to the terminal, but constitutes the communication between the program and the MCS as information about the message being handled.

15. During the execution of a SEND, ENABLE OUTPUT, or DISABLE OUTPUT statement, the contents of the data item referenced by *data-name-1* will indicate to the MCS the number of symbolic destinations that are to be used from the area referenced by *data-name-5.*

    The MCS finds the first symbolic destination in the first occurrence of the area referenced by *data-name-5,* the second symbolic destination in the second occurrence of the area referenced by *data-name-5* ..., up to and including the occurrence of the area referenced by *data-name-5* indicated by the contents of *data-name-1.*

    If during the execution of a SEND, ENABLE OUTPUT, or DISABLE OUTPUT statement the value of the data item referenced by *data-name-1* is outside the range of 1 through *integer-2,* an error condition is indicated and the execution of the SEND, ENABLE OUTPUT, or DISABLE OUTPUT statement is terminated.

16. The user must insure that the value of the data item referenced by *data-name-1* is valid at the time of execution of the SEND, ENABLE OUTPUT, or DISABLE OUTPUT statement.

17. As part of the execution of a SEND statement, the MCS will interpret the contents of the data item referenced by *data-name-2* to be the user's indication of the number of leftmost character positions of the data item referenced by the associated SEND identifier from which data is to be transferred. (See 13.3.5.)

18. Each occurrence of the data item referenced by *data-name-5* contains a symbolic destination previously known to the MCS. These symbolic destination names must follow the rules for the formation of system-names.

19. The contents of the data item referenced by *data-name-3* indicate the status condition of the previously executed SEND, ENABLE OUTPUT, or DISABLE OUTPUT statement.

    The actual association between the contents of the data item referenced by *data-name-3* and the status condition itself is defined in Table 13-1.

20. If, during the execution of a SEND, an ENABLE OUTPUT, or a DISABLE OUTPUT statement, the MCS determines that any specified destination is unknown, the contents of the data item referenced by *data-name-3* and all occurrences of the data items referenced by *data-name-4* are updated.

The contents of the data item referenced by *data-name-4* when equal to 1 indicate that the associated value in the area referenced by *data-name-5* has not been previously defined to the MCS. The contents of the data item referenced by *data-name-4* when equal to "A" indicate that the associated value referenced by *data-name-5* has been previously disabled. Otherwise, the contents of the data item referenced by *data-name-4* are set to zero.

All Formats

21. For Level 1, Table 13-1 indicates the possible contents of the data items referenced by *data-name-10* for Format 1 and by *data-name-3* for Format 2 at the completion of each statement shown. An "X" on a line in a statement column indicates that the associated code shown for that line is possible for that statement.

Table 13–1. Communication Status Key Condition in Level 1

RECEIVE	SEND	ACCEPT MESSAGE COUNT	ENABLE INPUT (without TERMINAL)	ENABLE OUTPUT	DISABLE INPUT (without TERMINAL)	DISABLE OUTPUT	STATUS KEY CODE	
X	X	X	X	X	X	X	00	No error detected. Action completed.
	X						10	Destination is disabled. Action completed
			X	X	X	X	15	One or more queues or destinations already disabled/enabled
	X			X		X	20	Destination unknown. No action taken for unknown destination. *Data-name-4* (ERROR KEY) *indicates* unknown
X		X	X		X		20	One or more queues or sub-queues unknown. No action taken.
	X			X		X	30	Content of DESTINATION COUNT invalid. No action taken.
			X	X	X	X	40	Password invalid. No enabling/disabling action taken.
	X						50	character count greater than length of spending field. No action taken.
	X			X		X	80	A combination of at least two status key conditions 10, 15, and 20 has occurred.
	X						90	Output queue capacity exceeded.

NOTE: *STATUS KEY codes 15, 21, 80, and 90 are items defined by Unisys.*

22. For Level 2, Table 13–2 indicates the possible contents of the data items referenced by *data-name-10* for Format 1 and by *data-name-3* for Format 2 at the completion of each statement shown. An "X" on a line in a statement column indicates that the associated code shown for that line is possible for that statement.

Table 13–2. Communication Status Key Condition in Level 2

RECEIVE	SEND	ACCEPT MESSAGE COUNT	ENABLE INPUT (without TERMINAL)	ENABLE INPUT (with TERMINAL)	ENABLE OUTPUT	DISABLE INPUT (without TERMINAL)	DISABLE INPUT (with TERMINAL)	DISABLE OUTPUT	STATUS KEY CODE	
X		X	X	X	X	X	X	X	00	No error detected. Action completed.
									10	One or more destinations are disabled. Action Completed.
			X	X	X	X	X	X	15	One or more queues or destination already disabled/enabled.
	X					X		X	20	One or more destindation unknown. Action completed for known destinations. No action taken for unknown destinations. *Data-name-4.* (ERROR KEY) *indicates known or unknown.*
X		X	X			X			20	One ir more queues or sub-queues unknown. No action taken.
				X			X		21	Symbolic source is unkown. No action taken.
	X				X			X	30	Content of DESTINATION COUNT invalid. No action taken.
			X	X	X	X	X	X	40	Password invalid. No enabling/disabling action taken.
	X								50	Character count greater than length of sending field. No action taken.
	X								60	Partial segment with either zero character count or no sneding area specified. No action taken.
	X				X			X	80	A combination of at least two status key conditions 10, 15, and 20 has occurred.
	X								90	Output queue capacity exceeded.

NOTE: *STATUS KEY codes 15, 21, 80, and 90 are items defined by Unisys.*

13.3. Procedure Division

13.3.1. The ACCEPT MESSAGE COUNT Statement

13.3.1.1. Function

The ACCEPT MESSAGE COUNT statement causes the number of messages in a queue to be made available.

13.3.1.2. General Format

```
ACCEPT cd-name MESSAGE COUNT
```

13.3.1.3. Syntax Rules

The parameter *cd-name* must reference an input CD.

13.3.1.4. General Rules

1. The ACCEPT MESSAGE COUNT statement causes the MESSAGE COUNT field specified for *cd-name* to be updated to indicate the number of messages that exist in a queue, sub-queue-1, ...
2. Upon execution of the ACCEPT MESSAGE COUNT statement, the contents of the area specified by a communication description entry must contain at least the name of the symbolic queue to be tested. Testing the condition causes the contents of the data items referenced by *data-name-10* (STATUS KEY) and *data-name-11* (MESSAGE COUNT) of the area associated with the communication entry to be appropriately updated. (See 13.2.2.)

13.3.2. The DISABLE Statement

13.3.2.1. Function

The DISABLE statement notifies the MCS to inhibit data transfer between specified output queues and destinations for output or between specified sources and input queues for input.

13.3.2.2. General Format

```
DISABLE { INPUT [ TERMINAL ] } cd-name WITH KEY { identifier-1 }
        { OUTPUT            }                   { literal-1    }
```

13.3.2.3. Syntax Rules

1. The parameter *cd-name* must reference an input CD when the INPUT phrase is specified.
2. The parameter *cd-name* must reference an output CD when the OUTPUT phrase is specified.
3. *Literal-1* or the contents of the data item referenced by *identifier-1* must be defined as alphanumeric.

13.3.2.4. General Rules

1. The DISABLE statement provides a logical disconnection between the MCS and the specified sources or destinations. When this logical disconnection is already in existence, or is to be handled by some other means external to this program, the DISABLE statement is not required in this program. The logical path for the transfer of data between the COBOL programs and the MCS is not affected by the DISABLE statement.
2. When the INPUT phrase with the optional word TERMINAL is specified, the logical path between the source and all queues and sub-queues is deactivated. Only the contents of the data item referenced by *data-name-7* (SYMBOLIC SOURCE) of the area referenced by *cd-name* are meaningful.
3. When the INPUT phrase without the optional word TERMINAL is specified, the logical paths for all of the sources associated with the queues and sub-queues specified by the contents of *data-name-1* (SYMBOLIC QUEUE) through *data-name-4* (SYMBOLIC SUB–QUEUE–3) of the area referenced by *cd-name* are deactivated.
4. When the OUTPUT phrase is specified, the logical path for destination, or the logical paths for all destinations, specified by the contents of the data item referenced by *data-name-5* (SYMBOLIC DESTINATION) of the area referenced by *cd-name* are deactivated.
5. *Literal-1* or the contents of the data-name referenced by *identifier-1* will be matched with a password built into the system. The DISABLE statement will be honored only if *literal-1* or the contents of the data item referenced by *identifier-1* matches the system password. When *literal-1* or the contents of the data item referenced by *identifier-1* do not match the system password, the value of the STATUS KEY item in the area referenced by *cd-name* is updated.

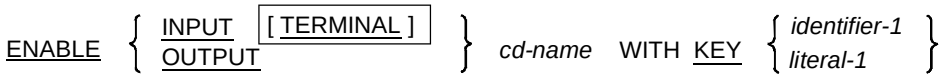
   The MCS is capable of handling a password of from one to ten characters inclusive.
6. The MCS will ensure that the execution of a DISABLE statement will cause the logical disconnection at the earliest time the source or destination is inactive. The execution of the DISABLE statement will never cause the remaining portion of the message to be terminated during transmission to or from a terminal.

13.3.3. The ENABLE Statement

13.3.3.1. Function

The ENABLE statement notifies the MCS to allow data transfer between specified output queues and destinations for output or between specified sources and input queues for input.

13.3.3.2. General Format



13.3.3.3. Syntax Rules

1. The parameter *cd-name* must reference an input CD when the INPUT phrase is specified.
2. The parameter *cd-name* must reference an output CD when the OUTPUT phrase is specified.
3. *Literal-1* or the contents of the data item referenced by *identifier-1* must be defined as alphanumeric.

13.3.3.4. General Rules

1. The ENABLE statement provides a logical connection between the MCS and the specified sources or destinations. When this logical connection is already in existence, or is to be handled by some other means external to this program, the ENABLE statement is not required in this program. The logical path for the transfer of data between the COBOL programs and the MCS is not affected by the ENABLE statement.
2. When the INPUT phrase with the optional word TERMINAL is specified, the logical path between the source and all associated queues and sub-queues which are already enabled is activated. Only the contents of the data item referenced by *data-name-7* (SYMBOLIC SOURCE) of the area referenced by *cd-name* are meaningful to the MCS.
3. When the INPUT phrase without the optional word TERMINAL is specified, the logical paths for all of the sources associated with the queue and sub–queues specified by the contents of *data-name-1* (SYMBOLIC QUEUE) through *data-name-4* (SYMBOLIC SUB–QUEUE–3) of the area referenced by *cd-name* are activated.
4. When the OUTPUT phrase is specified, the logical path for destination, or the logical paths for all destinations, specified by the contents of the data item referenced by *data-name-5* (SYMBOLIC DESTINATION) of the area referenced by *cd-name* are activated.
5. *Literal-1* or the contents of the data item referenced by *identifier-1* will be matched with a password built into the system. The ENABLE statement will be honored only if *literal-1* or the contents of the data item referenced by *identifier-1* match the system password. When *literal-1* or the contents of the data item referenced by *identifier-1* do not match the system password, the value of the STATUS KEY item in the area referenced by *cd-name* is updated.

   The MCS is capable of handling a password of from one to ten characters inclusive.

13.3.4. The RECEIVE Statement

13.3.4.1. Function

The RECEIVE statement makes available to the COBOL program a message, message segment, or a portion of a message or segment, and pertinent information about that data from a queue maintained by the Message Control System. The RECEIVE statement allows for a specific imperative statement when no data is available.

13.3.4.2. General Format

```
RECEIVE cd-name { MESSAGE          } INTO identifier-1 [ ; NO DATA imperative-statement ]
                { [ SEGMENT ]      }
```

13.3.4.3. Syntax Rule

The parameter *cd-name* must reference an input CD.

13.3.4.4. General Rules

1. The contents of the data items specified by *data-name-1* (SYMBOLIC QUEUE) through *data-name-4* (SYMBOLIC SUB–QUEUE–3) of the area referenced by *cd-name* designate the queue structure containing the message. (See 13.2.2.)
2. The message, message segment, or portion of a message or segment is transferred to the receiving character positions of the area referenced by *identifier-1* aligned to the left without space-fill.
3. When, during the execution of a RECEIVE statement, the MCS makes data available in the data item referenced by *identifier-1,* control is transferred to the next executable statement, whether or not the NO DATA phrase is specified.
4. When, during the execution of a RECEIVE statement, the MCS does not make data available in the data item referenced by *identifier-1*:
   a. If the NO DATA phrase is specified, the RECEIVE operation is terminated with the indication that action is complete (see general rule 5), and the imperative statement in the NO DATA phrase is executed.
   b. If the NO DATA phrase is not specified, execution of the object program is suspended until data is made available in the data item referenced by *identifier-1.*
   c. If one or more queues or sub-queues is unknown to the MCS, control passes to the next executable statement, whether or not the NO DATA phrase is specified. (See Table 13-1 and Table 13-2.)
5. The data items identified by the input CD are appropriately updated by the Message Control System at each execution of a RECEIVE statement. (See 13.2.2.)
6. A single execution of a RECEIVE statement never returns to the data item referenced by *identifier-1* more than a single message (when the MESSAGE phrase is used) or a single segment (when the SEGMENT phrase is used). However, the MCS does not pass any portion of a message to the object program until the entire message is available in the input queue, even if the SEGMENT phrase of the RECEIVE statement is specified.
7. When the MESSAGE phrase is used, end of segment indicators are ignored, and the following rules apply to the data transfer:
   a. If a message is the same size as the area referenced by *identifier-1,* the message is stored in the area referenced by *identifier-1.*
   b. If a message size is less than the area referenced by *identifier-1*, the message is aligned to the leftmost character position of the area referenced by *identifier-1* with no space-fill.

c. If a message size is greater than the area referenced by *identifier-1*, the message fills the area referenced by *identifier-1* left to right starting with the leftmost character of the message. In Level 1, the disposition of the remainder of the message is undefined. In Level 2, the remainder of the message can be transferred to the area referenced by *identifier-1* with subsequent RECEIVE statements referring to the same queue, sub-queue ... . The remainder of the message, for the purposes of applying rules 7a, 7b, and 7c, is treated as a new message.

8. When the SEGMENT phrase is used, the following rules apply:

   a. If a segment is the same size as the area referenced by *identifier-1,* the segment is stored in the area referenced by *identifier-1*.

   b. If the segment size is less than the area referenced by *identifier-1,* the segment is aligned to the leftmost character position of the area referenced by *identifier-1* with no space-fill.

   c. If a segment size is greater than the area referenced by *identifier-1,* the segment fills the area referenced by *identifier–1* left to right starting with the leftmost character of the segment. The remainder of the segment can be transferred to the area referenced by *identifier-1* with subsequent RECEIVE statements calling out the same queue, sub-queue ... . The remainder of the segment, for the purposes of applying rules 8a, 8b, and 8c, is treated as a new segment.

   d. If the text to be accessed by the RECEIVE statement has associated with it an end of message indicator or end of group indicator, the existence of an end of segment indicator associated with the text is implied and the text is treated as a message segment according to general rule 8.

9. Once the execution of a RECEIVE statement has returned a portion of a message, only subsequent execution of RECEIVE statements in that run unit can cause the remaining portion of the message to be returned.

10. After the execution of a STOP RUN statement, the disposition of a remaining portion of a message partially obtained in that run unit is undefined.

13.3.5. The SEND Statement

13.3.5.1. Function

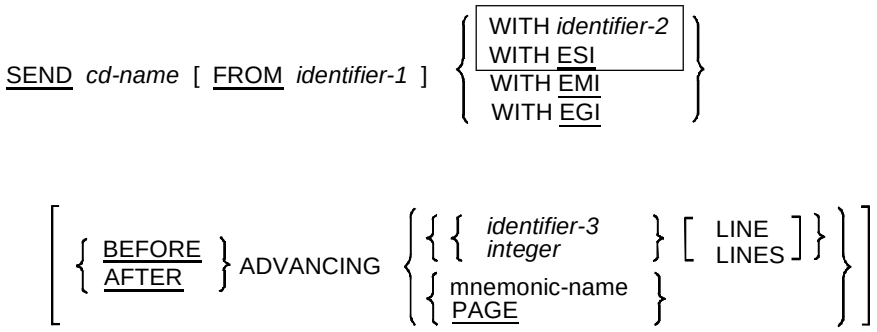
The SEND statement causes a message, a message segment, or a portion of a message or segment to be released to one or more output queues maintained by the Message Control System.

13.3.5.2. General Format

Format 1

```
SEND cd-name FROM identifier-1
```

Format 2



13.3.5.3. Syntax Rules

1. *Cd-name* must reference an output CD.
2. *Identifier-2* must reference a 1-character integer without an operational sign.
3. When *identifier-3* is used in the ADVANCING phrase, it must be the name of an elementary integer item.
4. When the *mnemonic-name* phrase is used, the name must be associated with PAGE. The *mnemonic-name* is defined in the SPECIAL–NAMES paragraph of the Environment Division.
5. *Integer* or the value of the data item referenced by *identifier-3* may be zero.

13.3.5.4. General Rules

All Formats

1. When a receiving communication device (printer, display screen, etc.) is oriented to a fixed line size:
   a. Each message or message segment begins at the leftmost character position of the physical line.
   b. A message or message segment that is smaller than the physical line size is released so as to appear space-filled to the right.
   c. Excess characters of a message or message segment will not be truncated. Characters will be packed to a size equal to that of the physical line and then sent to the device. The process continues on the next line with the excess characters.
2. When a receiving communication device (another computer, etc.) is oriented to handle variable length messages, each message or message segment will begin on the next available character position of the communications device.
3. As part of the execution of a SEND statement, the MCS will interpret the contents of the data item referenced by *data-name-2* (TEXT LENGTH) of the area referenced by *cd-name* to be the user's indication of the number of leftmost character positions of the data item referenced by *identifier-1* from which data is to be transferred.

   If the contents of the data item referenced by *data-name-2* (TEXT LENGTH) of the area referenced by *cd-name* are zero, no characters of the data item referenced by *identifier-1* are transferred.

If the contents of the data item referenced by *data-name-2* (TEXT LENGTH) of the area referenced by *cd-name* are outside the range of zero through the size of the data item referenced by *identifier-1* inclusive, an error is indicated by the value of the data item referenced by *data-name-3* (STATUS KEY) of the area referenced by *cd-name,* and no data is transferred. (See Table 13-1 and Table 13-2.)

4. As part of the execution of a SEND statement, the contents of the data item referenced by *data-name-3* (STATUS KEY) of the area referenced by *cd-name* is updated by the MCS. (See 13.2.2.)

5. The effect of having special control characters within the contents of the data item referenced by *identifier-1* is undefined.

6. A single execution of a SEND statement for Format 1 releases only a single portion of a message or of a message segment to the MCS.

A single execution of a SEND statement of Format 2 never releases to the MCS more than a single message or a single message segment as indicated by the contents of the data item referenced by *identifier-2* or by the specified indicator ESI, EMI or EGI.

However, the MCS will not transmit any portion of a message to a communications device until the entire message is placed in the output queue.

7. During the execution of the run unit, the disposition of a portion of a message not terminated by an EMI or EGI is undefined. However, the message does not logically exist for the MCS and hence cannot be sent to a destination.

After the execution of a STOP RUN statement, any portion of a message transferred from the run unit via a SEND statement, but not terminated by an EMI or EGI, is purged from the system. Thus no portion of the message is sent.

8. Once the execution of a SEND statement has released a portion of a message to the MCS, only subsequent execution of SEND statements in the same run unit can cause the remaining portion of the message to be released.

Format 2

9. The contents of the data item referenced by *identifier-2* indicate that the contents of the data item referenced by *identifier-1* are to have associated with it an end of segment indicator, an end of message indicator or an end of group indicator according to the following schedule:

If the content of the data item referenced by *identifier–2* is	**then the contents of data item referenced by *identifier–1* have associated with it**	**which means**
0	no indicator	no indicator
1	ESI	an end of segment indicator
2	EMI	an end of message indicator
3	EGI	an end of group indicator

Any character other than "1", "2", or "3" will be interpreted as "0".

If the content of the data item referenced by *identifier–2* is other than "1", "2", or "3", and *identifier-1* is not specified, then an error is indicated by the value in the data item referenced by *data-name-3* (STATUS KEY) of the area referenced by *cd-name,* and no data is transferred.

10. The ESI indicates to the MCS that the message segment is complete. The EMI indicates to the MCS that the message is complete.

    The EGI indicates to the MCS that the group of messages is complete.

    The MCS will recognize these indications and establish whatever is necessary to maintain group, message, and segment control.

11. The hierarchy of ending indicators is EGI, EMI, and ESI. An EGI need not be preceded by an ESI or EMI. An EMI need not be preceded by an ESI.

12. The ADVANCING phrase allows control of the vertical positioning of each message or message segment on a communication device where vertical positioning is applicable. Otherwise, vertical positioning is device dependent.

13. If *identifier-2* is specified and the content of the data item referenced by *identifier-2* is zero, the ADVANCING phrase is ignored by the MCS.

14. On a device where vertical positioning is applicable and the ADVANCING phrase is not specified, automatic advancing will be provided to act as if the user had specified AFTER ADVANCING 1 LINE.

15. If the ADVANCING phrase is implicitly or explicitly specified and vertical positioning is applicable, the following rules apply:

    a. If *identifier-3* or integer is specified, characters transmitted to the communication device will be repositioned vertically downward the number of lines equal to the value associated with the data item referenced by *identifier-3* or *integer*.

    b. If the BEFORE phrase is used, the message or message segment is represented on the communication device before vertical repositioning according to general rule 15a.

    c. If the AFTER phrase is used, the message or message segment is represented on the communication device after vertical repositioning according to general rule 15a.

    d. If PAGE or mnemonic-name is specified, characters transmitted to the communication device will be represented on the device before or after (depending upon the phrase used) the device is repositioned to the next page. If PAGE is specified but page has no meaning in conjunction with a specific device, then advancing will depend on the device specified.

13.4. System Considerations

See 20.4 for information on system considerations and also see the *OS 2200 ASCII COBOL Runtime Technical Reference Manual*.

Section 14
Asynchronous Processing

14.1. General

14.1.1. Asynchronous Control System (ACS)

Two levels of asynchronous processing capability may be established. The first provides the capability to control the overlapping execution of several programs within the same COBOL run unit. The second, which may be thought of as a subset of the first, provides the capability to control the overlapping execution of several USE procedures within the same program. The implementation of either level requires the specifications of an Asynchronous Control System (ACS).s

The scope of the ACS is the COBOL run unit; however, asynchronous activity is restricted to the USE procedure level. A cycle may be extended serially to more than one program via execution of CALL and ENTER statements. An asynchronous overlapping execution can only be initiated via the PROCESS statement in which a USE procedure within the program containing the PROCESS statement is specified.

OS 2200 ASCII COBOL provides the following set of asynchronous language features:

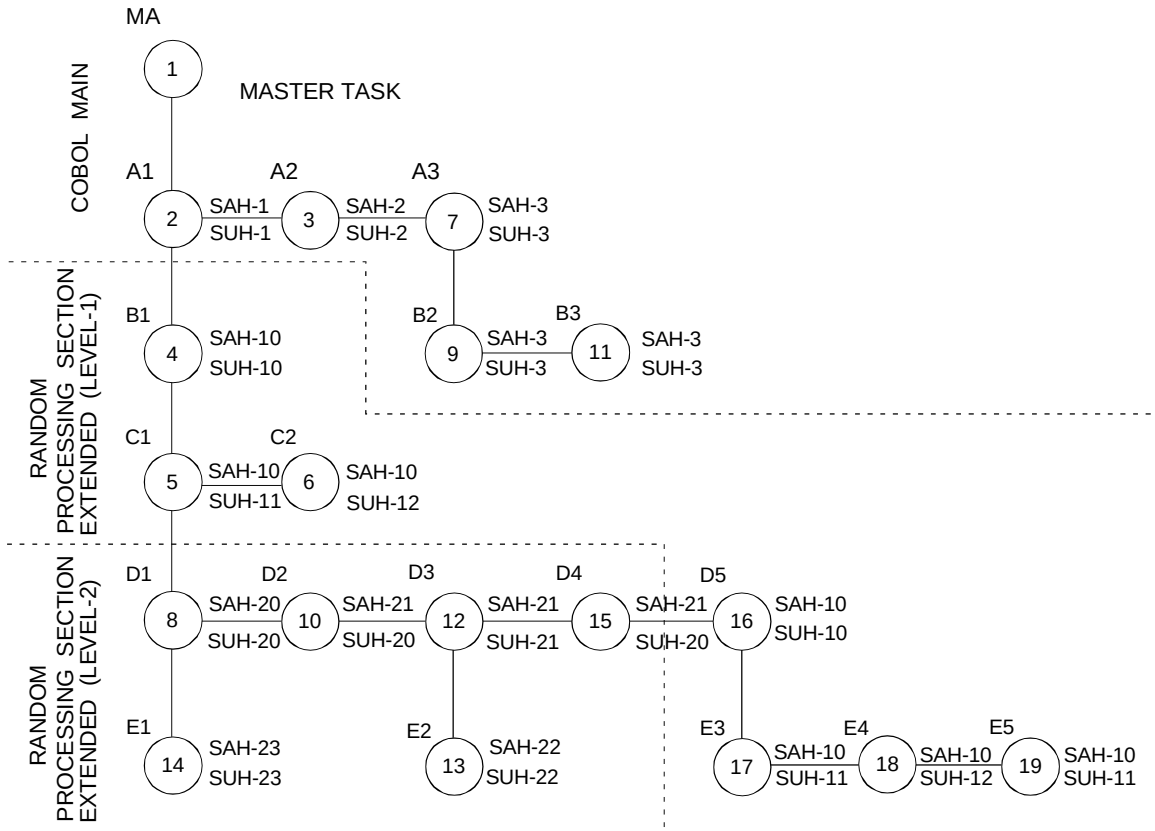
1. The dedication of a block of storage conforming to a specified Working–Storage description for use exclusively with a particular execution of a specified procedure (see 14.1.2).
2. The area may be initialized (see 14.4.5).
3. The specification of the maximum number of parallel executions that can be concurrently active.
4. The priority for each execution may be specified.
5. The current status of the set of all (or a subset of all) progeny-cycles may be determined and can cause temporary suspension of a specified cycle until the set of all (or subset of all) progeny-cycles of the specified cycle have terminated. (See 14.4.1 for the ALL condition, 14.4.2 for the ANY condition, 14.4.3 for the HOLD statement, 14.4.5 for the PROCESS statement, and 14.4.7 for the USE statement.)

Since cycles become host cycles through execution of PROCESS statements, and PROCESS statements may be placed anywhere within COBOL nondeclarative procedures and USE FOR RANDOM PROCESSING procedures, the cycle progeny tree for a given run unit may compound to any degree of complexity. The only limiting factor other than the arbitrary limits imposed by the user is the amount of storage available to a user program. A profile of a run unit's progeny tree is shown in Figure 14-1.

The ACS will register up to a maximum of three Executive activities per run unit (depending upon the current cycle count) through successive invocations of the Executive "activity initiation and termination" functions, FORK$ and EXIT$. This maximum can be changed by modifying the run-time library.

14.1.2. Local and Global Data

Two basic classes of data are available to a cycle. They are shared data items and exclusive data items. Shared data items are global in nature and include all fields defined in File, Working–Storage, Linkage and Common–Storage Sections. Exclusive data items are local to each active cycle. There are two types of exclusive data items, the record of the Saved–Area associated with the PROCESS statement for the cycle, or the copy of the record–area of the files and CD in the USING clause of the USE FOR RANDOM PROCESSING statement of the associated cycle.



MA: Master cycle

A1, A2, A3: slave-cycles emanating from host-cycle MA.

B1: slave-cycle emanating from host-cycle A1.

B2, B3: slave-cycle emanating from host-cycle A3.

C1, C2: slave-cycle emanating from host-cycle B1.

D1, D2, D3, D4, D5: slave-cycles emanating from host-cycle C1.

E1: slave-cycle emanating from host-cycle D1.

E2: slave-cycle emanating from host-cycle D3.

E3, E4, E5: slave-cycle emanating from host-cycle D5.

1. A circle represents a cycle.
2. SAH-i denotes a SAH associated with Saved-Area-name-i.
3. SUH-j denotes a schedule-unit-name-j.
4. The number within the circle indicates the initiation order.

NOTE:

Horizontal connectives define sibling chains, and vertical connectives define host-slave chains.

Figure 14-1. Example of a COBOL Cycle Structure

14.2. Environment Division

14.2.1. Input–Output Section

14.2.1.1. The FILE–CONTROL Paragraph

14.2.1.1.1. Function

The FILE–CONTROL paragraph in the INPUT-OUTPUT section specifies file-related information.

14.2.1.1.2. General Format

```
FILE-CONTROL. [ , file-control-entry ]  . . .
```

14.2.1.2. The File Control Entry

14.2.1.2.1. Function

The file control entry may specify the order in which logical records are processed by the COBOL program.

14.2.1.2.2. General Format

```
[ PROCESSING MODE IS RANDOM  [ FOR integer RECORDS ] ]
```

14.2.1.2.3. General Rules

1. When PROCESSING MODE IS RANDOM is specified, the logical records are processed asynchronously. The *integer* in the FOR *integer* RECORDS phrase specifies the number of areas allocated to contain logical records. In the random processing mode, the COBOL program normally contains one or more USE FOR RANDOM PROCESSING declarative sections which access the file. The maximum number of asynchronous processing cycles which can be accessing the file concurrently is limited to the value of *integer* in the FOR *integer* RECORDS phrase. If the maximum number of asynchronous processing cycles are executing, and another cycle is started, ACS waits until a logical record area becomes available.
2. When files are asynchronously processed the OPEN and CLOSE statements should be executed as part of the main cycle. An exception to this is when they are contained in RANDOM PROCESSING declaratives which contain proper logic to control the sequence of operations.
3. The *integer* in the FOR *integer* RECORDS phrase represents the maximum number of records required for asynchronous cycle processing. An additional record will always be assigned for the main cycle.

4. The area allocated to contain the logical records is not compiled as part of the object program. Rather, the object program is dynamically extended prior to the execution of a RANDOM PROCESSING declarative. A logical record area is assigned at this time for each file accessed in the declarative section. When the last statement in the declarative section has been executed, these logical record areas are released. Therefore, an executing COBOL program will only use as much storage area as the execution of asynchronous cycles dictates.

14.3. Data Division

14.3.1. The USAGE Clause

14.3.1.1. Function

The USAGE clause specifies the use and the format of a data item in storage.

14.3.1.2. General Format

```
[ USAGE IS ] LOCK
```

14.3.1.3. Syntax Rules

1. A lock data item can be referenced explicitly only in a LOCK or UNLOCK statement, or in a conditional statement. When appearing in a conditional statement, it is meaningful only to test their state, locked or unlocked, which is equivalent to nonzero or zero, respectively.
2. The SYNCHRONIZED, JUSTIFIED, PICTURE, VALUE, and BLANK WHEN ZERO clause cannot be used to describe group or elementary items described with the USAGE IS LOCK clause.

14.3.1.4. General Rules

1. The USAGE clause can be written at any level. If the USAGE clause is written at a group level, it applies to each elementary item in the group. The USAGE clause of an elementary item cannot contradict the USAGE clause of a group to which the item belongs.
2. An elementary item described with the USAGE IS LOCK clause is called a lock data item and contains a value of nonzero or zero. The elementary item cannot be a conditional variable. If a group item is described with the USAGE IS LOCK clause the elementary items in the group are all lock data items. The group itself is not a lock data item and cannot be used in the LOCK or UNLOCK statement or in a relation condition.
3. A lock data item can be part of a group which is referred to in a MOVE or input-output statement, in which case no conversion will take place.
4. The external and internal format of an index format of an index data item is one word with implicit synchronization and implicit COMPUTATIONAL usage.

14.3.2. File Section

14.3.2.1. The Saved–Area Description—Complete Entry Skeleton

14.3.2.1.1. Function

The Saved–Area description furnishes information concerning the physical structure and attributes of a local storage area required for random processing. Associated with each cycle is a copy of a Saved–Area dedicated to the exclusive use of this cycle. The storage occupied by the copy is returned to the dynamic storage pool for reuse upon termination of the cycle.

14.3.2.1.2. General Format

```
SA  area-name

        ;  AREA  CONTAINS  integer-1  RECORDS

    [   ;  WITH  RANK  OF  integer-2  ]   .
```

14.3.2.1.3. Syntax Rules

1. An SA entry is required when random processing is specified.
2. A level indicator SA is used to identify the beginning of a Saved-Area description entry and must precede the unique *area-name.*
3. An SA descriptor names the Saved-Area, establishes an upper bound the number of copies that can be allocated concurrently, and may specify the saved priority component.
4. The order in which the clauses appear is immaterial.
5. Saved-Area record descriptions, if any, must immediately follow the Saved-Area description for which they apply.
6. Data records within a Saved-Area represent overlays of the same record area. This is in no way altered by the presence of more than one type of data record within the Saved-Area.

14.3.2.2. The AREA CONTAINS Clause

14.3.2.2.1. Function

The AREA CONTAINS clause establishes an upper–bound to the number of copies that can be allocated concurrently.

14.3.2.2.2. General Format

```
AREA CONTAINS integer-1 RECORDS
```

14.3.2.2.3. Syntax Rules

1. This clause is required in every Saved-Area description entry.
2. The value of *integer-1* must be greater than zero and less than 512.

14.3.2.2.4. General Rules

Integer-1 specifies the maximum number of copies of the Saved-Area record that can be allocated concurrently. If the ACS is requested to allocate one beyond the maximum, the current cycle is suspended until another allocated copy is freed by the completion of its cycle.

14.3.2.3. The WITH RANK Clause

14.3.2.3.1. Function

The WITH RANK clause specifies the rank of the associated Saved-Area.

14.3.2.3.2. General Format

```
WITH RANK OF integer
```

14.3.2.3.3. Syntax Rules

1. The WITH RANK clause is optional.
2. The value of *integer* must be less than 512.

14.3.2.3.4. General Rules

1. If the WITH RANK clause is omitted, a value of zero is assumed for *integer*.
2. Cycle priority is computed as the sum of the rank of the Random Processing Section being executed and the rank of the associated Saved-Area. The greater the sum of the two numbers the higher the priority.

14.3.3. Communication Section

The PROCESSING MODE IS RANDOM clause may be specified in addition to the other clauses that may be specified in a communication description entry. (See 13.2.2.)

14.3.3.1. The PROCESSING MODE Clause

14.3.3.1.1. Function

The PROCESSING MODE clause is used to allow messages to be randomly processed. It specifies the maximum number of CD areas that may be allocated.

14.3.3.1.2. General Format

```
[ , PROCESSING MODE IS RANDOM [ FOR integer RECORDS ] ]
```

14.3.3.1.3. General Rules

1. PROCESSING MODE IS RANDOM specifies that messages are to be processed asynchronously. In this case, the compiler ensures that each PROCESS statement specifying a RANDOM PROCESSING declarative referencing the CD will allocate a separate CD area to be used for the execution of the declarative.
2. *Integer* RECORDS specifies to the compiler the maximum number of CD areas for this CD that can be concurrently active. During execution, when all CD areas are active, the execution of a process statement referencing this CD causes a wait status until a CD area becomes available.

14.4. Procedure Division

14.4.1. The ALL Conditional

14.4.1.1. Function

The ALL conditional provides a means for testing the current status of asynchronous activity with respect to the entire run unit.

14.4.1.2. General Format

```
ALL { INACTIVE } { FOR { area-name-1    } } ...
    { ACTIVE   }       { section-name-1 }
```

14.4.1.3. Syntax Rules

1. The parameter *section-name-1* used as an operand of the ALL statement must be a name of a section defined in a USE FOR RANDOM PROCESSING section in a USE declarative.
2. The parameter *area-name-1* used as an operand of the ALL statement must be a name of Saved-Areas defined by an SA level descriptor in the Data Division of the program.

14.4.1.4. General Rules

1. The ALL conditional uses the named Saved–Areas and *section-name-1* to test for the existence of the stated mode (ALL ACTIVE or ALL INACTIVE) with respect to the run unit. For the condition to be "true" when the ACTIVE option is used, each Saved-Area specified must have its maximum number of copies currently allocated and each *section-name-1* specified must have the maximum number of cycles currently being processed; otherwise, the condition is "false".
2. When the INACTIVE option is used, the condition is "false" if at least one copy of a specified Saved-Area is currently allocated or at least one cycle of a specified section-name is currently being processed; otherwise, the condition is "true".

14.4.2. The ANY Conditional

14.4.2.1. Function

The ANY conditional tests the current status of asynchronous activity with respect to the entire run unit.

14.4.2.2. General Format

```
ANY { INACTIVE } { FOR { area-name-1    } } ...
    { ACTIVE   }       { section-name-1 }
```

14.4.2.3. Syntax Rules

1. The parameter *section-name-1* used as an operand of the ANY statement must be a name of a section defined in a USE FOR RANDOM PROCESSING section in a USE declarative.
2. The parameter *area-name-1* used as an operand of the ANY statement must be the name of Saved-Areas defined by an SA level descriptor in the Data Division of the program.

14.4.2.4. General Rules

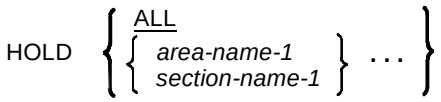
1. The ANY conditional uses the named Saved–Areas and *section-name-1* to test for the existence of the stated mode (ANY ACTIVE or ANY INACTIVE) with respect to the run unit. For the condition to be "true" when the ACTIVE option is used there must be at least one copy of a specified Saved-Area currently allocated or at least one cycle of the specified *section-name-1* currently being processed; otherwise, the condition is "false".
2. When the INACTIVE option is used, the condition is "false" if the maximum number of copies of each specified Saved–Area is currently allocated and if the maximum number of cycles is currently being processed for *section-name-1*; otherwise, the condition is "true".

14.4.3. The HOLD Statement

14.4.3.1. Function

The HOLD statement suspends execution of a cycle until a specified subset of progeny-cycles are inactive. (Inactive implies normally terminated, abnormally terminated, or never initiated.)

14.4.3.2. General Format



14.4.3.3. Syntax Rules

1. The section-names used as operands of the HOLD statement must be names of sections defined in a USE FOR RANDOM PROCESSING section in a USE declarative. (See 14.4.7.)
2. The area-names used as operands of the HOLD statement must be names of Saved-Areas defined by an SA level descriptor entry in the Data Division of the program. (See 14.3.2.1.)

14.4.3.4. General Rules

1. The ALL specifies that all cycles initiated from the cycle executing the HOLD statement (all progeny cycles) have been completed before any statements following the HOLD ALL statement are executed.
2. When a section-name or area-name is specified all cycles pertaining to the named sections or associated with the named saved–areas and initiated from the cycle executing the HOLD statement (and all progeny cycles) have been completed before any statements following the HOLD statement are executed.

14.4.4. The LOCK Statement

14.4.4.1. Function

The LOCK statement provides a means for obtaining exclusive use of a shared resource, such as, a shared data item, or procedure overlay area.

14.4.4.2. General Format

```
LOCK {identifier-1} . . .
```

14.4.4.3. Syntax Rule

Identifier-1 must be defined with a USAGE IS LOCK clause.

14.4.4.4. General Rules

1. The association of a lock data-item with a shared resource is conceptual in nature, and hence, a lock data-item can be considered simply as a 2-position status indicator: ON, when locked, and OFF, when unlocked.
2. When the LOCK statement is executed, all the identifiers are tested in order until a previously locked identifier is encountered or all identifiers are found to be unlocked. When a previously locked identifier is encountered, the cycle is effectively suspended. When the cycle is reactivated, all identifiers are again tested in order. After all have been tested and found to be unlocked, each identifier is locked and control transfers to the next executable statement following the LOCK statement.

14.4.5. The PROCESS Statement

14.4.5.1. Function

The PROCESS statement initiates a set of USE procedures under the control of an Asynchronous Control System (ACS).

14.4.5.2. General Format

```
PROCESS section-name [ [ FROM identifier ] USING area-name ]
```

14.4.5.3. Syntax Rules

1. *Section-name* specified in the PROCESS statement must be the name of a USE procedure defined in the Declarative Section with a USE FOR RANDOM PROCESSING clause in its header section. It identifies the Random Processing Section to be executed.
2. The USING phrase is required for programs containing more than one Saved–Area Description.
3. The *area-name* must be the name of a Saved-Area description entry (SA) defined in the File Section. (See 14.3.2.1.)

14.4.5.4. General Rules

1. The PROCESS statement causes the named Random Processing Section to be executed as an asynchronous USE cycle. Actual commencement of execution for the initiated cycle is a function of the Asynchronous Control System. Control for the initiating cycle is returned to the next statement following the PROCESS statement.
2. The cycle which contains the PROCESS statement will be suspended until another execution of the named Random Processing Section is completed, if activation of this cycle would exceed the maximum number of cycles stated in the FOR *integer-2* CYCLES clause of the named Random Processing Section.
3. When the optional phrase FROM is used, moving an identifier to the copy of the Saved-Area allocated for the cycle initiated by the PROCESS statement takes place in accordance with the rules specified for the MOVE statement without the CORRESPONDING option.
4. The PROCESS statement causes the allocation of one copy of the named Saved–Area and one copy of each PROCESSING MODE IS RANDOM or PROCESSING MODE IS RANDOM CD stated in the USING clause of the USE FOR RANDOM PROCESSING clause for the named Random Processing Section, to be dedicated to the exclusive use of the cycle initiated at execution of the PROCESS statement. Saved-Area copies are released for reassignment upon completion of the cycle to which they were dedicated. If no copies of the SA or of a record area of a PROCESSING MODE IS RANDOM file are available, the cycle which has the PROCESS statement is suspended until a copy of the record-area or SA is made available by completion of another cycle. (See 14.3.2.2.)
5. Exclusive use of shared data items can be assured only through explicit embedding of appropriate LOCK and UNLOCK statements throughout the source language programs. Data items which occupy portions of a word in conjunction with a shared data item must also be treated as an extension of the shared data item. Therefore, shared data items should be either implicitly or explicitly word synchronized.

14.4.6. The UNLOCK Statement

14.4.6.1. Function

The UNLOCK statement releases exclusive use of a shared resource, such as a shared data item or procedure overlay area.

14.4.6.2. General Format

```
UNLOCK {identifier-1} . . .
```

14.4.6.3. Syntax Rules

Identifier-1 must be defined with a USAGE IS LOCK clause.

14.4.6.4. General Rules

1. The association of a lock data item with a shared resource is conceptual in nature. Therefore, a lock data item can be considered simply as a two position status indicator: ON, when locked; and OFF, when unlocked.
2. When the UNLOCK statement is executed, all locked fields specified are unlocked. Control transfers to the next executable statement following the UNLOCK statement.

14.4.7. The USE Statement

14.4.7.1. Function

The USE FOR RANDOM PROCESSING statement identifies a set of procedural statements as a Random Processing Section to be processed asynchronously.

14.4.7.2. General Format

```
USE FOR RANDOM PROCESSING

    [ USING { file-name-1 } ... ]
            { cd-name-1   }

    [ WITH RANK OF integer-1 ] [ FOR integer-2 CYCLES ].
```

14.4.7.3. Syntax Rules

1. A USE statement, when present, must immediately follow a section header in the declaratives section and must be followed by a period followed by a space. The remainder of the section must consist of zero, one or more procedural paragraphs that define the procedures to be used.
2. The USE statement itself is never executed. It merely defines the conditions calling for the execution of the USE procedures.

3. The files implicitly or explicitly referenced in a USE statement need not all have the same organization or access.
4. *File-name-1* may not be a sort or merge file.
5. All file-names and cd-names referenced within a Random Processing Section must be named in the USING clause whether or not they specify PROCESSING MODE IS RANDOM in their descriptions.
6. The value of *integer-1* or *integer-2* must be less than 512.

14.4.7.4. General Rules

1. The WITH RANK phrase specifies the rank of its associated USE procedure.
2. If the WITH RANK phrase is omitted, a value of zero is assumed for *integer-1*.
3. Cycle priority is computed as the sum of the rank of the Random Processing Section being executed and the rank of the associated Saved-Area. The greater the sum of the two numbers the higher the priority.
4. The FOR *integer-2* CYCLES phrase establishes an upper-bound to the number of cycles of the associated USE procedure that can be concurrently active.
5. If the FOR *integer-2* CYCLES phrase is omitted, a value of one is assumed for *integer-2*.
6. If ACS is requested to activate one more cycle than is stated in the FOR *integer-2* CYCLES phrase, it will suspend that cycle containing the PROCESS statement until another cycle of the Random Processing Section is completed.
7. If the file-name or cd-name contains a PROCESSING MODE IS RANDOM FOR *integer* RECORDS clause, a copy of the record area for this file-name or cd-name is allocated for exclusive use of this cycle. This record area will be released for reassignment upon completion of the cycle. If ACS is requested to allocate one more record area than is stated in the FOR *integer* RECORDS phrase, the cycle which contains the PROCESS statement for this Random Processing Section will be suspended.
8. In the statements of the USE procedure, the execution of an ALTER or PERFORM statement affects that particular cycle only; that is, any other cycle of this USE procedure is not affected by another concurrent cycle in which an ALTER or PERFORM statement is executed.

Section 15
IBM Tape Processing

15.1. General

15.1.1. Function

The IBM Tape Processing module can access records of an IBM formatted tape file in an established sequence. The sequence is established as a result of writing the records to the file. It also provides for the specification of rerun points and the sharing of storage areas among files.

15.1.2. Language Concepts

15.1.2.1. Organization

Each record of a sequential file except the first has a unique predecessor record, and each record except the last has a unique successor record. These predecessor-successor relationships are established by the order of WRITE statements when the file is created. Once established, the predecessor-successor relationships do not change.

15.1.2.2. Access Mode

In the sequential access mode, the sequence in which records are accessed is the order in which the records were originally written.

15.1.2.3. Current Record Pointer

The current record pointer is a conceptual entity used to specify the next record to be accessed within a given file. The concept of the current record pointer has no meaning for a file opened in the output mode. The setting of the current record pointer is affected only by the OPEN and READ statements.

15.1.2.4. The AT END Condition

The AT END condition can occur as a result of the execution of a READ statement. For details of the causes of the condition, see 15.4.3.

15.2. Environment Division

15.2.1. Input–Output Section

15.2.1.1. The FILE–CONTROL Paragraph

15.2.1.1.1. Function

The FILE–CONTROL paragraph in the INPUT-OUTPUT section names each file and allows specification of other file-related information.

15.2.1.1.2. General Format

```
FILE-CONTROL.  {  file-control-entry  }  . . .
```

15.2.1.2. The File Control Entry

15.2.1.2.1. Function

The file control entry names a file and may specify other file-related information.

15.2.1.2.2. General Format

```
SELECT [ OPTIONAL ] file-name

    ASSIGN TO implementor-name-1 [ , implementor-name-2 ] . . .

    [ ; RESERVE { integer-1 } ALTERNATE [ AREA  ] ]
                { NO        }           [ AREAS ]

    [ ; ACCESS MODE IS SEQUENTIAL ]

    [ ; ORGANIZATION IS SEQUENTIAL ]
```

15.2.1.2.3. Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses that follow the SELECT clause may appear in any order.
2. Each file described in the Data Division must be named once and only once as *file-name* in the FILE–CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. The *implementor-name-1* is UNISERVO™.
4. If the *implementor-name-2* is specified, it is a 1 to 12- character name used to link the ASCII COBOL file to OS 2200 Executive File Control and should appear on an @ASG or @USE statement in the run stream for executing the object program. (See 15.5).
5. When *implementor-name-2* is not specified, the first 12 characters of the file-name are used to link the ASCII COBOL file to the OS 2200 Executive File Control. (See 15.5).

6. All recursions of *implementor-name-1* and *implementor-name-2* are for documentation only.
7. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.
8. When the ORGANIZATION IS SEQUENTIAL clause is not specified, the ORGANIZATION IS SEQUENTIAL clause is implied.
9. The OPTIONAL phrase may only be specified for input files. Its specification is required for input files that are not necessarily present each time the object program is executed.

15.2.1.2.4. General Rules

1. The ASSIGN clause specifies the association of the file referenced by *file-name* to a storage medium.
2. The RESERVE clause allows the user to affect the number of input-output areas allocated. If the RESERVE clause is not specified or if *integer-1* is specified, two input-output areas are allocated. If NO is specified, one input-output area is allocated.
3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
4. Records in the file are accessed in the sequence dictated by the file organization. This sequence is specified by predecessor-successor record relationships established by the execution of WRITE statements when the file is created.

15.2.1.3. The I–O–CONTROL Paragraph

15.2.1.3.1. Function

The I–O–CONTROL paragraph specifies the points at which rerun is to be established, and the storage area which is to be shared by different files.

15.2.1.3.2. General Format

```
I-O-CONTROL .

[ ; RERUN ON implementor-name    EVERY { [ END OF ] REEL       }
                                       { integer-1 RECORDS     }

                  OF file-name-1 ] ...

[ ; SAME [ RECORD ] AREA FOR { file-name-2 } ... ] ... .
```

15.2.1.3.3. Syntax Rules

1. The I–O–CONTROL paragraph is optional.
2. *File-name-1* may be either an input or output file.
3. The allowable *implementor–names* are MASS–STORAGE and UNISERVO.

4. More than one RERUN clause may be specified for a given *file-name-1*, subject to the following restrictions:
   a. When multiple *integer-1* RECORDS clauses are specified, no two of them may specify the same *file-name-1*.
   b. When multiple END OF REEL clauses are specified, no two of them may specify the same *file-name-1*.
5. The two forms of the SAME clause (SAME AREA, SAME RECORD AREA) are considered separately in the following:

   More than one SAME clause may be included in a program. However:
   a. A file-name must not appear in more than one SAME AREA clause.
   b. A file-name must not appear in more than one SAME RECORD AREA clause.
   c. If one or more file-names of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the file-names in that SAME AREA clause must appear in that SAME RECORD AREA clause. However, additional file-names not appearing in that SAME AREA clause may also appear in that SAME RECORD AREA clause. The rule that only one of the files mentioned in a SAME AREA clause can be open at any given time takes precedence over the rule that all files mentioned in a SAME RECORD AREA clause can be open at any given time.
6. The files referenced in the SAME AREA or SAME RECORD AREA clause need not all have the same organization or access.

15.2.1.3.4. General Rules

1. The RERUN clause specifies when and where the rerun information is recorded.
2. Rerun information is written in a separate file whose file-name is RERUN$. The two forms of the RERUN clause are based on the following conditions under which rerun points can be established:
   a. When the END OF REEL clause is used. In this case, the rerun information is written whenever the END OF REEL condition occurs on *file-name-1*.
   b. When the *integer-1* RECORDS is used. In this case, the rerun information is written whenever *integer-1* records of *file-name-1* have been processed.
3. The SAME AREA clause specifies that two or more files that do not represent sort or merge files are to use the same main storage area during processing. The area being shared includes all storage area assigned to the files specified; therefore, it is not valid to have more than one of the files open at the same time. (See syntax rule 5c in 15.2.1.3.3.)
4. The SAME RECORD AREA clause specifies that two or more files are to use the same storage area for processing of the current logical record. All of the files may be open at the same time. A logical record in the SAME RECORD AREA is considered as a logical record of each opened output file whose file-name appears in this SAME RECORD AREA clause and of the most recently read input file whose file-name appears in this SAME RECORD AREA clause. This is equivalent to an implicit redefinition of the area; in other words, records are aligned on the leftmost character position.

15.3. Data Division

15.3.1. File Section

In a COBOL program the file description entry (FD) represents the highest level of organization in the File Section. The File Section header is followed by a file description entry consisting of a level indicator (FD), a file-name and a series of independent clauses. The FD clauses specify the size of the logical and physical records, the presence or absence of label records, the value of Unisys defined label items, and the names of the data records which comprise the file. The entry itself is terminated by a period.

15.3.2. The File Description—Complete Entry Skeleton

15.3.2.1. Function

The file description furnishes information concerning the physical structure, identification, and record names pertaining to a given file.

15.3.2.2. General Format

```
FD file-name

[ ; BLOCK CONTAINS [ integer-1 TO ] integer-2 { RECORDS / CHARACTERS } ]

[ ; RECORD CONTAINS [ integer-3 TO ] integer-4 CHARACTERS ]

  ; LABEL { RECORD IS / RECORDS ARE } { STANDARD / OMITTED }

[ ; VALUE OF { implementor-name-1 IS { data-name-1 / literal-1 } } ... ]

[ ; DATA { RECORD IS / RECORDS ARE } { data-name-2 } ... ]

  ; RECORDING MODE IS { F / U / V } [ AN ].
```

15.3.2.3. Syntax Rules

1. The level indicator FD identifies the beginning of a file description and must precede *file-name*.
2. The clauses that follow the name of the file are optional in many cases, and their order of appearance is immaterial.
3. One or more record description entries must follow the file description entry.

15.3.3. The BLOCK CONTAINS Clause

15.3.3.1. Function

The BLOCK CONTAINS clause specifies the size of a physical record.

15.3.3.2. General Format

```
BLOCK CONTAINS  [ integer-1 TO ]  integer-2  { RECORDS    }
                                             { CHARACTERS }
```

15.3.3.3. General Rules

1. This clause is required except when:
   a. A physical record contains one and only one complete logical record.
   b. RECORDING MODE U is specified.
2. The size of the physical record may be stated in terms of RECORDS, unless one of the following situations exists, in which case the RECORDS phrase must not be used.
   a. The physical record contains padding (area not contained in a logical record).
   b. Logical records are grouped in such a manner that an inaccurate physical record size would be implied.
3. When the word CHARACTERS is specified, the physical record size is specified in terms of the number of ASCII character positions required to store the physical record, regardless of the types of characters used to represent the items within the physical record.
4. If only *integer-2* is shown, it represents the exact size of the physical record.

   If *integer-1* and *integer-2* are both shown, they refer to the minimum and maximum size of the physical record, respectively.

15.3.4. The DATA RECORDS Clause

15.3.4.1. Function

The DATA RECORDS clause serves only as documentation for the names of data records with their associated file.

15.3.4.2. General Format

```
DATA { RECORD  IS  }
     { RECORDS ARE } { data-name-1 } ...
```

15.3.4.3. Syntax Rules

Data-name-1 represents the names of the data records and must have 01 level-number record descriptions, with the same names, associated with them.

15.3.4.4. General Rules

The presence of more than one data-name indicates that the file contains more than one type of data record. These records may be of different sizes, different formats, etc. The order in which they are listed is not significant.

Conceptually, all data records within a file share the same area. This is in no way altered by the presence of more than one type of data record within the file.

15.3.5. The LABEL RECORDS Clause

15.3.5.1. Function

The LABEL RECORDS clause specifies whether labels are present.

15.3.5.2. General Format

```
LABEL { RECORD IS  } { STANDARD }
      { RECORDS ARE} { OMITTED  }
```

15.3.5.3. Syntax Rules

This clause is required in every file description entry.

15.3.5.4. General Rules

1. OMITTED specifies that no explicit labels exist for the file or the device to which the file is assigned.
2. STANDARD specifies that EBCDIC labels exist for the file or the device to which the file is assigned.

15.3.6. The RECORD CONTAINS Clause

15.3.6.1. Function

The RECORD CONTAINS clause specifies the size of data records.

15.3.6.2. General Format

```
RECORD CONTAINS [ integer-1 TO ] integer-2 CHARACTERS
```

15.3.6.3. General Rules

1. The size of each data record is completely defined within the record description entry, therefore this clause is never required. When present, however, the following notes apply:
   a. *Integer-2* may not be used by itself unless all the data records in the file have the same size. In this case, *integer-2* represents the exact number of characters in the data record. If *integer-1* and *integer-2* are both shown, they refer to the minimum number of characters in the smallest size data record and the maximum number of characters in the largest size data record, respectively.

b. The size is specified in terms of the number of character positions required to store the logical record, regardless of the types of characters used to represent the items within the logical record. The size of a record is determined by the sum of the number of characters in all fixed length elementary items plus the sum of the maximum number of characters in any variable length item subordinate to the record. This sum may be different from the actual size of the record. See 1.3.3.3.4, 15.3.11, and 15.3.12.

15.3.7. The RECORDING MODE Clause

15.3.7.1. Function

The RECORDING MODE clause is used to specify IBM OS/360 tape compatibility.

15.3.7.2. General Format

RECORDING MODE IS { F / U / V } [AN]

15.3.7.3. General Rules

1. The structure of the physical data blocks may be defined by one of the following forms:
   a. The F phrase specifies fixed blocked records.
   b. The U phrase specifies variable length unblocked records.
   c. The V phrase specifies variable length blocked records.
2. When the AN option is specified the records are assumed to consist entirely of ASCII characters internally and of EBCDIC characters on magnetic tape. The character translation is performed when records are transferred to or from storage.
3. When the AN option is not specified, the following data conversions are performed:
   a. On input all DISPLAY data items are converted from EBCDIC to ASCII.
   b. On output all DISPLAY data items are converted from ASCII to EBCDIC.
   c. When a record is read from tape, in every numeric DISPLAY field leading blanks are converted to leading zeros.
   d. Signed numeric DISPLAY fields are written to tape with an overpunched sign on the low order digit.
   e. On input all COMPUTATIONAL, COMPUTATIONAL–1, COMPUTATIONAL–2, and COMPUTATIONAL–3 data items are converted from IBM OS/360 format to OS 2200 ASCII COBOL internal format.
   f. On output all COMPUTATIONAL, COMPUTATIONAL–1, COMPUTATIONAL–2 and COMPUTATIONAL–3 data items are converted from OS 2200 ASCII COBOL internal format to IBM OS/360 format.
   g. Records written to tape are adjusted such that slack bytes are inserted where necessary to ensure proper IBM OS/360 alignment.

h. For RECORDING MODES U and V files, Format 2 of the OCCURS clause is allowed in the record description and the following processing is performed:

- A data item whose size is variable is expanded on input using the value of the data item referenced by *data-name-1* of the OCCURS clause (see 3.2.1).
- A data item whose size is variable is compressed on output using the value of the data item referenced by *data-name-1* of the OCCURS clause (see 3.2.1).

i. If a REDEFINES clause is used within a record description, then the original definition is used rather than the redefinition for the purposes of record conversion.

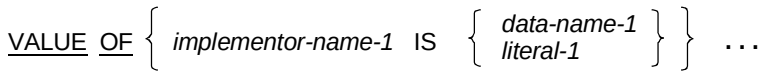
4. If AN is not specified and the file contains multiple record descriptions, then a record selector field within each record description is required. (See 15.3.14.)

15.3.8. The VALUE OF Clause

15.3.8.1. Function

The VALUE OF clause particularizes the description of an item in the label records associated with a file.

15.3.8.2. General Format



15.3.8.3. Syntax Rules

1. The allowable implementor-names are FILE–ACCESS, FILE–ID, and PURGE–DATE. These implementor-names have the implied definition of:

```
FILE-ACCESS    PIC    X.
FILE-ID        PIC    X(17).
PURGE-DATE     PIC    9(6).
```

2. *Data-name-1* should be qualified when necessary, but cannot be subscripted or indexed, nor can they be items described with the USAGE IS INDEX clause.
3. *Data-name-1* must be in the Working-Storage Section.
4. *Data-name-1* must be described as USAGE DISPLAY.

15.3.8.4. General Rules

1. For an input file, the appropriate label routine checks to see if the value of *implementor-name-1* is equal to the value of *literal-1* or of *data-name-1*, whichever has been specified. (See 15.4.2 and Appendix E.)

   For an output file, at the appropriate time the value of *implementor-name-1* is made equal to the value of *literal-1* or of a *data-name-1*, whichever has been specified. (See 15.4.2 and Appendix E.)

2. A figurative constant may be substituted in the format wherever a literal is specified.

15.3.9. The Record Description—Complete Entry Skeleton

A record description consists of a set of data description entries that describe the characteristics of a particular record. Each data description entry consists of a level-number followed by a data-name, if required, followed by a series of independent clauses as required. A record description has a hierarchical structure and therefore the clauses used with an entry may vary considerably, depending upon whether or not it is followed by subordinate entries. The structure of a record description is defined in 1.3.3.3.2.

15.3.9.1. Function

A data description entry specifies the characteristics of a particular item of data.

15.3.9.2. General Format

Format 1

level-number { *data-name-1* / FILLER } [; REDEFINES *data-name-2*]

[; { PICTURE / PIC } IS *character-string*]

[; [USAGE IS] { COMPUTATIONAL / COMP / COMPUTATIONAL-1 / COMP-1 / COMPUTATIONAL-2 / COMP-2 / COMPUTATIONAL-3 / COMP-3 / DISPLAY / DISP }]

[; { SYNCHRONIZED / SYNC } [LEFT / RIGHT]]

[; { JUSTIFIED / JUST } RIGHT]

[; BLANK WHEN ZERO]

[; { VALUE IS / VALUES ARE } { *literal-1* [{ THROUGH / THRU } *literal-2*] } ...]

[; RANK IS *integer-1*] .

Format 2

66 *data-name-1* ; RENAMES *data-name-2* [{ THROUGH / THRU } *data-name-3*] .

Format 3

```
88 condition-name ; { VALUE IS  } literal-1 [ { THROUGH } literal-2 ]
                    { VALUES ARE}             { THRU    }

    [ , literal-3 [ { THROUGH } literal-4 ] ] ... .
                    { THRU    }
```

15.3.9.3. Syntax Rules

1. The clauses may be written in any order with two exceptions. The *data-name-1* or FILLER clause must immediately follow the *level-number*. The REDEFINES clause, when used, must immediately follow the *data-name-1* clause.
2. The PICTURE clause must be specified for every elementary item except floating–point data items, in which case use of this clause is prohibited.
3. The words THRU and THROUGH are equivalent.

15.3.9.4. General Rules

Format 1

1. A PICTURE character-string containing one or more 1's is not allowed.
2. The RANK, SYNCHRONIZED, USAGE, and VALUE (in the File Section) clauses are described in this section. (See 15.3.10 for the RANK clause, 15.3.11 for the SYNCHRONIZED clause, 15.3.12 for the USAGE clause, and 15.3.13 for the VALUE clause.)
3. Excluding the restrictions listed in the previous General Rules, the elements shown in the General Format are documented in 2.5.2.

Format 2 and Format 3

4. See 2.5.2.

15.3.10. The RANK Clause

15.3.10.1. Function

The purpose of the RANK clause is to increase the efficiency of the process of record selection.

15.3.10.2. General Format

```
RANK IS [ integer-1 ]
```

15.3.10.3. Syntax Rules

1. The RANK clause is optional.
2. The RANK clause can only be used in the description of a record selector. Record selectors are described in 15.3.14.
3. *Integer-1* is an unsigned integer in the range 1 through 63.

15.3.10.4. General Rules

1. When the RANK clause is omitted, a value of 63 is assumed for the record selector.
2. When two or more record selectors in the same record description have the same RANK, the textual order determines the order in which the selectors are tested.
3. When a record description contains multiple record selectors, the value of the RANK assigned to each record selector need not be consecutive.
4. When a record description contains more than one record selector, the order in which the record selectors are tested is determined by the RANK of each selector. Record selectors of rank *n* are tested before any record selector of rank *n* + 1; for example, RANK 1 has a higher precedence than a record selector of RANK 2.
5. When a record description contains only one record selector, the RANK clause is permitted but it has no meaning.

15.3.11. The SYNCHRONIZED Clause

15.3.11.1. Function

The SYNCHRONIZED clause specifies the alignment of an elementary item on the natural boundaries of the computer main storage when IBM OS/360 allocation and alignment rules apply.

15.3.11.2. General Format

```
{ SYNCHRONIZED } [ LEFT  ]
{ SYNC         } [ RIGHT ]
```

15.3.11.3. Syntax Rules

1. This clause may only appear with an elementary item.
2. SYNC is an abbreviation for SYNCHRONIZED.

15.3.11.4. General Rules

1. A full description of data allocation and alignment may be found in 15.3.15.
2. This clause specifies that the subject data item is to be aligned in memory such that no other data item occupies any of the character positions between the leftmost and rightmost natural boundaries delimiting this data item. If the number of character positions required to store this data item is less than the number of character positions between those natural boundaries, the unused character positions (or portions thereof) must not be used for any other data item. Such unused character positions, however, are included in:
   a. The size of any group items to which the elementary item belongs; and
   b. The character positions redefined when this data item is the object of a REDEFINES clause.
3. The LEFT and RIGHT phrases serve only for documentation.

4. The effect of the SYNCHRONIZED phrase and the USAGE clause on the alignment of elementary data items is summarized in the following table:

Usage	Alignment
COMPUTATIONAL (1– 4 digits)	Half Word boundary
COMPUTATIONAL (5 –18 digits)	Word boundary
COMPUTATIONAL–1	Word boundary
COMPUTATIONAL–2	Double-Word boundary
COMPUTATIONAL–3	Quarter-Word boundary
DISPLAY	Quarter-Word boundary

5. Whenever a SYNCHRONIZED item is referenced in the source program, the original size of the item, as shown in the PICTURE clause, is used in determining any action that depends on size, such as justification, truncation, or overflow.
6. If the data description of an item contains the SYNCHRONIZED clause and an operational sign, the sign of the item appears in the normal operational sign position, regardless of whether the item is SYNCHRONIZED LEFT or SYNCHRONIZED RIGHT.
7. When the SYNCHRONIZED clause is specified in a data description entry of a data item that also contains an OCCURS clause, or in a data description entry of a data item subordinate to a data description entry that contains an OCCURS clause, then:
   a. Each occurrence of the data item is SYNCHRONIZED.
   b. Any implicit FILLER generated for other data items within that same table are generated for each occurrence of those data items.
8. The elementary items associated with this clause are handled as follows:
   a. Any necessary generation of implicit FILLER, if the elementary item immediately preceding an item containing the SYNCHRONIZED clause does not terminate at a word boundary, is not included as part of the elementary item. Such automatically generated FILLER positions are included in:
      - The size of any group to which the FILLER item belongs; and
      - The number of character positions allocated when the group item of which the FILLER item is a part appears as the object of a REDEFINES clause.
9. In the absence of the SYNCHRONIZED clause, the ASCII COBOL compiler will align data items on the next available quarter-word boundary.

15.3.12. The USAGE Clause

15.3.12.1. Function

The USAGE clause specifies the format of a data item in the computer storage.

15.3.12.2. General Format

```
[ USAGE IS ]  { COMPUTATIONAL   }
              { COMP            }
              { COMPUTATIONAL-1 }
              { COMP-1          }
              { COMPUTATIONAL-2 }
              { COMP-2          }
              { COMPUTATIONAL-3 }
              { COMP-3          }
              { DISPLAY         }
              { DISP            }
```

15.3.12.3. Syntax Rules

1. The PICTURE character-string of a COMPUTATIONAL, and COMPUTATIONAL-3 item can contain only 9's, the operational sign character "S", the implied decimal point character "V", and one or more P's. (See 2.5.7.)
2. COMP, COMP–*n*, and DISP are abbreviations for COMPUTATIONAL, COMPUTATIONAL–*n*, and DISPLAY, respectively.
3. COMPUTATIONAL–1, and COMPUTATIONAL–2 usage specifications may not appear in a data description which has a PICTURE clause, because the computer storage format of these items is fixed.

15.3.12.4. General Rules

1. A full description of data allocation and alignment may be found in 15.3.15.
2. The USAGE clause can be written at any level. If the USAGE clause is written at a group level, it applies to each group or elementary item in the group. The USAGE clause of an elementary item cannot contradict the USAGE clause of a group to which the item belongs.
3. If the group usage is DISPLAY or COMPUTATIONAL, the group is considered to be an alphanumeric string of ASCII characters.
4. If the USAGE clause is not specified on a group item, the group is considered to be an alphanumeric string of ASCII characters.

   If the USAGE clause is not specified for an elementary item, or for any group to which the item belongs, the usage is implicitly DISPLAY.
5. This clause specifies the manner in which a data item is represented in the storage of a computer. It does not affect the use of the data item, although the specifications for some statements in the Procedure Division may restrict the USAGE clause of the operands referred to. The USAGE clause may affect the radix or type of character representation of the item.

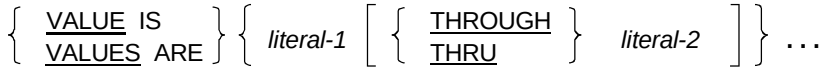
6. A COMPUTATIONAL or COMPUTATIONAL–*n* item is capable of representing a value to be used in computations and must be numeric. If a group item is described as COMPUTATIONAL or COMPUTATIONAL–*n*, the elementary items in the group are COMPUTATIONAL or COMPUTATIONAL–*n*. The group item itself is not COMPUTATIONAL or COMPUTATIONAL–*n* (cannot be used in computations).
7. The USAGE IS DISPLAY clause indicates that the format of the data is a standard data format.

15.3.13. The VALUE Clause

15.3.13.1. Function

The VALUE clause is used to specify a record selector. (See 15.3.14.)

15.3.13.2. General Format



15.3.13.3. Syntax Rules

1. The words THRU and THROUGH are equivalent.
2. The VALUE clause cannot be stated for any items whose size is variable (see 3.2.1).
3. A signed numeric literal must have associated with it a signed numeric PICTURE character-string.
4. All numeric literals in a VALUE clause of an item must have a value that is within the range of values indicated by the PICTURE clause, and not have a value that would require truncation of nonzero digits. Nonnumeric literals in a VALUE clause of an item must not exceed the size indicated by the PICTURE clause.

15.3.13.4. General Rules

1. The VALUE clause must not conflict with other clauses in the data description of the item or in the data description within the hierarchy of the item. The following rules apply:
   a. If the category of the item is numeric, all literals in the VALUE clause must be numeric. The literal is aligned according to the standard alignment rules. (See 1.3.3.3.6.)
   b. If the category of the item is alphabetic, alphanumeric, alphanumeric edited, or numeric edited, all literals in the VALUE clause must be nonnumeric literals. The literal is aligned as if the data item had been described as alphanumeric. (See 1.3.3.3.6.)
2. A figurative constant may be substituted wherever a literal is specified.
3. Wherever the THRU phrase is used, *literal-1* must be less than *literal-2*.
4. The VALUE clause must not be stated in a data description entry that contains an OCCURS clause, or in an entry that is subordinate to an entry containing an OCCURS clause.

5. The VALUE clause must not be stated in a data description entry that contains a REDEFINES clause, or in an entry that is subordinate to an entry containing a REDEFINES clause.
6. If the VALUE clause is used in an entry at the group level, the literal must be a figurative constant or a nonnumeric literal. The VALUE clause cannot be stated at the subordinate levels within this group.
7. The VALUE clause must not be written for a group containing items with descriptions including JUSTIFIED, SYNCHRONIZED, or USAGE (other than USAGE IS DISPLAY).

15.3.14. Record Selector Fields

A record selector field is identified to the compiler by the appearance of a VALUE clause in the record description. (See 15.3.13.)

When the file is read, the record selectors are tested to determine which record conversion must be performed. The record selectors in the first record are tested; if any of the selectors fail the next record is considered. If all the record selectors match, the first record description is chosen. This process is repeated for all the data descriptions, until a record description is encountered, all of whose selectors match. If no record description can be chosen, a diagnostic is issued.

In any given record description, the record selectors are listed in their textual order. This ordering can be changed by using the RANK clause. (See 15.3.10.)

The RANK clause does not affect the record description chosen; its purpose is to increase the efficiency of the record selection process.

The number of record selectors in any single record description must not exceed 63. Each record description in a given file description need not contain the same number of selectors. In addition, the selectors need not be in the same position (internal or external) in each record description, and they need not have the same USAGE or PICTURE. However, it is the user's responsibility to ensure that this does not lead to inadvertent record selection.

15.3.15. Data Allocation

Data allocations for items of various usages differ between OS 2200 ASCII COBOL and IBM OS/360 COBOL internal allocation. To provide greater compatibility with the IBM OS/360 allocation scheme, OS 2200 ASCII COBOL provides several compiler options which allow for the handling and allocation of data items according to IBM OS/360 COBOL or OS 2200 ASCII COBOL allocation rules without a change in the source program. These options may be specified in the Extra Options field of the @ACOB processor call statement. Following are the Extra Options and their descriptions:

Extra Option	Description
A	Use standard OS 2200 allocation rules for RECORDING MODE F, U, and V (EBCDIC) records.
B	Allocate COMPUTATIONAL–3 fields internally as DISPLAY fields outside RECORDING MODE F, U, and V records.
C	Use EBCDIC allocation and alignment rules outside the File Section for DISPLAY, COMPUTATIONAL, COMPUTATIONAL–1 and COMPUTATIONAL–2 fields.

Table 15-1 summarizes conditions under which IBM OS/360 COBOL or OS 2200 ASCII COBOL allocation and alignment rules apply. Table 15-2 describes the meaning and internal allocation of USAGE clauses. Table 15-3 gives the alignment of these clauses.

Table 15-1. Allocation and Alignment Rules

Options specified in Extra Options field of @ ACOB processor call statement	File Section				Non-File Section	
	Recording Mode F, U, and V Records		Non-Recording Mode F, U, and V Records			
	USAGE DISPLAY COMPUTATIONAL, COMPUTATIONAL-1, COMPUTATIONAL-2	USAGE COMPUTATIONAL-3	USAGE DISPLAY COMPUTATIONAL, COMPUTATIONAL-1, COMPUTATIONAL-2	USAGE COMPUTATIONAL-3	USAGE DISPLAY COMPUTATIONAL, COMPUTATIONAL-1, COMPUTATIONAL-2	USAGE COMPUTATIONAL-3
Not A, B, C	IBM OS/360	IBM OS/360	OS 2200	IBM OS/360	OS 2200	IBM OS/360
A	OS 2200	OS 2200	OS 2200	IBM OS/360	OS 2200	IBM OS/360
B	IBM OS/360	IBM OS/360	OS 2200	OS 2200	OS 2200	OS 2200
C	IBM OS/360	IBM OS/360	OS 2200	IBM OS/360	IBM OS/360	IBM OS/360
AB	OS 2200	OS 2200	OS 2200	OS 2200	OS 2200	OS 2200
AC	OS 2200	OS 2200	OS 2200	IBM OS/360	IBM OS/360	IBM OS/360
BC	IBM OS/360	IBM OS/360	OS 2200	OS 2200	IBM OS 360	OS 2200
ABC	OS 2200	OS 2200	OS 2200	OS 2200	IBM OS/360	OS 2200

Table 15-2. Meaning and Allocation of Data Types by USAGE

Usage	Meaning	Allocation
DISPLAY	ASCII characters	9 bits, quarter word, per allocatable character represented in PICTURE clause
COMPUTATIONAL	Fixed-point binary, also called ASCII sync-binary	OS 2200 Allocation: 1 – 2 digits, 1/4 word, 9 bits 3 – 5 digits, 1/2–word, 18 bits 6 – 7 digits, 3/4 word, 27 bits 8 – 10 digits, 1 word, 36 bits 11 – 13 digits, 1–1/4 word, 45 bits 14 – 15 digits, 1–1/2 word, 54 bits 16 – 18 digits, 1–3/4 word, 63 bits IBM OS/360 Allocation: 1 – 4 digits, 1/2 word, 18 bits 5 – 9 digits, 1 word, 36 bits 10 – 18 digits, 2 words, 72 bits
COMPUTATIONAL–1	Single precision signed floating point	OS 2200 Allocation: 36 bits, synchronized in one computer word IBM OS/360 Allocation: 36 bits, 9 bit synchronized
COMPUTATIONAL–2	Double precision signed floating point	OS 2200 Allocation: 72 bits synchronized in two computer words IBM OS/360 Allocation: 72 bits synchronized, 9 bit synchronized
COMPUTATIONAL–3	Numeric characters converted to or from packed decimal on output or input for F, U, V recording modes	OS 2200 Allocation: 9 bits, quarter-word, per digit
	Packed decimal*	IBM OS/360 Allocation: 1 digit, 1/4 word, 9 bits 2 – 3 digits, 1/2 word, 18 bits 4 – 5 digits, 3/4 word, 27 bits 6 – 7 digits, 1 word, 36 bits 8 – 9 digits, 1–1/4 words, 45 bits 10 – 11 digits, 1–1/2 words, 54 bits 12 – 13 digits, 1–3/4 words, 63 bits 14 – 15 digits, 2 words, 72 bits 16 – 17 digits, 2–1/4 words, 81 bits 18 digits, 2–1/2 words, 90 bits

** Each 9-bit byte consists of one slack bit and two 4-bit decimal digits, except for the right-most byte which contains one slack bit, one 4-bit decimal digit and one 4-bit sign field. This sign field may contain a hexadecimal digit "A" through "F". The sign values "A", "C", "E", and "F" will be treated as plus signs and "B" and "D" will be treated as minus signs. Data Manipulation Library (DML) routines and initial value generation only deposit the sign values "C" and "D" in signed COMPUTATIONAL–3 items, and "F" in unsigned COMPUTATIONAL–3 items.*

Table 15-3. Alignment of Data Types by USAGE

Usage	Alignment
DISPLAY	Quarter-word boundary
COMPUTATIONAL	Quarter-word boundary
COMPUTATIONAL–1	OS 2200 Allocation Word boundary IBM OS/360 Allocation Quarter-word boundary
COMPUTATIONAL–2	OS 2200 Allocation Word boundary IBM OS/360 Allocation Quarter-word boundary
COMPUTATIONAL–3	Quarter-word boundary

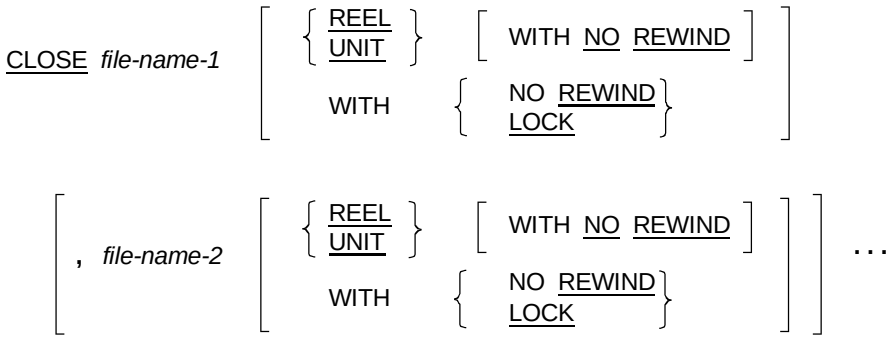
15.4. Procedure Division

15.4.1. The CLOSE Statement

15.4.1.1. Function

The CLOSE statement terminates the processing of reels/units and files with optional rewind and/or lock where applicable.

15.4.1.2. General Format



15.4.1.3. Syntax Rule

The files referenced in the CLOSE statement need not all have the same organization or access.

15.4.1.4. General Rules

Except where otherwise stated in the general rules that follow, the terms "reel" and "unit" are synonymous and completely interchangeable in the CLOSE statement.

1. A CLOSE statement may only be executed for a file in an open mode.
2. For the purpose of showing the effect of various types of CLOSE statements as applied to various storage media, all files are divided into the following categories:
   a. Sequential single-reel/unit. A sequential file that is entirely contained on one reel/unit.
   b. Sequential multi-reel/unit. A sequential file that is contained on more than one reel/unit.
3. The results of executing each type of CLOSE for each category of file are summarized in Table 15-4.

Table 15-4. Relationship of Categories of Files and the Formats of the CLOSE Statement

CLOSE Statement Format	File Category	
	Sequential Single-Reel/Unit	Sequential Multi-Reel/Unit
CLOSE	F	C,F,A
CLOSE WITH LOCK	F,D	C,F,E,A
CLOSE WITH NO REWIND	C,B	C,B,A
CLOSE REEL/UNIT	X	E,F
CLOSE REEL/UNIT WITH NO REWIND	X	E

The definitions of the symbols in the table follow. Where the definition depends on whether the file is an input or output file, alternate definitions are given; otherwise, a definition applies to input and output files.

A. Previous Reels/Units Unaffected

Input Files:

All reels/units in the file prior to the current reel/unit are processed according to the standard reel/unit swap procedure, except those reels/units controlled by a prior CLOSE REEL/UNIT statement. If the current reel/unit is not the last in the file, the reels/units in the file following the current one are not processed. (See Appendix E.)

Output Files:

All reels/units in the file prior to the current reel/unit are processed according to the standard reel/unit swap procedure, except those reels/units controlled by a prior CLOSE REEL/UNIT statement. (See Appendix E.)

B. No Rewind of Current Reel

The current reel/unit is left in its current position.

C. Close File

Input Files:

If the file is positioned at its end and label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If the file is positioned at its end and label records are not specified for the file, label processing does not take place but other closing operations are executed. If the file is positioned other than at its end, the closing operations are executed, but there is no ending label processing. (See Appendix E.)

Output Files:

If label records are specified for the file, the labels are processed according to the standard label convention. The behavior of the CLOSE statement when label records are specified but not present, or when label records are not specified but are present, is undefined. Closing operations are executed. If label records are not specified for the file, label processing does not take place but other closing operations are executed. (See Appendix E.)

D. File Lock

This file cannot be opened again during this execution of this run unit.

E. Close Reel/Unit

Input Files:

The following operations take place:

1. A reel/unit swap.
2. The standard beginning reel/unit label procedure is executed.

The next executed READ statement for that file makes available the next data record on the new reel/unit.

Output Files:

The following operations take place:

1. The standard ending reel/unit label procedure is executed.
2. A reel/unit swap.
3. The standard beginning reel/unit label procedure is executed.

   The next executed WRITE statement that references that file directs the next logical data record to the next reel/unit of the file.

F Rewind

The current reel is positioned at its physical beginning.

X Illegal

This is an illegal combination of a CLOSE option and a file category. The results at object time are undefined.

4. The action taken if the file is in the open mode when a STOP RUN statement is executed or the action taken for a file that has been opened in a called program and not closed in that program prior to the execution of a CANCEL statement for that program are undefined. No implicit CLOSE procedures will be initiated. This could result in the condition which would cause the file to be unusable on subsequent accesses to it.
5. If the OPTIONAL phrase has been specified for the file in the FILE– CONTROL paragraph of the Environment Division and the file is not present, the standard end–of–file processing is not performed for that file.
6. If a CLOSE statement without the REEL or UNIT phrase has been executed for a file, no other statement (except the SORT or MERGE statements with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly, unless an intervening OPEN statement for that file is executed.
7. Following the successful execution of a CLOSE statement without the REEL or UNIT phrase, the record area associated with file–name is no longer available. The unsuccessful execution of such a CLOSE statement leaves the availability of the record area undefined.

15.4.2. The OPEN Statement

15.4.2.1. Function

The OPEN statement initiates the processing of files. It also performs checking and writing of labels and other input-output operations.

15.4.2.2. General Format

```
OPEN { INPUT file-name-1 [ WITH NO REWIND ] [ , file-name-2 [ WITH NO REWIND ]]...
     { OUTPUT file-name-3 [ WITH NO REWIND ] [ , file-name-4 [WITH NO REWIND ]]... } ...
```

15.4.2.3. Syntax Rule

The files referenced in the OPEN statement need not all have the same organization or access.

15.4.2.4. General Rules

1. The successful execution of an OPEN statement determines the availability of the file and results in the file being in an open mode.
2. The successful execution of an OPEN statement makes the associated record area available to the program.
3. Prior to the successful execution of an OPEN statement for a given file, no statement (except for a SORT or MERGE statement with the USING or GIVING phrases) can be executed that references that file, either explicitly or implicitly.
4. An OPEN statement must be successfully executed prior to the execution of any of the permissible input-output statements. In Table 15-5, an "X" at an intersection indicates that the specified statement, used in the sequential access mode, may be used with the sequential file organization and open mode given at the top of the column.

Table 15-5. Permissible Statements

Statement	Open Mode	
	Input	**Output**
READ	X	
WRITE		X

5. A file may be opened with the INPUT and OUTPUT phrases in the same program. Following the initial execution of an OPEN statement for a file, each subsequent OPEN statement execution for that same file must be preceded by the execution of a CLOSE statement, without the REEL, UNIT, or LOCK phrase, for that file.
6. Execution of the OPEN statement does not obtain or release the first data record.
7. If label records are specified for the file (see Appendix E), the beginning labels are processed as follows:
   a. When the INPUT phrase is specified, the execution of the OPEN statement causes the labels to be checked in accordance with the conventions for input label checking.
   b. When the OUTPUT phrase is specified, the execution of the OPEN statement causes the labels to be written in accordance with the conventions for output label writing.

   The behavior of the OPEN statement when label records are specified but not present, or when label records are not specified but are present, is undefined.
8. The file description entry for *file-name-1* or *file-name-2* must be equivalent to that used when this file was created.
9. If an input file is designated with the OPTIONAL phrase in its SELECT clause, the object program causes an interrogation for the presence or absence of this file. If the file is not present, the first READ statement for this file causes the AT END condition to occur. (See 15.4.3.)

10. The NO REWIND phrase can only be used with sequential single reel/unit files. (See 15.4.1.)
11. Since the storage medium for the file permits rewinding, the following rules apply:
    a. When the NO REWIND phrase is not specified, execution of the OPEN statement causes the file to be positioned at its beginning.
    b. When the NO REWIND phrase is specified, execution of the OPEN statement does not cause the file to be repositioned; the file must be already positioned at its beginning prior to execution of the OPEN statement.
12. For files being opened with the INPUT phrase, the OPEN statement sets the current record pointer to the first record currently existing within the file. If no record exists in the file, the current record pointer is set such that the next executed READ statement for the file will result in an AT END condition.
13. Upon successful execution of an OPEN statement with the OUTPUT phrase specified, a file is created. At that time the associated file contains no data records.

15.4.3. The READ Statement

15.4.3.1. Function

The READ statement makes available the next logical record from a file.

15.4.3.2. General Format

```
READ file-name RECORD [ INTO identifier ] ; AT END imperative-statement
```

15.4.3.3. Syntax Rule

The INTO phrase must not be used when the input file contains logical records of various sizes as indicated by their record descriptions. The storage area associated with identifier and the record area associated with *file–name* must not be the same storage area.

15.4.3.4. General Rules

1. The associated file must be open in the INPUT mode at the time this statement is executed. (See 15.4.2.)
2. The record to be made available by the READ statement is determined as follows:
    a. If the current record pointer was positioned by the execution of the OPEN statement, the record pointed to by the current record pointer is made available.
    b. If the current record pointer was positioned by the execution of a previous READ statement, the current record pointer is updated to point to the next existing record in the file and then that record is made available.
3. Regardless of the method used to overlap access time with processing time, the concept of the READ statement is unchanged in that a record is available to the object program prior to the execution of any statement following the READ statement.

4. When the logical records of a file are described with more than one record description, these records share the same storage area. This is equivalent to an implicit redefinition of the area. The contents of any data items which lie beyond the range of the current data record are undefined at the completion of the execution of the READ statement.
5. If the INTO phrase is specified, the record being read is moved from the record area to the area specified by *identifier* according to the rules specified for the MOVE statement without the CORRESPONDING phrase. The implied MOVE does not occur if the execution of the READ statement was unsuccessful. Any subscripting or indexing associated with *identifier* is evaluated after the record has been read and immediately before it is moved to the data item.
6. When the INTO phrase is used, the record being read is available in both the input record area and the data area associated with *identifier*.
7. If, at the time of execution of a READ statement, the position of current record pointer for that file is undefined, the execution of that READ statement is unsuccessful.
8. If the end of a reel or unit is recognized during execution of a READ statement, and the logical end of the file has not been reached, the following operations are executed:
   a. The standard ending reel/unit label procedure. (See Appendix E.)
   b. A reel/unit swap.
   c. The standard beginning reel/unit label procedure. (See Appendix E.)
   d. The first data record of the new reel/unit is made available.
9. If a file described with the OPTIONAL phrase is not present at the time the file is opened, then at the time of execution of the first READ statement for the file, the AT END condition occurs and the execution of the READ statement is unsuccessful. The standard end-of-file procedures are not performed. (See 15.2.1.1 and 15.4.2.) Execution of the program then proceeds as specified in general rule 11.
10. If, at the time of the execution of a READ statement, no next logical record exists in the file, the AT END condition occurs, and the execution of the READ statement is considered unsuccessful.
11. When the AT END condition is recognized, control is transferred to the AT END *imperative-statement*.

    When the AT END condition occurs, execution of the input–output statement which caused the condition is unsuccessful.
12. Following the unsuccessful execution of any READ statement, the contents of the associated record area and the position of the current record pointer are undefined.
13. When the AT END condition has been recognized, a READ statement for that file must not be executed without first executing a successful CLOSE statement followed by the execution of a successful OPEN statement for that file.

15.4.4. The WRITE Statement

15.4.4.1. Function

The WRITE statement releases a logical record for an output file.

15.4.4.2. General Format

```
WRITE record-name [ FROM identifier ]
```

15.4.4.3. Syntax Rules

1. *Record-name* and *identifier* must not reference the same storage area.
2. The *record-name* is the name of a logical record in the File Section of the Data Division and may be qualified.

15.4.4.4. General Rules

1. The associated file must be open in the OUTPUT mode at the time of the execution of this statement. (See 15.4.2.)
2. The logical record released by the execution of the WRITE statement is no longer available in the record area unless the associated file is named in a SAME RECORD AREA clause or the execution of the WRITE statement was unsuccessful due to a boundary violation. The logical record is also available to the program as a record of other files referenced in the same SAME RECORD AREA clause as the associated output file, as well as to the file associated with *record-name*.
3. The results of the execution of the WRITE statement with the FROM phrase is equivalent to the execution of:

   a. The statement:

      ```
      MOVE identifier TO record-name
      ```

      according to the rules specified for the MOVE statement, followed by:

   b. The same WRITE statement without the FROM phrase.

   The contents of the record area prior to the execution of the implicit MOVE statement have no effect on the execution of this WRITE statement.

   After execution of the WRITE statement is complete, the information in the area referenced by *identifier* is available, even though the information in the area referenced by *record-name* may not be. (See general rule 2.)
4. The current record pointer is unaffected by the execution of a WRITE statement.
5. The maximum record size for a file is established at the time the file is created and must not subsequently be changed.
6. The execution of the WRITE statement releases a logical record to the operating system.

7. After the recognition of an end-of-reel or an end-of-unit of an output file that is contained on more than one physical reel/unit, the WRITE statement performs the following operations:
   a. The standard ending reel/unit label procedure. (See Appendix E.)
   b. A reel/unit swap.
   c. The standard beginning reel/unit label procedure. (See Appendix E.)

15.5. IBM Tape File Assignment

The link between the ASCII COBOL defined file name and the OS 2200 File Control System name should be specified in the runstream (@ASG or @USE) prior to the execution of any program containing IBM tape file processing.

15.5.1. Assignment Rules

1. The file must be assigned to a valid tape device.
2. The Q (quarter-word) data transfer format must be specified in the files assignment.
3. When hardware translation is specified, the processor code must be ASCII and the tape code must be EBCDIC.
4. The presence of the J option in the file's assignment will cause ASCII COBOL to process the file's labels. (See E.3.2). For ASCII COBOL to process the labels, Exec configuration tag TLEBCDIC must be set to 0 or 1. TLEBCDIC set to 1 is preferred.
5. The absence of the J option in the file's assignment will cause the Exec (40R3 and higher) to process the file's labels. (See E.3.2). For the Exec to process the labels, Exec configuration tag TLEBCDIC must be set to 1 or 2. TLEBCDIC set to 1 is preferred.

For more information, see the *Exec System Software Installation and Configuration Guide*.

15.5.2. Assignment Examples

The following are specific examples of the OS 2200 Executive @ASG control statements for IBM tape files. Consult the *OS 2200 Executive System Software Executive Control Language (ECL) End Use and Programming Reference Manual* (7830 7949) for general rules and formats.

```
@ASG,TJ IBM1.,U9S/////Q,5678

@ASG,T IBM2.,U9S/////Q,2345

@ASG,TJ IBM3.,U9V////ASCII/EBCDIC/Q,SCRTCH

@ASG,T IBM4.,HICL//////Q////BUFON,483
```

15.5.3. COBOL Program Links

The links between the COBOL-defined files and the File Control System files named in 15.5.2 are provided by the following COBOL source language statements:

```
SELECT IBM-FILE ASSIGN TO UNISERVO IBM1.

SELECT EBCDIC ASSIGN TO UNISERVO IBM2.

SELECT IBM-HT ASSIGN TO UNISERVO IBM3.

SELECT IBM-CART ASSIGN TO UNISERVO IBM4.
```

The COBOL file IBM–FILE would be linked with an unlabeled tape number 5678 with the File Control System name IBM1.

The COBOL file EBCDIC would now be associated with labeled tape number 2345 with the File Control System name IBM2.

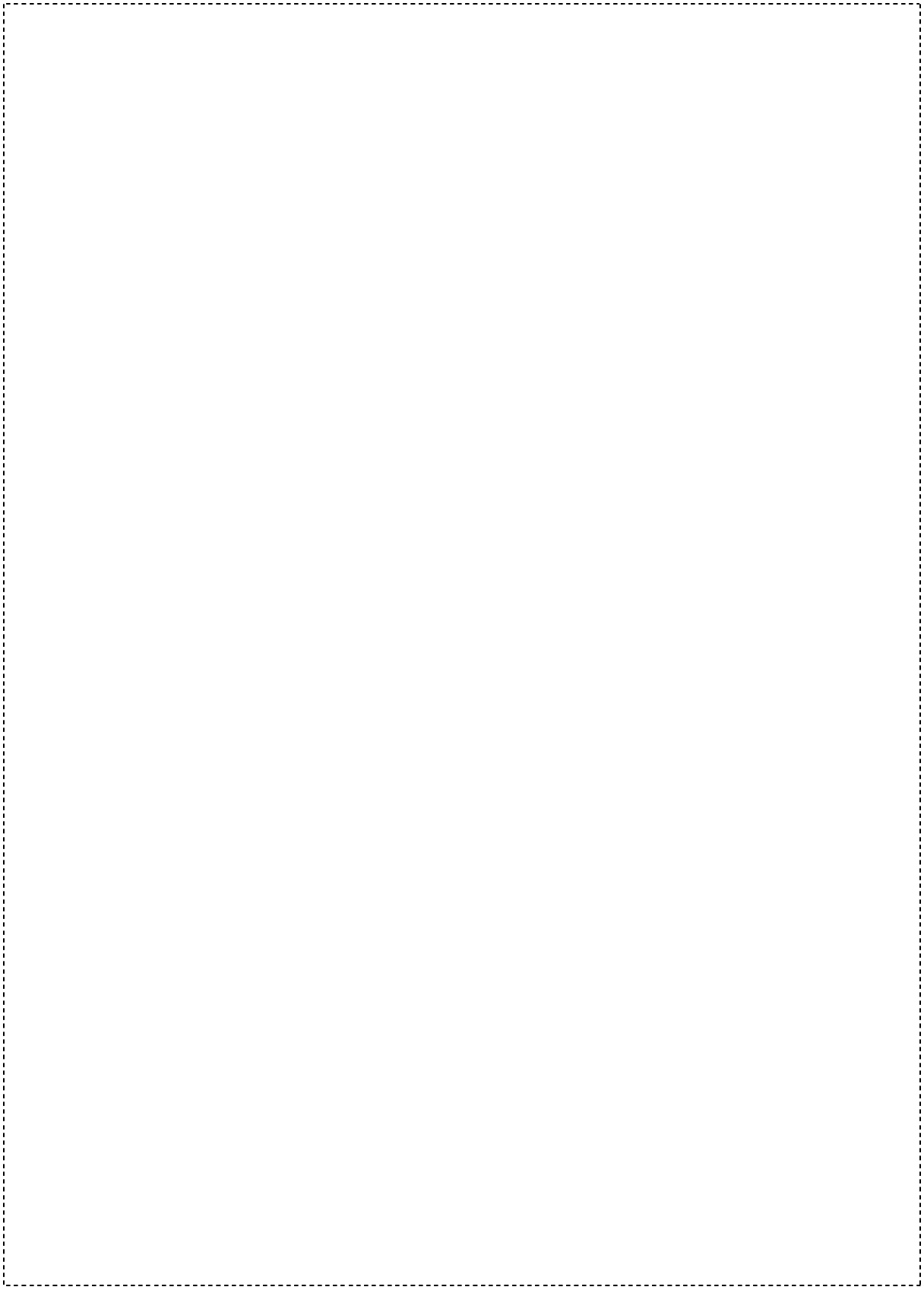
The COBOL file IBM–HT would now be associated with an unlabeled SCRTCH tape with the File Control System name IBM3, and will be hardware–translated to and from ASCII and EBCDIC.

The COBOL file IBM–CART would now be linked with labeled tape 483, where tape 483 is a half-inch cartridge volume assigned to a 0899 cartridge tape device in buffered write mode.

15.6. EBCDIC Character Codes

The EBCDIC character codes for the characters '!', '[' and ']' differ depending upon the operating system (OS/3 vs. IBM OS/360). When processing EBCDIC data, the ASCII COBOL run-time library defaults to the OS/3 character code set. If an IBM OS/360 character code set is needed, an ASCII COBOL build must be performed to apply a local change to the run-time library element C$DJAE (change tag OS3$ to zero).

If hardware translation is specified, the OS/3 character code set is always used.



Section 16
Information Interchange Tape Processing Module

16.1. General

The Information Interchange Tape Processing Module is a subset of the Sequential I–O Module which refers to tape files. Except for the restrictions imposed by this section, all formats, syntax rules and general rules which apply to tape files also apply to Information Interchange files. However, the formats, syntax rules, and general rules that apply to Information Interchange files are Unisys defined.

The Information Interchange Tape Processing Module is specifically designed to process tapes which are in the formats described in the X3.27–1987 (R 1998) Magnetic Tape Labels for Information Interchange. Tapes in this format are preferred for exchange of data between the systems of different computer manufacturers.

16.2. Environment Division

16.2.1. Input–Output Section

16.2.2.The FILE–CONTROL Paragraph

16.2.2.1. Function

The FILE–CONTROL paragraph names each file and allows specification of other file-related information.

16.2.2.2. General Format

```
FILE-CONTROL. { file-control-entry }   . . .
```

16.2.3. The File Control Entry

16.2.3.1. Function

The file control entry names a file and may specify other file-related information.

16.2.3.2. General Format

```
SELECT [ OPTIONAL ] file-name

    ASSIGN TO implementor-name-1  [ , implementor-name-2 ]...

    [ ; RESERVE integer-1 [ AREA | AREAS ] ]

    [ ; ORGANIZATION IS SEQUENTIAL ]

    [ ; ACCESS MODE IS SEQUENTIAL ]

    [ ; FILE STATUS IS data-name-1 ].
```

16.2.3.3. Syntax Rules

1. The SELECT clause must be specified first in the file control entry. The clauses that follow the SELECT clause may appear in any order.
2. Each file described in the Data Division must be named once and only once as file-name in the FILE–CONTROL paragraph. Each file specified in the file control entry must have a file description entry in the Data Division.
3. The only allowable *implementor-name-1* is INTERCHANGE.
4. If *implementor-name-2* is specified, it is a 1- to 12-character name used to link the COBOL file to OS 2200 Executive File Control and should appear on an @ASG or @USE statement in the run stream for executing the object program.
5. When *implementor-name-2* is not specified, the first 12 characters of *file-name* are used to link the COBOL file to OS 2200 Executive File Control.
6. All recursions of *implementor-name-1*, *implementor-name-2*, are for documentation only.

7. If the ACCESS MODE clause is not specified, the ACCESS MODE IS SEQUENTIAL clause is implied.
8. *Data-name-1* must be defined in the Data Division as a 2-character data item of the category alphanumeric and must not be defined in the File Section, the Report Section, or the Communication Section. *Data-name-1* must be defined as USAGE DISPLAY.
9. *Data-name-1* may be qualified.
10. When the ORGANIZATION IS SEQUENTIAL clause is not specified, the ORGANIZATION IS SEQUENTIAL clause is implied.
11. The OPTIONAL phrase may only be specified for input files. Its specification is required for input files that are not necessarily present each time the object program is executed.

16.2.3.4. General Rules

1. The ASSIGN clause specifies the association of the file referenced by *file-name* to a storage medium.
2. If the RESERVE clause is specified, it is for documentation only. Two input-output areas are always allocated.
3. The ORGANIZATION clause specifies the logical structure of a file. The file organization is established at the time a file is created and cannot subsequently be changed.
4. Records in the file are accessed in the sequence dictated by the file organization. This sequence is specified by predecessor-successor record relationship established by the execution of WRITE statements when the file is created or extended.
5. When the FILE STATUS clause is specified, a value will be moved by the ASCII COBOL runtime system into the data item specified by *data-name-1* after the execution of every statement that references that file either explicitly or implicitly. This value indicates the status of execution of the statement. (See 4.1.3.4.)

16.3. Data Division

16.3.1. File Section

16.3.2. The RECORDING MODE Clause

16.3.2.1. Function

The RECORDING MODE clause is used to specify a tape format specified in X3.27–1987 (R1998) Magnetic Tape Labels for Information Interchange.

16.3.2.2. General Format

RECORDING MODE IS { D / F / S / U }

16.3.2.3. General Rule

The structure of the physical data blocks may be defined by one of the following:

1. The D phrase specifies variable length blocked records.
2. The F phrase specifies fixed blocked records.
3. The S phrase specifies spanned blocked records.
4. The U phrase specifies variable length unblocked records.

16.3.3. Record Description Restrictions

Data items in a record description for an INTERCHANGE file abide by the following restrictions:

- The usage for each data item must be DISPLAY.
- The description of each signed numeric data item must include the SIGN IS LEADING SEPARATE CHARACTER clause.

16.4. Information Interchange Tape File Assignment

The link between the ASCII COBOL defined file name and the OS 2200 File Control System name should be specified in the runstream (@ASG or @USE) prior to the execution of any program containing Information Interchange tape file processing.

16.4.1. Assignment Examples

The following are specific examples of the OS 2200 Executive @ASG control statements for Information Interchange files. Consult *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899) for general rules and formats.

```
@ASG,T TAPE3.,HICL//////Q,1234

@ASG,TJ TAPE4.,HIS//////Q,9012
```

16.4.2. COBOL Program Links

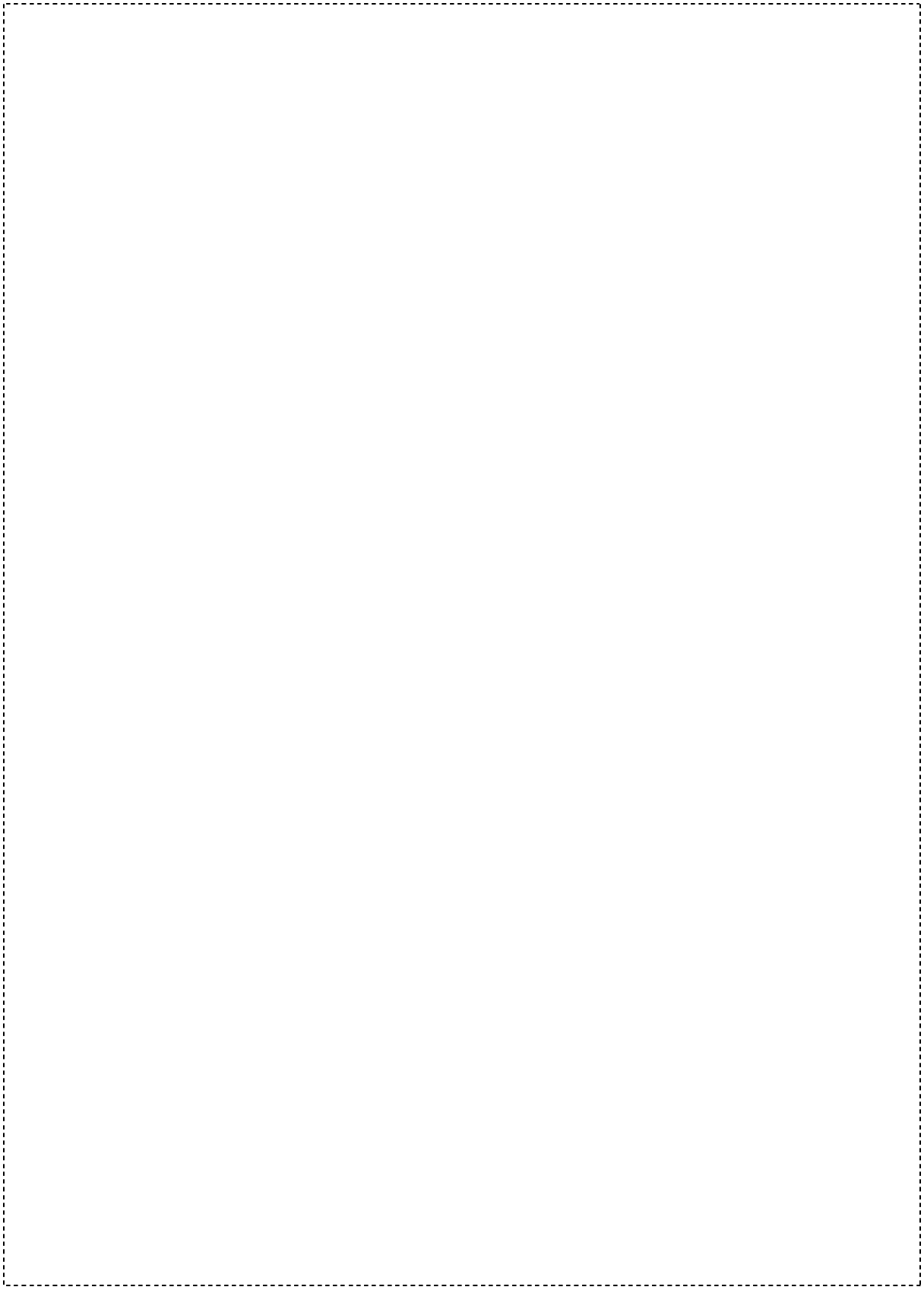
The links between COBOL-defined files and the File Control System files named in 16.4.1 are provided in the following COBOL source language statements:

```
SELECT NAME-FILE ASSIGN TO INTERCHANGE TAPE3.

SELECT INTERCHANGE-FILE ASSIGN TO INTERCHANGE TAPE4.
```

The COBOL file NAME–FILE would be associated with labeled tape number 1234 which has the File Control System name TAPE3.

The COBOL file INTERCHANGE–FILE would be associated with unlabeled tape number 9012 which has the File Control System name TAPE4.



Section 17
Optimization

17.1. General

COBOL programmers can easily optimize a COBOL program by utilizing the language features and hints documented in this section.

17.2. Programming Hints

This section contains a collection of hints for writing efficient and maintainable COBOL programs. Many of these suggestions may be found in the other sections of this document and many are common sense approaches to good COBOL programming techniques. The majority of the following recommendations are based on the COBOL language and should, therefore, apply to most COBOL compilers. Others, however, are based on special considerations for the OS 2200 Executive environment or for the ASCII COBOL implementation.

17.2.1. Program Maintainability

1. Do not use the ALTER statement.
2. Use meaningful names. Do not use numeric paragraph and section names.
3. Avoid the use of the OPEN REVERSED facility.
4. Use 01 level-number instead of the 77 level-number.
5. Use the Report Writer facility for report generation.
6. Use the COPY statement for common data structures and common procedures.
7. Use comment lines freely.
8. Use indentation to clearly depict the control flow of conditional statements.
9. The readability (and maintainability) of a program will be improved if structured control statements (for example, IF, PERFORM) are used in place of GO TO statements in as many instances as possible.

17.2.2. Execution Speed

Increased execution speed can generally be obtained in the following areas.

17.2.2.1. General

1. Moves and arithmetic operations are performed faster on unsigned data than signed data.
2. Arithmetic operations are generally faster if they are done on binary fields (COMPUTATIONAL). If input is in characters, yet considerable calculations are done, it is more efficient to force a conversion by moving DISPLAY data items to appropriate COMPUTATIONAL items, and then reference the COMPUTATIONAL items in arithmetic statements or expressions.
3. Where possible, grouping of references to the same operands enables the compiler to better optimize generated code.
4. The use of multiple receiving fields is more efficient than repeating calculations.

5. When multiple receiving fields are stated they should be grouped by usage and size, and the groups arranged in an order to simplify conversion. That is, if a COMPUTE will result in a single precision floating-point intermediate result, receiving fields should be ordered.

   a. Floating-point

   b. Binary, grouping like size items together

   c. DISPLAY or COMPUTATIONAL–3

   If the computation produces a DISPLAY result, the order of receiving fields should be reversed.

6. Indexing is more efficient than subscripting.

7. Group and alphanumeric moves are faster than numeric moves.

8. PERFORM VARYING is less efficient than PERFORM *n* TIMES with the user writing the ADD statements to emulate the VARYING option.

9. Use of the SEARCH verb is more efficient in scanning tables especially if the SEARCH ALL option is used.

17.2.2.2. Input/Output

1. DISPLAY and ACCEPT are less efficient than writing to a printer file or reading from a card-reader file.

2. Multi-key Indexed files should use the RESERVE clause to allow as many buffer areas as possible for processing. If a RESERVE clause is specified, the COBOL compiler causes the use of (4 + *n* + *a*) buffers at run time where *n* = *integer* - 1 of the RESERVE clause and *a* = number of alternate keys. When no RESERVE clause is stated, the number of buffers will be (5 + *a*). Thus, when there are no alternate keys, the minimum number of buffers allowed is 5. The compiler-generated number of buffers can be altered by means of the Define File Processor (see 20.2).

3. Block size for a Multi-Key Indexed file is determined by the compiler in the following manner. In the absence of a BLOCK clause in the file description entry, the compiler generates a block size which is the larger of a. and b. which follow, rounded up to a multiple of 448 words.

   a. 200*(*kl* + 1 + *x*) + 8

      where *kl* = largest key length in words; *x* = 1 if duplicate keys are allowed, otherwise *x* = 0.

   b. 10*(*mrl* + 2 + *ndk*) + 6

      where *mrl* = maximum record length in words; *ndk* = number of keys with duplicates.

This block size is the minimum the COBOL compiler will allow, even in the case where a BLOCK clause is specified. If a BLOCK CONTAINS *integer–2* CHARACTERS clause is specified, the COBOL compiler rounds *integer–2* (ASCII) characters up to a number of words and then rounds up again to a multiple of 448 words. This computed block size must be at least as large as the minimum acceptable block size. If a BLOCK CONTAINS *integer–2* RECORDS clause is specified, the compiler calculates the block size, rounded up to a multiple of 448 words, as:

integer–2 * (*mrl* + 2 + *ndk*) + 6

This computed block size is checked against the minimum acceptable block size. In any case, block size should be sufficient to allow for at least 10 maximum length records or 200 maximum length keys. The compiler-generated block size can be altered by means of the Define File Processor (see 20.2).

4. When a Multi-Key Indexed file is created and *integer-1* of the BLOCK clause is specified, the ratio of *integer-1* divided by *integer-2* of the BLOCK clause indicates the portion of each data block and index block which is filled. The remaining portion of the block (called the block loading factor when expressed in words) is then available for insertion of additional records in the file within the range of keys originally put into that block. The compiler-generated block loading factor can be altered by use of the Define File Processor (see 20.2). If a significant amount of activity on the file involves deletions or insertions of records, the portion of reserved space in each block should be higher at file creation time than if few deletions or insertions are anticipated.

5. Initial creation of a Multi-Key Indexed file is slightly faster if the access mode is sequential compared to the random or dynamic access modes. This does, however, impose the restriction that the prime record keys must be presented in an ascending sequence.

6. Blocksize for a RELATIVE DISC file is determined in the following manner:

```
MINBUF = MRL + NCW + 2*(PF-1)+1
```

where:

MRL = maximum record length in words,

NCW = number of control words per record = (MRL+2046)/2047

PF = prepping factor (default is 112)

If the resulting MINBUF is less than 1792 words (in other words, 1 track), then 1792 will be assumed.

Through the use of the Define File Processor, the default of 1792 can be overridden. A DFP specified blocksize, in terms of characters, will be used if it exceeds the MINBUF as calculated above. A DFP specified blocksize in terms of records, will cause a multiple of MINBUF to be used. DFP may also be used to alter PF in the MINBUF calculation (see 20.2).

7. Blocksize for a SEQUENTIAL DISC file is always rounded up to a multiple of the prepping factor. The default prepping factor is 112, but it may be increased to 224, 448, 896, or 1792 by means of the Define File Processor (see 20.2).

17.2.2.3. Sort

1. Allows perform a disk sort (in other words, assign three or more equally sized scratch files).
2. Perform an extended mode sort versus a basic mode sort. Assign the file R$CORE with a maximumn reserve of at least 211 granules. This size represents a sort memory size in thousands of words. The larger the size generally implies a more efficient sort.

17.2.2.4. Data Alignment and Data Types

1. Conversion from one data format to another is costly. Consistent data descriptions will generally result in faster execution.
2. For indexing or subscripting, better code is generated if all the dimensions have integral word sizes; in other words, all the items with OCCURS clauses have sizes which are an integral multiple of 4 ASCII bytes.
3. Data should be aligned as much as possible to word boundaries.
4. Alphanumeric moves and compares are most efficient if both operands start and end on the word boundary. Equal start byte moves are more efficient than unequal start byte moves.
5. Alphanumeric moves and compares of unequal length operands are more costly than equal length moves of similar length. An exception to this rule is when the longer operand is longer by 3 words or more.
6. Certain operations on decimal fields are done more efficiently if the "S" is omitted. For example, a DISPLAY data item which is defined with an operational sign would perhaps require more elaborate handling to convert it to a binary format.

17.2.2.5. Execution of Extended Instruction Set (EIS) Instruction

Execution speed generally can be increased by using the EIS instructions generated by the compilier. See 2.4.1.2.4 for the steps necessary to cause EIS instructions to be generated.

17.2.2.6. Generation of Inline Code

When EIS instructions are not generated the following recommendation will improve code efficiency:

1. For small fields (1 – 4 ASCII bytes or 1 – 36 bits) which are frequently referenced in arithmetic move and compare statements, the best inline code is generated when the fields are j-factorable; in other words, allocated in one whole word, a physical half-word, or a physical quarter-word. Avoid ASCII fields of 3 bytes.
2. Inline code will be generated for the following functions:
   a. Alphanumeric moves when the sending and receiving fields have equal start bytes and size of receiver is less than 512 bytes in length; if both sender and receiver start at byte 0 there is no restriction on length.

b. All comparisons where both fields have equal start bytes, are less than 512 bytes in length, and are equal in length, or if after scaling and padding two items of different lengths are less than two words in length.

c. ASCII to binary conversion for 1 – 4 bytes, unsigned.

d. Binary to ASCII conversions for 1 – 6 bytes unsigned.

e. Moving 1 or 2 characters ASCII to any alphanumeric field.

17.2.2.7. Programmer-Controlled Optimization

1. Optimization regions should be determined. For smaller programs this region could be the entire Procedure Division. For larger programs the optimization regions should be high-use areas. A frequency analysis of programs can be made by using the 7 option in the Extra Options field of the @ACOB statement, the USE FOR DEBUGGING syntax, and the object listing of the program. (See 17.3.)
2. When the optimization regions are determined the following techniques can be applied.
   a. For cases where inline code is not being generated, operands could possibly be changed so that inline code is generated.
   b. Moves of equal size fields with different starting bytes, which cannot be changed to generate inline code, could be changed to several moves of smaller parts of the original fields through use of REDEFINES clause, redefining the fields to j-factorable parts of the receiving field.
   c. Inefficient conversions and scalings could be eliminated.
   d. The COBOL algorithms could be changed, for example, the most likely pattern in a complex IF statement should be the first comparison made.

17.2.3. Compilation Speed

The following items have a direct bearing on compiler speed:

1. Using FILLER instead of named but unreferenced fields is more efficient.
2. Reducing the number of nonunique names and qualified references will increase compilation speed. Avoid including unnecessary qualification.
3. CORRESPONDING ADD, SUBTRACT, and MOVE statements require more time to compile than do the explicit simple MOVE, ADD, or SUBTRACT statements.
4. Large numbers of user names (procedural and data) are processed more efficiently if they are formed by appending a varying prefix or suffix to a constant.
5. Fieldata source programs are converted to ASCII upon input to the compiler.

17.3. Frequency Table Generation

Frequency table generation allows COBOL programmers to accumulate a frequency analysis of their program while running live data. This feature is invoked via the 7 option in the Extra Options field of the @ACOB statement and requires the USE FOR DEBUGGING syntax.

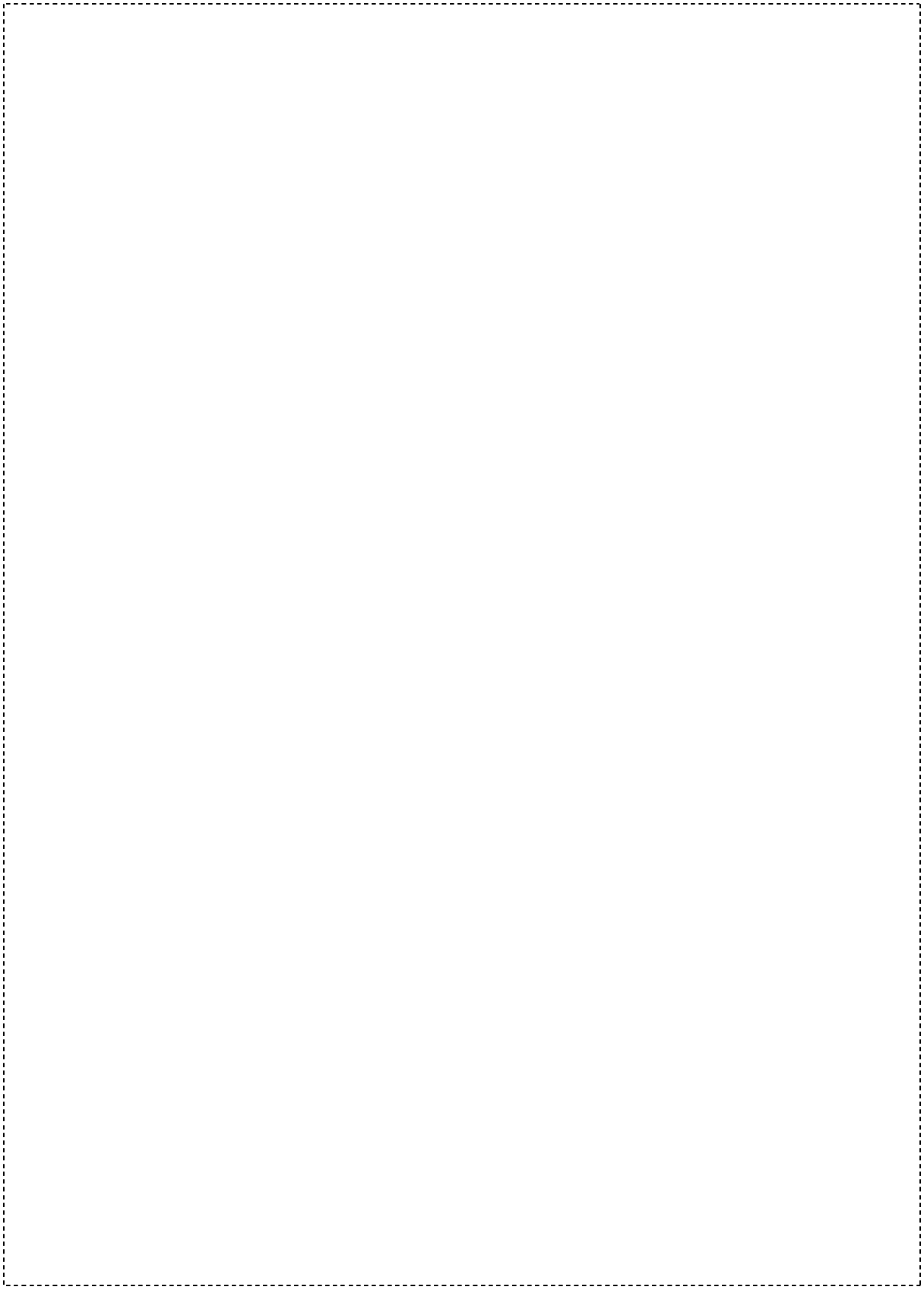
When a COBOL program is compiled with the 7 option in the Extra Options field, all procedures that are presently being debugged, via USE FOR DEBUGGING, will automatically be tabled for frequency analysis. Each time the program enters one of these tabled procedures, its frequency count is incremented by one. Upon reaching STOP RUN this frequency information is tabulated for each procedure referenced, sorted in frequency order, and printed in a report.

In order to perform such a frequency analysis, for example, on a COBOL run unit consisting of one main program named MAIN, and two subprograms named SUB1 and SUB2, the three COBOL programs MAIN, SUB1, and SUB2 must be changed to add the following syntax:

1. `WITH DEBUGGING MODE clause on the SOURCE-COMPUTER paragraph.`
2. `USE FOR DEBUGGING ON ALL PROCEDURES in the DECLARATIVES.`

Recompile each program with the 7 option in the Extra Options field of the @ACOB statement. When mapped and executed, the frequency table will be printed upon detection of a STOP RUN. The @XQT statement with D option will inhibit the building of DEBUG–ITEM but will still do the frequency count.

This feature should identify high-use sections of code in programs while running with live data. Those high-use sections identified are then candidates for examination and optimization.



Section 18
System Interface

18.1. Processor Call Statement

The @ACOB processor call statement conforms to the standard format for calling an OS 2200 processor. See OS 2200 Executive System Software Executive Control Language (ECL) Operations and Programming Reference Manual.

The option letters that are unique to the ASCII COBOL compiler are documented in the following section. The ASCII COBOL compiler uses the Source Input Routine to control the input and output of the source language elements. (See *OS 2200 System Service Routines Library (SYSLIB) Programming Reference Manual*.)

18.1.1. General Format

```
@ACOB,options SI,RO,SO,EO
```

18.1.2. General Rules

1. Options specifies the options consisting of letters. See Table 18-1 for a description of the allowable options.
2. *SI* (Source Input) specifies the program file and symbolic element name of the source language to be processed. (See *OS 2200 Executive System, Software Executive Control Language (ECL) Operations and Programming Reference Manual*.)
3. *RO* (Relocatable Output) specifies the program file and element name where the relocatable element will be placed. For a segmented program, this specifies the program file and element name given to the symbolic MAP and the main Procedure Division (Sections 0–49) relocatable element.
4. *SO* (Source Output) specifies the program file and element name given to the updated source language element. (See *OS 2200 Executive System, Software Executive Control Language (ECL) Operations and Programming Reference Manual*.)
5. *EO* (Extra Options) specifies the extra options consisting of a string of numbers or letters. See Table 18–2 for a description of the allowable options.

Table 18-1. @ACOB Control Statement Options

Option	Description
A	Accept element even if fatal errors are detected. Overrides X option.
B	Double spacing listing.
C	List library text from COPY statement. S or O option must also be present.
D	Print allocation listing.
E	Print all diagnostic messages.
F	Print all diagnostics except Remark and Warning. Overrides E option.
G	Source Input Routine option.
H	Source Input Routine option.
I	Source Input Routine option.
J	Source Input Routine option.
K	Source Input Routine option.
L	Print all listings. Same as specifying the C, D, E, O,R, and S options. Overrides all other listing options except N.
M	Ignore MONITOR statements.
N	Suppress all listings. Overrides all other listing options.
O	Print object listing. (UNLIST in column 1 turns option OFF. LIST in column 1 turns option ON. Option must be on initially for this feature to work. LIST/UNLIST only controls the option in the REPORT SECTION and the PROCEDURE DIVISION.)
P	Source Input Routine option.
Q	Source Input Routine option.
R	Print cross-reference listing. Overrides the 9 extra option.
S	Print source listing. (S–OFF in column 1 turns option OFF. SLIST in column 1 turns option ON. Option must be on initially for this feature to work.
T	Reverse DISPLAY and DISPLAY–1; implicit and DISPLAY usage mean Fieldata. Reverse COMPUTATIONAL and COMPUTATIONAL–4. This option does not affect data items described with a PICTURE clause containing 1's (exact binary). This option does not affect the special registers.
U	Source Input Routine option.
V	Indicates a called program. No starting address and no symbolic element containing collector source language will be produced for this compilation. The INITIAL clause must not be present in any CD entry.
W	Source Input Routine option.
X	Abort run if fatal errors occur.
Y	All listings written in Fieldata with PRINT$, instead of ASCII with APRINT$. (Except for W option correction lines.)
Z	Internal compiler debugging option.

Table 18-2. @ACOB Control Statement Extra Options

Extra Option	Description
0	Search the Source Input file before the file COB$PF for COPY statements not specifying a library-name.
1	Do not generate initial values for the Common-Storage section when the processor call statement has the V option set.
2	Check sequence numbers in column 1-6 of the source program.
3	Check sequence numbers in column 1-6 of the library text.
4	Initial state subprogram entry. This option is useful for low use subprograms since it saves space at the expense of time. The Compiler Temporary Area (CTA), the File Directory and the Operations Control Block (OCB) are allocated upon entrance to the subprogram and deallocated upon exit. This subprogram cannot be collected in a fixed overlayable segment or an independent segment if it contains either the APPLY EXDEF clause or the PROCESS statement. This option overrides the 8 extra option.
5	Perform syntax scan only. This option is intended for initial source program syntax checking to save compile time. No object code is generated. The following listings are not produced: allocation, cross reference, and object. All diagnostics that are requested based upon the compiler options will be issued except for diagnostics numbered 901-1000 and certain diagnostics numbered 1001-1100.
6	Allocate DEBUG–ITEM and the subprogram name hash table during the compilation of the main program. Useful only on the main program and/or the called program having the largest identifier and using the DEBUG module.
7	Frequency table generation. Also requires added syntax. (See 17.3.)
8	Allocate the compiler temporary area (CTA) of this program or subprogram during compilation rather than at execution time. Use this option only when required by size constraints; do not use it in a self-initializing or reentrant TIP transaction. The 4 extra option overrides this option.
9	Print a cross-reference listing with only the referenced data-names listed. The L, N, and R standard options, and the 5 extra option override this option.
A	Use standard OS 2200 allocation rules for RECORDING MODE F, U, and V (EBCDIC) records.
B	Allocate COMPUTATIONAL–3 fields internally as DISPLAY fields outside RECORDING MODE F, U, and V records.
C	Use EBCDIC allocation and alignment rules outside the FILE SECTION for DISPLAY, COMPUTATIONAL, COMPUTATIONAL–1, and COMPUTATIONAL–2 fields.
D	Use octal 0377 for HIGH–VALUES (instead of octal 0177).
E	Process American National Standard 1968 COPY statements and library text only. American National Standard 1974 COPY statements and library text cannot be processed with this option.
F	Reserved for future use.

Table 18-2. @ACOB Control Statement Extra Options

Extra Option	Description
G	Generate the parameter list for the CALL statement in I-bank. When the parameters are generated in the I-bank, the called program cannot be banked.
H	Diagnose violations of the High (or High-Intermediate if the I extra option is set) level of Federal Standard COBOL syntax. This option will also set the M and V extra options. To avoid flagging the USAGE COMPUTATIONAL clause as a Unisys defined item, the N extra option must be set.
I	Diagnose violations of the High-Intermediate (if the H extra option is set or Low-Intermediate (if the L extra option is set) level of Federal Standard COBOL syntax. This extra option requires that either the H or the L extra option be set.
J	Invoke ASCII COBOL LOAD and GO feature, it will print program storage map with additional extra option W specified. This option also turns off K extra option (INFO–010 tables) if the K extra option is specified.
K	Generate PADS symbolic debugging tables (see 11.5).
L	Diagnose violations of the Low (or Low-Intermediate if the I extra option is set) level of Federal Standard COBOL syntax (see 18.4.). This option will also set the M and V extra options. To avoid flagging the USAGE COMPUTATIONAL clause as a Unisys defined item, the N extra option must be set.
M	The lowercase characters a, b, . . ., z are not "folded" to the uppercase characters A, B, . . ., Z, respectively.
N	COMPUTATIONAL and COMPUTATIONAL–4 usages are treated as implicit DISPLAY and DISPLAY–1 usages, respectively.
O	Process American National Standard 1968 CODE clause, which implies fixed–length images. American National Standard 1974 CODE clause, which implies variable-length images, will not be processed.
P	Reserved for future use.
Q	Reserved for future use.
R	Diagnose violation of American National Standard COBOL, X3.23–1974. This option will also set the M and V extra options. To avoid flagging the USAGE COMPUTATIONAL clause as a Unisys defined item, the N extra option must be set.
S	In an attempt to reduce a program's I-bank, the ASCII COBOL compiler will attempt to generate calls to run-time library routines instead of generating in-line code.
T	All COMPUTE statements and numeric expressions will be done in double-precision floating-point mode.
U	Ignore the WITH DEBUGGING MODE clause. Stop the generation of debugging lines. This option does not affect the USE FOR DEBUGGING statement.

Table 18-2. @ACOB Control Statement Extra Options

Extra Option	Description
V	Accept the quotation mark as the delimiter for nonnumeric literals instead of the apostrophe.
W	Use with extra option J to invoke ASCII COBOL LOAD and GO feature and print program storage map.
X	Perform subscript range check for every table reference in the program. If the data item is not the object of a USE FOR TABLE ERROR procedure and the subscript of the data item is out of range, the name of the data item and its source line will be displayed and execution is terminated.

18.2. XQT Options

The @XQT statement initiates the execution of a program that is in absolute element format. (See *Series 2200 Executive System, EXEC Programmer Reference* (7830 9945))

The option letters that are unique to ASCII COBOL are documented here.

18.2.1. General Format

```
@XQT,options  file-name.element-name
```

18.2.2. General Rule

See Table 18-3 for a description of the allowable options.

Table 18-3. @XQT Options

Option	Description
B	The presence of the B option causes the ACCEPT and DISPLAY statements to operate as they do in batch mode. (See 2.6.4 and 2.6.6.)
D	If present, the effects of USE FOR DEBUGGING are inhibited. See 11.1.3.3. (This option has no effect on debugging lines.)

18.3. Auto-Flagging Compiler

The symbolic release tape contains two versions of the ASCII COBOL compiler. One version is the normal ASCII COBOL compiler as released prior to level 5R2CY. The other compiler is the flagging compiler. The normal compiler has full capabilities of all parts of ASCII COBOL and all options work the same as before. The flagging compiler diagnoses all non-standard uses of COBOL by DEFAULT, and can be turned off by option. In addition, some options and features of ASCII COBOL have been disabled or modified.

The flagging compiler is intended for sites that want to use the basic COBOL environment or write programs that comply with the American National Standards Institute COBOL. Therefore, use careful judgment when you install the flagging compiler if your site previously used the normal compiler.

The following EXTRA OPTIONS have been DISABLED:

C EBCDIC allocation and alignment rules outside the FILE SECTION are not used.

E Procedures in the 1968 American National Standard standard format may not be used to satisfy a COPY.

T Forcing numeric expressions and COMPUTE statements to use floating point for intermediate results is not possible.

The flagging compiler has the following STANDARD OPTION set:

M ALL MONITOR statements are ignored.

The flagging compiler has the following EXTRA OPTIONS set:

D HIGH–VALUES is represented internally by the octal value 0377 instead of 0177.

M Lowercase letters are NOT folded to uppercase letters.

N Data items with USAGE COMP or COMP–4 are treated as though DISP or DISP–1 were specified. (See the following special note on effects of using this option with the flagging compiler).

O The source input file is searched before COB$PF when looking for a procedure to satisfy a COPY without the optional library name specified.

R All violations of American National Standard COBOL X3.23.1974 are diagnosed. (See the following special note on effects of using this option with the flagging compiler).

V All nonnumeric literals must be delimited by quotation marks rather than apostrophes. (See the following special note on effects of using this option with the flagging compiler).

When used with the flagging compiler, the following EXTRA OPTIONS are MODIFIED:

N Using the N extra option with the flagging compiler causes the NORMAL meaning to the USAGEs COMP and COMP–4. (Reverse of normal).

R Using the R extra option with the flagging compiler disables flagging, requires apostrophes for nonnumeric literals, and folds lowercase letters to uppercase letters except in nonnumeric literals. (Unsets the R, V, and M extra options. Reverse of normal).

V Using the V extra option with the flagging compiler requires that apostrophes be used to delimit nonnumeric literals. (Reverse of normal).

In addition to the options changes, the segmentation feature has been disabled and all compilations produce one relocatable element (two relocatables for subprograms). In other words, all segmentation directives to the compiler are ignored.

A special note about using the R and V EXTRA OPTIONS together:

Using the R and V EXTRA OPTIONS together with the flagging compiler disables the flagging and folds lowercase letters to uppercase letters. However, nonnumeric literals must be delimited by quotation marks. (In other words, unsets the EXTRA OPTIONS R and M, but not V).

Using the N and R EXTRA OPTIONS with the flagging compiler makes the flagging compiler act as though it is a normal compiler, with the exception of the feature and options that have been disabled, using 0377 instead of 0177 for HIGH–VALUES, and the searching of the source input file before COB$PF to satisfy a COPY without the optional library name specified.

In addition, the flagging compiler always issues diagnostic 85 when any 1985 American National Standard Institute COBOL reserve word is used as a data name, paragraph name, or section name. The new reserve words are:

ALPHABETIC–LOWER
ALPHABETIC–UPPER
ALPHANUMERIC
ALPHANUMERIC–EDITED
BINARY
BINARY–1
CLASS
COMMON
CONTENT
CONTINUE
CONVERSION
CONVERTING
DAY–OF–WEEK
DISP–2
DISPLAY–2
END–ADD
END–CALL
END–COMPUTE
END–DELETE
END–DIVIDE
END–EVALUATE
END–IF
END–MULTIPLY
END–PERFORM
END–READ
END–RECEIVE
END–RETURN
END–REWRITE
END–SEARCH
END–START
END–STRING
END–SUBTRACT
END–UNSTRING
END–WRITE
EVALUATE
EXTERNAL
FALSE
GLOBAL
INITIALIZE
MODE–1
MODE–2
MODE–3
NUMERIC–EDITED
ORDER
OTHER
PACKED–DECIMAL
PADDING
PURGE
REFERENCE
REPLACE
STANDARD–2
TEST
THEN
TRUE

18.4. FIPS Flagger

The ASCII COBOL compiler can monitor user source programs, at compile time, for each of the four levels of Federal Standard COBOL and American National Standard COBOL X3.23–1974.

American National Standard COBOL and the four levels of Federal Standard COBOL are defined in Table 18-4. The numbers within the table refer to the level within the Functional Processing Module (FPM) or Nucleus, and a dash (–) indicates that the corresponding FPM is omitted in that level. Unisys defined items (as denoted in this manual by broken line boxing) and items not in Standard COBOL will also be monitored. Level 2 components of the Nucleus or FPM are enclosed in solid line boxing in this manual. All nonboxed components in this manual are considered level 1 unless specifically stated otherwise.

Table 18-4. Standard COBOL Levels

	Federal Standard COBOL				American National Standard COBOL
	Low Level	Low Intermediate Level	High Intermediate Level	High Level	High Level
Nucleus (NUC) FPMs	1	1	2	2	2
Table Handling (TBL)	1	1	2	2	2
Sequential I–O (SEQ)	1	1	2	2	2
Relative I–O (REL)	–	1	2	2	2
Indexed I–O (INX)	–	–	–	2	2
Sort–Merge (SRT)	–	–	1	2	2
Report Writer (RPW)	–	–	–	–	1
Segmentation (SEG)	–	1	1	2	2
Library (LIB)	–	1	1	2	2
Debug (DEB)	–	1	2	2	2
Interprogram Communications (IPC)	–	1	2	2	2
Communication (COM)	–	–	2	2	2

The ASCII COBOL user may choose the level at which the source program is monitored by specifying one of the following extra options on the @ACOB statement (see Table 18-5).

Table 18-5. Flagging Options

Option Letters	Standard COBOL Level
L	Low Federal Standard COBOL
L and I	Low Intermediate Federal Standard COBOL
H and I	High Intermediate Federal Standard COBOL
H	High Federal Standard COBOL
R	High American National Standard COBOL
None or any invalid sequence of H, I, L, or R	None (No leveling diagnostics are issued.)

Any syntax used in the source program that does not conform to that allowed by the selected level of Standard COBOL will be diagnosed.

The diagnostic will either point to the column number of the offending syntax or give the line number of the statement where the offending syntax occurred. The diagnostic will specify either the level at which the offending syntax is valid or that the syntax is not part of Standard COBOL.

The diagnostics issued are:

73 L LEVEL VIOLATION, USE LOW INTERMED FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW level but the offending syntax is available only in the LOW–INTERMEDIATE level or above.

74 L LEVEL VIOLATION, USE HIGH INTERMED FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW or LOW–INTERMEDIATE level but the offending syntax is available only in the HIGH–INTERMEDIATE level or above.

75 L LEVEL VIOLATION, USE HIGH FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW, LOW–INTERMEDIATE or HIGH–INTERMEDIATE level, but the offending syntax is available only in the HIGH level.

76 L LEVEL VIOLATION, NOT FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked but the offending syntax is available only as a OS 2200 ASCII COBOL extension, or an American National Standard 1974 feature not included in any of the FIPS PUB 21–1 levels of Federal Standard COBOL.

80 L NOT 1974 AMERICAN NATIONAL STANDARD COBOL

All syntax that violates the American National Standard COBOL, X3.23–1974 will be diagnosed.

MOD is the American National Standard 1974 module abbreviation (see Table 18-4), and N is the American National Standard 1974 level of the given syntax.

All leveling diagnostics have severity L.

Level Chosen	Potential Leveling Diagnostics
L	73–76
L and I	74–76
H and I	75–76
H	76
R	80
None, or any invalid sequence of H, I, L, or R	None

18.5. COPY Procedures

18.5.1. Library Preparation

To prepare an ASCII COBOL source library, source images are processed by the Procedure Definition Processor (@PDP). The PDP output produces an ASCII COBOL library, which can be accessed by COBOL programs via the COPY statement.

Several procedures (text items) may be grouped together in one element, but each separate text-name must appear in the label field of a PROC statement.

The format for a text item input to PDP is:

```
text-name PROC

    (COBOL statements to be copied)

 END
```

The parameter *text-name* may consist only of the characters "A" through "Z", "0" through "9", and "-" except that the "-" may not appear as the first or last character. The parameter *text-name* must begin in column 1 of the source image and END must begin in column 2. Library text must conform to the rules for ASCII COBOL reference format.

18.5.2. Library Search Algorithm

The various COPY statements in the source program can reference text from more than one library. Only the library-name specified is searched to satisfy the COPY request.

When the library-name is not specified, the text can be found in any of the following libraries:

SI

Source Input File.

COB$PF

A user-defined file with the attached name COB$PF that the compiler references to satisfy COPY requests. If this file is not assigned, the compiler attempts to dynamically assign it. If COB$PF is dynamically assigned, it will not be freed at the end of compilation. If a file with the attached name COB$PF is not present, the search of this file is skipped.

SYSLIB*PROC$

A file created and maintained by COMUS and SOLAR to contain system procedures that the compiler references to satisfy COPY requests. If this file is not assigned, the compiler dynamically assigns it. If it is dynamically assigned, it will not be freed at the end of compilation. If SYSLIB*PROC$ is not present, the search of this file is skipped.

SYS$*RLIB$

A file built during a system generation run.

Table 18-6 specifies the library search order when attempting to satisfy COPY requests.

Table 18-6. Library Search Order

Zero Extra Option NOT Specified	Zero Extra Option Specified
COB$PF (If present)	SOURCE INPUT
SOURCE INPUT	COB$PF (If present)
SYSLIB*PROC$ (If present)	SYSLIB*PROC$ (If present)
SYS$*RLIB$	SYS$*RLIB$

18.6. Sample COBOL Compilation

Following is an ASCII COBOL compilation with the available listing options numbered:

```
@ACOB,DERS EXAMPLE,RB,,HI                (1)

ACOB 7R3FQ1  R SL76R3 09/03/02 14:04:57 (0)        2200 ASCII COBOL 05/31/02 08:57
     (2)             (3)          (4)                                  (5)
```

The listing options are:

(1) Control Statement

(2) Version Number of Compiler

(3) Date of Compilation

(4) Time of Day

(5) Date of Compiler Creation

18.6.1. Source Listing

```
                                1100 ASCII COBOL SOURCE LISTING    (1)

 1              IDENTIFICATION DIVISION.                           (2)
 2              PROGRAM-ID.  EXAMPLE-PROGRAM.
 3              AUTHOR.  JOE PROGRAMMER.
 4              DATE-COMPILED.   SEP 03, 2002.
 5             ************************************************************
 6             * THIS PROGRAM READS IMAGES FROM THE CARD-READER AND    *
 7             * PRINTS THEM ON THE PRINTER.                           *
 8             ************************************************************
 9              ENVIRONMENT DIVISION.
10              CONFIGURATION SECTION.
11              SOURCE-COMPUTER.  UNISYS-1100.
12              OBJECT-COMPUTER.  UNISYS-1100.
13              SPECIAL-NAMES.
14                  PRINTER IS PRT.
15              INPUT-OUTPUT SECTION.
16              FILE-CONTROL.
17                  SELECT CARD-FILE ASSIGN TO CARD-READER.
18                  SELECT PRINT-FILE ASSIGN TO PRINTER.
19              I-O-CONTROL.
20                  SAME RECORD AREA FOR CARD-FILE, PRINT-FILE.
21              DATA DIVISION.
22              FILE SECTION.
23              FD  CARD-FILE
24                  LABEL RECORD IS OMITTED.
25              01  IN-REC.
26                  02  NAME        PIC X(10).
27                  02  ADDRESS     PIC X(10).
28                  02  CITY        PIC X(10).
29                  02  STATE       PIC X(10).
30                  02  ZIP         PIC X(10).
31                  02  FILLER      PIC X(10).
32              FD  PRINT-FILE
```

```
      33                 LABEL RECORD IS OMITTED
      34                 LINAGE IS 88 LINES.
**  335 **    SEV. R      LINE    32         LINAGE OR DENSITY CLAUSE        (3)
                                                MAY CAUSE FORMS MISALIGNMENT
      35             01  PRT-REC.
      36                 02  P-NAME      PIC X(10).
      37                 02  FILLER      PIC X(15).
      38                 02  P-ADDRESS   PIC X(10).
      39                 02  FILLER      PIC X(15).
      40                 02  P-CITY      PIC X(10).
      41                 02  FILLER      PIC X(15).
      42                 02  P-STATE     PIC X(10).
      43                 02  FILLER      PIC X(15).
      44                 02  P-ZIP       PIC X(10).
      45                 02  FILLER      PIC X(22).
      46             WORKING-STORAGE SECTION.
      47             01  T-VALUE PIC 9(5) USAGE COMP.
                                           ^
**   76 **    SEV. L      COL. 35            LEVEL VIOLATION, NOT FED STD COBOL.
      48             01  P-VALUE PIC 9(5) USAGE COMP.
                                           ^
**   76 **    SEV. L      COL. 35            LEVEL VIOLATION, NOT FED STD COBOL.
      49             01  R-VALUE PIC 9(5) USAGE COMP.
                                           ^
**   76 **    SEV. L      COL. 35            LEVEL VIOLATION, NOT FED STD COBOL.
      50             PROCEDURE DIVISION.
      51             DECLARATIVES.
      52             PRINT-FILE-ERROR SECTION.
      53                 USE AFTER STANDARD ERROR PROCEDURE ON PRINT-FILE.
      54             DISPLAY-ERROR-MESSAGE.
      55                 DISPLAY "ERROR ON PRINT FILE" UPON PRT.
      56             END DECLARATIVES.
      57             INIT SECTION.
      58             OPEN-FILES.
      59                 OPEN INPUT CARD-FILE.
      60                 OPEN OUTPUT PRINT-FILE.
      61                 MOVE SPACES TO PRT-REC.
      62             READ-LOOP.
      63                 READ CARD-FILE AT END GO TO ALL-DONE.
      64                 MOVE NAME      TO P-NAME.
      65                 MOVE ADDRESS   TO P-ADDRESS.
      66                 MOVE CITY      TO P-CITY.
      67                 MOVE STATE     TO P-STATE.
      68                 MOVE ZIP       TO P-ZIP.
      69                 WRITE PRT-REC AFTER ADVANCING2 LINES.
                                              ^
**  774 **    SEV. S      COL. 32          UNDEFINED DATA ITEM; STATEMENT OMITTED
                                                                    ^
**  602 **    SEV. R      COL. 48          ATTEMPT RECOVERY HERE AFTER PREVIOUS ERROR
      70                 GO TO READ-LOOP.
                         ^
**  724 **    SEV. S      COL. 12          PREVIOUS ERRORS CAUSE LOSS OF OBJECT CODE
      71             ALL-DONE.
      72                 CLOSE CARD-FILE CLOSE PRINT-FILE.
      73                 STOP RUN.
```

(1) Source listing title

(2) Source line number

(3) Compiler diagnostics

(4) Pointer to entry that caused

18.6.2. Allocation Listing (D Option)

This is an example of D option usage. See 18.6.4 for descriptions of items (1) through (10).

```
                                        1100 ASCII COBOL ALLOCATIONS LISTING

(2)                              (3)    (4)  (5)  (6) (7)  (8)     (9)     (10)
NAME                            DEFN  TYPE USAGE DIM SIZE SB    ADDRESS  FILE NAME

TALLY                                 SPEC  UBIN 0     36  0         02 TEMPORARY
MCFLAG                                SPEC  UBIN 0     36  0         03 TEMPORARY
CARD-FILE                       23    FILE                            CARD-FILE
NAME                            26    DATA ALPH9 0     10  0          0 CARD-FILE
ADDRESS                         27    DATA ALPH9 0     10 18         02 CARD-FILE
CITY                            28    DATA ALPH9 0     10  0         05 CARD-FILE
STATE                           29    DATA ALPH9 0     10 18         07 CARD-FILE
ZIP                             30    DATA ALPH9 0     10  0        012 CARD-FILE
FILLER                          31    DATA ALPH9 0     10 18        014 CARD-FILE
IN-REC                          25    DATA ALPH9 0     60  0          0 CARD-FILE
PRINT-FILE                      32    FILE                            PRINT-FILE
P-NAME                          36    DATA ALPH9 0     10  0          0 PRINT-FILE
FILLER                          37    DATA ALPH9 0     15 18         02 PRINT-FILE
P-ADDRESS                       38    DATA ALPH9 0     10  9         06 PRINT-FILE
FILLER                          39    DATA ALPH9 0     15 27        010 PRINT-FILE
P-CITY                          40    DATA ALPH9 0     10 18        014 PRINT-FILE
FILLER                          41    DATA ALPH9 0     15  0        017 PRINT-FILE
P-STATE                         42    DATA ALPH9 0     10 27        022 PRINT-FILE
FILLER                          43    DATA ALPH9 0     15  9        025 PRINT-FILE
P-ZIP                           44    DATA ALPH9 0     10  0        031 PRINT-FILE
FILLER                          45    DATA ALPH9 0     22 18        033 PRINT-FILE
PRT-REC                         35    DATA ALPH9 0    132  0          0 PRINT-FILE
T-VALUE                         47    DATA  UBIN 0     18  0         01 WORKING-ST
P-VALUE                         48    DATA  UBIN 0     18  0         02 WORKING-ST
R-VALUE                         49    DATA  UBIN 0     18  0         03 WORKING-ST
LINAGE-COUNTER                  32    DATA  UBIN 0     18 18        041 PRINT-FILE
PRINT-FILE-ERROR                52    SECT                            023 PROCEDURE
DISPLAY-ERROR-MESSAGE           54    PARA                            025 PROCEDURE
INIT                            57    SECT                            032 PROCEDURE
OPEN-FILES                      58    PARA                            032 PROCEDURE
READ-LOOP                       62    PARA                            047 PROCEDURE
ALL-DONE                        71    PARA                           0123 PROCEDURE

                TOTAL NUMBER OF USER NAMES =       32
```

18.6.3. Cross Reference Listing (R option)

This is an example of R option usage. For an explanation of notes 1–12, see 18.6.4.

```
(1)                                    1100 ASCII COBOL CROSS REFERENCE LISTING

(2)                          (3)    (4)  (5)  (6) (7)  (8)     (9)      (10)
NAME                        DEFN  TYPE USAGE DIM SIZE SB    ADDRESS  FILE NAME  REFS

ADDRESS                       27  DATA ALPH9 0     10 18         02 CARD-FILE    65
ALL-DONE                      71  PARA                        0123 PROCEDURE    63
CARD-FILE                     23  FILE                             CARD-FILE    59   63  72
CITY                          28  DATA ALPH9 0     10  0         05 CARD-FILE    66
DISPLAY-ERROR-MESSAGE         54  PARA                         025 PROCEDURE
IN-REC                        25  DATA ALPH9 0     60  0          0 CARD-FILE
INIT                          57  SECT                         032 PROCEDURE
LINAGE-COUNTER                32  DATA  UBIN 0     18 18        041 PRINT-FILE
MCFLAG                            SPEC  UBIN 0     36  0         03 TEMPORARY
NAME                          26  DATA ALPH9 0     10  0          0 CARD-FILE    64
OPEN-FILES                    58  PARA                         032 PROCEDURE
P-ADDRESS                     38  DATA ALPH9 0     10  9         06 PRINT-FILE   65*
P-CITY                        40  DATA ALPH9 0     10 18        014 PRINT-FILE   66*
P-NAME                        36  DATA ALPH9 0     10  0          0 PRINT-FILE   64*
P-STATE                       42  DATA ALPH9 0     10 27        022 PRINT-FILE   67*
P-VALUE                       48  DATA  UBIN 0     18  0         02 WORKING-ST
P-ZIP                         44  DATA ALPH9 0     10  0        031 PRINT-FILE   68*
PRINT-FILE                    32  FILE                             PRINT-FILE   60   72
PRINT-FILE-ERROR              52  SECT                         023 PROCEDURE
PRT-REC                       35  DATA ALPH9 0    132  0          0 PRINT-FILE   61*  69
R-VALUE                       49  DATA  UBIN 0     18  0         03 WORKING-ST
READ-LOOP                     62  PARA                         047 PROCEDURE    70
STATE                         29  DATA ALPH9 0     10 18         07 CARD-FILE    67
T-VALUE                       47  DATA  UBIN 0     18  0         01 WORKING-ST
TALLY                             SPEC  UBIN 0     36  0         02 TEMPORARY
ZIP                           30  DATA ALPH9 0     10  0        012 CARD-FILE    68

               TOTAL NUMBER OF USER NAMES =       26

END ACOB DIAGNOSTIC TOTALS      0 WARNING    0 MINOR      2 SERIOUS    0 FATAL      3 LEVELING
COMPILE TIME IS    3.72 SECONDS   CORE: MIN=  65150 / 65536  MAX=  67584 WORDS
```

18.6.4. Definition of Cross Reference and Allocation Listings

The following describes the usage of the cross reference and allocation listings.

(1) Title of page

(2) NAME

List of user names which have been defined within the program. Use of the R option is for cross reference in alphabetic order; the D option is for allocation listings only.

(3) DEFN

The internal line number at which the data item is defined.

(4) TYPE

Type of item; in other words, data, section, file, etc.

(5) USAGE

Type of usage; in other words, alphanumeric, numeric, or binary.

(6) DIM

Dimension. A zero in the dimension field indicates nondimensioned; a number greater than zero indicates the number of dimensions.

(7) SIZE

Size in bytes for character data; size in bits for binary data.

(8) SB

Starting bit. (The convention is left to right with the zero bit being the leftmost bit.)

(9) ADDRESS

Relative address within the area defined under FILE NAME.

(10) FILE NAME

The file name, working storage area, etc., described in the program.

(11) REFS**

All source lines containing reference to this location, excluding the originally defined location DEFN.

(12) Starred references mean that the contents of data–names are changed by this reference. An example would be the object of a MOVE.

Note: *If a COPY statement is used, there will be two sets of line numbers listed on the left side of the source listing. In this case, DEFN refers to the internal line number listed on the leftmost side of the source listing.*

18.7. Interprogram Communication Considerations

18.7.1. CALL Identifier

Usage of the CALL identifier format of the CALL statement requires some extra work at program collection time. The addresses of all the potential called programs, to be referenced by identifier, must be made available to the called program interface routine. This is accomplished by including a relocatable element (C$S333) in the collection. This table element contains all necessary information concerning potential called programs referenced by the identifier. If identifier references a called program name not found in the C$S333 element, the calling program terminates in error.

The C$S333 is a @MASM symbolic element which is found in the @ACOB run-time library file. The user must reassemble this element with MASM adding the list of subroutines to be referenced by CALL identifier to the source. The MASM-created relocatable C$S333 is collected with the program.

The subroutines can be collected in common banks, program banks, or segments.

Each subroutine referenced by using the CALL identifier format of the CALL statement must be specified in the source of C$S333 using the following format:

```
CREATE subroutine entry-point-name,n
```

where *n* is zero (0), an unsigned positive integer or is not specified. The intent of the CREATE statement is to generate two external references for the subroutine entry-point-name, one for the I-bank *entry-point-name* and the other for the D-bank C$B *entry-point-name*. If *n* is specified as zero or is not specified, two external references are generated. If only an I-bank external reference is required, the D-bank external reference can be omitted by specifying *n* as an unsigned positive integer. Note that the D-bank external reference is required if the "MEMORY 3 MODULES" clause is used in the called program and that the absence of the D-bank external reference during collection can cause the Collector to issue the following error message:

```
*ERROR:  entry point C$Bentry-point-name referenced from element element-name is
         undefined name
```

The current maximum number of external references allowed in this element is 1024 as specified in the *OS 2200 Data Structures Programming Reference Manual*.

An example of a MASM of C$S333 and a collection using that relocatable follows:

```
@MASM,SE SYS$LIB$*ACOB. C$S333,RBFILE.C$S333
-40
         CREATE 'SUBPROGRAM1'
         CREATE 'SUB3'
         CREATE 'WRITER'
         CREATE 'SUN-BREAK'
 @EOF

@MAP   ,PROGRAM
IN   MAIN
IN   RBFILE.C$S333
LIB  SUBPGMFILE.
LIB  SYS$LIB$*ACOB.
END
```

Note: *C$S333 symbolics are found in the ASCII COBOL product file. In this example: SYSLIB*ACOB.*

An alternative to this C$S333 method is to place the identifier subprogram I-bank entry-point-names in a DEF list (Collector directive). For example:

```
@MAP   ,PROGRAM
IN   MAIN
LIB  SUBPGMFILE.
DEF  SUBPROGRAM1,SUB3,WRITER,SUN-BREAK
LIB  SYS$LIB$*ACOB.
```

If the "MEMORY 3 MODULES" clause was used to compile the called programs, another DEF defining the D-bank entry-points must be included. For example:

```
@MAP   ,PROGRAM
IN   MAIN
LIB  SUBPGMFILE.
DEF  SUBPROGRAMS,SUB3,WRITER,SUN-BREAK
DEF  C$BSUBPROGRAM1,C$BSUB3,C$BWRITER,C$BSUN-BREAK
LIB  SYS$LIB$*ACOB.
```

All called programs must be included in the MAP with a DEF of the I-bank entry-points and, if the MEMORY clause is used, a DEF of the D-bank entry-points. This means that the called programs cannot be in a program bank or common bank when using this method. The C$S333 method allows usage of banked called programs. When using the DEF list approach, if a particular called program is not found in the C$S333 list, the MAP DEF directives will be searched.

18.7.2. ASCII COBOL Programs Calling Non–COBOL Programs

18.7.2.1. Preserving the COBOL Environment

The non-COBOL program being called must preserve the COBOL program's environment. That is, the called program must save and restore those registers which COBOL programs assume to be nonvolatile (normally the volatile registers are A0 through A5 and R1 through R3), should not alter interrupt trap locations set and used by ASCII COBOL, must not get its own dynamic storage areas, must not change the processor state word, and must not initiate random processing cycles except through the ASCII COBOL asynchronous processing facility, etc.

18.7.2.2. ENTER MASM Calling Sequence

When ENTER MASM is specified, the calling sequence will be:

```
$(1).
        L,U     A0,parameters
        LXI,U   X11,BDICALL$ + subpgm
        IBJ$    X11,subpgm
$(8).
parameters
    +           C$Bsubpgm,n
    +           Word 1 of parameterû1
    +           Word 2 of parameterû1
    .
    .
    .
    +           Word 1 of parameterûn
    +           Word 2 of parameterûn
$(1).
```

where:

subpgm

is the I-bank entry-point.

C$B*subpgm*

is the reference to an entry-point in the called program's D-bank and is the name derived from the I-bank entry-point. This allows called programs to utilize based D-bank references and is the technique used to pass the called program's D-bank from the calling bank to the called program common bank.

C$B*subpgm* is derived from the entry-point. If the entry-point is nine or fewer characters in length, the entire entry-point is appended to C$B. If the entry-point is greater than nine characters, the first three and last six characters are appended to C$B.

n

is a value equal to twice the number of operands contained in the USING phrase of the ENTER statement. If the USING phrase of the ENTER statement is not specified, only the LXI,U and IBJ$ instructions are generated.

Two words will be generated for each operand contained in the USING phrase of the ENTER statement. For a description of the parameter formats, see 18.7.2.4. For a description of the number and types of parameters which may be passed to MASM, see 12.4.4.

18.7.2.3. ENTER ASM Calling Sequence

When ENTER ASM is specified, the calling sequence will be:

```
LMJ         X11,subpgm
+        2*n
+        Word 1 of parameter-1
+        Word 2 of parameter-1
           .
           .
           .
+        Word 1 of parameter-n
+        Word 2 of parameter-n
Next instruction
```

where:

subpgm

is the I-bank entry-point.

n

contains a value equal to the number of operands contained in the USING phrase of the ENTER statement. If the USING phrase of the ENTER statement is not specified, only the LMJ instruction is generated.

Two words will be generated for each operand contained in the USING phrase of the ENTER statement. For a description of the parameter formats, see 18.7.2.4. For a description of the number and types of parameters which may be passed to ASM, see 12.4.4.

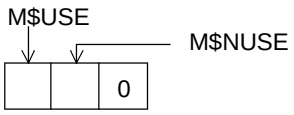
18.7.2.4. Parameter Format for MASM and ASM

Two words are generated for each operand contained in the USING phrase of the ENTER statement. These two words have the following format:

C	D			F	H	I
0	1	2	5	6 11	12 17	18 35
A	G		B J	K	E	L
0	1		4 5	6 11	12 17	18 35

Field	Location (in bits)	Size (bits)	Description of Use
C	0	1	If = 1 this item is array nonsynchronized (all occurrences do not have the same starting byte). If = 0 this item is array synchronized.
D	1	1	If = 1 this item is a constant.
	2–5	4	Unused.
F	6–11	6	The field containing the offset in bytes or bits depending on Field E. If MAJOR USE (M$USE) is 3 this field is expressed in bits. Otherwise, it is expressed in either 6 or 9 bit bytes depending on ASCII flag.
H	12–17	6	Contains the index register if one is used.
I	18–35	18	Contains the address used to reference the first word of the data item.
A	0	1	If = 1, this item is a procedure-name.
G	1	1	If = 1, field L contains a File Position Number for C$INFO call.
	2-3	2	Unused.
B	4	1	If = 1, this item is ASCII (ASCII flag). If = 0, this item is Fieldata.
J	5	1	If = 1, this item is double precision. If = 0, this item is single precision.
K	6–11	6	Scale that is expressed in a signed 6-bit binary field.
E	12–17	6	See the explanation following this listing.
L	18–35	18	Size of the item in character positions (bits if binary) or, the File Position Number for the file named in *argument-1* of the C$INFO call when field G is set.

E consists of three subfields of two bits each:



where M$USE is the major usage and M$NUSE is the minor usage:

M$NUSE \ M$USE	0 Alphanumeric	1 Numeric Edited	2 Numeric Unedited	3 Binary
0	Unedited	Lock	Unsigned	Unsigned
1	Unedited & Justified Right	Zero Suppress Editing	Signed	Signed
2	Edited	Special Editing	Unsigned Comp-3	Floating Point
3	Edited & Justified Right	General Editing	Signed Comp-3	Index Item

18.7.2.4.1. Evaluation of Subscripted Arguments

For either array-sync or non-array-sync table elements, the operand address can be determined by examining the F, H and I fields of the first word of the parameter. These fields are not maintained dynamically, however, the H field may reference an X-register that contains:

H1 = unnormalized bit offset

H2 = word offset

To locate the array element in question, multiply the F field value by either 9 to convert to bits unless MAJOR USE (M$USE) is 3 in which case it is bits already. Add this to the unnormalized bit offset (H1 of the X-register) and convert the result to words and bits by dividing by 36. Add the words just calculated to the sum of the I field value and the word offset (H2 of the X-register). This gives the array element word address while the remainder (bit count resulting from the division by 36) gives the start bit of the element within that word.

18.7.2.4.2. Evaluation of Arguments Containing an ODO Clause

When an item containing an ODO clause is used as an argument the maximum length of the item is passed in field L since this length is created at compile time and is not altered dynamically at run time.

18.7.2.5. ENTER PL1 Calling Sequence

When `ENTER PL1 "ROUTINENAME" USING` *dn1* is specified, it will cause the following code sequence to be generated:

```
$(1).
       L,U     A0,$(8)                    . parameter list address
       LXI,U   A0,1                       . number of parameters
       LMJ     X11,C$M706                 . PL1 prologue
       LXI,U   X11,BDICALL$+ROUTINENAME
       IBJ$    X11,ROUTINENAME
$(8)   +       dn1
$(1)   LMJ     X11,C$M707                 . PL1 epilogue
```

where *dn1* is an elementary or group item with one of the following COBOL usages:

(1) USAGE IS DISPLAY,

(2) USAGE IS COMPUTATIONAL (4 or 8 bytes in length starting on a word boundary),

(3) USAGE IS COMPUTATIONAL–1,

(4) USAGE IS COMPUTATIONAL–2,

(5) Exact Binary.

The corresponding allowable PL1 data types are as follows:

(1) CHARACTER (nonvarying),

(2) BINARY FIXED, DECIMAL FIXED,

(3) DECIMAL FLOAT (8),

(4) DECIMAL FLOAT (16),

(5) BIT.

If a COBOL main program passes an array or a structure as an argument to a PL/I subprogram, care must be taken in how the parameter is declared in the PL/I subprogram. PL/I requires aggregate descriptors whereas COBOL does not generate them. COBOL does create string descriptors for arguments passed to PL/I subprograms. However, for arrays and structures, the length used in the descriptor is the total length of the array or structure. Therefore, in the PL/I subprogram, the parameter must be declared as a character string with the array or structure defined in the parameter so that the elements of the array or the members of the structure can be referenced.

Example:

COBOL main program:

```
01  NUMBER-BUFFER USAGE DISPLAY.
    02  NUMBER-ARRAY OCCURS 5 PIC X(12).
        ENTER PL1 'ACOB' USING NUMBER-BUFFER.
```

PL/I subprogram:

```
DCL ARRAY CHARACTER (60);
DCL ARRAY1 (5) PIC '(12)X' DEFINED (ARRAY);
```

18.7.2.6. ENTER FTN Calling Sequence

When `ENTER FTN "ROUTINENAME" USING` *dn1* is specified, it will cause the following code sequence to be generated:

```
$(1).
      L,U    A0,$(8)                  . parameter list address
      LXI,U  A0,1                     . number of parameters
      LMJ    X11,C$M708               . FTN prologue
      LXI,U  X11,BDICALL$+ROUTINENAME
      IBJ$   X11,ROUTINENAME
$(8)  +      dn1
$(1)  LMJ    X11,C$M709               . FTN epilogue
```

where *dn1* is an elementary item with one of the following COBOL usages:

(1) USAGE IS DISPLAY,

(2) USAGE IS COMPUTATIONAL (4 bytes in length starting on a word boundary),

(3) USAGE IS COMPUTATIONAL–1,

(4) USAGE IS COMPUTATIONAL–2.

The parameter *dn1* may also be a COBOL paragraph-name.

The corresponding allowable FTN data types are as follows:

(1) CHARACTER,

(2) INTEGER,

(3) REAL,

(4) DOUBLE PRECISION.

When *dn1* is a group item, the corresponding FTN data type may be anything but CHARACTER.

18.7.2.7. Procedure Name Arguments

If procedure-name arguments are being passed to a non-COBOL called program, the called program has to be modified in the area referencing the procedure-name parameters. An X-register is loaded with the program BDI and the base I-bank address. The parameter contains a pointer to the X-register in S3 and the procedure-name offset in H2. When referencing the procedure-name, H2 of the X-register pointed to by S3 of the parameter is added to H2 of the parameter to form the address of the procedure-name. The BDI for the procedure-name is found in H1 of the X-register referenced by S3 of the parameter. An example of a CALL follows:

```
CALL "sub" USING D–N P1.
```

where "*sub*" is the non-COBOL subprogram, *D-N* is a data-name argument, and *P1* is a procedure-name argument. The code generated for this CALL is:

```
       L,U    Xa,$LCB(1)           . address 0 control counter 1
$(1).                              . this address gets reloaded to the
                                   . first I-bank address
       LXI,U  Xa,BDI$
       L,U    A0,PARAMS
       LXI,U  X11,BDICALL$+sub
       IBJ$   X11,sub
$(8).
PARAMS.
       +      C$Bsub,2
       +      0,address            . address of D–N
       +      Xa, offset           . offset of P1
$(1).
```

where X*a* is the X-register (usually X10) used for basing the procedure-name, *address* is the address of the data-name argument, and *offset* is the relative offset from the I-bank base address where the procedure-name argument is found.

18.7.3. Non-COBOL Programs Calling ASCII COBOL Programs

18.7.3.1. Considerations for Entering ASCII COBOL Programs

An ASCII COBOL program, when entered, must have the COBOL environment established either by being called from an ASCII COBOL program or via the mechanism described in 18.7.3.2. Upon entry to the program, a storage area called the Compiler Temporary Area (CTA) will be allocated by the run-time library. If the called program contains I–O statements, a File Directory and storage area called the Operations Control Block (OCB) will also be allocated by the run-time library. This allocation takes place upon the first entry to the called program per cycle and the area remains for the duration of the cycle. Hence, the first call on a called program has much more overhead than subsequent calls.

When the 4 Extra Option is present, the CTA, File Directory, and OCB areas are allocated upon entering a called program and deallocated upon exit. Hence, the storage area needed for these tables is in existence only for the time spent in the program. This option is useful for low-use called programs. Use of this option has the following restrictions: ALTER in a called program is in its initial state upon each entry to the called program. If the called program contains PROCESS or EXDEF file statements, it cannot be collected in an overlay segment. The called program cannot be the entry point to an HVTIP bank.

Files defined in the called program must be closed upon exit if the called program is in an overlay, and the segment is overlaid before the next call to the called program.

18.7.3.2. Establishing the ASCII COBOL Environment

Before and after execution of a run that contains ASCII COBOL programs, certain ASCII COBOL/System interface programs must be executed. The purpose of this code, in addition to providing a system interface, is to initialize the ASCII COBOL execution environment.

The appropriate code is generated for all ASCII COBOL programs which are not identifiable as ASCII COBOL called programs. (The presence of the V option on an @ACOB processor call statement or a USING or ENTRY clause on the PROCEDURE DIVISION header suppresses generation of this code.) The interface code is automatically executed for ASCII COBOL main programs specified as such to the Collector.

When ASCII COBOL is called by an ASM main program, registers A4, A5, R11, and R12 must be saved at the beginning of execution of the main program and restored before the interface prologue routine is called.

Once the interface prologue has been executed, the ASCII COBOL environment is established and it is the user's responsibility to preserve it. Preservation includes the saving and restoring of all registers, maintaining the processor state word, and the contents of system interrupt trap locations, etc.

Thus, when an ASCII COBOL called program is invoked and the main program is not an ASCII COBOL program, the user must ensure that this necessary initialization has been performed. The simplest solution is to always have an ASCII COBOL main program, or to insert the appropriate calls in an assembly language main program. Whichever way is selected, the initialization must be performed once and only once for a run unit which contains one or more collected ASCII COBOL programs.

If the assembly language main program method is used, the following sequence:

```
PP$    FORM     10,4,4,18
P      PROC
IBJ$*  NAME.
       PP$      IBJ$,P(1,1),0,P(1,2)
       END
$(1).
       L,U      A3,TAG
       LXI,U    X11,BDICALL$+C$S31A
       IBJ$     X11,C$S31A
$(8).
TAG.
       +        0,C$S300
$(1).
       LXI,U    X11,BDICALL$+C$S410
       IBJ$     X11,C$S410        . not necessary for FTN or PL/I main program
```

must be executed as a run unit interface prologue and:

```
       LXI,U    X11,BDICALL$+C$S320
       IBJ$     X11,C$S320         . only necessary at program termination
```

must be executed as a run unit interface epilogue.

If users intend to invoke PADS during execution of their programs, they must preserve the values that were set in registers A4, A5, R1-R3, and R11-R15 when the programs were loaded by the operating system (see 11.5).

18.7.3.3. Calling Sequence

Assuming that the ASCII COBOL environment is created and maintained, ASCII COBOL programs may be called with the following calling sequence:

```
$(1).
          L,U    A0,address of parameters
          LXI,U  X11,BDICALL$+program-id or entry-point-name
          IBJ$   X11,program-id or entryu-point-name
$(8).
parameters.
          +      C$Bprogid, number of parameters
          +      address of parameter1
          +      address of parameter2
          .
          .
          .
          +      address of parametern
$(1).
```

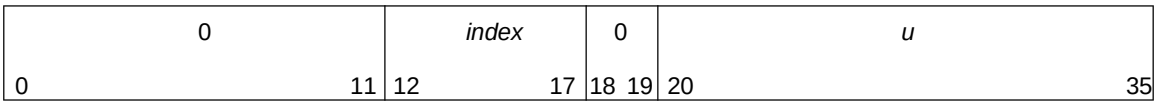
C$B*progid* is a reference to an entry-point in the called program's D-bank and is the name derived from the I-bank entry-point (*program-id* or *entry-point-name*). For ASCII COBOL called programs, the compiler will define this entry-point in the called program's D-bank. This allows called programs to utilize based D-bank references and is the technique used to pass the called program's D-bank from the calling bank to the called program common bank.

C$B*progid* is derived from the entry-point (*program-id* or *entry-point-name*). If the entry-point is nine or less characters in length, the entire entry–point is appended to C$B. If the entry-point is greater than nine characters, the first three and last six characters are appended to C$B.

The symbol *n* represents the number of parameters being passed. It must be greater than or equal to zero and less than 32.

18.7.3.4. Parameter Format

The parameter addresses are constructed in the following form:



where:

u

is either a relocatable address of the argument item or actual offset from a base index.

index

is a number from 0 through 26. If *index* is 0, *u* is the address of the argument. Otherwise, *index* indicates the register (1 = X1, 2 = X2, 3 = X3, 4 = X4, 5 = X5, ...,18 = A6, 19 = A7, 20 = A8, 21 = A9, 22 = A10, 23 = A11, 24 = A12, 25 = A13, 26 = A14) that contains the base address needed to compute the actual location of the argument as follows:

Actual location of argument = u + contents of the register indicated by index

Other bits in the word are zero.

18.8. Executive Request Call Statements

18.8.1. Call ER ACSF$

This feature provides a means of issuing an Executive Request (ER) to ACSF$ directly from a COBOL program. This is done via a CALL statement to the entry point ERACSF$. The ER packet is established and initialized within the COBOL program and passed by the USING clause in a CALL statement. Use of this feature assumes a knowledge of the ER being used as well as what is expected in the packet. The CALL has the form:

```
CALL "ERACSF$" USING data-name-1,data-name-2[,data-name-3]
```

where:

data-name-1

contains the ACSF$ image defined as PIC X up to 80 characters.

data-name-2

contains the returned status word defined as PIC S1(36).

data-name-3

contains the size in words of the ACSF$ image defined as PIC 9(10) COMP.

Data-name-3 must be specified if the ACSF$ image defined as *data-name-1*, is less than 80 characters, unless the ACSF$ image contained in *data-name-1* is terminated by the character sequence ' **.** ' .

For an explanation of the CSF$ Executive Request Status Codes, see the *OS 2200 Exec System Software Executive Requests Programming Reference Manual*.

18.8.2. Call ER SYMB$

This implementation allows for the initial language interface with the 777-printer. The interface is performed by issuing an ER to SYMB$ directly from a COBOL program. This is done via a CALL statement to the entry point ERSYMB$. The SYMB$ packet must be established and initialized within the COBOL program and passed by the USING clause in a CALL statement. Use of this feature requires a knowledge of the ER being used as well as what is expected in the packet. The CALL has the form:

```
CALL 'ERSYMB$' USING data-name-1,data-name-2[,data-name-3]
```

where:

data-name-1

is the SYMB$ packet as required by the 'SYMB$' ER. See the *OS 2200 Exec System Software Executive Requests Programming Reference Manual* for packet description.

data-name-2

is the area to be used as the image-address. This address will be placed in H2 of word 4 of SYMB$ packet.

data-name-3

is the area to be used as the image-address. This address will be placed in H2 or word 010 of the SYMB$ packet. For WR function only.

18.8.3. Call ER 'ERERR$'

This feature provides a means of issuing an ER to ERR$ directly from a COBOL program. If PADS was collected with the absolute, PADS is invoked instead of issuing an ER to ERR$. The CALL has the form:

```
CALL 'ERERR$'.
```

18.8.4. Call ER 'EREABT$'

This feature provides a means of issuing an ER to EABT$ directly from a COBOL program. If PADS was collected with the absolute, PADS is invoked instead of issuing an ER to EABT$. The CALL has the format:

```
CALL 'EREABT$'.
```

18.9. Current Date Service Routine (C$CURDATE)

The ASCII COBOL current date service routine (C$CURDATE) provides a method to obtain the current date with a 4-digit year in the format YYYYMMDD. A 4-digit year can also be obtained from the ACCEPT statement (see 2.6.4). The interface to this C$CURDATE service routine is:

```
CALL 'C$CURDATE' USING data-name-1
```

where *data-name-1* is described as a PIC 9(8) USAGE DISPLAY or PIC X(8) USAGE DISPLAY data item.

18.10. Messages to the Operator Console

18.10.1. ACCEPT . . . FROM CONSOLE

While running in the batch mode, an ACCEPT command causes a console message:

```
"ENTER DATA"-----
```

for every 50 characters required (see 2.6.4). Each message requires the addition of 50 characters for input.

18.10.2. Blank Tape

A blank tape, assigned without a specified reel number, which is required to be labelled, causes a console message ANSWER WITH REEL NO. ON *device*. Any 6-character reply is accepted.

18.10.3. DISPLAY . . . UPON CONSOLE

While running in the batch mode, a DISPLAY command causes a console message for every 50 characters displayed (see 2.6.8.)

18.10.4. Unlabeled Tape

An ER TLBL$ request on an output tape reel that is not 8–bit labeled causes the following operator's console message to be issued:

```
LOAD PROPERLY LABELLED REEL ////// file-name AE
```

The parameter ////// specifies the logical device name on which the prelabeled reel is to be mounted. The *file-name* is the external file name specified in the ASSIGN clause for the file.

The operator is given an opportunity to mount a properly labeled tape. The operator may then respond with an "A" or and "E". A response of "A" causes the ER TLBL$ request to be issued again. A response of "E" causes a diagnostic message 1161A to be issued and the cycle to abort.

18.10.5. VALUE OF CARTRIDGE–ID

If a file has specified a VALUE OF CARTRIDGE–ID clause, execution of either the OPEN statement for a nonreport file or the INITIATE statement for a report file may cause the following console messages:

```
sname xx IN,yy REQUESTED USE IN? YN
```

```
sname MOUNT CARTRIDGE yy A
```

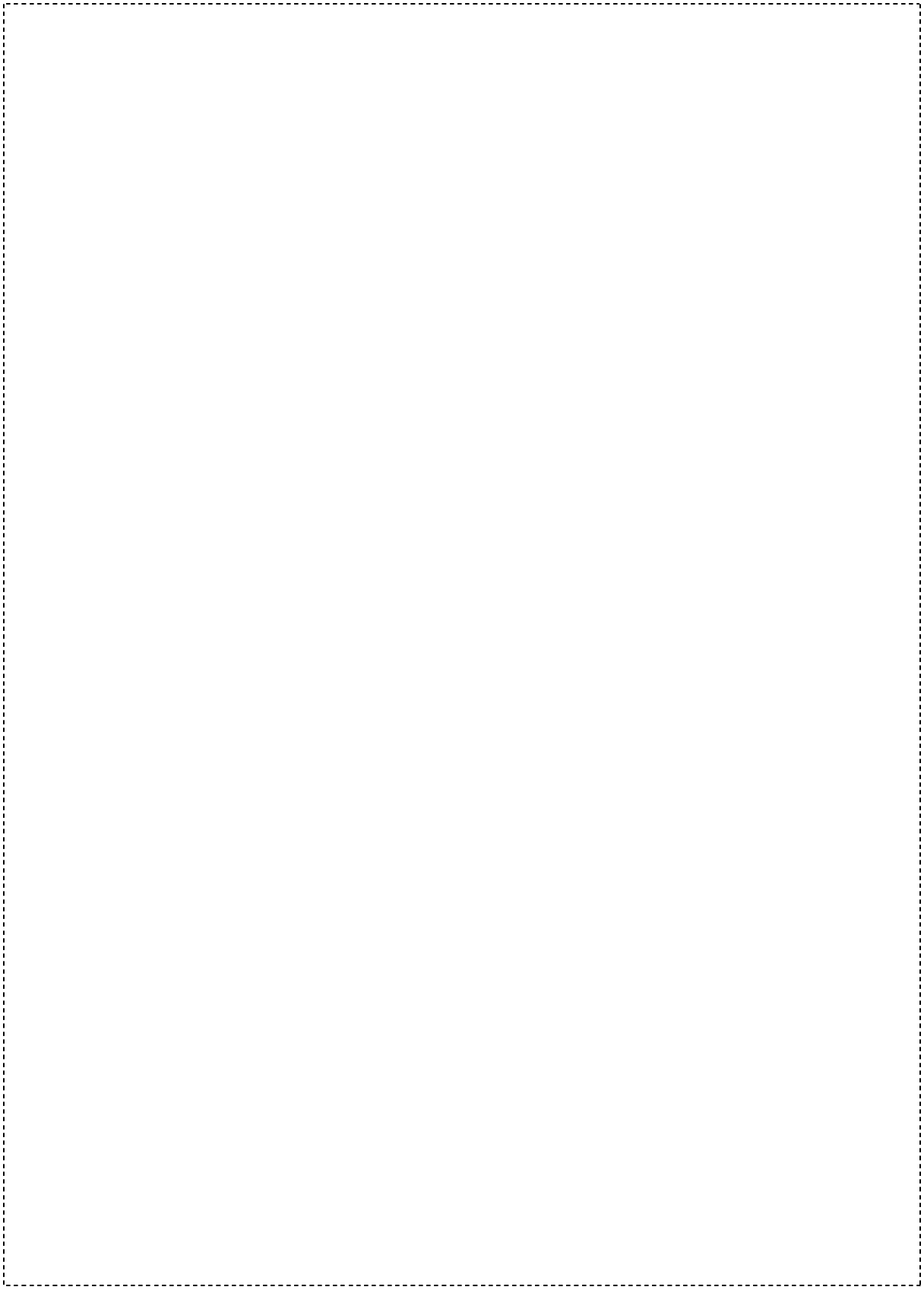
A complete description of these messages may be found in *OS 2200 Exec System Software Operations Reference Manual* (7831 0281).

18.10.6. VALUE OF FORMS–REQUEST

If a file has specified a VALUE OF FORMS–REQUEST clause, execution of either the OPEN statement for a nonreport file or the INITIATE statement for a report file causes the following operators console message:

```
X – SPECIAL FORMS REQ
```

where *x* is the 24-character alphanumeric field specified in the VALUE OF FORMS–REQUEST clause. The field may not contain any periods.



Section 19
Collection of Programs

19.1. General

This section discusses the ASCII COBOL compiler element naming conventions and ASCII COBOL program collection methods and restrictions. For a discussion on TIP collection considerations, see Section 21.

19.2. Element Naming Conventions

The ASCII COBOL compiler produces one relocatable element for nonsegmented main programs, two relocatable elements for subprograms, and several relocatable elements and a symbolic MAP element for segmented main programs. In all cases the compiler produces external symbols that are unique. They are constructed using the relocatable output (RO) field of the processor call statement.

The external symbols and the element names of segmented programs are constructed as follows. Truncate RO using the leftmost six characters, or if RO is less than six characters, "$" fill to the right for a total of six characters. The resulting six-character symbol [SYMB] is concatenated with six characters supplied by the compiler. [SYMB] must be unique to avoid duplicate name problems.

19.2.1. Main Programs

For main programs, one relocatable binary element is created with the name RO. In addition, four external symbols are defined. They are: [SYMB]C$MCBL, [SYMB]C$CTAL, [SYMB]C$MENT, and [SYMB]C$EPLG.

19.2.2. Segmented Programs

For segmented programs, the RO field is used as the name of the MAP symbolic. If the MAP symbolic name is the same as the source input (SI) or source output (SO), it is given the version name 'MAP'.

A relocatable element is produced for the main segment, for the data, and for each segment. (There may not be a one-for-one correspondence between the number of elements and the number of segments. The actual number depends on how the source program is arranged.) The relocatables are placed in the same file as the RO field and are named as follows:

```
[SYMB] MAIN
[SYMB] SYSDTA
[SYMB] xxxxxx
```

where one [SYMB] *xxxxxx* is created for each segment. The value of *xxxxxx* is unique and is determined by the compiler by numbering the procedure and section names in a program from 0 to *n* (where *n* is the last procedure in a program). The parameter *xxxxxx* is the *n* associated with the section name that defines the section.

The four external symbols defined for nonsegmented programs are defined for segmented programs as well. In addition, sets of the following are defined:

Entry-Point

[SYMB]P*xxxxx* in each element

Segment fall through

[SYMB]S*xxxxx* in prologue

Location counter
[SYMB]L*xxxxx*

where *xxxxx* is unique and determined by the compiler by numbering the procedure and section names in a program from 0 to *n* (where *n* is the last procedure in a program). The parameter *xxxxx* is the *n* associated with the section name that defines the section.

Segment names for the MAP are created as follows:

```
BAS000     for base segment
OVRyyy     for fixed overlayable segment
INDzzz     for independent segment
```

where *yyy* and *zzz* are the priority number.

19.2.3. Subprograms

For subprograms, two relocatable elements are created. One element contains exclusively I-bank and is given the name in the RO field. The other element contains exclusively D-bank and is given the name [SYMB] SYSDTA.

The I-bank element contains an entry–point for the PROGRAM–ID name plus one for each name on the WITH ENTRY POINTS clause. Also, the epilog entry-point is defined as [SYMB]C$EPLG.

The D-bank element contains an entry–point which is defined at relative address 0 of control counter 8, C$B[ENTRY–POINT]. [ENTRY–POINT] is derived from each PROCEDURE DIVISION entry-point in the called program. If the entry-point is less than or equal to nine characters, [ENTRY–POINT] will be the entry-point. If the entry-point is greater than nine characters, [ENTRY–POINT] is derived by concatenating the first three and last six characters of the entry-point. Also defined as entry-points are [SYMB]C$CTAL and [SYMB]L*nnnnn.* There is a [SYMB]L*nnnnn* for each location counter in the element where *nnnnn* is the value of the location counter.

19.3. Collection of ASCII COBOL Programs

This subsection discusses programming and collection restrictions, installation configuration considerations that affect program collection, and ASCII COBOL collection methods.

ASCII COBOL programs may be collected using the following methods:

- Bank-implied collection
- Bank-named collection
- Segmented collection

All discussions and examples in this subsection assume that ASCII COBOL was installed using COMUS or SOLAR. The mode in which ASCII COBOL was installed may impact the collection methods to be used. Further information on collection restrictions relating to the configuration of ASCII COBOL can be found in 19.3.1.2.

19.3.1. Restrictions

19.3.1.1. General Restrictions

The following are general restrictions which must be observed regardless of the method used to collect an ASCII COBOL program or the configuration of ASCII COBOL on the system.

- I-banks must not exceed 65K (in other words, the address octal 177777).
- Only one D-bank is allowed for all ASCII COBOL programs and subprograms.
- When the D-bank exceeds 65K (in other words, the address octal 177777), the MEMORY 3 MODULES clause must be specified in the OBJECT–COMPUTER paragraph on ASCII COBOL programs.

19.3.1.2. Installation Configuration Considerations

The ASCII COBOL run-time library can be installed and used in the following methods:

- Relocatable
- Partial Common Banked (in other words, excluding C$DML)
- Full Common Banked (in other words, including C$DML)

The form of the library that is used in the collection of a particular program depends on which version of the element CBEP$$ACOB is included. The following table relates the element CBEP$$ACOB to the form of the library.

CBEP$$ACOB Element	Library
CBEP$$ACOB/NO–LIB–CB	Relocatable
CBEP$$ACOB/PART–LIB–CB	Partial Common Banked
CBEP$$ACOB/FULL–LIB–CB	Full Common Banked

No special collection considerations are required to use the relocatable or partial common banked libraries. To use the full common banked library (including the C$DML common bank), a collection method using C$DML must be followed (see 19.3.3 through 19.3.5).

19.3.2. Bank-Implied Collections

The bank-implied collection method is the simplest collection method and should be used unless the size of the program prohibits its use or the full common banked library is used. The following is an example of the Collector directives necessary for a banked implied collection of a main program and one subprogram:

```
IN MAIN
IN SUB
IN SUB$$$SYSDTA
```

19.3.3. Collections Using C$DML

The collection method described in this subsection must be followed if the full common banked library is used. This method should not be followed if the relocatable or partial common banked library is used. The Collector directives required for this collection method are basically the same whether the element CBEP$$ACOB/FULL–LIB–CB is implicitly included from the ASCII COBOL library (SYSLIB*ACOB) or explicitly included from the ASCII COBOL C$DML CBEP file (SYS$LIB$*ACOB–DML). The collection method varies depending on whether an ASCII or Fieldata version of the C$DML common bank is installed.

Generally, the ASCII version of the C$DML common bank is installed, but it is important to verify this fact. To determine which version of the C$DML common bank is installed enter:

```
@PRT,TR SYS$LIB$*ACOB.C$D202
```

where SYSLIB*ACOB is the ASCII COBOL product file.

If the relocatable is found, an ASCII version is installed; otherwise, a Fieldata version is installed.

The following example shows the Collector directives for a collection of a main program and one subprogram using C$DML:

```
IBANK,UX   C$DML
IBANK,M    I1,022000                      . See Note 1
  IN       MAIN
  IN       SUB
DBANK,MUC  D1
  IN       MAIN
  IN       SUB$$$SYSDTA
  IN       SYS$LIB$*ACOB-DML.CBEP$$ACOB/FULL-LIB-CB
END
```

Note: *The collection illustrated assumes that an ASCII C$DML is installed. If a Fieldata version is installed, 022000 must be changed to 037000 on the IBANK directive.*

19.3.4. Bank-Named Collection

This subsection describes the method required to collect an ASCII COBOL program that requires two or more user I-banks. If only one user I-bank is required, the collection method described in 19.3.2 or 19.3.3 should be used.

Collection of ASCII COBOL programs using multiple I-banks can be extremely complicated because the compiler generates LMJs and direct references to many run-time library I-bank elements. Since the compiler generates code in this manner, all run-time library elements that are referenced by an LMJ or a direct reference must be visible when the reference occurs. To ensure this visibility, the library elements must be collected in one of the following ways:

1. In the D-bank
2. In a separate bank that is always based
3. In each bank that references the library element

The third technique listed is the most complex and should only be used when the other techniques fail. The following are sample multibanked collections of a main program and three subprograms.

Not Using C$DML — Version 1:

```
LIB (D1/$ALL)
LIB SYS$LIB$*ACOB()
IBANK,M  I1,01000
    IN      MAIN
    IN      SUB2
IBANK    I2,01000
    IN      SUB1
IBANK    I3,01000
    IN      SUB3
DBANK,MC D1,(040000,I1,I2,I3)
    IN      MAIN
    IN      SUB1$$SYSDTA
    IN      SUB2$$SYSDTA
    IN      SUB3$$SYSDTA
END
```

Not Using C$DML — Version 2:

```
LIB (IL/$ODD, D1/$EVEN)
LIB SYS$LIB$*ACOB()
IBANK,UL IL,01000
    IN()    SYS$LIB$*ACOB.C$RET0
IBANK,M  I1,(IL)
    IN      MAIN
    IN      SUB2
    IN()    SYS$LIB$*ACOB.C$RET0
IBANK    I2,(IL)
    IN      SUB1
    IN()    SYS$LIB$*ACOB.C$RET0
IBANK    I3,(IL)
    IN      SUB3
    IN()    SYS$LIB$*ACOB.C$RET0
DBANK,MC D1,(040000,I1,I2,I3)
    IN      MAIN
    IN      SUB1$$SYSDTA
    IN      SUB2$$SYSDTA
    IN      SUB3$$SYSDTA
END
```

Using C$DML:

```
LIB (D1/$ALL)
LIB SYS$LIB$*ACOB()
IBANK,UX     C$DML
IBANK,M      I1,022000
    IN          MAIN
    IN          SUB2
IBANK        I2,022000
    IN          SUB1
IBANK        I3,022000
    IN          SUB3
DBANK,MC     D1,(040000,I1,I2,I3)
    IN          MAIN
    IN          SUB1$$SYSDTA
    IN          SUB2$$SYSDTA
    IN          SUB3$$SYSDTA
    IN          SYS$LIB$*ACOB-DML.CBEP$$ACOB/FULL-LIB-CB
END
```

The previous collection examples illustrate techniques 1 and 2. If collection errors are received, attempt to resolve the errors by either using the C$DML common bank or dividing an I-bank into multiple I-banks. If collection errors are not resolved using either or both of these suggestions, ensure that all ASCII COBOL programs have the MEMORY 3 MODULES clause specified. If collection errors still occur, the last technique must be used. This technique requires the examination of an S or L option collection listing. Every C$Dxxx and C$M7xx element that is included in the collection, as well as the element C$RET0 must be locally INed in each bank that references the element. All other ASCII COBOL library elements may be collected into a separate I-bank.

The following is a bank named collection not using the C$DML common bank. This collection shows the local inclusion of the library elements as required.

```
LIB (ILIB/$ODD, D1/$EVEN)
LIB SYS$LIB$*ACOB()
IBANK,M  I1,01000
   IN       MAIN
   IN       SUB2
   IN()     C$RET0
   IN()     C$D322
IBANK    I2,01000
   IN       SUB1
   IN()     C$RET0
IBANK    I3,01000
   IN       SUB3
   IN()     C$RET0
   IN()     C$D313,C$D326
IBANK    ILIB,01000
DBANK,MC D1
   IN       MAIN
   IN       SUB1$$SYSDTA
   IN       SUB2$$SYSDTA
   IN       SUB3$$SYSDTA
END
```

19.3.5. Segmented Collections

19.3.5.1. Compiler Generated Maps

Map elements produced by the ASCII COBOL compiler for programs using the segmentation module of the COBOL language should be used without modification. If the program is to be collected with the full common banked library or contain calls to subprograms, the map element must be modified accordingly.

19.3.5.2. Non-Compiler Segmented Collections

Whenever possible, multibanked collections should be used instead of segmentation. Additional collection restrictions apply when using segmentation. These restrictions are:

- Only the I-bank should be segmented. If the D-bank is segmented, the data-items in that segment contain their initial values when the subprogram is called for a second time and the segment was overlaid by another segment.
- If the indirect load segment method is not used, the programmer is responsible for loading the segments.
- The call to a subprogram must not cause a segment to be loaded which overlays any subprogram that has been called and has not returned control to the calling program.

The following examples show the Collector directives for a segmented collection using the indirect load method with a main program and four subprograms. In these examples, SUB1 and SUB3 can not directly or indirectly call each other because they are included in segments that overlay one another.

Not Using C$DML:

```
IBANK,M  I1,01000
SEG     MAIN
    IN  MAINPROG
    IN  SUB2
SEG     SEG1*,(MAIN)
    IN  SUB1
SEG     SEG2*,(MAIN)
    IN  SUB3
SEG     SEG3*,(SEG1,SEG2)
    IN  SUB4
DBANK,MC  D1
    IN  MAINPROG
    IN  SUB1$$SYSDTA
    IN  SUB2$$SYSDTA
    IN  SUB3$$SYSDTA
    IN  SUB4$$SYSDTA
END
```

Using C$DML:

```
IBANK,UX  C$DML
IBANK,M   I1,022000
SEG      MAIN
    IN   MAINPROG
    IN   SUB2
SEG      SEG1*,(MAIN)
    IN   SUB1
SEG      SEG2*,(MAIN)
    IN   SUB3
SEG      SEG3*,(SEG1,SEG2)
    IN   SUB4
DBANK,MC  D1
    IN   MAINPROG
    IN   SUB1$$SYSDTA
    IN   SUB2$$SYSDTA
    IN   SUB3$$SYSDTA
    IN   SUB4$$SYSDTA
    IN   SYS$LIB$*ACOB-DML.CBEP$$ACOB/FULL-LIB-CB
END
```

Large programs may require that the D-bank be segmented or the program split into two or more programs. If the program cannot be split, the D-bank may be segmented as long as location counter 6 of all ASCII COBOL programs is collected in the main segment. The following examples show the Collector directives for a segmented I-bank and segmented D-bank collection.

Not Using C$DML:

```
IBANK,M   I1,01000
  SEG     MAIN
     IN   MAINPROG
     IN   SUB2
  SEG     SEG1*,(MAIN)
     IN   SUB1
  SEG     SEG2*,(MAIN)
     IN   SUB3
  SEG     SEG3*,(SEG1,SEG2)
     IN   SUB4
  DBANK,MC  D1
  SEG     MAIN
     IN   MAINPROG
     IN   SUB2$$SYSDTA
  SEG     SEG1*,(MAIN)
     IN   SUB1$$SYSDTA($ALLBUT,6)
  SEG     SEG2*,(MAIN)
     IN   SUB3$$SYSDTA($ALLBUT,6)
  SEG     SEG3*,(SEG1,SEG2)
     IN   SUB4$$SYSDTA($ALLBUT,6)
  END
```

Using C$DML:

```
IBANK,UX  C$DML
IBANK,M   I1,022000
SEG      MAIN
    IN   MAINPROG
    IN   SUB2
SEG      SEG1*,(MAIN)
    IN   SUB1
SEG      SEG2*,(MAIN)
    IN   SUB3
SEG      SEG3*,(SEG1,SEG2)
    IN   SUB4
DBANK,MC  D1
SEG      MAIN
    IN   MAINPROG
    IN   SUB2$$SYSDTA
    IN   SYS$LIB$*ACOB-DML.CBEP$$ACOB/FULL-LIB-CB
SEG      SEG1*,(MAIN)
    IN   SUB1$$SYSDTA($ALLBUT,6)
SEG      SEG2*,(MAIN)
    IN   SUB3$$SYSDTA($ALLBUT,6)
SEG      SEG3*,(SEG1,SEG2)
    IN   SUB4$$SYSDTA($ALLBUT,6)
END
```

Section 20
Related Processors

This section documents the ASCII COBOL interface with related processors. The following processor interfaces are described in this section:

- Define File Processor (DFP)
- Processor Common Input/Output System (PCIOS)
- Shared File System (SFS)
- Sort-Merge

20.1. Define File Processor (DFP) Interface

The Define File Processor gives the ASCII COBOL user the facility to specify certain attributes of a file at execution time without the need to change the COBOL source statements, recompile, or recollect.

For additional information on the Define File Processor call and keyword parameters, refer to the *OS 2200 Define File Processor (DFP) Administration and Programming Reference Manual* (7830 7816).

Prior to file initialization time, the use name DFP$ must be attached to the file containing the Define File Blocks (DFB). At file initialization time, ASCII COBOL searches the DFP$ file for an omnibus element of the same name as specified as the file name in the ASSIGN clause.

20.1.1. Parameters

20.1.1.1. Non-MSAM Files

You may experience difficulties in running COBOL programs accessing files, especially where several files are accessed simultaneously or where the file records are very large. The reason is that the COBOL compiler tries to optimize the number of buffers and the size of the buffers so that the PCIOS processor may operate more efficiently. Thus, this parameter provides the ability to override the BLOCK CONTAINS clause and number of buffers allocated without recompiling or remapping, and thus, to adjust the current operating environment.

The BLOCK keyword parameter list is as follows:

```
BLOCK = bs/bo/bn/bl/bu
```

where:

bs

is the BLOCK CONTAINS clause value of *integer-2*. It is an integer in the range $0 < bs < 131072$.

bo

is the buffer offset parameter. This parameter is valid only for input tape files assigned to INTERCHANGE. It is an integer in the range $0 \leq bo \leq 99$.

bn

is the number of buffers parameter. It is an integer in the range $0 < bn < 64$. *However, values of bn greater than 2 are meaningful only for INDEXED files.*

bl

is the replacement for BLOCK CONTAINS clause value of *integer–1*. It is an integer in the range $0 < bl < 131072$. This parameter is meaningful only for INDEXED files. Also, this parameter is meaningful only when the bs parameter is specified and $bl \leq$ bs.

bu

is the BLOCK CONTAINS clause RECORDS or CHARACTERS option. It is coded as one of the following:

```
RECORDS

CHARACTERS
```

In the absence of a *bu* parameter, CHARACTERS is assumed. This parameter is meaningful only when the *bs* parameter is specified.

20.1.1.2. MSAM Files

A method for altering the compiler defaults for buffer size and number of buffers for 1974 Standard Indexed files (in other words, PCIOS MSAM files) is provided.

The BLOCK keyword parameter list is as follows:

```
BLOCK = bs//bn/bl//bu
```

where:

bs

buffer size (see 20.1.1.2.1)

bn

number of buffer parameters ($4 \leq bn \leq 63$); for OUTPUT and I–O mode, *bn* must be ≥ 5 (see 20.1.1.2.2)

bl

block loading factor (see 20.1.1.2.3)

bu

blocking unit, WORDS, CHARACTERS or RECORDS (ASCII characters are assumed)

20.1.1.2.1. Buffer Size

Through the DFP, you can alter the compiler-generated buffer size by specifying *bs* or *bs* plus *bu*.

If *bu* = CHARACTERS or *bu* parameter absent, the buffer size used is *bs* rounded up to words then rounded up again to a multiple of 448 words.

If *bu* = WORDS, the buffer size used is *bs* rounded up to a multiple of 448 words.

If *bu* = RECORDS, the buffer size used is the following (rounded up to a multiple of 448 words):

```
bs * (mrl + 2 + ndk) + 6
```

This formula is the same formula used by ASCII COBOL (see 17.2.2.2); however, no minimum size check is made.

20.1.1.2.2. Number of Buffers

Through the DFP, you can alter the compiler generated number of buffers by specifying a DFP BLOCK *bn* parameter in the range 5 through 63 for OUTPUT and I–O and 4 through 63 for INPUT. If *bn* is less than the compiler generated value, only the top level index block for the RECORD key is locked into main storage at run-time. If *bn* is 4, no index blocks are locked into main storage.

20.1.1.2.3. Block Loading Factor

The DFP *bl* parameter is equivalent to the *integer–1* field in a BLOCK CONTAINS *integer-1* TO *integer-2* ... clause.

The block loading factor is the number of words reserved in a block at file creation time and is calculated as:

```
((bs - bl) / bs) * buffer size
```

If *bs* is not present, or if *bl* is not less than *bs*, the *bl* parameter is ignored.

20.1.1.3. Record Formats

The *rf* parameter of the RECORD keyword parameter list provided by the DFP is allowed for INTERCHANGE and System Data Format (SDF) files only. No validation checks on the parameter values specified are made at run time.

```
RECORD = /rf
```

where *rf* can be:

U	Undefined
F	Fixed
D	Variable
FB	Fixed-blocked
DB	Variable-blocked
FS	Fixed-segmented
DS	Variable-segmented
FBS	Fixed-blocked-segmented
VBS	Variable-blocked-segmented

20.1.1.4. Prep Factors

The PREP keyword parameter is valid only for mass-storage files

```
PREP = pf
```

where *pf* can be:

0	MASS–STORAGE
28	MASS–STORAGE–28
56	MASS–STORAGE–56
112	MASS–STORAGE–112

For RELATIVE files (direct access), the PF parameter may be coded as 112, 224, 448, 896, or 1792.

Note: *Any other values of pf are ignored for mass-storage files and rounded down to the next value allowed for RELATIVE files.*

20.1.1.5. Label Records

The LABEL keyword parameter list is:

```
LABEL = lt/li/le/la
```

where *lt* can be:

```
OMITTED

STANDARD

USER
```

Although this parameter is provided via the DFP, complete freedom of choice is not possible because of the label routine collection requirements. Also, in the case of user labels, declaratives must be coded in the source program in order to make this parameter meaningful. Therefore, this parameter may change the amount of label handling performed on a file only to a lower state than was declared in the source program. More precisely:

Source Program Stated	Allowable *lt* values
USER (data-name)	USER, STANDARD, OMITTED
STANDARD	STANDARD, OMITTED
OMITTED	OMITTED

If this parameter is used to change the label processing functions specified in the source program, the physical files that are to be processed must conform to the "new" mode of processing stated.

li

is the file identifier parameter. It may be from 1 to 17 characters in length. It is left justified, space filled.

le

is the file expiration period parameter. It is an integer in the range $0 \leq le \leq 131072$. It specifies the number of days the file is to be retained.

la

is the file accessibility code parameter. It is a 1-character code. The allowable characters are the digits 1 through 7 and the space.

Note: *The "li" parameter may also be specified for SDF sequential and relative DISC files.*

20.1.1.6. Verify Parameter

The VERIFY parameter specifies the file identifier for an existing file that requires verification before it can be overwritten using a new file identifier as follows:

```
VERIFY = vi
```

The *vi* parameter specifies the file identifier to be verified.

20.1.1.7. EBCDIC Translate

The OPTIONS keyword parameter is:

```
OPTIONS = A
```

The *A* parameter indicates that for an EBCDIC file, a 256-character EBCDIC/ISO8859-1 translate table is used. Absence of the *A* parameter indicates a 128-character translate table is used.

20.1.1.8. OPEN Parameter

The OPEN parameter gives the status of the file at open time. It is coded as:

```
OPEN = os
```

where *os* is the open status and is interpreted as:

NEW

removes the effect of the OPEN statement EXTEND option if specified for a standard TAPE file.

OLD

if specified for a file, an @ASG,A of the file will be attempted if the file is not assigned at open time.

EXTEND

if specified for a standard TAPE file opened for output, the action is the same as if the OPEN statement had the EXTEND option specified.

20.1.1.9. CLOSE Parameter

The CLOSE parameter specifies the disposition of the file at close time. It is coded as:

```
CLOSE = cd
```

where *cd* is the close disposition parameter and is interpreted as:

REWIND

if specified for 1974 Standard TAPE or INTERCHANGE file, the file is rewound regardless of a no rewind option on the CLOSE statement.

LEAVE

if specified for 1974 Standard TAPE or INTERCHANGE file, the file is not rewound at close time.

FREE

if specified for 1974 Standard TAPE or INTERCHANGE file, or SDF file, the file is freed (@FREE) at close time.

20.1.1.10. SHARE Parameters

The SHARE keyword parameter specifies that a file is shared (processed by the Shared File System) and that the Image Transformation Facility (ITF 1100) will be involved in processing the file. This parameter is only allowed for RELATIVE and INDEXED I–O files. The format is:

```
SHARE = shr/itf
```

where:

shr

is the SHARE parameter, and has one of the following values:

YES

the file is to be processed by the Shared File System (SFS 2200).

NO

the file is to be processed by PCIOS.

itf

is the ITF indicator, and has one of the following values:

ITF

the Image Transformation Facility System (ITF) is used to view the file's records. This file must be processed by SFS.

NOTIF

the Image Transformation Facility System (ITF) is not used to view the file's records.

20.1.2. Concatenation of Data Files

Concatenation of data files allows separate ASCII COBOL physical files to be processed as one ASCII COBOL logical file. This is accomplished by linking each file's Define File Block (DFB). The first and subsequent DFBs must be available via the general interface mechanism described at the beginning of this subsection. All DFB keyword parameters are interpreted from the first DFB; only LABEL–ID, OPEN, and CLOSE parameters are interpreted from each subsequent DFB.

File concatenation is limited to the input (forward only) mode of 74 ANSI TAPE, INTERCHANGE, and SDF sequential DISC files. Concatenation of a set of ASCII COBOL files is allowed only on component files of the same file format, and uniform labeling conventions must exist for all component files. A component file is one of several files that compose a concatenated file, and a concatenated file is the entire collection of COBOL files that are processed as a single logical file.

When a data file of the type just mentioned is opened for input, the concatenation field of the DFB is checked if a DFB exists for that file. If the field is zero, concatenation terminates. If nonzero, the field contains the element name for the DFB of the next file in the concatenation chain. All DFBs must be accessible in a file DFP$ either by assigning the file as DFP$ or by linking it to DFP$ via an @USE statement.

When an end-of-file is detected on an input file that has a DFB, the concatenation field of the DFB is checked. If concatenation is specified, the present file is automatically closed observing the DFB CLOSE parameter, the next DFB is retrieved, and the data file specified in the DFB is opened. If the DFP OPEN parameter for this file is OPEN=OLD and the file was cataloged, it is also automatically assigned. Processing of the input file continues as if no end-of-file had occurred. A true end-of-file condition is not raised until the last record of the final component file is processed.

The SELECT statement in the COBOL program for the concatenation file may specify the OPTIONAL clause but, if the first component file is present, all nonfirst component files must be present. If the first component file is not present, the entire concatenation file is treated as not present.

The following example shows how the Define File Processor is used to produce the Define File Blocks required to concatenate three component files:

```
@DFP,E F1.,DFP$.FA,DFP$.FB
  Keyword parameters describing F1*
@EOF
@DFP,E F2.,DFP$.FB,DFP$.FC
  Keyword parameters describing F2*
@EOF
@DFP,E F3.,DFP$.FC
  Keyword parameters describing F3*
@EOF
```

File DFP$ must be assigned. Files F1, F2, and F3 must be preassigned, or they must be cataloged with OPEN parameters of OPEN=OLD, in which case they are dynamically assigned by the run.

The COBOL source lines to process these files as a single logical file would have the form:

```
SELECT FA ASSIGN TO implementor-name.
    .
    .
    .
OPEN INPUT FA.
READ FA AT END imperative-statement.
```

The *imperative-statement* is not executed until an end-of-file is reached on file F3.

20.2. Processor Common Input/Output System (PCIOS) Interface

The Processor Common Input/Output System (PCIOS) is used as the ASCII COBOL Mass Storage Control System (MSCS) and the ASCII COBOL Tape Control System and serves as the interface between COBOL programs and Input/Output devices. All ASCII COBOL nonsymbiont files that conform to the American National Standard Institute (ANSI) COBOL X3.23–1974 specifications are processed by PCIOS (except for shared files).

If you are using a common banked ASCII COBOL library, then common banked PCIOS is required.

If you are using a relocatable COBOL library, then either common banked (IN CBEP$$PCIOS) or relocatable PCIOS (LIB SYSLIB*PCIOS) can be used.

For more information concerning PCIOS, refer to the *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588).

20.3. Shared File System (SFS 2200) Interface

The Shared File System (SFS) can be used as the ASCII COBOL Mass Storage Control System (MSCS) for Relative I–O (DSDF) and Indexed I–O (MSAM) files and serves as an interface between COBOL programs and input/output devices.

To access files through the OS 2200 Shared File System (SFS 2200), you must install the SFS 2200 common I–O modules and banks. This interface allows multithread access to Indexed I–O (MSAM) and Relative (DSDF) files only. You can process these files singly or concurrently in a multithread environment and designate them as recoverable. PCIOS must co-exist with SFS 2200 to allow single-thread processing of the nonshared file access modes (SSDF, DSDF, MSAM, ANSI TAPE, ISAM). All programs that interface with SFS should use explicit thread control via the Relational Syntax Analyzer (see the *Relational Database Server for ClearPath 2200 SQL End Use Guide*).

The SFS interface in ASCII COBOL common banks references the default application group (UDSSRC). To reference an alternate application group, use one of the following methods:

- Modify your collection to reference a relocatable ASCII COBOL library (such as LIB ACOBLIB) and include the SFS CBEP$ element for the alternate application group (for example, IN UDS$$TWO*SFS.CBEP$$SFS). This method will always ensure that the alternate application group is referenced. You can also optionally modify your ASCII COBOL program as described in the second method.
- If the SFS code banks in the default and alternate application groups are compatible, modify the ASCII COBOL program to specify the alternate application group name using an RSA BEGIN THREAD command, for example:

```
BEGIN THREAD <thread-name> FOR APPLICATION APPTWO ...
```

The ASCII COBOL Relative and Indexed I–O interface modules (C$SCIM and C$SIIO) reference the SFS 2200 common banked entry points if the SFS 2200 interface is present when you install the COBOL libraries. One of the following methods must be used to designate a file as shared:

1. At compile time, specify A–SHARED–FILE as the *implementor-name-1* on the ASSIGN clause in the SELECT clause. This method will cause all DFP processing for the file to be bypassed.
2. At runtime, use the SHARE parameter of the Define File Processor (see 20.1.1.10, DFP Interface).

For more information about SFS 2200, see the *Shared File System (SFS 2200) Interface Description* (7831 0786).

20.3.1. Rollback and Deadlock Situations

The Shared File System provides for concurrent file access as well as recovery. When a rollback or deadlock status is returned by the Shared File System, Table 20–1 summarizes the actions that ASCII COBOL will take during its error processing. These actions depend upon whether the user program specified a FILE STATUS clause in the file control entry of the shared file and/or defined a USE procedure for the shared file in the declaratives portion of the Procedure Division. It must be recognized that when an internal rollback or deadlock occurs and the file is declared as recoverable, all updates not yet committed are lost.

Table 20–1. ACOB Rollback/Deadlock Error Processing

	FILE STATUS SPECIFIED	NO FILE STATUS SPECIFIED
USE PROCEDURE SPECIFIED	1. Define File Status as: Status Key 1 = 9 Status Key 2 = binary 20 (deadlock) binary 21 (rollback) 2. Close all open shared files 3. Process the declarative 4. Normal return	1. Close all open shared files 2. Issue warning diagnostic 1502 (deadlock) 1503 (rollback) 3. Process the declarative 4. Normal return
NO USE PROCEDURE SPECIFIED	1. Define File Status as: Status Key 1 = 9 Status Key 2 = binary 20 (deadlock) binary 21 (rollback) 2. Close all open shared files 3. Normal return	Issue abort diagnostic 1504 (rollback) 1505 (deadlock)

20.4. Sort-Merge Interface

The Sort-Merge interface provides the ASCII COBOL user the facility to order one or more files of records or combine two or more identically ordered files of records, according to a set of user-specified keys.

20.4.1. Main Storage Size

At execution time, significant main storage is required to accomplish a sort or merge. The user can dynamically change the amount of main storage needed for a sort or merge by using the R$CORE feature. This feature requires mass storage assignment of a file name R$CORE whose maximum granularity on this assign will be used to determine the amount of main storage that is required for the sort or merge. The ASCII COBOL library multiplies the maximum granularity on the @ASG,T R$CORE statement by 1,000 to determine the actual amount of storage to dynamically assign. In the absence of an R$CORE statement, 22,500 words are used as the storage assignment. An example showing how to specify a 140,000 word Sort-Merge area is:

```
@ASG,T R$CORE.,F///140
```

20.4.2. Sort–Merge Files

Special Sort-Merge files must be assigned prior to the execution of the program using the external names "XA" through "XZ" for the required mass storage or tape. See the *OS 2200 Sort/Merge Programming Guide* (7831 0687) for a more detailed discussion of these assignments.

20.4.3. Suppression of SORT Console and Log Messages

By default, the console and/or log messages are displayed when a program executes SORT/MERGE statements. ASCII COBOL provides the user with a run-time facility to suppress or enable these messages. To suppress or enable the console and/or log messages, the following statements can be specified in a COBOL program:

To suppress console messages, use:

```
CALL 'SUPCON$'.
```

To suppress log messages, use:

```
CALL 'SUPLOG$'.
```

To suppress console and log messages, use:

```
CALL 'SUPCONLOG$'.
```

To enable console messages, use:

```
CALL 'ENACON$'.
```

To enable log messages, use:

```
CALL 'ENALOG$'.
```

To enable console and log messages, use:

```
CALL 'ENACONLG$'.
```

When the CALL statement to suppress the console and/or log message is executed, these are suppressed until a CALL statement to enable these messages is executed or the end of the program is reached.

Section 21
Transaction Processing

21.1. General

This section describes programming considerations, restrictions, and special collection requirements for executing ASCII COBOL programs in the TIP and HVTIP environments. The ASCII COBOL TIP library, normally SYSLIB*ACOB–TIPLIB, contains Message Control System (MCS) routines with TIP interfaces and main storage buffer elements.

21.2. Message Control System (MCS)

The ASCII COBOL MCS routines with TIP interfaces provide a direct interface between the ASCII COBOL SEND and RECEIVE statements and TIP. It is assumed that the ASCII COBOL programs are registered as TIP programs, and that each program will be initiated to receive and process one transaction. Each program may also send a message (or messages) to communications devices or other programs. All scheduling and routing is under the control of the TIP system, and the Communications Management System (CMS 2200) is used to enqueue and dequeue messages rather than MCS. The ASCII COBOL MCS routines use COMPOOL interfaces for all messages. No Message Control Bank (MCB) interfaces are provided.

21.2.1. The RECEIVE Statement

To receive the text of an input transaction, the program executes a RECEIVE statement. You do not have to update the QUEUE field of the INPUT CD since there is, at most, one message available for a single execution of a transaction program at any time. After the execution of the RECEIVE statement, the fields of the INPUT CD are set as follows:

QUEUE
: is initialized to TIP.

SUB–QUEUE–1
: contains spaces.

SUB–QUEUE–2
: contains spaces.

SUB–QUEUE–3
: contains spaces.

MESSAGE TIME
: contains spaces.

MESSAGE DATE
: contains spaces.

SYMBOLIC SOURCE
: if the transaction came from a terminal, this field contains the Position ID (PID) for that device. If the transaction was from another ASCII COBOL program, this field contains the PID of that program's transaction. The PID is right justified in the last four characters of this field and zero filled. If the SOURCE (in other words, the PID) is unknown, this field contains spaces.

TEXT LENGTH
: is the number of characters transferred to the specific identifier. (On a transmission from some devices, the character covered by the cursor is transmitted. This may result in a blank character at the end of the message.)

END KEY

is 0 until the entire message has been received into the ASCII COBOL program, at which time it is 2.

STATUS KEY

see Table 13-1 and Table 13-2.

MESSAGE COUNT

is 1 until the complete message has been received by the program, at which time it is 0.

The concept of message segments is meaningless in this environment. End of Segment Indicators (ESI) are ignored, and End of Group Indicators (EGI) and End of Message Indicators (EMI) are equivalent. If no message text is available during the execution of a RECEIVE statement, the imperative statement in the NO DATA phrase is executed.

21.2.2. The SEND Statement

To execute a SEND statement, you must update the fields of an OUTPUT CD. Since ASCII COBOL is interfacing through CMS to TIP programs and devices, the following conventions must be followed:

For program to program, SYMBOLIC DESTINATIONS must be:

PROG *xxxxxx*

One space after PROG is required. The *xxxxxx* parameter must contain the TIP transcode for the called program, left justified and space filled.

For program to output device, SYMBOLIC DESTINATIONS must be:

DEVICE *nnnn*

The word DEVICE is not required, but the last four characters *nnnn* of this field must be numeric digits reflecting a valid TIP PID. Normally, you should move the SYMBOLIC SOURCE of transaction received to this field for output to the same device. A SEND without EMI is permitted, as well as multiple destinations. However, a message for any destination is not stored into COMPOOL until a complete message with EMI is sent for that destination.

The ENABLE and DISABLE statements are transparent, but are syntactically allowed in a TIP environment.

Since a program is only scheduled when a message is available, the MESSAGE COUNT returned on execution of an ACCEPT MESSAGE COUNT is 1 until the complete message is received, at which time MESSAGE COUNT is 0.

21.3. Main Storage Requirements

Since efficiency is important in the transaction environment, all main storage should be obtained at collection time. For this purpose, 99 main storage elements reside in the ASCII COBOL TIP library. They are named C$SMC*n*, where *n* is the number of 512-word main storage blocks reserved in the C$SMC*n* element. One of these elements must be explicitly included in the collection of every ASCII COBOL transaction program. If this element is the last element explicitly included in the program's collection, an ER MCORE$ will be allowed to obtain additional space if necessary; otherwise, an ER MCORE$ will be prohibited and transaction programs will abort with diagnostic 1403A if insufficient space is allocated.

The exact size of the buffer element required will depend upon the program content. Use the following guidelines for determining the appropriate size of the element to be included in a non-HVTIP program collection.

- Begin with one to five core blocks for general COBOL use.
- Increase the size by the space required for file record areas, file buffer areas, and the SORT/MERGE main storage requirements.
- Add to that size the space requirements of other products (for example, DPS) that are used by the transaction.

If an element larger than C$SMC99 is required, you may generate your own buffer element by creating a MASM element as follows:

```
$(0)                    .
C$FIRST*                .
   $RES   size          .
C$LAST*                 .
   $RES   1             .
   $END                 .
```

where *size* is the number of words of main storage required.

21.4. Transaction Interface Processing (TIP) Considerations

The following subsections describe restrictions and collection requirements for non–HVTIP ASCII COBOL TIP transactions.

21.4.1. Restrictions

ASCII COBOL programs may be registered with TIP as a self-destructive, self-initializing, or reentrant transaction. The block-transfer loading and sticking options may be used with self-initializing and reentrant transactions. The following programming restrictions must be observed for self-initializing and reentrant transactions:

- The contents of data items with initial values can not be altered.
- Asynchronous processing is not allowed.
- I–O to nonprint files which are not processed by PCIOS or SFS is not allowed.
- Report writer is not allowed.
- I–O to a file specified in an APPLY EXREF/EXDEF clause is not allowed.

21.4.2. Collection Considerations

The collection of all ASCII COBOL transaction programs require that two elements (in other words, C$S400/TIP and C$SMCn) be included from the ASCII COBOL TIP library (SYSLIB*ACOB–TIPLIB). Reentrant transactions are restricted to one static I-bank, which must be the control bank, and one D-bank.

The collection of ASCII COBOL communication programs may require special collection considerations depending upon whether or not the default ASCII COBOL library contains TIP interfaces. The collection examples in the following subsections assume that SYSLIB*ACOB contains the ASCII COBOL library, SYSLIB*ACOB–TIPLIB contains the ASCII COBOL TIP library, and TIP$*TIPLIB$ contains the TIP primitive library. In addition, the collection examples assume that the common bank C$DML is not being used. If you want to use C$DML, the collection examples must be modified accordingly (see 19.3.3).

21.4.2.1. Noncommunication Program Example

If the relocatable element C$S205/TIP exists in the default library, ASCII COBOL communication and noncommunication programs should be collected using one of these sample collections.

```
@MAP,IS ,PROG
IN FILE.PROG
IN SYS$LIB$*ACOB-TIPLIB.C$S400/TIP
IN SYS$LIB$*ACOB-TIPLIB.C$SMCn
```

If a reentrant online program is required, the program must be collected with the I-bank as the control bank, as follows:

```
@MAP,IS ,PROG
IBANK,CM IB
IN FILE.PROG
LIB (IB/$ODD,DB/$EVEN)
LIB SYS$LIB$*ACOB(),TIP$*TIPLIB$()
DBANK,M DB
IN MEM$ERR
IN FILE.PROG
IN SYS$LIB$*ACOB-TIPLIB.C$S400/TIP
IN SYS$LIB$*ACOB-TIPLIB.C$SMCn
```

where C$SMCn is one of the main storage buffer elements described in 21.3.

21.4.2.2. Communication Program Examples

If the relocatable element C$S205 exists in the default library, ASCII COBOL communication programs must be collected using one of these sample collections.

```
@MAP,IS ,PROG
IN FILE.PROG
IN SYS$LIB$*ACOB-TIPLIB.C$S400/TIP
IN SYS$LIB$*ACOB-TIPLIB.C$SMCn
LIB SYS$LIB$*ACOB-TIPLIB,SYS$LIB$*ACOB
LIB SYS$LIB$*ACOB-TIPLIB,TIP$*TIPLIB$
NOT SYS$LIB$*ACOB.C$S200,.C$S205,.C$S210,.C$S230
NOT SYS$LIB$*ACOB.C$S240,.C$S250,.C$S260,.C$S400
NOT SYS$LIB$*ACOB-TIPLIB.C$D8BT,.C$RET0,.C$S30S
NOT SYS$LIB$*ACOB-TIPLIB.C$S310,.C$S390,.C$S395
```

If a reentrant online program is required, the program must be collected with the I-bank as the control bank, as follows:

```
@MAP,IS ,PROG
IBANK,CM ,IB
IN FILE.PROG
LIB (IB/$ODD,DB/$EVEN)
LIB SYS$LIB$*ACOB-TIPLIB(),SYS$LIB$*ACOB()
LIB SYS$LIB$*ACOB-TIPLIB(),TIP$*TIPLIB$()
DBANK,M DB
IN MEM$ERR
IN FILE.PROG
IN SYS$LIB$*ACOB-TIPLIB.C$S400/TIP
IN SYS$LIB$*ACOB-TIPLIB.C$SMCn
NOT SYS$LIB$*ACOB.C$S200,.C$S205,.C$S210,.C$S230
NOT SYS$LIB$*ACOB.C$S240,.C$S250,.C$S260,.C$S400
NOT SYS$LIB$*ACOB-TIPLIB.C$D8BT,.C$RET0,.C$S30S
NOT SYS$LIB$*ACOB-TIPLIB.C$S310,.C$S390,.C$S395
```

where C$SMCn is one of the main storage buffer elements described in 21.3.

21.5. HVTIP Considerations

21.5.1. General

ASCII COBOL allows programs to be executed in the high volume TIP (HVTIP) environment. The D-bank is made self-initializing by utilizing the relocatable segment (RSEG) capability. The following presumes a working knowledge of the HVTIP program environment.

An HVTIP transaction system consists of a set of independently collected program banks that link to each other dynamically at run time, and can be independently updated; in other words, recompiled, recollected, and reloaded online without need for other changes. CALL$ and XFR$ (in other words, CALL and TRANSFER) are the two TIP executive requests that cause the I-bank of an HVTIP program to be switched. They are analogous to the effect of an LIJ instruction in a non-HVTIP program.

To create high volume banks from COBOL subprograms involves writing a main transaction driver program and several special case transaction processing subprograms. Both the main and the subprograms can be written in COBOL. The main program must define all global work space in Common–Storage (this includes global COBOL data and the DMCA record area). Subprograms then have global data in Common-Storage and local data in Working–Storage.

Subprograms may be defined as either HVTIP bank entry-points subprograms or as "local" subprograms. HVTIP bank entry-point subprograms are defined by an IBJLNK Collector directive and can be referenced by an HVTIP bank. "Local" subprograms are defined by IN or LIB directives and can be referenced only by programs within the HVTIP bank.

If a COBOL subprogram is defined at collection time as existing under a TRANSFER IBJLNK directive, all previously allocated user work space (except for that of the main program) is released at run time before its own local work space is allocated and initialized.

A subprogram defined at collection time under a CALL IBJLNK directive, at run time, causes its local work space to be initialized at first call and to remain in its last used state for subsequent calls within the transaction. Local work spaces accumulate as the transaction cascades through several HVTIP banks via CALL$, and are released only at transaction termination or on a transfer (in other words, XFR$), or by explicitly programmed cancellation (in other words, CANCEL statement).

Basically, the ACOB/HVTIP interface is handled entirely within the main program or subprogram prologue routines. The collection process is the major consideration for creating HVTIP applications.

21.5.2. Creating Module Name Equate Elements

To define the HVTIP bank-to-bank calling relationships, either one or two Module Name Equate (MNQ) elements are required. These elements relate the entry-point name of each HVTIP subprogram bank, as used on a COBOL CALL statement, to the library and bank numbers the Exec uses to identify the bank.

Normally, there are two such elements defined for the complete set of HVTIP transaction programs that make up the application system. They should be created as part of the overall system design then placed in an appropriate user file from where they can be accessed during the collection of each HVTIP bank. You should place both the source and relocatable form of these elements in the user file, and use the names CALL$MNQ and XFR$MNQ for these elements.

One element, CALL$MNQ, defines all those subprogram banks that are to be entered using a CALL$. The other, XFR$MNQ, is for all those to be entered using a XFR$.

Note: *A subprogram bank should not be entered by both CALL$ and XFR$.*

Each entry point in an MNQ element must be defined as a 36–bit value as follows:

```
S1     library number
S2,S3  bank number
H2     (not used)
```

The following is an example of creating the MNQ elements:

```
@MASM,I     APPL.CALL$MNQ
SUB1*     $EQU      010001000000      .
C$BSUB1*  $EQU      0                 .
SUB3*     $EQU      010003000000      .
C$BSUB3*  $EQU      0                 .
          $END

@MASM,I     APPL.XFR$MNQ              .
SUB2*     $EQU      010002000000      .
C$BSUB2*  $EQU      0                 .
          $END
```

These elements define an HVTIP system consisting of two CALL$ subprogram banks and one XFR$ subprogram bank contained in HVTIP library number 01 as banks 01, 02, and 03, respectively, in addition to any ICPs defined by VALTAB. Entry points of the form C$B*xxxx* are included for completeness to satisfy references to the data areas of the called subprograms that are otherwise undefined.

COBOL names may contain a minus character, but such a name is not acceptable as a MASM label. In such a case, you should use a $INFO 6 directive to define the COBOL entry point.

21.5.3. TIP Interface

A program in the ICP bank (possibly the main program) should do a COMPOOL primitive call to read the incoming message or do a RECEIVE. The ICP then contains logic to process the message or branch to a subprogram module to process it. Alternatively, a COMPOOL primitive call can be made in one of the subprogram banks.

21.5.4. CALL Statement

The CALL statement is used to effect a CALL$ or a XFR$ from one HVTIP bank to another and to transfer control to a local subprogram. The execution of a CALL$ or XFR$ is determined at collection time by reference to the MNQ elements.

A subprogram called by using an XFR$ cannot thereafter execute a return from that subprogram, and the calling program never resumes execution unless called again, at which time it is reinitialized. No parameters can be passed on an XFR$ call.

The CALL *data-name* form of the CALL statement can be used to result in either form of HVTIP bank to bank switch, in which case the value of *data–name* should correspond to an entry point in one of the MNQ elements.

21.5.5. EXIT Statement

The EXIT PROGRAM statement is used to return from one HVTIP bank to another, except as noted earlier (XFR$).

21.5.6. CANCEL Statement

The CANCEL statement can be used to cause the local data space of a previously called HVTIP bank subprogram to be freed and, when next called, to be reallocated and reinitialized. A "local" subprogram must not be explicitly cancelled, but is automatically cancelled upon the cancellation of its HVTIP bank entry–point subprogram. Common banked subprograms are handled as though they were locally contained in the HVTIP bank making the call.

21.5.7. Writing MASM Subprograms for COBOL/HVTIP

Using HVTIP, the data area of a subprogram is located at an address that is assigned only at run-time. The COBOL compiler and library ensure that the data of a COBOL subprogram is addressed correctly. Since neither the COBOL compiler nor the COBOL library has control over a program written in MASM, it is the programmer's responsibility to know where the address of the MASM subprogram's data area can be found at run-time.

The data area address is available in H1 of the word pointed to by register A0 when the subprogram is entered at run time. The value contained in this half word is the address of the tag C$BXXXX if the MASM subprogram entry point has the name XXXX. The programmer should dedicate a register in the program to contain this value. All references to data contained in the MASM subprogram must use this register. The data area itself should be allocated following the tag C$BXXXX.

In the following example, register A3 is loaded with the address of the subprogram's data area immediately upon entry. The data area itself is contained in the same element as the code that references it.

```
          AXR$
$(0)
C$BASMEP*                          .
DUMMY     +015                     .
DATA      $EQUF     0,A3           .
   .
   .
   .
$(1)                               .
ASMEP*                             .
          L,H1       A3,0,A0       .
          L          A2,DATA
   .
   .
   .
          $END                     .
```

When collected, the $ODD and $EVEN location counters are split between the HVTIP bank and the relocatable segment (RSEG) of the D–bank. This data area is local to each HVTIP bank, so that different copies of this routine in different banks each access their own private data. To access a MASM-defined global data area from different HVTIP banks requires interaction with the COBOL run-time control tables and should be done only with extreme care.

21.5.8. Restrictions

The following is a list of source program and compiler option restrictions that must be adhered to in order for successful execution of ASCII COBOL programs in the HVTIP environment.

- The OBJECT–COMPUTER paragraph of all ASCII COBOL programs must contain the MEMORY 3 MODULES clause.
- APPLY EXREF and APPLY EXDEF are not allowed.
- Asynchronous processing is not allowed.
- Program segmentation is not allowed.
- Entry-point subprograms cannot be compiled with the 4 extra options.
- The ON OVERFLOW phrase of the CALL statement has no effect.
- The only ENTER statements allowed are ENTER ASM and ENTER MASM.
- Subprograms cannot be compiled with the 7 extra option.

21.5.9. Main Storage Requirements

As in the TIP environment, efficiency is important in the HVTIP environment, therefore, adequate storage should be allocated at collection time. The Initial Control Program (ICP) should reserve sufficient main storage for the entire transaction. HVTIP transactions require more main storage than TIP transactions. Thus, you must determine the main storage requirements based on the guidelines in 21.3 plus the requirements for all HVTIP bank RSEGs.

Upon transaction initialization, the ICP's RSEG is loaded into dynamic memory and remains in storage throughout the entire transaction. Each time a subprogram bank is called, the RSEG for that bank must be loaded into dynamic memory, unless it already resides in memory from a previous call. The RSEG will remain in memory until either the entry-point subprogram is CANCELled or a subprogram bank is called via XFR$.

21.5.10. Automatic Collection of ASCII COBOL HVTIP Transactions

Collecting ASCII COBOL/HVTIP programs are nontrivial. For this reason, a runstream called MAPHVTIP is available in the ASCII COBOL TIP library. The runstream uses an SSG skeleton.

Before using the skeleton, the source of the two MNQ elements, CALL$MNQ and XFR$MNQ, must exist in some user file as described in 21.5.2. If no CALL$ or XFR$ entry points exist, the corresponding element should exist but be empty. An empty element should never contain an END image. The last image in a nonempty element must be an END image. In addition, the relocatables for all ASCII COBOL programs must be located in a user file.

If you use a common-banked ASCII COBOL library, the element, CBEP$$ACOB, contains entry points C$S30S, C$S327, C$S35X, C$S335, C$S310, C$S31A, and C$S31C. Since elements C$S310 and C$S30S must be collected in HVTIP banks, these entry points cause the Collector to give a message saying that these entry points are already defined. This is not a problem. Leaving these entry points in CBEP$$ACOB allows the C$S310 and C$S30S routines in the C$INITIAL common bank to be used by your non-HVTIP applications.

To use the skeleton, enter as input:

```
@ADD SYS$LIB$*ACOB-TIPLIB.MAPHVTIP
user SGSs
@EOF
@EOF
```

The skeleton is designed so that each run can collect one or more HVTIP banks; i.e., ICP or subprogram banks. The HVTIP and LOCAL SGSs describe the individual banks and may occur any number of times. All other SGSs describe information common to all the banks and must occur only once. The only required SGSs are the HVTIP SGSs, one per bank to be collected, and the DMS SGS if DMS 2200 is being used.

Fields should be given in the sequence indicated. Optional fields are shown enclosed in [brackets]. Where needed, file names may include a qualifier, but periods are not allowed in file names or element names. All version-names are optional and default to blanks. Where the name of a COBOL relocatable element is indicated, it means the element containing the code. The SYSDTA elements are handled automatically.

21.5.10.1. User SGSs

The following subsections describe the user SGSs that can be supplied to the MAPHVTIP skeleton.

21.5.10.1.1. HVTIP

Format:

```
HVTIP prog-type,prog-id [ bank-type ] ;

[ IN,[element],[version] ] ;

[ OUT,[element],[version] ]
```

where:

prog-type
ICP or SUBPROGRAM

prog-id
the COBOL prog-id or ENTRY name of the entry program

bank-type
STATIC or DYNAMIC

Each HVTIP SGS calls for the collection of either an ICP or subprogram bank. The entry program (in other words, main program if an ICP, CALL$ subprogram if a subprogram bank) is the relocatable with element and version names given as IN, and the collected absolute has element and version names given as OUT. The default for both element names is the prog-id of the entry program. The default *bank-type* is the globally defined *bank-type* (see 21.5.10.1.7).

21.5.10.1.2. LOCAL

Format:

```
LOCAL local-to-id [ FILE,file ] ;

    lang,element[,version] ;

    [ lang,element[,version] ] . . . .
```

where:

lang
is COBOL, ASM, or MASM.

local-to-id
is TO, *prog–id* or ALL.

These SGSs name the relocatable elements of subprograms local to the HVTIP bank whose entry program has the given *prog–id*. If ALL is specified, the listed elements are to be included in all the HVTIP banks. There may be any number of LOCAL SGSs for each bank. FILE names the file containing the relocatables on each LOCAL SGS and defaults to the REL file. There may be one or more *lang* fields on each SGS.

21.5.10.1.3. REL

Format:

```
REL file
```

The REL SGS names the file containing the relocatables of all entry programs and all local subprograms named on LOCAL SGSs without a FILE. The default for REL is the file TPF$. If the REL file defaults to TPF$, the CONTROL SGS must be specified, and it must specify a file other than TPF$.

21.5.10.1.4. ABS

Format:

```
ABS file
```

The ABS SGS names the file to which the generated absolutes are directed. The default is the file specified on the REL SGS.

21.5.10.1.5. CONTROL

Format:

```
CONTROL file
```

The CONTROL SGS names the file containing the source of the equate elements CALL$MNQ and XFR$MNQ, if this type of bank switching is used. The default is the file specified on the REL SGS.

21.5.10.1.6. MAP

Format:

```
MAP file
```

The MAP SGS names the file that contains the MAP processor to be used for collections. The default is the system installed Collector (@MAP).

21.5.10.1.7. IBANKS

Format:

```
IBANKS ARE bank-type
```

where *bank-type* is STATIC or DYNAMIC.

The IBANKS SGS defines the type of I-bank generated for those banks that do not have a bank-type given on their HVTIP SGS. The default is STATIC.

21.5.10.1.8. DBANK

Format:

```
DBANK IS decimal-or-octal integer
```

The DBANK SGS defines the D-bank requirement for the transaction. The default is 4096 words.

21.5.10.1.9. USING

Format:

```
USING [ MCS ] [ SORT ] [ DEBUG ]
```

The USING SGS must specify MCS if the COBOL SEND/RECEIVE syntax is used for communicating messages with TIP, and/or must specify SORT if the COBOL SORT facility is used in the transaction, and/or must specify DEBUG only if frequency task generation is invoked in the initial control program (ICP).

21.5.10.1.10. LIST

Format:

```
LIST list-type
```

where *list-type* is SHORT, NORMAL, or LONG.

The LIST SGS gives the type of listing required from the MAP processor. The default is NORMAL.

21.5.10.1.11. COBOL

Format:

```
COBOL [ LIB,libfile ] [ TIPVERSION,tipfile ] ;

      [ COMMON BANKED [ CBEP$,cbfile ] [ UTILITY,util-size ] ]
```

where:

util-size

is ASCII, FIELDATA, and NONE.

libfile

is the COBOL library file. This file defaults to SYSLIB*ACOB.

tipfile

is the ASCII COBOL TIP relocatable library. This file defaults to the file SYSLIB*ACOB–TIPLIB.

cbfile

is the file containing the CBEP$$ACOB relocatable element banks. This file defaults to the libfile..

The UTILITY option says whether the utility-based common bank, C$DML, contains only ASCII (the default) or also FIELDATA routines. This prevents the HVTIP I-banks from overlapping with the C$DML library common bank. If you do not want to use the C$DML common bank, UTILITY, NONE should be specified.

21.5.10.1.12. TIP

Format:

```
TIP LIB,file
```

The LIB file contains the TIP primitives including the RSEGLOAD routine. The default for this file is TIP$*TIPLIB$.

21.5.10.1.13. DMS

Format:

```
DMS [ LIB,file ] [ S$WORK,[file],[subschema] ] ;

    [ SORTBUF ] [ D$WORK,[file],[impart-prog] ]
```

The DMS SGS is required if the transactions use DMS 2200. However, all fields can be omitted.

Note: *This skeleton is not usable with single-thread DMR.*

The LIB file contains the DMS 2200 elements to be included in the HVTIP banks (COBLNK, LINKER, CLSEG$, SORTBUF). The default is SYS$*RLIB$.

The S$WORK and D$WORK files are where those elements can be found, and both default to the REL file. The parameter *subschema* is used as a version name to S$WORK and, if not specified, any subschema in that file is acceptable. The *impart–prog* is the COBOL prog-id of the program in the transaction that executes the IMPART statement, and defaults to the main program prog-id.

SORTBUF need only be specified if a set whose mode is pointer array is going to be loaded.

21.5.10.1.14. DPS

Format:

```
DPS local-to-id [FILE,file]
```

where *local-to-id* is TO,*prog-id* or ALL.

The DPS SGS names the HVTIP banks that are to include the DPS elements CBEP$$DPS and ACOB/INTERFACE. FILE names the file that contains these relocatables. The default is SYSLIB*DPS. If TO,*prog-id* is specified, only that HVTIP Ibank whose entry program is *prog-id* will include both elements. If ALL is specified, all HVTIP Ibanks will include both elements. This SGS assumes that DPS is installed as a common bank. LOCAL SGS must be used if DPS is not installed as a common bank. Multiple DPS SGSs may be specified, but only one DPS SGS can apply to each HVTIP Ibank.

21.5.10.1.15. RDMS

Format:

```
RDMS local-to-id [FILE,file]
```

where *local-to-id* is TO,*prog-id* or ALL.

The RDMS SGS names the HVTIP banks that are to include the RDMS elements CBEP$$RSA and RDMR-ACOBDAT. FILE names the file that contains these relocatables. The default is SYSLIB*RSA. If TO,*prog-id* is specified, only that HVTIP Ibank whose entry program is *prog-id* will include CBEP$$RSA. If ALL is specified, all HVTIP Ibanks will include CBEP$$RSA.

RDMR-ACOBDAT will be included in each HVTIP Dbank. Multiple RDMS SGSs may be specified, but only one RDMS SGS can apply to each HVTIP Ibank.

21.5.10.1.16. OWN

Format:

```
OWN LIB,file[,file. . . .]
```

These LIB files are user files to be searched first, in the order given, during the collections.

21.5.10.1.17. DBANKADDRESS

Format:

```
DBANKADDRESS start-address
```

The DBANKADDRESS SGS defines the D-bank start address of the HVTIP transaction that must be at least 040000 and greater than the maximum address calculated for any I-bank called by the transaction using the following formula:

```
LASTI$ + (19/18 * size of the RSEG) + 4
```

where LASTI$ is the Collector defined variable for the highest I-bank address.

The default D-bank start address is:

040000 if the DMS SGS is not specified.

0150000 if the DMS SGS is specified.

21.5.10.1.18. NOCOMMON

Format:

```
NOCOMMON
```

The NOCOMMON SGS indicates that the ICP does not contain a COMMON-STORAGE section. If this SGS is not specified, BLANK$COMMON will be included in the ICP's collection.

21.5.10.2. Examples of Map SGSs

Example 1:

```
HVTIP  ICP,MAIN
LOCAL  TO,MAIN  COBOL,XYZ  COBOL,ABC
HVTIP  SUBPROGRAM,SUB
LOCAL  TO,SUB  FILE,A*B  ASM,SHARED
LOCAL  TO,SUB  COBOL,P3
REL    MYFILE
DBANK  IS 512
COBOL  LIB,SYS$*ACOBLIB$
```

This example calls for the collection of one ICP and one subprogram bank. The ICP contains the main program in element MAIN and subprograms in elements XYZ and ABC. The subprogram bank contains the CALL$ entry program in element SUB and local subprograms SHARED and P3. All relocatables are in file MYFILE, except for the MASM routine SHARED in file A*B. The two absolutes are also generated in file MYFILE.

The initial transaction D-bank size is 512 words. A relocatable COBOL library is used in file SYS$*ACOBLIB$. The TIP library is file TIP$*TIPLIB$. The ASCII COBOL TIP relocatable library is SYSLIB*ACOB–TIPLIB. DMS 2200 is not used.

Example 2:

```
HVTIP  ICP,B1 STATIC  IN,Q1
HVTIP  SUBPROGRAM,B2  IN,Q2
HVTIP  SUBPROGRAM,B3  IN,Q3
HVTIP  SUBPROGRAM,B4  IN,Q4
MAP    UTILITY
ABS    MASTER
IBANKS ARE DYNAMIC
COBOL  TIPVERSION,COBTIP  COMMON BANKED
TIP    LIB,SYS$*RLIB$
DMS    S$WORK,,SS4
```

This example calls for the collection of one ICP and three subprogram banks. Each bank contains just one COBOL program. All the relocatables are in the TPF$ file, with element Q1 for prog-id B1 (the main program) and elements Q2, Q3, and Q4 for subprograms with prog-id B2, B3, and B4, respectively.

The subprogram banks are dynamic, while the ICP is static with an initial D-bank of 4096 words. The absolutes are placed in file MASTER, after collecting with the MAP processor in file UTILITY.

A common-banked COBOL library with C$DML is used with the ASCII COBOL TIP library in file COBTIP. The DMS 2200 subschema definition is in TPF$.S$WORK/SS4.

21.5.11. Program Example

21.5.11.1. General

The following is an example of a COBOL HVTIP transaction program and the steps that were taken to execute it.

21.5.11.2. COBOL Source Programs

The transaction program in this example consists of a COBOL main program and two COBOL subprograms. The main program uses the CALL identifier and the Message Control System (MCS). The data entered in the main program is the name of the subroutine to be called. The name is retrieved by executing a RECEIVE statement. Each of the subprograms are placed in a bank separate from the one containing the main program.

21.5.11.2.1. Main Program Source

The source code for the COBOL main program follows:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. COBHV1.
ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. UNISYS-1100.
OBJECT-COMPUTER. UNISYS-1100 MEMORY SIZE IS 3 MODULES.
DATA DIVISION.
COMMUNICATION SECTION.
CD     COMM-INPUT FOR INPUT
       TEXT LENGTH IS LENGTHI
       SYMBOLIC SOURCE IS MSG-SOURCE.
CD     COMM-OUTPUT FOR OUTPUT
       DESTINATION COUNT IS OP-DEST-CNT
       TEXT LENGTH IS OP-TXT-L
       SYMBOLIC DESTINATION IS OP-DEST.
COMMON-STORAGE SECTION.
01     COMCOM        PIC         X(80).
WORKING-STORAGE SECTION.
01     MY-DATA.
       02 MY-ACT     PIC         X(6).
       02 FILLER     PIC         X.
       02 MY-SUB     PIC         X(6).
       02 FILLER     PIC         X(67).
01     CALL-DATA.
       02 CALLID     PIC         X(6).
       02 FILLER     PIC         X(74).
01     MPARM1     PIC         H9(10) VALUE 1.
PROCEDURE DIVISION.
BEGIN. DISPLAY 'BEFORE RECEIVE' LENGTHI MSG-SOURCE UPON PRINTER.
       RECEIVE COMM-INPUT MESSAGE INTO MY-DATA.
       DISPLAY 'AFTER RECEIVE' LENGTHI MSG-SOURCE UPON PRINTER.
       DISPLAY MY-DATA UPON PRINTER.
       MOVE MY-SUB TO CALLID.
       DISPLAY CALLID UPON PRINTER.
       DISPLAY 'BEFORE CALL', MPARM1 UPON PRINTER.
       CALL CALLID USING MPARM1.
       DISPLAY 'AFTER CALL', MPARM1 UPON PRINTER.
       MOVE MSG-SOURCE TO OP-DEST.
       MOVE 80 TO OP-TXT-L.
       MOVE 1 TO OP-DEST-CNT.
       DISPLAY OP-DEST-CNT OP-DEST OP-TXT-L UPON PRINTER.
       SEND COMM-OUTPUT FROM MY-DATA WITH EMI.
       DISPLAY 'MSG SENT' UPON PRINTER.
       DISPLAY 'END COBHV1' UPON PRINTER.
       CANCEL CALLID.
       STOP RUN.
```

The start byte, length, and USAGE of the data item MY–SUB are key to the retrieval of the name of the subprogram to be called (see 21.5.11.6).

21.5.11.2.2. COBOL Subprogram COBHV2

```
            IDENTIFICATION DIVISION.
            PROGRAM-ID. COBHV2.
            ENVIRONMENT DIVISION.
            CONFIGURATION SECTION.
            SOURCE-COMPUTER. UNISYS-1100.
            OBJECT-COMPUTER. UNISYS-1100 MEMORY SIZE IS 3 MODULES.
            DATA DIVISION.
            COMMON-STORAGE SECTION.
            01  ACOMCOM                  PIC X(80).
            WORKING-STORAGE SECTION.
            01  SUB1-DATA                PIC H9(10) VALUE 2.
            LINKAGE SECTION.
            01  SUB1-PARM1               PIC H9(10).
            PROCEDURE DIVISION USING SUB1-PARM1.
            P1.
                DISPLAY 'IN             COBHV2' UPON PRINTER
                MOVE SUB1-DATA TO SUB1-PARM1.
                DISPLAY 'LEAVING COBHV2' UPON PRINTER.
            P2.
                EXIT.
```

21.5.11.2.3. COBOL Subprogram COBHV3

```
            IDENTIFICATION DIVISION.
            PROGRAM-ID. COBHV3.
            ENVIRONMENT DIVISION.
            CONFIGURATION SECTION.
            SOURCE-COMPUTER. UNISYS-1100.
            OBJECT-COMPUTER. UNISYS-1100 MEMORY SIZE IS 3 MODULES.
            DATA DIVISION.
            COMMON-STORAGE SECTION.
            01  COMCOM                   PIC X(80).
            WORKING-STORAGE SECTION.
            01  SUB2-DATA                PIC H9(10) VALUE 3.
            LINKAGE SECTION.
            01  SUB2-PARM1               PIC H9(10).
            PROCEDURE DIVISION USING SUB2-PARM1.
            P1.
                DISPLAY 'IN   COBHV3' UPON PRINTER.
                MOVE SUB2-DATA TO SUB2-PARM1.
                DISPLAY 'LEAVING COBHV3' UPON PRINTER.
            P2.
                EXIT.
```

21.5.11.3. C$S333

Because the CALL identifier is used (see 18.7.1), element C$S333 is updated as follows:

```
@MASM,E             SYS$LIB$*ACOB.C$S333,COBOL*CLIB.
-4O
                    CREATE 'COBHV2'
                    CREATE 'COBHV3'
@EOF
```

The file into which the relocatable element is placed must be prepped (@PREP) prior to running MAPHVTIP.

21.5.11.4. MNQ Elements

Since the COBOL main program used in this example passes parameters to each subroutine it may call, the element XFR$MNQ is empty. The entry-point-names of the subprograms called are COBHV2 and COBHV3. The MNQ elements are created as follows:

```
@ED,I    MYFILE.XFR$MNQ
@EOF
@ED,I    MYFILE.CALL$MNQ
COBHV2*             $EQU  010002000000
C$BCOBHV2*          $EQU  0
COBHV3*             $EQU  010003000000
C$BCOBHV3*          $EQU  0
                    $END
 @EOF
```

Both subprograms are placed in library file number 1.

21.5.11.5. SGS Input to MAPHVTIP

The following SGS was used as input to MAPHVTIP:

```
HVTIP               ICP,COBHV1
HVTIP               SUBPROGRAM,COBHV2
HVTIP               SUBPROGRAM,COBHV3
REL                 COBOL*TPF$
ABS                 COBOL*TPF$
CONTROL             COBOL*MYFILE
MAP                 SYS$LIB$*MAP
IBANKS    ARE       STATIC
DBANK     IS        4096
USING               MCS
LIST                LONG
COBOL               LIB,COBOL*CLIB TIPVERSION,COBOL*TIPTIP
                    UTILITY,NONE
TIP                 LIB,TIP$*TIPLIB$
```

Given the preceding input, the skeleton MAPHVTIP was instructed to find the relocatable elements produced by the COBOL compiler for the main program and subprograms in file TPF$. It was also instructed to find the source for the MNQ elements in file MYFILE.

The updated element C$S333 is in file COBOL*CLIB, and the ASCII COBOL TIP library is in file COBOL*TIPTIP. Both CLIB and TIPTIP must be prepped (@PREP) prior to running MAPHVTIP.

For the given SGSs, the skeleton produces three absolute elements: COBHV1, COBHV2, and COBHV3, and leaves them in file TPF$.

21.5.11.6. Program Input

Before you execute the transaction, a VALTAB entry must be created, a TIP library must be created, TIP storage must be allocated, and the three absolutes must be copied to the library file. See the *ASCII COBOL Runtime Technical Reference Manual* for more information.

The following data image was entered on a TIP terminal, beginning in column 1:

```
COBHV1 COBHV2
```

Note: *Only one blank space appears between COBHV1 and COBHV2. The cursor was positioned on the first blank space following the "2" in "COBHV2" when this line was transmitted.*

21.5.11.7. Program Output

The following output appeared on the printer. Each line of output starts in column 1.

```
BEFORE RECEIVE
AFTER RECEIVE 0014000000000008
COBHV1 COBHV2
COBHV2
BEFORE CALL 0000000001
IN COBHV2
LEAVING COBHV2
AFTER CALL 0000000002
00010080000000000008
MSG SENT
END COBHV1
```

The length of the text received (via RECEIVE) (14 characters) includes the blank character positioned under the cursor when the input line COBHV1 COBHV2 was transmitted.

Appendix A
Reserved Words

The following words are an inherent part of the OS 2200 American National Standard ASCII COBOL and cannot be used for data or procedure names:

A-SHARED-FILE
ACCEPT
ACCESS
ACTIVE
ACTUAL
ADD
ADVANCING
AFTER
ALL
ALPHABET
ALPHABETIC
ALSO
ALTER
ALTERNATE
AN
AND
ANS74-PRINTER
ANY
APPLY
ARE
AREA
AREAS
ASCENDING
ASM
ASSIGN
AT
AUTHOR

BEFORE
BEGINNING
BLANK
BLOCK
BOTTOM
BY

CALL
CANCEL
CARD-PUNCH
CARD-READER
CARTRIDGE-ID
CD
CF
CFH
CH
CHANGED
CHARACTER
CHARACTERS
CLASS-NAME
CLOCK-UNITS
CLOSE
COBOL
CODE
CODE-SET
COLLATING
COLUMN
COMMA
COMMON-STORAGE
COMMUNICATION
COMP
COMP-1
COMP-2
COMP-3
COMP-4
COMPACT
COMPUTATIONAL
COMPUTATIONAL-1
COMPUTATIONAL-2
COMPUTATIONAL-3
COMPUTATIONAL-4
COMPUTE
CONFIGURATION
CONSOLE
CONTAINS
CONTROL
CONTROLS
CONVERSION

COPY
CORE-INDEX
CORR
CORRESPONDING
COUNT
CREATION-DATE
CURRENCY
CYCLES

DATA
DATE
DATE-COMPILED
DATE-TIME
DATE-WRITTEN
DAY
DE
DEBUG-CONTENTS
DEBUG-ITEM
DEBUG-LINE
DEBUG-NAME
DEBUG-SUB-1
DEBUG-SUB-2
DEBUG-SUB-3
DEBUGGING
DECIMAL-POINT
DECLARATIVES
DELETE
DELIMITED
DELIMITER
DENSITY
DEPENDING
DESCENDING
DESTINATION
DETAIL
DIRECT
DISABLE
DISC
DISP
DISP-1
DISPLAY
DISPLAY-1
DIVIDE
DIVISION
DOWN
DUPLICATES
DYNAMIC

EGI
ELSE
EMI
ENABLE
END
ENDING

END-OF-PAGE
ENTER
ENTRY
ENVIRONMENT
EOP
EQUAL
EQUALS
ERROR
ESI
EVERY
EXAMINE
EXCEEDS
EXCEPTION
EXDEF
EXHIBIT
EXIT
EXREF
EXTEND

FD
FILE
FILE-ACCESS
FILE-CONTROL
FILE-ID
FILE-LIMIT
FILE-LIMITS
FILE-QUALIFIER
FILLER
FINAL
FIRST
FOOTING
FOR
FORM01
FORM02
FORM03
FORMS-REQUEST
FORTRAN
FREE
FROM
FTN

GENERATE
GIVING
GO
GREATER
GROUP

HEADING
HIGH-VALUE
HIGH-VALUES

HOLD

I-O
I-O-CONTROL
IDENTIFICATION
IF
IN
INACTIVE
INDEX
INDEXED
INDICATE
INITIAL
INITIATE
INLINE
INPUT
INPUT-OUTPUT
INSPECT
INSTALLATION
INTERCHANGE
INTERLOCK
INTERNAL
INTO
INVALID
IS

JUST
JUSTIFIED

KEY

LABEL
LAST
LEADING
LEFT
LENGTH
LESS
LIMIT
LIMITS
LINAGE
LINAGE-COUNTER
LINE
LINE-COUNTER
LINES
LINKAGE
LION
LOCATION
LOCK
LOW-VALUE
LOW-VALUES
LOWER-BOUND
LOWER-BOUNDS

MASM
MASS-STORAGE
MASS-STORAGE-28
MASS-STORAGE-56
MASS-STORAGE-112
MCFLAG
MEMORY
MERGE
MESSAGE
MESSAGES
MODE
MODULES
MONITOR
MOVE
MULTIPLE
MULTIPLY

NAMED
NATIVE
NEGATIVE
NEXT
NO
NOT
NOTE
NUMBER
NUMERIC

OBJECT-COMPUTER
OCCURS
OF
OFF
OMITTED
ON
OPEN
OPTIONAL
OR
ORGANIZATION
OTHERWISE
OUTPUT
OVERFLOW

PAGE
PAGE-COUNTER
PERFORM
PF
PH
PIC
PICTURE
PLACE
PLACES
PLUS

PL1
POINT
POINTER
POINTS
POSITION
POSITIVE
PRINTER
PRINTING
PROCEDURE
PROCEDURES
PROCEED
PROCESS
PROCESSING
PROGRAM
PROGRAM-ID
PROTECT
PURGE-DATE

QUEUE
QUOTE
QUOTES

RANDOM
RANK
RD
READ
RECEIVE
RECORD
RECORDING
RECORDS
REDEFINES
REEL
REFERENCES
RELATIVE
RELEASE
REMAINDER
REMARKS
REMOVAL
RENAMES
REPLACING
REPORT
REPORTING
REPORTS
RERUN
RESERVE
RESET
RETURN
REVERSED
REWIND
REWRITE
RF
RH
RIGHT

ROUNDED
RUN

SA
SAME
SD
SDF
SEARCH
SECTION
SECURITY
SEGMENT
SEGMENT-LIMIT
SELECT
SEND
SENTENCE
SEPARATE
SEQUENCE
SEQUENTIAL
SEQUENTIAL-FILE
SET
SET-ID
SIGN
SIZE
SORT
SORT-MERGE
SOURCE
SOURCE-ALPHABET
SOURCE-COMPUTER
SPACE
SPACES
SPECIAL-NAMES
SPERRY-SYSTEM-11
SPERRY-1100EIS
STANDARD
STANDARD-1
START
STATUS
STOP
STRING
SUB-QUEUE-1
SUB-QUEUE-2
SUB-QUEUE-3
SUBTRACT
SUM
SUPPRESS
SWITCH-1
SWITCH-2
SWITCH-3
SWITCH-4
SWITCH-5

SWITCH-6

SWITCH-7
SWITCH-8
SWITCH-9
SWITCH-10
SWITCH-11
SWITCH-12
SWITCH-13
SWITCH-14
SWITCH-15
SWITCH-16
SWITCH-17
SWITCH-18
SWITCH-19
SWITCH-20
SWITCH-21
SWITCH-22
SWITCH-23
SWITCH-24
SYMBOLIC
SYNC
SYNCHRONIZED

TABLE
TALLY
TALLYING
TAPE
TERMINAL
TERMINATE
TEXT
THAN
THROUGH
THRU
TIME
TIMES
TO
TOP
TRAILING
TRANSFORM
TYPE

UNEQUAL
UNISERVO
UNISERVOS
UNISYS-1100EIS
UNISYS-1100-90
UNISYS-2200
UNIT
UNIVAC-1110
UNIVAC-1100-40
UNIVAC-1100-60EIS
UNIVAC-1100-70EIS
UNIVAC-1100-80
UNIVAC-1100-90
UNLOCK
UNSTRING
UNTIL
UP
UPON
UPPER-BOUND
UPPER-BOUNDS
USAGE
USE
USING

VALUE
VALUES
VARYING

WHEN
WITH
WORDS
WORKING-STORAGE
WRITE
WRITE-ONLY

ZERO
ZEROES
ZEROS

+
-
*
/
**
>
<
=
770-PRINTER

Appendix B
Character Codes

Table B–1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order

ASCII			Character Name	Sym	Card Punches	EBCDIC*			Fieldata			
Octal	Hex	Dec				Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
0	0	0	NUL		12-0-9-8-1	0	0	0	No conversion			
1	1	1	SOH		12-9-1	1	1	1	No conversion			
2	2	2	STX		12-9-2	2	2	2	No conversion			
3	3	3	ETX		12-9-3	3	3	3	No conversion			
4	4	4	EOT		9-7	37	55	67	No conversion			
5	5	5	ENO		0-9-8-5	2D	45	55	No conversion			
6	6	6	ACK		0-9-8-6	2E	46	56	No conversion			
7	7	7	BEL		0-9-8-7	2F	47	57	No conversion			
10	8	8	BS		11-9-6	16	22	26	No conversion			
11	9	9	HT		12-9-5	05	5	5	No conversion			
12	A	10	LF		0-9-5	25	37	45	No conversion			
13	B	11	VT		12-9-8-3	0B	11	13	No conversion			
14	C	12	FF		12-9-8-4	0C	12	14	No conversion			
15	D	13	CR		12-9-8-5	0D	13	15	No conversion			
16	E	14	SO		12-9-8-6	0E	14	16	No conversion			
17	F	15	SI		12-9-8-7	0F	15	17	No conversion			
20	10	16	DLE		12-11-9-8-1	10	16	20	No conversion			
21	11	17	DC1		11-9-1	11	17	21	No conversion			
22	12	18	DC2		11-9-2	12	18	22	No conversion			
23	13	19	DC3		11-9-3	13	19	23	No conversion			
24	14	20	DC4		9-8-4	3C	60	74	No conversion			
25	15	21	NAK		9-8-5	3D	61	75	No conversion			
26	16	22	SYN		9-2	32	50	62	No conversion			
27	17	23	ETB		0-9-6	26	38	45	No conversion			
30	18	24	CAN		11-9-8	18	24	30	No conversion			
31	19	25	EM		11-9-8-1	19	25	31	No conversion			
32	1A	26	SUB		9-8-7	3F	63	77	No conversion			
33	1B	27	ESC		0-9-7	27	39	47	No conversion			
34	1C	28	FS		11-9-8-4	1C	28	34	No conversion			
35	1D	29	GS		11-9-8-5	1D	29	35	No conversion			

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B-1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order (continued)

ASCII			Character Name	Sym	Card Punches	EBCDIC*			Fieldata			
Octal	Hex	Dec				Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
36	1E	30	RS		11-9-8-6	1E	30	36	No conversion			
37	1F	31	US		11-9-8-7	1F	31	37	No conversion			
40	20	32	SP, Space			40	64	100		5	5	SP
41	21	33	Exclamation Point	!	12-8-7	4F	79	117		45	55	11-0
42	22	34	Quotation Mark Dieresis	"	8-7	7F	127	177	▭	62	76	0-7-8
43	23	35	Number Sign Pound Sign	#	8-3	7B	123	173		3	3	12-7-8
44	24	36	Dollar Sign	$	11-8-3	5B	91	133		39	47	11-3-8
45	25	37	Percent Sign	%	0-8-4	6C	108	154		42	52	0-5-8
46	26	38	Ampersand	&	12	50	80	120		38	46	2-8
47	27	39	Apostrophe Acute Accent	'	8-5	7D	125	175		58	72	4-8
50	28	40	Opening Parenthesis	(	12-8-5	4D	77	115		41	51	0-4-8
51	29	41	Closing Parenthesis	)	11-8-5	5D	93	135		32	40	12-4-8
52	2A	42	Asterisk	*	11-8-4	5C	92	134		40	50	11-4-8
53	2B	43	Plus Sign	+	12-8-6	4E	78	116		34	42	12
54	2C	44	Comma Cedilla	,	0-8-3	6B	107	153		46	56	0-3-8
55	2D	45	Minus Sign Hyphen	-	11	60	96	140		33	41	11
56	2E	46	Period Decimal Point	.	12-8-3	4B 61	75	113		61	75	12-3-12
57	2F	47	Slash Virguie Solidus	/	0-1	47	97	141		60	74	0-1
60	30	48	0, Zero	0	0	F0	240	360		48	60	0
61	31	49	1, One	1	1	F1	241	361		49	61	1
62	32	50	2, Two	2	2	F2	242	362		50	62	2
63	33	51	3, Three	3	3	F3	243	363		51	63	3
64	34	52	4, Four	4	4	F4	244	364		52	64	4
65	35	53	5, Five	5	5	F5	245	365		53	65	5
66	36	54	6, Six	6	6	F6	246	366		54	66	6
67	37	55	7, Seven	7	7	F7	247	367		55	67	7
70	38	56	8, Eight	8	8	F8	248	370		56	70	8
71	39	57	9, Nine	9	9	F9	249	371		57	71	9
72	3A	58	Colon	:	8-2	7A	122	172		43	53	5-8
73	3B	59	Semicolon	;	11-8-6	5E	94	136		59	73	11-6-8
74	3C	60	Less Than	<	12-8-4	4C	76	114		35	43	12-6-8
75	3D	61	Equal Sign	=	8-6	7E	126	176		36	44	3-8
76	3E	62	Greater Than	>	0-8-6	6E	110	156		37	45	6-8
77	3F	63	Question Mark	?	0-8-7	6F	111	157		44	54	12-0

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B-1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order (continued)

ASCII				Sym	Card Punches	EBCDIC*			Fieldata			
Octal	Hex	Dec	Character Name			Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
100	40	64	Commercial At Symbol	@ 1	8-4	7C	124	174		0	0	7-8
101	41	65	Capital A	A	12-1	C1	193	301		6	6	12-1
102	42	66	Capital B	B	12-2	C2	194	302		7	7	12-2
103	43	67	Capital C	C	12-3	C3	195	303		8	10	12-3
104	44	68	Capital D	D	12-4	C4	196	304		9	11	12-4
105	45	69	Capital E	E	12-5	C5	197	305		10	12	12-5
106	46	70	Capital F	F	12-6	C6	198	306		11	13	12-6
107	47	71	Capital G	G	12-7	C7	199	307		12	14	12-7
110	48	72	Capital H	H	12-8	C8	200	310		13	15	12-8
111	49	73	Capital I	I	12-9	C9	201	311		14	16	12-9
112	4A	74	Capital J	J	11-1	D1	209	321		15	17	11-1
113	4B	75	Capital K	K	11-2	D2	210	322		16	20	11-2
114	4C	76	Capital L	L	11-3	D3	211	323		17	21	11-3
115	4D	77	Capital M	M	11-4	D4	212	324		18	22	11-4
116	4E	78	Capital N	N	11-5	D5	213	325		19	23	11-5
117	4F	79	Capital O	O	11-6	D6	214	326		20	24	11-6
120	50	80	Capital P	P	11-7	D7	215	327		21	25	11-7
121	51	81	Capital Q	Q	11-8	D8	216	330		22	26	11-8
122	52	82	Capital R	R	11-9	D9	217	331		23	27	11-9
123	53	83	Capital S	S	0-2	E2	226	342		24	30	0-2
124	54	84	Capital T	T	0-3	E3	227	343		25	31	0-3
125	55	85	Capital U	U	0-4	E4	228	344		26	32	0-4
126	56	86	Capital V	V	0-5	E5	229	345		27	33	0-5
127	57	87	Capital W	W	0-6	E6	230	346		28	34	0-6
130	58	88	Capital X	X	0-7	E7	231	347		29	35	0-7
131	59	89	Capital Y	Y	0-8	E8	232	350		30	36	0-8
132	5A	90	Capital Z	Z	0-9	E9	233	351		31	37	0-9
133	5B	91	Opening Bracket	[1	12-8-2	4A	74	112		1	1	12-5-8
134	5C	92	Reverse Slash	\	0-8-2	EO	224	340		47	57	0-6-8
135	5D	93	Closing Bracket	]	11-8-2	5A	90	132		2	2	11-5-8
136	5E	94	Circumfiex	^	11-8-7	5F	95	137	n	4	4	11-7-8
137	5F	95	Underline		0-8-5	6D	109	155	□	63	77	0-2-8
140	60	96	Grave Accent	‾	8-1	79	121	171	@	0	0	7-8
141	61	97	Small A	a	12-0-1	81	129	201	A	5	6	12-1
142	62	98	Small B	b	12-0-2	82	130	202	B	7	7	12-2
143	63	99	Small C	c	12-0-3	83	131	203	C	8	10	12-3
144	64	100	Small D	d	12-0-4	84	132	204	D	9	11	12-4
145	65	101	Small E	e	12-0-5	85	133	205	E	10	12	12-5
146	66	102	Small F	f	12-0-6	86	134	206	F	11	13	12-6
147	67	103	Small G	g	12-0-7	87	135	207	G	12	14	12-7
150	68	104	Small H	h	12-0-8	88	136	210	H	13	15	12-8
151	69	105	Small I	i	12-0-9	89	137	211	I	14	16	12-9
152	6A	106	Small J	j	12-11-1	91	145	221	J	15	17	11-1
153	6B	107	Small K	k	12-11-2	92	146	222	K	16	20	11-2
154	6C	108	Small L	l	12-11-3	93	147	223	L	17	21	11-3
155	6D	109	Small M	m	12-11-4	94	148	224	M	18	22	11-4

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B-1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order (continued)

ASCII				Sym	Card Punches	EBCDIC*				Fieldata		
Octal	Hex	Dec	Character Name			Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
156	6E	110	Small N	n	12-11-5	95	149	225	N	19	23	11-5
157	6F	111	Small O	o	12-11-6	96	150	226	O	20	24	11-5
160	70	112	Small P	p	12-11-7	97	151	227	P	21	25	11-7
161	71	113	Small Q	q	12-11-8	98	152	230	Q	22	26	11-8
162	72	114	Small R	r	12-11-9	99	153	231	R	23	27	11-9
163	73	115	Small S	s	11-0-2	A2	162	242	S	24	30	0-2
164	74	116	Small T	t	11-0-3	A3	163	243	T	25	31	0-3
165	75	117	Small U	u	11-0-4	A4	164	244	U	26	32	0-4
166	76	118	Small V	v	11-0-5	A5	165	245	V	27	33	0-5
167	77	119	Small W	w	11-0-6	A6	166	246	W	28	34	0-6
170	78	120	Small X	x	11-0-7	A7	167	247	X	29	35	0-7
171	79	121	Small Y	y	11-0-8	A8	168	250	Y	30	36	0-8
172	7A	122	Small Z	z	11-0-9	A9	169	251	Z	31	37	0-9
173	7B	123	Opening Brace	{	12-0	C0	192	300	>	44	54	12-0
174	7C	124	Vertical Line	|	12-11	6A	106	152	?	47	57	0-6-8
175	7D	125	Closing Brace	}	11-0	D0	208	320	,	45	55	11-0
176	7E	126	Overline, Tilde	~	11-0-1	A1	161	241	h	4	4	11-7-8
177	7F	127	DEL, Delete		12-9-7	07	7	7	□	63	77	0-2-8
200	80	128			11-0-9-8-1	20	32	40		No conversion		
201	81	129			0-9-1	21	33	41		No conversion		
202	82	130			0-9-2	22	34	42		No conversion		
203	83	131			0-9-3	23	35	43		No conversion		
204	84	132			0-9-4	24	36	44		No conversion		
205	85	133			11-9-5	15	21	25		No conversion		
206	86	134			12-9-6	06	6	6		No conversion		
207	87	135			11-9-7	17	23	27		No conversion		
210	88	136			0-9-8	28	40	50		No conversion		
211	89	137			0-9-8-1	29	41	51		No conversion		
212	8A	138			0-9-8-2	2A	42	52		No conversion		
213	8B	139			0-9-8-3	2B	43	53		No conversion		
214	8C	140			0-9-8-4	2C	44	54		No conversion		
215	8D	141			12-9-8-1	09	9	11		No conversion		
216	8E	142			12-9-8-2	0A	10	12		No conversion		
217	8F	143			11-9-8-3	1B	27	33		No conversion		
220	90	144			12-11-0-9-8-1	30	48	60		No conversion		
221	91	145			9-1	31	49	61		No conversion		
222	92	146			11-9-8-2	1A	26	32		No conversion		
223	93	147			9-3	33	51	63		No conversion		
224	94	148			9-4	34	52	64		No conversion		
225	95	149			9-5	35	53	65		No conversion		
226	96	150			9-6	36	54	66		No conversion		
227	97	151			12-9-8	08	8	10		No conversion		
230	98	152			9-8	38	56	70		No conversion		
231	99	153			9-8-1	39	57	71		No conversion		
232	9A	154			9-8-2	3A	58	72		No conversion		
233	9B	155			9-8-3	3B	59	73		No conversion		
234	9C	156			12-9-4	04	4	4		No conversion		
235	9D	157			11-9-4	14	20	24		No conversion		

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B-1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order (continued)

ASCII				Sym	Card Punches	EBCDIC*			Fieldata			
Octal	Hex	Dec	Character Name			Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
236	9E	158			9-8-6	3E	62	76				No conversion
237	9F	159			11-0-9-1	E1	225	341				No conversion
240	A0	160			12-0-9-1	41	65	101				No conversion
241	A1	161			12-0-9-2	42	66	102				No conversion
242	A2	162			12-0-9-3	43	67	103				No conversion
243	A3	163			12-0-9-4	44	68	104				No conversion
244	A4	164			12-0-9-5	45	69	105				No conversion
245	A5	165			12-0-9-6	46	70	106				No conversion
246	A6	166			12-0-9-7	47	71	107				No conversion
247	A7	167			12-0-9-8	48	72	110				No conversion
250	A8	168			12-8-1	49	73	111				No conversion
251	A9	169			12-11-9-1	51	81	121				No conversion
252	AA	170			12-11-9-2	52	82	122				No conversion
253	AB	171			12-11-9-3	53	83	123				No conversion
254	AC	172			12-11-9-4	54	84	124				No conversion
255	AD	173			12-11-9-5	55	85	125				No conversion
256	AE	174			12-11-9-6	56	86	126				No conversion
257	AF	175			12-11-9-7	57	87	127				No conversion
260	B0	176			12-11-9-8	58	88	130				No conversion
261	B1	177			11-8-1	59	89	131				No conversion
262	B2	178			11-0-9-2	62	98	142				No conversion
263	B3	179			11-0-9-3	63	99	143				No conversion
264	B4	180			11-0-9-4	64	100	144				No conversion
265	B5	181			11-0-9-5	65	101	145				No conversion
266	B6	182			11-0-9-6	66	102	146				No conversion
267	B7	183			11-0-9-7	67	103	147				No conversion
270	B8	184			11-0-9-8	68	104	150				No conversion
271	B9	185			0-8-1	69	105	151				No conversion
272	BA	186			12-11-0	70	112	160				No conversion
273	BB	187			12-11-0-9-1	71	113	161				No conversion
274	BC	188			12-11-0-9-2	72	114	162				No conversion
275	BD	189			12-11-0-9-3	73	115	163				No conversion
276	BE	190			12-11-0-9-4	74	116	164				No conversion
277	BF	191			12-11-0-9-5	75	117	165				No conversion
300	C0	192			12-11-0-9-6	76	118	166				No conversion
301	C1	193			12-11-0-9-7	77	119	167				No conversion
302	C2	194			12-11-0-9-8	78	120	170				No conversion
303	C3	195			12-0-8-1	80	128	200				No conversion
304	C4	196			12-0-8-2	8A	138	212				No conversion
305	C5	197			12-0-8-3	8B	139	213				No conversion
306	C6	198			12-0-8-4	8C	140	214				No conversion
307	C7	199			12-0-8-5	8D	141	215				No conversion
310	C8	200			12-0-8-6	8E	142	216				No conversion
311	C9	201			12-0-8-7	8F	143	217				No conversion
312	CA	202			12-11-8-1	90	144	220				No conversion
313	CB	203			12-11-8-2	9A	154	232				No conversion
314	CC	204			12-11-8-3	9B	155	233				No conversion
315	CD	205			12-11-8-4	9C	156	234				No conversion
316	CE	206			12-11-8-5	9D	157	235				No conversion

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B-1. Correspondence of ASCII, EBCDIC, and Fieldata, in ASCII Order (continued)

ASCII				Sym	Card Punches	EBCDIC*			Fieldata			
Octal	Hex	Dec	Character Name			Hex	Dec	Octal	SYM **	Dec	Octal	Card Punches
317	CF	207			12-22-8-6	9E	158	236	No conversion			
320	D0	208			12-11-8-7	9F	159	237	No conversion			
321	D1	209			11-0-8-1	A0	160	240	No conversion			
322	D2	210			11-0-8-2	AA	170	252	No conversion			
323	D3	211			11-0-8-3	AB	171	253	No conversion			
324	D4	212			11-0-8-4	AC	172	254	No conversion			
325	D5	213			11-0-8-5	AD	173	255	No conversion			
326	D6	214			11-0-8-6	AE	174	256	No conversion			
327	D7	215			11-0-8-7	AF	175	257	No conversion			
330	D8	216			12-11-0-8-1	B0	176	260	No conversion			
331	D9	217			12-11-0-1	B1	177	261	No conversion			
332	DA	218			12-11-0-2	B2	178	262	No conversion			
333	DB	219			12-11-0-3	B3	179	263	No conversion			
334	DC	220			12-11-0-4	B4	180	264	No conversion			
335	DD	221			12-11-0-5	B5	181	265	No conversion			
336	DE	222			12-11-0-6	B6	182	266	No conversion			
337	DF	223			12-11-0-7	B7	183	267	No conversion			
340	E0	224			12-11-0-8	B8	184	270	No conversion			
341	E1	225			12-11-0-9	B9	185	271	No conversion			
342	E2	226			12-11-0-8-2	BA	186	272	No conversion			
343	E3	227			12-11-0-8-3	BB	187	273	No conversion			
344	E4	228			12-11-0-8-4	BC	188	274	No conversion			
345	E5	229			12-11-0-8-5	BD	189	275	No conversion			
346	E6	230			12-11-0-8-6	BE	190	276	No conversion			
347	E7	231			12-11-0-8-7	BF	191	277	No conversion			
350	E8	232			12-0-9-8-2	CA	202	312	No conversion			
351	E9	233			12-0-9-8-3	CB	203	313	No conversion			
352	EA	234			12-0-9-8-4	CC	204	314	No conversion			
353	EB	235			12-0-9-8-5	CD	205	315	No conversion			
354	EC	236			12-0-9-8-6	CE	206	316	No conversion			
355	ED	237			12-0-9-8-7	CF	207	317	No conversion			
356	EE	238			12-11-9-8-2	DA	218	332	No conversion			
357	EF	239			12-11-9-8-3	DB	219	333	No conversion			
360	F0	240			12-11-9-8-4	DC	220	334	No conversion			
361	F1	241			12-11-9-8-5	DD	221	335	No conversion			
362	F2	242			12-11-9-8-6	DE	222	336	No conversion			
363	F3	243			12-11-9-8-7	DF	223	337	No conversion			
364	F4	244			11-0-9-8-2	EA	234	352	No conversion			
365	F5	245			11-0-9-8-3	EB	235	353	No conversion			
366	F6	246			11-0-9-8-4	EC	236	354	No conversion			
367	F7	247			11-0-9-8-5	ED	237	355	No conversion			
370	F8	248			11-0-9-8-6	EE	238	356	No conversion			
371	F9	249			11-0-9-8-7	EF	239	357	No conversion			
372	FA	250			12-11-0-9-8-2	FA	250	372	No conversion			
373	FB	251			12-11-0-9-8-3	FB	251	373	No conversion			
374	FC	252			12-11-0-9-8-4	FC	252	374	No conversion			
375	FD	253			12-11-0-9-8-5	FD	253	375	No conversion			
376	FE	254			12-11-0-9-8-6	FE	254	376	No conversion			
377	FF	255	EO. Eight Ones		12-11-0-9-8-7	FF	255	377	No conversion			

* When 128-character ASCII or EBCDIC translation is in effect, all ASCII characters in the range 128 through 255 decimal will be converted to 0. See 20.1.1.7 for selecting 256-character ASCII/EBCDIC translate.

** Fieldata symbols are listed only when different from ASCII symbols.

Table B–2. Correspondence Between ASCII and EBCDIC in EBCDIC Order

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
00	0	0	000	0	NUL	12-0-9-8-1
01	1	1	001	1	SOH	12-9-1
02	2	2	002	2	STX	12-9-2
03	3	3	003	3	ETX	12-9-3
04	4	4	(234)	156	K28	12-9-4
05	5	5	011	9	HT	12-9-5
06	6	6	(206)	134	K6	12-9-6
07	7	7	177	127	DEL	12-9-7
08	10	8	(227)	151	K23	12-9-8
09	11	9	(215)	141	K13	12-9-8-1
0A	12	10	(216)	142	K14	12-9-8-2
0B	13	11	013	11	VT	12-9-8-3
0C	14	12	014	12	FF	12-9-8-4
0D	15	13	015	13	CR	12-9-8-5
0E	16	14	016	14	SO	12-9-8-6
0F	17	15	017	15	SI	12-9-8-7
10	20	16	020	16	DLE	12-11-9-8-1
11	21	17	021	17	DC1	11-9-1
12	22	18	022	18	DC2	11-9-2
13	23	19	023	19	DC3	11-9-3
14	24	20	(235)	157	K29	11-9-4
15	25	21	(205)	133	K5	11-9-5
16	26	22	010	8	BS	11-9-6
17	27	23	(207)	135	K7	11-9-7
18	30	24	030	24	CAN	11-9-8
19	31	25	031	25	EM	11-9-8-1
1A	32	26	(222)	146	K18	11-9-8-2
1B	33	27	(217)	143	K15	11-9-8-3
1C	34	28	034	28	FS	11-9-8-4
1D	35	29	035	29	GS	11-9-8-5
1E	36	30	036	30	RS	11-9-8-6
1F	37	31	037	31	US	11-9-8-7
20	40	32	(200)	128	K0	11-0-9-8-1
21	41	33	(201)	129	K1	0-9-1
22	42	34	(202)	130	K2	0-9-2
23	43	35	(203)	131	K3	0-9-3
24	44	36	(204)	132	K4	0-9-4
25	45	37	012	10	LF	0-9-5
26	46	38	027	23	ETB	0-9-6
27	47	39	033	27	ESC	0-9-7
28	50	40	(210)	136	K8	0-9-8
29	51	41	(211)	137	K9	0-9-8-1
2A	52	42	(212)	138	K10	0-9-8-2
2B	53	43	(213)	139	K11	0-9-8-3
2C	54	44	(214)	140	K12	0-9-8-4
2D	55	45	005	5	ENQ	0-9-8-5
2E	56	46	006	6	ACK	0-9-8-6
2F	57	47	007	7	BEL	0-9-8-7
30	60	48	(220)	144	K16	12-11-0-8-1
31	61	49	(221)	145	K17	9-1

Table B-2. Correspondence Between ASCII and EBCDIC in EBCDIC Order (continued)

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
32	62	50	026	22	SYN	9-2
33	63	51	(223)	147	K19	9-3
34	64	52	(224)	148	K20	9-4
35	65	53	(225)	149	K21	9-5
36	66	54	(226)	150	K22	9-6
37	67	55	004	4	EOT	9-7
38	70	56	(230)	152	K24	9-8
39	71	57	(231)	153	K25	9-8-1
3A	72	58	(232)	154	K26	9-8-2
3B	73	59	(233)	155	K27	9-8-3
3C	74	60	024	20	DC4	9-8-4
3D	75	61	025	21	NAK	9-8-5
3E	76	62	(236)	158	K30	9-8-6
3F	77	63	032	26	SUB	9-8-7
40	100	64	040	32	SP	No punches
41	101	65	(240)	160	N0	12-0-9-1
42	102	66	(241)	161	N1	12-0-9-2
43	103	67	(242)	162	N2	12-0-9-3
44	104	68	(243)	163	N3	12-0-9-4
45	105	69	(244)	164	N4	12-0-9-5
46	106	70	(245)	165	N5	12-0-9-6
47	107	71	(246)	166	N6	12-0-9-7
48	110	72	(247)	167	N7	12-0-9-8
49	111	73	(250)	168	N8	12-8-1
4A	112	74	133	91	[	12-8-2
4B	113	75	056	46	.	12-8-3
4C	114	76	074	60	<	12-8-4
4D	115	77	050	40	(	12-8-5
4E	116	78	053	43	+	12-8-6
4F	117	79	041	33	!	12-8-7
50	120	80	046	38	&	12
51	121	81	(251)	169	N9	12-11-9-1
52	122	82	(252)	170	N10	12-11-9-2
53	123	83	(253)	171	N11	12-11-9-3
54	124	84	(254)	172	N12	12-11-9-4
55	125	85	(255)	173	N13	12-11-9-5
56	126	86	(256)	174	N14	12-11-9-6
57	127	87	(257)	175	N15	12-11-9-7
58	130	88	(260)	176	N16	12-11-9-8
59	131	89	(261)	177	N17	11-8-1
5A	132	90	135	93	]	11-8-2
5B	133	91	044	36	$	11-8-3
5C	134	92	052	42	*	11-8-4
5D	135	93	051	41	)	11-8-5
5E	136	94	073	59	;	11-8-6
5F	137	95	136	94	_	11-8-7
60	140	96	055	45	_	11
61	141	97	057	47	/	0-1
62	142	98	(262)	178	N18	11-0-9-2
63	143	99	(263)	179	N19	11-0-9-3

Table B-2. Correspondence Between ASCII and EBCDIC in EBCDIC Order (continued)

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
64	144	100	(264)	180	N20	11-0-9-4
65	145	101	(265)	181	N21	11-0-9-5
66	146	102	(266)	182	N22	11-0-9-6
67	147	103	(267)	183	N23	11-0-9-7
68	150	104	(270)	184	N24	11-0-9-8
69	151	105	(271)	185	N25	0-8-1
6A	152	106	174	124	¦	12-11
6B	153	107	054	44	,	0-8-3
6C	154	108	045	37	%	0-8-4
6D	155	109	137	95	-	0-8-5
6E	156	110	076	62	>	0-8-6
6F	157	111	077	63	?	0-8-7
70	160	112	(272)	186	N26	12-11-0
71	161	113	(273)	187	N27	12-11-0-9-1
72	162	114	(274)	188	N28	12-11-0-9-2
73	163	115	(275)	189	N29	12-11-0-9-3
74	164	116	(276)	190	N30	12-11-0-9-4
75	165	117	(277)	191	N31	12-11-0-9-5
76	166	118	(300)	192	N32	12-11-0-9-6
77	167	119	(301)	193	N33	12-11-0-9-7
78	170	120	(302)	194	N34	12-11-0-9-8
79	171	121	140)	96	\	8-1
7A	172	122	072	58	:	8-2
7B	173	123	043	35	#	8-3
7C	174	124	100	64	@	8-4
7D	175	125	047	39	'	8-5
7E	176	126	075	61	=	8-6
7F	177	127	042	34	"	8-7
80	200	128	(303)	195	N35	12-0-8-1
81	201	129	141	97	a	12-0-1
82	202	130	142	98	b	12-0-2
83	203	131	143	99	c	12-0-3
84	204	132	144	100	d	12-0-4
85	205	133	145	101	e	12-0-5
86	206	134	146	102	f	12-0-6
87	207	135	147	103	g	12-0-7
88	210	136	150	104	h	12-0-8
89	211	137	151	105	i	12-0-9
8A	212	138	(304)	196	N36	12-0-8-2
8B	213	139	(305)	197	N37	12-0-8-3
8C	214	140	(306)	198	N38	12-0-8-4
8D	215	141	(307)	199	N39	12-0-8-5
8E	216	142	(310)	200	N40	12-0-8-6
8F	217	143	(311)	201	N41	12-0-8-7
90	220	144	(312)	202	N42	12-11-8-1
91	221	145	152	106	j	12-11-1
92	222	146	153	107	k	12-11-2
93	223	147	154	108	l	12-11-3
94	224	148	155	109	m	12-11-4
95	225	149	156	110	n	12-11-5

Table B-2. Correspondence Between ASCII and EBCDIC in EBCDIC Order (continued)

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
96	226	150	157	111	o	12-11-6
97	227	151	160	112	p	12-11-7
98	230	152	161	113	q	12-11-8
99	231	153	162	114	r	12-11-9
9A	232	154	(313)	203	N43	12-11-8-2
9B	233	155	(314)	204	N44	12-11-8-3
9C	234	156	(315)	205	N45	12-11-8-4
9D	235	157	(316)	206	N46	12-11-8-5
9E	236	158	(317)	207	N47	12-11-8-6
9F	237	159	(320)	208	N48	12-11-8-7
A0	240	160	(321)	209	N49	11-0-8-1
A1	241	161	176	126	~	11-0-1
A2	242	162	163	115	s	11-0-2
A3	243	163	164	116	t	11-0-3
A4	244	164	165	117	u	11-0-4
A5	245	165	166	118	v	11-0-5
A6	246	166	167	119	w	11-0-6
A7	247	167	170	120	x	11-0-7
A8	250	168	171	121	y	11-0-8
A9	251	169	172	122	z	11-0-9
AA	252	170	(322)	210	N50	11-0-8-2
AB	253	171	(323)	211	N51	11-0-8-3
AC	254	172	(324)	212	N52	11-0-8-4
AD	255	173	(325)	213	N53	11-0-8-5
AE	256	174	(326)	214	N54	11-0-8-6
AF	257	175	(327)	215	N55	11-0-8-7
B0	260	176	(330)	216	N56	12-11-0-8-1
B1	261	177	(331)	217	N57	12-11-0-1
B2	262	178	(332)	218	N58	12-11-0-2
B3	263	179	(333)	219	N59	12-11-0-3
B4	264	180	(334)	220	N60	12-11-0-4
B5	265	181	(335)	221	N61	12-11-0-5
B6	266	182	(336)	222	N62	12-11-0-6
B7	267	183	(337)	223	N63	12-11-0-7
B8	270	184	(340)	224	G0	12-11-0-8
B9	271	185	(341)	225	G1	12-11-0-9
BA	272	186	(342)	226	G2	12-11-0-8-2
BB	273	187	(343)	227	G3	12-11-0-8-3
BC	274	188	(344)	228	G4	12-11-0-8-4
BD	275	189	(345)	229	G5	12-11-0-8-5
BE	276	190	(346)	230	G6	12-11-0-8-6
BF	277	191	(347)	231	G7	12-11-0-8-7
C0	300	192	173	123	I	12-0
C1	301	193	101	65	A	12-1
C2	302	194	102	66	B	12-2
C3	303	195	103	67	C	12-3
C4	304	196	104	68	D	12-4
C5	305	197	105	69	E	12-5
C6	306	198	106	70	F	12-6
C7	307	199	107	71	G	12-7

Table B-2. Correspondence Between ASCII and EBCDIC in EBCDIC Order (continued)

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
C8	310	200	110	72	H	12-8
C9	311	201	111	73	I	12-9
CA	312	202	(350)	232	G8	12-0-9-8-2
CB	313	203	(351)	233	G9	12-0-9-8-3
CC	314	204	(352)	234	G10	12-0-9-8-4
CD	315	205	(353)	235	G11	12-0-9-8-5
CE	316	206	(354)	236	G12	12-0-9-8-6
CF	317	207	(355)	237	G13	12-0-9-8-7
D0	320	208	175	125	I	11-0
D1	321	209	112	74	J	11-1
D2	322	210	113	75	K	11-2
D3	323	211	114	76	L	11-3
D4	324	212	115	77	M	11-4
D5	325	213	116	78	N	11-5
D6	326	214	117	79	O	11-6
D7	327	215	120	80	P	11-7
D8	330	216	121	81	Q	11-8
D9	331	217	122	82	R	11-9
DA	332	218	(356)	238	G14	12-11-9-8-2
DB	333	219	(357)	239	G15	12-11-9-8-3
DC	334	220	(360)	240	G16	12-11-9-8-4
DD	335	221	(361)	241	G17	12-11-9-8-5
DE	336	222	(362)	242	G18	12-11-9-8-6
DF	337	223	(363)	243	G19	12-11-9-8-7
EO	340	224	134	92	\	0-8-2
E1	341	225	(237)	159	K31	11-0-9-1
E2	342	226	123	83	S	0-2
E3	343	227	124	84	T	0-3
E4	344	228	125	85	U	0-4
E5	345	229	126	86	V	0-5
E6	346	230	127	87	W	0-6
E7	347	231	130	88	X	0-7
E8	350	232	131	89	Y	0-8
E9	351	233	132	90	Z	0-9
EA	352	234	(364)	244	G20	11-0-9-8-2
EB	353	235	(365)	245	G21	11-9-8-8-3
EC	354	236	(366)	246	G22	11-0-9-8-4
ED	355	237	(367)	247	G23	11-0-9-8-5
EE	356	238	(370)	248	G24	11-0-9-8-6
EF	357	239	(371)	249	G25	11-0-9-8-7
F0	360	240	060	48	0	0
F1	361	241	061	49	1	1
F2	362	242	062	50	2	2
F3	363	243	063	51	3	3
F4	364	244	064	52	4	4
F5	365	245	065	53	5	5
F6	366	246	066	54	6	6
F7	367	247	067	55	7	7
F8	370	248	070	56	8	8
F9	371	249	071	57	9	9

Table B-2. Correspondence Between ASCII and EBCDIC in EBCDIC Order (continued)

EBCDIC			ASCII			
Hex	Octal	Dec	Octal	Dec	Graphic	Card Punches Codes
FA	372	250	(372)	250	G26	12-11-0-9-8-2
FB	373	251	(373)	251	G27	12-11-0-9-8-3
FC	374	252	(374)	252	G28	12-11-0-9-8-4
FD	375	253	(375)	253	G29	12-11-0-9-8-5
FE	376	254	(376)	254	G30	12-11-0-9-8-6
FF	377	255	(377)	255	E0	12-11-0-9-8-7

The following Fieldata and ASCII conversion tables are in conformance with USA Standard X3.4 — 1967 and 1968.

Fieldata			ASCII			Fieldata			ASCII			Fieldata			ASCII		
DEC	OCT	SYM	DEC	OCT	SYM*	DEC	OCT	SYM	DEC	OCT	SYM*	DEC	OCT	SYM	DEC	OCT	SYM*
0	00	@	64	100		22	26	Q	81	121		43	53	:	58	72	
1	01	[	91	133		23	27	R	82	122		44	54	?	63	77	
2	02	]	93	135		24	30	S	83	123		45	55		33	41	
3	03	#	35	43		25	31	T	84	124		46	56	,	44	54	
4	04	△	94	136	^	26	32	U	85	125		47	57	\	92	134	
5	05	SP	32	40		27	33	V	86	126		48	60	0	48	60	
6	06	A	65	101		28	34	W	87	127		49	61	1	49	61	
7	07	B	66	102		29	35	X	88	130		50	62	2	50	62	
8	10	C	67	103		30	36	Y	89	131		51	63	3	51	63	
9	11	D	68	104		31	37	Z	90	132		52	64	4	52	64	
10	12	E	69	105		32	40	)	41	51		53	65	5	53	65	
11	13	F	70	106		33	41	-	45	55		54	66	6	54	66	
12	14	G	71	107		34	42	+	43	53		55	67	7	55	67	
13	15	H	72	110		35	43	<	60	74		56	70	8	56	70	
14	16	I	73	111		36	44	=	61	75		57	71	9	57	71	
15	17	J	74	112		37	45	>	62	76		58	72	.	39	47	
16	20	K	75	113		38	46	&	38	46		59	73	;	59	73	
17	21	L	76	114		39	47	$	36	44		60	74	/	47	57	
18	22	M	77	115		40	50	*	42	52		61	75	.	46	56	
19	23	N	78	116		41	51	(	40	50		62	76	□	34	42	..
20	24	O	79	117		42	52	%	37	45		63	77	≠	95	137	-
21	25	P	80	120													

* All Symbols are identiacl except the three identified in the ASCII SYM column.

Overpunch Sign Conversions

The overpunched sign condition is represented (for +0 and -0) as follows:

Overpunch Sign Condition	Fieldata	ASCII	EBCDIC
+0	54	173 077	300
-0	55	175 041	320

Translation from Fieldata to ASCII is clear from the preceding conversion tables. Fieldata 54 and 55 are translated to ASCII 77 and 41, respectively.

Translation from EBCDIC to ISO8859-1 is given in the preceding conversion tables. The translation from ISO8859-1 to EBCDIC depends on the ASCII COBOL definition of the data item.

	ASCII	EBCDIC			
	(OCTAL)	(OCTAL)	(HEX)		
+0	077	300	C0	if	PIC S9
	173	300	C0	if	PIC S9
	077	360	F0	if	PIC 9
	173	360	F0	if	PIC 9
	077	156	6F	if	PIC X
-0	041	320	D0	if	PIC S9
	175	320	D0	if	PIC S9
	041	300	F0	if	PIC 9
	175	300	F0	if	PIC 9
	041	117	4F	if	PIC X

Overpunch Sign Condition	ISO8859-1	EBCDIC	EBCDIC	EBCDIC	EBCDIC	EBCDIC	EBCDIC
	(Octal)	(Octal)	(Octal)	(Octal)	(Hex)	(Hex)	(Hex)
		if PIC S9	if PIC 9	if PIC X	if PIC S9	if PIC 9	if PIC X
+0	077 173	300	360	156	C0	F0	6F
-0	041 175	320	300	117	D0	F0	4F

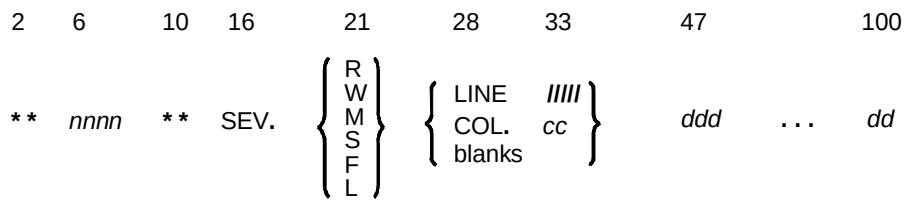
Appendix C
Diagnostic Messages

C.1. Compiler Diagnostics

This section describes the diagnostics produced during compilation. These diagnostics appear in the compilation listings in the format illustrated and explained as follows.

Column:

Column:



where:

nnnn

The 4-digit diagnostic number.

R, W, M, S, F, L

Severity codes which have the following meanings:

R (Remark)

Explains action taken by compiler but does not indicate an error in the source program.

W (Warning)

Denotes source code that deviates from the specification and may therefore produce unacceptable results. The diagnostic is merely precautionary. Compilation continues and code is generated.

M (Minor)

Indicates an instance of incorrect usage or omission. The compiler makes a reasonable assumption of the user's intent and continues, but further errors may result if compiler assumptions were inappropriate.

S (Serious)

Indicates a serious source error for which the compiler is unable to compensate or make a reasonable assumption of the user's intent. Either no code at all or incomplete code is generated for the statement in error.

F (Fatal)

Either a serious source program error or deficiency has been encountered. No code will be generated unless the A option is used on the @ACOB processor call statement.

L (Leveling)

Indicates a source code level violation error. The H, I, L, and R options in the Extra Options field of the @ACOB processor call statement determine the level of Standard COBOL at which leveling checks are performed. (See 18.4.)

/////

The number of the source line where the error was detected.

cc

The column number of the COBOL syntactical unit in error.

ddd...dd

The text of the diagnostic message.

In general, diagnostic messages are printed immediately after the source line image containing the error. Some fatal diagnostics cannot be associated with specific lines or columns in the source. In this case, columns 28 through 37 will be blank.

The following table (based upon diagnostic number ranges) indicates when the diagnostic is detected and during which listing the diagnostic will be issued.

Diagnostic Number	Condition Detected	Diagnostic Issued
1-100	While the source program is read.	Throughout the Source listing.
101-400	While processing the IDENTIFICATION, ENVIRONMENT, or DATA DIVISIONS (except the REPORT SECTION).	During the Source Listing of the IDENTIFICATION, ENVIRONMENT or DATA DIVISIONS.
401-600	While processing the REPORT SECTION.	During the Source Listing of the REPORT SECTION.
601-900	While processing the PROCEDURE DIVISION	During the Source Listing of the PROCEDURE DIVISION.
901-1000	During the code generation phase. Not detected or issued if 5 extra option specified.	After the Source Listing or during the Object Listing.
1001-1100	Throughout the compilation.	At any point during the compilation.

The E and F options from the control statement options field and the leveling options from the Extra Options field (L, L and I, H and I, H, or R) control the printing of the diagnostics.

Table C-1 shows the severity of the diagnostics printed for all combinations of E and F, and for the leveling options considered as a single category.

Table C-1. Severity of Printed Diagnostics

Leveling Options *	E Options Set	E and F Options Set	F Options Set	E and F OptionsNot Set	Leveling Diagnostic Total
Set	R, W, M, S, F, L**	M, S, F	M, S, F	W, M, S, F, L	Equal to number of leveling diagnostics issued
Not Set	R, W, M, S, F	M, S, F	M, S, F	S, F	Always equal to zero

* The leveling options are L, L and I, H and I, H, or R.

** R, W, M, S, F, and L are the severity codes described at the beginning of this section.

The diagnostic totals for the W, M, S and F diagnostics are always given, even though the diagnostics may not be printed.

The following is a list, in numeric order, of the diagnostic message text, its severity, and an explanation of the error. Diagnostics that refer to specific COBOL syntactical units fill in the units in parentheses for clarification.

001 F IDENTIFICATION DIVISION HEADER IS MISSING

The first word of the source program is not IDENTIFICATION; possibly a misspelled control statement was treated as an initial image. Compilation continues as if an Identification Division header had been processed.

002 W PROGRAM–ID HEADER IS MISSING

The second line of source text does not have the paragraph header PROGRAM–ID. Compilation continues as if a paragraph header had been present.

003 W PERIOD AFTER PREVIOUS LINE IS MISSING

The line should have a period after the header or at the end of the line. The period is assumed present.

004 W ILLEGAL/UNSEQUENCED PARAGRAPH HEADER IN ID–DIVISION

The paragraph header is not allowed or it is out of sequence in the Identification Division. Permissible headers are PROGRAM–ID, AUTHOR, INSTALLATION, DATE–WRITTEN, DATE–COMPILED, SECURITY, and REMARKS. Processing continues, ignoring the inadmissible Identification Division paragraph header.

005 M TEXT–NAME IS NOT FOLLOWED BY (OF, IN, OR REPLACING)

In a COPY statement, the only allowable words after the *text-name* are OF, IN or REPLACING. If the *text-name* is qualified by *library-name*, the only allowable word after *library-name* is REPLACING. When the E option in the Extra Options field is used, the only allowable word after *text-name* is REPLACING. If the word is the first word on a source line, it is treated as if it were the beginning of the statement which follows the COPY statement. Otherwise, the word and any remaining words on the source line are ignored.

006 S SUBJECT–WORD NOT FOLLOWED BY (BY)

The REPLACING phrase of a COPY statement should contain a sequence of phrases in the form subject-word BY object-word. If the word BY is not found after the subject-word, the compiler diagnoses the absence and ignores the REPLACING phrase of the COPY.

007 S LIBRARY HEADER NOT EQUAL TO COPY STATEMENT HEADER

The first line of the library element in American National Standard 1968 COBOL libraries (signified by the E option appearing in the Extra Options field of the processor call statement) has a header (*procedure-name* or level indicator) which does not match the header preceding the COPY statement. The library element header may be omitted in American National Standard 1968 COBOL libraries (except for Procedure Division sections). If headers exist and do not match, the compilation will terminate the COPY.

008 M ILLEGAL DUPLICATE PARAGRAPH/SECTION NAME

Either the same *section-name* has been defined twice in the Procedure Division or the same paragraph name has been defined twice within one section. The duplicate definition is accepted, but may cause erroneous execution if referenced.

009 W DECIMAL POINT SHOULD BE COMMA

The DECIMAL–POINT IS COMMA clause appeared in the Environment Division but a numeric literal has a period-decimal marker instead of a comma-decimal. The period was treated as if a comma had been used.

010 W ILLEGAL HYPHEN IN COLUMN 7

The next line is a *comment-entry* in the Identification Division or the current line ends with a punctuation mark or a literal–delimiting quote, followed by a blank. In either case the continuation hyphen is unnecessary and therefore ignored. If within a COPY proc, the previous line contains a hyphen although the line prior to it ended with a separator period. The hyphen is illegal in this case, and is ignored.

011 M (') MISSING; IT IS ASSUMED TO BE AFTER COL. 72

The last word on the line is an alphanumeric literal with no ending delimiter, and the next line has no continuation hyphen. An ending delimiter quotation mark is assumed after column 72.

012 M OVERSIZE NAME/LITERAL, THE ITEM WILL BE TRUNCATED

A *user-name* over 30 characters or numeric literal over 18 digits or alphanumeric literal over 132 characters was found. The word is truncated at the maximum size, the balance is treated as a new word.

013 W CONTINUATION (') MISSING BEFORE FIRST NON–BLANK ITEM

A continued alphanumeric literal does not have a quotation mark before the first nonblank character. The quotation mark is assumed.

014 W LITERAL CONTINUATION (') APPEARS BEFORE COL. 12

The continuation quotation mark for an alphanumeric literal should be in the Area B (column 12 or higher). The quotation mark was accepted as is.

015 W WORD SHOULD CONTINUE IN AREA B

The continuation part of a word should start in the Area B (column 12 or higher). The continued word was accepted as is.

016 S UNABLE TO FIND TEXT–NAME, COPY IGNORED

The *text-name* cannot be found in the user's Library Index of files, therefore COPY cannot be completed.

017 S ILLEGAL DATA ITEM IN SOURCE CLAUSE OR USE STATEMENT

In CONTROL FOOTING, PAGE HEADING, PAGE FOOTING, and REPORT FOOTING report groups, SOURCE clauses and USE statements may not reference: (a)group data items containing control data items, (b) data items subordinate to a control data item, or (c) a redefinition or renaming of any part of a control data item. In PAGE HEADING and PAGE FOOTING report groups, SOURCE clauses and USE statements must not reference control data-names.

018 S SUBSTITUTION TABLE OVERFLOW, REPLACING TERMINATED HERE

Too many REPLACING items in a COPY statement caused the compiler table to overflow. All replacements up to but not including the current one are honored. Others are ignored.

019 M DUPLICATE (') AT BEGINNING OF LITERAL; IGNORED

It is illegal for an alphanumeric literal to begin with two quotation marks. If an imbedded quotation mark is desired as the first character of the literal, the stated literal should start with three quotation marks. Extraneous quotation mark is ignored.

020 M ILLEGAL VERB/USER NAME FOLLOWS PAR/SEC NAME, IGNORED

The paragraph name or section name is followed by something other than a period or the word SECTION. The extraneous word is ignored.

021 W BLANK MISSING BEFORE OPERATOR

Any arithmetic or comparison symbol must be preceded by a blank. The blanks are assumed to be present and the symbol is treated as a correct operator or punctuation character.

022 W BLANK MISSING AFTER ARITHMETIC OR RELATIONAL OPERATOR

An arithmetic or relational operator must be followed by a blank. A blank is assumed present and the symbol is treated as an arithmetic or relational operator.

023 R NOTE VERB ALLOWED ONLY IN PROCEDURE DIVISION

The NOTE statement appears illegally in the Identification, Environment, or Data Division. It is processed in the same manner as a legal Procedure Division NOTE statement.

024 S WORD (COPY) AT THIS POINT IS SYNTACTICALLY INVALID

It is illegal for library text to contain a COPY statement. The library COPY statement is ignored and the original COPY is terminated.

025 F PROCEDURE DIVISION HEADER MISSING, COMPILE TERMINATED

The source program does not contain a valid Procedure Division header. Code generation is suppressed unless the @ACOB processor call statement had the A option, in which case an unexecutable program will result.

026 M HYPHEN AT END OF WORD; IS TREATED AS PART OF NAME

A *user-name* must not end with a hyphen. If the hyphen is intended as a minus sign, it must be preceded by a blank. The hyphen is accepted as part of the name.

027 M ILLEGAL CHARACTER IN WORD; TRUNCATED

Each character in a *user-name* must be a letter, a digit, or a hyphen. The remaining characters in the name, starting with the illegal one, are truncated.

028 M ILLEGAL CHARACTER IN NUMERIC LITERAL; TRUNCATED

A sequence of digits followed by a letter is treated as a legal *user-name*. A sequence of digits with a decimal point is assumed to be a numeric literal and, if a letter or other nondigit appears, it is an illegal character. The numeric literal is truncated starting with that character.

029 W ILLEGAL (.) (,) (;); TREATED AS PUNCTUATION

When a period, comma, or semicolon is used as punctuation (not in a PICTURE or a literal), it must be followed by a blank. The character is accepted as punctuation and compilation continues as if a blank had been present.

030 W BLANK MISSING BETWEEN PLUS/MINUS AND A USER–NAME

A blank must appear between a plus or minus sign and a *user-name*. Processing continues as if a blank had been present.

031 R MORE THAN 99 DIAGNOSTICS FOR THIS LINE

A maximum of 99 diagnostic messages can be printed for one source image. Excessive diagnostics are suppressed.

032 W ILLEGAL WORD IN AREA A; WORD ACCEPTED

This word is not an allowable section, paragraph, or entry header and therefore should not start in Area A (between columns 8 and 11). The word is accepted and processed as if it were properly positioned.

033 R ATTEMPT RECOVERY HERE AFTER ERRONEOUS SOURCE SKIPPED

After the syntax error in an earlier image or column, the compiler is attempting to resume normal syntax analysis at this point.

034 M THIS USER NAME IS INADEQUATELY QUALIFIED

This *user-name* is duplicated and therefore requires qualification when referenced. The first declaration of the name is used if: (1) the *user-name* had no qualifiers at all; or (2) the *user-name* is improperly qualified but has no duplicates. If the *user-name* is inadequately qualified and has duplicates then the second declaration is used.

035 M LAST IDENTIFIER IN THIS LINE INADEQUATELY QUALIFIED

The same as 034 except that user name and its qualifiers are on separate source images and therefore require that the message be worded differently to identify the source of error.

036 M THIS QUALIFIER INAPPROPRIATE FOR USER NAME

There is no declaration of this *user-name* which fits the existing set of qualifiers. The first declaration of this *user-name* is used if it had no duplicates; otherwise the last declaration is used.

037 M QUALIFIERS INAPPROPRIATE FOR LAST NAME IN THIS LINE

For the last identifier on the indicated line, there is no declaration of the first *user-name* which fits the existing set of qualifiers. The first declaration of this *user-name* is used if it had no duplicates; otherwise the last declaration is used.

038 M EXTRANEOUS OR UNRECOGNIZABLE TOKEN(S) IN LINE: IGNORED

Tokens not belonging to the text are deleted.

039 W BLANK MUST PRECEDE STARTING QUOTE

A blank must precede a starting quotation mark.

040 M ITEM SIZE GREATER THAN 262K; MAXIMUM USED

A data item, elementary or group, must not contain more than 262,140 characters or 65,535 words; 262,140 characters are allocated.

041 M ILLEGAL CHARACTER IN PICTURE; ILLEGAL ONES IGNORED

The PICTURE contains an illegal character; only the following are allowed:

0, 1, 9, A, B, C, D, P, R, S, V, X, Z, +, -, /, currency sign, comma, decimal point, and parenthesized integers.

The illegal character or characters are ignored and the rest of the PICTURE is processed as if illegal characters had not been present.

042 M ILLEGAL ORDER OF CHARACTERS; CONFLICTING ONES IGNORED

The PICTURE contains a pair of characters which either must not appear in the same PICTURE or must not appear in the stated order. Cases of the first type are: S with an editing character; 1 with anything but S or 1; S, V, or P with A or X; two decimal markers (V or .); two different floating characters (Z, *, pair of +'s, $'s). Cases of the second type are: + or – after $; floating symbol after 9; S in any but the high–order position; C not followed by R; D not followed by B; P's not high or low order. Any PICTURE character which conflicts with an earlier character is ignored. Any PICTURE string is processed as if conflicting characters had not been present.

043 M ILLEGAL REPEAT COUNT; ILLEGAL CHARACTER IGNORED

The repeat count following a left parenthesis either has a nondigit in it, or has more than five digits, or the PICTURE ends before the right parenthesis occurs. PICTURE is processed as if illegal characters were not present.

044 M ILLEGAL CHARACTER WITH FLOAT/SUPPRESSION SYMBOL; IGNORE

It is illegal to have both a floating insertion or zero suppression symbol (+, –, $, *, or Z) and a 9 to the right of the decimal point in an edited PICTURE string. The string is processed as if floating insertion or zero suppression were specified for the entire item.

045 W (X) IN NUMERIC EDIT PICTURE; X TREATED AS 9

An item is numeric edited if BLANK WHEN ZERO is present or if its PICTURE contains any of the following characters:

0, /, B, C, R, D, B, Z, *, +, -, $, comma, decimal point

Any X in the PICTURE is treated as a 9.

046 M NUMERIC EDITING MAXIMUM SIZE 31; EXCESS TRUNCATED

A numeric-edited item must not be larger than 31 characters. The excess characters in the PICTURE are truncated on the right.

047 M ALPHANUMERIC EDITING MAXIMUM SIZE 255; TRUNCATED

An item is alphanumeric edited if its PICTURE contains at least one A or X, and at least one editing character B or O, and no other numeric editing characters. An alphanumeric edited item must not be larger than 255 characters. The excess characters in the PICTURE are truncated on the right.

048 M EXACT BINARY MAXIMUM 72 BITS; EXCESS TRUNCATED

An item is exact-binary if its PICTURE contains all 1's with an optional S. An unsigned exact–binary item must not be larger than 71 bits or, if signed, it must not be larger than 72 bits. The excess bits are truncated on the right if the size is greater than 72 bits.

049 M MAXIMUM DECIMAL SIZE 18 DIGITS; EXCESS TRUNCATED

An item is numeric if its PICTURE contains only the following characters:

1, 9, S, V, P

If a numeric PICTURE contains 9's, it must not contain more than 18 of them. The excess is truncated on the right.

050 M ILLEGAL POINT LOCATION CLAUSE; CLAUSE IGNORED

The POINT LOCATION clause can be stated only for an item with an exact–binary PICTURE (all 1's with a possible S). In all other cases, the POINT LOCATION clause is ignored.

051 M ILLEGAL JUSTIFIED (RIGHT) CLAUSE; CLAUSE IGNORED

The JUSTIFIED (RIGHT) clause can be stated only for an alphanumeric or alphanumeric edited item. In all other cases it is ignored.

052 M ILLEGAL BLANK WHEN ZERO CLAUSE; CLAUSE IGNORED

The BLANK WHEN ZERO clause can be stated only with a numeric (other than exact–binary) or numeric edited PICTURE. In all other cases, it is ignored.

053 M ILLEGAL CHARACTER IN CURRENCY LITERAL

The literal in the CURRENCY SIGN clause must not match any of the allowable PICTURE characters or punctuation characters: A, B, C, D, L, P, R, S, V, X, Z, +, –, *, /, comma, decimal point, left parenthesis, right parenthesis, semicolon, space, and the digits 0 through 9. The illegal currency sign will not be recognized as a currency sign but as a regular character in the picture. The currency sign ($) will be accepted (in other words, CURRENCY SIGN clause is ignored).

054 W MORE THAN 18 NUMERIC CHARACTERS; TREAT AS ALPHANUMERIC

A PICTURE containing only 9's, and more than 18 of them, is treated as if it contained X's instead of 9's; in other words, it has the indicated size but is considered alphanumeric instead of numeric.

055 M MORE THAN 18 SCALING POSITIONS; SCALE REDUCED TO 18

The maximum number of scaling positions, to the left or right of the low–order end of the PICTURE, is 18. An excess scale is reduced to 18.

056 F SAVE TABLE FOR COPY OVERFLOWS HERE, COPYING TERMINATED

The save table overflows during COPY REPLACING due to a very large copy procedure. Reduce the size of the Copy Proc or use fewer characters in your data-names.

057 M ILLEGAL USAGE WITH EXACT BINARY; USAGE IS IGNORED

It is illegal to state any of the usages with an exact–binary PICTURE. The usage clause is ignored.

058 M PICTURE AND USAGE CONFLICT; USAGE IGNORED

Alphanumeric PICTURE valid only for DISPLAY and DISPLAY–1 usage.

059 W NUMERIC TEXT–NAME IS ILLEGAL, ACCEPTED

The library name must be a user defined word or text-word. The name is accepted and copying continues normally.

060 M SECTION PRIORITY MAY NOT EXCEED 2 DIGITS, ZERO ASSUMED

A section-number is specified as other than an integer with a value 00 through 99. Zero is assumed, thus establishing the section as a part of the program's fixed permanent segment.

061 M EXPONENT MAY NOT EXCEED 3 DIGITS, TRUNCATED

A floating-point literal is written with four or more digits. Low order or rightmost three digits used.

062 M EXPONENT MUST BE AN INTEGER

Nonnumeric characters are specified as an exponent in a floating-point literal. Exponent is derived from the digits which precede the invalid character. Remaining characters are treated as the next word.

063 W ILLEGAL CHARACTER IN COL 7 OF NEXT CARD, BLANK ASSUMED

An "*", "/", "–" and "D" are the only valid characters in column 7.

064 W CARD SEQUENCE ERROR, ACCEPTED

The sequence numbers are not ascending.

065 W THIS CHARACTER IS IMPROPER SEPARATOR

At this point only one of the following characters is legal: space, period, comma, right parenthesis, or semicolon.

066 M ILLEGAL USAGE TYPE, IGNORED

A record description has an assumed USAGE which is incompatible with the file's recording mode.

067 S LIBRARY MISSING, COPY IGNORED

The named library has not been assigned, therefore COPY cannot be completed.

068 M PIC 1(72) IS ILLEGAL; ACCEPTED

A data item is exact-binary if its PICTURE contains all 1's with an optional S. An unsigned exact-binary item must not be larger than 71 bits. The compiler will accept this item and allocate 72 bits, however unpredictable results can occur if the left-most bit is set.

069 W PROGRAM CONTAINS PARAGRAPHS BEFORE THE FIRST SECTION

If one paragraph is in a section, then all paragraphs must be in sections. This program contains sections, and the previous paragraphs are not contained in a section. The previous paragraphs are assumed to be in a section.

070–072 Not issued.

073 L LEVEL VIOLATION, USE LOW INTERMED FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW level but the offending syntax is available only in the LOW–INTERMEDIATE level or later.

MOD is the American National Standards Institute 1974 module abbreviation (see Table 18-4), and N is the American National Standards Institute 1974 level (not to be confused with the Federal level) of the given syntax.

074 L LEVEL VIOLATION, USE HIGH INTERMED FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW or LOW–INTERMEDIATE level but the offending syntax is available only in the HIGH–INTERMEDIATE level or above.

MOD is the American National Standards Institute 1974 module abbreviation (see Table 18-4), and N is the American National Standards Institute 1974 level (not to be confused with the Federal level) of the given syntax.

075 L LEVEL VIOLATION, USE HIGH FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked at the LOW, LOW–INTERMEDIATE or HIGH–INTERMEDIATE level but the offending syntax is available only in the HIGH level.

MOD is the American National Standards Institute 1974 module abbreviation (see Table 18-4), and N is the American National Standards Institute 1974 level (not to be confused with the Federal level) of the given syntax.

076 L LEVEL VIOLATION, NOT FED STD COBOL. MODN

Federal Standard COBOL leveling is invoked but the offending syntax is available only as an OS 2200 ASCII COBOL extension, or as an American National Standards Institute 1974 feature not included in any of the FIPS PUB 21–1 levels of Federal Standard COBOL.

MOD is the American National Standards Institute 1974 module abbreviation (see Table 18-4), and N is the American National Standards Institute 1974 level (not to be confused with the Federal level) of the given syntax. MODN consists of four blanks for OS 2200 ASCII COBOL extensions.

077 S USER NAME SHOULD HAVE APPEARED HERE; KEYWORD ACCEPTED

A *user-name* must be used as PROGRAM–ID. Possibly an intended *user-name* accidentally matches a reserved word.

078 M IDENTIFIER OR PSEUDO–TEXT ILLEGAL HERE

In the REPLACING clause of a COPY statement, when the E option appears in the Extra Options field of the processor statement, an identifier or pseudo-text item is not allowed as a replacement entry, nor is a pseudo-text item allowed as a replacing entry.

079 M ILLEGAL NULL REPLACEMENT PSEUDO–TEXT

In the REPLACING clause of a COPY statement, a replacement pseudo-text must not be null, nor may it consist solely of spaces, nor may it consist solely of comment lines.

080 L NOT 1974 AMERICAN NATIONAL STANDARD COBOL

All syntax that violates American National Standard COBOL, X3.23–1974, will be diagnosed. In addition, any violations of the above standard rules such as USAGE COMPUTATIONAL data items, which violate the standard alignment rules, will be diagnosed.

081 W ILLEGAL WORD IN B MARGIN

A section header must begin in margin A.

082 S BLOCK OR RECORD SIZE EXCEEDS 5 DIGIT MAXIMUM

The maximum block or record size for tape files is limited to 5 digits in order to fit into the label fields.

083 M RECORD SIZE & RECORD CONTAINS CLAUSE CONFLICT; IGNORED

A conflict exists between the RECORD CONTAINS clause and the record size. If only *integer-2* was specified, the record size was not equal to *integer-2*. If *integer-1* and *integer-2* were both specified, either *integer-1* does not equal the length of the smallest record or *integer-2* does not equal the length of the largest record. The record size is completely defined within the record description. The RECORD CONTAINS clause is ignored.

084 W KEY WORD DIVISION IS EXPECTED HERE, ASSUMED PRESENT

A period is detected after key word identification. It is ignored and processing continues as normal.

085 W 1985 ANSI COBOL RESERVED WORD, IGNORED

086 W BLOCK SIZE OR BLOCKING FACTOR EXCEEDS 262143 LIMIT

The blocking factor or the computed block size for UNISERVO files exceeds 262143 limit and truncation in internal tables may lead to erroneous results. Please correct the value specified in the BLOCK CONTAINS clause and retry.

087 W BLOCK SIZE EXCEEDS 65,520 WORDS; MAXIMUM ASSUMED

All PCIOS files have an absolute maximum block size of 65,535 words. For performance reasons ASCII COBOL has established the maximum value at 65,520 words. The block size (based on the BLOCK CONTAINS clause and/or the record or Report Description entry) exceeds 65,520 words for this PCIOS file. The block size will be reset to ASCII COBOLs maximum value of 65,520 words.

088–100 Not issued.

101 W PERIOD AFTER PREVIOUS LINE MISSING

The line either contains a section header such as Working–Storage Section, or it contains the end of a file or item description. In both cases the line should terminate with a period. Processing continues as if a period had been present.

102 S LEVEL INDICATOR MISSING; ASSUME MISSING FD

The line after the File Section header should contain one of the headers FD, SD, SA, or CD. The compiler assumes that an FD header was omitted, and it treats the following text as a description of a dummy file in order to analyze it for diagnostics.

103 M REQUIRED KEYWORD MISSING; ASSUME KEYWORD

The clause being used requires a certain keyword (probably IS) at this point in the text. The compiler assumes that the omitted keyword is present and continues to scan the text, starting with the current word.

104 M KEYWORD (DIVISION) WAS EXPECTED AND IS ASSUMED

This word is expected to be the keyword DIVISION. The compiler assumes that it is, and continues the scan with the next word.

105 M KEYWORD (SECTION) WAS EXPECTED AND IS ASSUMED

This word is expected to be the keyword SECTION. The compiler assumes that it is, and continues the scan with the next word.

106 M CONFIGURATION SECTION MISSING

The Input–Output Section header has been reached and neither the Configuration Section header nor any of its contained paragraph headers has been found. The Input–Output Section clauses are processed as if the Configuration Section had been present.

107 M SOURCE–COMPUTER PARAGRAPH MISSING; ASSUMED PRESENT

The line after the Configuration Section header must be the Source–Computer paragraph header. The compiler assumes that a Source–Computer paragraph has been stated.

108 M OBJECT–COMPUTER PARAGRAPH MISSING; ASSUMED PRESENT

The line after the Source–Computer paragraph must be the Object–Computer paragraph header. The compiler assumes that an Object–Computer paragraph has been stated

109 F DATA DIVISION HEADER MISSING; NO SYNTAX ANALYSIS

No Data Division header was found in the program; therefore, no syntax analysis is performed. The diagnostics in the source listing reflect only limited analysis such as literal construction, punctuation, etc. The seriousness of this diagnostic will result in subsequent fatal diagnostics. The compiler will abort.

110 S ENVIRONMENT DIVISION HEADER MISSING

No Environment Division header was found in the program. Syntax analysis proceeds from the Data Division header.

111 M CONFIGURATION HEADER MISSING; ASSUMED TO HAVE APPEARED

The line after the Environment Division header must be the Configuration Section header. In this case the section header is missing but one or more of the appropriate paragraph headers is present. The compiler assumes that the missing section header was present and proceeds to process configuration paragraphs and clauses.

112 M INPUT–OUTPUT HEADER MISSING; ASSUMED TO HAVE APPEARED

The line after the end of the Configuration Section must be the Input– Output Section header. The compiler assumes that it has already appeared.

113 M LITERAL–2 NOT GREATER THAN LITERAL–1, CLAUSE OMITTED

A test has been made on the validity of range and an impossible range has been detected.

114 M INCORRECT LITERAL HERE; CLAUSE OMITTED

A particular type of literal (alphanumeric or numeric integer) is required at this point. In its absence the clause is omitted.

115 M USER NAME SHOULD HAVE APPEARED HERE; CLAUSE OMITTED

A *user-name* must appear here; possibly an intended *user-name* accidentally matches a keyword. The clause is omitted.

116 S SEQUENTIAL ORGANIZATION IS ILLEGAL WITH DYNAMIC ACCESS

DYNAMIC ACCESS is not valid for a sequential file, only for an ORGANIZATION INDEXED or an SDF DIRECT ORGANIZATION file.

117 M VALUE CLAUSE MISSING HERE

No VALUE clause was found in the description of the *condition-name*. The compiler resumes processing at the next level entry and leaves the *condition-name* definition incomplete. Additional diagnostics will be issued if this *condition-name* is referenced. The use of class–name clause without a VALUE clause requires the presence of a SOURCE–ALPHABET clause. Since only one SOURCE–ALPHABET clause is allowed, all class–name clauses except one, must have a VALUE clause.

118 M CONDITION VALUE ILLEGAL OR MISSING; CLAUSE OMITTED

The VALUE clause literal is expected here. The clause is omitted.

119 M CURRENCY CLAUSE LITERAL MISSING; CLAUSE OMITTED

The CURRENCY SIGN clause literal is expected here. The clause is omitted.

120 M KEYWORD (COMMA) MISSING; DECIMAL–POINT CLAUSE OMITTED

The keyword COMMA is expected here, as part of the DECIMAL–POINT clause. The clause is omitted; periods will be assumed as the decimal point character.

121 M INTEGER LITERAL MISSING HERE

A numeric integer literal must appear at this point. The integer 0 is assumed.

122 M NEGATIVE INTEGER ILLEGAL; ZERO IS ASSUMED

A negative numeric literal is illegal here. A literal of 0 is assumed.

123 M ILLEGAL LABEL RECORD OPTION; ASSUME OMITTED

The LABEL RECORDS ARE STANDARD/OMITTED/*data-name* are the only permissible options. Option is either specified incorrectly (misspelled) or is in conflict with file assignment. In either case OMITTED is assumed.

124 M ILLEGAL FILE–LIMIT WORD; CLAUSE IGNORED

The FILE–LIMIT words must be *user-name*s or numeric integer literals. The clause is ignored if neither is present.

125 W OPTIONAL FILE AND RANDOM ACCESS; IGNORE OPTIONAL

The OPTIONAL phrase of the SELECT clause is valid only for a file which has a sequential access mode. For a file which has random access specified, the OPTIONAL phrase is ignored.

126 M AREA CONTAINS CLAUSE MISSING FROM SA

An SA entry requires an AREA CONTAINS clause and one is not found. The compiler assumes AREA CONTAINS 1 RECORDS.

127 M SYNTAX ERROR IN QUALIFICATION; STRING TRUNCATED

In a qualified identifier, the word after IN or OF must be a *user-name*. The qualification string is truncated after the last valid *user-name*, and may therefore result in an incorrect reference.

128 M MISSING RIGHT PARENTHESIS; SUBSCRIPT NOT USED

After the last valid subscript element (integer literal or *user-name* with possible + or – literal) in a subscript expression, a right parenthesis is expected but not found. Since subscripting is not allowed in the Data Division, a malformed subscript is diagnosed and the entire subscript expression is ignored.

129 M ILLEGAL SUBSCRIPT; SUBSCRIPT NOT USED

This word is not a legal subscript element (integer literal or *user-name*) after the left parenthesis starting a subscript expression, or a comma within one. Since subscripting is not allowed in the Data Division, the malformed subscript is diagnosed and the entire subscript expression is ignored.

130 W INTERLOCK OR PROTECT WITH SEQUENTIAL ACCESS; IGNORED

The clause is meaningless with sequential access files as a READ of a sequential access record in a cycle causes the next logical record to be accessed; therefore two cycles can never access the same record simultaneously.

131 W FILE SECTION HEADER MISSING; ASSUMED TO HAVE APPEARED

Between the Data Division header and the first File Description (FD) header, the File Section header was not found. It is assumed.

132 M INTEGER MUST BE 6, 8 OR 12; DENSITY CLAUSE OMITTED

The only correct values for the DENSITY clause are 6, 8 or 12. The clause is ignored.

133 M ILLEGAL USAGE WORD; DEFAULT IS TO (DISPLAY)

This word is not one of the allowable usages:

DISPLAY, DISP, DISPLAY-1, DISP-1, COMPUTATIONAL, COMPUTATIONAL-1, COMPUTATIONAL-2, COMPUTATIONAL-3, COMPUTATIONAL-4, COMP, COMP-1, COMP-2, COMP-3, COMP-4, INDEX, LOCK

The default usage DISPLAY is assumed.

134 M FIGCON CONFLICTS WITH PICTURE, CLAUSE OMITTED

The figurative constant does not agree with PICTURE. The clause is omitted.

135 W MISSING LABEL RECORD CLAUSE; (OMITTED) IS ASSUMED

The LABEL RECORDS clause is required in every File Description. LABEL RECORDS ARE OMITTED is assumed.

136 W FILE HAS LINAGE CLAUSE; PLUS REPORT

A file containing one or more reports should not have a LINAGE clause since the line-spacing mechanisms activated by the two clauses will probably conflict. Both clauses are accepted.

137 M ERROR IN RECORDING MODE CLAUSE; CLAUSE IGNORED

Illegal syntax has occurred in a RECORDING MODE clause. The clause is omitted.

138 S RECORD LEVEL NOT 01; ENTRY IS SKIPPED

A level number other than 01 has been used where a 01 level number is required. The entry started by the illegal level number is skipped.

139 M ERROR IN POINT LOCATION CLAUSE; CLAUSE IGNORED

A syntax error has occurred in a POINT LOCATION clause. The clause is ignored and the decimal location is assumed to be immediately to the right of the rightmost 1 in the exact binary picture.

140 M (SEQUENTIAL/RANDOM) IS REQUIRED HERE; SEQUENTIAL USED

In an ACCESS MODE or PROCESSING MODE clause, this word must be SEQUENTIAL or RANDOM. SEQUENTIAL is assumed.

141 M ILLEGAL DEPENDING ON VARIABLE; CLAUSE IGNORED

An illegal, missing or unrecognizable DEPENDING ON variable has been specified. The clause is ignored and the array will be treated as a fixed-length table of the specified maximum size.

142 M ERROR IN APPLY CLAUSE; CLAUSE OMITTED

A syntax error has occurred in an APPLY clause. Possibly the APPLY type-word is not one of the allowable ones: EXREF, EXDEF, PROTECT, CORE–INDEX, WRITE–ONLY, INTERLOCK. The clause is omitted.

143 W VALUE CLAUSE NOT PERMITTED HERE, VALUE CLAUSE IGNORED

The VALUE IS clause cannot be used to specify the initial value of a data item in the File Section (except as a record selector) or the Linkage Section. The VALUE IS clause cannot be specified following a variable array or subordinate to another VALUE IS clause. In all sections, the VALUE IS clause can be used to specify a *condition-name* entry. In addition, the VALUE OF clause cannot be specified in a file that has APPLY EXREF specified. The VALUE clause is ignored.

144 M ILLEGAL RENAMES CLAUSE; CLAUSE IS OMITTED

A syntax error has occurred in a RENAMES clause. The clause is omitted.

145 S FILE NOT ASSIGNED TO TAPE, FILE IGNORED IN THIS CLAUSE

The file in a multiple file clause has not been assigned to tape.

146 M (INPUT/OUTPUT) MISSING IN CD; USE INPUT ENTRY

The keyword INPUT or OUTPUT must appear in the Communication Description (CD) entry. INPUT is assumed.

147 M NO ASSIGN CLAUSE; (DISC) USED

Within a SELECT sentence, an ASSIGN clause is expected. DISC is assumed.

148 M DEVICE WORD ILLEGAL; (DISC) USED

This is not one of the allowable device-names in the ASSIGN clause. DISC is assumed.

149 W CONFLICT IN USE OF SAME [RECORD/SORT] AREA CLAUSE

A *file-name* must not appear in more than one SAME AREA clause. A *file-name* must not appear in more than one SAME RECORD AREA clause.

If one or more *file-name*s of a SAME AREA clause appear in a SAME RECORD AREA clause, all of the *file-name*s in that SAME AREA clause must appear in the SAME RECORD AREA clause. Clause ignored.

150 M ERROR IN RERUN CLAUSE; CLAUSE OMITTED

A syntax error has occurred in the RERUN clause. The clause is omitted.

151 M ILLEGAL LEVEL NUMBER; ENTRY IS SKIPPED

The first item in the Working–Storage, Common–Storage, and Linkage Sections must be either a Level 77 or Level 01 item. The allowable level numbers are: 01 to 49, 66, 77, and 88. The entry started by the illegal number is skipped.

152 M EXTERNAL FILE IN SAME CLAUSE; CLAUSE IGNORED

A file named in an APPLY EXDEF/EXREF clause is an external file. External files may not be named in a SAME AREA or SAME RECORD AREA clause. The file name is ignored.

153 M (KEY,LINAGE) COMBINATION ILLEGAL; LINAGE OMITTED

It is illegal for a file to have an ACTUAL KEY clause (FILE–CONTROL paragraph) and a LINAGE clause (File Description). The LINAGE clause is omitted.

154 S KEY SIZE GREATER 63 COMPUTER WORDS; EXCESS TRUNCATED

The ACTUAL KEY for the file whose FD appears on the indicated line must not exceed 63 words in length. This is 252 ASCII bytes or 378 Fieldata bytes. The field will be truncated to 63 words when used as a key at object time.

155 M FD CLAUSE CONFLICTS WITH AN ASSIGNED DEVICE NAME

One of the File Description clauses conflicts with the assigned device-name in the FILE–CONTROL paragraph. For example, a nonprint file cannot have a LINAGE clause or a card input file cannot have a REPORT clause. In general, the FD clause which causes the conflict is ignored. However, if a LINAGE clause is specified for a nonprint file or a REPORT clause is specified for a card input file, the device assignment is revised to PRINTER.

156 W LABEL RECS OMITTED ILLEGAL IN THIS FILE; ASSUME STD

A tape file which appears in the MULTIPLE FILE TAPE clause specified that the LABEL RECORDS ARE OMITTED. For purposes of automatic tape positioning, label records are required and STANDARD is assumed.

157 W UNDEFINED VALUE OF DATANAME; IGNORED

The data-name found in the VALUE OF clause must be defined in the WORKING–STORAGE or COMMON–STORAGE sections; if it is not, the entry is ignored.

158 S KEY CLAUSE MISSING – RANDOM OR ORGANIZATION–INDEX FILE

A RANDOM ACCESS or ORGANIZATION IS INDEXED file must have an ACTUAL KEY/RECORD KEY/RELATIVE KEY clause stated for it in the FILE–CONTROL paragraph. Possibly the ACTUAL KEY/RECORD KEY/RELATIVE KEY clause was skipped because of a syntax error earlier in the SELECT sentence. Processing continues as if the error had not occurred.

159 M VALUE OF CLAUSE AND RECORDING MODE CONFLICT, IGNORED

The VALUE OF clause is not implemented for noninternal recording mode files. LABEL RECORDS STANDARD does not use the VALUE OF clause when doing labeling on a noninternal recording mode file.

160 W (SAME) CLAUSE HAS ONLY 1 FILE–NAME HENCE IS IGNORED

The SAME AREA or SAME RECORD AREA clause must state at least two valid *file-name*s. Only one is found in the above clause, so it is ignored.

161 M REDEFINE SUBJECT IS EXDEF OR EXREF RECORD; REDEFINES IGNORED

In A REDEFINES B, A is the subject and B is the object. The EXDEF or EXREF Working–Storage record must not be the subject of a REDEFINES clause. The REDEFINES clause is omitted.

162 S FILE/WORKING–STORAGE GREATER THAN 65K; TRUNCATED

The maximum main storage space that can be allocated for any file (including its current record area), for all Working–Storage, or for all Common–Storage is 65,534 words. The excess is truncated.

163 S MULTIPLE FILE CLAUSE & RECORDING MODE CONFLICT; IGNORED

Automatic file positioning at open time is not honored for LION, CFH, FORM01, FORM02, FORM03, U, V, or F files. The MULTIPLE FILE TAPE clause is ignored.

File positioning for foreign files can be accomplished by specifying the NO REWIND clause for execution of a series of OPEN and CLOSE statements referencing the preceding series of files on the physical reel.

164 M CD NAME IS DUPLICATE; FIRST DECLARATION USED

A data-name in a CD entry is duplicated. The first of the multiple definitions is accepted; the rest are ignored.

165 M LABEL RECORD AND RECORDING MODE CONFLICT; ASSUME STD

LABEL RECORD IS data-name may not be specified for LION, CFH, FORM01, FORM02, or FORM03 files. LABEL RECORDS ARE STANDARD is assumed.

166 M INTEGER–2 NOT GREATER THAN INTEGER–1; INTEGER–1 USED

When processing the OCCURS clause where *integer-1* and *integer-2* are both specified a check is made on the values of *integer-1* and *integer-2* to ensure that *integer-2* is larger than *integer-1*. If not, the diagnostic is issued and *integer-1* is used.

167 M VALUE OF CLAUSE & LABEL RECORDS CLAUSE CONFLICT; IGNORE

The VALUE OF data-name clause can only be used with the LABEL RECORD IS data-name clause. The VALUE OF clause is ignored.

168 M SUBSCRIPT ILLEGAL HERE AND IS OMITTED

It is illegal to reference a dimensioned item here. Consequently the subscript expression is a syntax error. The subscript expression is ignored.

169 S FILE–NAME APPEARS IN PREVIOUS FD; PREVIOUS IS OVERLAID

This *file-name* appeared in a previous File Description paragraph. The current description will override the previous one.

170 S UNSELECTED FILE; SYNTAX CHECKED BUT NO CODE GENERATED

This *file-name* did not appear in a SELECT sentence in the FILE–CONTROL paragraph. Its File Description and Record Descriptions will be checked for diagnostics, but no input–output statements will be generated for the file.

171 M NUMERIC ALL LITERAL; ALL IGNORED

The figurative constant ALL can be specified only for alphanumeric literals. The word ALL is ignored.

172 M LABEL RECORD NAME NOT SUBJECT OF A RECORD DESCRIPTION

The names of label records must be the subject of a Record Description associated with the file.

173 M NUMERIC ITEM HAS A/N LITERAL; BAD VALUE MAY RESULT

The Working–Storage item has class numeric; therefore, the initial VALUE should be a numeric literal or ZERO. The character string in the alphanumeric literal is treated as a numeric literal. If there are any nonnumeric characters in it, the initial value is unpredictable.

174 M ITEM UNSIGNED, VALUE SIGNED: OMIT SIGN

The Working–Storage numeric item is unsigned (no S in the PICTURE); therefore, the initial VALUE literal should not have a sign. The sign is omitted.

175 M VALUE LITERAL IS TOO LARGE; TRUNCATED LEFT

The initial VALUE or condition VALUE literal is too large for the item. It will be truncated at the left end.

If the item is decimal numeric, the literal has too many digits to the left of the decimal point.

If the item is binary, the literal when converted to binary has too many significant bits.

If the item is alphanumeric justified, the literal has too many characters.

176 W ITEM ALPHA, VALUE NUMERIC; USE ALPHA

The Working–Storage item is class alphanumeric, which includes numeric edited. Therefore the initial VALUE literal should be alphanumeric or a figurative constant. The sequence of digits in the numeric literal is treated as an alphanumeric literal, after any sign or decimal point have been stripped.

177 W CLAUSE CONFLICTS WITH IMPLEMENTOR–NAME

The FILE CONTROL entry for a file contains clauses which conflict with the *implementor-name*.

178 M VALUE LITERAL IS TOO LARGE; TRUNCATED RIGHT

The initial VALUE or condition VALUE is too large for the alphanumeric item. It will be truncated at the right end.

179 M LITERAL SCALE GREATER THAN ITEM SCALE; TRUNCATED RIGHT

The numeric initial VALUE literal has a larger scale (more positions to the right of the decimal point) than the numeric item. The excess fractional digits at the right end are truncated.

180 M KEY ILLEGAL IN UNIT RECORD FILE; KEY IS OMITTED

A file which has been assigned to a symbiont device (device-name has CARD–READER, PRINTER or CARD–PUNCH in it) must not have an ACTUAL KEY. The KEY is omitted.

181 W BLANK WHEN ZERO INCOMPATIBLE WITH * ZERO SUPPRESSION

BLANK WHEN ZERO may not be used in an entry that has the * used for zero suppression. BLANK WHEN ZERO is ignored.

182 M FCBW3T TABLE OVERFLOW; THIS ENTRY IGNORED

The internal compiler table FCBW3T cannot contain this entry. If this diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

183 M ERROR IN QUALIFICATION IN DEPENDING ON; CLAUSE OMITTED

A syntax error has occurred in the qualification string in a DEPENDING ON clause. The DEPENDING clause is omitted and the table is treated as if a fixed number of occurrences had been stated.

184 M DEPENDING OBJECT NOT ELEMENTARY; CLAUSE OMITTED

The object of the DEPENDING ON clause must be an elementary item. The clause is omitted, and the table will include the maximum number of occurrences.

185 M DEPENDING OBJECT NOT NUMERIC; CLAUSE OMITTED

The object of the DEPENDING ON clause must be a numeric item. The clause is omitted, and the table will include the maximum number of occurrences.

186 M DATA ITEM ERROR IN LINAGE CLAUSE, CLAUSE OMITTED

The data items specified in the LINAGE clause must be elementary unsigned numeric integers.

187 W NO 01 RECORD IN FD

No data descriptions exist for this file, hence no record space is allocated.

188 M RECORDING MODE CLAUSE CONFLICTS WITH FILE TYPE; IGNORED

The recording mode specification is illegal because of the files device assignment. The RECORDING MODE clause is ignored and defaults to INTERNAL.

189 W RECORD STRUCTURE BYTE ALIGNMENT ERROR

A record description is considered to be "byte-consistent" if it meets one of these two criteria:

1. All its elementary items, other than exact-binary, are 9-bit usage (DISPLAY or COMPUTATIONAL) and any exact binary items occupy integral 9-bit bytes, either individually or in groups of consecutive items.
2. All its elementary items, other than exact-binary, are 6-bit usage (DISPLAY–1 or COMPUTATIONAL–4) and any exact-binary items occupy integral 6-bit bytes, either individually or in groups of consecutive items.

It is dangerous for a COMPACT record to not be byte-consistent, because the bit padding in the compressed record on external media depends dynamically on the variable length items and arrays.

190 M DEPENDING CLAUSE USED WITH EXACT BINARY ITEM; OMITTED

The DEPENDING ON clause must not be used if the OCCUR phrase item is exact-binary. The clause is omitted, and the table size will include the maximum number of occurrences.

191 W AREA CONTAINS MORE THAN 511 RECORDS; ASSUME 511

The maximum number of records that can be requested in an AREA CONTAINS clause is 511. The maximum is assumed.

192 W VALUE CLAUSE NOT PERMITTED HERE, VALUE CLAUSE IGNORED

The VALUE clause must not be stated in a data description entry that contains an OCCURS or a REDEFINES clause, or in an entry that is subordinate to an entry containing an OCCURS or a REDEFINES clause, unless the VALUE clause is associated with a *condition-name* entry. A Working–Storage item contained in a group item with a VALUE clause must not have a VALUE clause of its own. The current VALUE clause is ignored.

193 W INCORRECTLY FORMED DEBUG STATEMENT (ACCEPTED)

An error was encountered on the Source–Computer paragraph. The WITH DEBUGGING MODE is assumed to be set.

194 S CONDITION VALUE CLASS CONFLICT; DATA OMITTED

If the conditional variable preceding the *condition-name* is class numeric, the condition values associated with the *condition-name* should be numeric literals or ZERO. If the conditional variable is class alphanumeric (this includes numeric edited) the condition values should be alphanumeric literals or figurative constants. The data is omitted.

195 S CONDITION VALUE SCALE CONFLICT; DATA OMITTED

The CONDITION VALUE literal has a larger scale (more positions to the right of the decimal point) than the associated conditional variable. The data is omitted.

196 M PRINT FILES RECORD LENGTH > 63 words; 63 ASSUMED

The maximum record length for a file assigned to PRINTER (in other words, SELECT *file-name* ASSIGN TO PRINTER) must not be greater than 63 words. The maximum record length of 63 words will be assumed.

197 M FILE ILLEGAL IN SAME (RECORD) AREA CLAUSE; FILE OMITTED

It is illegal for a RANDOM PROCESSING file (in other words, PROCESSING MODE IS RANDOM) to be specified in the SAME AREA or SAME RECORD AREA clause. It is also illegal for a shared file (in other words, ASSIGN TO A–SHARED FILE) to be specified in the SAME AREA clause. The file is omitted from the list of SAME (RECORD) AREA files.

198 M FILE > 132 OR 160 CHARACTERS, POSSIBLE SYMBIONT ERROR

This print file contains more than 132 characters. If this print file is assigned to a 0770 printer it contains more than 160 characters. The output will be unpredictable. This message is only available with Extra Option O specified.

199 M TOO MANY (FD),(SD),(CD),(SA),(EXREF) RECORDS DEFINED

Each file, CD, or saved described in the program is assigned a unique location counter in the object module. In addition, each Working–Storage record named in an APPLY EXREF clause is assigned a unique location counter. A maximum of 49 location counters can be assigned to files, Saved–Areas, and EXREF Working–Storage records.

200 M SORTFILE HAS ILLEGAL CLAUSES

A sort-file (distinguished by an SD instead of an FD in the Data Division) has a SELECT clause with an ASSIGN clause. The only other Environment Division clauses that can be stated for a sort–file are SAME RECORD AREA, and SAME AREA. Illegal clauses are ignored.

201 M USAGE ILLEGAL WITH FILE REC MODE OR DEVICE ASSIGNMENT

A record description contains a USAGE which conflicts with the file's recording mode or device assignment (for example, a data item with usage other than DISPLAY in an IBM file when the AN option is specified on the RECORDING MODE clause or an exact binary item in a PRINTER file). This USAGE is accepted.

202 M REDEFINE OBJECT ILLEGALLY DIMENSIONED; IGNORED

The REDEFINES object (the name following the keyword REDEFINES) must not be dimensioned; in other words, it must not have, or be subordinate to, an OCCURS clause. The REDEFINES is ignored.

203 M ILLEGAL LEVEL NUMBER; NOT 1–49,66,77 OR 88

The integer is in the appropriate syntactic position for a level number but is not one of the allowable values 1–49, 66, 77, or 88. Level number 49 assumed.

204 M APPLY INTERLOCK IS ILLEGAL FOR SDF FILE – IGNORED

Record protection via the APPLY INTERLOCK is not valid for a RECORDING MODE IS SDF file.

205 M SAVED AREA NAME ILLEGAL/MISSING; (DUMMY) IS USED

The word after the level indicator SA must be a *user-name*. The SA name DUMMY is assumed.

206 M FILE NAME ILLEGAL/MISSING; (DUMMY) IS USED

The word after the level indicator FD must be a *user-name*. The file name DUMMY is assumed.

207 M ILLEGAL OR UNDEFINED DEP ON VARIABLE; IGNORED

An illegal or undefined DEPENDING ON variable has been specified. The clause is ignored and the array will be treated as a fixed-length table of the specified maximum size.

208 S NO FD IN DATA DIVISION FOR SELECTED FILE

A file was SELECTed in the File–Control paragraph but no FD file description for that *file-name* appeared in the Data Division. Subsequent references to file will be undefined or illegal.

209 M SYNTAX ERROR AT THIS WORD; RECOVERY ATTEMPTED

This word violates the syntax rules. A recovery is attempted.

210 M NO ALLOCATION FOR THIS ELEMENTARY ITEM

This item description is not followed by another at a lower level (higher level number); therefore this item is assumed to be elementary. An elementary item description must include a PICTURE clause or must have USAGE INDEX, LOCK, COMPUTATIONAL–1, or COMPUTATIONAL–2. No memory is allocated for erroneous item.

211 M REDEFINES SUBJ ALLOCATION > OBJ; USE SUBJECT ALLOCATION

In A REDEFINES B, A is the REDEFINES subject and B is the REDEFINES object. The subject whose description appears on this line or whose last contained item is described on this line, has an ending word and byte that is greater than the object's ending word and byte. The next item, if any, is allocated after the subject. Possible causes for this diagnostic are:

- The subject and object specify a different number of character positions.
- The subject or object specify the SYNCHRONIZED clause.

212 W DUPLICATE INDEX NAME; IGNORED

This *index-name* has been declared earlier in the program. While other data names can be declared several times and resolved with qualification, *index-names* cannot. The *index-name* can be used in references to tables other than the one it is declared with, providing the two dimension sizes are the same. (The dimension size is the size of the item with the OCCURS clause.)

213 M INDEX NAME DECLARED MORE THAN ONCE

The same *index-name* is stated more than once in a single INDEXED BY clause. Redundancy is unnecessary and is ignored.

214 M MAXIMUM 49 QUALIFIERS; TRUNCATED

A maximum of 49 qualifiers can be written for one qualified name. The excess qualifiers are ignored.

215 M MAXIMUM 3 DIMENSIONS; IGNORE OCCURS

An OCCURring item contained within an OCCURring item is a 2-dimensional item, and an OCCURring item contained within a 2-dimensional item is a 3-dimensional item. Since three dimensions are the maximum allowed, a 3-dimensional item cannot contain an OCCURring item. The OCCURS clause is ignored.

216 W ITEM USAGE AND GROUP USAGE DIFFERENT BUT ACCEPTED

A group item containing this item had a USAGE clause, and this item has a USAGE clause with a different usage. The usage of this item is that which is explicitly stated in the items USAGE clause. Those items following this item which are also contained in the group item with the USAGE clause, will continue to take their usage from the group.

217 M IMPROPER LEVEL NUMBER; ADJUST TOWARDS 01

This item has a level number which is smaller than the preceding level number, but does not match any previous level number within the record. To avoid ambiguity, the level number is implicitly decreased until it matches some previous level number in the record, possibly 01. This may cause spurious diagnostic 218 messages to be produced for the succeeding items.

218 M GROUP LEVEL WITH ELEMENTARY CLAUSE; IGNORE CLAUSES

This item description is followed by another item at a lower level (higher level number); therefore it is assumed to be a group item. A group item description must not contain any of the following clauses: PICTURE, BLANK WHEN ZERO, JUSTIFIED, or SYNCHRONIZED.

219 M PICTURE AND USAGE CONFLICT; USAGE IGNORED

The USAGE INDEX, LOCK, COMPUTATIONAL–1, or COMPUTATIONAL–2 clause must not be stated for an item with a PICTURE clause. The USAGE clause is ignored.

220 S REDEFINE OBJECT NAME NOT THE SAME AS PREVIOUS

In A REDEFINES B, A is the REDEFINES subject and B is the REDEFINES object. The REDEFINES object must be the most recently declared item, in the same record and of the same level number, that is not itself a REDEFINES subject. In other words, the first REDEFINES subject must refer, as the object, to the immediately previous item at the same level, and any successive REDEFINES subjects at that level must refer to the same object. The "REDEFINES" is ignored.

221 W EXTERNAL FILENAME > 12 CHARACTERS; EXCESS IGNORED

The first 12 characters of a *file-name* or a 12-character external name specified in the SELECT statement are used to identify a file to the Series 2200 Operating System. If files are to be the subject of separate @ASG statements it is important that *file-name*s or external-names be unique in the first 12 characters. If, however, files are the subject of the same @ASG (as for multiple file tape reels), names for such files must be identical in the first 12 characters.

222 W MORE THAN 10 KEY CLAUSES: IGNORED

A maximum of 10 KEY items, ASCENDING and/or DESCENDING, may be specified for a given OCCURS. The first 10 KEY items are accepted. The rest are ignored.

223 M LEVEL 88 USAGE INDEX ITEM; IGNORED

This level 88 item (a *condition-name*) is associated with a USAGE INDEX item which is illegal for a *condition-name*. The level 88 entry is ignored.

224 M PREVIOUS WORD IS DUPLICATE CLAUSE; IGNORED

The previous word started a descriptor clause which duplicates a previous descriptor clause for this item. The duplicate clause is ignored.

225 M DUPLICATE NAME; ACCEPTED; MAY LEAD TO AMBIGUITY

This data name is the same as one declared earlier with the same containing group item; in other words, this item has a previous sibling with the same name. The name is accepted but all attempted references to this item will be diagnosed as ambiguous and treated as a reference to the previous item.

226 M MAXIMUM OF 60 LITERALS ALLOWED HERE – IGNORED

A maximum of 60 literals can be stated in one *condition-name* or record selector entry. A literal range, of the form literal-1 THROUGH literal-2, counts as two literals for this purpose. Additional literals are ignored.

227 M ILLEGAL SEARCH KEY QUALIFICATION; IGNORED

A KEY item cannot be resolved because of incorrect or missing qualification. The KEY reference is ignored and processing resumes with the next key in the KEY clause as if the improper one had not been present.

228 M ERROR IN 2ND RENAMES OBJECT; USE 1ST

In A RENAMES B [THRU C], A is the subject and B and C are objects. In the stated RENAMES clause a syntax error has occurred in the second RENAMES object; therefore, the RENAMES is treated as if there were only one object.

229 M RENAMES OBJECT ADDRESS ERROR; 2ND OBJECT IGNORED

When there are two RENAMES objects, B THRU C, the second object's allocation must begin after the first object's allocation ends. If this is not the case, the second object is ignored.

230 M DUPLICATE NAME IN CD; IGNORED

Within a non-RANDOM CD (Communication Description) all the CD–field names must be unique. Any duplicate names are ignored.

231 M DUPLICATE CD NAME; IGNORED

Non–RANDOM cd-names must be unique. Any such name that matches another name of any type is ignored.

232 M APPLY PROTECT IS ILLEGAL FOR SDF FILE – IGNORED

Record protection via the APPLY PROTECT is not valid for a RECORDING MODE IS SDF file.

233 S ILLEGAL USE OF DYNAMIC ACCESS – SEQUENTIAL ACCESS USED

DYNAMIC ACCESS can only be used with ORGANIZATION DIRECT files if the file also is RECORDING MODE IS SDF. Internal recording mode direct organization files do not have a DYNAMIC ACCESS capability.

234 F PREVIOUS ERROR(S) MEANS NO OBJECT MODULE

The previous diagnostic indicates a catastrophic error. No object module file will be generated by the compiler.

235 R RECOVER HERE AFTER PREVIOUS ERROR

All words, literals, and punctuation since the last one to be diagnosed have been skipped and syntax analysis has been resumed at this point.

236 M MAXIMUM 5 QUALIFIERS IN (KEY) CLAUSE

The identifier in an ACTUAL/RECORD/ALTERNATE/RELATIVE KEY clause can have at most five qualifiers. Additional qualifiers are ignored.

237 M DUPLICATE FILE IN (SAME) CLAUSE; IGNORED

It is illegal for the same *file-name* to appear twice in one SAME [RECORD] AREA clause. The second appearance is ignored.

238 M UNSELECTED/UNDEFINED FILE IN (SAME/RERUN/APPLY) CLAUSE

An unselected file (not appearing in a SELECT clause) or an undefined file (not appearing in a File Description paragraph) has appeared in a SAME [RECORD] AREA, RERUN, or APPLY clause. The illegal reference is ignored.

239 M RENAMES OBJECT NOT IN RECORD; IGNORED

In A RENAMES B, B is the object. The RENAMES object must be an item contained within the current record. The RENAMES entry is ignored.

240 M 2ND RENAMES OBJECT NOT IN RECORD; IGNORED

The *user-name* associated with the THROUGH is not defined in the current record. The RENAMES item is treated as if the THROUGH phrase had not been written.

241 M SPECIAL NAME NOT UNIQUE; IGNORED

All mnemonic-names declared in the SPECIAL–NAMES paragraph must be unique. The duplicated name is ignored.

242 M EXREF OR EXDEF NAME NOT UNIQUE IN FIRST 12 CHARACTERS

The full name of EXREFed or EXDEFed items must appear in the APPLY clause. It may contain more than 12 characters; however, only the first 12 are used at object time, so the names must be unique in the first 12 characters and must not conflict with other object program external–names.

243 M MAXIMUM 100 EXREF NAMES; EXCESS IGNORED

A maximum of 100 names can appear in APPLY EXREF clauses. The excess names are ignored.

244 M DUPLICATE EXREF NAME; IGNORED

It is illegal for the same name to appear twice in APPLY EXREF clauses. The second appearance is ignored.

245 M MAXIMUM 100 EXDEF NAMES; EXCESS IGNORED

There is a 101-name maximum for externally defined items and entry points. Since all programs have at least one entry point, no more than 100 names may appear in an APPLY EXDEF. Extraneous names are ignored.

246 M DUPLICATE EXDEF NAME; IGNORED

It is illegal for the same name to appear twice in APPLY EXDEF clauses. The second appearance is ignored.

247 M FILE–LIMITS CLAUSE IS NOT IMPLEMENTED FOR SDF –IGNORED

The FILE–LIMITS clause is not valid for RECORDING MODE IS SDF.

248 M RERUN ON SDF FILE–NAME IS NOT IMPLEMENTED – IGNORED

RERUN ON for a file which has RECORDING MODE IS SDF is not allowed.

249 M RERUN EVERY E–O–R OF SDF FILE NOT IMPLEMENTED –IGNORED

RERUN EVERY END OF REEL on a RECORDING MODE IS SDF file is not allowed.

250 M INTEGER LARGER THAN 6 DIGITS; SET TO ZERO

A numeric literal in the Environment or Data Divisions, other than in a RERUN or VALUE clause, must not exceed six digits in length. The value zero is assumed as *integer-1*.

251 S DEVICE TYPE AND EXTRA OPTION CONFLICT, REPORT IGNORED

When the O option is specified in the Extra Options field, the only allowable implementor-names for a report file are UNISERVO(S) and MASS–STORAGE. The REPORT clause is ignored.

252 M (KEY) QUALIFIER ERROR, USE FIRST

The identifier that appeared in an ACTUAL KEY clause was insufficiently or improperly qualified. The compiler chooses the first item whose declared name (without qualifiers) matches the base name of the identifiers.

253 M ILLEGAL CONDITION NAME; IGNORE LEVEL 88

The level 88 entry does not follow an item description which can have legal *condition-name*s. The level 88 entry is ignored.

254 M LEVEL 01 REDEFINE, FILE SECTION; IGNORED

The REDEFINES clause in a level 01 entry in the File Section is unnecessary since all the level 01 items within one file are definitions of the current record area of that file, and therefore implicitly redefine each other.

255 M MORE THAN 31 LINKAGE PARAMETERS; TREAT AS 31

Each level 77 or level 01 entry in the Linkage Section is called a parameter item. A maximum of 31 parameter items can be described. The excess items are ignored. In order to stay within the limit, several parameter items can be described as level 02 items within one level 01 item.

256 Not issued.

257 M MORE THAN 60 INDEX NAMES/KEYS; IGNORED

The number of keys plus the number of *index-names* for this occurs is greater than 60. Errors may occur during code generation if the *index-name* is referenced.

258 W 77/01 SHOULD BE IN AREA A

Level numbers 01 and 77 must appear in the Area A of the source line (columns 8–11). Other level numbers can be indented for purposes of readability. The level 01 or 77 item description is accepted.

259 M OCCURS AT 01 OR 77 LEVEL IS ILLEGAL; IGNORE OCCURS

The OCCURS clause is not permitted in a level 01 or level 77 entry. This OCCURS clause is ignored. The item with the illegal OCCURS clause should be given level number 02 with a containing level 01 item in front of it.

260 M FILLER AT 77 LEVEL IS IGNORED

It is illegal for a level 77 entry to be FILLER. The level 77 entry is ignored.

261 M REDEFINES AFTER FILLER IS IGNORED

It is illegal for a FILLER entry to be a REDEFINES subject. The REDEFINES clause is ignored.

262 M ERROR IN DEPENDING RANGE; IGNORED

An error has occurred in a DEPENDING clause. Either the syntax has been violated, or the second integer is not greater than the first. The DEPENDING clause is ignored. The OCCURS clause is treated as a fixed OCCURS clause.

263 M MAXIMUM NO. OF DEP. ON VARIABLES EXCEEDED – IGNORE

When too many DEPENDING ON variables have been defined which refer to items defined elsewhere in the Data Division, the references are ignored and the affected array is treated as a fixed length table. The problem can be resolved by changing the compiler limit of 275 words for DEPTAB in PH2L.

264 M LABEL RECORD MUST BE OMITTED IF FORM01,OMITTED ASSUMED

If FORM01 is used as the recording mode, label records may not be standard or D–N. Label records omitted is assumed.

265 M (FOOTING) MUST NOT BE 0 OR GREATER THAN LINAGE,OMITTED

In the LINAGE clause the value of the FOOTING phrase must be greater than zero and not greater than the value of LINAGE. The clause is omitted.

266 M DIMENSION ERROR IN FIRST RENAMES OBJECT

It is illegal for a RENAMES object to be a dimensioned item. The entire RENAMES entry is ignored.

267 M DIMENSION ERROR IN SECOND RENAMES OBJECT

It is illegal for a RENAMES THRU object to be a dimensioned item. The THRU portion of the RENAMES clause is ignored.

268 M CURRENT RECORD CANNOT BE RENAMED; NO STRUCTURE PRESENT

An 01, 77, 88, or 66 level item may not be the object of a RENAMES. The current level 66 item is ignored.

269 M FORWARD REF TABLE EXHAUSTED – DEP. VARIABLE IGNORED

DEPENDING ON identifier references which refer to data items not yet defined must be preserved for later resolution. The table for this purpose has been filled and the current DEPENDING ON clause ignored. Correction may entail rearranging the section so that the DEPENDING ON identifiers are defined prior to the OCCURS clause referencing them.

270 M PERIPHERAL DEVICE AND USAGE CONFLICT

All the items in a symbiont file (ASSIGNed to PRINTER, 770–PRINTER, CARD–READER, or CARD–PUNCH) should have USAGE DISPLAY or DISPLAY–1.

271 M NON–UNIQUE FILE NAME – USE DUMMY

It is illegal for an item name, at any level, to match a file name. The declared name is accepted and references to it might lead to correct code, depending on qualifications.

272 M MAXIMUM 11 DATANAMES IN INPUT CD LIST; TRUNCATE EXCESS

In an INPUT CD, one CD-name and 11 CD-field-names can be stated. Any names beyond these first 11 are truncated.

273 M UNRECOGNIZABLE WORD; IGNORED

The current word violates the syntax of the particular paragraph or descriptor. This error could be caused by a reserved word being specified in a context where a *user-name* was expected, a *user-name* being specified in a context where a reserved word was expected, or an incorrect reserved word being specified. The word is ignored and recovery is attempted.

274 Not issued.

275 M NON–FLOATING ITEM WITH FLOATING PT. VALUE; VALUE IGNRD

Floating-point numeric literals (10.2E–5) may be stated as initial values only for COMPUTATIONAL–1 or COMPUTATIONAL–2 data items. The current VALUE clause is ignored.

276 M ZERO OCCURS COUNT; ASSUME ONE

The literal in the OCCURS literal TIMES clause must not be zero. It is assumed to be 1.

277 W REDEFINE OBJ ALLOCATION > SUBJ; USE OBJECT ALLOCATION

In A REDEFINES B, A is the REDEFINES subject and B is the REDEFINES object. The object has an ending word and byte that is greater than the subject's ending word and byte. The next item, if any, is allocated after the object. Possible causes for this diagnostic are:

– The subject and object specify a different number of character positions.

– The subject or object specify the SYNCHRONIZED clause.

278 M EXTERNAL ITEM NOT IN (FD SD SA CD 01–WORKING–STORAGE)

An EXREFed item must be an identifier or *file-name* associated with an FD, SD, SA, or CD level indicator or an 01 level Working–Storage record. The information provided to the system for this item will be incomplete and therefore may result in incorrect execution.

279 M LABEL RECORD TABLE OVERFLOW; EXCESS IGNORED

The number of label records specified for this file exceeds the internal compiler table size. If received please submit UCF.

280 M TOO MANY LIMITS

The number of file limits values for this file causes an internal compiler table to overflow. Correction requires reducing the number of file limits values stated. This diagnostic indicates possible execution errors in detecting INVALID KEY conditions.

281 M FORWARD REF TABLE OVERFLOWED

An internal compiler table used to process forward data references in FILE–LIMITS, VALUE OF (label parameter) is *data-name*, ASCENDING/DESCENDING KEY items, and DEPENDING ON variables has overflowed. The source program should be structured to reduce the number of these types of forward references.

282 W LENGTH OF LITERAL NOT EQUAL TO 2 CHARACTERS

The literal specified in the literal is CODE in the Special Names paragraph is larger than two characters. The first two characters are used.

283 M FLOATING ITEM, ALPHA VALUE – ASSUME BINARY ZEROS

A floating-point data item (USAGE COMPUTATIONAL–1 or COMPUTATIONAL–2) is initialized with an alphanumeric literal. The entire floating-point item is preset to binary zeros which is an invalid floating-point value which will cause execution errors if used arithmetically.

284 M VALUE TOO LARGE OR TOO SMALL FOR FLOATING ITEM; USE 0

The numeric value specified for a COMPUTATIONAL–1 or COMPUTATIONAL–2 data item cannot be completely contained in the one or two word floating-point representation of the OS 2200 Systems. An initial value of binary zeros is assumed. Since all zeros is an invalid floating-point value, execution errors will result if this data item is used before being properly initialized.

285 M CLAUSE MEANINGLESS IN THIS FILE; IGNORED

An FD clause contains conflicting specifications that require programmer resolution (for example, LABEL RECORDS ARE STANDARD for a PRINTER file or ACTUAL KEY for a tape file).

286 M ILLEGAL CD NAME; DUMMY USED

The word after the level indicator CD must be a *user-name*. The CD-name DUMMY is assumed.

287 M NO COMPACTION WITH THIS RECORDING MODE, MAX SIZE USED

A variable length array has been declared in a record description which is in a file whose recording mode is LION, SIGN, BLANK, CFH. These recording modes do not allow compaction; the array is treated as a fixed length array. This situation also occurs with recording modes F, U, or V with the AN option. In order to use compaction, the AN option should be removed.

288 M ZERO IGNORED, ONE ASSUMED

The integer zero appeared in a clause which required a positive, nonzero integer. The integer one was assumed.

289 M PREPPING FACTOR ILLEGAL WITH DIRECT ORGANIZATION

One of the implementor names MASS–STORAGE–NNN was specified for a direct file. Only MASS–STORAGE is legal for a direct file.

290 M ISAM FILE REQUIRED, CLAUSE IGNORED

The APPLY CORE–INDEX clause is specified for a non–ISAM file. The clause is not allowed for MSAM files (1974 standard). The *file-name* is ignored.

291 W MULTIPLE DATA RECORDS REQUIRE RECORD SELECTOR ON INPUT

A file which contains multiple data records and which requires record selectors in each record, contains a record with a missing record selector.

292 S DYNAMIC/RANDOM ACCESS ILLEGAL WITH SEQUENTIAL FILES

A sequential file may not specify ACCESS IS DYNAMIC/RANDOM.

293 M SYNTAX ERROR IN APPLY INLINE STATEMENT

This error is issued if an APPLY INLINE statement cannot be found within the INLINE SECTION.

294 W A NESTED VARIABLE ARRAY IS ILLEGAL, ASSUME FIXED

Variable arrays must not be subordinate to a fixed or variable array. The nested variable array is assumed to be a fixed length.

295 M FILE ORG INVALID FOR SHARED FILE; DISC ASSUMED

A–SHARED–FILE is specified as implementor–name–1 on the ASSIGN clause and the ORGANIZATION clause did not specify either RELATIVE or INDEXED. A file whose organization is SEQUENTIAL cannot be processed by the Shared File System. Implementor–name–1 is assumed to be DISC and the file will be processed as a non–shared file.

296 W INTERCHANGE FILE – RM F SPECIFIED FOR VAR LENGTH RECS

RECORDING MODE F is specified for an Information Interchange file and the record descriptions are variable in length. RECORDING MODE D is assumed.

297 W RECORDING MODE U SPECIFIED – UNBLOCKED MODE SET

RECORDING MODE U and a BLOCK CONTAINS integer RECORDS clause with integer greater than 1 or BLOCK CONTAINS integer CHARACTERS where integer CHARACTERS are greater than maximum record length are specified. The BLOCK CONTAINS clause is ignored.

298 W VARIABLE RECORD LENGTHS CALCULATED – RM D ASSUMED

No Recording Mode is specified and the record descriptions for the file are variable in length. RECORDING MODE D is assumed.

299 W BLOCK SIZE EXPANDED TO MAXIMUM RECORD LENGTH

One or more of the record descriptions for the file is greater in length than the integer CHARACTERS specified. The block size is assumed to be the size of the largest record description.

300 S RANK CLAUSE ILLEGAL HERE – CLAUSE IGNORED

The RANK clause only has a meaning in the description of a record selector.

301 M RANK NOT IN RANGE 1 TO 63 – CLAUSE IGNORED

The literal in the RANK clause must be an unsigned integer in the range 1 to 63.

302 M MAXIMUM OF 63 RECORD SELECTORS ALLOWED – IGNORED

At most 63 record selectors are allowed in any one record description.

303 W RECORD SELECTORS HAVE DIFFERENT DESCRIPTIONS – ALLOWED

A record description is now permitted to contain more than one record selector. In addition, the record selectors in different record descriptions in a File Description need not have the same addresses or descriptions. For compatibility, the first record selector in the first record description in a File Description is compared with the first record selection in each of the following record descriptions. If the descriptions do not match, a warning diagnostic is issued.

304 W NON–TRAILING OCCURS DEP – ASSUME FIXED LENGTH

OCCURS DEPENDING ON clause may only be followed within that record description by items that are subordinate to it. This record will be treated as a fixed length record.

305 M ILLEGAL NESTING OF OCCURS DEP CLAUSE – IGNORE DEP

OCCURS DEPENDING ON clause is subordinate to an OCCURS clause. OCCURS cannot be specified on an item of variable length. The DEPENDING clause will be ignored and the record treated as fixed length.

306 M KEY HAS DIFF DIMENSION THAN OCCURS ENTRY – OMIT KEY

There is an OCCURS clause between the KEY IS phrase and the data description of the items specified in the KEY IS phrase. The keys will be ignored.

307 W INTEGER–1 IN OCCURS RANGE EQUAL TO ZERO–ACCEPTED

Nonstandard for an item to OCCUR zero TIMES. The zero will be accepted.

308 M DEPENDING ON OBJECT HAS DIMENSIONS–CLAUSE OMITTED

The OCCURS DEPENDING ON data-name is dimensioned. The record will be treated as fixed length.

309 M DEP VAR NOT UNSIGNED INTEGER –CLAUSE OMITTED

The object of the OCCURS DEPENDING ON clause must be an unsigned integer. The record will be treated as fixed length.

310 M SPECIFICATION OF INTEGER–1 ILLEGAL WITHOUT DEPENDING

A range of occurrences has been specified. This is meaningless without a DEPENDING ON clause.

311 M INTEGER–1 MISSING; INTEGER–1 = ONE ASSUMED

An OCCURS DEPENDING ON clause is present but no range was specified. One will be used as the lower limit of the range.

312 M PROGRAM COLLATING SEQUENCE IN ERROR; STATEMENT OMITTED

A syntax error has occurred in a program collating sequence statement. The statement is omitted.

313 M ILLEGAL ALPHABET–NAME; STATEMENT OMITTED

The *alphabet-name* is invalid (in other words, reserved word, duplicate alphabet-name, file name). The statement is omitted.

314 M NUMBER OF ALPHABET NAMES EXCEEDS 16; STATEMENT OMITTED

The maximum number of alphabet names allowed is 16. The statement is omitted.

315 M ALPHABET–NAME CLAUSE SYNTAX ERROR; STATEMENT OMITTED

One of the following syntax errors has been made in defining an alphabet–name: the same character is specified more than once; a numeric literal is floating point, signed or greater than 0377; a literal associated with a THROUGH or ALSO is more than one character in length; a literal is missing preceding a THROUGH or following a THROUGH or ALSO; or the token encountered does not correspond to any valid syntax allowed in the alphabet-name clause. The statement is omitted.

316 M PROGRAM COLLATING SEQUENCE IGNORED; DEFAULT TO NATIVE

The OBJECT–COMPUTER paragraph specifies a program collating sequence which is not explicitly defined in the SPECIAL–NAMES paragraph. Default to the NATIVE collating sequence.

317 M IGNORE D–OPTION; HIGH–VALUE DEFINED IN PROG COLLAT SEQ

The D option has been used in the Extra Options field of the COBOL control statement. This option will be ignored since a program collating sequence has been defined. Thus, the HIGH–VALUE will be as specified in the ALPHABET–NAME clause associated with the program collating sequence.

318 M ERROR IN SIGN CLAUSE;CLAUSE IGNORED

A syntax error has occurred in a SIGN clause. The clause is omitted.

319 M SIGN CLAUSE & USAGE/FILE CONFLICT;SIGN CLAUSE IGNORED

Sign clause valid only for data items whose usage is DISPLAY or DISPLAY–1. Also the SIGN clause is invalid for an IBM file. The SIGN clause is ignored.

320 M SIGN CLAUSE SPECIFIED FOR UNSIGNED ITEM; CLAUSE IGNORED

SIGN clause is valid only for signed data items; in other words, those with an "S" in the picture character-string. The SIGN clause is ignored.

321 S IMPROPER ALLOCATION – EXPLICIT GROUP USAGE NECESSARY

In some cases groups are misallocated when rules restricting moving modes are not adhered to. This diagnostic is issued when an ASCII group does not start on an ASCII boundary or a Fieldata group does not start on a Fieldata boundary. Putting a USAGE clause on the group item will correct this problem. (See D.2.5, "Alignment in Group Items.")

322 W STATUS KEYS IN ERROR – IGNORED

STATUS KEYS are undefined or are not specified as 2 alphanumeric ASCII characters or they are defined in the File Section, the Report Section or the Communication Section.

323 W LIMIT GREATER THAN 49, IGNORED

The section-number should be an integer in the range 1 through 49. The clause is ignored.

324 M DUPLICATE FILE NAME IN MULTIPLE FILE CLAUSE(S) IGNORED

The file name indicated in error has previously been associated with the same or another MULTIPLE FILE clause. This file name does not become associated with the other files in this MULTIPLE FILE clause.

325 Not issued.

326 S ILLEGAL ALPHABET–NAME SPECIFIED IN CODE–SET CLAUSE

The alphabet-name referenced by the CODE–SET clause must not specify a literal phrase. The CODE–SET clause is ignored.

327 S ILLEGAL FILE–TYPE IN CODE–SET CLAUSE–IGNORE CODE–SET

The device type for the file with the CODE–SET clause was not external media (PRINTER, CARD–PUNCH, TAPE, INTERCHANGE, or CARD– READER). When a REPORT clause is specified in the file description a CODE–SET clause is illegal.

328 S MORE THAN ONE CODE–SET CLAUSE SPECIFIED FOR THIS FILE

The second CODE–SET clause for this file will be ignored.

329 S ILLEGAL USAGE OR SIGN WITH CODE–SET – IGNORE CODE–SET

All data in a file with a CODE–SET clause must be specified as DISPLAY. Any signed numeric data must have the SIGN IS SEPARATE clause. If this is an INTERCHANGE file the sign must also be LEADING.

330 S REDF SUBJECT & OBJECT DON'T BEGIN/END ON SAME BOUNDARY

When redefining an elementary exact binary item by an item which is not exact binary, the redefined exact binary item must begin and end on the same type of boundary (ASCII or Fieldata) as the redefining group. Redefines clause is ignored.

331 W A REPORT MAY NOT BE SENT TO MORE THAN 1 FILE

A report may not be sent to more than one file. The report is accepted.

332 W BOTH DATA RECORDS & REPORT CLAUSES NOT ALLOWED

The DATA RECORDS clause and the REPORT clause are mutually exclusive. This is accepted, however.

333 S ACTUAL/RECORD/RELATIVE KEY(S) SPECIFICATION ERROR

The specified ACTUAL/RECORD/RELATIVE/ALTERNATE keys are not defined as data-items. Or, a RECORD or ALTERNATE key is not defined as a data-item of the category alphanumeric within a record description entry associated with the *file-name*. Or, a RELATIVE key is defined in a record description entry associated with the *file-name*. Or, a RELATIVE key is not defined as an unsigned integer. Or, an ACTUAL KEY is not defined as an integral numeric item.

334 R THIS CLAUSE SHOULD BE SPECIFIED FOR AN ALTERNATE FILE

Specification of this clause for a primary output file will affect all the tasks that follow in this run unit. Specification of this clause for an alternate file will only affect that printer file until a close is executed for that file.

335 R LINAGE OR DENSITY CLAUSE MAY CAUSE FORMS MISALIGNMENT

Depending on the logical page size, physical page size, density, and position on the page, the presence of the LINAGE or DENSITY clause without the VALUE OF FORMS–REQUEST clause may or may not cause the misalignment of forms when a close is executed for this file. It is suggested that the LINAGE or DENSITY clause be specified for an alternate print file in conjunction with the VALUE OF FORMS–REQUEST clause.

336 M USAGE BINARY WITH THIS RECORDING MODE IS ILLEGAL

A file with a recording mode of LION, CFH, F, U, V, FORM01, FORM02, or FORM03 may not contain a record whose usage is binary.

337 S RESERVED WORD NOT DEVICE NAME, IGNORED

In an ASSIGN clause, the use of a reserved word other than a device name is illegal.

338 S TABLE WITH MIXED USAGE & SIZE NOT MULTIPLE OF 18 BITS

A group contained a mixture of 6-bit, 9-bit, and exact binary data items, and the size of the group is not a multiple of half words (18 bits). The alignment of elementary items in the dimensioned group is not predictable. In some cases the alignment is corrected by putting 6-bit explicit usage (DISPLAY–1) on the group, but to ensure correct alignment make the group a multiple of 18 bits.

339 S RENAMES OBJECT OUT OF ASCII/FD BOUNDARY

Renames objects do not start and/or end on the same boundary (ASCII/FD). This may cause bad results at execution time.

340 S MAXIMUM NO. OF ALTERNATE KEYS EXCEEDED, KEY IGNORED

The area which holds alternate keys has been exceeded. Extra keys will be ignored. If received please submit a UCF.

341 S INTERCHANGE FILE MUST HAVE SIGN IS LEADING SEPARATE

Information interchange files are required to have a SIGN IS LEADING SEPARATE clause when the data item is signed.

342 S USAGE CONFLICT WITH PREVIOUS USAGE IN THIS FILE – IGNORE

This usage is conflicting with a previous usage (within the same record or different record) in this file. Two different usages are not allowed for the same file. This usage is ignored.

343 W 01/77 REDEFINES HAVE UNEVEN SIZES

The 01/77 REDEFINES have uneven sizes. This diagnostic is for information purposes only. Conforms to ANSI standard.

344 F PROGRAM EXCEEDS MAXIMUM SIZE ALLOWED – ITEM IGNORED

A data item not unique was encountered. Link required for this nonunique data item exceeds the maximum allowed. Please submit a UCF or make this name unique.

345 S ITEM EXCEEDS MAXIMUM SIZE ALLOWED – MAXIMUM ASSUMED

Item found with size greater than the maximum allowed by the compiler. The maximum size permitted is assumed. This may cause unpredictable results during execution of the program.

346 R UNEQUAL RECORD LENGTHS IN THIS FILE

The 01 data descriptions for this file have uneven sizes. This diagnostic is for information purposes only.

347 W DATANAME OF VALUE OF CLAUSE MUST BE DISPLAY (ASCII)

Data-name–1, data-name–2, ..., must be described as USAGE DISPLAY (ASCII) in the VALUE OF clause.

348 S DUPLICATE COMMON/WORKING – STORAGE SECTION – IGNORE

Only one COMMON–STORAGE/WORKING–STORAGE SECTION is allowed.

349 M DUPLICATE RENAMES SUBJECT – MAY LEAD TO AMBIGUITY

This data name is the same as one declared earlier. The name is accepted but all attempted references to this item will be diagnosed as ambiguous and treated as a reference to the previous item.

350 W REDEFINES SUBJECT AND OBJECT HAVE UNEVEN SIZES

IN A REDEFINES B, A is the REDEFINES subject and B is the REDEFINES object. The REDEFINES subject must occupy the same amount of storage space as the REDEFINES object. This is accepted.

351 M SYNC RIGHT/SYNC LEFT REDEFINES CONFLICT

In A REDEFINES B, A is the REDEFINES subject and B is the REDEFINES object. REDEFINES subject and object are adjusted to opposite sides (RIGHT/LEFT). This can cause improper allocation if the size of both items is not multiples of a full word.

352 S MAXIMUM NUMBER OF BUFFERS EXCEEDED — MAX. ASSUMED

The maximum number of buffers allowed for 1974 Standard Indexed files has been exceeded. 63 is assumed. The number of buffers used by COBOL run time is (4 + n + a), where n = integer-1 of the RESERVE clause (1 if no RESERVE clause), a = number of alternate keys. If 4+n+a > 63, the number of buffers used is reduced to 63.

353 S THE SIZE OF EACH OCCURRENCE EXCEEDS MAXIMUM ALLOWED

The size of each occurrence exceeds the maximum value allowed by the OPFILE. If this item is referenced in the PROCEDURE DIVISION it is possible to get bad code generated or erroneous results.

354 M LABEL RECORD DATA ITEM > 80 CHARACTERS, TRUNCATED

The data item described in the LABEL RECORDS clause is limited to 80 ASCII characters in length. The item size has been truncated.

355 W NO SELECT CLAUSE FOLLOWING FILE–CONTROL PARAGRAPH

The FILE–CONTROL paragraph should not be present if it is not immediately followed by a SELECT statement.

356 M RECORD CONTAINS CLAUSE IN ERROR, CLAUSE IS IGNORED.

The RECORD CONTAINS [*integer-1* TO] *integer-2* CHARACTERS clause contains an error such as an invalid integer or *integer-1* is greater than *integer-2*. The record size becomes defined in the record description and the RECORD CONTAINS clause is ignored.

357 W LABEL REC & VALUE OF CLAUSE CONFLICT; ASSUME STANDARD

VALUE OF clauses were found for a file that specified LABEL RECORDS OMITTED. LABEL RECORDS are set to STANDARD and the VALUE OF clauses are processed accordingly.

358 W RESERVE CLAUSE NOT IMPLEMENTED FOR REC. MODE SDF FILES

The RESERVE integer ALTERNATE AREAS clause is not allowed for files with RECORDING MODE IS SDF. The clause is ignored and two input output areas are allocated for the file.

359 W EXREF'ED FILE USES FILE STATUS FROM EXDEF'ING PROGRAM

If a file is specified in an APPLY EXDEF clause, then any FILE STATUS data–item associated with that file must either be defined in the COMMON STORAGE SECTION of all programs that reference it, or it must be EXDEF'ed and defined in WORKING STORAGE. In this case, any program where the file is EXREF'ed must either define the FILE STATUS in the LINKAGE SECTION, or EXREF it in WORKING STORAGE. If it is EXREF'ed in WORKING STORAGE, then the FILE STATUS clause in the EXREF'ed file's SELECT clause will be ignored, and the FILE STATUS from the EXDEF'ing program will be used.

360 R EIS CODE GENERATION SUPPRESSED – INCOMPATIBLE USAGE

The generation of EIS instructions is not possible due to the use of DISP–1 or COMP–4 data items within the program. This could also be the result of using the T option on the compile statement. The program content is correct, however code will be generated as though EIS was not specified on the OBJECT–COMPUTER clause. To enable EIS instructions to be generated, all DISP–1 and COMP–4 items must be removed from the program, or the T option should be removed from the compile statement if it is specified.

361 R VALUE CLAUSE LITERAL EXCEEDS PIC SIZE FOR BINARY ITEM

The numeric literal in a VALUE clause associated with a binary item (COMPUTATIONAL or COMPUTATIONAL–4) has a value, which is outside the range of values indicated by the picture clause. No truncation of the literal occurs when it is converted to binary since it can be entirely contained within the item without loss of significant bits, however, use of this item for non–computational purposes (in other words, DISPLAY or certain MOVE's) may give erroneous results.

362 W RECORD SIZE OF THIS FILE EXCEEDS MAXIMUM ALLOWED

PCIOS does not allow tape files assigned to INTERCHANGE with the RECORDING MODE D, to have record length greater than 9999 bytes, which includes 4 bytes for control information. Therefore, the maximum record size for an INTERCHANGE file is 9995. This is an AMERICAN NATIONAL STANDARD limitation for data block structure. PCIOS does not allow files assigned to TAPE to have a record size larger than 9992.

363 W SELECT OPTIONAL IS ILLEGAL FOR INDEXED/RELATIVE FILES

The optional clause in a SELECT statement is not valid for files with ORGANIZATION INDEXED or RELATIVE. The optional clause will be ignored.

364 M RECORD SELECTOR FIELD TOO LARGE – CLAUSE IGNORED

The size of the record selector field cannot be greater than 132 characters.

365 W FD CONTAINS A REPORT CLAUSE AND RECORD DESC; ACCEPTED

The file description contains a REPORT clause and a record description entry which is illegal but will be accepted. The file's maximum record length is the largest record size of the record description for the file or the largest record size of the record description for the report.

366–369 Reserved

370 M INTEGER LARGER THAN 10 DIGITS; 9999999999 IS ASSUMED

Integer-1 of the RERUN clause must not exceed ten digits in length. The value 9999999999 is assumed for *integer-1*.

371 M OCCURRENCE NUMBER TOO LARGE; MAXIMUM ASSUMED

This occurrence number is larger than 262143. The maximum occurrence number of 262143 will be assumed.

372 W ALLOCATION CHANGE WHEN USAGE COMP–1/COMP–2/INDEX/LOCK

This diagnostic indicates that the allocation for this group item has changed between this level and any released level lower than 6R2C. This diagnostic will be issued when:

- A USAGE of COMP–1, COMP–2, INDEX or LOCK is implicitly or explicitly specified on a group item.
- The group does not start on a word boundary.
- The REDEFINES clause is not specified.

373 W GROUP VALUE CONFLICTS WITH JUST/SYNC/USAGE; ACCEPTED

The VALUE clause must not be specified for a group containing items with descriptions explicitly specifying a JUSTIFIED, SYNCHRONIZED, or USAGE clause (other than DISPLAY or DISPLAY–1). The VALUE clause will be accepted.

374 M PERIOD WITHIN FORMS–REQUEST CLAUSE IS ILLEGAL

A period was found within the literal specified for the FORMS–REQUEST clause. The period can cause the program to abort at execution time. All periods must be removed from the FORMS–REQUEST literal.

375 W DATA ALLOCATION CHANGED; POTENTIAL INCOMPATIBILITY

The data allocation has been changed between this level and any release level of the ASCII COBOL compiler higher that 6R1 and up to 7R1F for cases when a level 77 data description entry is specified after a level 66 data description entry or a level 02–49, level 88 data description entry is specified between a level 66 and a level 77 data description entry in the WORKING STORAGE, COMMON–STORAGE or LINKAGE SECTION. The data allocation has been changed for the level 66 and all the following data description entries in that section. This may result in incompatibilities due to different execution results being produced because of the change in the data allocation.

376 W RECORD/ALTERNATE KEY HAVE THE SAME START BIT; ACCEPTED

The file description for an ORGANIZATION IS INDEXED file incorrectly specifies a RECORD key and an ALTERNATE record key that start at the same location within the record. This syntax is illegal but will be accepted for compatibility reasons.

377–400 Not issued.

401 M RD HEADER MISSING; ASSUMED PRESENT

The first image after the REPORT SECTION header must start with RD, for Report Description. The RD header is assumed present.

402 S USER NAME MISSING AFTER (RD); NO REPORT GENERATED

The first word after the keyword RD must be a *user-name*. The rest of the report is scanned but the report cannot be written at object time since it has no associated file.

403 M POSITIVE INTEGER REQUIRED HERE; CLAUSE OMITTED

A positive integer is expected at this point in the syntax. The clause is omitted.

404 M SYNTAX ERROR IN QUALIFIER STRING; STRING TRUNCATED

A syntax error has occurred within a string of qualifiers. The string is truncated at a point before the error.

405 M MNEMONIC FOR CODE MUST BE 2 CHARACTER LITERAL, IGNORED

CODE mnemonic-name is not associated with a 2–character literal. CODE is ignored.

406 M SYNTAX ERROR IN SUBSCRIPT; SUBSCRIPT OMITTED

A syntax error has occurred within a subscript expression. The entire expression is omitted.

407 M NEXT GROUP VALUE OUTSIDE PAGE LAYOUT; ADJUSTED

The following table shows the upper and lower limits for an absolute NEXT GROUP clause:

Type	Lower Limit	Upper Limit
REPORT HEADING		FIRST DETAIL
BODY GROUP	FIRST DETAIL	FOOTING
PAGE FOOTING		PAGE LIMIT

It is adjusted to the value it exceeded.

408 W (CODE) MEANINGLESS FOR PRINTER; OMIT

The CODE clause is intended only for multiple reports which are written to a nonprinter report file for later processing. Therefore, CODE clause associated with a printer file is ignored.

409 M 01 LEVEL EXPECTED HERE; ASSUMED PRESENT

The first level number after an RD paragraph must be 01. It is assumed.

410 W PERIOD MISSING AT END OF PREVIOUS LINE; ASSUMED

The previous image completed a header, paragraph, or descriptor, and should have a period at the end. It is assumed.

411 S REPORT NAME NOT SPECIFIED IN AN FD REPORT CLAUSE

The *user-name* after the keyword RD does not match any report-name declared in a REPORTS clause in an FD. The report is scanned but will not be written at object time.

412 S USAGE CONFLICT WITH PREVIOUS USAGE IN THIS FILE – IGNORE

This usage is conflicting with a previous usage (within the same record or different record) in this Report File. Two different usages are not allowed for the same report file. This usage is ignored.

413 M SYNTAX ERROR IN CODE CLAUSE; CODE CLAUSE IGNORED

The alphanumeric literal or the mnemonic-name associated with an alphanumeric literal in the CODE clause was not found. The clause is ignored.

414 M (FINAL TOTAL) NOT FIRST CONTROL FIELD; OMITTED

If FINAL is one of the control fields in the CONTROL clause, it must be stated first in the sequence. Here FINAL is ignored.

415 S CONTROL DATA ITEMS MAY NOT BE SUBSCRIPTED OR INDEXED

Each control field (other than FINAL) must be an undimensioned data item declared in the Data Division. The current word does not meet these specifications and is ignored.

416 M NO VALID FIELDS IN (CONTROLS) CLAUSE; CLAUSE OMITTED

The CONTROL clause has been completely scanned and no valid control fields have been found. No control items are associated with report.

417 M CBTABS OVERFLOW. FURTHER CONTROLS NOT ENTERED IN TABLE

CBTABS is used to maintain a list of all control fields so that reference to them within USE procedures may be diagnosed. Control fields for this RD are not included in CBTABS. May prevent proper issuing of diagnostic 17 in the DECLARATIVE sections. If this diagnostic is received, please submit a UCF.

418 M KEYWORD (DETAIL) IS MISSING; ASSUMED TO BE PRESENT

In a FIRST DETAIL or LAST DETAIL clause, the keyword DETAIL was not found. It is assumed.

419 R RECOVER HERE AFTER PREVIOUS ERROR

All words, literals, and punctuation since the last one to be diagnosed have been skipped and syntax analysis has been resumed at this point.

420 M (ZERO) EXPECTED HERE; ASSUMED TO BE PRESENT

In a BLANK WHEN ZERO clause, the keyword ZERO was not found. It is assumed.

421 S DUPLICATE (TYPE) CLAUSE IN GROUP; USE LAST ONE

The TYPE clause appeared more than once in a level 01 entry. The last TYPE stated is used and any earlier ones are ignored.

422 S (HEADING) OR (FOOTING) NOT FOUND; (HEADING) ASSUMED

Neither the keyword HEADING nor the keyword FOOTING was found after the keyword CONTROL, PAGE, or REPORT in a TYPE clause. HEADING is arbitrarily assumed.

423 S SYNTAX ERROR IN TYPE CLAUSE; (TYPE DETAIL) ASSUMED

A syntax error has occurred in a TYPE clause. TYPE DETAIL is assumed.

424 S CONTROL FIELD NAME MISSING; OMIT ENTIRE GROUP

In a TYPE CONTROL HEADING or FOOTING clause, the control field name was not found. The report group is scanned but no object time writing will be done for it.

425 S CONTROL FIELD NAME NOT DEFINED; OMIT GROUP

The control field name in a TYPE CONTROL HEADING or FOOTING clause does not match any of the control field names in the CONTROL clause in the RD. The report group is scanned but no object time writing will be done for it.

426 M POSITIVE INTEGER REQUIRED; PLUS 1 ASSUMED

No positive integer was found in a LINE NUMBER clause. LINE NUMBER IS PLUS 1 is assumed.

427 M (PAGE) NOT FOUND IN (NEXT GROUP) CLAUSE; ASSUMED

The keyword PAGE in a NEXT PAGE specification (in a LINE NUMBER or NEXT GROUP clause) was not found. It is assumed.

428 M (GROUP) NOT FOUND IN (NEXT GROUP) CLAUSE; ASSUMED

The keyword GROUP in a NEXT GROUP clause was not found. It is assumed.

429 M RESET MUST NOT SPECIFY LOWER LEVEL CONTROL; IGNORED

RESET phrase has specified a lower level control than the associated control for the report group in which the RESET phrase appears. The RESET phrase is ignored.

430 S (FINAL) NOT LISTED IN CONTROLS CLAUSE; REPORT OMITTED

FINAL has appeared in a TYPE CONTROL HEADING or FOOTING clause, but FINAL did not appear in the CONTROL clause in the RD. The report group is scanned but no writing will be done at object time.

431 M SYNTAX ERROR IN VALUE CLAUSE; CLAUSE IGNORED

A syntax error has occurred in a VALUE clause. The clause is ignored.

432 M REPORT ASSIGNED TO MIXED PRINTER TYPES, USED LARGEST

Report is assigned to both 132-column printer and a 160-column printer. Both reports will be treated as if they were assigned to a 160-column printer.

433 M SUM OBJECT IS SUM COUNTER IN HIGHER LEVEL; IGNORED

A SUM object which is a sum counter must be defined either in the same report group that contains the SUM clause or in a report group which is at a lower level in the control hierarchy of this report. The clause is ignored.

434 M SYNTAX ERROR IN (UPON) CLAUSE; CLAUSE OMITTED

A syntax error has occurred in an UPON clause. The preceding SUM clause is treated as if no UPON clause had appeared.

435 M SYNTAX ERROR AT THIS WORD; RECOVERY ATTEMPTED

This word, literal, or punctuation character violates the syntax rules. A recovery is attempted.

436 W REPORT DOES NOT CONTAIN AT LEAST ONE BODY GROUP

The description of a report must include at least one CONTROL HEADING, CONTROL FOOTING, or DETAIL in order to be processed as a printable body group.

437 M LINE NUMBER CLAUSE MISSING, ASSUMED PRESENT

Each report item that contains a COLUMN clause must contain a LINE NUMBER clause or be subordinate to an item that contains a line number clause.

438 M ONLY DISPLAY USAGE PICTURE ALLOWED

The only USAGE permitted in report items is DISPLAY or DISPLAY–1. Other usages are ignored and DISPLAY is assumed.

439 M NO REPORT GROUP (01) FOR THIS REPORT

A report must have at least one report group (level 01). No report groups have been found for this report. No reports will be written for this RD.

440 S SOURCE FORWARD REFERENCE TABLE OVERFLOW; IGNORED

The number of forward reference SOURCE items exceeds the internal compiler table size. This entry is ignored. If this diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

441 S CLAUSE ILLEGAL ON 01 LEVEL, GROUP OMITTED

A SUM, SOURCE, or VALUE clause may only be used on a level number 02 to 48 inclusive. The entire group is ignored.

442 M MORE THAN 2 QUALIFIERS FOR ONE FR OBJECT – IGNORED

There are too many qualifiers in a forward reference source object. This reference is ignored.

443 W NEXT GROUP CLAUSE MUST BE IN REPORT GROUP DESCRIPTOR

The NEXT GROUP clause must appear in the report group (level 01) descriptor. The clause here has appeared in a lower level descriptor. It is assumed to apply to the containing report group.

444 M SOURCE OBJECT CAN NOT BE SOURCE/VALUE SUBJECT, IGNORED.

The source object for a report section item must be either a PAGE–COUNTER, LINE–COUNTER, or sum counter.

445 M NEXT GROUP CLAUSE ILLEGAL WITH (PH/RF); IGNORED

The NEXT GROUP clause must not be specified in a PAGE HEADING report group or in a REPORT FOOTING report group. This NEXT GROUP clause is ignored.

446 M ONLY ONE (NEXT) CLAUSE ALLOWED PER GROUP; USE FIRST

It is illegal to have more than one NEXT GROUP clause within a complete report group description (level 01 and all subordinates). The first one is used.

447 M ABS. LINE LESS THAN PREVIOUS LINE IN GROUP, ASSUME + 1

The absolute LINE NUMBER stated is less than the previous LINE NUMBER in the group. The LINE NUMBER is handled as if a PLUS 1 had been stated.

448 M DUPLICATE CLAUSE FOR THIS DESCRIPTOR; USE LAST

It is illegal to have more than one USAGE, COLUMN, LINE, VALUE, SOURCE, NEXT GROUP, RESET, SUM, or PICTURE clause in a descriptor. The last one is used.

449 M LINE NUMBER IS LESS THAN UPPER LIMIT; USED UPPER LIMIT

See the PAGE Regions Table in 8.2.16.5.

450 M LINE NUMBER GREATER THAN LOWER LIMIT; USED LOWER LIMIT

See the PAGE Regions Table in 8.2.16.5.

451 M LINE NUMBER EXCEEDS GROUPS PRINTABLE AREA; +1 ASSUMED

A relative LINE NUMBER cannot exceed the area specified for a report group, see the PAGE Regions Table in 8.2.16.5. The LINE NUMBER is handled as if a PLUS 1 had been stated.

452 M NEXT GROUP NEXT PAGE ILLEGAL FOR PAGE FOOTING; IGNORED

The NEXT PAGE phrase of the NEXT GROUP clause must not be specified in a PAGE FOOTING report group. This NEXT GROUP clause is ignored.

453 F SECTION OUT OF ORDER; SKIP TO END OF DIVISION

The sections in the DATA DIVISION are not in the correct sequence. Skip to the end of the DATA DIVISION.

454 M ILLEGAL DUPLICATE NAME; ACCEPTED MAY LEAD TO AMBIGUITY

This name duplicates one of the following:

- another name in the CONTROL clause,
- file–name,
- 01 or 77 level in Working–Storage,
- parent or sibling
- RD entry.

The name is accepted but all attempted references to this item will be diagnosed as ambiguous and treated as a reference to the previous item.

455 S SUM COUNTER TABLE OVERFLOW; PREVIOUS ENTRY DESTROYED

The number of sum counters in this program exceeds the internal compiler table size. This entry will destroy the previous entry. If this diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

456 M OUT OF ORDER COLUMN NUMBER; POSSIBLE COLUMN OVERLAP

The elementary report items within the scope of one LINE NUMBER specification must be declared in ascending order of COLUMN NUMBER. A possible cause of a sequence of out-of-order column numbers is an accidentally omitted LINE NUMBER clause in the first. The out-of-order COLUMN NUMBER is accepted but the user is warned of possible column overlapping at object time.

457 S NO TYPE CLAUSE ON 01 GROUP, GROUP OMITTED

A report group description (level 01) must include a TYPE clause. This group is omitted.

458 S SOURCE OBJECT UNDEFINED; IGNORED

The object of a SOURCE clause has not been defined in any section of the Data Division, and it is not a PAGE–COUNTER, LINE–COUNTER, or a sum counter of the report within which the SOURCE clause appears. The SOURCE clause is ignored.

459 M UNDEFINED DATA NAME; IGNORED

The indicated word is expected to be a data item, but it is not defined anywhere in the Data Division. The item is ignored.

460 M ILLEGAL USE OF NEXT PAGE PHRASE; ADJUSTED

This message is caused by two different errors. One error is the use of the illegal syntax LINE NUMBER IS NEXT PAGE. The line number is adjusted to conform to the PAGE Regions Table described in 8.2.16.5. The other error is that a NEXT PAGE phrase has appeared in a report group other than a body group or a REPORT FOOTING. In this case the line number is accepted.

461 M SUM SUBSCRIPT TABLE OVERFLOW; THIS ENTRY IGNORED

In A SUM B, A is the subject and B is the object. The number of SUM objects with subscripts exceeds the internal compiler table size. This entry is ignored. If this diagnostic is received, please submit a UCF.

462 M WORD AFTER (+) NOT INTEGER; IGNORE WORD AND (+)

When a plus sign (+) appears in a subscript indicator the next character-string must be an integer literal. The plus sign and word are ignored.

463 M WORD PRECEDING + IS NOT INDEX–NAME; ACCEPTED

When a plus sign (+) appears in a subscript indicator, the preceding character-string must be an *index-name*. In this case it is a data item, which is accepted.

464 W COMMA OR SEMICOLON NOT FOLLOWED BY IDENTIFIER; IGNORED

A series connector (comma or semicolon) must be followed by another identifier in series. The connector here is ignored.

465 M TOO MANY SUBSCRIPTS; EXCESS IGNORED

The number of subscripts in the subscript expression is greater than the number required by the dimensionality of the referenced item. This includes the case of a subscript expression in a reference to an undimensioned item. The excess subscripts (on right) are ignored.

466 M TOO FEW SUBSCRIPTS; 1ST OCCURRENCES USED ON RIGHT

The number of subscripts in the subscript expression (if present) is less than the number required by the dimensionality of the referenced item. This includes the case of an unsubscripted referenced to a dimensioned item. The missing subscripts are assumed to reference the first occurrence of each of the missing dimensions.

467 M INDEX DATA ITEM USED AS SUBSCRIPT; ACCEPTED

It is illegal for an index item to appear as the object of a SOURCE clause. The index item is accepted here as a binary integer, but will probably cause an incorrect reference at object time.

468 W UNRELATED INDEX–NAME, BUT DIMENSION SIZE CORRECT

An *index-name* should be used as a subscript only for items whose controlling OCCURS clause mentioned that *index-name*. However, as long as the two dimension sizes (the allocated sizes of the items with the OCCURS clause) are equal, the subscripted reference will be handled correctly at object time.

469 M UNRELATED INDEX–NAME, BUT DIMENSION SIZE DIFFERENT

An *index-name* should be used as a subscript only in references to items whose controlling OCCURS clause mentioned that *index-name*. In this case the two dimension sizes (the allocated sizes of the items with the OCCURS clauses) are not equal, and the subscripted reference will work incorrectly at object time.

470 M INDEX–NAME WITH NON–INDEXED ARRAY; ACCEPTED

An *index-name* should not be used as a subscript in reference to items whose controlling OCCURS clause did not include an INDEXED BY phrase. The subscript is accepted but will probably work incorrectly at object time.

471 M SUBSCRIPT NOT INTEGER; TREATED AS INTEGER

A data item used as a subscript must be an integer; that is, its scale (number of decimal positions) must be zero. The noninteger data item is accepted and will be treated as an integer at object time.

472 S SUM SUBJECT TABLE OVERFLOW; PREVIOUS ENTRY DESTROYED

In A SUM B, A is the SUM subject (named sum counter) and B is the object. The number of named sum counters exceeds the internal compiler table size. The previous entry is destroyed. If received please submit a UCF.

473 M SUBSCRIPT OUT OF RANGE; ACCEPTED

A numeric literal used as a subscript must not exceed the maximum number of occurrences stated in the OCCURS clause. The oversize literal is accepted here, but the object time reference will access data outside the declared table.

474 M LINE CLAUSE ILLEGAL HERE; ACCEPTED

The report item that contains this LINE NUMBER clause is subordinate to an item that also contains a LINE NUMBER clause. The LINE NUMBER will be accepted for presentation of this report item and any items subordinate to it.

475 M PICTURE MISSING, ITEM OMITTED

The PICTURE clause must be stated in every elementary report item description.

476 M SUM CLAUSE OUT OF PLACE, CLAUSE OMITTED

The SUM clause is permitted only within a complete report group description (level 01 and all subordinates) of TYPE CONTROL FOOTING. Any other SUM clause is ignored.

477 M SUM OBJECT DEFINED IN REPORT SECTION NOT A SUM COUNTER

On the SUM clause, identifier–1 and identifier–2, when defined in the Report Section, must be the names of the sum counters.

478 S PREVIOUS GROUP HAS SAME (TYPE); PREV. WILL BE IGNORED

Within a report there can be only one (if any) report group of TYPE REPORT HEADING, REPORT FOOTING, PAGE HEADING, or PAGE FOOTING. For each control level there can be only one (if any) report group of type CONTROL HEADING or CONTROL FOOTING. This report group has the same type as a previous one. The previous one will be ignored at object time.

479 R PREVIOUS ERRORS CAUSE REPORT TO BE OMITTED

An earlier diagnostic is considered catastrophic for this report and no Report Control Block will be generated. Statements in the Procedure Division referencing this report will lead to unexecutable coding.

480 M (GROUP INDICATE) ONLY IN (TYPE DETAIL); IGNORED HERE

The GROUP INDICATE clause can appear only within a TYPE DETAIL report group description. The clause is ignored here.

481 M VALUE,SUM,OR SOURCE CLAUSE REQUIRED HERE; ITEM OMITTED

Every elementary report item descriptor must include either a SOURCE, a VALUE, or a SUM clause. In the absence of any of these clauses, the report item has no effect and is ignored.

482 W FIRST LINE NUMBER WITHIN A PF MUST BE ABS.; ACCEPTED

The first LINE NUMBER clause specified within a PAGE FOOTING report group must be an absolute LINE NUMBER clause. The LINE NUMBER clause is accepted.

483 S MORE THAN 9 CONTROL FIELDS; EXCESS ARE IGNORED

A maximum of nine control fields (FINAL, if present, counts as one field) can be stated in a CONTROL clause in the RD. The excess names are ignored.

484 S MAXIMUM NUMBER OF DETAILS EXCEEDED; THIS ONE IGNORED

This detail exceeds the allowable number of details per report and is ignored. If it is received please submit a UCF.

485 S UPON D–N TABLE OVERFLOW, CURRENT DATA–NAME IGNORED

The number of DETAIL report groups named in the UPON phrase exceeds the present table size. This entry is ignored. If this diagnostic is received, please submit a UCF.

486 M UPON PHRASE ILLEGAL WHEN SPECIFIED WITH SUM COUNTER

If the UPON phrase is specified, any identifiers in the associated SUM clause must not be sum counters.

487 M SUM OBJECT MUST BE UNEDITED AND < 18 DIGITS

The SUM object must be a legal arithmetic operand; in other words, it must be unedited and not longer than 18 digits. This SUM object is ignored.

488 M ABS. LINE NO. ILLEGAL AFTER RELATIVE; DEFAULT PLUS 1

The LINE NUMBER IS integer clause indicates an absolute line number in a report page. This designation may not follow relative positionings specified by LINE NUMBER IS PLUS integer. The absolute designation is ignored and the line is treated as if PLUS 1 had been stated.

489 M ONLY REL. LINE NO. IF PAGE LIMIT MISSING; +1 ASSUMED

If the PAGE clause is omitted from a given Report Group Description entry, only relative LINE NUMBER clauses can be specified in any Report Group Description entry within that report. The value specified for the LINE NUMBER clause is ignored and the line is treated as if PLUS 1 had been stated.

490 M PAGE LIMIT SPECIFICATION RULE BROKEN; SPEC ALTERED

The PAGE LIMIT specifications must obey the following rules:

- HEADING not greater than FIRST DETAIL;
- FIRST DETAIL not greater than LAST DETAIL;
- LAST DETAIL not greater than FOOTING;
- FOOTING not greater than PAGE LIMIT;
- PAGE LIMIT not greater than 511.

If a rule is violated, one of the specifications is altered to obey the rule.

491 M ONLY REL. NEXT GROUP IF PAGE LIMIT MISSING; +1 ASSUMED

If the PAGE clause is omitted from the Report Description entry only a relative NEXT GROUP clause may be specified in any Report Group Description entry with that report. The value specified for the NEXT GROUP clause is ignored and is treated as if PLUS 1 had been stated.

492 M COL. NO. + SIZE–1 > 132 OR 160,POSSIBLE SYMBIONT ERROR

The report is assumed to be printed eventually on *nnn* column paper. For a given report item its rightmost printed column equals column number plus size minus 1. If this rightmost printed column is greater than *nnn*, the size is reduced appropriately so that it is *nnn*, where *nnn* is normally 132. When the report is assigned to a 770–PRINTER *nnn* is equal to 160. This message can only be available with Extra Option O specified.

493 S (PH/PF) ILLEGAL WHEN PAGE LIMIT MISSING; OMITTED

PAGE HEADING and PAGE FOOTING report groups may be specified only if a PAGE clause is specified in the corresponding Report Description entry. The PAGE HEADING or PAGE FOOTING is omitted.

494 W (SECTION) MISSING AFTER (REPORT); ASSUMED PRESENT

The header keyword REPORT must be followed by the keyword SECTION, at the beginning of the Report Section. The keyword SECTION is assumed.

495 M NEXT PAGE LEGAL ONLY IN FIRST LINE OF GROUP; USED +1

Within a given Report Group Description entry a NEXT PAGE phrase can appear only once and, if present, must be in the first LINE NUMBER clause in that Report Group Description entry. The line is treated as if PLUS 1 had been stated.

496 M UNDEFINED SUM OBJECT; NO ACCUMULATION CODE GENERATED

The object of a SUM clause on the indicated line was never defined, either in the non-Report Data Division or as a SUM subject. No accumulation code is generated.

497 S SUM OBJECT TABLE OVERFLOW; PREVIOUS ENTRY DESTROYED

In SUM B, B is the SUM object. The number of SUMs exceeds the internal compiler table size. The previous entry is ignored. If this message appears, please submit a UCF.

498 W (SOURCE/VALUE) OBJECT TOO LARGE; EXCESS TRUNCATED

The object of a SOURCE or VALUE clause has more characters (or digits) than the report item's PICTURE clause. The SOURCE or VALUE object will be truncated when moved to this report item at object time.

499 W LINE/PAGE/SUM –COUNTER USED ILLEGALLY; ACCEPTED

A sum counter or the data names LINE–COUNTER and PAGE–COUNTER cannot be used as subscripts within the Report Section. The item is accepted.

500 W NO (COLUMN) CLAUSE ASSOCIATED WITH VALUE

A report item with a VALUE clause must have a COLUMN NUMBER clause associated with it.

501 S PAGE LIMIT GREATER THAN INTERNAL TABLE SIZE, USE MAX

The value of the integer specified for the PAGE LIMIT clause exceeds the capacity of an internal table. The maximum value has been used. If received, please submit a UCF.

502 R MAY CAUSE MISALIGNMENT OF FORMS AT FILE TERMINATION

Depending on logical page size, physical page size, density, and position on the physical page, this clause may or may not cause the misalignment of forms when a CLOSE statement is executed for this file. It is suggested that this clause be specified for an alternate print file in conjunction with the VALUE OF FORMS – REQUEST clause.

503 M SYNTAX ERROR IN SUM CLAUSE, CLAUSE OMITTED

A syntax error has occurred in the SUM clause. The clause is omitted.

504 W ILLEGAL MOVE: SOURCE IS ALPHANUMERIC EDITED

As specified in the X3.23–74 Standard, a move of an alphanumeric edited field to a numeric integer, numeric noninteger, or numeric edited field is not allowed. This also applies to the SOURCE clause.

505 W ILLEGAL MOVE: SOURCE IS NUMERIC NONINTEGER

As specified in the X3.23–74 Standard, a move of a numeric noninteger field to an alphanumeric edited field or an alphanumeric field is not allowed. This applied to the SOURCE clause.

506 M ILLEGAL MOVE: SOURCE IS NUMERIC EDITED

Since the SOURCE clause conforms to the rules of the MOVE statement, a numeric edited data item cannot be moved to a numeric or numeric edited data item. Code will be generated except when the receiving field is described as numeric edited and EIS code generation is specified.

507 R LEVEL NUMBER DOES NOT REFLECT CORRECT HIERARCHY

The hierarchy within a report group is derived solely from the syntax used, not from specified level numbers (except 01). However, level numbers should reflect the hierarchy within a report group.

508 S CLAUSE ILLEGAL WITH LEVEL NUMBER SPECIFIED

The TYPE clause may only be used at an 01 level. The clause is ignored.

509 S NEXT GROUP ILLEGAL WITHOUT LINE CLAUSE, IGNORED

A report group entry must not contain a NEXT GROUP cause unless the description of that report group contains at least one LINE NUMBER clause. The NEXT GROUP clause is ignored.

510 S CODE SPECIFIED FOR ONE REPORT IN FD MUST BE FOR ALL

If the CODE clause is specified for any report in a file, then it must be specified for all reports in the same file.

511 M REPORT LINE GREATER THAN 63 WORDS; MAX CHAR ASSUMED

A report line for a file assigned to PRINTER (in other words, SELECT *file-name* ASSIGN TO PRINTER) must not be greater than 63 words. The maximum number of characters, 252 for ASCII or 378 for Fieldata, will be assumed.

512 W CONTROL CANNOT HAVE VARIABLE LENGTH; MAX SIZE USED.

A control item cannot have subordinate to it a data item which OCCURS a variable number of times. The CONTROL item will be treated as though it had fixed length.

513–600 Not issued.

601 M (END DECLARATIVES) MISSING

The header DECLARATIVES appeared and all the following sections were considered to be Declarative Sections. The end-of-program delimiter (the end of the source program file) appeared before an END DECLARATIVES header appeared. The END DECLARATIVES header should appear after the last Declarative Section.

602 R ATTEMPT RECOVERY HERE AFTER PREVIOUS ERROR

All words, literals, or punctuation since the last one to be diagnosed have been skipped and syntax analysis has been resumed at this point.

603 W EXECUTABLE STATEMENTS FOUND IN EXIT PARAGRAPH; ACCEPT

The EXIT or the EXIT PROGRAM statement must be the only statement in a paragraph. The statement is accepted here. In particular, the EXIT PROGRAM statement will return control to the calling program.

604 M REFERENCE REQUIRES QUALIFICATION; 1ST DEFINITION USED

The indicated reference is to a *paragraph-name* which has been defined more than once. Each definition must be in a different section and each reference must be qualified with a *section-name*. The first definition within the Procedure Division is assumed.

605 S ILLEGAL QUALIFIER; STATEMENT OMITTED

A syntax error occurred in a qualified reference. The statement is omitted.

606 S QUALIFIER IS NOT A PROCEDURE NAME; STATEMENT OMITTED

The qualifier of a paragraph reference must be a *section-name*. The statement is omitted.

607 M INVALID (USING) PARAMETER; PARAMETER IGNORED

The USING parameters in a PROCEDURE DIVISION header must be declared as level 77 or level 01 items in the Linkage Section. A level 02 (or below) Linkage Section item cannot be used as a parameter. The indicated parameter is ignored.

608 S ILLEGAL FILE LIMIT SPECIFIED; OMIT OPEN

File limit identifiers must be defined as numeric integer data items. The OPEN for the file with noninteger file limits is ignored.

609 W EXTERNAL FILE IN (USE) DECLARATIVE; IGNORE REFERENCE

It is illegal to write a Declarative Section in which the USE statement refers by name or by mode (input, output, I–O) to a file named in an APPLY EXREF clause. All such declarative procedures must be contained in the externally defined program. The indicated reference is ignored.

610 S SYNTAX ERROR IN LOCATE STATEMENT; STATEMENT OMITTED

Internal compiler error during generation of relocatable element. If this message is received, please submit a UCF.

611 S SYNTAX ERROR IN SUBSCRIPT; STATEMENT OMITTED

A syntax error has occurred in a subscript expression (subscript indicator or right parenthesis not found, unexpected keyword, etc.). The statement containing the subscript expression is omitted.

612 S SYNTAX ERROR IN SECTION HEADER; SKIP TO NEXT SECTION

A syntax error has occurred in a section header. The section header is omitted.

613 S (USE) MISSING IN (DECLARATIVES); SECTION IGNORED

Every section header in the Declarative Area (between the headers DECLARATIVES and END DECLARATIVES) must have a USE statement in it. The entire section is ignored.

614 S SYNTAX ERROR IN (USE); SECTION OMITTED

A syntax error has occurred in a USE statement in a Declarative Section header. The entire section is omitted.

615 W KEY APPEARS BEFORE IN CLAUSE; IGNORE PREVIOUS

An ascending or descending key item appears more than once in the WHEN condition of a SEARCH ALL statement. Only the last conditional with this key is used.

616 W PERIOD MISSING IN THE LINE; ASSUMED

The current image should have a period after the header or at the end of the image.

617 M EXTRA PERIOD; SECOND ONE IGNORED

Two periods occurred where one was required or allowed. The second one is ignored.

618 W SECTION HEADER MISSING AFTER DECLARATIVES; SUPPLIED

The first image after the header DECLARATIVES must be a section header. A section header is assumed present.

619 S ILLEGAL OFFSET IN SUBSCRIPT; STATEMENT OMITTED

When a plus sign (+) appears in a subscript indicator, the next word must be an integer literal whose size does not exceed ten (10) significant digits. The statement is omitted.

620 W DEVICE USAGE AND STATEMENT INCOMPATIBLE; ACCEPTED

Either the I–O statement or some clause within it contradicts one of the FILE–CONTROL, I–O–CONTROL, or file description clauses previously stated for the file. Examples are CLOSE UNIT on a DISC file and a WRITE ... ADVANCING phrase for a file not assigned to PRINTER. The conflicting statement or clause is accepted.

621 S MISSING PROCEDURE HEADER; SKIP TO PROCEDURE

The first image after the PROCEDURE DIVISION header or the END DECLARATIVES header must be a section header. A dummy section header is inserted and this image is accepted.

622 M PARAMETERS MISSING AFTER (USING); CLAUSE IGNORED

The keyword USING in a Procedure Division header must be followed by one or more Linkage Section parameters. No parameters were found and the USING phrase is ignored.

623 M SUBSCRIPT VALUE OUT OF RANGE; ACCEPTED

A numeric literal used as a subscript must not exceed the maximum number of occurrences stated in the OCCURS clause. The oversize literal is accepted here, but the object time reference will access data outside the declared table.

624 M SUBSCRIPT ITEM NOT AN INTEGER; TREATED AS IF IT WERE

A data item used as a subscript must be an integer, that is, its scale (number of decimal positions) must be zero. The noninteger data item is accepted and will be treated as an integer at object time.

625 M WORD NOT INDEX–NAME; TREATED AS INDEX–NAME.

When a plus sign (+) appears in a subscript indicator, the preceding word must be an *index-name*. In this case, it is a data item, which is accepted.

626 W PREVIOUS COMMA OR SEMI–COLON EXTRANEOUS; IGNORED

Comma (,) or semicolon (;) may be used to separate statements or consecutive items in a series. An extra comma or semicolon that appears at the end of a series of items is ignored.

627 M TOO MANY SUBSCRIPTS; EXCESS IGNORED

The number of subscript indicators in the subscript expression is greater than the number required by the dimensionality of the referenced item. This includes the case of a subscript expression in a reference to an undimensioned item. The excess subscript indicators are truncated and the reference is accepted.

628 M TOO FEW SUBSCRIPTS; 1ST OCCURRENCES USED ON RIGHT

The number of subscript indicators in the subscript expression is less than the number required by the dimensionality of the returned item. The one or two missing subscript indicators are assumed to have the value 1 and the reference is accepted.

629 M SUBSCRIPTED SCALAR; SUBSCRIPT IGNORED

The subscripted reference here is to an undimensioned item. The subscript expression is ignored and the item reference is accepted.

630 M INDEX–NAME USED IN PLACE OF SUBSCRIPT; ACCEPTED

An *index-name* should not be used as a subscript indicator in a reference to an item whose controlling OCCURS clause did not include an INDEXED BY phrase. The subscript indicator is accepted but will probably cause an incorrect reference at object time.

631 W UNRELATED INDEX–NAME, DIMENSION SIZES EQUAL; ACCEPTED

An *index-name* should be used as a subscript indicator only in reference to items whose controlling OCCURS clause mentioned that *index-name*. However, as long as the two dimension sizes (the allocated sizes of the two items with the controlling OCCURS clauses) are equal, the subscripted reference will be handled correctly at object time.

632 M UNRELATED INDEX–NAME, DIM SIZES NOT EQUAL; ACCEPTED

An *index-name* should be used as a subscript indicator only in references to items whose controlling OCCURS clause mentioned that *index-name*. In this case, the two dimension sizes (the allocated sizes of the two items with the controlling OCCURS clauses) are unequal, and the subscripted references will be handled incorrectly at object time.

633 M INDEX–ITEM MAY NOT BE USED AS SUBSCRIPT; ACCEPTED

An index item (USAGE IS INDEX) must not be used as a subscript indicator. It is accepted here, but will probably cause an incorrect reference at object time.

634 W PROCEDURE REFERENCED WITHIN DECLARATIVES; ACCEPTED

Within a Declarative Section, any references (in PERFORM, GO, and ALTER statements) to procedures in the main body of the program are illegal. The reference is accepted.

635 W THIS SUBPROGRAM HAS NO (EXIT PROGRAM); ACCEPTED

The only way a called program can return control to a calling program is by executing an EXIT PROGRAM statement. This subprogram has no EXIT PROGRAM statements.

636 M UNALTERED, NULL (GO TO) STATEMENTS IN PROGRAM

The only purpose of a paragraph consisting solely of a GO TO statement is to serve as an alterable switch. This program contains such GO TO paragraphs which are nowhere altered.

637 S PARAGRAPH NOT ALTERABLE; OMIT ALTER BUT ACCEPT GO TO

This diagnostic points either to an ALTER statement or to the paragraph which a previous ALTER statement is attempting to alter. The paragraph being altered by the ALTER statement must consist solely of a GO TO statement. The ALTER statement is omitted.

638 M SYNTAX ERROR IN MONITOR STATEMENT; IGNORE STATEMENT

A syntax error occurred in a MONITOR statement. The statement is ignored.

639 R EMPTY PROCEDURE PARAGRAPH

The previous paragraph consisted solely of a paragraph header and had no statements in it.

640 S WORD NOT A VERB – SCAN SKIPS TO NEXT SENTENCE

The previous statement was completed by the word, literal, or punctuation prior to the current one; the current word, therefore, is expected to start a new statement and must be a verb. Words are skipped until the next sentence or procedure header appears.

641 M KEYWORD EXPECTED HERE IS MISSING; ASSUMED TO BE HERE

The clause being used requires a certain keyword at this point in the text. The compiler assumed that the omitted keyword is present and continues to scan the text starting with the current word.

642 M DUPLICATE EXECUTE CONDITION FOR SAME FILE; USE THIS

The USE statement in the Declarative Section calls for its execution under the same circumstances (begin label, end label, error; input, output, I–O) for a given file as a previous Declarative Section. The current one will be executed under the appropriate circumstances and the previous one will be ignored.

643 S FD FILE NAME IS REQUIRED HERE; STATEMENT IGNORED

The statement being used requires the current word to be a *file-name*. The statement is ignored.

644 M READ/WRITE/REWRITE NOT ALLOWED FOR REPORT FILES

A file described with a report clause cannot be referenced by any input–output statement except the OPEN or CLOSE statement.

645 S (REPORT) TYPE REQUIRED HERE; SECTION IGNORED

The USE BEFORE REPORTING statement must reference a level 01 report group of TYPE other than DETAIL. The Declaration Section is ignored.

646 S ARRAY ITEM ILLEGAL HERE – OMIT STATEMENT

An array item can not be used at this location in the system.

647 M BEGIN/END LABEL APPLIED TO NON–LABELED FILE; ACCEPTED

The BEGINNING and ENDING LABEL options in the USE statement should not be applied to a file for which LABEL RECORDS ARE OMITTED. The USE statement is accepted.

648 S SYNTAX ERROR IN MOVE STATEMENT; STATEMENT OMITTED

A syntax error occurred in a MOVE statement. The MOVE statement is omitted.

649 W NO (PROCESS) FOR RP SECTION; THUS IT WONT BE EXECUTED

A Random Processing Section (USE FOR RANDOM PROCESSING declarative), which appeared in the Declarative Section, has not been referenced in any PROCESS statements in the program and therefore will never be executed.

650 S TOO MANY EXTERNAL REFERENCES; IGNORE THIS STATEMENT

Each unique entry-point-name appearing in a CALL statement is placed in the EXREF Table, which also contains any names appearing in APPLY EXREF clauses. This table allows a maximum of 100 names. The entry-point-name in this CALL statement exceeds that maximum, and the statement is ignored.

651 S RP SECTION NAME REQUIRED HERE; PROCESS STMNT OMITTED

The PROCESS statement requires the name of an Random Processing Section (USE FOR RANDOM PROCESSING declarative) at this point. The PROCESS statement is omitted.

652 S ILLEGAL MOVE SOURCE OR DESTINATION NAME; MOVE OMITTED

This word does not represent an allowable data item in a MOVE statement (in other words, it is a *file-name*, a condition-name, a hardware mnemonic-name, an *index-name*, or an index item). The MOVE statement is omitted.

653 M DANGLING ELSE/WHEN/OTHERWISE; TREATED AS AN IMPERATIVE

Each ELSE clause, in a sequence of ELSE clauses, must be paired with an IF statement in a nested fashion. A sequence of WHEN clauses must be preceded by a SEARCH statement. Each OTHERWISE clause must be paired with an ON statement. This ELSE, WHEN, or OTHERWISE is not preceded by the appropriate statements and is therefore treated as an imperative statement.

654 S CONDITIONAL REQUIRES IMPERATIVE HERE; COND. IGNORED

The conditional sentence being used requires that an imperative statement appear at this point. The entire conditional sentence is omitted.

655 S SYNTAX ERROR IN (NEXT SENTENCE) CLAUSE; CLAUSE OMITTED

A syntax error occurred in a NEXT SENTENCE phrase. The clause is omitted.

656 S (SEARCH) REQUIRES (INDEXED) ITEM HERE; SEARCH OMITTED

The SEARCH statement requires the current word to be a dimensioned item with an INDEXED clause in its OCCURS clause. The SEARCH statement is omitted.

657 S (SEARCH) REQUIRES (WHEN) HERE; SEARCH OMITTED

The SEARCH statement requires a WHEN clause at this point. The SEARCH statement is omitted.

658 M (NEXT SENTENCE) ILLEGAL HERE; IGNORED

The NEXT SENTENCE phrase is not permitted at this point. It is omitted.

659 S USER NAME REQUIRED HERE; STATEMENT OMITTED

The syntax requires a *user-name* at this point rather than a keyword or a literal. The containing statement is omitted.

660 s LITERAL MUST BE A POSITIVE INTEGER; STATEMENT OMITTED

A numeric literal that is either zero, negative, or not an integer has been used in a place where a positive integer is required. The statement containing the invalid literal is omitted.

661 W SEPARATE SENTENCE SHOULD BE USED FOR NOTE AND MONITOR

The MONITOR statement should be in a sentence by itself. It is accepted here.

662 S (ALTER) REQUIRES PARAGRAPH NAME HERE; ALTER OMITTED

The ALTER statement requires this word to be a paragraph name. The ALTER statement is omitted.

663 S PARAGRAPH OR SECTION NAME REQUIRED HERE; OMIT STATEMNT

The syntax of the PERFORM, ALTER, or GO statement requires this word to be either a paragraph or a *section-name*. The statement is omitted.

664 S OPERAND IS TOO LARGE; OMIT STATEMENT

It is illegal for an item whose size is more than 18 characters to be used as an operand in an arithmetic statement. The statement is omitted.

665 S ILLEGAL ARITHMETIC OPERAND; STATEMENT OMITTED

It is illegal for an alphanumeric (including numeric edited) item to be used as an operand in an arithmetic statement.

666 M RESERVED WORD USED AS PARAGRAPH OR SECTION NAME IGNORE

It is illegal to use a reserved word for a paragraph or *section-name*.

667 F ATTEMPT TO PROCESS NON–RP SECTIONS; FATAL ERROR

The object of a PROCESS statement is not the name of a USE FOR RANDOM PROCESSING Declarative Section. Code generation is suppressed unless the A option is present on the @ACOB processor call statement.

668 S (TO/FROM/BY/GIVING/INTO) MISSING; STATEMENT OMITTED

This arithmetic statement violates syntax rules and needs one of the keywords (TO, FROM, BY, GIVING, or INTO) to be legal. The statement is omitted.

669 S ILLEGAL ARITHMETIC RESULT; STATEMENT OMITTED

It is illegal for an item (other than numeric edited) whose size is greater than 18 characters to be the result field in an arithmetic statement. The statement is omitted.

670 S MISSING DESTINATION IN ALTER STATEMENT; OMIT STATEMENT

An ALTER statement lacks an object following the TO. The ALTER is omitted.

671 S PROGRAM FLOW WILL NOT REACH THESE STATEMENTS

The EXIT PROGRAM, STOP RUN, and simple GO statement must be the last statement in the sentence. The further statements in the sentence are accepted but they can never be executed.

672 W PROCEDURE REFERENCE FROM OUTSIDE DECLARATIVES;ACCEPTED

The main body of the Procedure Division must not reference procedures contained in the Declarative Area. This reference is accepted.

673 S PROCEDURE NAME OR (DEPENDING) REQUIRED HERE; OMITTED

The current word in a GO...DEPENDING statement must either be another procedure name or the key word DEPENDING. The error is probably a misspelled procedure name. The GO...DEPENDING statement is omitted.

674 S (DEPENDING) IDENTIFIER MUST BE INTEGER; OMIT STATEMENT

The identifier in the DEPENDING clause must be an integer numeric field. The GO ...DEPENDING statement is omitted.

675 S INTEGER LITERAL OR IDENTIFIER REQUIRED HERE; OMITTED

The syntax requires an integer numeric literal or identifier at this point. The statement is omitted.

676 M WORD FOLLOWING (STOP) ASSUMED TO BE (RUN)

The word following STOP must either be a literal or the keyword RUN. The keyword RUN is assumed.

677 W (STOP RUN) SHOULD NOT BE USED IN SUBPROGRAMS;ACCEPTED

STOP RUN statements should not be present in called programs (ENTRY POINTS in the Procedure Division header) because they will prevent control from returning to the calling program at object time. The STOP RUN statement is accepted.

678 W (EXIT PROGRAM) SHOULDN'T BE USED IN MAIN PROG; OMITTED

EXIT PROGRAM statements should not be present in a main program (no ENTRY POINTS in the Procedure Division header) because the code generated for them is intended to return control to a calling program, not to the system. The EXIT statement is omitted.

679 S (GENERATE/INITIATE/TERMINATE) ILLEGAL HERE AND OMITTED

It is illegal for a USE BEFORE REPORTING declarative section to have a GENERATE, INITIATE or TERMINATE statement in it. Any such statements are omitted.

680 S REPORT ITEM REQUIRED HERE; STATEMENT OMITTED

This word in a GENERATE statement must be either a TYPE DETAIL report group name or a report-name. The statement is omitted.

681 S REPORT NAME REQUIRED HERE; STATEMENT OMITTED

The identifiers in an INITIATE or TERMINATE statement must be report-names. The statement is omitted.

682 S NON–NUMERIC LIT OR FIG CON REQUIRED AFTER (ALL); OMIT

The word following ALL must be an alphanumeric literal or a figurative constant (in an IF statement it can also be a Saved-Area condition word, ACTIVE or INACTIVE). The containing statement is omitted.

683 M (ALL) NOT PERMITTED HERE; STATEMENT OMITTED

The syntax does not permit an ALL literal at this point. The containing statement is omitted.

684 Not issued.

685 S SA MUST APPEAR WHEN 2 OR MORE EXIST; OMIT STATEMENT

The PROCESS statement must include a USING saved-area-block or record clause. The PROCESS statement is omitted.

686 S SA NAME OR RECORD REQUIRED HERE; STATEMENT OMITTED

The syntax requires either a saved-area-block name or a saved-area-record name at this point. The containing statement is omitted.

687 S SORT FILE NAME REQUIRED HERE; STATEMENT OMITTED

The RELEASE statement requires a sort-file-record name at this point. The RELEASE statement is omitted.

688 S SORT–FILE NAME REQUIRED HERE; RETURN STATEMENT OMITTED

If a *user-name* appears immediately after the word SORT, it must be a sort-file name, and the syntax of a RETURN statement requires a sort-file name here. The statement is omitted.

689 S SYNTAX ERROR IN SORT/MERGE STATEMENT; STATEMENT OMITTED

A syntax error occurred in the SORT/MERGE statement. The statement is omitted.

690 S ILLEGAL (SORT) KEY; STATEMENT OMITTED

The KEY items in a SORT statement must be contained in a sort–file record, if a sort–file is named, or in a USING–file record, if no sort–file is named. The SORT statement is omitted.

691 M ILLEGAL SORT PROCEDURES; ACCEPTED

The SORT INPUT PROCEDURE and SORT OUTPUT PROCEDURE both must be a section or a range of sections. This paragraph or range of paragraphs are illegal, but accepted.

692 S ERROR IN SORT/PERFORM RANGE; STATEMENT OMITTED

A syntax error occurred in stating the procedure range in a SORT or PERFORM statement. The statement is omitted.

693 S ILLEGAL FILE TYPE FOR SORT/MERGE STATEMENT; OMIT STMT

A CD, SA, SD, random, or dynamic file is not allowed with the USING/GIVING phrase of the SORT/MERGE statement and a giving file may not have ORGANIZATION INDEXED specified in the FILE–CONTROL paragraph.

694 S DEVICE USAGE AND STATEMENT INCOMPATIBLE; OMIT STMT

The I–O statement contradicts one of the FILE–CONTROL, I–O–CONTROL or FD clauses previously stated for the file. Examples are: OPEN INPUT, OPEN I–O, OPEN EXTEND or READ of a file ASSIGNed to CARD–PUNCH or PRINTER; OPEN OUTPUT, OPEN I–O or WRITE of a file ASSIGNed to CARD–READER; WRITE ... ADVANCING phrase for a file not ASSIGNed to PRINTER; END–OF–PAGE phrase for a file with no LINAGE clause; OPEN EXTEND of a file that is specified in a MULTIPLE FILE TAPE clause. The illegal statement is omitted.

695 S SYNTAX ERROR IN SET STATEMENT; SET IS OMITTED

A syntax error has occurred in a SET statement. The SET statement is ignored.

696 S MISSING INDEX NAME ITEM OR INTEGER; STATEMENT OMITTED

Either the VARYING clause of a SEARCH statement or the TO object of a SET statement is missing an *index-name* or integer. The SET or SEARCH statement is ignored.

697 S UNEXPECTED PROGRAM END; INDICATES PREVIOUS ERROR

The end-of-program delimiter (after the last image of the source) has been reached at a point where the syntax analyzer is unprepared for it. This indicates some prior catastrophic syntax error which interfered with a normal recovery.

698 S INCOMPATIBLE (SET) OPERANDS; STATEMENT OMITTED

In a SET...UP BY or SET...DOWN BY statement, all of the operands preceding UP or DOWN must be index names. In a SET...TO statement, the operands preceding TO must all be of one type; the type permitted depends upon the type of operand that follows TO. This SET statement contains an invalid combination of operands and the statement is omitted.

699 W POSSIBLE RIGHT TRUNCATION OF RECEIVING FIELD

One of the operands in an arithmetic statement has a larger scale (more decimal positions) than the result field. This means that some digits will be truncated from the right end of the result value which might have affected the result field if ROUNDED had been specified. Since ROUNDED is not specified, these digits are ignored.

700 W ROUNDED CLAUSE MEANINGLESS HERE; IGNORED

In the ADD or SUBTRACT statement, none of the operands has a larger scale (more decimal positions) than the result field. Therefore, there can be no digits to truncate at the right end, and the ROUNDED option cannot affect the result field. The ROUNDED option is ignored.

701 S ILLEGAL (SET) OPERAND; STATEMENT OMITTED

Every operand in a SET statement must be either an *index-name*, an index item, an integer item, or an integer literal. The SET statement is omitted.

702 S AN (FD) RECORD REQUIRED HERE; STATEMENT OMITTED

The syntax requires a File Section level 01 record at this point. The statement is omitted.

703 S SYNTAX ERROR IN (ADVANCING) OPTION; STATEMENT OMITTED

A syntax error occurred in the ADVANCING option of a WRITE statement. The statement is omitted.

704 S DATA ITEM REQUIRED HERE; STATEMENT OMITTED

The syntax requires a data item at this point. The following are not permitted: a literal, *file-name*, *condition-name*, mnemonic–name, *index-name*, or index item. The containing statement is omitted.

705 W RESULTANT SIGN STRIPPED UPON STORAGE

One (or more) of the operands in this MOVE or arithmetic statement is signed (PICTURE includes S) but the receiving operand is unsigned. This means that the absolute value of the result value will be stored, which might mean that a minus sign is thrown away.

706 S (AT END/INVALID KEY) MISSING; STATEMENT OMITTED

Every American National Standard 1968 COBOL READ statement must have either an AT END clause or an INVALID KEY clause in it. The READ statement is omitted.

707 S (LOCK/UNLOCK) IDENT IMPROPERLY DEFINED; OMIT IDENT

Each identifier in a LOCK or UNLOCK statement must be a 1–word binary field. The illegal identifier is omitted.

708 S RP OR IDENT USED IN (LOCK/UNLOCK); IDENT OMITTED

The identifier in a LOCK or UNLOCK statement must not be contained in a saved-area record or a random processing file record. The illegal identifier is omitted.

709 M TOO MANY PARAMETERS FOR EXHIBIT; IGNORE PARAMETER

A maximum of 14 items may be specified in an EXHIBIT CHANGED statement, 9 in an EXHIBIT NAMED or NAMED CHANGED. Parameters in excess of this limit are ignored. The problem can be resolved by writing multiple EXHIBIT statements.

710 S (INVALID KEY) UNNECESSARY OR NON–KEYED FILE; OMIT STMT

An INVALID KEY clause can appear in a READ, WRITE, REWRITE, START or DELETE statement only for a KEY file (RECORD/ACTUAL/RELATIVE KEY clause in FILE–CONTROL paragraph; access can be RANDOM, DYNAMIC). The statement is omitted.

711 M ARITHMETIC OPERATOR ILLEGAL WITH SEARCH KEY; ACCEPTED

Conditional statement in this SEARCH statement is limited to relation equal or single value *condition-name* condition. Arithmetic expression in condition is accepted.

712 S MOVE CORR – SAME SENDER AS RECEIVER – STATEMENT OMITTED

The SENDER and RECEIVER have the same name and the same qualification.

713 S SYNTAX ERROR IN COMPUTE STATEMENT; STATEMENT OMITTED

A syntax error occurred in a COMPUTE statement. The statement is omitted.

714 S (CORRESPONDING) OPERANDS MUST BE GROUP; OMIT STATEMENT

All the operands in a CORRESPONDING clause must be group items so there can be at least one matching pair of subordinate items. The statement is omitted.

715 S (EXAMINE/INSPECT) IDENT MUST BE DISPLAY; STMT OMITTED

This identifier on the EXAMINE or INSPECT statement must be character not binary. USAGE must be DISPLAY or DISPLAY–1. The statement is omitted.

716 S SYNTAX ERROR IN EXAMINE/INSPECT; STATEMENT OMITTED

A syntax error occurred in an EXAMINE or INSPECT statement. The statement is omitted.

717 M (EXAMINE) LIT WITH MORE THAN ONE CHARACTER; ACCEPTED

All the literals in an EXAMINE statement must be 1-character literals. This statement contains a multicharacter literal, but the statement is accepted.

718 S SYNTAX ERROR IN CONDITIONAL STATEMENT; STMNT OMITTED

A syntax error occurred in a conditional statement. The statement is omitted.

719 S SYNTAX ERROR IN OPEN; STATEMENT OMITTED

A syntax error has occurred in an OPEN statement. A common error is omitting the keyword INPUT or OUTPUT. The statement is omitted.

720 M (REVERSED/NO REWIND) MEANINGLESS; IGNORED

The REVERSED clause of the OPEN statement cannot be specified for a nonsequential file or for a file whose open mode is output, I–O or extend. The NO REWIND clause of the OPEN or CLOSE statement cannot be specified for a nonsequential file or for a file whose open mode is I–O or extend. The clause is ignored.

721 S PERIOD MISSING IN PREVIOUS CONDITIONAL

A period is missing in a conditional expression. The end of the paragraph has been detected before the period which should end the conditional expression. The statement is accepted but it may cause errors in code generation.

722 S USE STATEMENT NOT ALLOWED FOR LION/CFH FILES

Declarative processing not allowed for LION or CFH files. The USE statement will be omitted.

723 S NO SEARCH KEY IN PREVIOUS RELATIONAL; OMIT STATEMENT

The conditional in a SEARCH statement must test a KEY field in the table. The entire SEARCH statement is ignored.

724 S PREVIOUS ERRORS CAUSE LOSS OF OBJECT CODE

No code is generated for the current statement due to previous errors.

725 S ARRAY USED AS SUBSCRIPT; STATEMENT OMITTED

The data item used as a subscript indicator must not itself be a dimensioned item. The statement is omitted.

726 S SYNTAX ERROR IN PERFORM; STATEMENT IGNORED

A syntax error occurred in a PERFORM statement. The statement is omitted.

727 S ILLEGAL SYNTAX FOLLOWING UNARY + OR –; OMIT STATEMENT

The only words, literals, or punctuation that can legally follow a unary (+) or (–) are: an elementary data item, a numeric literal, and a left parenthesis. The containing statement is omitted.

728 S ILLEGAL SYNTAX FOLLOWING EXPONENTIATION; OMIT STMNT

The only words, literals, or punctuation that can legally follow an exponentiation symbol (**) are: an elementary data item, a numeric literal, and a left parenthesis. The containing statement is omitted.

729 S ILLEGAL SYNTAX AFTER MULTIPLY/DIVIDE; OMIT STATEMENT

The only words, literals, or punctuation that can legally follow the symbols (*), (/), MULTIPLIED BY, or DIVIDED BY are: an elementary data item, a numeric literal, and a left parenthesis. The containing statement is omitted.

730 S ILLEGAL SYNTAX AFTER PLUS OR MINUS; STATEMENT OMITTED

The only words, literals, or punctuation that can legally follow the symbols (+) or (–) are: an elementary data item, a numeric literal, and a left parenthesis. The containing statement is omitted.

731 S (TO,GIVING) MAY NOT APPEAR TOGETHER; STATEMENT OMITTED

The syntax "ADD A TO B GIVING C" is illegal; either TO or GIVING but not both can be stated. The ADD statement is omitted.

732 S MNEMONIC–NAME OR ALPHABET–NAME REQUIRED; OMIT STATEMENT

After "ACCEPT ... FROM", "DISPLAY ... UPON" or "COLLATING ... IS" the next word must be a mnemonic–name defined in the SPECIAL–NAMES paragraph.

733 S DATA ITEM OR LITERAL REQUIRED HERE; STATEMENT OMITTED

The syntax requires either a data or a literal at this point. Keywords or other types of names (*file-name*, *condition-name*s, mnemonic–names, *index-names*, and index items) are not legal. The statement is omitted.

734 S CALL NAME MUST BE 'LIT' OR ALPHANUMERIC;STMT OMITTED

The entry-point in a CALL, ENTER, or CANCEL statement must be expressed as an alphanumeric literal or as an identifier. The statement is omitted.

735 S SYNTAX ERROR IN EXPRESSION; OMIT STATEMENT

The only words, literals, and punctuation that can legally follow a left parenthesis are: an elementary data item, a numeric literal, a unary (+), a unary (–), and another left parenthesis. The containing statement is omitted.

736 S RELATIONAL OPERATOR MISSING HERE; STATEMENT OMITTED

The syntax of a simple condition requires a relation operator (equal, less, greater, etc.) at this point. The entire conditional statement is omitted.

737 S CONDITIONAL OPERATOR MISSING HERE; STATEMENT OMITTED

The syntax of a conditional statement requires a simple condition at this point. A simple condition is one of the following; relation test, class test, zero test, *condition-name* condition, or Saved–Area condition. The entire conditional statement is omitted.

738 S CONDITIONAL EXPRESSION ILLEGAL HERE; STATEMENT OMITTED

The special conditions are: ON SIZE ERROR, AT END, INVALID KEY, AT END– OF–PAGE, WHEN, AT NONE, and ON ALL INACTIVE. The syntax of each of these special conditions requires that an imperative statement follow which does not allow a conditional expression. The conditional expression is omitted.

739 S CONDITIONAL EXPRESSION REQUIRED HERE; OMIT STATEMENT

The keywords IF, WHEN, and UNTIL must all be followed by a conditional expression, which must begin with a data item, a literal, a left parenthesis or a unary sign. The containing statement is omitted.

740 M PROGRAM NAME EXCEEDS 12 CHARACTERS; MIDDLE OMITTED

The entry-point-name in a CALL, CANCEL, or ENTER statement must be 12 or fewer characters in length. The excess characters in the literal are omitted. The first six and last six characters are used.

741 Not issued.

742 W SUBJECT AND CLASS TEST INCOMPATIBLE; ACCEPTED

The ALPHABETIC class test should not be performed on a numeric item, and the NUMERIC class test should not be performed on an edited item, and neither class test should be performed on a binary item. The test is accepted here.

743 S RELATION CONDITION HAS NO SUBJECT; STATEMENT OMITTED

A relation condition with a missing subject occurred (for example, IF A EQUAL B ... IF EQUAL C). This is legal if a full relation test occurred earlier in the conditional statement, from which the subject can be "borrowed". In this case, no earlier subject exists. The conditional statement is omitted.

744 S IMPLIED SUBJECT AND OBJECT ABBREVIATION ILLEGAL; OMIT

A relation test with an implied subject and object (for example, IF A EQUAL B ... IF GREATER ...) is not permitted. The conditional statement is omitted.

745 W SUBJECT AND OBJECT ARE BOTH NUMERIC LITERALS; ACCEPTED

A relation test in which the subject and object are both numeric literals (for example, IF 7 GREATER THAN 5) does not make sense. It is accepted here.

746 M ILLEGAL COMPARE INDEX–NAME/ITEM WITH NON–INDEX; OMIT

It is illegal to compare an index item (USAGE INDEX) with anything but another index item or an *index-name*. The relation test is accepted here. The index item is treated as a binary integer item.

747 S ILLEGAL COMPARE WITH ALPHANUMERIC ITEM; OMIT STMNT

It is illegal to compare an alphanumeric item to a binary item or to compare an alphanumeric group item to a numeric expression. The conditional statement is omitted.

748 M MISSING RIGHT PAREN; ASSUMED TO BE HERE

A compound condition has apparently ended with an insufficient number of right parentheses, in other words, the condition has more left than right parentheses. The missing right parentheses are assumed to be present and the condition is accepted.

749 S CONDITIONAL OR EXPRESSION TOO LARGE; STATEMENT OMITTED

Each nested pair of parentheses (logical or algebraic) in the conditional sentence or COMPUTE statement causes an entry in a parentheses-level table. The maximum number of entries allowed in this table has been exceeded. The conditional sentence or COMPUTE statement must be broken into smaller statements. Subsequent fatal errors may occur as a result of this error. If these fatal errors still occur after the statement has been broken into smaller statements, please submit a UCF.

750 S ILLEGAL (CALL) PARAMETER; STATEMENT OMITTED

It is illegal for a USING parameter in a CALL statement to be a keyword, a *condition-name*, a mnemonic-name, an *index-name*, or an index item. The parameter is omitted.

751 S BINARY OPERAND ILLEGAL HERE; OMIT STATEMENT

Binary items may not be the subject of NUMERIC or ALPHABETIC class tests. The entire statement containing the illegal condition is ignored.

752 M (MESSAGE/SEGMENT) REQUIRED HERE; ASSUME MESSAGE

The RECEIVE statement requires, after the cd-name, the keyword MESSAGE or the keyword SEGMENT. The keyword MESSAGE is assumed.

753 S (FROM) REQUIRED HERE; STATEMENT OMITTED

The syntax requires the keyword FROM at this point. The containing statement is OMITTED.

754 S SYNTAX ERROR IN (STRING/UNSTRING); STATEMENT OMITTED

A syntax error occurred in a STRING or UNSTRING statement. The statement is omitted.

755 M MORE THAN 49 QUALIFIERS; EXCESS TRUNCATED

A qualified reference must not have more than 49 qualifiers. The excess qualifiers are truncated.

756 M DUPLICATE PROC NAME; FIRST DEFINITION CHOSEN

A *procedure-name* reference requires qualification for uniqueness. The first occurrence of *procedure-name* in Procedure Division is assumed.

757 M PARAGRAPH NAME ILLEGAL AS QUALIFIER; QUALIFIER OMITTED

Only a *section-name* can appear as a qualifier in a reference to a *paragraph-name*. The indicated character-string is a *paragraph-name* used illegally as a qualifier. The qualifier is omitted.

758 M ILLEGAL TO QUALIFY SECTION; QUALIFIER OMITTED

A *section-name* cannot be qualified in a reference. The indicated character-string is an illegal qualifier for a *section-name*. It is omitted.

759 M COMPOSITE OPERAND GREATER THAN 18 DIGITS; ACCEPTED

The composite of the operands in an arithmetic statement is an artificial decimal item with just enough integral and just enough fractional positions of all the operands. For exact binary operands the largest number of decimal digits that can be represented by the binary size is used as a decimal equivalent with the provision that 30–36 bits gives 10 digits, and 57–72 bits gives 18 digits. It is illegal for the composite to exceed 18 digits. The arithmetic statement is accepted, and, at object time, digits will be truncated from the left end of the operands with excess integral digits.

760 M ILLEGAL MNEMONIC NAME; CONSOLE ASSUMED

The syntax requires a mnemonic-name which has been associated with a valid device-name in the SPECIAL–NAMES paragraph. The default device-name CONSOLE is assumed, and the ACCEPT/DISPLAY statement is accepted.

761 M SEARCH KEY IN PREVIOUS OBJECT; ACCEPTED

A key name should not appear as an object in a SEARCH statement. The statement is accepted.

762 M ENTRY POINT IS DUPLICATE PARAGRAPH NAME; USE 1ST DEF

The names in the WITH ENTRY POINTS clause in the Procedure Division header must be *section-name*s or unique *paragraph-name*s. The indicated name is a duplicated *paragraph-name*. The first definition is used.

763 M ENTRY POINT NAME 12 CHARACTERS; MIDDLE OMITTED

The entry-point names in the WITH ENTRY POINTS clause in the Procedure Division header must not exceed 12 characters externally. The excess characters are omitted for external reference purposes (the first six and last six characters are used). The full name is used internally.

764 M ENTRY POINT NAME APPEARS MORE THAN ONCE; IGNORE THIS

The same name appears twice in the WITH ENTRY POINTS clause in the Procedure Division header. The duplication is ignored.

765 M GREATER THAN 100 EXDEF NAMES; EXTRAS ARE IGNORED

The entry-point names in the WITH ENTRY POINTS clause in the Procedure Division header are placed in an EXDEF table, which also contains any names mentioned in an APPLY EXDEF clause in the I–O CONTROL paragraph. The EXDEF table holds a maximum of 101 names. Excess names are not entered.

766 W ALPHANUMERIC ITEM REQUIRED HERE; NUMERIC ITEM ACCEPTED

The syntax requires an alphanumeric item (includes all edited items) at this point. The numeric item is accepted.

767 W IMPERATIVE REQUIRED HERE; STATEMENT ACCEPTED

The syntax requires an imperative statement at this point in a conditional sentence. The sentence is accepted, and treated as if it had a null imperative statement.

768 W EDITING OF THIS ITEM IGNORED HERE

The editing specified in an item's description is relevant only when that item appears as a receiving item in a MOVE or arithmetic statement. In other statements, or in input operand positions, the editing is ignored. Specifically, no de-editing of input is implied. The editing item is treated as an alphanumeric item.

769 S (DISPLAY) USAGE REQUIRED HERE; IGNORED

The INSPECT and STRING statements require a character item (USAGE DISPLAY or DISPLAY–1) at this point; a binary item is illegal. The statement is omitted.

770 S SYNTAX ERROR IN (ENABLE/DISABLE) STATEMENT

A syntax error occurred in an ENABLE or DISABLE statement. The statement is ignored.

771 S OPERAND EXCEEDS 18 CHARACTERS; STATEMENT OMITTED

It is illegal for any arithmetic operand (other than an edited receiving field) to be more than 18 characters in size. The arithmetic statement is omitted.

772 S MISSING LINAGE CLAUSE IN (FD); STATEMENT OMITTED

The END–OF–PAGE phrase in the WRITE statement is legal only for a file which has a LINAGE clause in its File Description paragraph. The WRITE statement is omitted.

773 S ILLEGAL SEARCH ALL CONDITION; STATEMENT OMITTED

The SEARCH ALL condition may be either a single equal relational or a *condition-name* condition which is associated with a single value. Other conditional expressions are invalid and cause the statement to be ignored.

774 S UNDEFINED DATA ITEM; STATEMENT OMITTED

The indicated word is expected to be a data item, but it is not defined anywhere in the Data Division. The containing statement is omitted.

775 S PARTIALLY DEFINED DATA ITEM; STATEMENT OMITTED

The indicated word is a data item which is defined incompletely in the Data Division. This generally means that a PICTURE clause was not found. The containing statement is omitted.

776 M PROCEDURE NAME DUPLICATES DATA NAME; ACCEPTED

It is illegal for a *procedure-name* to match a data-name, regardless of qualifiers. The indicated reference or definition of a *procedure-name* matches a data-name; it is accepted and the object code will work correctly.

777 R SENDING DEFINITION LINE NUMBER OF MATCH

The internal line number associated with the message is the line number on which the definition of a matched sending field occurred for a CORRESPONDING ADD, SUBTRACT, or MOVE.

778 R RECEIVING DEFINITION LINE NUMBER OF MATCH

The internal line number associated with this message is the line number of the definition of the receiving field which matches the sending field indicated in the previous line message (777).

779 W DUPLICATE NAME IN CORRESPONDING CLAUSE; 1ST DEF CHOSEN

While searching for matching names subordinate to the group items in a CORRESPONDING clause, the compiler has discovered the same name declared twice with the same hierarchy of qualifiers. The first definition is chosen for one of the matching pairs.

780 M MISSING ARITHMETIC OPERATOR; (MULTIPLIED BY) ASSUMED

The syntax error in the algebraic expression (two consecutive operands or subexpressions with no algebraic symbol between them) appears to be a missing MULTIPLIED BY (*).

781 S ONLY BASIC NUMERIC ITEM ALLOWED HERE; OMIT STATEMENT

An alphanumeric (including edited) item or an index item should not appear in an algebraic expression. It is rejected.

782 S EXPRESSION CONTAINS ILLEGAL ITEMS; STATEMENT OMITTED

Nondata items are not permitted in algebraic expressions. Nondata items are: *file-name*s, *condition-name*s, *index-names*, and mnemonic–names. The containing statement is omitted.

783 S BASIC NUMERIC ITEM REQUIRED HERE; OMIT STATEMENT

The receiving field in an arithmetic statement must be numeric or numeric edited. The alphanumeric receiving field is rejected.

784 S BASIC UNEDITED NUMERIC ITEM REQUIRED; OMIT STATEMENT

The input operands in an arithmetic statement must be numeric (unedited) items. The statement is omitted.

785 S INCOMPLETELY DEFINED CONDITION NAME; STATEMENT OMITTED

The *condition-name* referenced here was incompletely defined in the Data Division; it probably had no literal values. The condition is omitted.

786 S MISSING OBJECT AND RELATIONAL; CONDITION OMITTED

After the subject operand in a simple condition, no relation word or object operand has been found. A possible error is a misspelled or improperly defined *condition-name*. The condition is omitted.

787 S SENTENCE OR STATEMENT TOO LARGE; STATEMENT OMITTED

An internal compiler table overflowed at this point, and the entire sentence or statement is omitted. For a conditional sentence, the solution is to break the compared conditions (ANDs and ORs) or the nested conditions (IF ... IF ... ELSE) into two or more smaller conditional sentences. For an imperative statement, the overflow is caused by multiple operands (MOVE A TO B, C, D ...); the solution is to break the statement into two or more statements with fewer operands.

This error can sometimes occur after DEBUGGING, MONITOR, or subscript range checks have been added to a program. This is because additional entries are created for an internal table to handle these features. As a result of these additional entries, the table can overflow on statements that normally compile without these features.

788 M INDEX NAME OR ITEM ILLEGAL HERE

The syntax does not permit an *index-name* or index item at this point.

789 W EDITED ALPHA TO NUMERIC IS ILLEGAL; ACCEPTED

It is illegal to move an alphanumeric edited item to a numeric item. The sending item is accepted as an integer item.

790 M NON–INTEGER NUMERIC TO ALPHANUMERIC IS ILLEGAL; OMIT

It is illegal to move a noninteger numeric item to an alphanumeric item.

791 S ILLEGAL MOVE TO NUMERIC; OMIT STATEMENT

It is illegal to move a numeric edited item to a numeric item. The sending item is accepted as a numeric item with the scale implied by the edited picture.

792 M NO CORRESPONDING MATCHES FOUND; STATEMENT OMITTED

No matching pairs of subordinate names were found for the two group items in the CORRESPONDING clause. The statement is omitted.

793 S IDENT AFTER SEARCH MUST HAVE AN INDEX; OMIT STATEMENT

The identifier after SEARCH must be a dimensioned item with an associated *index-name*. The SEARCH statement is omitted.

794 M ILLEGAL QUALIFIER; LAST DEFINITION USED

The qualifier is either illegal or not appropriate for this *procedure-name*. The last definition of the qualified name is used.

795 S SYNTAX ERROR IN IF STATEMENT; STATEMENT OMITTED

A syntax error occurred in the IF statement. The statement is omitted.

796 S NOT AN ARRAY; STATEMENT OMITTED

The identifier after SEARCH must contain an OCCURS clause. The SEARCH statement is omitted.

797 M OVER 80 CORRESPONDING MATCHES: EXCESS IGNORED

Each matching pair of subordinate items for the two group items in a CORRESPONDING clause are entered in match table. The match table allows a maximum of 80 matching pairs. The excess pairs are omitted. Code will be generated for the first 80 pairs.

798 W (CALL) PARAMETERS MUST BE (77/01); ACCEPTED

The USING parameters in a CALL statement must be level 77 or level 01 items. The indicated parameter has level 02 or below; it is accepted and will be transmitted correctly providing the item is word–synchronized in its allocation; in other words, the item plus its allocation padding occupies all of one or more computer words.

799 M MORE THAN 31 (USING) PARAMETERS; EXCESS ARE OMITTED

A maximum of 31 USING parameters can appear in a CALL statement. The excess ones are omitted. Problem can be corrected by grouping parameters into a single 01 or by declaring them in the Common–Storage section of both the calling and the called program.

800 S MORE THAN 10 SEARCH KEYS; STATEMENT OMITTED

A SEARCH ALL object array may have no more than 10 key fields. Excessive keys are ignored.

801 W (MONITOR ALL) IN FORCE; MONITOR STATEMENT IGNORED

The current procedure is in the range of a MONITOR ALL statement. Therefore any further MONITOR statements are redundant.

802 S NO KEYS ASSOCIATED WITH THIS ARRAY; STATEMENT OMITTED

The SEARCH ALL statement requires that the searched array be defined with at least one KEY item. The array specified here does not meet this requirement; thus, the SEARCH is ignored.

803 W NUMERIC LITERAL IN INSPECT STATEMENT; ACCEPTED

The syntax requires an alphanumeric literal at this point. The numeric literal is accepted and treated as if it had been written with quotation mark delimiters; however, any (+) or (–) sign or decimal point have been removed.

804 S ONLY MNEMONIC NAME FOR SWITCH 13–14 VALID OMIT STMNT

A syntax error occurred in the SET mnemonic-name statement. The statement is omitted.

805 W (REPLACING) SUBJECT OR OBJECT SIZE ERROR IN INSPECT

In a REPLACING clause in an INSPECT statement, the substring to be replaced is the subject and the substring to insert is the object. Either the subject or the object or both can be literals or character data items. The subject and the object must have the same size in characters. When they differ, the smaller of the two sizes is used and the large item is truncated on the right to the new size. If the CHARACTERS phrase is used, both subject and object must be one character in length.

806 W ELEMENTARY ITEM REQUIRED, STATEMENT ACCEPTED

An elementary item is required in the INSPECT statement. The statement is accepted.

807 M (SUPPRESS PRINTING) ILLEGAL HERE; STATEMENT OMITTED

The SUPPRESS PRINTING statement is legal only when it appears in a USE BEFORE REPORTING Declarative Section. The SUPPRESS statement is omitted.

808 M ILLEGAL (USING) PARAMETER; PARAMETER OMITTED

USING parameters on a Procedure Division header must be basic data items. *File-name*s, *index-names*, report-names, etc., will not be accepted. For the C$INFO call the first USING parameter was not a *file-name* or the second parameter was not a basic data item.

809 S SYNTAX ERROR IN (ON) STATEMENT; OMITTED

A syntax error in the ON statement causes the entire statement to be ignored.

810 M IMPROPER CD NAME FOLLOWS RECEIVE/SEND; ACCEPTED

The identifier after RECEIVE must be an input cd-name. The identifier after SEND must be an output cd-name. The input or output cd-name is accepted, but will probably not work at execution time.

811 W ITEM SHOULD BE ONE NUMERIC DISPLAY; ACCEPTED

The data item after WITH in the SEND statement must be one character in length. The data item is accepted, and at object time only the first character will be used to control the transmission termination.

812 W ITEM IN RANGE OF A PREVIOUS MONITOR; THIS ONE IGNORED

The indicated MONITOR statement contains an identifier which is already in the range of an earlier MONITOR statement and is therefore redundant.

813 W VARIABLE LENGTH ITEM ILLEGAL HERE; TREATED AS INTEGER

A variable length item is not permitted in arithmetic statements. It is accepted, and is treated as a numeric integer item, ignoring the byte-count word.

814 W (DELIMITED BY) CLAUSE MISSING; STATEMENT ACCEPTED

The DELIMITER and COUNT options in the UNSTRING statement are meaningless if no explicit DELIMITED BY clause appears. The statement is accepted.

815 W NO MORE ITEMS CAN BE MONITORED; REST IGNORED

At any point, a maximum of 20 identifiers can be currently monitored. The excess identifiers are ignored. Monitor ALL if more information is necessary.

816 S USE STATEMENT NOT ALLOWED FOR FORM01, 02, 03 FILES

A USE procedure in the DECLARATIVES section will not be executed for any file which has a recording mode of FORM01, FORM02, or F0RM03 with or without the USE phrase.

817 S SYNTAX ERROR IN EXHIBIT; STATEMENT OMITTED

A syntax error in the EXHIBIT statement results in ignoring this statement.

818 S VARIABLE LENGTH ITEM ILLEGAL HERE; STATEMENT OMITTED

Variable length items are illegal in EXHIBIT statements. The entire EXHIBIT statement is ignored.

819 S PRIORITY NUMBER CONFLICT; STATEMENT OMITTED

The rules for referencing procedures among fixed segments, independent segments, and overlay segments are violated by this ALTER or PERFORM. The statement is omitted.

820 S FILE NOT ON MASS STORAGE; STATEMENT OMITTED

The I–O statement implies that this file resides on MASS–STORAGE (for example, DELETE/REWRITE). The statement is ignored.

821 S NO INTERLOCK OR PROTECT SPECIFIED; STATEMENT OMITTED

A READ WITH PROTECT or READ WITH NO PROTECT statement references a file that is not referenced in an APPLY PROTECT or APPLY INTERLOCK clause, respectively. The READ is ignored.

822 S SA NAME OR RP SECTION REQUIRED HERE; STATEMENT OMITTED

A HOLD or ANY/ALL conditional statement specifies illegal operands. The statement is ignored.

823 S RP DECLARATIVE SECTION REQUIRED HERE; OMIT STATEMENT

The object of the PROCESS statement must be a USE FOR RANDOM PROCESSING *section-name*. The PROCESS statement is ignored.

824 S INCORRECT ACCESS MODE FOR THIS VERB; STATEMENT OMITTED

A file mentioned in START must be sequential. The START statement is ignored.

825 S SYNTAX ERROR IN TRANSFORM; STATEMENT OMITTED

A syntax error in this TRANSFORM statement causes the statement to be ignored.

826 S ILLEGAL TRANSFORM IDENTIFIER; STATEMENT OMITTED

The data-names and/or literals in the TRANSFORM statement do not conform to the rules. The statement is ignored.

827 W IMPROPER REFERENCE TO (LOCK) ITEM; ACCEPTED

This USAGE LOCK identifier may be referenced legally only in a LOCK or UNLOCK statement. The reference is accepted and the item is treated as a binary numeric data item.

828 S FILE NAME OR CD NAME REQUIRED HERE; SECTION OMITTED

Operands in the USE FOR RANDOM PROCESSING USING clause must be *file-name*s or
cd-names. The USE section is ignored.

829 S SYNTAX ERROR IN READ; STATEMENT OMITTED

A syntax error in a READ results in the READ being ignored.

830 S MONITOR ONLY STMT IN SENTENCE; THE REST ARE IGNORED

MONITOR must be in a sentence by itself. No other verbs are allowed.

831 S INCOMPLETE KEY TESTS FOR SEARCH ALL; STATEMENT OMITTED

If a SEARCH ALL condition tests an array's KEY field all other keys of greater precedence must also be tested.

832 S STATEMENT AND DEVICE USAGE INCOMPATIBLE; OMIT STMNT

Either the I–O statement or some clause within it contradicts one of the FILE–CONTROL, I–O–CONTROL, or FD clauses previously stated for the file. Examples are: OPEN INPUT, OPEN I–O, or READ of a file ASSIGNED TO CARD–PUNCH or PRINTER; OPEN OUTPUT, OPEN I–O, or WRITE of a file assigned to CARD–READER; WRITE...ADVANCING phrase for a file not assigned to PRINTER; END–OF–PAGE phrase for a file with no LINAGE clause. The illegal statement or clause is ignored.

833 S SAVED AREA NAME REQUIRED HERE; STATEMENT OMITTED

The PROCESS statement USING clause required sa-names or SA 01 level record names as operands. The PROCESS statement is ignored.

834 M LABEL ITEM REFERENCED OUTSIDE DECLARATIVES; ACCEPT

The addresses of label data items are defined only at label processing time. It is, therefore, likely that references to this item will not be correctly executed.

835 W INLINE PROC–NAME–2 FOUND BEFORE PROC–NAME–1

The *procedure-name*–2 specified on an APPLY INLINE statement was found before the corresponding *procedure-name*–1.

836 W INLINE CODE GENERATED–SEQ ERROR OR DUPLICATE PROC–NAME

This means that either the *procedure-name* in the INLINE clause was inadequately qualified or that diagnostic 835 has already been issued and that all specified inline code generation will occur until the end of the source element.

837 W INLINE CODE NOT GENERATED BECAUSE PROC–1 NOT FOUND

A *procedure-name*–1 specified on the APPLY INLINE statement was not found.

838 W ASCII PARAMETER ON ENTER FD OR FORTRAN SUBROUTINE

A parameter specified on an ENTER statement calling an FD COBOL or FORTRAN subroutine is allocated on a quarter word boundary. Since only Fieldata items should be expected, this warning is given even though the item will be passed.

839 W MORE THAN 9 PARAMETERS ON ENTER FD COBOL SUBROUTINE

An FD COBOL subroutine can be passed only nine parameters. If more than this are passed the subroutine may not execute properly.

840 W NUMERIC LIT IN STOP/DISPLAY MUST BE UNSIGNED INTEGER

The numeric literal is accepted, but is displayed as an unsigned integer.

841 W ALPHA TO NUMERIC MAY GIVE UNPREDICTABLE RESULTS

An field being moved to a numeric field may give unpredictable results. The move will be generated.

842 W NON–WORD ALIGNED ARRAY PARAM ON ENTER FD SUBROUTINE

The parameter is an array item which does not start at a word boundary for every occurrence. A Fieldata COBOL subroutine will not be able to handle bit and byte offsets.

843 S SYNTAX ERROR IN USE FOR DEBUGGING; STATEMENT OMITTED

A reserved word is missing or misplaced within a USE FOR DEBUGGING statement. The USE statement is omitted.

844 S ILLEGAL USER ITEM; STATEMENT OMITTED

A *user-name* appearing on a USE FOR DEBUGGING statement is not an identifier, cd–name, *file-name*, or *procedure-name*.

845 M MAXIMUM NO. ITEMS REACHED TABLE OVERFLOW; OMITTED

An internal table has overflowed, forcing the statement to be omitted. This table can overflow as a result of too many *procedure-name*s or data items being DEBUGGED. The number of items that the table can hold is dependent upon the number of characters in the data or *procedure-name*s.

846 M PROCEDURE NAME ILLEGAL – ALL PROCEDURES ENCOUNTERED

The *procedure-name* is ignored since debugging on ALL PROCEDURES has already been specified on either the same USE statement or another USE FOR DEBUGGING statement.

847 M INADEQUATE QUALIFICATION FIRST ENCOUNTER ASSUMED

Inadequate qualification for either a *procedure-name* or identifier has occurred. The first occurrence of the item is assumed.

848 W SIZE OF FIRST RECORD USED FOR INTO CLAUSE

The subject of a READ INTO statement contains more than one record; the size of the first record was used as the size of the receiving field.

849 M D. P. FLOATING DIVIDE REMAINDER NOT POSSIBLE; IGNORED

There is no remainder on a double precision floating divide instruction. The clause will be ignored.

850 M USE PROCEDURE NOT ALLOWED FOR THIS FILE TYPE – IGNORED

The specified USE PROCEDURE DECLARATIVE has not been implemented for this file type. This includes any DECLARATIVE for RECORDING MODE SDF files, or LABEL DECLARATIVE for TAPE, DISC or INTERCHANGE files.

851 S SYNTAX ERROR IN CLOSE STATEMENT – STATEMENT OMITTED

A syntax error has occurred in the CLOSE statement. The statement is omitted.

852 S ERROR IN KEY CLAUSE; STATEMENT OMITTED

Key in Indexed I–O READ/START statement does not match specified file keys.

853 W BINARY TO ALPHA MAY GIVE UNPREDICTABLE RESULTS

Binary values may not have a corresponding match in the valid characters set of the alpha item. The results are unpredictable if a conversion takes place.

854 S ACTUAL KEY ILLEGALLY SUBSCRIPTED – STATEMENT OMITTED

A dimensioned item cannot be used as a symbolic key.

855 S ILLEGAL OPEN – INPUT MODE REQUIRED FOR OPTIONAL FILES

An optional file was illegally opened for OUTPUT or I–O.

856 S TOO MANY LINKAGE SECTION ITEMS FOR CALL/USING; OMIT

Only 11 registers are available for passing Linkage Section parameters on a CALL/ENTER FD, FORTRAN, ASM, or FD ASM statement. If the program is over 65K only 10 LINKAGE SECTION parameters can be used. For any other CALL/ENTER statement, the limit is 31 Linkage Section parameters. The statement is ignored.

857 W PARAMETERS MUST BE IN LINKAGE SECTION; ACCEPTED

PROCEDURE DIVISION USING parameters must be in the Linkage Section.

858 S COMP–3 ITEM NOT ALLOWED AS SORT/MERGE KEY – IGNORED

A data-item referenced as an ASCENDING or DESCENDING KEY is allocated as a packed-decimal item (COMPUTATIONAL–3 usage without the B option in the Extra Options field of the processor call statement). These types of keys cannot be passed to the system sort package.

859 S SYNTAX ERROR IN (SEND) STATEMENT; STATEMENT OMITTED

Format 2 of the SEND statement requires a WITH identifier/ESI/EMI/EGI phrase when a BEFORE/AFTER ADVANCING phrase is used. This SEND statement does not conform to the required syntax and it is omitted.

860 S LINE–COUNTER OR LINAGE–COUNTER CANNOT BE ALTERED HERE

Procedure Division statements cannot modify the value of LINAGE–COUNTER or LINE–COUNTER.

861 W SIGNED ELEMENTARY ITEMS ILLEGAL FOR NUMERICS,ACCEPTED

The numeric test cannot be used with a group item composed of elementary items described as signed. It is accepted.

862 S ILLEGAL PRIORITY NUMBER ON DECLARATIVE SECTION; 0 USED

A segment-number greater than 49 is illegal for a Declarative section; the section will be considered part of the main segment.

863 S SUMMARY REPORT CONTAINS MORE THAN 1 DETAIL OMIT STMNT

In summary reporting, only one detail group is allowed.

864 S REPORT NOT DESCRIBED IN REPORT SECTION – IGNORED

There should be a report description for this report item in the Report Section.

865 S AT END CLAUSE ILLEGAL FOR DYNAMIC ACCESS READ – IGNORED

READ *file-name* AT END is illegal for dynamic access files. Use either READ NEXT *file-name* AT END for sequential retrieval of records, or READ *file-name* INVALID KEY for random retrieval of records. This statement is ignored.

866 M ENTER MASM REQUIRED FOR C$INFO CALL, MASM ASSUMED

The call to the C$INFO function should name MASM. MASM is assumed.

867 W C$INFO CALL EXCEEDS 2 PARAMETERS; EXCESS IS IGNORED

The C$INFO call requires two parameters only. The excess is ignored.

868 W GROUP ITEM ILLEGAL HERE: ACCEPTED

A group item may not be specified as identifier–7 of a STRING statement.

869 W AN INITIAL CLAUSE ON A CD IS ILLEGAL IN A SUBPROGRAM

ANSI COBOL specifies that an INITIAL clause on a CD must not appear in a program that has the USING phrase on the Procedure Division header. Also watch that only one INITIAL clause is allowed per program.

870 S ALPHABET–NAME NOT PERMITTED HERE, STATEMENT OMITTED

The special-name item cannot be used in a transfer statement such as MOVE, ADD, or SUBTRACT.

871 S ILLEGAL DATA ITEM FOR SORT/MERGE DATE KEY-STMT OMITTED

When the DATE reserved word precedes a data name in the ASCENDING/DESCENDING KEY phrase of a SORT or MERGE statement the data item must have a USAGE of DISPLAY and be at least 2 characters in length. In addition, for numeric or numeric edited items, the data item must be unsigned and have no scale. The data item specified does not meet this criteria and the SORT or MERGE statement is omitted.

872 W SEARCH REQUIRES THE OCCURS DATA–NAME HERE

This is a warning that the incorrect data-name has been used in place of identifier–1 in the SEARCH statement. Code is generated.

873 W OCCURS DEPENDING ON NOT ALLOWED HERE–MAX SIZE USED

The operand is a group item which has in its description a subordinate, trailing item containing the OCCURS DEPENDING ON clause. This type of variable length operand is not allowed at this point in the syntax and the maximum length of the group item will be used in the operation instead.

874 S FROM/INTO IDENTIFIER INVALID

The FROM/INTO identifier in a READ, WRITE, REWRITE, RELEASE, or RETURN statement references the same storage area as the record area associated with the file.

875 M UNEQUAL NO. PARAMETERS IN LINKAGE SEC. & USING PHRASE

The number of parameters in the PROCEDURE DIVISION HEADER does not equal the number of parameters processed in the LINKAGE SECTION.

876 S RESERVED WORD USED AS PARAGRAPH OR SECTION NAME IGNORE

A reserved word has been placed in margin A and is followed by a period. It is being used as a paragraph or section name and is ignored.

877 S SYNTAX ERROR IN SEARCH STATEMENT – STATEMENT OMITTED

A syntax error was detected in a SEARCH ALL statement. This message is issued in the case where the AT END and WHEN phrases have been interchanged in the statement.

878 M ILLEGAL VERB/USER NAME FOLLOWS PAR/SEC NAME, IGNORED

This diagnostic is issued when the token following the paragraph or section name is not 'SECTION' or a period.

879 M (FOR REMOVAL) NOT VALID WITH FILE TYPE

The FOR REMOVAL clause on the CLOSE statement is only applicable if the file to be closed is selected TAPE.

880 S INVALID SYNTAX FOR SORT/MERGE FIRST DATE-STMT OMITTED

The correct syntax for the FIRST DATE phrase of a SORT or MERGE statement is:

FIRST DATE IS *literal*

Either incorrect syntax was detected in the FIRST DATE phrase or the *literal* specified was not an unsigned integer between 0 and 99. The SORT or MERGE statement is omitted.

881 W INDEX AND SUBSCRIPT MIX NOT PERMITTED, ACCEPTED

COBOL requires that either INDEXES or SUBSCRIPTS be used alone or with literals and not be mixed. Statement accepted.

882 M ALL ENCOUNTERED FOR SECOND TIME IN A USE STATEMENT

It is illegal for the statement USE FOR DEBUGGING ON ALL PROCEDURES or USE FOR TABLE ERROR REFERENCES ON ALL to appear more than once in a program. The second USE statement is ignored.

883 M IDENTIFIER ILLEGAL – ALL REFERENCES ALREADY ENCOUNTERED

The identifier is ignored since TABLE REFERENCES ON ALL has already been specified on either the same USE statement or on another USE FOR TABLE ERROR statement.

884 F SERIOUS ERRORS IN LARGE IF STATEMENT – NO RB GENERATED

A relocatable element is not generated because serious errors in a large IF statement caused internal information needed to generate correct code for the entire IF statement to be incomplete. The incomplete information could not be discarded because portions of it had already been written out to mass storage from internal tables so as to prevent the internal tables from overflowing.

The remainder of the source program following the large IF statement will still be checked for syntax errors. If the serious errors cannot be removed or if fatal diagnostics are received after removing the serious errors, please contact your local support center so that a UCF can be submitted on your behalf.

885 M SUBSCRIPT RANGE CHECK NOT ALLOWED WITH DEBUGGING MODE

The extra option x or USE FOR TABLE ERROR is not allowed when the WITH DEBUGGING MODE clause is on the SOURCE–COMPUTER clause. In order to use the table handling facility, DEBUGGING must be turned off. In this case table handling is ignored.

886 M UNEQUAL SORT/MERGE RECORD SIZES MAY CORRUPT RECORDS

The actual record size of all logical records involved in a sort or merge must be the same. Unequal record sizes may force truncation of the record or may cause erroneous data to appear as part of the record. This diagnostic also indicates that possible execution errors may occur when the size of the SD record is smaller than the largest input record.

887 S (DECLARATIVES) EXPECTED — NO RB GENERATED

DECLARATIVES in END DECLARATIVES is either missing or misspelled. Compiler will skip to end of source code and will not generate an RB. This error could cause many other error messages.

888 S ILLEGAL SUBSCRIPT USED IN (SEARCH ALL)

An illegal subscript is used as an index name in a SEARCH ALL when it is not the first (or only) index name in the OCCURS clause or it is not an index name.

889 S COMPARISON OF NON–INTEGER TO ALPHANUMERIC IS ILLEGAL

Comparison of a non-integer numeric item to an alphanumeric item is illegal.

890 M LITERAL EXCEEDS MAXIMUM OCCURRENCES

The value of an *index-name* after the execution of a SET ... TO statement must represent an occurrence number of an element in the associated table. For a SET ... (UP/DOWN) BY statement, the value of an *index-name* must represent an occurrence number of an element in the associated table BEFORE and AFTER the execution of the SET statement. The literal specified in the SET statement will cause an *index-name* to exceed the maximum occurrence number of an element in the associated table. Incorrect results may occur.

891 S SUBJECT AND OBJECT ARE BOTH LITERALS; OMIT STATEMENT

A statement that contains a relation condition in which the subject and the object are: alphanumeric literals, or a figurative constant and an alphanumeric literal, or a figurative constant (other than ZERO) and a numeric literal, or a numeric literal and an alphanumeric literal is not accepted here. A relation condition must contain at least one reference to a variable. This statement is omitted.

892 W RESULTANT SIGN STRIPPED UPON STORAGE

One (or more) of the operands in this COMPUTE statement is signed (PICTURE includes S), but the receiving field is unsigned. This means the absolute value of the result will be stored and the operational sign will not be stored.

893 M DATA ITEM SIZE TOO LARGE – MAY GIVE INCORRECT RESULTS

A Fieldata item is greater than 262143 bytes. This exceeds the maximum size allowed by the COBOL compiler internally, and may cause unpredictable results during execution.

894 M MAXIMUM NO. OF ITEMS REACHED TABLE OVERFLOW; OMITTED

An internal table has overflowed forcing the statement to be omitted. This table can overflow as a result of too many array items being checked for subscript out of range. The number of items that the table can hold is dependent upon the number of characters in the data names.

895 W OVERLAPPING OPERANDS MAY CAUSE UNPREDICTABLE RESULTS

The sending and receiving data items share a part of their storage areas. The results after the execution of the MOVE statement are undefined.

896 W COMPARISON OF NON–INTEGER TO FIG CONSTANT; ACCEPTED

A comparison between a non-integer numeric data item and a figurative constant (other than ZERO) is illegal. The statement is accepted.

897 W REVERSED/NO REWIND AND FILE TYPE INCOMPATIBLE; INGORED

The REVERSED or NO REWIND clause is incompatible with implementor–name–1 in the SELECT clause. The NO REWIND clause should not be specified in an OPEN or CLOSE statement for a card-reader, card-punch, printer or sequential mass storage file. The REVERSED clause should not be specified in an OPEN statement for a card-reader card-punch, printer, sequential mass storage, sequential SDF, or sequential disk file. The clause is ignored.

898–900 Not issued.

901 S DUPLICATE EXTERNAL DEFINITION FOUND

Names which are externally defined by this compilation are not unique in the first 12 characters. Either duplicate names appear in the EXDEF clause, or there is a duplication among the program-id name, EXDEF names, WITH ENTRY POINT names, or internal compiler-generated symbols.

902–906 Not issued.

907 F SROR ERROR RETURN

The SYSLIB routine, SROR, returned an error code to ASCII COBOL as the result of receiving an error return from the Executive Request PFWL$ while creating a relocatable output element. The error status returned by ER PFWL$ can be found in register A2 of the A register snap printed prior to this diagnostic. The compilation has been aborted.

908 F ROR ERROR RETURN

The SYSLIB routine, ROR, returned an error code to ASCII COBOL as the result of receiving an I/O error or detecting an internal inconsistency while creating a relocatable output element. The I/O error status returned can be found in register A1 of the A register snap printed prior to this diagnostic. The I/O error is related to the Relocatable Output (RO) file. The compilation has been aborted.

909 F EROR ERROR RETURN

The SYSLIB routine, EROR, returned an error code to ASCII COBOL as the result of receiving an I/O error or detecting an internal inconsistency while creating a relocatable output element. The I/O error status returned can be found in register A1 of the A register snap printed prior to this diagnostic. The I/O error is related to the Relocatable Output (RO) file. The compilation has been aborted.

910 F SSOR ERROR RETURN

The SYSLIB routine, SSOR, returned an error code to ASCII COBOL as the result of receiving an error return from the Executive Request PFWL$ while creating a Collector symbolic output element. The error status returned by ER PFWL$ can be found in register A2 of the A register snap printed prior to this diagnostic. The compilation has been aborted.

911 F SOR ERROR RETURN

The SYSLIB routine, SOR, returned an error code to ASCII COBOL as the result of receiving an I/O error from the SYSLIB routine SDFO while creating a Collector symbolic output element. The I/O error status returned can be found in register A5 of the A register snap printed prior to this diagnostic. The I/O error is related to the Relocatable Output (RO) file. The compilation has been aborted.

912 F ESOR ERROR RETURN

The SYSLIB routine, ESOR, returned an error code to ASCII COBOL as the result of receiving an I/O error or an error return from the Executive Request PFI$ SDFO while creating a Collector symbolic output element. If an I/O error status was returned, it can be found in register A5 of the A register snap printed prior to this diagnostic. If an error status was found from an ER PFI$, the error status can be found in register A2 and register A5 will be zero in the A register snap prior to this diagnostic. The error encountered are related to the Relocatable Output (RO) file. The compilation has been aborted.

913 F TBLWR ERROR RETURN

The SYSLIB routine, TBLWR, returned an error code to ASCII COBOL as the result of receiving an I/O error or an error return from the Executive Request PFI$ while creating a relocatable output element. If an I/O error status was returned, it can be found in register A1 of the A register snap printed prior to this diagnostic. If an error status was found from an ER PFI$, the error status can be found in register A2 and register A1 will be zero in the A register snap prior to this diagnostic. The errors encountered are related to the Relocatable Output (RO) file. The compilation has been aborted.

914 F ILLEGAL RELOCATION TYPE

This is an internal compiler debugging diagnostic. If this message is received, please submit a UCF.

915 Not issued.

916 F ILLEGAL LOC COUNTER FOR TEXT WORD POSITION

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

917 F ILLEGAL LOC COUNTER FOR RELOC OF RIGHT PART TEXT WORD

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

918 F ILLEGAL LOC COUNTER FOR RELOC OF LEFT PART TEXT WORD

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

919 F ILLEGAL RELOCATION OF ADDRESS FIELD IN OVER 65K PROG.

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

920 W OPTIMIZATION FREQUENCY ZERO WHEN OPERAND REFERENCED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

921 F OPZ SUBSCRIPT NOT COMPUTED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

922 F ILLEGAL OP CODE IN TRPT SEQUENCE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

923 F ILLEGAL INST PARAMETER IN RELOC INFO OR OP CODE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

924 F ILLEGAL LIST REQUEST FOR OBJECT LISTING

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

925 W INCONSISTENT BASING REGISTER ALLOCATION – RELOAD BASE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

926 W UNNECESSARY OPTIMIZATION FREQUENCY REGISTER ALLOCATION

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

927 F DUPLICATE INTERNAL LABEL

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

928 F ENTRY POINT PROCEDURE NAME NOT IN EXDEF TABLE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

929 F ILLEGAL INITIAL VALUE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

930 F ILLEGAL DAC INDEXING

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

931 F ILLEGAL TEMP INDEXING

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

932 F NON ZERO ADDRESS IN EXTERNAL REFERENCING INSTRUCTION

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

933 F FORWARD REFERENCE TABLE OVERFLOW

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

934 F EQUATED INTERNAL LABEL ALREADY DEFINED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

935 F ZERO WORD COUNT IN OPERATOR FILE

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

936 F NON–J FACTOR SUBSCRIPT DETECTED BY LOCA/PWD

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

937 F ILLEGAL LOCATION COUNTER DETECTED BY LOCA

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

938 W INCONSISTENT OPTIMIZATION REGISTER ALLOCATION – RELOAD

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

939 F PRFTAB INCOMPLETE PURGE; FORWARD REFS NOT FULFILLED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

940 F ILLEGAL DATA MODULE FILE ENTRY ID

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

941 F REGMEM ID IS PARTIAL RESULT; OSTK SAYS RESULT NOT REG

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

942 F REGMEM ID IS PARTIAL RESULT; OSTK NOT FLUSHED REGISTER

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

943 W UNNECESSARY REGISTER SAVE REQUESTED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

944 F ILLEGAL REGMEM ID DETECTED WHILE FLUSHING

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

945 W UNNECESSARY OPTIMIZATION REGISTER SAVE REQUESTED

If this internal compiler debugging diagnostic is received, please contact your local support center so that a UCF can be submitted on your behalf.

946 F REVERSE MOVE DYMSTK TABLE OVERFLOW

The number of independent moves required to effect a STRING, UNSTRING, EXAMINE, or INSPECT statement exceeds compiler capability. Reduce the complexity of the statement to correct the error. Generally this problem arises when COUNT, POINTER, and TALLY identifiers are not single–precision binary data items.

947 F PARAMETER WORD TABLE OVERFLOW IN INSPECT/UNSTRING

The complexity of the INSPECT or UNSTRING statement caused an internal compiler table to overflow. Code generation for this statement is incomplete or incorrect and may cause an illegal operation halt. Reduce the complexity of the INSPECT or UNSTRING statement to correct the error.

948 F PARAMETER WORD TABLE OVERFLOW IN STRING/DISPLAY

An internal compiler parameter word table overflows while generating code for an extremely complex STRING statement (for example, a STRING statement that contains a large number of DELIMITED identifiers) or for a DISPLAY statement that specifies a large number of operands where at least one of the operands contains an OCCURS DEPENDING ON clause. Code generated for this statement is incomplete or incorrect and may cause an illegal operation interrupt (IOPR) if executed. To correct this error, reduce the complexity of the STRING or DISPLAY statement.

949 S MOVE OF NON–NUMERIC TO NUMERIC GIVES UNDEFINED RESULTS

A move of a figurative constant (other than ZERO, ZEROS, or ZEROES) to a numeric field may have unpredictable results. The move will be generated.

950 F REGISTER OVERFLOW; ALL PROTECTED, NONE AVAILABLE

An extremely complex statement or group of related statements caused all usable registers to be used. Reduce the complexity of the code and recompile. If the error persists, please report it as a possible compiler error.

951 F OSTK ID IS OPZ ON ENTRY TO PREP00

If this internal compiler debugging diagnostic is received, please submit a UCF.

952 F MORE THAN 5 UNCOMPUTED LINKED OPZ ENTRIES

If this internal compiler debugging diagnostic is received, please submit a UCF.

953 F ILLEGAL MODE TRANSFORMATION DETECTED BY PREP00

If this internal compiler debugging diagnostic is received, please submit a UCF.

954 F SENDER OSTK ID IS OPZ; ILLEGAL SITUATION BY PREP00

If this internal compiler debugging diagnostic is received, please submit a UCF.

955 F OPZ ITEM NOT REGISTER OR TEMP; ILLEGAL DAC LOAD LOGIC

If this internal compiler debugging diagnostic is received, please submit a UCF.

956 F PREP00 CALLED TO PREPARE OPT NOT ARRAY, SCALER, INDEX

If this internal compiler debugging diagnostic is received, please submit a UCF.

957 S REFERENCE TO A DECLARATIVE SECTION IN ILLEGAL CONTEXT

A declarative *section-name* may be directly referenced only in a PROCESS statement. The reference is accepted, but, incorrect execution is probable.

958 F ILLEGAL LBLST POINTER (RTL PROG NAME) DETECTED

If this internal compiler debugging diagnostic is received, please submit a UCF.

959 F ILLEGAL OPERAND DETECTED – MALFORMED OPT IN LOCA/MOVG

If this internal compiler debugging diagnostic is received, please submit a UCF.

960 F MALFORMED PROTAB DETECTED

If this internal compiler debugging diagnostic is received, please submit a UCF.

961 F FILE OPERAND EXPECTED, NOT FOUND

If this internal compiler debugging diagnostic is received, please submit a UCF.

962 F BAD FILE TYPE OR LBLST POINTER

If this internal compiler debugging diagnostic is received, please submit a UCF.

963 F CONSTANT OR IDENTIFIER EXPECTED; NOT FOUND

If this internal compiler debugging diagnostic is received, please submit a UCF.

964 F ILLEGAL SOURCE FIELD DETECTED IN STOR00

A previously diagnosed probable error which was accepted conditionally by the Procedure Division syntax analysis has been rejected by the code generator and will cause the production of incomplete or incorrect code. Correct the source error.

965 F FLOATING POINT SOURCE DETECTED IN STORE OPERATION

If this internal compiler debugging diagnostic is received, please submit a UCF.

966 F ILLEGAL CHARACTER TO BINARY MOVE DETECTED

A previously diagnosed probable error, which was accepted conditionally by the Procedure Division syntax analysis, has been rejected by the code generator and will cause the production of incomplete or incorrect code. Correct source error.

967 F SUBSCRIPT NOT COMPUTED FOR STORE OPERATION

If this internal compiler debugging diagnostic is received, please submit a UCF.

968 Not Issued.

969 F ILLEGAL INDEXING OF A CONSTANT IN AN OVER 65K PROGRAM

If this internal compiler debugging diagnostic is received, please submit a UCF.

970 F ILLEGAL PROCEDURE REFERENCE IN A SEGMENTED PROGRAM

If this internal compiler debugging diagnostic is received, please submit a UCF.

971 F INTERNAL LABEL TABLE OVERFLOW – REDUCE STMT COMPLEXITY

An overly complex sentence containing nested IF, PERFORM, or SEARCH statements caused an overflow in a compiler table. Simplify the sentence by reducing the number of such statements. Execution of the code generated will result in an execution interrupt. The diagnostic will point to the last line of the sentence.

972 F FORWARD REFERENCE TABLE OVERFLOW – REDUCE FREFS

An overly complex sentence containing nested IF, PERFORM, or SEARCH statements requires an excessive number of forward references to compiler-generated internal labels. Such references will not be resolved and will produce unpredictable results if executed.

973 S SPRINGBOARD TABLE OVERFLOW: TOO MANY PERFORM PARAGRAPHS

There are too many PERFORM paragraphs and sections in the compilation. Change the source program (decrease the number of paragraphs and sections which are referenced by a PERFORM statement).

974 W MONITOR/DISPLAY/EXHIBIT FIELD TRUNCATED TO 512 BYTES

The data item being monitored or the data items specified in the DISPLAY or EXHIBIT statement is greater than 512 characters. The first 512 characters of the items will be sent to a hardware device.

975 F ILLEGAL TXTTAB POINTER DETECTED IN LIST PHASE

If this internal compiler debugging diagnostic is received, please submit a UCF.

976 F TEMP TABLE OVERFLOW

An internal compiler table which controls the allocation and use of temporary storage has overflowed. This is caused by a combination of global optimization involving large data items, large data items in ACTUAL KEY, ACCEPT, DISPLAY, MONITOR, or EXHIBIT statements, or extremely complex statements. Incorrect or incomplete code will be generated.

977 F MALFORMED OPT REGURGITATED BY PHASE 4 IN IF CLUSTER

If this internal compiler debugging diagnostic is received, please submit a UCF.

978 F ATTEMPT TO POOL A CONSTANT OF ZERO LENGTH

If this internal compiler debugging diagnostic is received, please submit a UCF.

979 F ILLEGAL RECEIVING FIELD IN GROUP MOVE

A group sending item is treated as an alphanumeric item in a move. The specified receiving field is binary and is therefore incompatible. Generated code for this move is incomplete and will probably result in an error termination if executed.

980 F FLOATING POINT LITERAL EXCEEDS SIZE OF RECEIVER

The context in which a floating-point literal is used is such that the literal's binary magnitude is too large (in other words, the exponent is inappropriately large). The value zero is used.

981 F OVERFLOW OF SEGMENT LOAD TABLE FOR DECLARATIVES

More than 100 DECLARATIVE sections with section-number greater than SEGMENT–LIMITS have been encountered.

982 W AT END/INVALID KEY CLAUSE/USE DECL MISSING IN I–O STMT

The INVALID KEY phrase or the AT END phrase must be specified if no applicable USE procedure is specified for the file name in this I–O statement.

983 W INTERMEDIATE RESULT SIZE > 21; INTEGER TRUNCATION

The size determined for an intermediate operation exceeds 21 digits; the integer portion in the intermediate result will be truncated left to right. Refer to arithmetic intermediate results table in 2.6.3.7.

984 W INTERMEDIATE RESULT SIZE > 21; DECIMAL TRUNCATION

The size determined for an intermediate operation exceeds 21 digits; the decimal portion in the intermediate result will be truncated right to left up to the decimal point and then the integer portion will be truncated left to right, if necessary. Refer to arithmetic intermediate results chart in 2.6.3.7.

985 R SENDER LITERAL EXCEEDS PIC SIZE FOR RECEIVER ITEM

On a MOVE of a literal to a numeric item, the literal has more digits than was specified in the picture clause of the receiver.

986–987 Not issued.

988 S LITERAL SPECIFIED IN CONDITION TOO LARGE; CODE OMITTED

The numeric literal specified in a condition is too large for proper code to be generated. This error is most likely caused by attempting to compare an *index-name* to a literal that is greater than the maximum occurrence number of an element in the associated table. Unpredictable results can occur if the program is executed due to incomplete code generation.

989–1000 Not issued.

1001 F I–O ERROR RETURN FROM PREPRO

The system routine, PREPRO, returned an error code. The compilation was aborted.

1002 F CONTROL CARD ERROR, BAD TAPE POSITION, OR UNDEFINED PF

The system routine, PREPRO, returned an error code. The compilation was aborted.

1003 F I–O ERROR RETURN FROM SIR ON OPEN

The system routine, SIR, returned an error condition when attempting to open a file. Probably system or hardware error. The compilation was aborted.

1004 F I–O ERROR RETURN FROM SIR ON CLOSE

The system routine, SIR, returned an error condition while attempting to close a file. Probable system or hardware error. The compilation was aborted.

1005 F CSF$ ERROR – A0 = value FILE NAME

The system routine, CSF$, returned an error condition, in register A0, when the compiler requested that the compiler file, named in the diagnostic text, be assigned or attached. The compilation is aborted. The "A0 = value" is the status code returned. Any explanation of these codes may be found in the OS 2200 Exec System Software Executive Requests Programming Reference Manual (7830 7899).

1006 F I–O ERROR RETURN FROM SIR ON INPUT

System routine, SIR, returned an error condition while attempting to read the source element from the program file or input stream.

1007 W CORRECTION CARDS OUT OF SEQUENCE

The line correction statements are not in ascending order.

1008 F I–O ERROR ON DIAGNOSTIC FILE

An error has occurred while the compiler diagnostic file is being written or read. The compilation was aborted. The error is probably related to a hardware or system failure.

1009 F SDFO ERROR – A5 = value FILE NAME

The system routine, SDFO, which is used to output most of the compiler's internal files, has experienced an unrecoverable I–O error. The "A5 – value" shows the SDFO I–O status code which is explained in the *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899). The name is the compiler's *file-name*. The compilation was aborted.

1010 F SDFI ERROR – A5 = value FILE NAM

The system routine, SDFI, which is used to input most of the compiler's internal files, has experienced an unrecoverable I–O error. The "A5 = value" shows the SDFI I–O status code, which is explained in the OS 2200 Exec System Software Executive Requests Programming Reference Manual (7830 7899). The name is the compiler's *file-name*. The compilation was aborted.

1011 F I–O ERROR ON TABLE FILE

The compiler Table File is used to input and output compiler tables (Name, Constant, Procedure, Description). An I–O error occurred during a read or write of this file. Compilation is aborted. The error is probably related to a hardware or system failure.

1012 F ATTEMPT TO REFERENCE PAST MAX ASSIGNMENT FOR TABLEFILE

The compiler Table File is initially assigned 1000 tracks of mass storage. An extremely complex and long source program requires more allocated storage. The program should be simplified or fragmented into separately compiled subprograms. The current compilation is aborted.

1013 F COMPILATION REQUIRES MORE TABLE SPACE THAN AVAILABLE

Compiler tables have been expanded until no more storage is available. This condition is generally caused by an extremely large program. Subprogramming may be necessary to correct the condition.

1014 F RFTI ERROR – A0 = *xx* FILE NAME

The Basic Service Package subroutine, RFTI (Read File Table Index), has experienced an unrecoverable I/O error while attempting to locate a COBOL COPY element. The *xx* in the A0 – field is the status returned from the Executive request IOW$. These status values are described in the *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899). The *file-name* contained in the diagnostic text is the name of the file being searched for a library element. This name is the internal name attached by a dynamic @USE command to the file's external name. The following table explains the internal and external names of the file being searched.

Internal @USE Name	External File-Name
CFN$$$	Specified file-name
COB$PF	@USE COB$PF.,file-name
C$$$PF	Source input file
ACOB$PROC$$	SYSLIB*PROC$
RLIB$	SYS$*RLIB$

1015 F RPFCPT ERR – A0 = value FILE NAME

The Basic Service Package subroutine, RPFCPT (Read Program file COBOL Procedure Table), has experienced an unrecoverable I–O error while attempting to read a COBOL library element. The A0 value and file name contained in the message are explained in message 1014's description.

1016 F CPTIS ERROR – A0 = value FILE NAME

The Basic Service Package subroutine, CPTIS (COBOL Procedure Table Item Search), has experienced an unrecoverable I–O error while attempting to locate a COPY library element. The A0 value and file name contained in the message are explained in message 1014's description.

1017 F TOO MANY FATAL ERRORS — COMPILATION TERMINATED

It would be meaningless to continue this compilation due to the large number of fatal errors. Therefore, this compilation has been aborted.

1018 F BAD OPTIONS IN FIELD4 OF THE PROCESSOR CALL CARD

A character other than 0–9 or A–Z has appeared in the Extra Option field of the processor call statement.

C.2. Compiler Errors

This section describes the Compiler Errors that cause termination of compilation. These diagnostics appear in the source program listing in the format illustrated and explained as follows.

Format:

```
Column

2       6                                   21      25

***  COMPILER ERROR  nnn  ***
```

where *nnn* is the 3-digit diagnostic number.

The following is a list, ordered by phase, of the diagnostic number along with its meaning.

Phase 0

400	OPENO called with invalid file-id
403	WRFILE called for a file not open for output
404	WRFILE called with an invalid record length
405	CLOSEO called with invalid file-id
406	OPENI called with invalid file-id
407	RDFILE called for a file not open for input
408	CLOSEI called with an invalid file-id
409	DIAGER called with an unassigned diagnostic number
500	Diagnostic message contains an invalid severity designator
501	The length of a diagnostic message is not 56 characters
502	Illegal attempt to expand NAMTAB
503	Illegal attempt to expand PROTAB
504	Illegal attempt to expand CONTAB
505	Illegal attempt to expand DESTAB

GNT, Picture Processor, Phase 2

421	Zero action (in other words, undefined code routine)
422	Stack over/underflow

Phase 3

431	Word count in CONTAB = 0

Phase 4

441 All Phase 4 error conditions

If any of these errors occur, please submit a UCF.

C.3. Execution Diagnostics

This section describes the diagnostics produced during program execution. These diagnostics are output on the program print file (printer or terminal for batch or demand mode execution, respectively) and therefore may be interspersed with program generated writes and displays.

The format of this message is:

Column:

1 6 8

nnnn { W / A / blank } *ddd . . . dd* *fff . . . ff*

where:

nnnn

is the 4-character diagnostic number.

W and A

are severity codes.

W (Warning)

Indicates that an error has occurred that probably affects execution correctness, but the execution is continued.

A (Abort)

Indicates an error which is severe enough that further execution is not possible. The main program, subprograms, and any active ACS cycles are terminated.

blank

Severity is used for several ACS diagnostics for which neither of the above suffice.

ddd ... dd

is the text of the message which may be followed by *fff ... ff*

fff ... ff

is the name or contents of the item or file to which the diagnostic text refers.

The following is a list, in numeric order, of the message number, text, and explanation of execution diagnostics.

0000–1100 Not issued.

1101 A ATTEMPTED TO OPEN FILE WHICH IS ALREADY OPEN

A program logic error results in attempting to open a file twice without an intervening close.

1102 A INVALID EQUIPMENT ASSIGNED TO FILE

The program-assigned device does not correspond to the run stream @ASG statement assignment.

1103 A NO EQUIPMENT ASSIGNED TO FILE

A file which was not selected as OPTIONAL has no associated @ASG statement in the run stream.

1104 A NO DATA AVAILABLE ON ACCEPT

The OS 2200 Executive symbiont which supports the device from which an ACCEPT is expected has no data available.

1105 A ATTEMPTED TO WRITE ON UNOPENED FILE

A program logic error results in a WRITE statement referencing a file which is currently closed.

1106 A ATTEMPTED TO OPEN FILE PREVIOUSLY CLOSED WITH LOCK

A program logic error results in an attempt to open a file which, because of a CLOSE WITH LOCK statement, can no longer be accessed by this run unit.

1107 A ATTEMPTED TO REWRITE RECORD LONGER THAN ORIGINAL

A record which is rewritten must be equal to or shorter than the record it replaces.

1108 A ATTEMPTED TO READ PAST MAXIMUM FASTRAND ASSIGNMENT

The @ASG statement for this file did not specify sufficient space for the data. Revise allocation upward.

1109 W CLOSE REEL SPECIFIED, NO REELS LEFT

Either no data was processed since the last CLOSE REEL, the entire file has already been closed, or no OPEN statement has been executed.

1110 A ATTEMPTED TO WRITE ON FILE OPENED FOR INPUT

A program error resulted in either incorrectly opening a file for input or writing a record to the wrong file.

1111 A UNRECOVERABLE HARDWARE ERROR STATUS (*xx*) ENCOUNTERED ON WRITE

The system was unable to write to the specified file. This resulted in octal I/O error status code xx being returned. See the Exec System Software Executive Requests Programming Reference Manual for a description of the error.

1112 A UNRECOVERABLE HARDWARE ERROR STATUS (*xx*) ENCOUNTERED ON READ

The system was unable to read from the specified file. This resulted in octal I/O error status code xx being returned. See the Exec System Software Executive Requests Programming Reference Manual for a description of the error.

1113 A A WRITE–ONLY FILE OPENED FOR INPUT

A file with an @ASG statement defining it as write-only has been opened for input. This indicates that either a program logic error or an incorrect job stream was set up.

1114 A A READ–ONLY FILE OPENED FOR OUTPUT

A file with an @ASG statement defining it as read-only has been opened for output. This indicates that either a program logic error or an incorrect job stream was set up.

1115 A PREMATURE END OF FILE REACHED ON MULTIFILE TAPE

A file on a multifile tape is not present at the position expected. For example, file four is requested but file three is followed by two tape marks. Either the specified tape is incorrect or malformed, or the position of the file in question was incorrectly stated in the COBOL program.

1116 A ATTEMPTED TO READ AN UNOPENED FILE

A program logic error resulted in a READ statement referencing a file which is currently closed.

1117 W ENDING LABEL NOT FOUND

A malformed tape does not contain an expected EOF label.

1118 W FILE–LIMITS ARE NOT IN SEQUENCE, IGNORED

The FILE–LIMITS clause does not specify values in ascending order. All file-limit values starting with the first value out of order are ignored.

1119 A CLOSE UNIT INVALID

The CLOSE UNIT statement applies only to files assigned to mass storage. The CLOSE statement is ignored.

1120 A READ–ONLY/WRITE–ONLY FILE IS OPENED FOR I–O

A file whose @ASG statement specified read-only or write-only is opened for I–O which could violate the assignment.

1121 A ABNORMAL RETURN FROM AREAD$/AREADA$

An abnormal return from the system routine, AREAD$ or AREADA$, was caused by an attempt to read a control card as data. That is, expected data in the run stream was not present. See the *OS 2200 Exec System Software Executive Requests Programming Reference Manual* (7830 7899) for an explanation of the error code printed with this message.

1122 A READ REVERSE ILLEGAL ON ASSIGNED TAPE UNIT

A file which is opened reversed must be assigned to a tape device which is capable of reading backwards. Reassign the file and rerun to correct the error.

1123 A INVALID KEY CONDITION WITH NO CLAUSE STATED

A program logic error resulted in an INVALID KEY condition being detected with no invalid key action being specified. For example, a WRITE statement does not contain a KEY clause yet causes a record to be placed outside of file limits.

1124 A ATTEMPTED TO OPEN MORE THAN 1 FILE ON MULTIFILE TAPE

When several files exist on the same tape, only one may be open at any point in time.

1125 W FILE IS ALREADY CLOSED

A program logic error results in a file being closed before it is opened. The CLOSE statement is ignored.

1126 A TOO MANY INDEX LEVELS FOR FILE

This error occurs when a program creates small logical records with very large keys, thus causing the index blocks to fill up very rapidly and new index blocks to be created. The error is corrected by using smaller keys and larger logical records. The maximum number of index blocks per file is 100.

1127 A START INVALID WHEN ACCESS IS RANDOM

The START statement can be executed only for ACCESS IS SEQUENTIAL input or I–O files. A source program correction is required.

1128 W REEL NO. IN VOL1 LABEL DOES NOT MATCH REEL ON ASG

The tape reel number in the VOL1 label does not match the reel number on the file's @ASG statement. This may mean an out-of-order reel in a multireel file, a malformed label, a mismounted tape, or an @ASG statement error. The tape is processed as if the error had not occurred.

1129 A FILE DESCRIPTION OR FILE NOT STATED IN USING CLAUSE

A program logic or source error causes this file to be currently unassigned. Probable causes in the COBOL program are a missing FD entry, serious errors in the FD or SELECT statements for the file, or accessing the file in a USE FOR RANDOM PROCESSING section which does not name the file in its USING clause. Correct the source error and recompile to correct the error.

1130 W STANDARD LABEL VALUE & LABEL DISAGREE/NOT COMPARED

The user specified VALUE OF value does not agree with the actual value in a standard label or the user specified VALUE OF value is not compared to the label because the file originator is not COBOL.

1131 A ATTEMPTED TO READ BEYOND END OF FILE

A READ statement has been executed after an AT END condition has been detected. This is probably a program logic error.

1132 A ATTEMPTED TO READ A FILE OPENED FOR OUTPUT

A program logic error resulted in executing READ statement referencing a file which is currently open for output.

1133 A UNRECOGNIZABLE LABEL

A label contains something other than HDR1, HDR2, VOL1, UHL, UVL, UTL, EOF in the initial character positions of the label. The malformed label was possibly caused by a program error during creation, or during hardware or system failure.

1134 W LABEL VALUE DOES NOT AGREE WITH USER LABEL

The user specified VALUE OF value does not agree with the actual value in a user label.

1135 A FILE NOT ASSIGNED TO MAGNETIC TAPE

A file selected and assigned to UNISERVO has been assigned via the @ASG statement to a nontape device.

1136 W BEGINNING LABEL NOT FOUND

A file being processed is missing a HDR1 label. File processing is continued as if the label had been found. The error indicates incorrect processing when the file was created.

1137 A ATTEMPTED TO OPEN A PRINT/PUNCH FILE AS INPUT OR I–O

A program error results in attempting to open an output-only device file for input. This message is usually caused by a misspelled *file-name* or improper assignment of a file.

1138 A ATTEMPTED TO OPEN A CARD–READER FILE AS OUTPUT OR I–O

A program error results in attempting to open an input-only device file for output. This is frequently caused by a misspelled *file-name* or improper assignment of a file.

1139 A ITEM SIZE IN LABEL DOES NOT MATCH RECORD DESCRIPTION

A VALUE OF item in a label (standard or user) does not agree with the size of the item in the label record description.

1140 W BLOCK COUNT DOES NOT MATCH END LABEL

The actual number of data blocks on a file does not agree with the block count value contained in the EOF label created when the file was written. Program error or hardware or system failure could cause this apparent malformed file or label.

1141 W CHECKPOINT FILE NOT ASSIGNED OR NOT ASSIGNED TO TAPE–CHECKPOINT NOT TAKEN

The checkpoint dump file, specified in the RERUN clause, must be appropriately assigned. If a *file-name* was specified, that file's external-name must appear on an @ASG statement. If UNISERVO or MASS–STORAGE was specified, the *file-name* RERUN$ must be the subject of an @ASG statement for the appropriate class of devices.

1142 W CHECKPOINT ERROR CODE (xxxxx) RETURNED FOR FILE

Checkpoint/Restart encountered an error while attempting a checkpoint. The checkpoint is ignored and processing will continue. The checkpoint error code xxxxx is printed in octal. If Checkpoint/Restart is not installed, further checkpoints will be inhibited for this file.

1143 A UNRECOVERABLE HARDWARE ERROR STATUS (xx) ENCOUNTERED ON I–O FILE

The system detected an unrecoverable hardware error while attempting to process an I–O file. The I–O error status xx is printed in octal.

1144 W CHECKPOINT DUMP NOT ALLOWED ON FILES OPENED FOR INPUT

The *file-name* specified in the RERUN clause must be opened for output prior to the execution of a checkpoint dump. No checkpoint dump is written.

1145 W INVALID CREATION/PURGE DATE – DATE IGNORED

The value specified as VALUE OF CREATION–DATE or PURGE–DATE must be valid dates of the form MMDDYY. If the date value is not equal to six characters or if the digits do not correspond to valid dates, today's date is used for both values.

1146 A INTERNAL BLOCK LENGTH FOR THIS FILE DOES NOT AGREE WITH EXTERNAL BLOCK LENGTH ON THIS TAPE

The BLOCK CONTAINS clause value for an input tape file does not represent the true physical characteristics of the file. There is probably an error in a BLOCK CONTAINS specification, a LABEL RECORDS OMITTED clause was specified for a labeled tape, or an illegal block was generated causing the number of words received and the number of words specified in the block control word to differ.

1147 A ATTEMPT TO WRITE BEYOND MAXIMUM FILE ASSIGNMENT FOR FILE

An attempt was made to write beyond the maximum granule assignment for this file.

1148 A DELETE INVALID WHEN OPENED FOR OUTPUT

A delete of a record has been attempted on an indexed file which is opened for output.

1149 A LABEL SPECIFICATION DOESN'T AGREE WITH LABELS ON THIS FILE

A mass-storage file was created by a program in which the label specifications for this file (omitted, standard, or user labels) were different from those specified in the current program. The offending cycle is aborted.

1150 A SEQUENTIAL READ TAPE ERROR

A nonzero status has been received from the OS 2200 Executive after an attempted tape file read.

1151 A SEQUENTIAL WRITE TAPE ERROR

A nonzero status has been received from the OS 2200 Executive after an attempted tape file write.

1152 A THE C$S333 LIST DOES NOT CONTAIN CALL IDENTIFIER NAME: *xxx*

The identifier in a CALL statement or C$S333 list is not properly initialized. The target of the search is *xxx*.

1153 A CLOSE REEL ATTEMPTED ON ILLEGAL UNIT

CLOSE REEL was attempted on a hardware device other than tape.

1154 A WRONG BLOCK TYPE

A particular type of label block was expected but not encountered on the last I–O.

1155 A I–O ERROR NON ZERO STATUS

A nonzero I–O status other than an end-of-file, end-of-tape, or abnormal frame count has been encountered.

1156 A BLOCK SIZE TOO LARGE

The maximum block size specified in the header block when opening a file is larger than the internal calculation of the maximum block size.

1157 A RECORD SIZE TOO LARGE

The maximum record size specified in the record control word when reading a record is larger than the internal calculation of maximum record size.

1158 A BAD DATA BLOCK

There is no stop code on a variable length input record.

1159 A ILLEGAL BLOCK SIZE USED WITH LION FILE

The internal calculated block size is not large enough to be compatible with the DOD–COBOL file size requirement of 14 words.

1160 A BAD STATUS (xx) FROM LABEL$ REQUEST FOR FILE

In a system with the Tape Labeling System turned on, an ER LABEL$ to process a label on an input or output reel returned an abnormal status. The "REEL IS NOT PRELABELLED" status is handled separately and will not be indicated by this diagnostic. The parameter xx is the error status returned from the LABEL$ request.

1161 A REEL IS NOT PRELABELLED

In a system with the Tape Labeling System turned on, an ER LABEL$ or TLBL$ to process a label on an input or output reel returned a status indicating that the reel was not prelabeled. A console message allowing a prelabeled tape to be mounted was answered with an "E".

1162 A IBM FILE ASSIGNED TO PRELABELLED REEL

An IBM output file has not been assigned with the J option and the reel was prelabeled. Use of a prelabeled reel with the Tape Labeling System turned on results in a reel which would be unreadable at an IBM site.

1163 A RECORDS ARE NOT FIXED LENGTH FOR RECORDING MODE F

A file described with the clause RECORDING MODE IS F [AN] is also described with record descriptions of different external lengths. The cycle in error is terminated.

1164 A INCOMPATIBLE ITEM SIZE

The fixed item size specified in the COBOL program does not agree with the fixed item size recorded in the label block.

1165 A SIZE OF DATA BLOCK IS INVALID

The size of the data block created for the input or I–O Indexed sequential file is not consistent in size with the data block used when the file was initially created.

1166 A SIZE OF ACTUAL KEY IS INVALID

The size of the actual key created for the input or I–O Indexed sequential file is not consistent in size with the actual key used when the file was initially created.

1167 W SEQUENTIAL FILE TREATED AS DIRECT

The internal file and the external file types are not identical but they are compatible. This message informs the user that the file in question will be treated as direct.

1168 A INTERNAL/EXTERNAL FILE TYPES ARE INCOMPATIBLE

The internal file and external file types are not identical and are not compatible. The user should check the file type (direct, sequential, etc.) on the assigned device and check the SELECT clause in the program and make the necessary changes.

1169 A CARD IMAGE LARGER THAN MAXIMUM RECORD LENGTH

A card image has been read which exceeds the largest record described for this file.

1170 A IBM FILE NOT ASSIGNED TO NINE TRACK UNIT

An attempt has been made to open a file with RECORDING F, U, or V and the tape is not assigned to a 9-track tape drive.

1171 W BAD STATUS (*xx*) FROM DFP IO$ REQUEST FOR FILE

In an attempt to read a Define File Block from the file DFP$, an abnormal I–O status *xx* occurred.

1172 W BAD STATUS (*xx*) FROM DFP PFS$ REQUEST FOR FILE

In an attempt to search file DFP$ for an omnibus element for use as a Define File Block, an abnormal PFS$ status *xx* occurred. The "no find" or "deleted" status is not included as abnormal.

1173 W NESTED SUBPROGRAM CALL: *xxx*

Subprogram *xxx* was reentered prior to exit.

1174 A BAD I–O STATUS (*xx*) DURING TLBL$ REQUEST

In an attempt to process an 8-bit tape label via a TLBL$ request, an abnormal I–O status *xx* occurred.

1175 A BLOCK SPECIFICATION NOT AS ON LABEL OF FILE

A mass-storage file being opened for input or I–O was created with a blocking description different from what is now being used.

1176 A I–O 05 ERROR STATUS WHILE PROCESSING LABELS

An attempt has been made to open a file for I–O which has never been written to.

1177 A I–O 05 ERROR STATUS WHILE READING DIRECT FILE

An attempt has been made to read a Direct file sequentially for a record that was never written, in other words, the file was created in random access mode. If the condition occurs on a random access read, an invalid key is signaled.

1178 A BAD STATUS (*xx*) FROM TLBL$ REQUEST FOR FILE

In an attempt to process an 8-bit tape label, an abnormal TLBL$ status *xx* occurred.

1179 W THE SUBPROGRAM BEING CANCELED IS STILL ACTIVE: *xxx*

Subprogram *xxx* has been canceled while it is still active.

1180 A TAPE LABELLING SYSTEM REQUIRED FOR LABELLED FILE

An attempt to open a 1968 Standard internal tape file which specifies LABEL RECORDS ARE STANDARD or data-name is terminated because the OS 2200 Executive does not include the Tape Labeling System.

1181 A WRITE OF TAPE LABELS VIA LABEL$ NOT ALLOWED

On Executive level 35 or later, an attempt to open for output a 1968 Standard internal tape file which specifies LABEL RECORDS ARE STANDARD or data-name is terminated because the reel contained 9-bit labels. A 9-bit labeled reel may only be used for input on this level of the Executive.

1182 A FAILED LABEL CHECK, ACCESS TO FILE NOT ALLOWED

An attempt to open a 1968 Standard internal tape file which specified LABEL RECORDS ARE STANDARD or data-name was terminated because the Tape Labeling System detected an inconsistency between the label on the reel and the manner in which the file was assigned.

Possible inconsistencies include differences between the reel label and the assigned file with respect to FILE–ID, FILE–QUALIFIER, FILE–ACCESS, or PURGE–DATE.

1183 W C$INFO FOUND FILE NOT OPEN OR NOT MSAM FILE

The routine C$INFO found that the file was not open, or the file is not a MSAM file. The user area has been zero filled before return of control.

1184 W C$INFO FOUND PARAMETER LIST IN ERROR

The routine C$INFO found that the parameter list was in error. The user area has not been altered before return of control.

1185 A ACCEPT IMAGE > SIZE OF A SINGLE DATA TRANSFER

Buffer space is allocated to accommodate the maximum size of a single data transfer. If this area overflows it will overlay whatever follows. See 2.6.4.4 for general rules for the ACCEPT statement to determine maximum data transfer sizes.

1186 A INAPPROPRIATE IBM HARDWARE TRANSLATE SPECIFIED

For IBM tape hardware translate, the RECORDING MODE must be F AN or U AN and the @ASG statement must specify ASCII for PROCESSOR code, EBCDIC for TAPE code, and Q for data transfer FORMAT.

1187 W MAXIMUM NO. OF BUFFERS EXCEEDED, MAXIMUM USED

The user has created a Define File Block for an Indexed I–O file and has specified a value for *bn* (the number of buffers) which is too large. The maximum number of buffers (63) is used.

1188 W TOO FEW BUFFERS SPECIFIED, MINIMUM USED

The user has created a Define File Block for an Indexed I–O file and has specified a value for *bn* (the number of buffers) which is too small. The minimum number of buffers (4) is used.

1189 W OVER 65K MODE IS REQUIRED FOR HVTIP SUBPROGRAM

COBOL requires that HVTIP subprograms be compiled with the OBJECT–COMPUTER paragraph containing the clause "MEMORY SIZE 3 MODULES" in order to allow D-bank addresses to become X-register based.

1190 A THIS SUBPROGRAM'S D–AREA IS MISSING:

COBOL's D-bank parameter address is too small. This can occur if the subprogram's D-bank element is not included (for example, IN RO$$$$SYSDTA) in the program's collection.

1191 W WRONG NUMBER OF USING OPERANDS FOR SUBPROGRAM

The number of USING parameters on the CALL statement does not agree with the number of USING parameters on the PROCEDURE DIVISION header.

1192 A DFP SPECIFIED BLOCK SIZE EXCEEDS MAXIMUM

A Define File Processor specified block size parameter caused a calculated block size to exceed the 5-digit field allowed in the label block.

1193 A END–OF–FILE MARK NOT FOUND WHILE POSITIONING TO FILE

While attempting to position a multiple file tape to a file not the first on the tape, a hardware tape mark status was not returned as expected during execution of a forward space file function (FSF$).

1194 A BAD EOF MARK COUNT ENCOUNTERED WHILE POSITIONING FILE

While attempting to position a multiple file tape to a file not the first on the tape, an inappropriate number of hardware tape marks were encountered. Three forward space file functions (FSF$) had been issued from the load point and a FITEM$ file's extended count did not match an expected number.

1195 A J OPTION ON @ASG, LABEL RECORDS MUST BE OMITTED

The J option was specified on the assign card for a file assigned to TAPE or INTERCHANGE, but LABEL RECORDS OMITTED was not stated.

1196 A ATTEMPTED A RELEASE OR RETURN WHEN NOT IN SORT OR MERGE

A program logic error resulted in executing a RELEASE or a RETURN statement when not under control of a SORT or a MERGE. The program aborts.

1197 A CSF$ ERROR STATUS (*xx*) ENCOUNTERED ON SYMBIONT

While attempting to close an alternate print file using an ER CSF$, an abnormal CSF$ status *xx* occurred.

1198 W CANCELED SUBPROGRAM NOT AN HVTIP BANK ENTRY POINT – IGNORE CANCEL: *xxx*

In HVTIP, the CANCEL statement can only reference the HVTIP bank entry point and cannot explicitly reference a common banked subprogram or a "local" subprogram (one that resides in the same HVTIP bank as the calling program). The subprogram *xxx* will not be marked as canceled, in other words, the CANCEL will be ignored.

1199 A FILE TYPE NOT RELATIVE OR IIO FOR SHARED FILE

A shared file was defined by specifying the SHARE option (SHARE=YES) with the Define File Processor. However, this type of file cannot be processed by the Shared File System (SFS). To be processed by SFS, the file must be defined as either a Relative file or an Indexed I–O (MSAM) file.

1200 W NO VALID CHARACTERS IN A SINGLE DATA TRANSFER TO CONSOLE

The statement DISPLAY ... UPON CONSOLE attempted to display no valid characters in a single data transfer to the operator's console. The single data transfer is ignored.

1201 A BAD CD DEFINITION OR CD NOT STATED IN USING CLAUSE

A cd-name is referenced by a Random Processing cycle that does not list the cd-name in the appropriate USE FOR RANDOM PROCESSING USING clause, or the CD being referenced had serious errors diagnosed during compilation. Correct the source and recompile to correct the error.

1202 A SYSTEM OR USER ERROR ENCOUNTERED BY MCS—ERROR CODE X

A fatal error was encountered by MCS. The error code, represented by the letter "X" in this message, consists of a single character and will have one of the following values:

@	C$S240 unknown message terminator
C	Illegal function code specified in network interface packet
D	I–O error in queue files
E	Cannot assign queue file
F	MCS network name illegal
H	USER/MPP already signed on network
K	USER/MPP not signed on network
L	Incorrect message header length
M	Message text length exceeds network maximum
O	Cannot free queue file

If the error code value is an @, C, or L, please contact you local support center so that a UCF can be submitted on your behalf.

1203 W MESSAGE RECEIVED WITHOUT A HEADER BLOCK. MESSAGE BYPASSED

A message in a COBOL-processed communication file does not contain header information generated by COBOL-produced messages. The message is ignored. Probable error exists in the message generator/formatter.

1204 A ICP RSEG CANNOT FIT IN HVTIP DBANK

Insufficient D-bank exists to load the RSEG of the Initial Control Program (ICP). This error occurs when the size of the element C$SMCn is less than the actual size of the ICP's RSEG.

1205 W ZERO PARAMETERS REQUIRED FOR XFR$ CALLED SUBPROGRAM: *xxx*

In HVTIP, no parameters can be passed to a subprogram when the subprogram is called via an ER XFR$. An attempt was made to pass parameters to the subprogram *xxx* which was called by an ER XFR$. Zero parameters are assumed.

1206 A ATTEMPTED TO EXECUTE AN ILLEGAL EXIT PROGRAM OR CALL STATEMENT

In the HVTIP environment, an attempt was made to execute an illegal EXIT PROGRAM or CALL statement. The error occurs either while attempting to return from a subprogram via an ER XFR$ or by calling a subprogram via an ER CALL$ which results in the program exceeding the maximum number of nested CALL$'s.

1207 W CANCELED SUBPROGRAM NOT RESIDENT – IGNORE CANCEL: *xxx*

Any subprogram being canceled must reside in memory before the execution of the CANCEL statement. Since the subprogram *xxx* was not resident in memory, the CANCEL will be ignored.

1208 A BLOCK TRUNCATED WHILE WRITING TO TAPE FOR FILE

The Exec indicated that a block being written to tape did not transfer all of the data requested. This situation probably occurred as a result of the file being assigned with quarter-word mode (Q-mode) enabled and a record containing data with a stop bit set (e.g., binary or Fieldata characters). The program is aborted as the file being written will contain corrupted information and probably will be unreadable. To avoid this problem assign the file without Q-mode.

1209–1300 Not issued.

1301 PROGRAM TERMINATED IN ERROR DUE TO PREVIOUS ERROR(S)

This error is caused by a program which destroyed part of a cycle's SAR (the cycle dependent work area) or by invoking the cycle termination routine, C$S320, without first executing the normal epilogue functions. Either may be caused by executing a program whose compilation resulted in S and/or F severity diagnostics. If correcting the source program does not eliminate this error, it should be reported as a possible compiler or system malfunction.

1302 MASTER CYCLE TERMINATED IN ERROR – PROGRAM ABORTED

This message follows a previously issued abort diagnostic for the master cycle.

1303 CYCLE BEING TERMINATED IN ERROR

This message follows a previously issued abort diagnostic for a cycle.

1304 SLAVE CYCLE TERMINATED DUE TO HOST CYCLE TERMINATION

An error condition caused cancellation of a host cycle which therefore required termination of subordinate cycles. The reason for termination of the host cycle is the subject of a previous diagnostic.

1305 INTERNAL PROGRAM DEADLOCK: A0 = A(LOCK ITEM), A1 = A(LOCK PROC)

An internal run–time library error has caused a cycle to attempt to process a locked out internal table which should have been unlocked. This error may be caused by incorrect code generation or by D-bank corruption.

For single cycle programs, an attempt has been made to relock a global table lock cell that was previously locked. For example, an attempt to execute a file operation in the Declaratives Section was made for a file which has previously invoked the declarative error procedure. Registers A0 and A1 may be helpful in determining which lock cell is involved.

1306 A INCORRECT NUMBER OF USING ARGUMENTS ON A CALL TO ERACSF$ OR ERSYMB$

A CALL statement to the ASCII COBOL routine ERACSF$ with zero USING arguments or ERSYMB$ with zero or one USING arguments is illegal. A CALL to ERACSF$ must contain 1, 2, or 3 USING arguments and a CALL to ERSYMB$ must contain 2 or 3 USING arguments.

1307 W INCORRECT NUMBER OF PARAMETERS PASSED TO C$CURDATE

The ASCII COBOL date service routine C$CURDATE requires one parameter. If no parameters are passed, the routine will not store the current date. If more than one parameter is passed, the current date will be stored into the first parameter and the other parameters will be ignored. This message could indicate a source program coding error or indicate the incorrect resolution of the entry point C$CURDATE.

1308-1310 Not issued.

1311 PROGRAM IS DEADLOCKED – PROGRAM TERMINATED

A program logic error, generally due to insufficient LOCK/UNLOCK and HOLD statements, results in each of two or more active cycles possessing control over resources required by the other cycles. Continued execution is impossible. Another cause of this error is the execution of a program that destroys main storage. If correcting S and F source diagnostics and/or deadlocking resource requirements does not eliminate the error, it should be reported as a possible compiler or system malfunction.

1312–1400 Not issued.

1401 A INTERNAL RTL ERROR – NEG. OR ZERO CORE BLOCK REQUESTED

The execution of a program whose compilation produced S or F diagnostics has caused the COBOL run-time system to request additional main storage incorrectly. If source program correction does not eliminate this problem, it should be reported as a possible compiler or system malfunction.

1402 A INTERNAL RTL ERROR – INVALID BLOCK ADDRESS FOR LCORE

The execution of a program whose compilation produced S or F diagnostics has caused a COBOL run-time library program to request the release of previously required main storage incorrectly. If source program correction does not eliminate this problem, it should be reported as a possible compiler or system malfunction.

This error has been known to occur when a subscript is out of range. Before reporting this problem, please verify that a subscript is not out of range (for example, X extra option, USE FOR TABLE ERROR).

1403 A REQUEST FOR ADDITIONAL CORE CANNOT BE SATISFIED – PROGRAM TERMINATED

An extremely large or complex run unit requires more main storage to execute than the system has available. Reduction in the number of concurrently processed files, CDs, or Saved–Areas may be required.

1404 W CYCLE BEING SUSPENDED DUE TO LACK OF CORE

A Random Processing cycle has requested more main storage than is currently available. If sufficient main storage is never made available, the program may eventually deadlock (message 1311).

1405 A NO SUBPROGRAM TABLE FOR A SUBPROGRAM

A run-time system error occurs while executing a subprogram. Please contact your local support center so that a UCF can be submitted on your behalf.

1406 W SUBPROGRAM TO BE IN INITIAL STATE ON NEXT CALL CONTAINS OPEN FILE:

A subprogram is in its initial state:

- The first time a subprogram is called in a run unit.
- The first time a subprogram is called after execution of a CANCEL statement referencing that subprogram.
- Every time a subprogram is called, if it was compiled with the 4 extra options specified on the ACOB processor call statement.

Thus, each file opened in a subprogram must be closed before exiting a subprogram that will be CANCELed or that was compiled with the 4 extra options.

This file was not explicitly closed and the ASCII COBOL run-time system did not implicitly close it. On the next call to this subprogram, this file will be considered as though it was never opened.

1407–1499 Not issued.

1500 W SHARED AREA ALREADY IN USE

Another file specified in the same SAME AREA clause is already in the open mode. The open continues normally. A concurrency count of opened files for each SAME AREA clause is maintained for future checks.

1501 W SHARED AREA USE COUNT IN ERROR

The concurrency count kept for opened files for each SAME AREA clause is out of the valid range of values.

1502 W DEADLOCK DETECTED – NO FILE STATUS ON SHARED FILE

A deadlock status has been detected by the locking subsystem while attempting an I–O function on a shared file. The deadlock status cannot be accessed and appropriate action cannot be taken by the program since no FILE STATUS exists for the file.

1503 W ROLLBACK DETECTED – NO FILE STATUS ON SHARED FILE

A rollback status has been detected by the locking subsystem while attempting an I–O function on a shared file. The rollback status cannot be accessed and appropriate action cannot be taken by the program since no FILE STATUS exists for the file.

1504 A ROLLBACK DETECTED – NO FILE STATUS/USE PROCEDURE ON SHARED FILE

A rollback status has been detected by the Shared File System while attempting an I–O function on a shared file. Since no FILE STATUS and no USE procedures have been specified for the file, the rollback status cannot be accessed and appropriate action cannot be taken by the program.

1505 A DEADLOCK DETECTED – NO FILE STATUS/USE PROCEDURE ON SHARED FILE

A deadlock status has been detected by the Shared File System while attempting an I–O function on a shared file. Since no FILE STATUS and no USE procedures have been specified for the file, the deadlock status cannot be accessed and appropriate action cannot be taken by the program.

1506 W RETURN FILE–NAME DOES NOT MATCH SORT/MERGE FILE–NAME

The RETURN statement's *file-name* does not match the active SORT/MERGE statement's *file-name*.

1507 W RELEASE RECORD–NAME DOES NOT MATCH SORT–FILE RECORD–NAME

The RELEASE statement's record-name does not match the active SORT/MERGE statement's record-name. The record will be released to SORT from the record area associated with the active SORT statement and execution will continue.

1508–1600 Not issued.

1601 A FILE CONCATENATION SPECIFIED, NO DFB FOUND

The previous Define File Processor call contained a nonzero concatenation field but the Define File Block specified could not be located.

1602–2200 Not issued.

2201 W DEPENDING ON VALUE OUT OF RANGE FOR SEARCH ALL STATEMENT AT LOCATION *value*

The current value of a DEPENDING variable is outside the range specified when the table was defined. The defined upper limit rather than the variable's value is used for the SEARCH statement identified by *value* in the message.

2202 A CURRENT VALUE OF DEPENDING ON DATA NAME EXCEEDS SPECIFIED UPPER LIMIT

An attempt has been made to reference an occurrence of an OCCURS DEPENDING ON item beyond the maximum number of occurrences specified when the item was defined.

2203 A CURRENT VALUE OF DEPENDING ON DATA NAME IS ZERO

The current value of a DEPENDING ON variable is zero in a comparison.

2204 A COLLATING SEQUENCE CHARACTER GREATER THAN 0377

During an alphanumeric comparison involving a user–defined collating sequence, a character conversion is attempted in which the character has a value greater than the greatest value defined in the collating sequence; for example, the character has a value greater than 0377.

2205 A REWRITE RECORD SIZE DIFFERS FROM ORIGINAL FOR FILE

For a Sequential, Relative, or Indexed file in sequential access mode, the record to be rewritten must have the same number of character positions as the (original) record from the previous READ statement. The specified file was not updated and execution was aborted because a USE procedure or a FILE STATUS clause was not specified for this file.

2206 A ATTEMPTING AN IIO FUNCTION ON A NON–IIO FILE

An attempt was made to perform an American National Standard 1974 Indexed I–O function on a file which is not an American National Standard 1974 Indexed I–O file.

2207 A I–O BUFFER TOO SMALL FOR IIO TO FUNCTION.

The buffer size is too small to allow an American National Standard 1974 Indexed I–O file to be built.

2208 A INDEXED KEY TABLE IN FILE & IN COBOL PROGRAM DO NOT MATCH

The Indexed Key Table from the file is a different size from that in the COBOL program.

2209 A I–O FILE CAN NOT BE RESTORED AFTER INVALID KEY CONDITION

An INVALID KEY condition occurred while executing a WRITE, DELETE, or REWRITE statement on an American National Standard 1974 Indexed I–O file. The file cannot be restored.

2210 A ATTEMPTING RANDOM READ ON SEQUENTIAL IIO FILE OR VICE VERSA

An attempt was made to perform a random read on an American National Standard 1974 Indexed I–O file which is in sequential access mode or vice versa.

2211 A ATTEMPTING IIO SEQUENTIAL READ WHEN KEY IS UNDEFINED

An attempt was made to perform a sequential read on an American National Standard 1974 Indexed I–O file when the key of reference was undefined.

2212 A START KEY IS LARGER THAN IIO FILE KEY OR EQUALS ZERO

The key specified in a START statement on an American National Standard 1974 Indexed I–O file is either larger than the file's record key or is equal to zero.

2213 A REWRITE IS ILLEGAL ON IIO FILE OPENED FOR OUTPUT

A REWRITE statement is being attempted on an American National Standard 1974 Indexed I–O file which is opened for output.

2214 A REWRITE/DELETE ILLEGAL WITHOUT PRIOR SUCCESSFUL READ FOR FILE

For Sequential, Relative, or Indexed file in sequential access mode, the last operation on the file was not a successful READ statement prior to the execution of a REWRITE or DELETE statement. The specified file was not updated and the execution was aborted because a USE procedure or a FILE STATUS clause was not specified for this file.

2215 A WRITE ILLEGAL ON SEQUENTIAL IIO FILE OPENED FOR I–O

An attempt was made to perform a write on an American National Standard 1974 Indexed I–O file in sequential access mode opened for input-output.

2216 A DELETE/REWRITE ILLEGAL WHEN IIO FILE OPENED FOR OUTPUT

An attempt was made to delete or rewrite a record on an American National Standard 1974 Indexed I–O file opened for output.

2217 A IIO KEY SPECIFICATION NOT CONSISTENT WITH FILE

The user-specified key specification for an American National Standard 1974 Indexed I–O file is inconsistent with the existing file.

2218 A INSUFFICIENT NO. OF I–O BUFFERS FOR IIO

Indexed I–O (PCIOS) requires more I–O buffers to operate. Reserve more I–O buffers for the Indexed I–O file. Refer to the OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual (7831 0588) for guidelines in sizing.

2219 A COMPILER GENERATED KEY PARAMETER IS INVALID

The record key has a length of zero or the start bit is greater than 35. This error is probably caused by invalid RECORD or ALTERNATE key specifications.

2220 A COMPILER GENERATED KEY OF REFERENCE IS INVALID

The key type parameter generated by the compiler is not valid. This error is probably caused by a invalid RECORD or ALTERNATE key specifications.

2221 A FILE CHANGE REQUESTS NOT ACCEPTED; REORGANIZE FILE

The maximum number of index levels has been reached. I–O requests requiring file modification are not accepted. The file requires reorganization. Refer to the *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588) or to the *OS 2200 UDS Shared File System (UDS SFS 2200) Administration and Support Reference Manual* (7831 0786).

2222 A ERROR CODE *xx* RETURNED BY MSAM/ISAM

An undefined error has been encountered while processing an Index Sequential Access Method (ISAM) file or a Multi–index Sequential Access Method (MSAM) file. *xx* is the octal error code returned by the Processor Common Input/Output System or by the Shared File System. Refer to the *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588) or to the *OS 2200 UDS Shared File System (UDS SFS 2200) Administration and Support Reference Manual* (7831 0786) for a description of the error.

2223 A MAXIMUM RECORD LENGTH HAS CHANGED ON INDEXED FILE

During the open of an ISAM or a MSAM file, the current maximum record length was found to differ from the maximum record length when the file was created. The current operation is not performed.

2224 A CURRENT MSAM RECORD LENGTH LESS THAN VALID MINIMUM

The current record length is not long enough to contain the record keys. The current operation is not performed.

2225 A ERROR *xxx* RETURNED BY SFS FOR SHARED MSAM FILE

A Shared File System (SFS) error has been returned while processing an Indexed I/O (MSAM) file where *xxx* is the octal error code. Refer to the *OS 2200 UDS Shared File System (UDS SFS 2200) Administration and Support Reference Manual* (7831 0786) for a description of the error.

2226 W REWRITE RECORD SIZE DIFFERS FROM ORIGINAL FOR FILE

For a Sequential, Relative, or Indexed I–O file in sequential access mode, the record to be rewritten must have the same number of character positions as the (original) record from the previous READ statement. The specified file was not updated, however, execution continued because a USE procedure or a FILE STATUS clause was specified for this file.

2227 W REWRITE/DELETE ILLEGAL WITHOUT PRIOR SUCCESSFUL READ FOR FILE

For a Sequential, Relative, or Indexed I–O file in sequential access mode, the last operation on the file was not a successful READ statement prior to the execution of a REWRITE or DELETE statement. The specified file was not updated, however, execution continued because a USE procedure or a FILE STATUS clause was specified for this file.

2228 W ACTUAL RECORD SIZE UNEQUAL TO PROGRAM RECORD SIZE ON READ OF FILE

The current record pointer specifies a record whose length is not equal to the file's maximum record length when the logical records of the file are described in the source program with only one record description. This situation will cause the FILE STATUS data item to be updated and the record to be truncated if its length exceeds the file's maximum record length. Likely factors causing this diagnostic to be issued for a sequential SDF file are:

- The file is incorrectly described in the source program.
- The external file has been corrupted or created improperly, or
- The program file has been linked (via @ASG or @USE) to the wrong OS 2200 Exec file.

2229 A WRITE/REWRITE/DELETE ILLEGAL FOR IIO FILE

An attempt was made to WRITE, REWRITE, or DELETE, a record on an American National Standard 1974 Indexed I–O file opened for input.

2230 W FILE DEFINED AS SHARED WILL BE PROCESSED AS A NONSHARED FILE

A file has been specified as shared; however, the ASCII COBOL run–time system does not contain the Shared File System (SFS) interface. Therefore, the file must be processed as nonshared by PCIOS.

2231 Not issued

2232 A MAXIMUM GRANULE LIMIT EXCEEDED FOR MASS STORAGE FILE

This COBOL mass storage file type has a maximum granularity specified in positions larger than 4095. The only COBOL file types which allow this are: standard or alternate symbiont files, sequential/direct/device-independent mass storage files specifying a RECORDING MODE of SDF, sequential disk files, or relative disk files.

2233 A SORT/MERGE GIVING FILE RETURN INVALID KEY CONDITION

An INVALID KEY condition occurred for an indexed file during its creation via a SORT ... GIVING or MERGE ... GIVING statement. The error condition was probably caused by a duplicate primary index key or a duplicate key for an index key which does not allow duplicates. No STANDARD EXCEPTION USE procedure was specified for an output operation to the indexed file. Therefore, the ASCII COBOL run–time system is aborting the execution of this program.

2234–2400 Not issued.

2401 W OVERFLOW IN IBM SINGLE PRECISION FLOATING TO UNISYS FLOATING CONVERSION

Input processing of an IBM-formatted file has experienced an overflow condition in conversion to compiler internal floating-point format. That is, the characteristic of the IBM-formatted single precision item is larger than a single precision item can contain. Binary zeros are placed in the converted field.

2402 W UNDERFLOW IN IBM SINGLE PRECISION FLOATING TO UNISYS FLOATING CONVERSION

Input processing of an IBM-formatted file has experienced an underflow condition in conversion to compiler internal floating-point format. Binary zeros are used in the converted item.

2403 W OVERFLOW IN UNISYS DOUBLE PRECISION FLOATING TO IBM FLOATING CONVERSION

Output processing of an IBM-compatible file has experienced an overflow condition in the conversion of COMPUTATIONAL–2 data to IBM floating-point data. Binary zeros are placed in the converted item.

2404 W UNDERFLOW IN UNISYS DOUBLE PRECISION FLOATING TO IBM FLOATING CONVERSION

Output processing of an IBM-compatible file has experienced an underflow condition in the conversion of COMPUTATIONAL–2 data to IBM floating-point data. Binary zeros are placed in the converted item.

2405 A NO DATA RECORDS ARE IDENTIFIABLE

Records which must contain a record selector field do not contain a recognizable value in the location specified as a selector.

2406 A ILLEGAL USAGE DECLARED IN RECORD DESCRIPTION

A program error causes the processing of a record which contains items with a usage that conflicts with the file's recording mode.

2407 W NO SELECTOR EXISTS, FIRST RECORD ALWAYS SELECTED

Records which must contain a record selector field do not contain a record selector field. The first record is selected.

2408 A OCCURS COUNT EXCEEDS LIMIT

An OCCURS DEPENDING ON data-name exceeds the maximum number of occurrences specified for an item in the record description of a COMPACT or IBM file. The cycle is aborted.

2409 W NO LABEL CHECKING PERFORMED ON 9–BIT LABELED FILE

ER LABEL$ is not available on EXEC levels 38 and higher. The labels are now processed by ER IOW$. No verification is performed on the input labels and no data is written on the output labels for the file.

2410 A BAD STATUS (xx) FROM INFO$ REQUEST FOR FILE:

In an attempt to determine tape unit information, an abnormal INFO$ status xx was returned during the execution of an OPEN statement.

2411 Not issued

2412 A TAPE REQUIRES J OPTION ASSIGNMENT FOR IBM FILE

An IBM file has been assigned without the J option when the Exec configuration parameter TLEBCDIC is set to zero. This causes the Tape Labeling System (TLS) to control label processing and write ASCII labels on the tape in addition to ASCII COBOL writing EBCDIC labels. The result is a file that is unreadable. You must assign the file with the J option specified (in other words, @ASG,TJ IBMFILE,U9S//////Q,REEL#).

2413–2500 Not issued.

2501 W EXPONENTIATION ERROR

One of the following exponentiation errors occurred:

1. Base = 0 and power ≤ 0

2. Power * LN (base) > 709.089

3. Base < 0 and power not an integer

The floating-point item is set to binary zeros.

2502–3100 Not issued.

3101 A REPORT ALREADY INITIATED

A program error results in the initiation of a report twice without executing an intervening TERMINATE statement.

3102 A REPORT NOT INITIATED

A program error results in a GENERATE or TERMINATE statement being executed before the report is initiated.

3103 A REPORT NOT INITIATED AT TIME OF A TERMINATE

A program error results when a TERMINATE statement is executed for a report before the report is initiated or after the report has been terminated.

3104 A FILE INITIATED BUT NOT OPENED

The report file must be opened before a report can be initiated.

3105–3200 Not issued.

XX01–XX16 Not issued.

XX17 A START IS ATTEMPTED ON A CLOSED OR OPEN FOR OUTPUT FILE

Executing a START statement is illegal for a file which is closed or opened for output.

XX18 A ERROR RETURN FROM SDF COMMON LIBRARY

The SDF common library has returned in error. The common library will print a message previous to this one which describes the internal SDF error.

XX19 A SDF EXTERNAL RECORD SIZE IS TOO LARGE

The record size on the external media is too large to fit into the record area reserved for this program. Make the input record size large enough to hold the record.

XX20 A A SEQUENTIAL SDF FILE NOT OPEN FOR INPUT OR I–O ON A READ

A read has been attempted on a sequential SDF file which is not opened correctly.

XX21 A DIRECT SDF FILE NOT OPEN FOR INPUT OR I–O ON A READ

A read has been attempted on a direct SDF file which is not opened correctly.

XX22 A A SEQUENTIAL SDF FILE NOT OPEN FOR OUTPUT ON A WRITE

A write has been attempted on a sequential SDF file that is not open for output.

XX23 A NON–SEQUENTIAL ACCESS DIRECT SDF FILE NOT OPEN FOR OUTPUT OR I–O ON A WRITE

A random or dynamic write was attempted on a direct SDF file but the file was not opened correctly.

XX24 A DELETE IS ILLEGAL FOR SEQUENTIAL SDF FILE

Execution of a DELETE statement is not legal for ORGANIZATION IS SEQUENTIAL SDF files.

XX25 A SDF READ NEXT NEEDS ORGANIZATION DIRECT AND ACCESS MODE DYNAMIC

A READ NEXT was attempted on an SDF file that was not defined as ORGANIZATION IS DIRECT and ACCESS IS DYNAMIC.

XX26 A DIRECT SDF FILE ASSIGNED TO TAPE DEVICE IS ILLEGAL

ORGANIZATION IS DIRECT cannot be used on a tape device. It is only meaningful for mass storage devices.

XX27 A REWRITE IS ILLEGAL FOR SEQUENTIAL SDF FILE NOT OPENED FOR I–O

An attempt was made to perform a rewrite on a sequential SDF file not opened for input-output.

XX28 A START IS ILLEGAL FOR SEQUENTIAL SDF FILE

An attempt was made to perform a start on a sequential SDF file.

XX29 A START IS ILLEGAL FOR ACCESS RANDOM DIRECT SDF FILE

START is only legal for direct SDF files with ACCESS IS DYNAMIC/SEQUENTIAL.

XX30 A ZERO KEY IS ILLEGAL FOR DIRECT SDF FILE

The value of an ACTUAL KEY for a direct SDF file or a RELATIVE KEY for a relative disk file is zero. Valid key values are one for the first record, two for the second record, etc.

XX31 A INVALID KEY RETURNED FROM SDF COMMON LIBRARY WHEN NOT EXPECTED

An INVALID KEY return was made from the common library for a function that should not have caused an INVALID KEY condition. This can mean that the file has been corrupted.

XX32 A DELETE IS ILLEGAL FOR DIRECT SDF FILE NOT OPENED FOR I–O

DELETE is not legal for a direct SDF file not opened in the I–O mode.

XX33 A REWRITE IS ILLEGAL FOR DIRECT SDF FILE NOT OPENED FOR I–O

REWRITE is not legal for a direct SDF file not opened in the I–O mode.

XX34 A SEQUENTIAL ACCESS DIRECT SDF FILE NOT OPENED FOR OUTPUT ON A WRITE

A write was attempted on a sequential access direct SDF file that was not opened in the output mode.

XX35 A OPEN I–O IS ILLEGAL FOR A TAPE FILE

OPEN I–O is illegal for sequential SDF files assigned to a tape device.

XX36 A SDF FILE HAS ALREADY BEEN CLOSED OR WAS NEVER OPENED

An attempt was made to close an SDF file that was already closed or was never opened.

XX37 A OPEN REVERSED IS ILLEGAL FOR FILES ASSIGNED TO MASS STORAGE

OPEN REVERSED is illegal for tape files which are assigned to a mass storage device at run-time.

XX38 A VALUE OF ID UNEQUAL TO INPUT FILE STANDARD LABEL

An open input/I–O was attempted on a nontape file with LABEL RECORDS STANDARD and a VALUE OF ID clause created by COBOL. The label does not equal the VALUE OF ID nor all binary zeros (older SDF file). The file originator may not have had a VALUE OF ID clause. The qualifier*external–name of the file up to 17 characters (qualifier truncated on right) is written to the label if no VALUE OF ID clause is present.

XX39 A MISSING VALUE OF ID FOR INPUT SDF STANDARD LABEL

An open input/I–O was attempted on a nontape SDF file with LABEL RECORDS STANDARD created by COBOL. The label is not the qualifier*external-name of the file up to 17 characters (qualifier truncated on right) nor all binary zeros (older SDF file). The file may have been created with a VALUE OF ID clause present.

XX40 A ERROR RETURN FROM ANSI COMMON LIBRARY

The Information Interchange common library has returned in error. The common library will print a message previous to this one which describes the internal Information Interchange error. If the previous error is a TLBL$ error, another message is printed with the error code returned from the TLBL$ request.

XX41 A ANSI EXTERNAL RECORD SIZE IS TOO LARGE

The record size on the external media is too large to fit into the record area reserved for this program. Make the input record size large enough to hold the record.

XX42 A ANSI FILE NOT OPEN FOR INPUT ON A READ

A read has been attempted on an Information Interchange file which was not opened correctly.

XX43 A ANSI FILE NOT OPEN FOR OUTPUT ON A WRITE

A write has been attempted on an Information Interchange file that was not opened in the output mode.

XX44 A ANSI FILE HAS ALREADY BEEN CLOSED OR WAS NEVER OPENED

An attempt was made to close an Information Interchange file that was already closed.

XX45 A ANSI FILE MUST USE MSA Q–MODE

The tape assignment for an Information Interchange file must include the MSA Q–mode (format "A") specification.

XX46 A ANSI FILE CAN NOT BE OPENED REVERSED

The REVERSED phrase must not be specified in the OPEN statement for an Information Interchange file.

C.4. PCIOS Error Messages

For Processor Common Input/Output System error diagnostics, see the *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588).

C.5. Sort/Merge Error Messages

For Sort–Merge error diagnostics, see the *OS 2200 Sort/Merge Programming Guide* (7831 0687).

C.6. OS 2200 Status Key 2 Values

When Status Key 1 contains a 9 (OS 2200 Defined Status), binary values are also contained in Status Key 2. Interrogation of Status Key 2 values is most easily performed through the redefinition of Status Key 2 with a description of:

```
...  PIC 99 USAGE IS COMPUTATIONAL.
```

The exact meaning of each value of Status Key 2 is defined by the corresponding diagnostic indicated in the following table. Field type abbreviations are: T = TAPE, S = SYMBIONTS, SD = SEQUENTIAL DISC, RD = RELATIVE DISC, ID = INDEXED DISC.

Binary Value in Status Key 2	Execution Diagnostic Number	File Types
1	Note 1	
2	Note 1	
3	Note 1	
4	Note 1	
5	1125	T,SD,RD,ID
6	Note 1	
7	1130	ID
8	Note 1	
9	1136	
10	Note 1	
11	1141	T,S,SD,RD,ID
12	1142	T,S,SD,RD,ID
13	1144	T,S,SD,RD,ID
14	1145	
15	Note 1	
16	1171	T,SD,RD,ID
17	1172	T,SD,RD,ID
18	1500	T,S,SD,RD,ID
19	1501	T,S,SD,RD,ID
20	1502	RD,ID
21	1503	RD,ID
22	2226	SD,RD,ID
23	2227	SD,RD,ID
24	2228	SD,RD

NOTE 1: Reserved for future usage.

Status Keys are not implemented for IBM tape files.

Appendix D
Data Allocation

D.1. General

The items discussed in this appendix do not include IBM–compatible data allocation options. (See Section 15.)

D.2. Data Allocation and Alignment

The physical size of the area in the computer's storage and the position of the beginning and ending of the area of any data item is determined by the PICTURE, USAGE, and SYNCHRONIZED clauses which define the item. The JUSTIFIED clause has no affect on allocation, it only effects moves into the field at object time.

Each data item, whether elementary or group, begins and ends on the same class of boundary. The divisions of the 36-bit Series 2200 computer word are shown in Figure D-1.

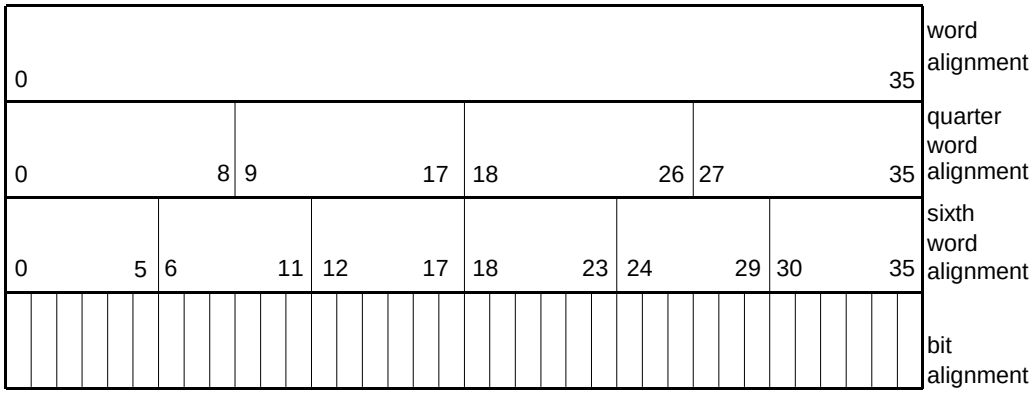


Figure D-1. OS 2200 Word Divisions

The alignment of elementary data items on these four types of boundaries is discussed in the following four sections. The special problems that arise when aligning group items are described in D.2.5.

D.2.1. Word-Aligned Elementary Items

General Rules

1. Word-aligned items use undivided 36-bit words.
2. Level 01 and 77 data items are aligned on word boundaries.
3. Any elementary item with a SYNCHRONIZED clause, USAGE of COMPUTATIONAL–1, COMPUTATIONAL–2, INDEX, or LOCK, will be aligned on word boundaries.
4. COMPUTATIONAL–1, INDEX, and LOCK usage items will cause one word to be allocated.
5. COMPUTATIONAL–2 items will cause two consecutive words to be allocated.
6. SYNCHRONIZED items will allocate the number of words or bytes represented by the PICTURE clause plus enough bits to bring the total allocation up to the next word boundary.
   a. These extra bits are called slack bits.
   b. Slack bits are not included in the size of an elementary item except when the elementary item is the object of a REDEFINES clause or when determining the size of the group containing the item.

c. Slack bits created by synchronization, if any, will either precede or follow the addressable portion of the allocated area depending on the type of synchronization specified (RIGHT or LEFT) or on the type of item (alphanumeric, alphabetic, or numeric). See 2.5.11.

D.2.2. Quarter-word Aligned Elementary Items

General Rules

1. Quarter-word aligned items use a 36-bit word, which is divided in four 9-bit bytes, called quarter words or ASCII bytes.
2. DISPLAY or COMPUTATIONAL usage will cause a data item to be aligned on the next available quarter word boundary.
3. If the SYNCHRONIZED clause is specified for quarter-word aligned items it will cause the data item to be aligned so that it begins and ends on quarter-word boundaries which are also word boundaries and may cause one to three 9-bit slack bytes to be allocated.
4. Disregarding the one, two, or three 9-bit slack bytes which may be allocated because of the SYNCHRONIZED clause, one byte will be allocated for each character position represented in a DISPLAY PICTURE clause (i.e., all PICTURE string symbols except "S", "P", and "V").
5. When the SEPARATE CHARACTER phrase of the SIGN clause is specified for a data item whose usage is DISPLAY, one ASCII byte will be allocated for the "S" symbol specified in the PICTURE clause.
6. Table D-1 shows the number of ASCII bytes allocated for the COMPUTATIONAL usage items.

D.2.3. Sixth-Word Aligned Elementary Items

General Rules

1. Sixth word aligned items use a 36-bit word which is divided into six 6-bit bytes, called sixth words or Fieldata bytes.
2. A USAGE clause of DISPLAY–1 or COMPUTATIONAL–4 will cause the data item to be aligned on the next available sixth word boundary.
3. If the SYNCHRONIZED clause is specified the data item will begin and end on sixth-word boundaries which are also word boundaries and may cause one to five 6-bit slack bytes to be allocated.
4. Disregarding the one, two, three, four, or five 6-bit slack bytes which may be allocated because of the SYNCHRONIZED clause, one byte will be allocated for each character position represented in a USAGE DISPLAY–1 PICTURE clause (i.e., all PICTURE string symbols except "S", "P", and "V").
5. When the SEPARATE CHARACTER phrase of the SIGN clause is specified for a data item whose usage is DISPLAY–1, one 6-bit byte will be allocated for the "S" symbol specified in the PICTURE clause.
6. Table D-1 shows the number of Fieldata bytes allocated for each "9" in a usage COMPUTATIONAL–4 PICTURE clause.

Table D-1. Allocation of COMP and COMP–4 Items

Number of Digit Positions in Picture Clause	COMP Usage Allocation		COMP–4 Usage Allocation	
	Bits	ASCII Bytes	Bits	Fieldata Bytes
1	9	1	6	1
2	9	1	12	2
3	18	2	12	2
4	18	2	18	3
5	18	2	18	3
6	27	3	24	4
7	27	3	30	5
8	36	4	30	5
9	36	4	36	6
10	36	4	36	6
11	45	5	42	7
12	45	5	42	7
13	45	5	48	8
14	54	6	48	8
15	54	6	54	9
16	63	7	60	10
17	63	7	60	10
18	63	7	66	11

D.2.4. Exact Binary Elementary Items

General Rules

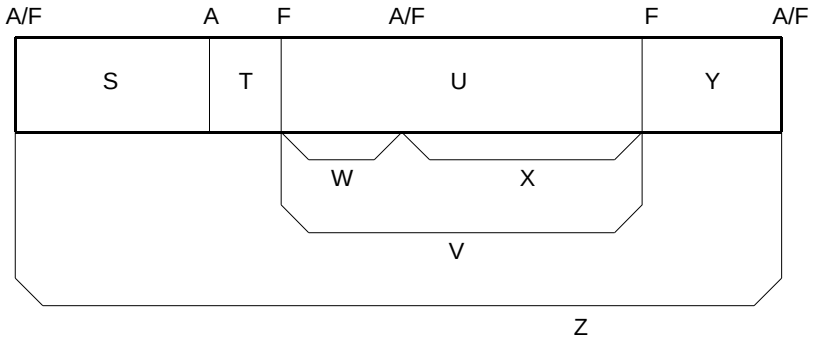
1. Exact binary items are aligned on any of the OS 2200 word's 36 individual bits.
2. An exact binary data item is one whose PICTURE string is composed of a single optional "S" and one or more 1's and is aligned on the next available bit position.
3. If the SYNCHRONIZED clause is specified the data item will begin and end on bit positions which are also word boundaries.
4. Disregarding the 1 to 35 slack bits which may be allocated because of the SYNCHRONIZED clause, one bit will be allocated for each character position of "1" specified in the PICTURE character string and no other slack bits are associated with the elementary item.
5. Since an exact binary item may end at any bit position, slack bits are possible between exact binary items and subsequent elementary items or groups.

6. When redefining an elementary exact binary item by a group item or an elementary item which is not exact binary, the redefined exact binary item must begin and end on the same type of boundary (ASCII or Fieldata) as the redefining group. This rule also applies to renaming an elementary exact binary item with the RENAMES clause.

For example:

```
02 S PIC X.
02 T PIC 1(3).
02 U PIC 1(18).
02 V REDEFINES U USAGE DISPLAY-1.
     03 W PIC X.
     03 X PIC 99.
2  Y PIC X DISPLAY-1.
66 Z RENAMES T THRU Y
```

Results in the following allocation of main storage:



A/F: ASCII or Fieldata boundry

F: Fieldata boundry

A: ASCII boundry

It is advisable to redefine aligned fields by exact binary fields rather than vice versa. The following example allows the programmer to reference the three fields which compose single and double precision floating-point items:

```
02 A COMP-1.
02 B REDEFINES A.
     03 FLOAT-SIGN PIC 1.
     03 FLOAT-CHARACTERISTIC PIC 1(8).
     03 FLOAT-MANTISSA PIC 1(27).
02 C COMP-2.
02 D REDEFINES C.
     03 DFLOAT-SIGN PIC 1.
     03 DFLOAT-CHARACTERISTIC PIC 1(11).
     03 DFLOAT-MANTISSA PIC 1(60).
```

Figure D-2. Data Allocation

Data attribute in Main Storage / usage / meaning / Definition Points	DISP DISP-1 with "T" option ASCII Code	DISP-1 DISP w/ "T" option COMP-3 with "T" option & "B" option Fieldata Code	COMP COMP-4 with "T" option Fixed Point Binary	COMP-1 Single Precision Floating Point Binary	COMP-2 Double Precision Floating Point Binary	COMP-3 Packed Decimal
Working-Storage Linkage Common-Storage	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-2	align: 1/4 word SYNC: 1 word ref-3	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1 word SYNC: 1 word	align: 1/4 word SYNC: 1 word ref-5
Working-Storage Linkage Common-Storage } with "C" xopt	align: 1/4 word SYNC: 1/4 word ref-1	align: 1/6 word SYNC: 1 word ref-2	align: 1/4 word SYNC: 1 or 1/2 word ref-4	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1/4 word SYNC: 2 words	align: 1/4 word SYNC: 1/4 word ref-5
Recording Mode INTERNAL SDF	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-2	align: 1/4 word SYNC: 1 word ref-3	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1 word SYNC: 1 word	align: 1/4 word SYNC: 1/4 word ref-5
Recording Mode COMPACT	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-2	align: 1/4 word SYNC: 1 word ref-3	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1/4 word SYNC: 1 word	align: 1/4 word SYNC: 1/4 word ref-5
Recording Mode IBM	align: 1/4 word SYNC: 1/4 word ref-1		align: 1/4 word SYNC: 1 or 1/2 word ref-4	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1 word SYNC: 2 words	align: 1/4 word SYNC: 1/4 word ref-5
Recording Mode IBM with "A" xopt	align: 1/4 word SYNC: 1 word ref-1		align: 1/4 word SYNC: 1 word ref-3	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits SYNC: 1 word	align: 1/4 word SYNC: 1 word ref-2
Recording Mode CFH LION		align: 1/6 word SYNC: 1 word ref-2				
Recording Mode BLANK/SIGN BLANK SIGN	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-2	align: 1/4 word SYNC: 1 word ref-3	align: 36 bits by 1 word SYNC: 1 word	align: 72 bits by 1 word SYNC: 1 word	align: 1/4 word SYNC: 1/4 word ref-5
Symbiont	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-2				
Recording Mode INTERCHANGE	align: 1/4 word SYNC: 1 word ref-1					

ref-1) 9 bits are allocated for each one of picture string except S, V, and P.

ref-2) 6 bits are allocated for each one of picture string except S, V, and P.

ref-3) 1 ~ 2 digits . . . 9 bits
3 ~ 5 digits . . . 18 bits
6 ~ 7 digits . . . 27 bits
8 ~ 10 digits . . . 36 bits
11 ~ 13 digits . . . 45 bits
14 ~ 15 digits . . . 54 bits
16 ~ 18 digits . . . 63 bits

ref-4) 1 ~ 4 digits . . . 18 bits
5 ~ 9 digits . . . 36 bits
10 ~ 18 digits . . . 72 bits
There are two kinds of SYNCHRONIZATION:
1 ~ 4 digits . . . 1/2 word synchronization
5 ~ 18 digits . . . 1 word synchronization

Data attribute in Main Storage / usage / meaning / Definition Points	COMP-3 with "B" option	COMP-4 COMP with "T" option	INDEX	LOCK	Exact Binary
	ASCII Code Decimal	Fixed Point Binary	Index Data	Operand for LOCK/UNLOCK Statement	Integer Binary
Working-Storage Linkage Common-Storage	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-6	align: 36 bits by 1 word	align: 36 bits by 1 word	align: 1 bit SYNC: 1 word ref-7
Working-Storage Linkage Common-Storage } with "C" xopt	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-6	align: 36 bits by 1 word	align: 36 bits by 1 word	align: 1 bit SYNC: 1 word ref-7
Recording Mode INTERNAL SDF	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-6	align: 36 bits by 1 word	align: 36 bits by 1 word	align: 1 bit SYNC: 1 word ref-7
Recording Mode COMPACT	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-6	align: 36 bits by 1 word	align: 36 bits by 1 word	align: 1 bit SYNC: 1 word ref-7
Recording Mode IBM	align: 1/4 word SYNC: 1/4 word ref-5				
Recording Mode IBM with "A" xopt	align: 1/4 word SYNC: 1 word ref-1				
Recording Mode CFH LION		align: 1/6 word SYNC: 1 word ref-6			
Recording Mode BLANK/SIGN BLANK SIGN	align: 1/4 word SYNC: 1 word ref-1	align: 1/6 word SYNC: 1 word ref-6	align: 36 bits by 1 word	align: 36 bits by 1 word	align: 1 bit SYNC: 1 word ref-7

ref-5) 1 digit . . . 9 bits
2 ~ 3 digits . . . 18 bits
4 ~ 5 digits . . . 27 bits
6 ~ 7 digits . . . 36 bits
8 ~ 9 digits . . . 45 bits
10 ~ 11 digits . . . 54 bits
12 ~ 13 digits . . . 63 bits
14 ~ 15 digits . . . 72 bits
16 ~ 17 digits . . . 81 bits
18 digits . . . 90 bits

ref-6) 1 digit . . . 6 bits
2 ~ 3 digits . . . 12 bits
4 ~ 5 digits . . . 18 bits
6 digits . . . 24 bits
7 ~ 8 digits . . . 30 bits
9 ~ 10 digits . . . 36 bits
11 ~ 12 digits . . . 42 bits
13 ~ 14 digits . . . 48 bits
15 digits . . . 54 bits
16 ~ 17 digits . . . 60 bits
18 digits . . . 66 bits

ref-7) 1 bit is allocated for each on of picture string

D.2.5. Alignment in Group Items

General Rules

1. A group item always begins and ends on the same type of boundary, either quarter or sixth word, even if it contains only exact binary.
2. All level 01 data items are word aligned.
3. Group items which are explicitly DISPLAY or COMPUTATIONAL (i.e. USAGE DISPLAY or COMPUTATIONAL clause specified at group level) will begin on the next available quarter word (ASCII) boundary and will contain an integral number of 9-bit bytes.
4. Group items which are explicitly DISPLAY–1 or COMPUTATIONAL–4 will begin on the next available sixth word (Fieldata) boundary and will contain an integral number of 6-bit bytes.
5. A group item that does not have a USAGE clause associated with it will be considered ASCII unless the first contained elementary non–FILLER item has USAGE DISPLAY–1 or COMPUTATIONAL–4, in which case the group is considered to have USAGE DISPLAY–1 or COMPUTATIONAL–4.

   For example:

   ```
   05 A.
        10 FILLER PIC X DISPLAY-1.
        10 B PIC 1.
   ```

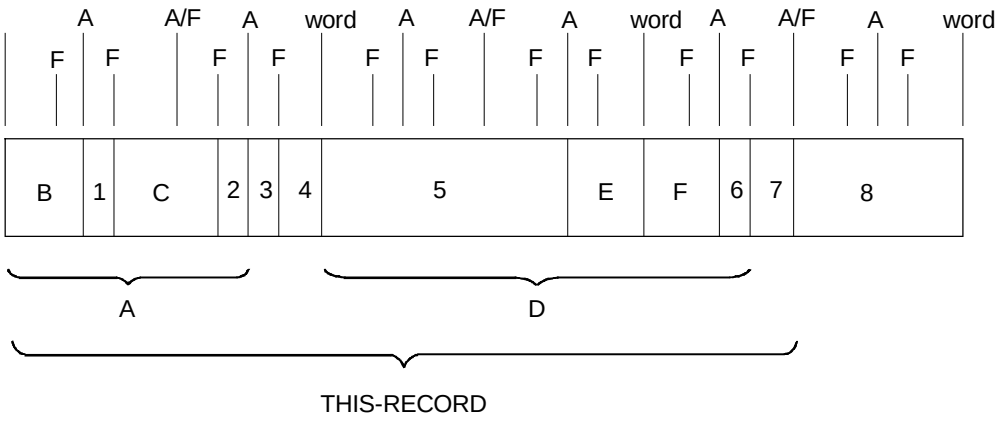
   In this case, the data item A is an ASCII group. This group may not have proper data allocation depending on the group preceding it. See rule 6.
6. When mixing ASCII and Fieldata items in the same group, each group must have explicit usage declared in order to ensure proper data allocation.
7. All the elementary items within a group, defined without explicit usage, take on ASCII characteristics unless the T option is specified on the ACOB processor call statement. In this case, except for exact binary items, they will be considered Fieldata (FD).
8. Group usage is determined from the T option on the ACOB processor call statement only if no group usage is specified and the group contains only FILLER items and condition names.
9. A level 66 item which renames a group or which contains the RENAMES clause with the THRU phrase is a group item whose usage and alignment is determined by the usage of the group being renamed.
10. Slack bits may be created because of the different permissible alignment boundaries of subordinate items. This slack is of importance if it is included in:
    a. A dimensioned item
    b. A group that is renamed
    c. A group that is redefined, or
    d. An application where there is a critical need to reduce the amount of storage required by a program.

11. When a group item has an explicit usage and an item within it has a conflicting usage, slack areas may be allocated which affect the size of the group.

    The following example was constructed to show worst–case alignment, so that the user may avoid the pitfalls implicit in mixing item usages.

```
01 THIS-RECORD.
    02 A.
         03 B PIC X.
         03 C PIC XX DISPLAY-1.
    02 D USAGE DISPLAY-1.
         03 E PIC X DISPLAY SYNCHRONIZED RIGHT.
         03 F PIC 9 COMPUTATIONAL.
01 NEXT-RECORD.
```

    is aligned and allocated as:



    The numbered areas are slack bits generated:

    a. Because of mixed elementary usages.

    b. Because an ASCII group, A, must terminate on an ASCII boundary.

    c. Because a Fieldata group, D, must begin on a Fieldata boundary.

    d. Because of an elementary item, E, usage is in conflict with its group's usage and requires word alignment.

    e. Because the field E is synchronized.

    f. Because a Fieldata group item, D, must terminate on a Fieldata boundary.

    g. Because an ASCII group, THIS–RECORD, must terminate on an ASCII boundary.

    h. Because a 01 level item must begin on a word boundary.

12. An additional way slack may be generated which affects the size of the group is in the alignment of dimensioned items (an item which contains or is subordinate to an OCCURS clause is a dimensioned item).

13. Each occurrence of a dimensioned group is aligned to the next available appropriate byte boundary.

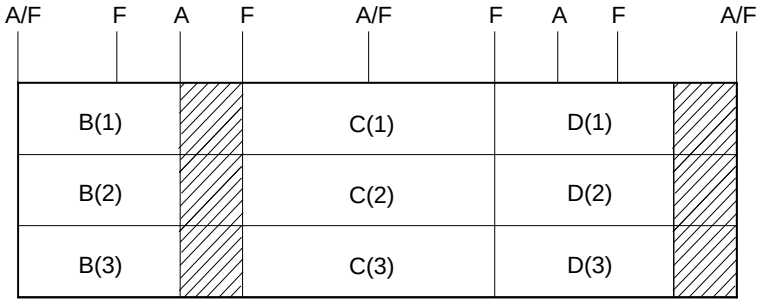
14. If a dimensioned group contains a mixture of 6-bit, 9-bit, and exact binary data items, the size of the group item, including slack must be an integral number of half words (18 bits). When this is not the case, alignment of elementary items in the dimensioned group is not predictable.

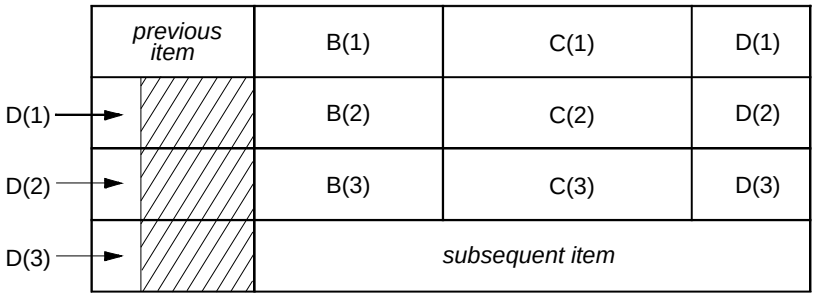
    For example:

    The following data items will be properly allocated because the total size of one occurrence of the group, including slack, is one word:

```
02 A OCCURS 3 TIMES
    03 B PIC X.
    03 C PIC XX USAGE DISPLAY-1.
    03 D PIC 1(9).
```

    The storage allocation for this sequence is one of the following two:





    where:

    A/F

    indicates a half word boundary which is available for either ASCII or Fieldata alignment.

    F

    indicates a Fieldata or sixth word boundary.

    A

    indicates an ASCII or quarter word boundary.

    Which of the two is actually used depends, in a given case, on the initial alignment of the group item. If the table in the preceding example is redefined, it is obvious that the precise location of the slack areas must be determined.

15. Dimensioned groups consisting of homogeneous usage elementary items need not be limited to lengths which are half word multiples.

16. Since level 01 data items, COMPUTATIONAL–1, COMPUTATIONAL–2, LOCK, INDEX, and SYNCHRONIZED items are always word aligned, either:

```
01 NEW-RECORD.
     02 A OCCURS 3 TIMES.
          .
          .
          .
```

    or:

```
02 FLD-1 COMPUTATIONAL-1.
02 A OCCURS 3 TIMES.
     .
     .
     .
```

    or:

    any number of similar means, may be used to ensure a predictable alignment and allocation.

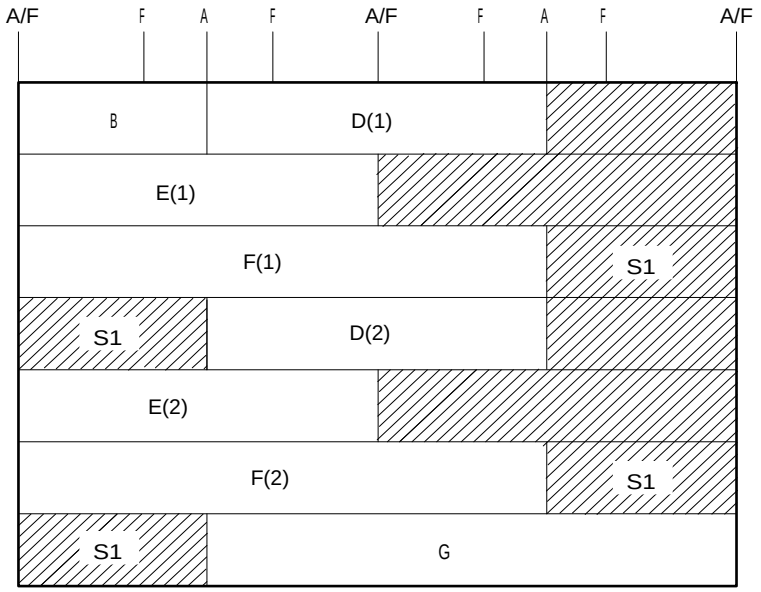
17. The mixture of synchronized and unsynchronized items in a dimensioned group may, even if consistent usages are specified, cause slack areas to be allocated which are of interest when the dimensioned area is redefined or renamed.

18. When a dimensioned group contains implicitly or explicitly synchronized items (SYNCHRONIZED clause on an item description; a usage of COMPUTATIONAL–1, COMPUTATIONAL–2, INDEX, or LOCK), slack bytes are provided between occurrences to ensure that the synchronized fields will always be located in the same relative position in each occurrence.

    For example:

```
01 A.
     02 B PIC X.
     02 C OCCURS 2 TIMES.
          03 D PIC XX.
          03 E PIC 99 SYNCHRONIZED LEFT.
          03 F PIC XXX.
     02 G PIC 999.
```

    will be allocated as follows.



The slack areas labeled S1 in the example ensure the proper positioning of the items in the dimensioned group. This area is not counted in determining the size of C(1) or C(2), but is, of course, used in determining the location of C(2), D(2), or G.

If MOVE ZERO TO C(1), C(2) were executed, ASCII zeros would be moved to all locations beginning with D(1) and ending with the last character of F(2) except for the two byte locations between F(1) and D(2) and between F(2) and G.

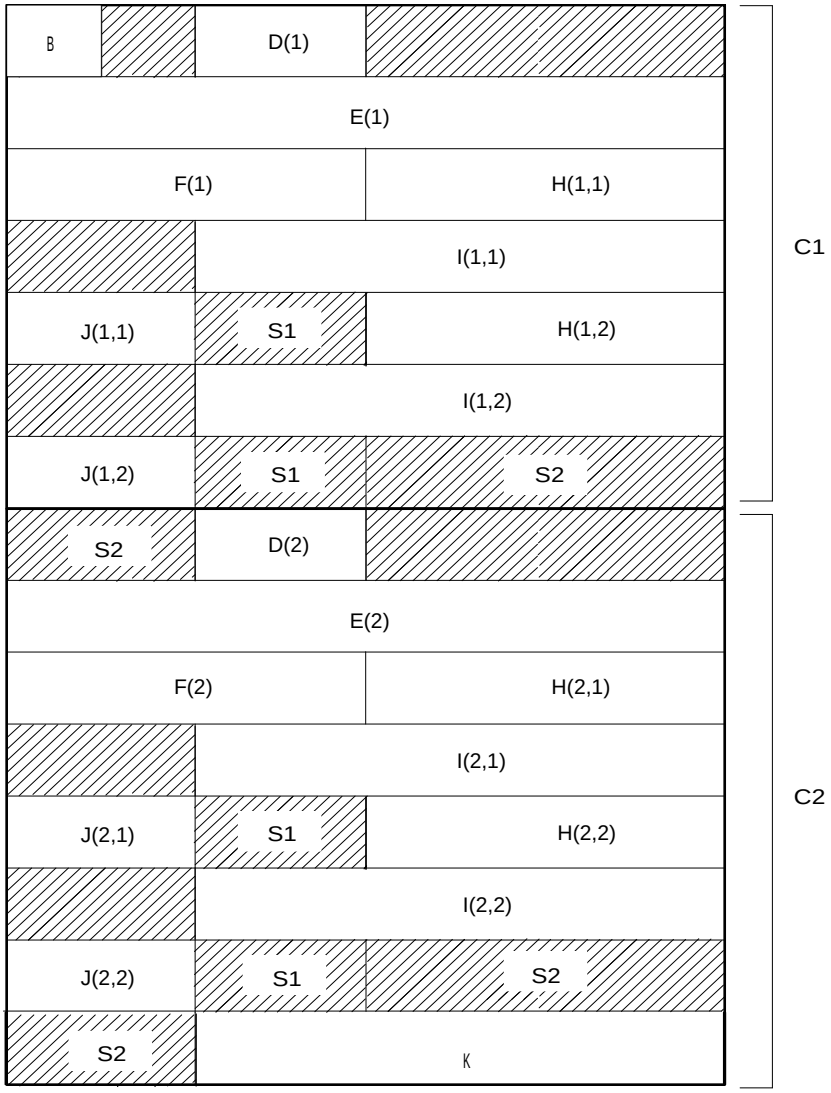
MOVE ZERO TO A would fill the entire illustrated area with zeros.

The preceding illustration showed a 1-dimension array. If two or three nested OCCURS are used, padding slack is provided as necessary between each occurring group.

For example:

```
01 A.
     02 B PIC 1(4).
     02 C OCCURS 2 TIMES
         03 D PIC X.
         03 E COMPUTATIONAL-1.
         03 F PIC XX.
         03 G OCCURS 2 TIMES.
             04 H PIC XX.
             04 I PIC XXX SYNCHRONIZED RIGHT.
             04 J PIC X.
      02 K PIC X(3).
```

is allocated as follows:



S1 indicates slack created because of the inner OCCURS group

S2 indicates slack created because of the outer OCCURS group

Other slack areas are those which result from normal synchronization and alignment.

The preceding example shows not only how nested dimensioned items are handled, but also illustrates the advisability of careful analysis when determining how to arrange dimensioned fields. The same data written:

```
01 A.
     02 B PIC 1(4).
     02 K PIC X(3).
     02 C OCCURS 2 TIMES.
         03 E COMPUTATIONAL-1.
         03 D PIC X.
         03 F PIC XX.
         03 G OCCURS 2 TIMES.
             04 J PIC X.
             04 I PIC XXX SYNCHRONIZED RIGHT.
             04 H PIC XX.
```

reduces the size of each occurrence of C by one word, and the total size by two words. If the table had been larger the reduction could dramatically reduce the amount of storage required.

Appendix E
Label Processing

E.1. Introduction to Label Processing

This appendix discusses all of the file label processing which is performed on disc and tape files. An explanation is given of the COBOL source statements which may alter labelling procedures. Label formats are shown and default label values are indicated. The sequence of label processing events is outlined for each COBOL I–O verb which affects label processing.

E.2. The LABEL RECORDS Clause

When the STANDARD phrase of the LABEL RECORDS clause is specified for any file, explicit label information is created and maintained on the file. The COBOL user may control the contents or the validation of selected portions of this label information through the use of the VALUE OF clause.

When the OMITTED phrase of the LABEL RECORDS clause is specified for any DISC, TAPE, or INTERCHANGE file, implicit label information is created and maintained on the file. In this case, the COBOL user may not use the VALUE OF clause to override any of the system default values used to create or validate this label information. However, if a TAPE or INTERCHANGE file is assigned with the J option, no implicit label processing will occur and the tape must be unlabeled.

When the OMITTED phrase of the LABEL RECORDS clause is specified for an IBM compatible tape file, no labels are created in the output mode and none are expected in the input mode.

E.3. Label Formats

The method by which label information is recorded on a file varies with the type of file.

E.3.1. TAPE and INTERCHANGE Files

The labels recorded on files which have TAPE or INTERCHANGE as *implementor-name-1* in the ASSIGN clause are maintained by the Tape Labeling System in the OS 2200 Operating System. These labels include VOL 1, HDR1, HDR2, EOF1, EOF2, EOV1, and EOV2.

E.3.1.1. Volume Header Label (VOL1)

The VOL 1 label exists as the first label block on every reel. It is maintained by the OS 2200 Executive to verify that the proper reel has been mounted. The COBOL program has no access to this label.

Character Positions	Length	Standard Content	Description
1-3	3	VOL	Label identifier
4	1	1	Label number
5-10	6	Reel number	Six alphanumeric characters identifying the physical reel
11	1	Blank or nonblank	Nonblank indicates restricted access, because the tape is privately owned.
12-37	26	Spaces	Not used
38-51	14	Account number or spaces*	Account number of owner if this is a privately owned tape reel.
52-79	28	Spaces	Not used
80	1	4	Indicates the tape label standard. The value 4 represents the ISO 1001-1986 and ANSI X3.27-1987 standards.

* The account number is placed in this label only in standard ASCII labels controlled by the Tape Labeling System, and then only when the proper configuration parameters are set. In all other cases, this field is set to spaces.

E.3.1.2. First File Header Label (HDR1)

The HDR1 label exists as the first label block on every file or file section on a tape reel. It is maintained by the OS 2200 Executive System to control access to the file.

Character Positions	Length	Standard Default Values	Description	VALUE OF Name
1–3	3	HDR	Label identifier	None
4	1	1	Label number	None
5-21	17	External *qualifier*file–name* (1)	File identifier	FILE–ID
22-27	6	Reel number of first reel of tape set (2)	Set identifier	None
28-31	4	0001	File section number (reel sequence number within a file)	None
32-35	4	0001	File sequence number (within a reel)	None
36-39	4	0001	Generation number	None
40-41	2	00	Generation version number	None
42-47	6	Date when file created	Creation date in Julian form preceded by one space or one zero	None
48-53	6	Creation date plus period from ASG statement *field–4* (or Operating System default)	Purge date in Julian form preceded by one space or one zero	PURGE–DATE
54	1	Space	Accessibility	FILE–ACCESS
55-60	6	000000	Block Count	None
61-73	13	U1100–1	Identifier of system that recorded the file	None
74-80	7	Spaces	Unused	None

(1) External qualifier*file–name consists of a character string which identifies the qualifier and file name used on the @ASG statement that created this file. If the number of character positions of the qualifier and file name (the sum of these two) exceeds 16 positions, the rightmost portion of the qualifier is truncated, but the entire file name is always maintained. For example, if the actual qualifier and file name used was: MYPROJECTID*CREATEDFILE, the base level inserts "MYPRO*CREATEDFILE" into this field. In the default case, the same character string occurs in this field for all files within a file set. In other words, all HDR1 records for all files on a volume or multiple volumes contain the same file identifier in the default case. The file identifier in the default case is left justified, space filled.

(2) For IBM files, if the file is initially opened in the current execution on a non–first REEL, the value "UNIVAC" will be inserted in this field.

E.3.1.3. Second File Header (HDR2)

The HDR2 label follows each HDR1 label. This label contains additional file format information.

Character Position	Length	Standard Default Values	Description
1-3	3	HDR	Label Identifier
4	1	2	Label number
5	1	D, F, S, or U for INTERCHANGE files or D for TAPE files	D – Variable number of characters in a record F – Fixed number of characters in a record S – Variable number of characters in a record and the records may span across two or more blocks. U – Variable number of characters in a record and only one record in each block
6-10	5	Block Length	Five numeric characters specifying the maximum number of characters per block
11-15	5	Record Length	Maximum record length including any control fields
16-50	35	Reserved for Operating Systems	Reserved for operating systems use. Any alphanumeric characters.
51-52	2	00	Buffer Offset. Two numeric characters specifying the length in characters of any additional field inserted before a data block, for example, block length. This length is included in the block length (character positions 6-10).
53-80	28	Spaces	Must be "spaces"

E.3.1.4. End of File Label (EOF1)

Except for character positions 1-3 which contain "EOF", the EOF1 label has the same format as the HDR1 label. The values are the same, except for the "Block count" field which is set to the number of physical data blocks since the previous HDR label group.

E.3.1.5. Second End of File Label (EOF2)

Except for character positions 1-3 which contain "EOF", the EOF2 label has the same format as the HDR2 label. This label is located on all tapes following the EOF1 label.

E.3.1.6. End of Volume Label (EOV1)

Except for character positions 1-3 which contain "EOV", the EOV1 label has the same format as the EOF1 label. This label is located at the end of a file section of a tape file which continues on another reel of tape.

E.3.1.7. Second End of Volume Label (EOV2)

Except for character positions 1-3 which contain "EOV", the EOV2 label has the same format as the EOF2 label. This label is located on all tapes following the EOV1 label.

E.3.2. IBM OS/360 Compatible Tape Files

The labels recorded on files which have UNISERVO as *implementor–name–1* in the ASSIGN clause and a RECORDING MODE F, U, or V clause in the File Description are maintained by ASCII COBOL on the OS 2200 Executive. These labels have formats similar to the TAPE or INTERCHANGE files except that the characters in the label are recorded in EBCDIC on the external media. (See Appendix B.)

E.3.3. INDEXED DISC Files

For files which have DISC as *implementor–name–1* in the ASSIGN clause and the ORGANIZATION IS INDEXED clause in the file–control entry the labels are maintained by ASCII COBOL. Only HDR1 and EOF1 labels are recorded on the files. These labels are similar in format to the HDR1 (see E.3.1.2) and EOF1 (see E.3.1.4) labels of the TAPE or INTERCHANGE files except that these labels consist of 80 9–bit ASCII characters. Character positions 22-27 of HDR1 contain "UNIVAC". Character positions 61-73 of HDR1 contain the file qualifier from the @ASG statement. The HDR1 and EOF1 labels are located at sector addresses 0 and 2, respectively. In the remaining words of sector 2, additional information about the file is maintained.

For a complete description of these words see *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588).

E.3.4. RELATIVE and SEQUENTIAL DISC Files

The labels recorded on files which have DISC as *implementor–name–1* in the ASSIGN clause and the ORGANIZATION IS RELATIVE or SEQUENTIAL clause in the file–control entry are maintained by the Processor Common Input/Output System (PCIOS) as System Data Format (SDF) labels. See *OS 2200 Processor Common Input/Output System (PCIOS) Administration and Programming Reference Manual* (7831 0588). ASCII COBOL uses the first 17 9–bit ASCII characters starting at word 14 in these SDF labels to contain the FILE–ID field.

E.4. The VALUE OF Clause

The VALUE OF clause allows the COBOL user to override system default values for certain fields within selected labels for unique label creation on output or validation on input. System default values generally originate on the @ASG statement for the file.

E.4.1. FILE–ACCESS

The VALUE OF FILE–ACCESS clause is allowed only for files assigned to tape. The only allowable functioning characters are ASCII "1" through "7" and space (octal values 061 through 067 and 040, respectively). Any other character will be treated as a space.

The function of the values of 061-067 correspond to the R, W, and F options on the @ASG statement; i.e., if the R option is on the assignment when the file is created, the file is read only until it expires.

Accessibility Character	Assign Options		
	F	W	R
" " or 040	X		
"1" or 061			
"2" or 062	X	X	
"3" or 063		X	
"4" or 064	X		X
"5" or 065			X
"6" or 066	X	X	X
"7" or 067		X	X

E.4.2. FILE-ID

The contents of the literal or the data item referenced by the data–name specified in the VALUE OF FILE–ID clause is used to override the *qualifier*file–name* specified on the @ASG statement for the file during label creation or validation.

E.4.3. PURGE-DATE

The VALUE OF PURGE–DATE clause is allowed only for files assigned to tape. The contents of the literal or the data item referenced by the data–name specified in the VALUE OF PURGE–DATE clause is used to override the purge date specified in *field–4* of the @ASG statement for the file during file creation. These contents must be of the form *mmddyy* where *mm* is the number of the month (01-12), *dd* is the number of the day (01-31), and *yy* is the last two digits of the year. For example, July 4, 1998, would be indicated as "070498".

E.5. Procedure Division I-O Statements

This section indicates the logical sequence of labeling operations that occurs implicitly during the execution of certain I-O statements. Since the labeling of files assigned as INTERCHANGE and TAPE files is under control of the OS 2200 Executive System Tape Labeling System, this logical sequence of operations often does not correspond directly to the physical sequence of operations performed on the file.

E.5.1. The CLOSE Statement

E.5.1.1. CLOSE File

E.5.1.1.1. Output Files

During the execution of a CLOSE statement without the REEL phrase for a file which is opened in the output mode, the following sequence of operations occurs:

- The EOF1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the EOF1 label are replaced by the user specified values.
- The EOF1 label is written to the external media.

In addition, for files assigned to tape, the following operations also occur:

- The EOF1 label is preceded on the tape by a tape mark.
- Following the EOF1 label, a EOF2 label and a double tape mark are also written on the tape.

E.5.1.1.2. Input Files

During the execution of a CLOSE statement without the REEL phrase for a file which is opened in the input mode, no label processing occurs if the file is positioned at a position other than its end. If the file is positioned at its end, the following sequence of operations occurs:

- The EOF1 label is read from the external media and standard label values are verified.
- If any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user–specified values.
- For tape files, the EOF2 label is also read from the external media.

E.5.1.1.3. I–O Files

During the execution of a CLOSE statement without the REEL phrase for a file which is opened in the I–O mode, label processing is identical to that for a file opened in the input mode. Subsequent to the other operations, the EOF1 label is rewritten to the external media. This includes the supplementary information contained in words 20 through 27 of sector 2 for indexed files.

E.5.1.1.4. Reversed Files

Only tape files can be opened in the reverse mode.

During the execution of a CLOSE statement for a file opened in the reverse mode, label processing occurs only if the beginning of file condition is encountered. If this condition has been encountered, label processing occurs similar to that stated for the CLOSE statement without the REEL phrase for a file opened in the input mode (see E.5.1.1.2) except the HDR labels are treated as EOF labels.

E.5.1.2. CLOSE REEL

The CLOSE statement with the REEL phrase applies only to tape files.

E.5.1.2.1. Output Files

During the execution of a CLOSE statement with the REEL phrase for a file which is opened in the output mode, the following sequence of operations occurs:

- The EOV1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the EOV1 label are replaced by the user-pecified values.
- A tape mark and the EOV1 label are written on the tape.
- The EOV2 label and a double tape mark are written on the tape.
- The current reel is swapped to the next reel in the sequence specified on the @ASG statement for the file.
- The labeling procedures continue on the new reel similar to those specified for an OPEN statement with the OUTPUT phrase (see E.5.2.1). The file section field in the HDR1 label is updated to indicate the new section of the file.

E.5.1.2.2. Input Files

During the execution of a CLOSE statement with the REEL phrase for a file which is opened in the input mode, no end of volume label processing occurs for the current reel if the end of the file section has not been encountered. In this case, label processing resumes with the swap to the next reel in sequence below. If the end of the file section has been encountered, the following label processing occurs:

- The EOV1 label is read from the tape and standard label values are verified.
- In any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user–specified values.
- The EOV2 label is also read from the tape.
- The current reel is swapped to the next reel in the sequence specified on the @ASG statement for the file.
- The labeling procedures continue on the new reel similar to those specified for an OPEN statement with the INPUT phrase (see E.5.2.2).

E.5.2. The OPEN Statement

E.5.2.1. Output Files

During the execution of an OPEN statement with the OUTPUT phrase, the following sequence of operations occurs:

- The HDR1 label is created with the standard values.
- If any VALUE OF clauses are specified for the file, the standard values in the HDR1 label are replaced by the user specified values.
- The HDR1 label is written to the external media.
- In addition, for files assigned to tape, the following operations also occur:
- If the file being created is the first file on a tape or tape set, the HDR1 label is preceded on the tape by a VOL 1 label.
- Following the HDR1 label, a HDR2 label and a tape mark are also written on the tape.

E.5.2.2. Input or I–O Files

During the execution of an OPEN statement with the INPUT or I–O phrase, the following sequence of operations occurs:

- The HDR1 label is read from the external media and standard label values are verified.
- If any VALUE OF clauses are specified for the file, the VALUE OF label entries are verified against the user–specified values.
- For nonindexed disc files, the end-of-file address is extracted from the labels and made available to the program for subsequent end-of-file checks.
- For sequential access files, the file is logically positioned at the first available data record.
- If the file being input is the first file on a tape or tape set, validation of the HDR1 label is preceded by the system validation of the VOL 1 label.

E.5.2.3. Reversed Files

Only tape files can be opened in the reverse mode.

During the execution of an OPEN statement for a file opened in the reverse mode, label processing occurs similar to that stated for the OPEN statement for a file opened in the input mode (see E.5.2.2) except the EOF labels are treated as HDR labels.

E.5.3. The READ Statement

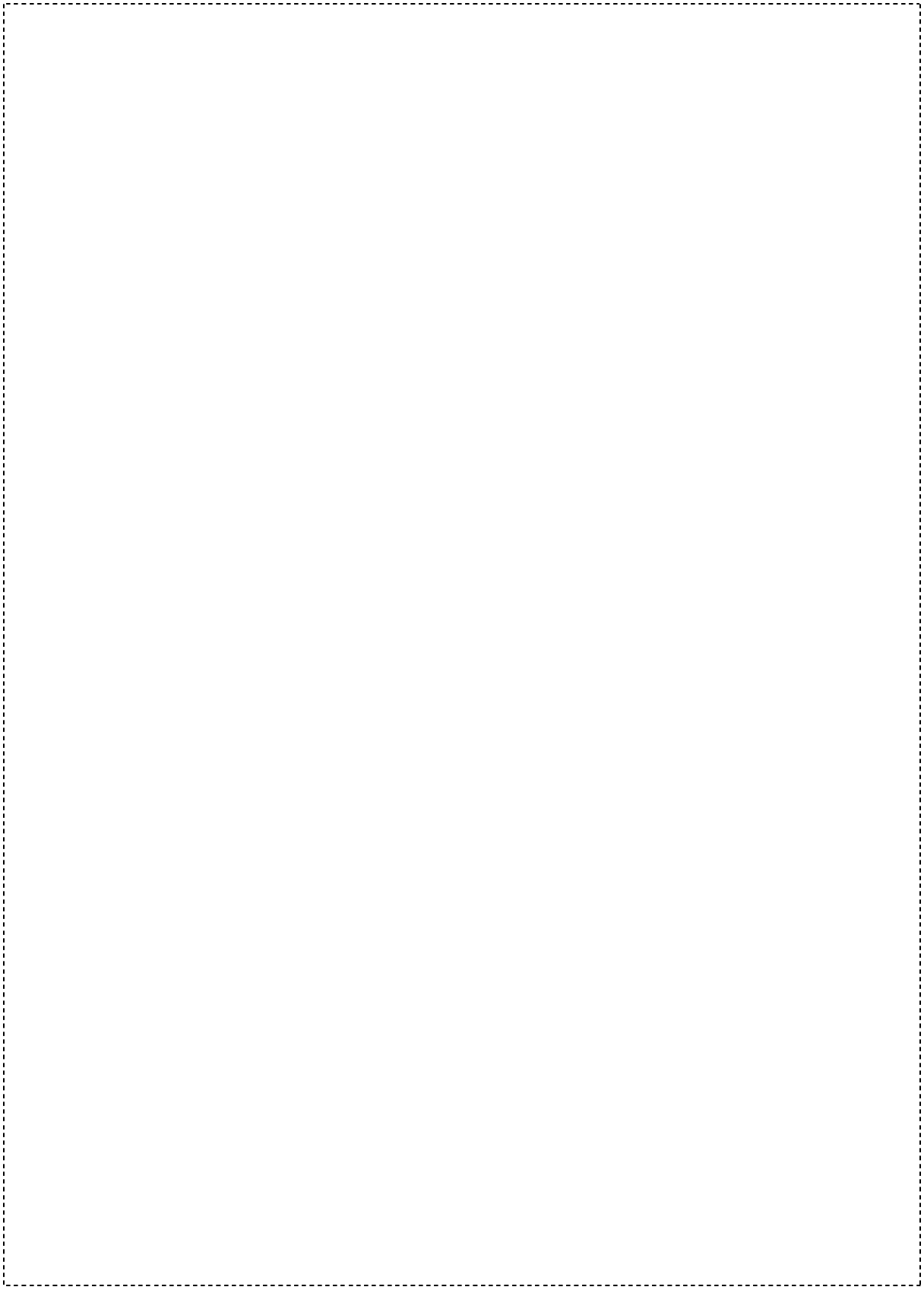
The READ statement may invoke label processing only in the case of a multiple reel tape file.

If during the execution of a READ statement an end–of–file section condition occurs, label processing occurs as stated in the execution of a CLOSE statement with the REEL clause for a file opened in the input mode (see E.5.1.2.2).

E.5.4. The WRITE Statement

The WRITE statement may invoke label processing only in the case of a multiple reel tape file.

If during the execution of a WRITE statement an end-of-tape condition occurs, label processing occurs as stated in the execution of a CLOSE statement with the REEL clause for a file opened in the output mode (see E.5.1.2.1).



Appendix F
Language Skeleton

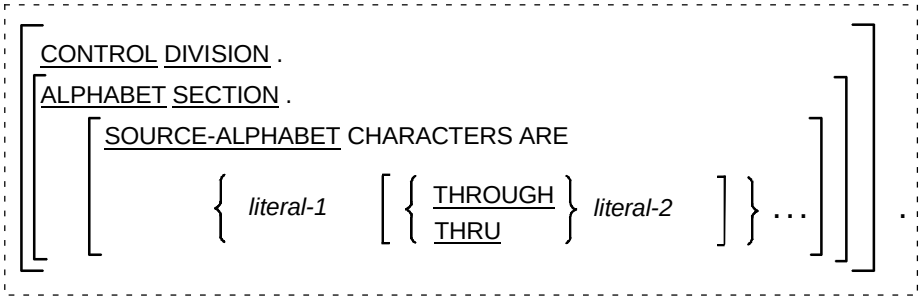
F.1. General Description

This appendix contains the composite language skeleton of ASCII COBOL. It displays the complete and correct syntax.

In F.2 through F.6 the leftmost margin is equivalent to margin A in a COBOL source program. The first indentation after the leftmost margin is equivalent to margin B.

In F.6.1 through F.6.46, the leftmost margin indicates the beginning of the format for a new COBOL verb. The first indentation after the leftmost margin indicates continuation of the format of the COBOL verb.

F.2. Control Division



F.3. Identification Division

IDENTIFICATION DIVISION.

PROGRAM-ID. *program-name*.

[AUTHOR. [*comment-entry*] . . .]

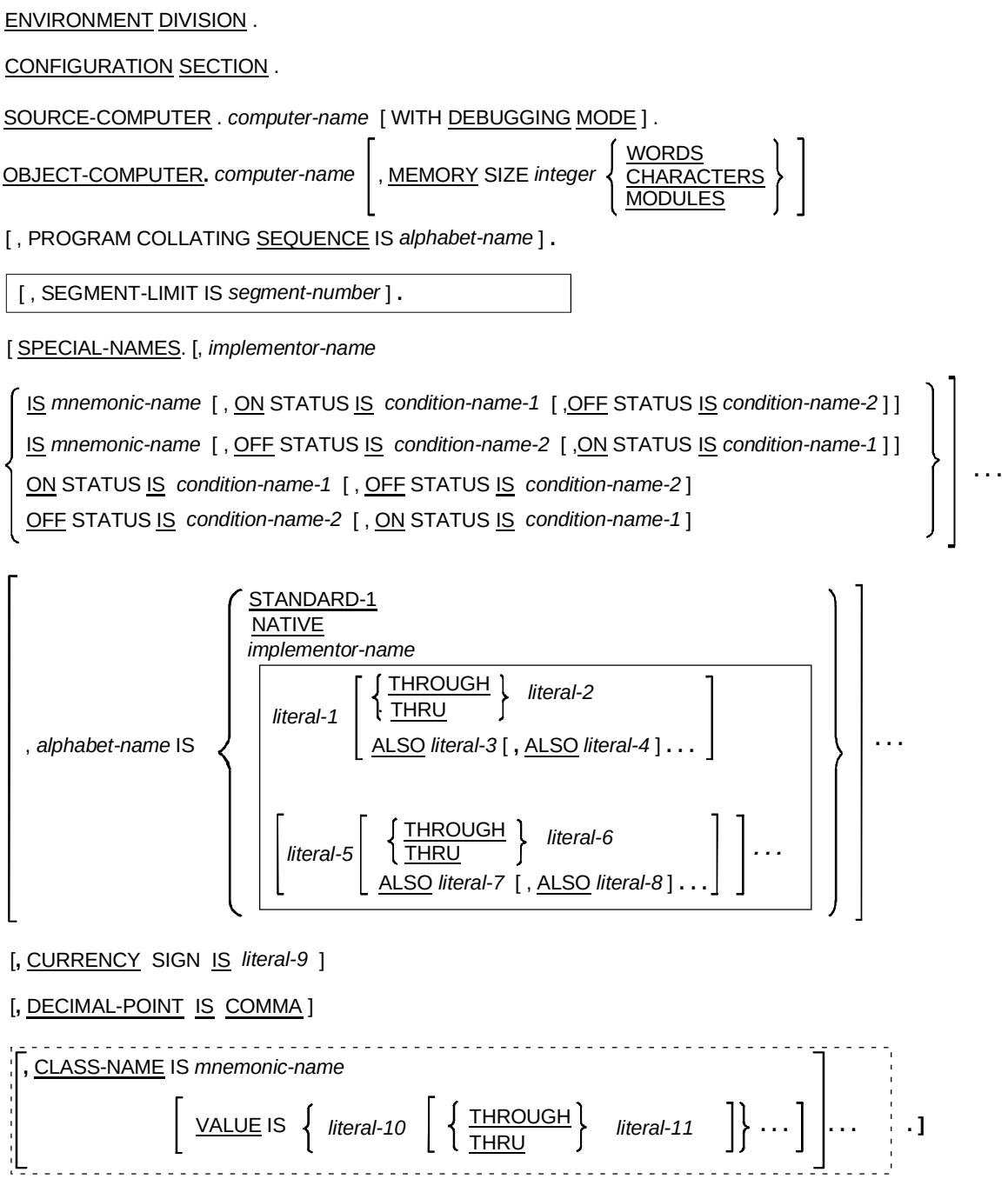
[INSTALLATION. [*comment-entry*] . . .]

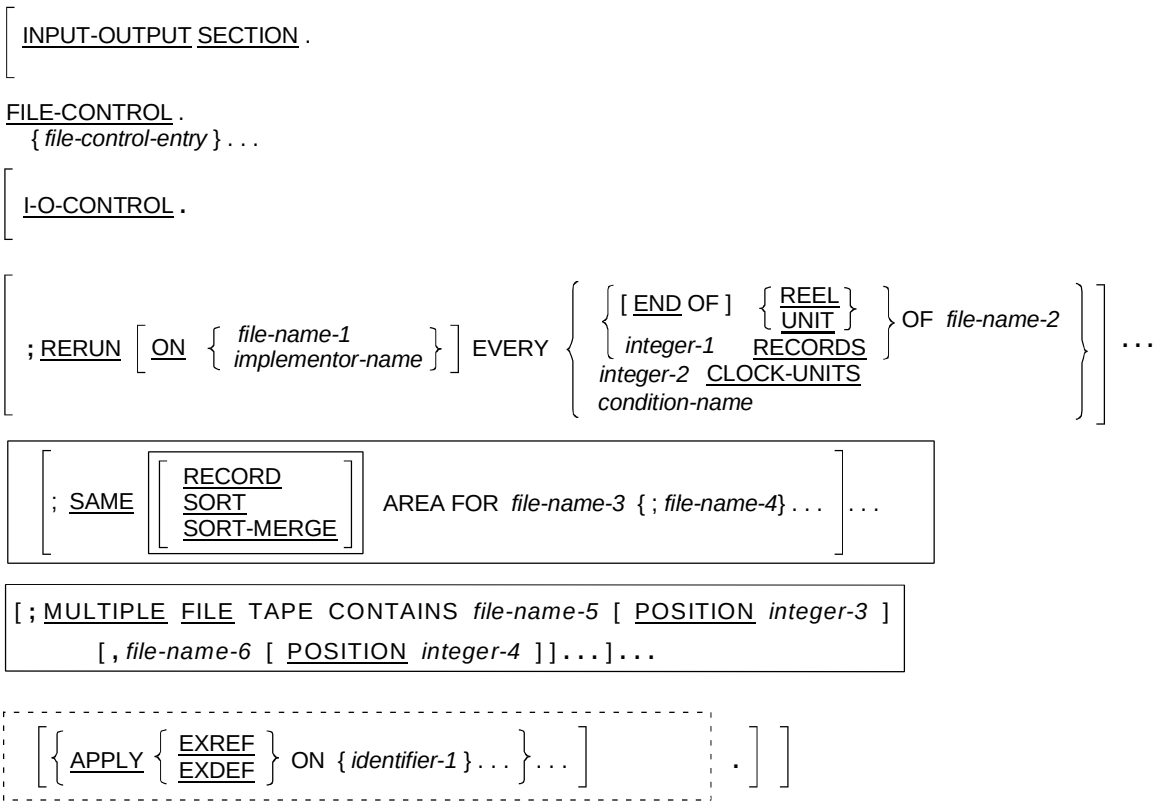
[DATE-WRITTEN. [*comment-entry*] . . .]

[DATE-COMPILED. [*comment-entry*] . . .]

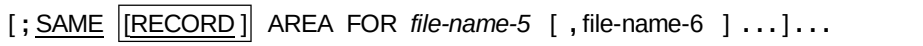
[SECURITY. [*comment-entry*] . . .]

F.4. Environment Division





- **Sequential I-O, Relative I-O, and Indexed I-O Format:**



- **Sort-Merge Format:**



F.4.1. File-Control Entry

- **Sequential I-O Format:**

```
SELECT [OPTIONAL] file-name

    ASSIGN TO implementor-name-1 [ , implementor-name-2 ] . . .

    [ ; RESERVE integer-1 [ AREA | AREAS ] ]
    [ ; ORGANIZATION IS SEQUENTIAL ]
    [ ; ACCESS MODE IS SEQUENTIAL ]
    [ ; FILE STATUS IS data-name-1 ]
    [ ; PROCESSING MODE IS RANDOM [ FOR integer-2 RECORDS ] ] .
```

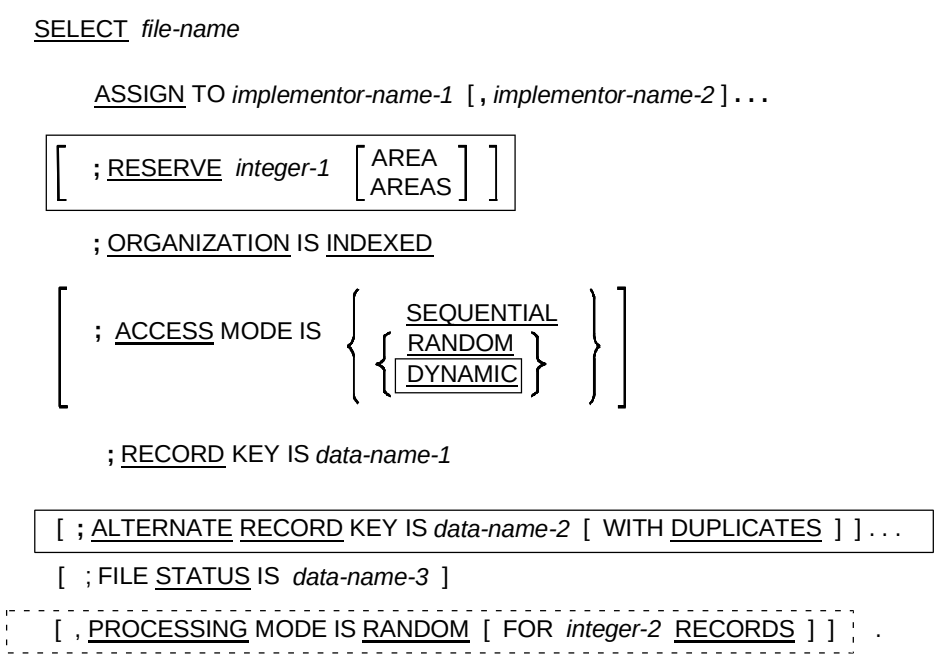
- **Relative I-O Format:**

```
SELECT file-name

    ASSIGN TO implementor-name-1 [ , implementor-name-2 ] . . .

    [ ; RESERVE integer-1 [ AREA | AREAS ] ]
    ; ORGANIZATION IS RELATIVE
    [ ; ACCESS MODE IS { SEQUENTIAL [ , RELATIVE KEY IS data-name-1 ]
                       { { RANDOM  }
                       { [ DYNAMIC ] }  , RELATIVE KEY IS data-name-1 } ]
    [ ; FILE STATUS IS data-name-2 ]
    [ ; PROCESSING MODE IS RANDOM [ FOR integer-2 RECORDS ] ] .
```

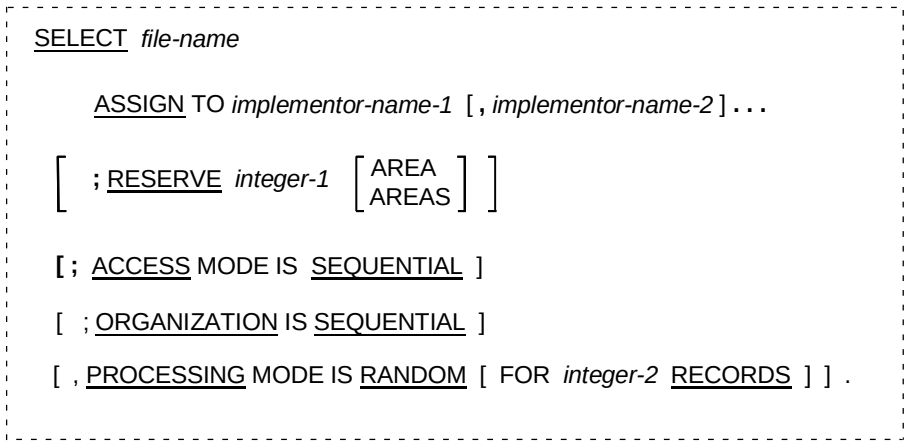
- **Indexed I-O Format:**



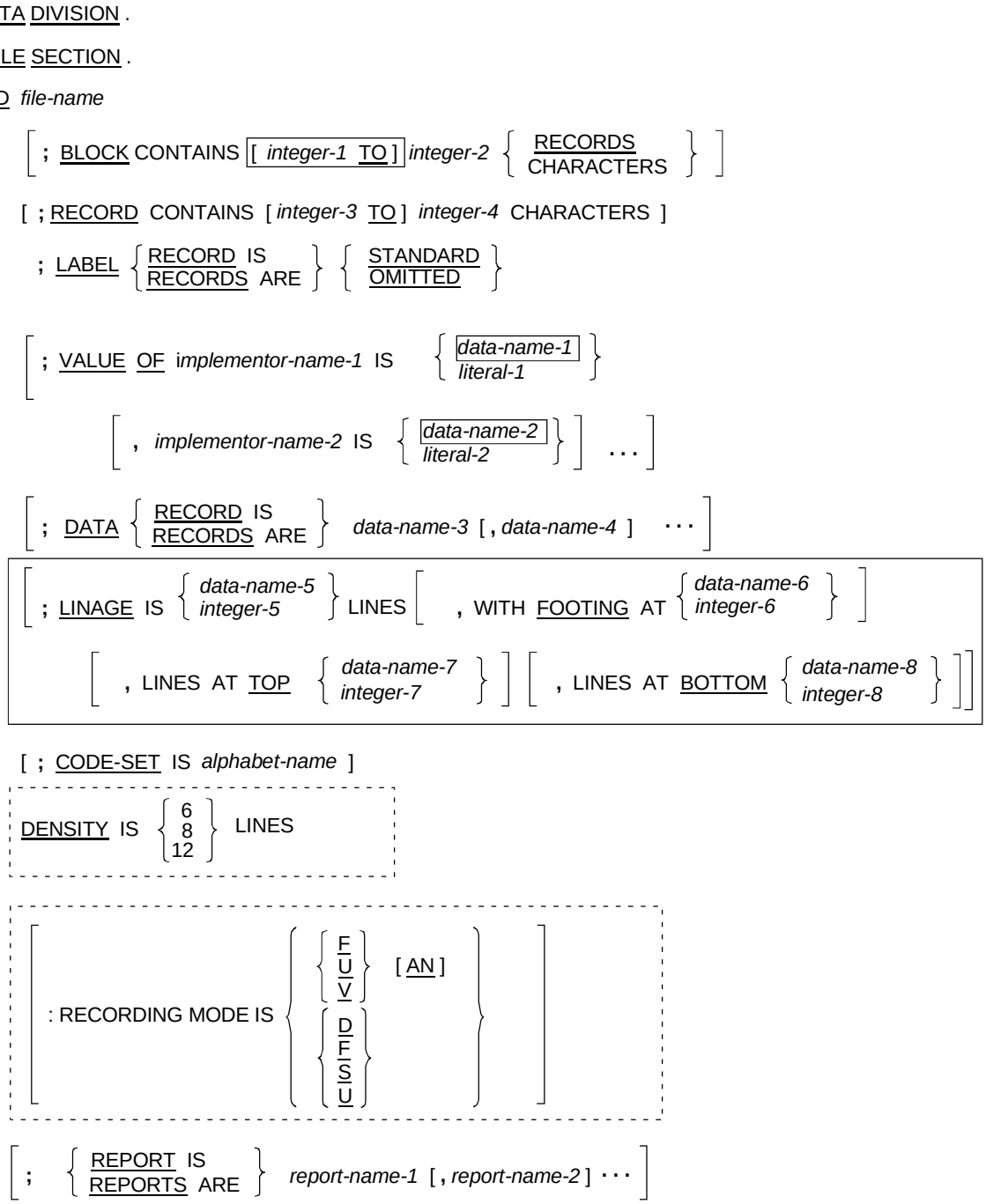
- **Sort-Merge Format:**

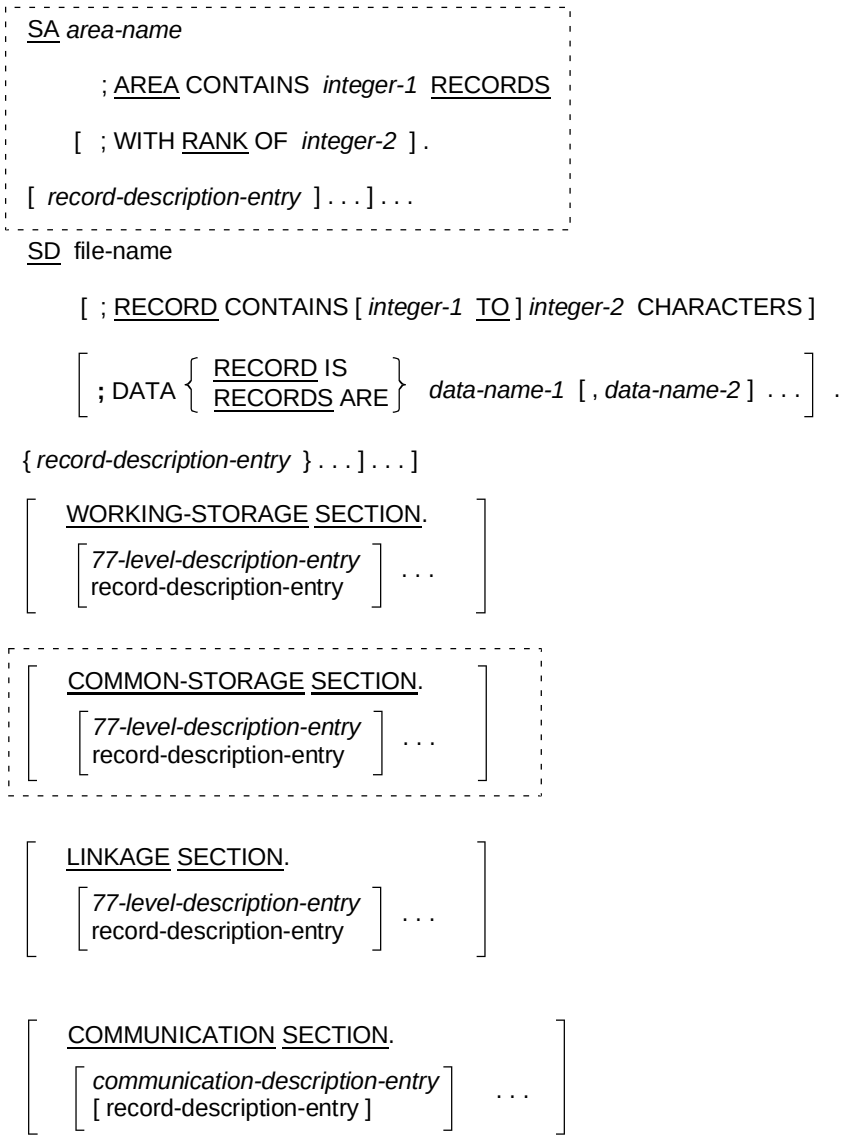
```
SELECT file-name ASSIGN TO implementor-name-1 [, implementor-name-2 ] ... .
```

- **IBM Tape Processing Format:**



F.5. Data Division




SA area-name
; AREA CONTAINS integer-1 RECORDS
[; WITH RANK OF integer-2] .
[record-description-entry] . . .] . . .
SD file-name
[; RECORD CONTAINS [integer-1 TO] integer-2 CHARACTERS]
; DATA RECORD IS / RECORDS ARE data-name-1 [, data-name-2] . . .
{ record-description-entry } . . .] . . .]
WORKING-STORAGE SECTION.
77-level-description-entry
record-description-entry
. . .
COMMON-STORAGE SECTION.
77-level-description-entry
record-description-entry
. . .
LINKAGE SECTION.
77-level-description-entry
record-description-entry
. . .
COMMUNICATION SECTION.
communication-description-entry
[record-description-entry]
. . .

```
[ REPORT SECTION .

RD  report-name

  [  ; CODE  literal-1  ]

  [  ;   { CONTROL IS  }  { data-name-1  [ , data-name-2  ] ...                    }  ]
         { CONTROLS ARE }  { FINAL  [ , data-name-1  [ , data-name-2  ] ... ]     }

  [  ; PAGE  [ LIMIT IS  ]  integer-1  [ LINE  ]  [ , HEADING integer-2 ]
             [ LIMITS ARE ]            [ LINES ]

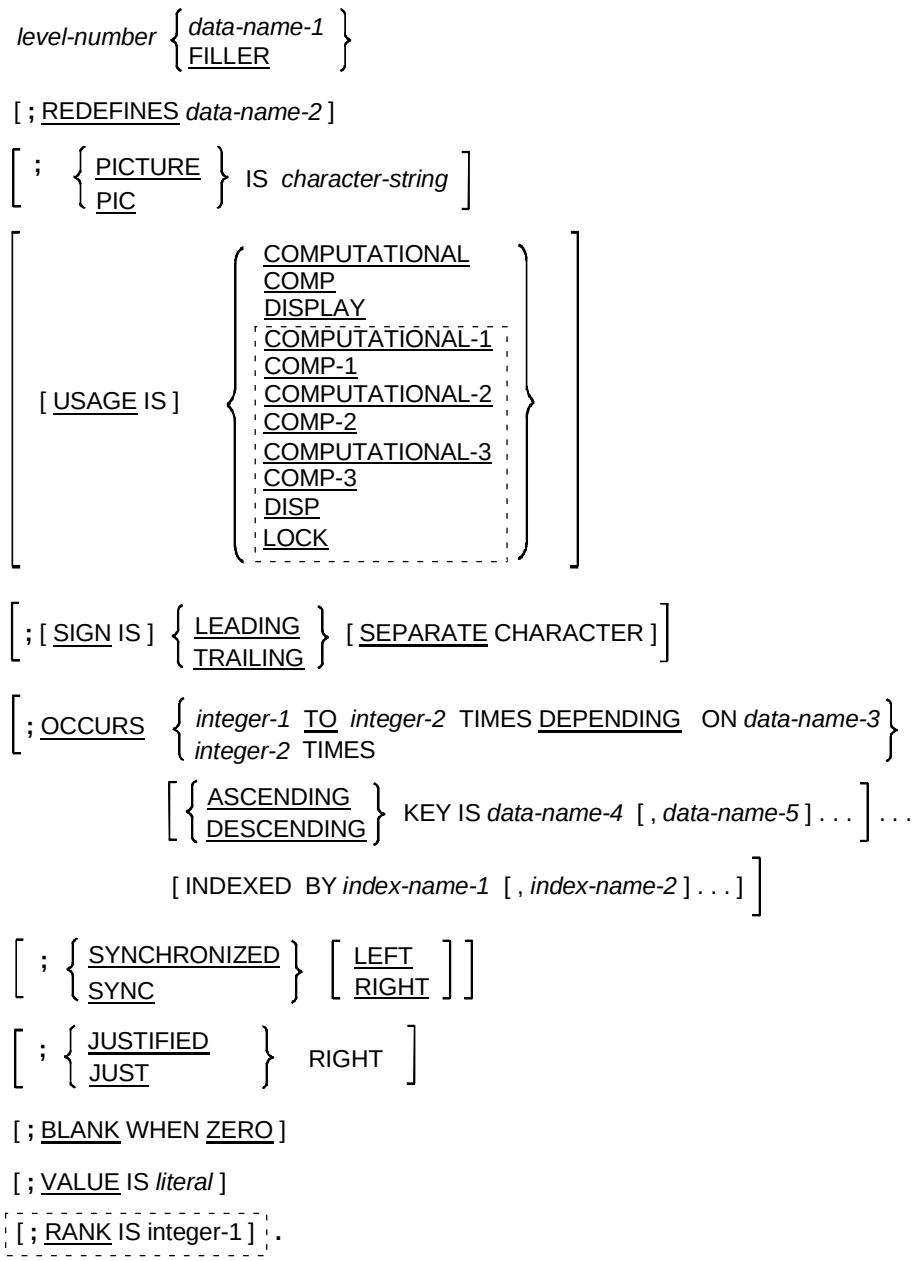
        [ , FIRST  DETAIL  integer-3  ] [ , LAST  DETAIL  integer-4  ]

        [ , FOOTING  integer-5  ]  ]  .

  { report-group-description-entry } . . . ] . . . ]
```

F.5.1. Data Description Entry

Format 1



Format 2

```
66 data-name-1; RENAMES data-name-2  [ { THROUGH } data-name-3 ]  .
                                       { THRU    }
```

Format 3

```
88 condition-name ; { VALUE IS  } literal-1  [ { THROUGH } literal-2 ]
                    { VALUES ARE }             { THRU    }

   [ , literal-3  [ { THROUGH } literal-4 ] ] ... .
                    { THRU    }
```

F.5.2. Communication Description Entry

Format 1

```
CD cd-name ;

                                                  [ ; SYMBOLIC QUEUE IS data-name-1 ]
                                                  [ ; SYMBOLIC SUB-QUEUE-1 IS data-name-2 ]
                                                  [ ; SYMBOLIC SUB-QUEUE-2 IS data-name-3 ]
                                                  [ ; SYMBOLIC SUB-QUEUE-3 IS data-name-4 ]
                                                  [ ; MESSAGE DATE IS data-name-5 ]
                                                  [ ; MESSAGE TIME IS data-name-6 ]
FOR [ INITIAL ] INPUT  [ [                        [ ; SYMBOLIC SOURCE IS data-name-7 ]      ]  .
                         [                        [ ; TEXT LENGTH IS data-name-8 ]          ]
                         [                        [ ; END KEY IS data-name-9 ]              ]
                         [                        [ ; STATUS KEY IS data-name-10 ]          ]
                         [                        [ ; MESSAGE COUNT IS data-name-11 ]       ]
                           [ data-name-1, data-name-2, ..., data-name-11 ]                   ]

  [ ; PROCESSING MODE IS RANDOM [ FOR integer-2 RECORDS ] ] .
```

Format 2

```
CD cd-name ;  FOR OUTPUT
    [ ; DESTINATION COUNT IS data-name-1 ]
    [ ; TEXT LENGTH IS data-name-2 ]
    [ ; STATUS KEY IS data-name-3 ]
    [ ; DESTINATION TABLE OCCURS integer-2 TIMES
        [ ; INDEXED BY index-name-1 [ , index-name-2 ] . . . ] ]
    [ ; ERROR KEY IS data-name-4 ]
    [ ; SYMBOLIC DESTINATION IS data-name-5 ]
  [ ; PROCESSING MODE IS RANDOM [ FOR integer-2 RECORDS ] ] .
```

F.5.3. Report Group Description Entry

Format 1

```
01 [ data-name-1 ]

[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] } ]
                   { PLUS integer-2             }

[ ; NEXT GROUP IS { integer-3     } ]
                  { PLUS integer-4 }
                  { NEXT PAGE      }

                                  { REPORT HEADING }
                                  { RH             }
                                  { PAGE HEADING   }
                                  { PH             }
                                  { CONTROL HEADING } { data-name-2 }
                                  { CH              } { FINAL       }
                                  { DETAIL         }
                                  { DE             }
                                  { CONTROL FOOTING } { data-name-3 }
                                  { CF              } { FINAL       }
                                  { PAGE FOOTING   }
                                  { PF             }
                                  { REPORT FOOTING }
                                  { RF             }
; TYPE IS

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

Format 2

level-number [*data-name-1*]

```
[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] }
                   { PLUS integer-2            } ]

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

Format 3

level-number [*data-name-1*]

```
[ ; BLANK WHEN ZERO ]

[ ; GROUP INDICATE ]

[ ; { JUSTIFIED } RIGHT ]
    { JUST      }

[ ; LINE NUMBER IS { integer-1 [ ON NEXT PAGE ] } ]
                   { PLUS integer-2            }

[ ; COLUMN NUMBER IS integer-3 ]

  ; { PICTURE } IS character-string
    { PIC     }

{ ; SOURCE IS identifier-1                                          }
{ ; VALUE IS literal                                                }
{ ; SUM identifier-2 [ , identifier-3 ] . . .                       }
{       [ UPON data-name-2 [ , data-name-3 ] . . . ] } . . .        }
{       [ RESET ON { data-name-4 } ] ]                              }
{                  { FINAL       }                                  }

[ ; [ USAGE IS ] { DISPLAY } ] .
                 { DISP    }
```

F.6. Procedure Division

Format 1

```
PROCEDURE DIVISION [ USING data-name-1 [, data-name-2 ] ... ]

[ WITH ENTRY POINTS { procedure-name-1 } ... ] .

[ DECLARATIVES .

{ section-name SECTION [ segment-number ]. declarative-sentence

[ paragraph-name. [ sentence] ... ] ... } ...

END DECLARATIVES. ]

{ section-name SECTION [ segment-number ] .

[ paragraph-name. [ sentence ] ... ] ... } ...
```

Format 2

```
PROCEDURE DIVISION [ USING data-name-1 [, data-name-2 ] ... ]

[ WITH ENTRY POINTS { procedure-name-1 } ... ] .

{ paragraph-name.  [ sentence ] ... } ...
```

F.6.1. The ACCEPT Statement

Format 1

```
ACCEPT identifier  [ FROM mnemonic-name ]
```

Format 2

```
ACCEPT identifier FROM { DATE [YYYYMMDD]
                       { DAY [YYYYDDD]
                       { TIME
                       { DATE-TIME [MMDDYYYY]
```

Format 3

```
ACCEPT cd-name MESSAGE COUNT
```

F.6.2. The ADD Statement

Format 1

```
ADD { identifier-1 } [ , identifier-2 ] ... TO identifier-m [ ROUNDED ]
    { literal-1    } [ , literal-2    ]

    [ ,identifier-n [ ROUNDED ] ] ... [ ; ON SIZE ERROR imperative-statement ]
```

Format 2

```
ADD { identifier-1 } , { identifier-2 } [ , identifier-3 ] ...
    { literal-1    }   { literal-2    } [ , literal-3    ]

    GIVING identifier-m [ ROUNDED ] [ , identifier-n [ ROUNDED ] ] ...

    [ ; ON SIZE ERROR imperative-statement ]
```

Format 3

```
ADD { CORRESPONDING } identifier-1 TO identifier-2 [ ROUNDED ]
    { CORR          }

[ ; ON SIZE ERROR imperative-statement ]
```

F.6.3. The ALTER Statement

```
ALTER procedure-name-1 TO [ PROCEED TO ] procedure-name-2

    [, procedure-name-3 TO [ PROCEED TO ] procedure-name-4 ] ...
```

F.6.4. The CALL Statement

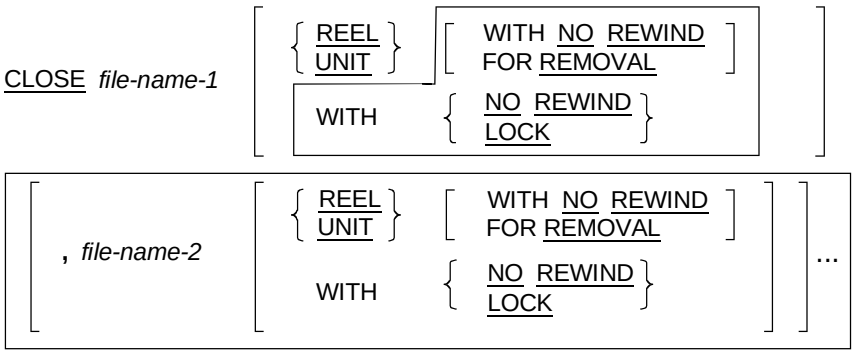
```
CALL { identifier-1 } [ USING { data-name-1 } [ , { data-name-2 } ] ... ]
     { literal-1    }         { literal-2    }     { literal-3    }

     [ ; ON OVERFLOW imperative-statement ]
```

F.6.5. The CANCEL Statement

```
CANCEL { identifier-1 } [ , identifier-2 ] ...
       { literal-1    } [ , literal-2    ]
```

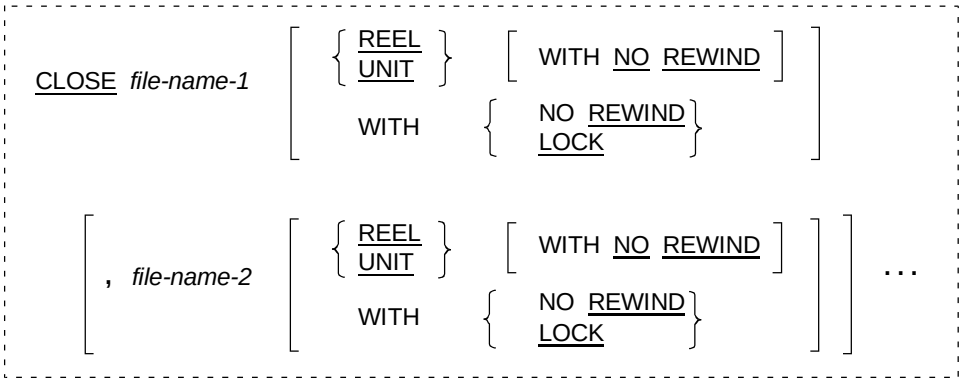
F.6.6. The CLOSE Statement



- **Relative and Indexed I-O Format:**

```
CLOSE file-name-1 [ WITH LOCK ] [, file-name-2 [ WITH LOCK ] ] ...
```

- **IBM Format:**



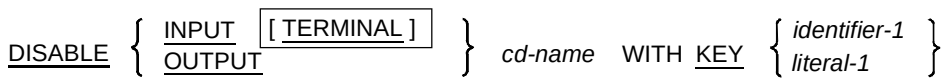
F.6.7. The COMPUTE Statement

```
COMPUTE identifier-1 [ ROUNDED ] [, identifier-2 [ ROUNDED ] ]...
    = arithmetic-expression [ ; ON SIZE ERROR imperative-statement ]
```

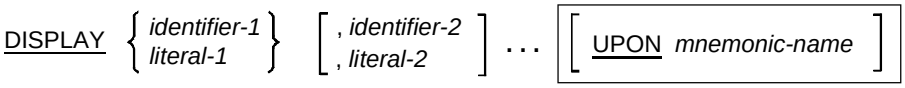
F.6.8. The DELETE Statement

```
DELETE file-name RECORD [; INVALID KEY imperative-statement ]
```

F.6.9. The DISABLE Statement

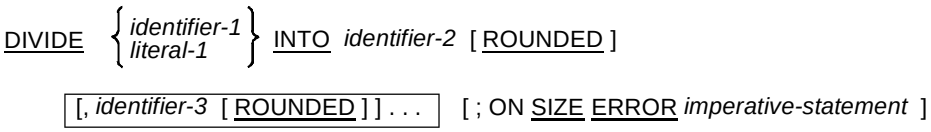


F.6.10. The DISPLAY Statement

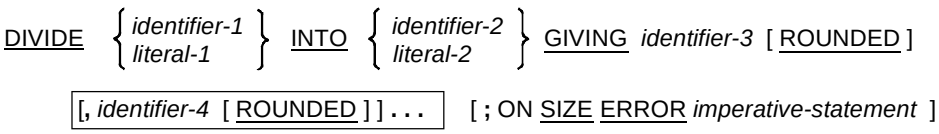


F.6.11. The DIVIDE Statement

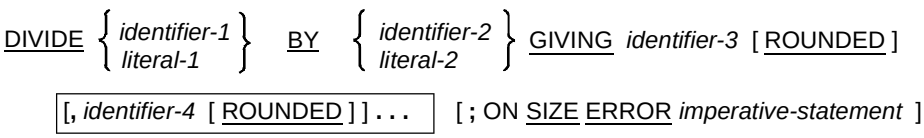
Format 1



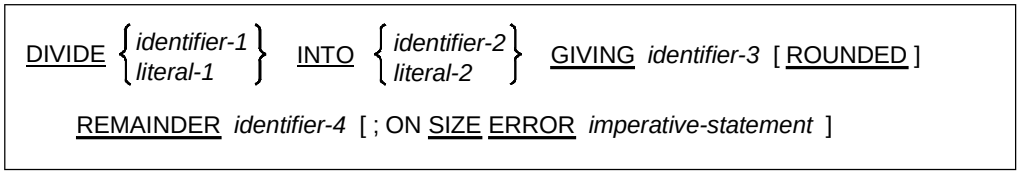
Format 2



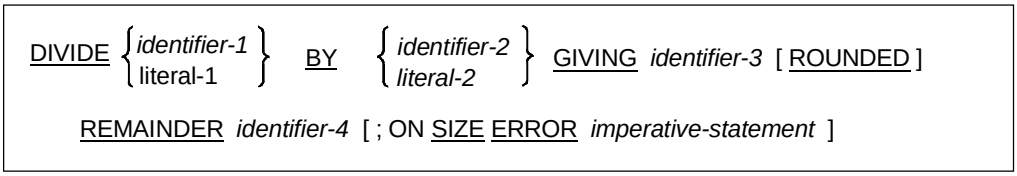
Format 3



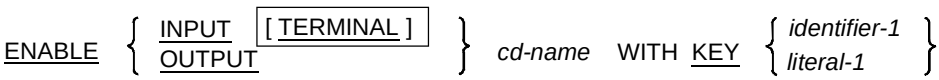
Format 4



Format 5

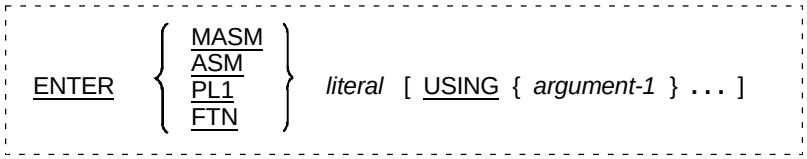


F.6.12. The ENABLE Statement



F.6.13. The ENTER Statement

Format 1



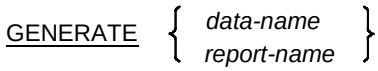
Format 2

```
ENTER MASM "C$INFO" USING file-name, data-name.
```

F.6.14. The EXIT Statement

```
EXIT [ PROGRAM ].
```

F.6.15. The GENERATE Statement



F.6.16. The GO TO Statement

Format 1

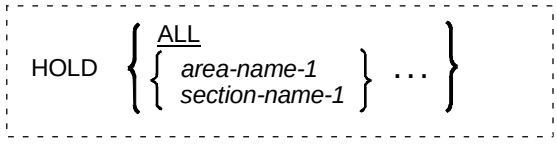
```
GO TO [ procedure-name-1 ]
```

Format 2

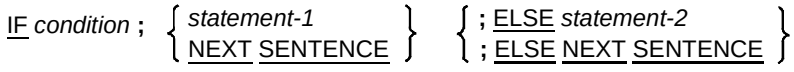
```
GO TO procedure-name-1 [, procedure-name-2 ] ... , procedure-name-n

    DEPENDING ON identifier
```

F.6.17. The HOLD Statement



F.6.18. The IF Statement

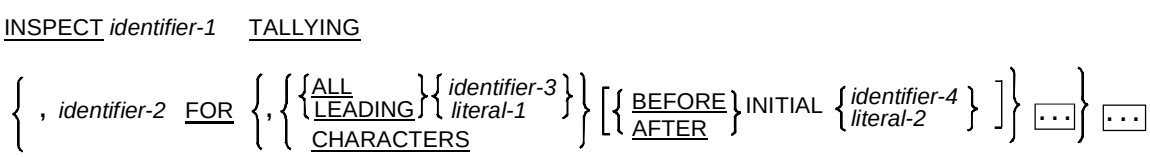


F.6.19. The INITIATE Statement

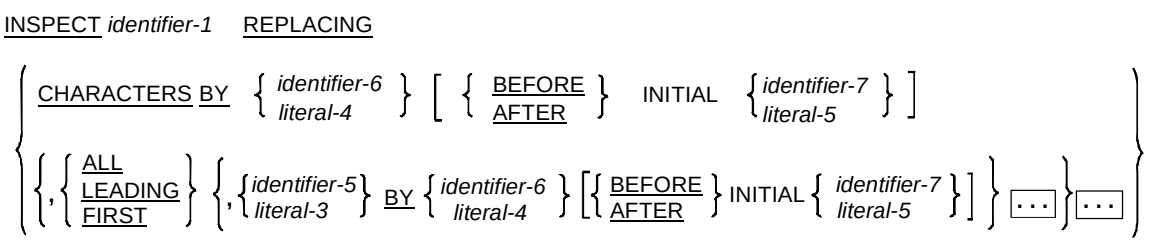
```
INITIATE report-name-1 [, report-name-2 ] ...
```

F.6.20. The INSPECT Statement

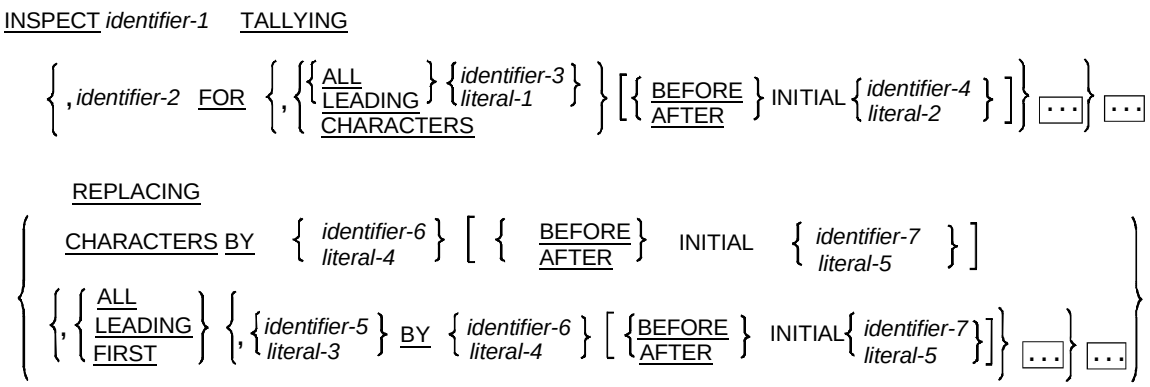
Format 1



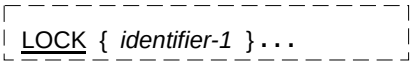
Format 2



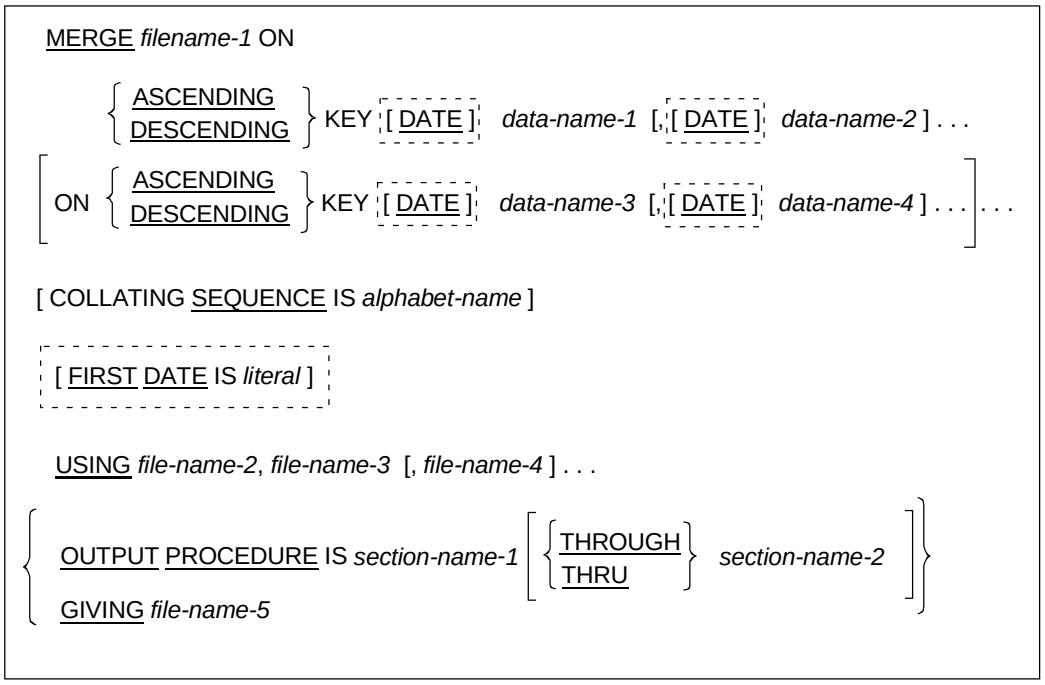
Format 3



F.6.21. The LOCK Statement



F.6.22. The MERGE Statement



F.6.23. The MOVE Statement

Format 1

MOVE { *identifier-1* / *literal* } TO *identifier-2* [, *identifier-3*] . . .

Format 2

MOVE { CORRESPONDING / CORR } *identifier-1* TO *identifier-2*

F.6.24. The MULTIPLY Statement

Format 1

MULTIPLY { *identifier-1* / *literal-1* } BY *identifier-2* [ROUNDED]

[, *identifier-3* [ROUNDED]] . . . [; ON SIZE ERROR *imperative-statement*]

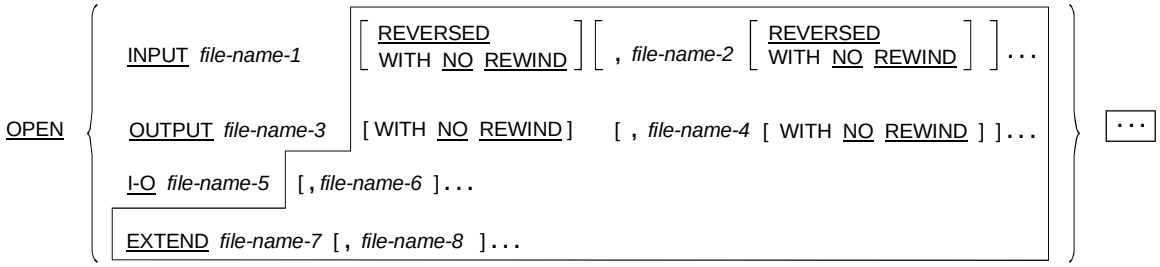
Format 2

MULTIPLY { *identifier-1* / *literal-1* } BY { *identifier-2* / *literal-2* } GIVING *identifier-3* [ROUNDED]

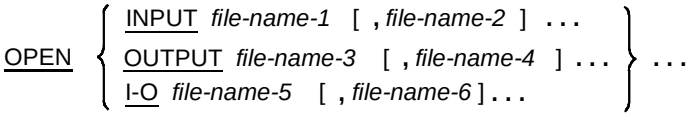
[, *identifier-4* [ROUNDED]] . . . [; ON SIZE ERROR *imperative-statement*]

F.6.25. The OPEN Statement

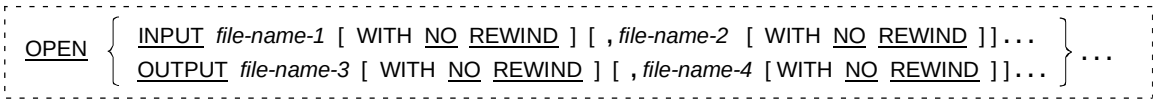
- **Sequential I-O Format:**



- **Relative and Indexed I-O Format:**

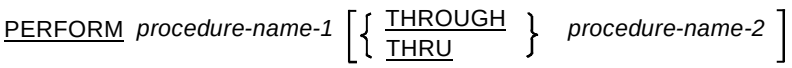


- **IBM Format:**

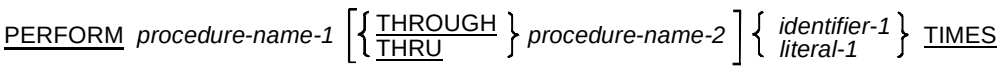


F.6.26. The PERFORM Statement

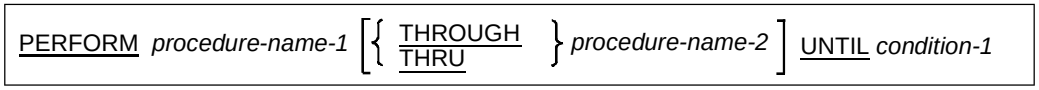
Format 1



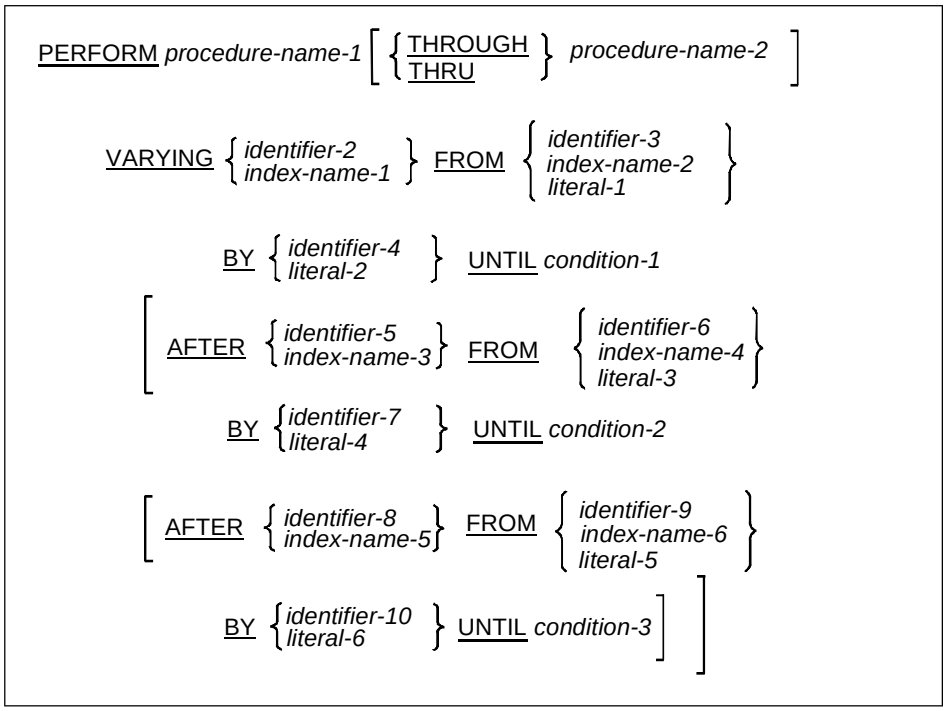
Format 2



Format 3



Format 4



F.6.27. The PROCESS Statement

PROCESS *section-name* [[FROM *identifier*] USING *area-name*]

F.6.28. The READ Statement

- **Sequential I-O Format:**

READ *file-name* RECORD [INTO *identifier*] [; AT END *imperative-statement*]

- **Relative I-O Format:**

Format 1

READ *file-name* [NEXT] RECORD [INTO *identifier*] [; AT END *imperative-statement*]

Format 2

READ *file-name* RECORD [INTO *identifier*] [; INVALID KEY *imperative-statement*]

- **Indexed I-O Format:**

Format 1

READ *file-name* [NEXT] RECORD [INTO *identifier*]

[; AT END *imperative-statement*]

Format 2

```
READ file-name RECORD [ INTO identifier ] [ ; KEY IS data-name ]
     [ ; INVALID KEY imperative-statement ]
```

• IBM Format:

```
READ file-name RECORD [INTO identifier ] AT END imperative-statement
```

F.6.29. The RECEIVE Statement

```
RECEIVE cd-name { MESSAGE          } INTO identifier-1 [ ; NO DATA imperative-statement ]
                { [ SEGMENT ]      }
```

F.6.30. The RELEASE Statement

```
RELEASE record-name [ FROM identifier ]
```

F.6.31. The RETURN Statement

```
RETURN file-name RECORD [ INTO identifier ] ; AT END imperative-statement
```

F.6.32. The REWRITE Statement

- **Sequential I-O Format:**

```
REWRITE record-name [ FROM identifier ]
```

- **Relative and Indexed I-O Format:**

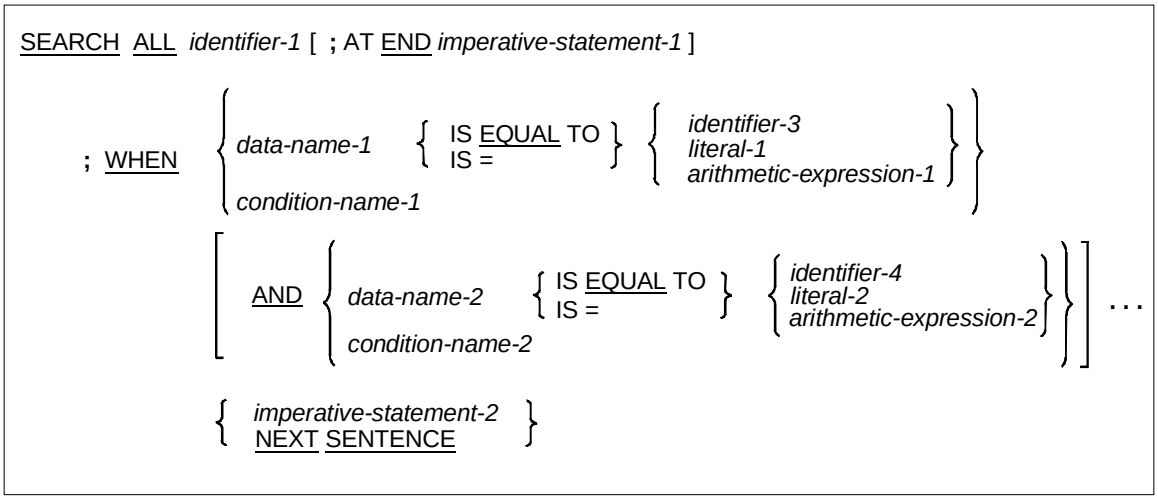
```
REWRITE record-name [ FROM identifier ] [; INVALID KEY imperative-statement]
```

F.6.33. The SEARCH Statement

Format 1

```
SEARCH identifier-1 [ VARYING { identifier-2  } ]
                              { index-name-1  }
    [ ; AT END imperative-statement-1 ]
      ; WHEN condition-1 { imperative-statement-2 }
                         { NEXT SENTENCE          }
   [  ; WHEN condition-2 { imperative-statement-3 } ] ...
                         { NEXT SENTENCE          }
```

Format 2

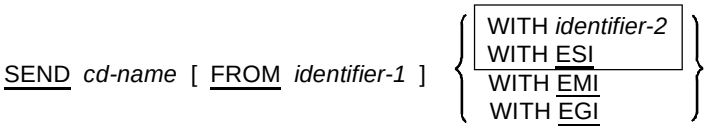


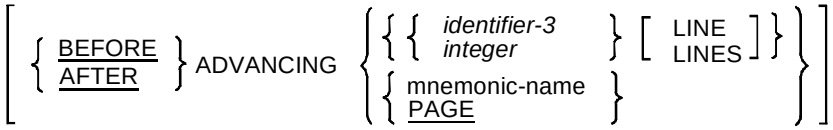
F.6.34. The SEND Statement

Format 1

SEND *cd-name* FROM *identifier-1*

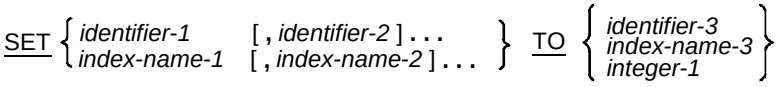
Format 2



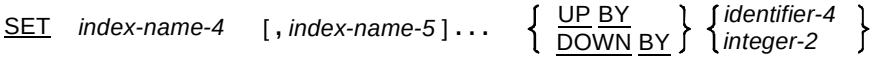


F.6.35. The SET Statement

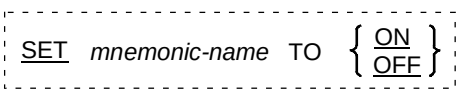
Format 1



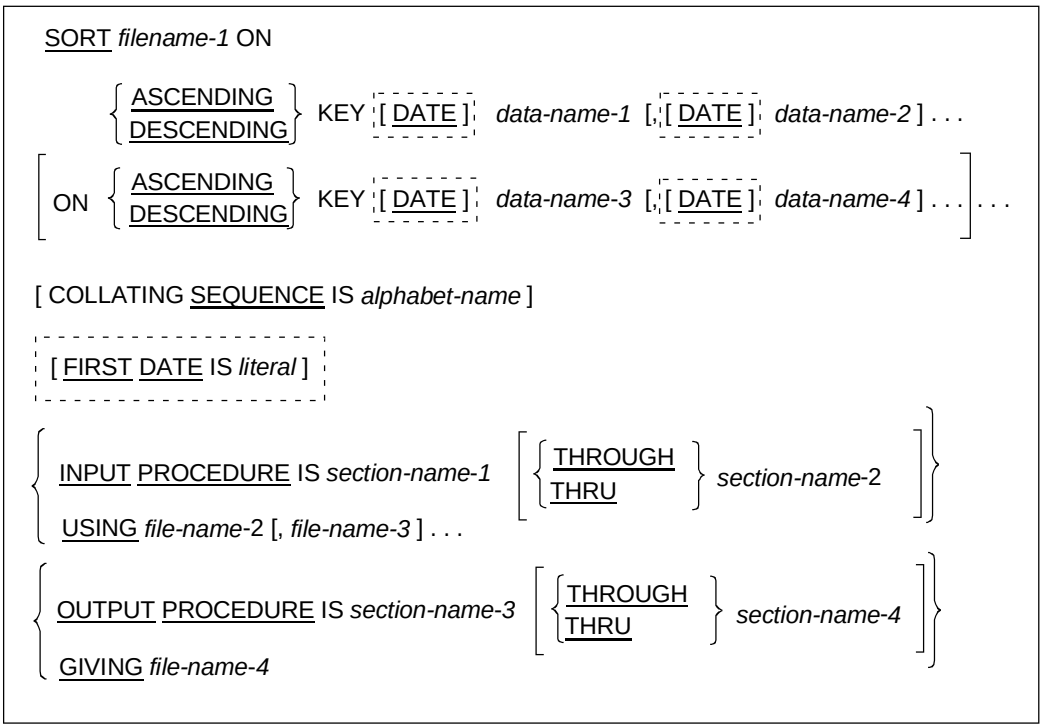
Format 2



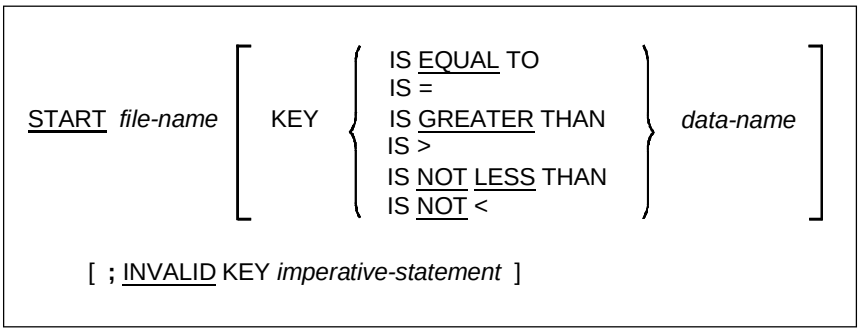
Format 3



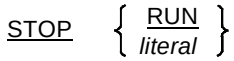
F.6.36. The SORT Statement



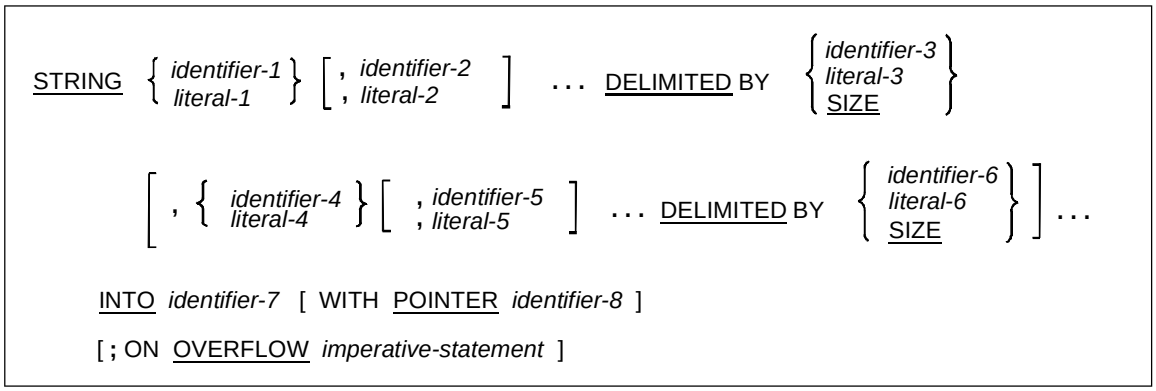
F.6.37. The START Statement



F.6.38. The STOP Statement

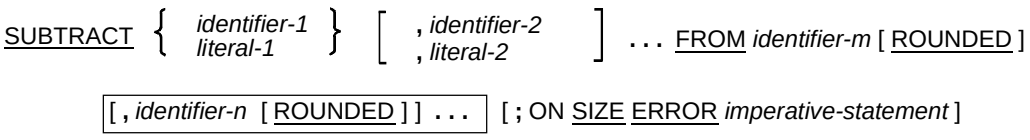


F.6.39. The STRING Statement

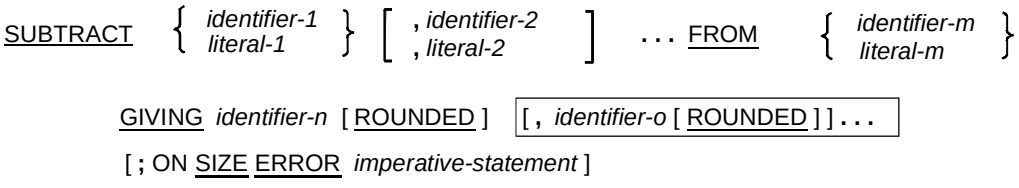


F.6.40. The SUBTRACT Statement

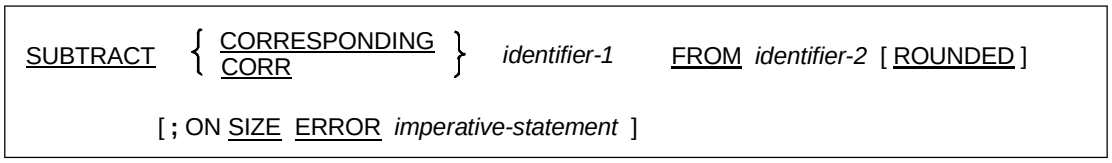
Format 1



Format 2



Format 3



F.6.41. The SUPPRESS Statement

```
SUPPRESS PRINTING
```

F.6.42. The TERMINATE Statement

```
TERMINATE report-name-1 [, report-name-2 ] ...
```

F.6.43. The UNLOCK Statement

```
UNLOCK { identifier-1 } ...
```

F.6.44. The UNSTRING Statement

```
UNSTRING identifier-1
    [ DELIMITED BY [ ALL ] { identifier-2 } [ , OR [ ALL ] ] { identifier-3 } ] ... ]
                           { literal-1    }                  { literal-2    }
    INTO identifier-4 [ , DELIMITER IN identifier-5 ] [ , COUNT IN identifier-6 ]
        [ , identifier-7 [ , DELIMITER IN identifier-8 ] [ , COUNT IN identifier-9 ] ] ...
    [ WITH POINTER identifier-10 ] [ TALLYING IN identifier-11 ]
    [ ; ON OVERFLOW imperative-statement ]
```

F.6.45. The USE Statement

- **Sequential I-O Format:**

```
USE AFTER STANDARD { EXCEPTION } PROCEDURE ON { file-name-1 [ , file-name-2 ] ... }
                   { ERROR     }              { INPUT                            }
                                              { OUTPUT                           }
                                              { I-O                              }
                                              { EXTEND                           } .
```

- **Relative and Indexed I-O Format:**

```
USE AFTER STANDARD { EXCEPTION } PROCEDURE ON { file-name-1 [ , file-name-2 ] ... }
                   { ERROR     }              { INPUT                            }
                                              { OUTPUT                           }
                                              { I-O                              } .
```

- **Report Writer Format:**

```
USE BEFORE REPORTING identifier.
```

- **Debug Format:**

```
USE FOR DEBUGGING ON { cd-name-1                                  }
                     { [ ALL REFERENCES OF ] identifier-1         }
                     { file-name-1                                }
                     { procedure-name-1                           }
                     { ALL PROCEDURES                             }

    [ , { cd-name-2                                  } ... ]
        { [ ALL REFERENCES OF ] identifier-2         }
        { file-name-2                                }
        { procedure-name-2                           }
        { ALL PROCEDURES                             } .
```

- **ACS Format:**

```
USE FOR RANDOM PROCESSING

      [ USING { file-name-1 } ... ]
              { cd-name-1   }

      [ WITH RANK OF integer-1 ] [ FOR integer-2 CYCLES ].
```

- **TABLE ERROR Format:**

```
USE FOR TABLE ERROR REFERENCES ON { ALL                  } .
                                  { { identifier-1 } ... }
```

F.6.46. The WRITE Statement

- **Sequential I-O Format:**

```
WRITE record-name [ FROM identifier-1 ]

[ { BEFORE } ADVANCING { { identifier-2 } [ LINE  ] } ]
  { AFTER  }           { { integer      } [ LINES ] }
                       { { mnemonic-name }          }
                       { { PAGE          }          }

[ ; AT { END-OF-PAGE } imperative-statement ]
       { EOP         }
```

- **Relative and Indexed I-O Format:**

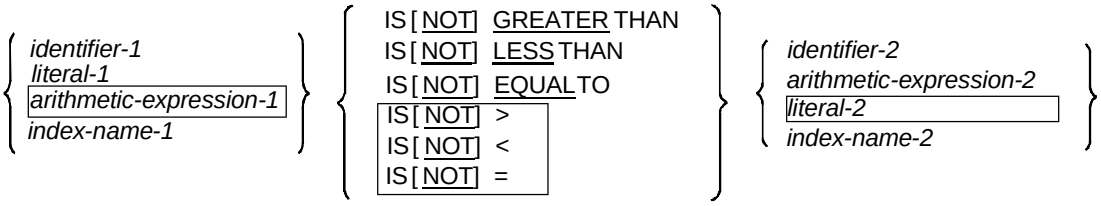
```
WRITE record-name [FROM identifier] [; INVALID KEY imperative-statement]
```

- **IBM Format:**

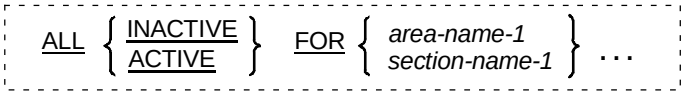
```
WRITE record-name [ FROM identifier ]
```

F.7. Conditions

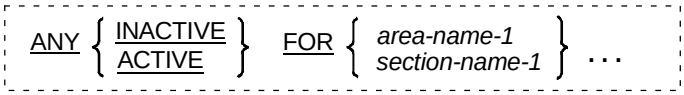
F.7.1. Relation Condition



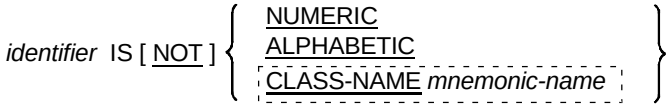
F.7.2. ALL Condition



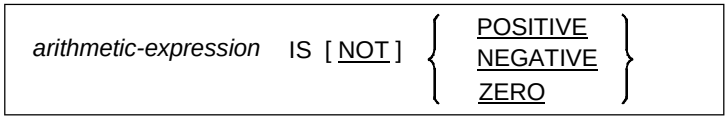
F.7.3. ANY Condition



F.7.4. Class Condition



F.7.5. Sign Condition



F.7.6. Condition-Name Condition

condition-name

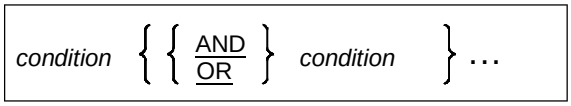
F.7.7. Switch-Status Condition

condition-name

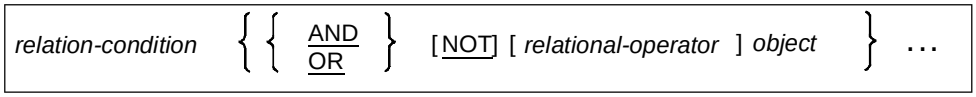
F.7.8. Negated Simple Condition

NOT *simple-condition*

F.7.9. Combined Condition



F.7.10. Abbreviated Combined Relation Condition



F.8. Miscellaneous Formats

F.8.1. Qualification

Format 1

```
{ data-name-1     }     [ { OF } data-name-2 ] ...
{ condition-name  }     [ { IN }             ]
```

Format 2

```
paragraph-name  [ { OF } section-name ]
                  { IN }
```

Format 3

```
text-name  [ { OF } library-name ]
             { IN }
```

F.8.2. Subscripting

```
{ data-name      }  ( subscript-1 [ , subscript-2 [ , subscript-3 ] ] )
{ condition-name }
```

F.8.3. Indexing

```
{ data-name      }  (  { index-name-1 [ { + } literal-2 ] }
{ condition-name }     { literal-1                       }

   [ , { index-name-2 [ { + } literal-4 ] } [ , { index-name-3 [ { + } literal-6 ] } ] ] )
       { literal-3                            { literal-5                           }
```

F.8.4. Identifier

Format 1

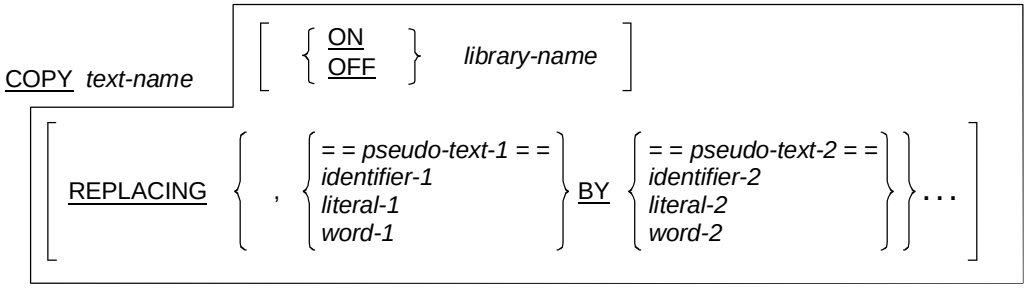
```
data-name-1  [ { OF } data-name-2 ] ...  [ ( subscript-1 [ , subscript-2 [ , subscript-3 ] ] ) ]
               { IN }
```

Format 2

```
data-name-1  [ { OF } data-name-2 ] ...  [ (  { index-name-1 [ { + } literal-2 ] }
               { IN }                        { literal-1                       }

   [ , { index-name-2 [ { + } literal-4 ] } [ , { index-name-3 [ { + } literal-6 ] } ] ] ) ]
       { literal-3                            { literal-5                           }
```

F.9. The COPY Statement



Glossary

A

abbreviated combined relation condition
The combined condition that results from the explicit omission of a common subject or a common subject and common relational operator in a consecutive sequence of relation conditions.

access mode
The manner in which records are to be operated upon within a file.

ACS
Abbreviation for Asynchronous Control System.

actual decimal point
The physical representation, using either the decimal point characters period (.) (American) or comma (,) (European), of the decimal point position in a data item.

alphabet name
A user-defined word in the SPECIAL-NAMES paragraph of the Environment Division that assigns a name to a specific character set or collating sequence.

Alphabet Section
The section of the Control Division that describes the extended COBOL character set and specifies what language to use when printing diagnostics.

alphabetic character
A character that belongs to the following set of letters: A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, and the space. Also includes the lowercase letters a, b, ..., z.

alphanumeric character
Any character in the ISO8859-1 native character set on Unisys OS 2200 systems.

alternate record key
A key, other than the prime record key, whose contents identify a record within an indexed file.

American Standard Code for Information Interchange (ASCII)
A system standard that assigns specific symbols and numeric codes to each control and graphic character.

ANSI

Abbreviation for American National Standards Institute.

area-name

A data-name associated with an SA level indicator.

arithmetic expression

An identifier of a numeric elementary item, a numeric literal, such identifiers and literals separated by arithmetic operators, two arithmetic expressions separated by an arithmetic operator, or an arithmetic expression enclosed in parentheses.

arithmetic operator

A single character or fixed two-character combination that belongs to the following set: subtraction (–), multiplication (*), division (/), or exponentiation (**).

Array-Sync

A data item is array-sync if the length of one occurrence is a multiple of a word.

ascending key

A key upon the value of which data is ordered starting with the lowest value of key up to the highest value of key in accordance with the rules for comparing data items.

ASCII

See American Standard Code for Information Interchange.

assumed decimal point

A decimal point position which does not involve the existence of an actual character in a data item. The assumed decimal point has logical meaning but no physical representation.

Asynchronous Control System (ACS)

A control system that directs, or schedules, the execution of asynchronous processing cycles.

asynchronous processing

The manner of processing logical records within USE procedures that are initiated in consecutive relation to one another but are not necessarily executed or completed in order in which they are initiated; no specific processing cycle is necessarily before subsequent cycle is initiated.

at end condition

A condition caused:

- During the execution of a READ statement for a sequentially accessed file, when no next logical record exists in the file, when the number of significant digits in the relative record number is larger than the size of the relative key data item, or when an optional file is not present.
- During the execution of a RETURN statement, when no next logical record exists for the associated sort or merge file.
- During the execution of a SEARCH statement, when the search operation terminates without satisfying the condition specified in any of the associated WHEN phrases.

B

block

A physical unit of data that is normally composed of one or more logical records. For mass storage files, a block can contain a portion of a logical record. The size of a block has no direct relationship to the size of the file within which the block is contained or to the size of the logical records that are either contained within the block or that overlap the block. The term is synonymous with physical record.

body group

Generic name for a report group of TYPE DETAIL, CONTROL HEADING, or CONTROL FOOTING.

C

called program

A program that is the object of a CALL or ENTER statement combined at object time with the calling program to produce a run unit.

calling program

A program that executes a CALL or ENTER statement to another program.

cd-name

A user-defined word that names an MCS interface area described in a Communication Description entry within the Communication Section of the Data Division.

character

The basic indivisible unit of the language.

character position

The amount of physical storage required to store a single standard data format character described as usage is DISPLAY.

character-string

A sequence of contiguous characters that form a COBOL word, a literal, a PICTURE character-string, or a comment-entry.

class condition

The proposition, for which a truth value can be determined, that the content of an item is wholly alphabetic or is wholly numeric or consists exclusively of those characters listed in the definition of a class name.

clause

A clause is an ordered set of consecutive COBOL character strings whose purpose is to specify an attribute of an entry.

COBOL character set

The complete COBOL character set consists of the following 77 characters:

Characters	Meaning
0, 1, ..., 9	Digits
A, B, ..., Z	Uppercase letters
a, b, ..., z	Lowercase letters
	Spaces
+	Plus sign
–	Minus sign (hyphen)
*	Asterisk
/	Slant (solidus)
=	Equal sign
$	Currency sign
,	Comma (European decimal point)
;	Semicolon
.	Period (decimal point, full stop)
"	Quotation mark
(	Left parenthesis
)	Right parenthesis
>	Greater than symbol
:	Colon

The COBOL character set can be extended to include characters not in the list of 77 characters. The SOURCE-ALPHABET clause of the Control Division can extend the COBOL character whose ordinal position is between 128 and 255 in the ASCII character set. If the SOURCE-ALPHABET clause includes any lowercase letters or the M extra option is specified on the processor call statement, then the lowercase letters are not included unless explicitly stated in the SOURCE-ALPHABET clause.

COBOL word

See word.

collating sequence

The sequence in which the characters that are acceptable in a computer are ordered for purposes of sorting, merging, comparing, and processing indexed files sequentially.

column

A character position within a print line. The columns are numbered from one, by one, starting at the leftmost character position of the print line and extending to the rightmost position of the print line.

combined condition
A condition that is the result of connecting two or more conditions with the AND or the OR logical operator.

comment-entry
An entry in the Identification Division that can be any combination of characters from the computer's character set.

comment line
A source program line represented by an asterisk (*) in the indicator area (column 7) of the line and any characters from the computer's character set in area A and area B of that line. The comment line is only for documentation in a program. A special form of comment line represented by a stroke (/) in the indicator area of the line and any characters from the computer's character set in area A and area B of that line causes page ejection prior to printing the comment.

Common-Storage Section
The section of the Data Division that describes data items, composed of either noncontiguous items, records, or both, that can be shared by two or more programs in a run unit.

Communication Description entry
An entry in the Communication Section of the Data Division that is composed of the level indicator CD, followed by a cd-name, and then followed by a set of clauses as required. It describes the interface between the Message Control System (MCS) and the COBOL program.

communication device
A mechanism (hardware or hardware/software) capable of sending data to a queue and/or receiving data from a queue. This mechanism may be a computer or a peripheral device. One or more programs containing communication description entries and residing within the same computer define one or more of these mechanisms.

Communication Section
The section of the Data Division that describes the interface areas between the MCS and the program, composed of one or more CD description entries.

compile time
The time during which a COBOL source program is translated, by a COBOL compiler, to a COBOL object program.

compiler directing statement
A statement, beginning with a compiler directing verb, that causes the compiler to take a specific act during compilation.

complex condition
A condition in which one or more logical operators act upon one or more conditions. (*See* negated simple condition, combined condition, negated combined condition.)

computer-name
A system-name that identifies the computer upon which the program is to be compiled or run.

condition

A status of a program at execution time for which a truth value can be determined. Where the term "condition" (*condition-1, condition-2* . . .) appears in these language specifications or in reference to "condition" (*condition-1, condition-2* . . .) of a general format, it is a conditional expression consisting of either a simple condition optionally parenthesized, or a combined condition consisting of the syntactically correct combination of simple conditions, logical operators, and parentheses, for which a truth value can be determined.

condition ANY/ALL

The proposition, for which a truth value can be determined, that asynchronously processed procedures and data areas are active and inactive at a given instant.

condition-name

A user-defined word assigned to a specific value, set of values, or range of values, within the complete set of values that a conditional variable may posses; or the user-defined word assigned to a status of an implementor-defined switch or device.

condition-name condition

The proposition, for which a truth value can be determined, that the value of a conditional variable is a member of the set of values attributed to a condition-name associated with the conditional variable.

condition word, system

The word of storage set by the Executive @SETC statement and used to determine the truth value of switch-status conditions.

conditional expression

A simple condition or a complex condition specified in an IF, PERFORM, or SEARCH statement. *See* simple condition and complex condition.

conditional statement

A statement specifying that the truth value of a condition is to be determined and that the subsequent action of the object program is dependent on this truth value.

conditional variable

A data item in which one or more values has a condition name assigned to it.

Configuration Section

A section of the Environment Division that describes overall specifications of source and object computers.

connective

A reserved word that is used to:

- Associate a data-name, paragraph-name, condition-name, or text-name with its qualifier.
- Link two or more operands written in a series.
- Form conditions (logical connectives). (See logical operator.)

contiguous items
Items that are described by consecutive entries in the Data Division, and that bear a definite hierarchic relationship to each other.

control break
A change in the value of a data item referred to in the CONTROL clause. More generally, a change in the value of a data item used to control the hierarchical structure of a report.

control break level
The relative position within a control hierarchy at which the most major control break occurred.

control data item
A data item, a change in whose contents can produce a control break.

control data-name
A data-name that appears in a CONTROL clause and refers to a control data item.

control footing
A report group presented at the end of the control group of which it is a member.

control group
A set of body groups presented for a given value of a control data item or of FINAL. Each control group can begin with a CONTROL HEADING, end with a CONTROL FOOTING, and contain DETAIL report groups.

control heading
A report group that is presented at the beginning of the control group of which it is a member.

control hierarchy
A designated sequence of report subdivisions defined by the positional order of FINAL and the data names within a CONTROL clause.

counter
A data item used for storing numbers or number representations in a manner that permits these numbers to be increased or decreased by the value of another number, or to be changed or reset to zero or to an arbitrary positive or negative value.

currency sign
The character "$" of the COBOL character set.

currency symbol
The character defined by the CURRENCY SIGN clause in the SPECIAL-NAMES paragraph. If no CURRENCY SIGN clause is present in a COBOL source program, the currency symbol is identical to the currency sign.

current record
In file processing, the record that is available in the record area associated with the file.

current record pointer
A conceptual entity that is used in the selection of the next record.

cycle

The execution of a subtask.

cycle-extension

Transfer of control to a procedure outside the current main program or random Processing declarative through execution of the GO, TO, PERFORM, CALL, and ENTER statements.

D

data clause

A clause that appears in a data description entry in the Data Division and provides information describing a particular attribute of a data item.

data description entry

An entry in the Data Division that is composed of a level number followed by a data name, if required, and then followed by a set of data clauses, as required.

data item

A character or a set of contiguous characters (excluding in either case literals) defined as a unit of data by the COBOL program.

data-name

A user-defined word that names a data item described in a data description entry in the Data Division. When used in the general formats, "data-name" represents a word that cannot be subscripted, indexed, or qualified unless specifically permitted by the rules of the format.

debugging line

Any line with D in the indicator area of the line.

debugging section

A section that contains a USE FOR DEBUGGING statement.

declaratives

A set of one or more special purpose sections written at the beginning of the Procedure Division, the first of which is preceded by the keyword DECLARATIVES and the last of which is followed by the keywords END DECLARATIVES. A declarative is composed of a section header, followed by a USE compiler directing sentence, followed by a set of zero, one, or more associated paragraphs.

declarative sentence

A compiler-directing sentence consisting of a single USE statement terminated by the separator period.

delimiter

A character or a sequence of contiguous characters that identify the end of a string of characters and separates that string of characters from the following string of characters. A delimiter is not part of the string of characters that it delimits.

derailment

A temporary suspension of a cycle execution by the ACS for any cause, such as waiting for I-O completion or waiting for a lock field to be unlocked. Upon removal of the cause, the cycle is eligible for resumption. When several cycles are eligible for resumption the master-task will be resumed if it has been suspended. Otherwise, the ACS will resume the cycle with the highest cycle priority, which is the sum of the ranks of the Random Processing Section and the associated Saved-Area. The greater the sum of the two numbers, the higher the priority.

descending key

A key upon the values of which data is ordered, starting with the highest value of a key down to the lowest value of key, in accordance with the rules for comparing data items.

destination

The symbolic identification of the receiver of a transmission from a queue.

digit position

A digit position is the amount of physical storage required to store a single digit. This amount can vary depending on the usage of the data item describing the digit position. If the data description entry specifies that usage is DISPLAY, then a digit position is synonymous with a character position.

division

A collection of zero, one, or more sections of paragraphs, called the division body, that are formed and combined in accordance with a specific set of rules. Each division consists of the division header and the related division body. There are five divisions in a COBOL program: Control, Identification, Environment, Data, and Procedure.

division header

A combination of words followed by a period and a space, that indicates the beginning of a division. The division headers are

```
CONTROL DIVISION.
IDENTIFICATION DIVISION.
ENVIRONMENT DIVISION.
DATA DIVISION.
PROCEDURE DIVISION   [ [ USING ] { data-name-1 } . . . ]
 [ [ WITH ] ENTRY [ POINTS ] { procedure-name-1 } . . . ] .
```

dynamic access

An access mode in which specific logical records can be obtained from or placed into a mass storage file in a nonsequential manner (see random access) and obtained from a file in a sequential manner (see sequential access) during the scope of the same OPEN statement.

E

editing character

A single character or a fixed two-character combination belonging to the following set:

character	meaning
B	space
0	zero
+	plus
-	minus
CR	credit
DB	debit
Z	zero suppress
*	check protect
$	currency sign
,	comma (decimal point)
.	period (decimal point)
/	stroke (virgule, slash)

elementary item

A data item described as not being further logically subdivided.

end of procedure division

The physical position in a COBOL source program after which no further procedures appear.

entry

Any descriptive set of consecutive clauses terminated by a separator period and written in the Identification, Environment, or Data Division of a COBOL source program.

environment clause

A clause that appears as part of an Environment Division entry.

execution time

The time during which an object module is executed. *Synonym for* object time.

extend mode

The state of a file after execution of an OPEN statement, with the EXTEND phrase specified, for that file and before the execution of a CLOSE statement without the REEL or UNIT phrase for that file.

F

Fieldata

A six-bit (sixth word) character code.

figurative constant

A compiler generated value referred to through the use of certain reserved words.

file
A collection of records.

file clause
A clause that appears as part of any of the following Data Division entries:

File description (FD)
Sort/merge file description (SD)
Communication description (CD)

file control entry
A SELECT clause and all its subordinate clauses that declare the relevant physical attributes of a file.

file description entry
An entry in the File Section of the Data Division composed of the level indicator FD, followed by a file-name, and then followed by a set of file clauses as required.

file-name
A user-defined word that names a file described in a file description entry or a sort-merge file description entry within the File Section of the Data Division.

file organization
The permanent logical file structure established when a file is created.

File Section
The section of the Data Division that contains file description entries and sort-merge file description entries together with their associated record descriptions.

FIPS
Abbreviation for Federal Information Processing Standard.

format
A specific arrangement of a set of data.

G

group item
A named contiguous set of elementary or group items.

H

high-order end
The leftmost character of a string of characters.

host-cycle
A cycle whose execution has included execution of at least one PROCESS statement, and hence, a cycle from which slave-cycles have emanated.

host-task

A sub-task that contains a PROCESS statement in its instruction sequence.

HVTIP

Abbreviation for High-Volume Transaction Processing.

I

I-O

Abbreviation for input-output.

I-O-CONTROL

The name of an Environment Division paragraph in which object program requirements for specific input-output techniques, rerun points, sharing of same areas by several data files, and multiple file storage on a single input-output device are specified.

I-O Mode

The state of a file after execution of an OPEN statement, with the I-O phrase specified, for that file and before the execution of a CLOSE statement for that file.

identifier

A data-name, followed as required, by the syntactically correct combination of qualifiers, subscripts, and indices necessary to make unique reference to a data item.

imperative statement

A statement that begins with an imperative verb and specifies an unconditional action to be taken. An imperative statement can consist of a sequence of imperative statements.

implementor-name

A system name that refers to a particular feature available in the Unisys OS 2200 Executive System.

index

A computer storage position or register, the contents of which represent the identification of a particular element in a table.

index data item

A data item in which the values associated with an index name can be stored.

index-name

A user-defined word that names an index associated with a specific table.

indexed data-name

An identifier that is composed of a data-name, followed by one or more index-names enclosed in parentheses.

indexed file

A file with indexed organization.

indexed organization
The permanent logical file structure in which each record is identified by the value of one or more keys within that record.

Inline Section
The section of the Control Division that describes where inline code is to be generated.

input file
A file opened in the input mode.

input mode
The state of a file after execution of an OPEN statement with the INPUT phrase specified for that file, and before the execution of a CLOSE statement without the REEL or UNIT phrase specified for that file.

input-output file
A file opened in the I-O mode.

Input-Output Section
The section of the Environment Division that names the files and the external media required by an object program and which provides information required for transmission and handling of data during execution of the object program.

input procedure
A set of statements that are executed each time a record is released to the sort file.

integer
A numeric literal or a numeric data item that does not include any character positions to the right of the assumed decimal point. Where the term "integer" appears in general formats, integer must not be a numeric data item, and must not be signed, nor zero unless explicitly allowed by the rules of that format.

invalid key condition
A condition, at object time, caused when a specific value of the key associated with an indexed or relative file is determined to be invalid.

ISO
Abbreviation for International Standards Organization.

ISO8859-1
An ISO 8-bit coded character set standard. Also called the Latin-1 alphabet.

K

key
A data item which identifies the location of a record, or a set of data items which serve to identify the ordering of data.

key of reference
The key, either prime or alternate, currently being used to access records within an indexed file.

key word
A reserved word whose presence is required when the format in which the word appears is used in a source program.

L

level indicator
Two alphabetic characters that identify a specific type of file or a position in a hierarchy. The level indicators in the Data Division are CD, FD, RD, and SD.

level number
A user-defined word which indicates the position of a data item in the hierarchical structure of a logical record or which indicates special properties of a data description entry. A level-number is expressed as a one or two digit number. Level numbers in the range 1 through 9 can be written either as a single digit or as a zero followed by a significant digit. Level numbers 66, 77, and 88 identify special properties of a data description entry.

library-name
A user-defined word that names a COBOL library to be used by the compiler for a given source program compilation.

library text
A sequence of character-strings and/or separators in a COBOL library.

line
See report line.

line number
An integer that denotes the vertical position of a report line on a page.

Linkage Section
The section in the Data Division of the called program that describes data items available from the calling program. These data items can be referred to by both the calling and the called program.

literal
A character string whose value is implied by the ordered set of characters comprising the string.

logical operator
One of the reserved words AND, OR, or NOT. In the formation of a condition, both or either of AND and OR can be used as logical connectives. NOT can be used for logical negation.

logical record
The most inclusive data item. The level-number for a record is 01. A record can be either an elementary item or a group item. *Synonym for* record.

low order end
The rightmost character of a string of characters.

M

mass storage
A storage medium on which data may be organized and maintained in both a sequential and nonsequential manner.

Mass Storage Control System (MSCS)
An input-output control system that directs, or controls, the processing of mass storage files.

mass storage file
A collection of records assigned to a mass storage medium.

master-cycle
The master-cycle is the execution of the master-task.

master-task
The master-task of the COBOL run unit includes the system entry of the COBOL main program and its extension to any level by virtue of GO TO, PERFORM, CALL, and ENTER statements imbedded in the master-task. Since only one COBOL main program can be collected with a COBOL run unit, the master-task is uniquely defined.

MCS
See Message Control System.

merge file
A collection of records to be merged by a MERGE statement. The merge file is created and can be used only by the merge function.

message
Data associated with an end of message indicator or an end of group indicator. (See message indicators.)

Message Control System (MCS)
A communication control system that supports the processing of messages.

message count
The count of the number of complete messages that exist in the designated queue of messages.

message indicators
End of Group Indicator (EGI), End of Message Indicator (EMI), and End of Segment Indicator (ESI) are conceptual indicators that serve to notify the MCS that a specific condition exists (end of group, end of message, end of segment).

Within the hierarchy EGI, EMI, and ESI of message indicators, an EGI is conceptually equivalent to an ESI, EMI, and EGI. An EMI is conceptually equivalent to an ESI and EMI. Thus, a segment may be terminated by an ESI, EMI, or EGI. A message may be terminated by an EMI or EGI.

message segment
Data that forms a logical subdivision of a message normally associated with an end of segment indicator. (See message indicators.)

mnemonic-name
A user-defined word that is associated in the Environment Division with a specific implementor-name.

MSCS
See Mass Storage Control System.

N

native character set
The character set defined by Unisys and associated with the computer specified in the OBJECT-COMPUTER paragraph.

native collating sequence
The collating sequence defined by Unisys and associated with the computer specified in the OBJECT-COMPUTER paragraph.

negated combined condition
The "NOT" logical operator immediately followed by a parenthesized combined condition.

negated simple condition
The "NOT" logical operator immediately followed by a simple condition.

next executable sentence
The next sentence to which control will be transferred after execution of the current statement is complete.

next executable statement
The next statement to which control will be transferred after execution of the current statement is complete.

next record
The record which logically follows the current record of a file.

Non-array-sync
A date item is non-array-sync if the length of one occurrence is not a multiple of a word.

noncontiguous items
Elementary data items in the Working-Storage, Linkage, and Common-Storage Sections that bear no hierarchical relationship to other data items.

nonnumeric item
A data item whose description restricts its contents to a value represented by characters chosen from the digits "0" through "9"; if signed, the item may also contain a "+", "-", or other representation of an operational sign.

nonnumeric literal
A literal bounded by quotation marks. The string of characters can include any character in the computer's character set.

numeric character
A character from the following set of digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

numeric item
A data item whose description restricts its contents to a value represented by characters chosen from the digits "0" through "9". If signed, the item can also contain a plus sign (+), minus sign (–), or other representation of an operational sign.

numeric literal
There are two types of numeric literals: fixed-point and floating-point. A fixed-point literal is a literal composed of one or more numeric characters that can contain a decimal point, an algebraic sign, or both. The decimal point must not be the rightmost character. The algebraic sign, if present, must be the leftmost character.

A floating-point literal is a literal composed of a fixed-point numeric literal that can have a decimal point in any character position (the mantissa) immediately followed by the letter E that is, in turn, immediately followed by a fixed-point literal consisting of one to three digits that does not contain a decimal point.

O

OBJECT-COMPUTER
The name of an Environment Division paragraph in which the computer environment, within which the object program is executed, is described.

object of entry
A set of operands and reserved words, within a Data Division entry, that immediately follows the subject of the entry.

object program
A set or group of executable machine language instructions and other material designed to interact with data to provide problem solutions. In this context, an object program is generally the machine language result of the operation of a COBOL compiler on a source program. Where there is no danger of ambiguity, the word "program" can be used in place of the phrase "object program."

object time
The time at which an object program is executed. *Synonym for* execution time.

open mode
The state of a file after execution of an OPEN statement for that file and before the execution of a CLOSE statement without the REEL or UNIT phrase for that file. The particular open mode is specified in the OPEN statement as either INPUT, OUTPUT, I-O, or EXTEND.

operand
Any lowercase words that appear in a statement or entry format. An operand is an implied reference to the data indicated by the operand.

operational sign
An algebraic sign, associated with a numeric data item or a numeric literal, to indicate whether its value is positive or negative.

optional word
A reserved word included in a specific format only to improve the readability of the language and whose presence is optional to the user when the format in which the word appears is used in a source program.

output file
A file opened in either the output mode or extend mode.

output mode
The state of a file after execution of an OPEN statement for that file, with the OUTPUT or EXTEND phrase specified, and before the execution of a CLOSE statement without the REEL or UNIT phrase for that file.

output procedure
A set of statements to which control is given during execution of a SORT statement after the sort function is completed, or during execution of a MERGE statement after the merge function has selected the next record in merged order.

P

page
A vertical division of a report representing a physical separation of report data, the separation being based on internal reporting requirements and external characteristics of the reporting medium.

page body
That part of the logical page in which lines can be written and spaced.

page footing
A report group presented at the end of a report page, as determined by the Report Writer Control System.

page heading
A report group presented at the beginning of a report page, as determined by the Report Writer Control System.

paragraph
In the Procedure Division, a paragraph name followed by a separator period and by zero, one, or more sentences. In the Identification and Environment Divisions, a paragraph header followed by zero, one, or more entries.

paragraph header

A reserved word, followed by the separator period, that indicates the beginning of a paragraph in the Identification Division and Environment Divisions. The permissible paragraph headers in the Identification Division are:

```
PROGRAM-ID.
AUTHOR.
INSTALLATION.
DATE-WRITTEN.
DATE-COMPILED.
SECURITY.
```

The permissible paragraph headers in the Environment Division are:

```
SOURCE-COMPUTER.
OBJECT-COMPUTER.
SPECIAL-NAMES.
FILE-CONTROL.
I-O-CONTROL.
```

paragraph-name

A user-defined word that identifies and begins a paragraph in the Procedure Division.

phrase

An ordered set of one or more consecutive COBOL character-strings that form a portion of a COBOL procedural statement or of a COBOL clause.

physical record

See block.

prime record key

A key whose contents uniquely identify a record within an indexed file.

printer file

A file assigned to ANS74-PRINTER or PRINTER.

printable group

A report group that contains at least one print line.

printable item

A data item, the extent and contents of which are specified by an elementary report entry. This elementary report entry contains a COLUMN NUMBER clause, a PICTURE clause, and a SOURCE, SUM, or VALUE clause.

PROC

A statement accepted by the Procedure Definition Processor that prepares library text in a COBOL library. *See also* library text.

procedure

A paragraph or group of logically successive paragraphs, or a section or group of logically successive sections, within the Procedure Division.

procedure-name
A user-defined word which is used to name a paragraph or section in the Procedure Division. It consists of a paragraph-name (which may be qualified) or a section-name.

processing cycle
A single execution of a defined USE procedure.

progeny-cycles
Progeny-cycles are always references with respect to a host-cycle to a host-cycle and include "children" slave-cycles, "grandchildren" slave-cycles, etc. All slave-cycles are progeny-cycles of master-cycle.

program-name
A user-defined word that identifies a COBOL source program.

pseudo-text
A sequence of character-strings and/or separators bounded by, but not including, pseudo-text delimiters.

pseudo-text delimiter
Two contiguous equal sign (=) characters used to delimit pseudo-text.

punctuation character
A character that belongs to the following set:

character	meaning
,	comma (decimal point)
;	semicolon
.	period
"	quotation mark
(	left parenthesis
)	right parenthesis
	space
=	equal sign

Q

qualified data name
An identifier that is composed of a data-name followed by one or more sets of either of the connectives OF and IN followed by a data-name qualifier.

qualifier
(1) A data-name which is used in a reference together with another data name at a lower level in the same hierarchy.

(2) A section-name which is used in a reference together with a paragraph name specified in that section.

(3) A library-name which is used in a reference together with a text name associated with that library.

queue
A logical collection of messages awaiting transmission or processing.

queue name
A symbolic name that indicates to the MCS the logical path by which a message or a portion of a completed message may be accessible in a queue.

R

random access
An access mode in which the program-specified value of a key data item identifies the logical record obtained from, deleted from, or placed into a relative or indexed file.

record
A sequence of values or characters treated as a unit. Records do not necessarily correspond to a particular physical representation. *See* logical record.

record area
A storage area allocated for the purpose of processing the record described in a record description entry in the File Section or the Data Division. In the File Section, the current number of character positions in the record area is determined by the explicit or implicit RECORD clause.

record description
See record description entry.

record description entry
The total set of data description entries associated with a particular record.

record key
A key whose contents identify a record within an indexed file. A record key is either the prime record key or an alternate record key, whose contents identify a record within an indexed file.

record name
A user-defined word that names a record described in a record description entry in the Data Division.

reel
An individual physical volume of magnetic tape that contains part of a file, all of a file, or any number of files. One or more files can reside on one or more reels of tape. *Synonym for* unit and volume.

reference format
A format that provides a standard method for describing COBOL source programs.

relation
See relational operator.

relation character
A character used to specify a comparison between two values. Relation characters are > (greater than), < (less than), or = (equal).

relation condition
A reserved word, relation character, group of consecutive reserved words, or group of consecutive reserved words and relation characters used in the construction of a relation condition.

relational operator
A reserved word, relation character, group of consecutive reserved words, or group of consecutive reserved words and relation characters used in the construction of a relation condition. The permissible operators and their meaning are:

Relative Operator	Meaning
IS [NOT] GREATER THAN IS [NOT} >	Greater than or not greater than
IS [NOT] LESS THAN IS [NOT] <	Less than or not less than
IS [NOT] EQUAL TO IS [NOT] =	Equal to or not equal to

relative file
A file with relative organization.

relative key
A key whose contents identify a logical record in a relative file.

relative organization
The permanent logical file structure in which each record is uniquely identified by an integer value greater than zero that specifies the record's logical ordinal position in the file.

report clause
A clause in the Report Section of the Data Division that appears in a report description entry or a report group description entry.

report description entry
An entry in the Report Section of the Data Division composed of the level indicator RD, followed by the report name, followed by a set of report clauses as required.

report file
An output file whose file description entry contains a REPORT clause. The contents of a report file consist of records that are written under control of the Report Writer Control System.

report footing
A report group presented only at the end of a report.

report group
In the Report Section of the Data Division, an 01 level-number entry and its subordinate entries.

report group description entry
An entry in the Report Section of the Data Division that is composed of the level number 01, an optional data name, a TYPE clause, and an optional set of report clauses.

report heading
A report group presented only at the beginning of a report.

report line
A division of a page representing one row of horizontal character positions. Each character position of a report line is aligned vertically beneath the corresponding character position of the report line above it. Report lines are numbered from 1, by 1, starting at the top of the page.

report name
A user-defined word that names a report described in a report description entry within the Report Section of the Data Division.

Report Section
The section of the Data Division that contains zero, one, or more report description entries and their associated report group description entries.

report writer control system (RWCS)
An object time control system that accomplishes the construction reports.

report writer logical record
A record that consists of the report writer print line and associated control information necessary for its selection and vertical positioning.

reserved word
A COBOL word specified in the list of words which can be used in a COBOL source program, but which must not appear in the program as user-defined words or system- names.

routine-name
A user-defined word that identifies a procedure written in a language other than COBOL.

run
A sequence of job executions following an @RUN statement and preceding the associated @FIN statement.

run unit
A set of one or more object programs which function, at object time, as a unit to provide problem solutions.

RWCS
The Report Writer Control System.

S

saved area description entry

An entry in the File Section of the Data Division that is composed of the level indicator SA, followed by a data-name that is the name assigned to particular saved area, and then followed by a set of file clauses as required.

saved-area

A storage area, described in the Data Division, which is composed of one or more data records.

schedule-unit

A slave-task, which is capable of executing asynchronously with other sub-tasks and is activated through execution or a PROCESS statement.

section

A set of zero, one, or more paragraphs or entries, called a section body, the first of which is preceded by a section header. Each section consists of the section header and the related section body.

section header

A combination of words followed by a period and a space that indicates the beginning of a section in the Control, Environment, Data, and Procedure Division.

In the Control, Environment, and Data Divisions, a section header is composed of reserved words followed by a period and a space. The permissible section headers are:

In the Control Division

ALPHABET SECTION.
INLINE SECTION.

In the Environment Division:

```
CONFIGURATION SECTION.
INPUT-OUTPUT SECTION.
```

In the Data Division:

```
FILE SECTION.
WORKING-STORAGE SECTION.
COMMON-STORAGE SECTION.
LINKAGE SECTION.
COMMUNICATION SECTION.
REPORT SECTION.
```

In the Procedure Division, a section header is composed of a section name followed by the reserved word SECTION, followed by a segment number (optional), followed by a period and a space.

section-name

A user-defined word that names a section in the Procedure Division.

segment-number
A user-defined word that classifies sections in the Procedure Division for segmentation. Segment numbers can contain only the characters 0, 1, ..., 9. A segment number can be expressed either as a 1- or 2-digit number.

sentence
A sequence of one or more statements, the last of which is terminated by a period followed by a space.

separator
A punctuation character used to delimit character-strings.

sequential access
An access mode in which logical records are obtained from or placed into a file in a consecutive predecessor-to-successor logical record sequence determined by the order of records in the file.

sequential file
A file with sequential organization.

sequential organization
The permanent logical file structure in which a record is identified by a predecessor-successor relationship established when the record is placed into the file.

shared data items
Shared data items include data-record areas for items defined in the File, Working-Storage, Linkage, and Common-Storage Sections.

sibling-cycles
Sibling-cycles are slave-cycles that are activated by the same host-cycle.

sign condition
The proposition, for which a truth value can be determined, that the algebraic value of a data item or an arithmetic expression is less than, greater than, or equal to zero.

simple condition
Any single condition chosen from this set:

relation condition
class condition
condition-name condition
switch-status condition
sign condition
(simple-condition)

slave-cycle
A slave-cycle is the execution of a slave-task.

slave-task

A slave-task is a Random Processing Declarative Section including extensions to any level by virtue of GO TO, PERFORM, CALL, and ENTER statements imbedded in the slave-task. Salve-tasks may be defined in any COBOL program within a COBOL run unit.

sort file

A collection of records to be sorted by a SORT statement. The sort file is created and can be used by the sort function only.

sort-merge file description entry

An entry in the File Section of the Data Division composed of the level indicator SD, followed by a file name, and then followed by a set of file clauses as required.

source

The symbolic identification of the originator of a transmission to a queue.

SOURCE-COMPUTER

The name of an Environment Division paragraph in which the computer environment, within which the source program is compiled, is described.

source item

An identifier designated by a SOURCE clause that provides the value of a printable item.

source program

A syntactically correct set of COBOL statements beginning with either the optional Control Division or an Identification Division and ending with the end of the Procedure Division. In contexts where there is no danger of ambiguity, the word "program" alone may be used in place of the phrase "source program."

special character

A character that belongs to the following set:

Character	Meaning
+	Plus sign
–	Minus sign
*	Asterisk
/	Stroke (virgule, solidus)
=	Equal sign
$	Currency sign
,	Comma (decimal point)
;	Semicolon
.	Period (decimal point, full stop)
"	Quotation mark
(	Left parenthesis
)	Right parenthesis
>	Greater than symbol
<	Less than symbol

special-character word

A reserved word which is an arithmetic operator or a relation character.

SPECIAL-NAMES
The name of an Environment Division paragraph in which implementor names are related to user specified mnemonic-names.

special registers
Compiler generated storage areas whose primary use is to store information produced in conjunction with the use of specific COBOL features.

statement
A syntactically valid combination of words and symbols written in the Procedure Division beginning with a verb.

sub-queue
A logical hierarchical division of a queue.

subject of entry
An operand or reserved word that appears immediately following the level indicator of the level-number in a Data Division entry.

subprogram
See called program.

subscript
An occurrence number represented by either an arithmetic expression or an index name optionally followed by an integer with the operator + or – that identifies a particular element in a table.

subscripted data name
An identifier that is composed of a data name followed by one or more subscripts enclosed in parentheses.

sub-task
A generic term used whenever it is unnecessary to differentiate between master-task and slave-task.

sum counter
A signed numeric data item established by a SUM clause in the Report Section of the Data Division. The sum counter is used by the Report Writer Control System to contain the result of designated summing operations that take place during production of a report.

switch-status condition
The proposition, for which a truth value can be determined, that a switch, capable of being set to an on or off status, has been set to a specific status.

system condition word
The word of storage set by the Executive @SETC statement and used to determine the truth value of switch-status conditions.

system-name
A COBOL word used to communicate with the operating environment.

T

table
A set of logically consecutive items of data that are defined in the Data Division by the OCCURS clause.

table element
A data item that belongs to the set of repeated items comprising a table.

terminal
The originator of a transmission to a queue, or the receiver of a transmission from a queue.

text-name
A user-defined word that identifies library text.

text word
Any character-string or separator, except space, in a COBOL library or in pseudo-text.

truth value
The representation of the result of the evaluation of a condition in terms of one of two values: true or false.

U

unary operator
A plus (+) or minus (–) sign which precedes a variable or a left parenthesis in an arithmetic expression and which has the effect of multiplying the expression by +1 or –1, respectively.

unit
For tape files, *synonym for* reel and volume. For mass storage files, no concept of multiple units exists. Each mass storage file resides entirely on one logical unit. Since mass storage file space allocation is controlled by the operating system, this restriction implies no limitations on the physical layout of a mass storage file on the available system resources.

user-defined word
A COBOL word that must be supplied by the user to satisfy the format of a clause or statement.

V

variable
A data item whose value may be changed by execution of the object program. A variable used in an arithmetic expression must be a numeric elementary item.

verb
A word that expresses an action to be taken by a COBOL compiler or object program.

W

word

A character string of not more than 30 characters that forms a user-defined word, a system name, or a reserved word.

Working-Storage Section

The section of the Data Division that describes working-storage data items, composed of noncontiguous items, working storage records, or both.

77-level description entry

A data description entry that describes a noncontiguous data item with the level number 77.

Index

A

ACCEPT MESSAGE COUNT statement
  USE FOR DEBUGGING statement, 11-7
ACCEPT statement
  Mnemonic-name, 2-10
  SPECIAL-NAMES paragraph, 2-10
ACCESS MODE
  SEQUENTIAL, 15-2
ACCESS MODE clause
  DYNAMIC, 5-5, 6-8
  RANDOM, 5-5, 6-8
  SEQUENTIAL, 4-4, 5-5, 16-2
ADD statement
  composite of operands, 2-67
  COMPUTE statement, 2-86
  CORRESPONDING, 2-65
  data conversion, 2-67
  decimal alignment, 2-67
  maximum operand size, 2-67
AFTER phrase
  INSPECT statement, 2-95
  PERFORM statement, 2-106
Alignment of data
  MOVE statement, 2-101
ALL
  INSPECT statement, 2-95
  SEARCH statement, 3-7
  USE FOR DEBUGGING statement, 11-6
ALL literal
  INSPECT statement, 2-95
  STOP statement, 2-113
  STRING statement, 2-114
  UNSTRING statement, 2-119
  Alphabet-name
  CODE-SET clause, 4-10
  SORT statement, 7-12
ALTER statement
  GO TO statement, 2-92
  segmentation, 9-7
  SORT statement, 7-15
  USE FOR DEBUGGING statement, 11-6
AND
  abbreviated combined relation, 2-61
  hierarchy, 2-62
  negated combined condition, 2-60
  SEARCH statement, 3-7
AREA/AREAS clause
  Information Interchange Tape Processing module, 16-2
Arithmetic expression
  COMPUTE statement, 2-86
  relation condition, 2-50
  sign condition, 2-58
ASCENDING Key phrase
  MERGE statement, 7-7
ASCENDING KEY phrase
  OCCURS clause, 3-2
  SEARCH statement, 3-9
  SORT statement, 7-13
ASCII COBOL environment
  establishing, 18-28
ASSIGN clause
  DISC, 6-8
  IBM Tape Processing module, 15-2
  Indexed I-O module, 6-8
  Information Interchange Tape Processing module, 16-2
  INTERCHANGE, 16-2
  Relative I-O module, 5-5
  UNISERVO, 15-2
asynchronous processing, 14-1
AT END condition
  READ statement, 4-26, 5-18
  RETURN statement, 7-12
  status key, 4-2, 5-2, 6-2
  USE statement, 4-28, 5-22, 6-27
AT END phrase
  READ statement, 5-17, 6-21
  RETURN statement, 7-12
  SEARCH statement, 3-7
  USE statement, 4-28, 5-22, 6-27

B

BEFORE phrase
SEND statement, 13-15
WRITE statement, 4-29
BLANK WHEN ZERO clause
USAGE IS INDEX clause, 3-6
USAGE IS LOCK clause, 14-6
VALUE clause, 2-44
BLOCK CONTAINS clause
IBM Tape Processing module, 15-6
Relative I-O module, 5-10
Report Writer module, 8-10
Sequential I-O module, 4-9
Blocking factors
MSAM files, 20-3
Non-MSAM files, 20-2
BY
DIVIDE statement, 2-89
MULTIPLY statement, 2-105
PERFORM statement, 2-106

C

CALL statement
Linkage Section, 12-4
Procedure Division header, 12-6
Calling COBOL programs
calling sequence, 18-29
non-COBOL programs, 18-27
parameter format, 18-30
Calling non-COBOL programs
parameter format, 18-22
calling sequence
ENTER PL1, 18-25
Calling sequence
calling COBOL programs, 18-29
ENTER ASM, 18-22
ENTER MASM, 18-21
CANCEL statement
CALL statement, 12-8
CLOSE statement, 4-20, 6-18
CARD-PUNCH
BLOCK CONTAINS clause, 4-10
CARD-READER
BLOCK CONTAINS clause, 4-10
CARTRIDGE-ID
Sequential I-O module, 4-16
category of data
VALUE clause, 2-43
Category of data
MOVE statement, 2-102
PICTURE clause, 2-19, 2-34
CHARACTERS
BLOCK CONTAINS clause, 4-9
RECORD CONTAINS cause, 15-7
RECORD CONTAINS clause, 4-15
Class of data
incompatible, 2-79
CLOSE statement
AT END condition, 4-26, 5-19, 6-23
IBM Tape Processing module, 15-20
Indexed I-O module, 6-17
I-O status, 4-2, 5-2, 6-2
OPEN statement, 4-22, 6-20
READ statement, 4-26, 5-18, 6-22
Relative I-O module, 5-13
Sequential I-O module, 4-17
TERMINATE Statement, 8-36
COBOL environment
preserving, 18-21
CODE-SET clause
Sequential I-O module, 4-10
COLLATING SEQUENCE phrase
MERGE statement, 7-7
SORT statement, 7-13
Comma
Library text-word, 10-3
restriction, 2-1
Comment line
debugging lines, 11-9
library [text], 10-3
comment-entry
DATE-COMPILED paragraph, 2-6
Communication
communication facility, 1-10
interprogram communication, 1-8
Communication Section
Asynchronous Processing module, 14-8
Comparison of
index data items, 3-7
index-names, 3-7
COMPUTE statement
composite of operands, 2-67
data conversion, 2-67
decimal alignment, 2-67
maximum operand size, 2-67
multiple results, 2-68
Condition
abbreviated combined relation condition, 2-61
class condition, 2-55

combined condition, 2-60
complex condition, 2-59
condition-name condition, 2-56
evaluation rules, 2-63
IF statement, 2-93
negated simple condition, 2-59
PERFORM UNTIL statement, 2-106
relation condition, 2-50
SEARCH statement, 3-7
sign condition, 2-58
SIZE ERROR condition, 2-64
switch-status condition, 2-57
condition-name
SPECIAL-NAMES paragraph, 2-9
VALUE clause, 2-43
Condition-name
level-number, 2-18
REDEFINES clause, 2-35
RERUN clause, 5-7, 6-10
Connective
restriction, 2-1
continuation of lines
comment entries, 2-6
Continuation of lines
debugging lines, 11-9
restriction, 2-1
control
transfers, 1-37
Control break
CONTROL clause, 8-13
Control Division
Alphabet Section, 2-3
Inline Section, 2-3
messages, 2-3
source-alphabet, 2-3
structure, 1-39
CORRESPONDING (CORR) phrase
MOVE statement, 2-101
SUBTRACT statement, 2-117
Cross reference & allocation listings
definition, 18-18
Current record pointer
DELETE statement, 6-19
Indexed I-O module, 6-2
OPEN statement, 4-23, 5-16, 6-20
READ statement, 4-25, 5-18, 6-22
Relative I-O module, 5-2
REWRITE statement, 4-27, 6-24
START statement, 5-21, 6-26
WRITE statement, 5-23, 6-29
Current Record Pointer
IBM Tape Processing module, 15-1

D

Data
Global, 14-2
Local, 14-2
Data alignment
execution speed, 17-2
Data allocation
IBM Tape Processing module, 15-17
Data description entry
Linkage Section, 12-4
Working-Storage Section, 2-13
Data Division
Asynchronous Processing module, 14-6
Communication module, 13-2
entries, 1-51
IBM Tape Processing module, 15-5
Indexed I-O module, 6-12
Information Interchange Tape Processing module, 16-4
Interprogram Communication module, 12-4
Nucleus, 2-13
Relative I-O module, 5-9
Report Writer module, 8-1
Sequential I-O module, 4-8
Sort-Merge module, 7-5
Structure, 1-42
Table Handling module, 3-2
DATA RECORDS clause
IBM Tape Processing module, 15-6
Indexed I-O module, 6-13
Relative I-O module, 5-10
Sequential I-O module, 4-11
Sort-Merge module, 7-5
Data storage
Interprogram data storage, 1-9
Debugging lines
within library [text], 10-3
DEBUGGING MODE clause
debugging lines, 11-9
Declaratives
segmentation, 9-5
USE BEFORE REPORTING statement, 8-37
USE statement, 4-28, 5-22, 6-27
DELETE statement
Indexed I-O module, 6-18
OPEN mode, 5-15
Relative I-O module, 5-14
DENSITY clause
Report Writer module, 8-14
Sequential I-O module, 4-11

DESCENDING KEY phrase
MERGE statement, 7-7
OCCURS clause, 3-2
SORT statement, 7-13
Diagnostic messages
Compiler diagnostics, C-1
Execution diagnostics, C-106
DISABLE statement
USE FOR DEBUGGING statement, 11-6
DISC
ASSIGN clause, 6-8
RERUN clause, 4-6, 6-10
DISPLAY statement
mnemonic-name, 2-10
SPECIAL-NAMES paragraph, 2-10
DIVIDE statement
composite of operands, 2-67
COMPUTE statement, 2-86
data conversion, 2-67
decimal alignment, 2-67
maximum operand size, 2-67
multiple results, 2-68

E

element naming conventions
main programs, 19-2
ENABLE statement
USE FOR DEBUGGING statement, 11-6
Environment Division
Asynchronous Processing module, 14-4
Debug module, 11-3
IBM Tape Processing module, 15-2
Indexed I-O module, 6-8
Information Interchange Tape Processing module, 16-2
Interprogram Communication module, 12-2
Nucleus, 2-7
Relative I-O module, 5-5
Segmentation module, 9-5
Sequential I-O module, 4-4
structure, 1-41
example
Sequential I-O, 4-33
examples
arithmetic expressions, 2-48
IF statement, 2-94
Examples
ALTER statement, 2-85
collating sequence, 2-53
conditional variable, 2-56
CORRESPONDING phrase, 2-66
DEPENDING phrase, 3-5
DISPLAY statement, 2-89
EXIT statement, 2-92
FILLER, 2-17
INSPECT statement, 2-100
JUSTIFIED, 2-17
library, 10-5
RENAME, 2-36
Report Writer, 8-40
ROUNDED phrase, 2-64
Sequential I-O, 4-33
Sort-Merge, 7-18
SOURCE-ALPHABET, 2-4
SPECIAL-NAMES, 2-12
STRING statement, 2-117
SUBTRACT statement, 2-118, 2-119
Switch-status condition, 2-57
UNSTRING statement, 2-122, 4-4
EXIT PROGRAM statement
CALL statement, 12-8
CANCEL statement, 12-9
Extra Options field
@ACOB control statement, 18-1

F

FD level indicator
IBM Tape Processing module, 15-5
Indexed I-O module, 6-13
Report Writer module, 8-4
Sequential I-O module, 4-9
Figurative constant
DISPLAY statement, 2-88
INSPECT statement, 2-97
restriction, 2-1
STOP statement, 2-113
STRING statement, 2-114
VALUE clause, 2-44
VALUE OF clause, 6-16
File control entry
Asynchronous Processing module, 14-4
Indexed I-O module, 6-8
Relative I-O module, 5-5
Sequential I-O module, 4-4
File Control Entry
IBM Tape Processing module, 15-2
Information Interchange Tape Processing module, 16-2

File description entry
Indexed I-O module, 6-12
Relative I-O module, 5-9
Report Writer module, 8-4
Sequential I-O module, 4-8
File organization
indexed organization, 1-7
relative organization, 1-7
sequential organization, 1-6
File Section
Asynchronous Processing module, 14-6
IBM Tape Processing module, 15-5
Indexed I-O module, 6-12
Information Interchange Tape Processing module, 16-4
Relative I-O module, 5-9
Sequential I-O module, 4-8
Sort-Merge module, 7-5
FILE STATUS clause
Indexed I-O module, 6-8
Information Interchange Tape Processing module, 16-2
Relative I-O module, 5-5
Sequential I-O module, 4-4
FILE STATUS data item
Indexed I-O module, 6-2
Relative I-O module, 5-2
Sequential I-O module, 4-2, 15-1
FILE-ACCESS
Sequential I-O module, 4-16
FILE-CONTROL paragraph
Asynchronous Processing module, 14-4
IBM Tape Processing module, 15-2
Indexed I-O module, 6-8
Information Interchange Tape Processing module, 16-2
Relative I-O module, 5-5
Sequential I-O module, 4-4
Sort-Merge module, 7-2
FILE-ID
Indexed I-O module, 6-15
Relative I-O module, 5-12
Sequential I-O module, 4-16
FINAL
CONTROL clause, 8-13
TYPE clause, 8-27
FORMS-REQUEST
Sequential I-O module, 4-16
FROM phrase
ACCEPT statement, 2-80
PERFORM varying statement, 2-106
RELEASE statement, 7-11
REWRITE statement, 4-27, 5-19, 6-24
SUBTRACT statement, 2-117
WRITE statement, 4-30, 5-23, 6-28, 15-27

G

GENERATE statement
CONTROL clause, 8-13
data-name, 8-13
SUM clause, 8-26
GIVING phrase
ADD statement, 2-83
MERGE statement, 7-7
MULTIPLY statement, 2-105
SORT statement, 7-13
SUBTRACT statement, 2-117
GO TO statement
ALTER statement, 2-85
PERFORM statement, 2-108
SORT statement, 7-15
USE FOR DEBUGGING statement, 11-5
GROUP INDICATE clause
VALUE clause, 2-44

H

handling
arithmetic expressions, 2-48
HIGH
restriction, 2-1
HOLD statement, 14-10
HVTIP Utilization
automatic mapping, 21-11
creating modulename equate elements, 21-8
general, 21-7
writing ASM subprograms, 21-9

I

IBM Tape Processing
USAGE clause, 15-14
Identification Division
structure, 1-40
Identifier
restriction, 2-1
IF
execution speed, 17-2

Implementor-name
Alphabet-name clause, 2-10
SPECIAL-NAMES paragraph, 2-9
Index data item
CONTROL clause, 8-13
initial value, 2-13
indexed I-O
statistics, 6-5
Indexing
CONTROL clause, 8-13
MOVE statement, 2-101
RETURN statement, 7-12
Index-name
OCCURS clause, 3-2
relation condition, 3-7
Information Interchange Tape Processing Module
record description restrictions, 16-4
INITIATE statement
SUM clause, 8-27
USE BEFORE REPORTING statement, 8-37
Input-Output Section
Asynchronous Processing module, 14-4
IBM Tape Processing module, 15-2
Indexed I-O module, 6-8
Information Interchange Tape Processing module, 16-2
Relative I-O module, 5-5
Sequential I-O module, 4-4
Sort-Merge module, 7-2
INTO
READ statement, 4-25, 5-17, 6-21
STRING statement, 2-114
INVALID KEY condition
DELETE statement, 5-14, 6-18
READ statement, 5-17, 6-21
REWRITE statement, 5-19, 6-24
START statement, 5-21, 6-25
WRITE statement, 5-23, 6-28
I-O status
Indexed I-O module, 6-2
Relative I-O module, 5-2
Sequential I-O module, 4-2
I-O Status
Relative I-O module, 5-2
I-O-CONTROL paragraph
IBM Tape Processing module, 15-3
Indexed I-O module, 6-10
Relative I-O module, 5-7
Sequential I-O module, 4-6
Sort-Merge module, 7-3

J

JUSTIFIED (JUST) clause
Condition-name, 2-15
USAGE IS INDEX clause, 3-6
VALUE clause, 2-45
JUSTIFIED clause
USAGE IS LOCK clause, 14-6

K

KEY data-names
MERGE statement, 7-7
SORT statement, 7-13
KEY phrase
DISABLE statement, 13-11
OCCURS clause, 3-2, 3-3, 3-4
Relative I-O relative key, 5-5
START statement, 6-25

L

LABEL RECORDS clause
IBM Tape Processing module, 15-7
Indexed I-O module, 6-14
Relative I-O module, 5-11
Report Writer module, 8-15
Sequential I-O module, 4-12
Level-number
Data description entry, 2-14
Report group description entry, 8-7
levels
concept, 1-30
Line number
absolute, 8-16
relative, 8-16
LINE NUMBER clause
presentation rules, 8-17
Linkage Section
VALUE clause, 2-44
Literal
CURRENCY SIGN clause, 2-12
nonnumeric literals, 1-25
numeric literals, 1-26
STOP statement, 2-113
LOCK statement, 14-11
USAGE IS LOCK clause, 14-6
LOW-VALUE/LOW-VALUES
restriction, 2-2

M

MASS-STORAGE
RERUN clause, 15-3
MERGE statement
OPEN statement, 4-21
Segmentation, 9-7
USE FOR DEBUGGING statement, 11-8
Message Control System (MCS)
TIP, 21-2
messages
operators console, 18-34
Mnemonic-name
ACCEPT statement, 2-80
DISPLAY statement, 2-87
SEND statement, 13-15
WRITE statement, 4-29
MOVE
execution speed, 17-2
MOVE statement
CORRESPONDING (CORR), 2-101
index data item, 3-6
overlapping operands, 2-67
MULTIPLY statement
compute statement, 2-86
data conversion, 2-67
maximum operand size, 2-67
multiple results, 2-68

N

naming conventions
segmented programs, 19-2
subprograms, 19-3
NEXT PAGE phrase
GROUP clause, 8-18
LINE NUMBER clause, 8-16
NEXT phrase
Indexed I-O module, 6-21
Relative I-O module, 5-17
NEXT SENTENCE phrase
IF statement, 2-93
SEARCH statement, 3-7
Noncontiguous elementary item
level-number 77, 2-18

O

OCCURS clause
MOVE statement, 2-103
SYNCHRONIZED Clause, 2-39
VALUE clause, 2-43
OPEN statement
CLOSE statement, 4-19, 5-14, 6-17
IBM Tape Processing module, 15-23
Indexed I-O module, 6-19
INITIATE statement, 8-35
I-O status, 4-1, 5-2, 6-2
LINAGE clause, 4-14
READ statement, 4-25, 5-18, 6-21
Relative I-O module, 5-15
Report Writer module, 8-1
REWRITE statement, 4-27, 5-19, 6-24
Sequential I-O module, 4-21
START statement, 5-21, 6-25
WRITE statement, 4-30, 5-23, 6-28
Operator
logical, 2-59
Operators console
messages, 18-34
optimization
programmer controlled, 17-6
OPTIONAL phrase
CLOSE statement, 4-20
options
@ACOB control statement, 18-1
OR logical connective
abbreviated combined relation, 2-61
hierarchy, 2-62
OUTPUT PROCEDURE phrase
MERGE statement, 7-7
SORT statement, 7-13
OVERFLOW phrase
CALL statement, 12-7
STRING statement, 2-114
UNSTRING statement, 2-119
Overlapping operands
Nucleus, 2-67
Table Handling module, 3-7

P

PAGE
SEND statement, 13-15
WRITE statement, 4-29
PAGE clause
FIRST DETAIL, 8-19
HEADING, 8-19
LAST DETAIL, 8-19
NEXT GROUP clause, 8-18
PAGE LIMIT, 8-19

Paragraph
header, 1-50
Parameter format
calling COBOL programs, 18-30
Parentheses
condition, 2-60
PICTURE clause, 2-20
Password
DISABLE statement, 13-12
ENABLE statement, 13-13
PERFORM
execution speed, 17-2
PERFORM statement
segmentation, 9-7
SORT statement, 7-15
USE statement, 4-28, 5-23, 6-28
PICTURE (PIC) clause
BLANK WHEN ZERO clause, 2-15
DECIMAL-POINT IS COMMA clause, 2-12
Linkage Section, 12-4
USAGE IS INDEX clause, 3-6
Working-Storage Section, 2-13
PICTURE clause
character<#106>string, 1-28
USAGE IS LOCK clause, 14-6
POINTER phrase
STRING statement, 2-114
UNSTRING statement, 2-121
Presentation rules
LINE NUMBER clause, 8-17
PRINTER
BLOCK CONTAINS clause, 4-10
Procedure Division
Asynchronous Processing module, 14-9
Communication module, 13-11
general description, 1-44
IBM Tape Processing module, 15-20
Indexed I-O module, 6-17
Interprogram Communication module, 12-6
nucleus, 2-46
Relative I-O module, 5-12
Report Writer module, 8-33
Sequential I-O module, 4-17
Sort-Merge module, 7-7
structure, 1-45
Table Handling module, 3-7
PROCESS statement, 14-11
PROCs
COBOL, 18-12
program-name
CANCEL statement, 12-9
Program-name
CALL statement, 12-7
PURGE-DATE
Sequential I-O module, 4-16

Q

Qualification
CD entry, 13-5
CONTROL clause, 8-13
CORRESPONDING phrase, 2-65
DEBUG-NAME, 11-7
LINE-COUNTER, 8-5
Linkage Section, 12-4
MERGE statement, 7-7
OCCURS clause, 3-2
READ statement, 6-21
RELEASE statement, 7-11
restriction, 2-1
REWRITE statement, 4-27, 5-20, 6-24
SORT statement, 7-13
START statement, 5-21, 6-25
VALUE OF clause, 4-16, 5-12, 6-15
Working-Storage Section, 2-13
WRITE statement, 4-29, 5-23, 6-28
Queues
concepts, 1-14
dequeuing, 1-14
enqueuing, 1-14
queue hierarchy, 1-15

R

READ statement
CLOSE statement, 4-19
DELETE statement, 5-14
IBM Tape Processing module, 15-25
Indexed I-O module, 6-21
OPEN statement, 4-22
Relative I-O module, 5-17
REWRITE statement, 5-20, 6-24
Sequential I-O module, 4-25
RECEIVE statement
USE FOR DEBUGGING statement, 11-6
RECORD CONTAINS clause
IBM Tape Processing module, 15-7
Indexed I-O module, 6-14
Relative I-O module, 5-11
Sequential I-O module, 4-15
Sort-Merge module, 7-6
RECORD CONTAINS CLAUSE
Report Writer module, 8-23

Record description entry
Indexed I-O module, 6-12
Relative I-O module, 5-9
Sequential I-O module, 4-8
Record Description entry
IBM Tape Processing module, 15-10
Record Description Restrictions
Information Interchange Tape Processing module, 16-4
RECORDING MODE clause
IBM Tape Processing module, 15-8
Information Interchange Tape Processing module, 16-4
Reference format
general description, 1-49
restriction, 2-2
Text-words, 10-4
Relation condition
abbreviated combined, 2-61
index data item, 3-7
index-name, 3-7
MERGE statement, 7-9
nonnumeric operands, 2-51
SORT statement, 7-14
RELATIVE KEY phrase
READ statement, 5-19
REWRITE statement, 5-20
START statement, 5-21
WRITE statement, 5-23
RENAMES clause
Level-number, 2-18
RERUN clause
IBM Tape Processing module, 15-3
Indexed I-O module, 6-10
Relative I-O module, 5-7
Sequential I-O module, 4-6
RESERVE AREA/AREAS clause
IBM Tape Processing module, 15-2
Relative I-O module, 5-5
RESERVE clause
Information Interchange Tape Processing module, 16-2
REWRITE statement
Indexed I-O module, 6-24
OPEN statement, 4-21, 5-15, 6-19
Relative I-O module, 5-19
Sequential I-O module, 4-27
USE FOR DEBUGGING statement, 11-6

S

SAME AREA clause
IBM Tape Processing module, 15-3
Indexed I-O module, 6-10
Relative I-O module, 5-7
Sequential I-O module, 4-6
SAME RECORD AREA clause
IBM Tape Processing module, 15-3
Indexed I-O module, 6-10
Relative I-O module, 5-7
Sequential I-O module, 4-6
SEARCH statement
USAGE IS INDEX clause, 3-6
segmentation
SORT statement, 7-16
Segmentation
CALL statement, 12-8
Segmentation restrictions
ALTER, 9-7
MERGE, 9-7
PERFORM, 9-7
SORT, 9-8
SELECT clause
IBM Tape Processing module, 15-2
Indexed I-O module, 6-8
Information Interchange Tape Processing module, 16-2
Relative I-O module, 5-5
Sequential I-O module, 4-4
Sort-Merge module, 7-2
SEND statement
SPECIAL-NAMES paragraph, 2-10
USE FOR DEBUGGING statement, 11-6
Separator
restriction, 2-1
SET statement
overlapping operands, 2-67
Overlapping operands, 3-7
SEARCH statement, 3-10
SIGN clause
MOVE statement, 2-103
PICTURE symbol, 2-22
SORT statement
OPEN statement, 4-21
segmentation, 9-8
USE FOR DEBUGGING statement, 11-8

Sort-Merge
files, 20-13
main storage size, 20-13
Sort-Merge module
relationship with Sequential I-O module, 7-1
Source program
COPY statement, 10-2
SOURCE-COMPUTER paragraph
WITH DEBUGGING MODE phrase, 11-3
Space
library text-word, 10-3
SPACE/SPACES
restriction, 2-2
special registers
LINAGE-COUNTER, 4-3
SPECIAL-NAMES paragraph
DISPLAY statement, 2-87
Switch-status condition, 2-57
START statement
Indexed I-O module, 6-25
OPEN statement, 5-15
READ statement, 5-19, 6-23
Relative I-O module, 5-21
USE FOR DEBUGGING statement, 11-6
status key
Relative I-O module, 5-2
Sequential I-O module, 4-2
Status key
Indexed I-O module, 6-2
STATUS KEY clause
ENABLE statement, 13-13
SEND statement, 13-16
STRING statement
overlapping operands, 2-67
Structure
Control Division, 1-39
Data Division, 1-42
Environment Division, 1-41
Identification Division, 1-40
Procedure Division, 1-45
Subscripting
CONTROL clause, 8-13
SUBTRACT statement
composite of operands, 2-67
COMPUTE statement, 2-86
CORRESPONDING (CORR), 2-117
maximum operand size, 2-67
multiple results, 2-68
Sum counter
INITIATE statement, 8-35
SYMBOLIC DESTINATION clause
DISABLE statement, 13-13
ENABLE statement, 13-13
RECEIVE statement, 13-14
SYNCHRONIZED (SYNC) clause
elementary data item, 2-15
IBM Tape Processing module, 15-12
USAGE IS INDEX clause, 3-6
SYNCHRONIZED clause
USAGE IS LOCK clause, 14-6

T

Table handling
table definition, 1-4
table items, 1-5
table searching, 1-6
TALLYING phrase
INSPECT statement, 2-95
UNSTRING statement, 2-119
TAPE
RERUN clause, 4-6, 6-10
TERMINATE statement
INITIATE statement, 8-35
SUM clause, 8-26
USE statement, 8-37
THROUGH (THRU)
MERGE statement, 7-7
RENAMES clause, 2-35
SORT statement, 7-13
VALUE clause, 2-43
Transfers of Control
explicit and implicit, 1-37

U

UNISERVO
RERUN clause, 15-3
UNLOCK statement, 14-13
UNSTRING statement
overlapping operands, 2-67
UPON phrase
DISPLAY statement, 2-87
SUM clause, 8-24
USAGE clause
INSPECT statement, 2-95
USAGE clause
INSPECT statement, 2-95
relation condition, 2-50
SIGN clause, 2-37
USAGE clause
STRING statement, 2-114

USAGE clause
IBM Tape Processing module, 15-14
USAGE IS INDEX clause
MOVE statement, 2-101
SEARCH statement, 3-8
Working-Storage Section, 2-13
USAGE IS LOCK clause
LOCK statement, 14-11
UNLOCK statement, 14-13
USE statement
READ statement, 6-21
USE statement
DELETE statement, 5-14, 6-18
INVALID KEY condition, 5-3, 6-4
READ statement, 4-25
Relative I-O module, 5-17, 5-22
REWRITE statement, 5-20
Sequential I-O module, 4-28
START statement, 5-21
WRITE statement, 4-29, 5-23
USE statement
REWRITE statement, 6-24
USE statement
START statement, 6-26
USE statement
Indexed I-O module, 6-27
USE statement
WRITE statement, 6-28
USE statement, 14-13
USE Statement
Asynchronous Processing module, 14-13
USING phrase
CALL statement, 12-7
Linkage Section, 12-4
MERGE statement, 7-7
Procedure Division header, 12-6
SORT statement, 7-13

V

VALUE clause
IBM Tape Processing module, 15-15
USAGE IS LOCK clause, 14-6
VALUE OF clause
IBM Tape Processing module, 15-9
Indexed I-O module, 6-15
Relative I-O module, 5-12
Report Writer module, 8-31
Sequential I-O module, 4-15
VARYING phrase
SEARCH statement, 3-10

W

Word
COBOL, 1-19
WRITE statement
IBM Tape Processing module, 15-27
Indexed I-O module, 6-28
OPEN statement, 4-22, 5-15, 6-20
Relative I-O module, 5-23
Sequential I-O module, 4-29
SPECIAL-NAMES paragraph, 2-10
USE FOR DEBUGGING statement, 11-6

Z

ZERO/ZEROS/ZEROES
restriction, 2-2

Special Characters

@XQT statement
D option, 11-2

© 2010 Unisys Corporation.
All rights reserved.

7830 7709–002